

ALG

THUẬT TOÁN, TƯ DUY GIẢI BÀI VÀ KỸ LUẬT LUYỆN TẬP

CÂY TRI THỨC GIẢI THUẬT

Trình bày theo mạch

bài toán → ý tưởng → bất biến → chi phí → chỗ hỏng

*“Thuật toán tốt không phải thuật toán thông minh nhất,
mà là thuật toán khai thác đúng cấu trúc mà bài toán vô tình để lộ ra.”*

luận điểm xuyên suốt cả quyển, xem Chương 1 và Chương 13

Năm phần A · Tư duy và nền tảng B · Cấu trúc dữ liệu C · Chiến lược thiết kế
D · Giới hạn và miền chuyên E · Hành nghề và luyện tập

Bản gộp toàn cây — Ngày 9 tháng 9 năm 2026

Bản đồ cây — quyển sách này là gì và đọc thế nào

TÓM TẮT

Đây là bản đồ của một quyển sách gồm 28 chương về thuật toán, được viết để *ôn lại* chứ không phải để học lần đầu. Nó chia làm năm phần: tư duy giải bài và nền tảng (A), cấu trúc dữ liệu (B), chiến lược thiết kế (C), giới hạn lý thuyết và các miền chuyên (D), và hành nghề — viết code, gỡ lỗi, phỏng vấn, luyện tập (E). Điểm khác biệt so với một giáo trình thuật toán thông thường: mỗi chương không chỉ nói thuật toán làm gì, mà nói *làm sao nghĩ ra được nó*, và cuối mỗi chương có ngân hàng câu hỏi kèm đáp án cộng danh sách bài tập kèm gợi ý. Nếu chỉ nhớ một điều: đọc chương 1 và chương 2 trước, vì hai chương đó quyết định bạn dùng 26 chương còn lại như thế nào.

A. Quyển sách này giải quyết vấn đề gì

Người học thuật toán lâu năm hay rơi vào một trong hai trạng thái khó chịu. Trạng thái thứ nhất: biết rất nhiều thuật toán, đọc tên nào cũng thấy quen, nhưng khi gặp một bài mới thì đứng im, vì không có cách nào để đi từ *đề bài* tới *cái tên thuật toán* cần dùng. Trạng thái thứ hai: giải được nhiều bài nhờ luyện tập, nhưng không giải thích được vì sao lời giải đúng, nên khi bài đổi một chi tiết nhỏ thì lời giải sụp mà không biết sụp ở đâu.

Cả hai trạng thái đều có chung một nguyên nhân: kiến thức được lưu dưới dạng *danh mục* chứ không phải dưới dạng *quan hệ*. Danh mục trả lời câu hỏi “có những gì”; nó không trả lời được “khi nào dùng cái nào” và “vì sao cái đó lại đúng”. Quyển sách này cố ý được tổ chức theo quan hệ: mỗi kỹ thuật xuất hiện kèm câu hỏi mà nó trả lời, giả định mà nó cần, giá phải trả, và dấu hiệu trên đề bài để nhận ra nó.

Có một điểm nữa ít giáo trình nói thẳng. Luyện thuật toán không chỉ là học thuật toán — nó là một dạng luyện tư duy giải quyết vấn đề, và tư duy đó có tài liệu riêng, có người nghiên cứu nghiêm túc, có phương pháp luyện đã được kiểm chứng. Pólya viết *How to Solve It* cho toán học nhưng bốn bước của ông áp thẳng vào lập trình thi đấu; Schoenfeld chỉ ra rằng người giải giỏi khác người giải kém chủ yếu ở khả năng *tự giám sát* chứ không ở lượng kiến thức; Ericsson chỉ ra rằng số giờ luyện tập chỉ có giá trị khi luyện đúng kiểu. Vì vậy Phần A và Phần E của quyển sách này nói về cách nghĩ và cách luyện, chứ không nói về thuật toán nào cả — và đó là hai phần nên đọc kỹ nhất.

B. Năm phần và vì sao chia như vậy

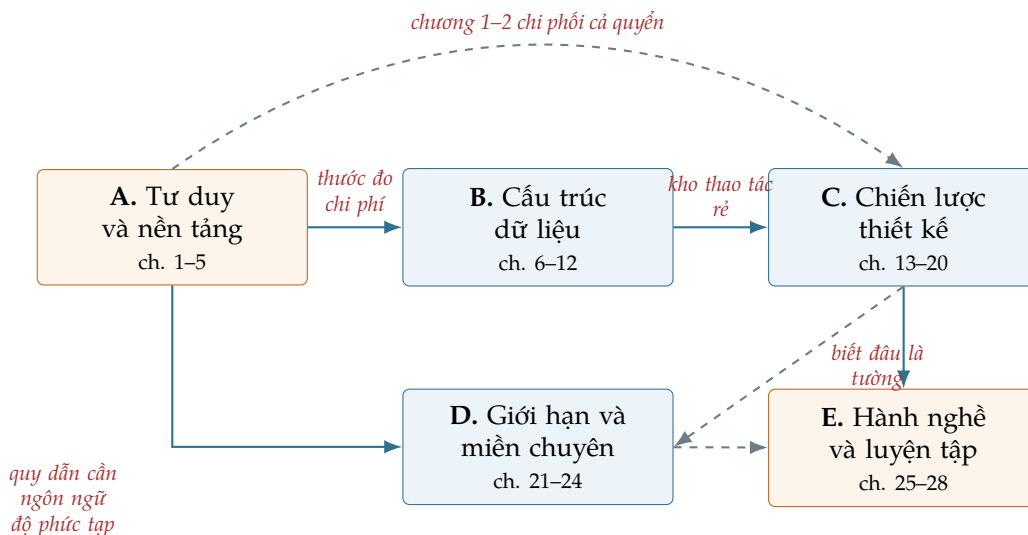
Cách chia phổ biến của giáo trình thuật toán là chia theo *đối tượng*: sắp xếp, cây, đồ thị, chuỗi, hình học. Cách chia đó tiện tra cứu nhưng dở khi cần vận dụng, vì đề bài thật không bao giờ nói cho bạn biết nó thuộc chương nào. Cây này chia theo *vai trò trong quá trình giải*:

- **Phần A — Tư duy và nền tảng (chương 1–5).** Trả lời câu hỏi “đứng trước một bài lạ thì làm gì” và “bằng thước đo nào mà tôi biết lời giải này đủ nhanh và đúng”. Gồm nghệ thuật giải bài, kinh nghiệm của những người giỏi nhất, mô hình chi phí và độ phức tạp, chứng minh tính đúng, và bộ toán tối thiểu.

- **Phần B — Cấu trúc dữ liệu (chương 6–12).** Trả lời câu hỏi “dữ liệu nên nằm ở dạng nào để thao tác tôi cần trở nên rẻ”. Mỗi chương là một nhóm cấu trúc chia theo *thao tác được làm rẻ đi*, không theo hình dạng.
- **Phần C — Chiến lược thiết kế (chương 13–20).** Trả lời câu hỏi “dùng khuôn tư duy nào để sinh ra lời giải”. Vết cạn, chia để trị, tham lam, quy hoạch động, đồ thị, luồng, tìm kiếm trên đáp án, ngẫu nhiên hoá. Đây là phần dày nhất và cũng là phần sinh ra nhiều lời giải nhất trên thực tế.
- **Phần D — Giới hạn và miền chuyên (chương 21–24).** Trả lời câu hỏi “khi nào thì nên ngừng tìm lời giải nhanh”. Số học và đại số tính toán, hình học tính toán, độ khó và quy dẫn, cận dưới cùng các mô hình tính toán khác.
- **Phần E — Hành nghề (chương 25–28).** Trả lời câu hỏi “từ ý tưởng đúng tới code chạy đúng, và làm sao luyện để giỏi lên”. Cài đặt và gỡ lỗi, thuật toán trong hệ thống thật, chiến thuật phỏng vấn và thi đấu, lộ trình luyện tập kèm ngân hàng bài tập lớn.

C. Phụ thuộc và thứ tự đọc

Hình 1 vẽ quan hệ phụ thuộc thật giữa năm phần. Mũi tên đặc là phụ thuộc bắt buộc; mũi tên nét đứt là “đọc trước thì dễ hơn nhưng không bắt buộc”.



Hình 1. Quan hệ phụ thuộc giữa năm phần. Hai phần tô hổ phách (A và E) là phần nói về *cách nghĩ và cách luyện*; ba phần còn lại là nội dung kỹ thuật. Người đã có nền có thể vào thẳng C, nhưng nên đọc chương 1–2 trước vì chúng quyết định cách dùng phần còn lại.

Có ba lối đọc hợp lý, tùy mục tiêu.

Ôn tập toàn diện (khuyến nghị): đọc tuần tự A → B → C, rồi D, rồi E; nhưng mỗi khi học xong một chương của C thì quay ngay sang mục bài tập tương ứng ở chương 28 và giải ít nhất năm bài. Kiến thức thuật toán không dính nếu không có bài tập đi kèm trong vòng vài ngày.

Chuẩn bị phỏng vấn (6–10 tuần): chương 1, 2, 3, rồi B toàn bộ, rồi C các chương 13–17, rồi chương 25 và 27. Bỏ qua D trừ mục nhận diện bài NP-khó ở chương 23 — nhận ra “bài này không có lời giải đa thức đã biết” là một tình huống phỏng vấn thật.

Thi đấu lập trình: A đầy đủ, B đầy đủ, C đầy đủ, D chương 21–22, E chương 25 và 27–28. Chương 23–24 đọc lướt.

D. Cốt lõi và tham khảo

Không phải chương nào cũng đáng đầu tư như nhau. Theo nguyên tắc 80/20, phần lõi tạo ra phần lớn năng lực giải bài gồm: chương 1 (bốn bước và bộ heuristic), chương 3 (mô hình chi phí), chương 4 (bất biến — công cụ chứng minh dùng nhiều nhất), chương 6–7 và 9–10 (mảng, hai con trỏ, heap, cây chỉ số, DSU), chương 14–17 (tham lam, quy hoạch động, đồ thị — chiếm tỉ lệ áp đảo trong mọi kỳ thi và phỏng vấn), chương 19 (tìm kiếm trên đáp án, kỹ thuật có tỉ lệ “bài khó hoá dễ” cao nhất), và chương 25 (cài đặt, gỡ lỗi, stress test).

Phần tham khảo — biết là có, tra khi cần — gồm: cây cân bằng ở mức cài đặt chi tiết (chương 9), suffix automaton (chương 12), luồng chi phí nhỏ nhất (chương 18), hình học với số thực (chương 22), và toàn bộ chương 24.

E. Quy ước dùng trong cả quyển

Mỗi chương mở đầu bằng khối **Tóm tắt**, khối **Kiến thức cần có trước** và bốn tới năm **câu hỏi mở đầu**; những câu hỏi đó được trả lời tường minh ở cuối chương trong khối **Trả lời câu hỏi mở đầu**. Sau đó là khối **Nói lại thật đơn giản** — toàn chương kể lại bằng ngôn ngữ thường, theo kỹ thuật Feynman.

Cuối mỗi chương có ba khối luyện: bảng **Active recall** (che cột phải, tự trả lời), **Câu hỏi ôn tập có đáp án** (câu hỏi dài hơn, đáp án viết đủ để tự chấm), và bảng **Bài tập** với cột “điều cần nhận ra” đóng vai trò gợi ý — che cột đó lại nếu muốn tự nghĩ. Mức độ đánh bằng chấm: • khởi động, •• chuẩn, ••• khó.

Ba loại hộp màu được dùng tiết chế: **Trực giác** (mô hình tư duy trước khi hình thức hoá), **Cạm bẫy** (lỗi thường gặp), **Cần nhớ** (điều duy nhất phải mang theo nếu quên hết phần còn lại).

Nhiều chương kết thúc bằng khối **Đọc thêm**: mỗi dòng ghi tên công trình và tác giả, *nơi công bố và năm*, một câu vì sao nên đọc và nên đọc phần nào, cùng mức ưu tiên — *nền tảng* nếu muốn hiểu chương tới gốc, *tra khi cần* nếu chỉ cần biết là có. Khối này chỉ liệt kê nguồn đã qua phản biện; nơi nào dẫn một bản tiền ấn thì có ghi rõ, và bạn nên đọc dòng đó như tin tức chứ không như kiến thức đã ổn định.

Về thuật ngữ: mỗi thuật ngữ chuyên môn xuất hiện lần đầu ở dạng *tiếng Việt (English)*, sau đó dùng nhất quán một dạng. Tên thuật toán và tên cấu trúc dữ liệu giữ nguyên tiếng Anh, vì đó là dạng bạn sẽ gặp khi tra cứu và khi đọc code.

MỤC LỤC

Bản đồ cây	i
A TƯ DUY VÀ NỀN TẢNG	1
1 Nghệ thuật giải bài (The Art of Problem Solving)	4
1.1 Bốn bước của Pólya, dịch sang ngôn ngữ lập trình	5
1.2 Mười hai câu hỏi để hỏi khi bí	7
1.3 Điều Schoenfeld quay được trên video	8
1.4 Một ví dụ đi hết bốn bước	9
1.5 Vòng lặp bí — thoát bằng cách nào	10
1.6 Gốc rễ: quy trình này từ đâu ra	10
1.7 Liên hệ ngang: cùng một quy trình ở những nơi khác	11
2 Kinh nghiệm của những người giỏi nhất (What the Masters Actually Do)	16
2.1 Bốn trường phái, một vòng phản hồi	17
2.2 Luyện tập có chủ đích, dịch sang việc giải thuật	19
2.3 Vòng upsolving — cỗ máy tiến bộ của dân thi đấu	20
2.4 Đọc sách thế nào cho có tác dụng	21
2.5 Những gì không bắt buộc được, và những gì bắt buộc được ngay	22
2.6 Gốc rễ: vì sao ta biết những điều này về luyện tập	22
2.7 Liên hệ ngang: nghề nào cũng phải giải bài toán luyện tập này	24
3 Mô hình máy và độ phức tạp (Cost Models and Complexity)	29
3.1 Mô hình máy: giả định mà mọi phân tích đều ngầm dùng	30
3.2 Ký hiệu tiệm cận, nói cho đúng	30
3.3 Từ giới hạn đề bài suy ra độ phức tạp cho phép	31
3.4 Hệ thức truy hồi: chi phí của thuật toán đệ quy	32
3.5 Phân tích khấu hao: khi một thao tác đắt trả cho nhiều thao tác rẻ	32
3.6 Chỗ mô hình nói dối: hằng số, bộ nhớ và cache	33
3.7 Đo bậc tăng trưởng thay vì đoán: phép thử nhân đôi	34
3.8 Gốc rễ: ký hiệu này mượn từ đâu và vì sao lại cần	35
3.9 Liên hệ ngang: so sánh với các cách đo chi phí khác	36

4	Tính đúng và chứng minh (Correctness and Proof)	42
4.1	Bất biến vòng lặp: ba việc phải kiểm	43
4.2	Tính dừng: thứ bất biến không hứa	44
4.3	Quy nạp: khuôn của mọi chứng minh về cấu trúc đệ quy	44
4.4	Lập luận đổi chỗ: vũ khí để chứng minh tham lam	44
4.5	Lập luận cắt-dán: nền của quy hoạch động	45
4.6	Tìm phản ví dụ: kỹ năng đối ngẫu với chứng minh	46
4.7	Chứng minh tới mức nào là đủ	46
4.8	Máy trạng thái: một khung duy nhất cho ba loại bất biến	46
4.9	Gốc rễ: chứng minh chương trình bắt đầu từ đâu	47
4.10	Liên hệ ngang: bất biến là ý tưởng dùng chung của nhiều ngành	48
5	Toán cho thuật toán (The Mathematics You Actually Reuse)	54
5.1	Tổng và chặn: đọc chi phí của một vòng lặp	55
5.2	Tổ hợp đếm: bao nhiêu khả năng, và đếm chúng thế nào	55
5.3	Xác suất và kỳ vọng: công cụ mạnh nhất là công cụ đơn giản nhất	56
5.4	Số học modulo: vì sao mọi bài đếm đều lấy dư	57
5.5	Đại số rời rạc và trò chơi: hai mảnh nhỏ nhưng hay dùng	57
5.6	Gốc rễ: những mẫu toán này ra đời để giải bài gì	57
5.7	Liên hệ ngang: cùng công cụ, khác miền	58
B	CẤU TRÚC DỮ LIỆU	63
6	Mảng, chuỗi và hai con trỏ (Arrays, Strings, Two Pointers)	66
6.1	Vì sao bắt đầu bằng mảng	67
6.2	Tổng tiền tố: trả trước một lượt quét, sau đó mọi truy vấn miễn phí	67
6.3	Mảng hiệu: phép ngược của tổng tiền tố	68
6.4	Hai con trỏ: vì sao hai vòng lặp lồng nhau lại tuyến tính	68
6.5	Sắp xếp như một nước đi mở đầu	69
6.6	Chuỗi: mảng có tính chất riêng	69
6.7	Nhìn cửa sổ di chuyển, rồi chủ động làm nó hỏng	70
6.8	Phản biện, nhược điểm và rủi ro triển khai	71
6.9	Gốc rễ: từ bảng từ tới ảnh tích phân	72
6.10	Liên hệ ngang	72

7	Ngăn xếp, hàng đợi và heap (Stacks, Queues, Heaps)	77
7.1	Ngăn xếp: cái mới nhất trước	78
7.2	Hàng đợi và hàng đợi hai đầu	78
7.3	Heap: chỉ biết ai nhỏ nhất, và chỉ cần thế	79
7.4	Cùng một khung, ba cấu trúc, ba thuật toán	80
7.5	Giảm khoá và bài học Fibonacci heap: khi cận và máy thật không cùng kết luận	80
7.6	Gốc rễ: từ máy tính không có đệ quy tới lịch trình hệ điều hành	81
7.7	Liên hệ ngang	82
8	Băm và tập hợp (Hashing and Sets)	87
8.1	Hàm băm và hai cách sống chung với va chạm	88
8.2	Chỗ mà giáo trình nói “trung bình” và thực tế nói “kẻ tấn công”	88
8.3	Bảo đảm mạnh hơn: băm cuckoo và băm bảng tra	89
8.4	Nghịch lý ngày sinh và băm chuỗi	90
8.5	Cấu trúc xác suất: chấp nhận sai một chiều để tiết kiệm bộ nhớ	90
8.6	Phản biện, nhược điểm và rủi ro triển khai	91
8.7	Gốc rễ: từ bảng ký hiệu của trình dịch hợp ngữ tới hệ phân tán	92
8.8	Liên hệ ngang	93
9	Cây tìm kiếm và cân bằng (Search Trees and Balance)	98
9.1	Bài toán thoái hoá và ý nghĩa của cân bằng	99
9.2	Phép quay: thao tác cơ bản duy nhất	99
9.3	Ngẫu nhiên hoá: cách rẻ nhất để cân bằng	100
9.4	Tăng cường: kỹ thuật đáng giá nhất của chương	100
9.5	B-cây: khi chi phí thật không phải là số phép so sánh	101
9.6	Splay và câu hỏi mở nổi tiếng nhất của ngành cấu trúc dữ liệu	102
9.7	Gốc rễ: từ bài toán chiều cao tới bài toán đĩa quay	102
9.8	Liên hệ ngang	103
10	Cây chỉ số và truy vấn đoạn (Fenwick, Segment Trees, Range Queries)	109
10.1	Cây phân đoạn: ý tưởng tổng quát nhất	110
10.2	Fenwick: ngắn hơn, nhanh hơn, hẹp hơn	111
10.3	Sparse table: trả trước tất cả, với điều kiện mảng tĩnh	111
10.4	Phân rã căn bậc hai và thuật toán Mo	112
10.5	Ba mở rộng nên biết là có	112
10.6	Biến cấu trúc tĩnh thành cấu trúc động: phép biến đổi Bentley–Saxe	113
10.7	Nhìn lowbit và kiểm hợp thành lazy bằng tay	113

10.8	Phản biện, nhược điểm và rủi ro triển khai	114
10.9	Gốc rễ: từ nén dữ liệu và hình học tới bài toán truy vấn đoạn	115
10.10	Liên hệ ngang	116
11	Hợp nhất tập hợp — DSU (Disjoint Set Union)	121
11.1	Từ ngây thơ tới gần như hằng số	122
11.2	Hạn chế cốt lõi: không xoá được	122
11.3	Tăng cường DSU: đếm, phe phái, khoảng cách	123
11.4	Gộp nhỏ vào lớn: người anh em tổng quát hơn	123
11.5	“Tối ưu” nghĩa là gì ở đây: hai cận dưới cho hợp nhất tập hợp	124
11.6	Gốc rễ: từ câu lệnh EQUIVALENCE của Fortran	125
11.7	Liên hệ ngang	125
12	Chuỗi và so khớp (Strings and Pattern Matching)	131
12.1	KMP: dùng lại thông tin của lần thất bại	132
12.2	Aho–Corasick: KMP cho nhiều mẫu cùng lúc	132
12.3	Băm chuỗi: lựa chọn ngắn với cái giá xác suất	133
12.4	Nhìn được mọi chuỗi con: trie, mảng hậu tố, automaton	133
12.5	Manacher và một mẫu tư duy đáng nhớ	134
12.6	Gốc rễ: từ tìm kiếm thư mục tới giải mã bộ gen	134
12.7	Liên hệ ngang	135
C	CHIẾN LƯỢC THIẾT KẾ	140
13	Vét cạn, quay lui và cắt tỉa (Brute Force, Backtracking, Pruning)	143
13.1	Ước lượng trước khi cài	144
13.2	Khuôn quay lui và bốn tầng cắt	144
13.3	Gặp nhau ở giữa: đổi bộ nhớ lấy thời gian	145
13.4	Khi vét cạn là câu trả lời đúng	146
13.5	Gốc rễ: từ trò chơi ô chữ tới bộ giải công nghiệp	146
13.6	Liên hệ ngang	147
14	Chia để trị (Divide and Conquer)	152
14.1	Ba câu hỏi thiết kế và hệ thức truy hồi	153
14.2	Sắp xếp: hai triết lý ngược nhau	153
14.3	Khi bước ghép đòi hỏi một ý tưởng: cặp điểm gần nhất	154
14.4	Nhanh hơn: giảm số bài con thay vì giảm chi phí ghép	155
14.5	Chia để trị ở những nơi không ngờ	155

14.6	Biên giới hiện nay: $O(n \log n)$ cho nhân số, và số mũ của nhân ma trận	156
14.7	Gốc rễ: một mẹo đại số phá vỡ một phỏng đoán	156
14.8	Liên hệ ngang	157
15	Tham lam (Greedy Algorithms)	163
15.1	Vì sao tham lam sai, nói cho chính xác	164
15.2	Hai khuôn chứng minh	164
15.3	Matroid: cấu trúc mà tham lam luôn đúng	165
15.4	Danh mục các bài tham lam kinh điển	165
15.5	Khi tham lam sai: xấp xỉ có bảo đảm	166
15.6	Vẽ phép đổi chỗ: điều gì được bảo toàn	167
15.7	Gốc rễ: điện khí hoá Moravia và một bài tập về nhà	167
15.8	Liên hệ ngang	168
16	Quy hoạch động (Dynamic Programming)	173
16.1	Quy trình năm bước	174
16.2	Bước khó nhất: trạng thái phải đủ	174
16.3	Sáu họ trạng thái hay gặp	175
16.4	Ba lô và câu hỏi “vì sao vẫn NP-khó”	176
16.5	Từ trên xuống hay từ dưới lên	176
16.6	Bốn kỹ thuật tăng tốc	176
16.7	Vì sao điểm tối ưu lại đơn điệu: bất đẳng thức tứ giác	177
16.8	Cùng một công thức, đổi chiều duyệt là đổi bài toán	178
16.9	Phản biện, nhược điểm và rủi ro triển khai	178
16.10	Gốc rễ: một cái tên được chọn để nguy trang	180
16.11	Liên hệ ngang	181
17	Đồ thị: mô hình hoá và duyệt (Graphs: Modeling and Traversal)	187
17.1	Mô hình hoá: cái gì là đỉnh, cái gì là cạnh	188
17.2	Hai phép duyệt và những gì chúng để lộ	189
17.3	Bốn thuật toán đường đi ngắn nhất, bốn giả thiết	189
17.4	Cây: các kỹ thuật riêng	190
17.5	Ba mẹo mô hình hoá đáng giá nhất	190
17.6	Phản ví dụ ba đỉnh để chọn đúng đường đi ngắn nhất	191
17.7	Dijkstra có phải là tận cùng không: rào cản sắp xếp	192
17.8	Phản biện, nhược điểm và rủi ro triển khai	192
17.9	Gốc rễ: bảy cây cầu, một quán cà phê, và một con robot	193
17.10	Liên hệ ngang	194

18	Luồng cực đại và ghép đôi (Max Flow, Matching, Min Cut)	200
18.1	Luồng, lát cắt, và một sự trùng khớp đáng kinh ngạc	201
18.2	Ba thuật toán và giả thiết của chúng	202
18.3	Ghép đôi hai phía và ba định lý anh em	202
18.4	Mô hình hoá: nơi chương này thật sự trả công	203
18.5	Cạnh ngược là quyền sửa một quyết định cũ	203
18.6	Biên giới hiện nay: từ đường tăng tới tối ưu hoá liên tục	204
18.7	Gốc rễ: mạng đường sắt Liên Xô và một bài toán về hôn nhân	204
18.8	Liên hệ ngang	205
19	Tìm kiếm trên đáp án (Binary Search on the Answer)	211
19.1	Khuôn chuẩn và điều kiện áp dụng	212
19.2	Dấu hiệu nhận biết trên đề	212
19.3	Ba cạm bẫy cài đặt	213
19.4	Tìm kiếm ba phân: khi hàm có một đỉnh	213
19.5	Khi không có cấu trúc nào: tìm kiếm cục bộ	214
19.6	Một vết chạy đầy đủ: chia mảng để giảm tổng lớn nhất	214
19.7	Gốc rễ: từ giải phương trình tới ủ kim loại	215
19.8	Liên hệ ngang	215
20	Ngẫu nhiên hoá (Randomized Algorithms)	220
20.1	Hai loại và cách chọn giữa chúng	221
20.2	Khuếch đại: vì sao “sai một phần tư” vẫn dùng được	221
20.3	Bốn đại diện đáng biết	222
20.4	Ngẫu nhiên hoá như một mẫu thiết kế	222
20.5	Gốc rễ: một ván bài trong bệnh viện và bài toán số nguyên tố	223
20.6	Liên hệ ngang	224
D	GIỚI HẠN VÀ MIỀN CHUYÊN	229
21	Số học và đại số tính toán (Number Theory and Algebra)	232
21.1	Euclid: thuật toán cổ nhất còn dùng	233
21.2	Sàng, phân tích thừa số, và ranh giới của cái dễ	233
21.3	“Kiểm tra thì dễ” — dễ tới mức nào, và từ khi nào	234
21.4	Định lý phần dư Trung Hoa: ghép các mảnh thông tin	234
21.5	Luỹ thừa nhanh và luỹ thừa ma trận	234
21.6	Biến đổi Fourier nhanh: đổi miền để nhân trở thành dễ	235

21.7	Đại số trên $\mathbb{F}_2$: khi cộng là XOR	235
21.8	Gốc rễ: từ Euclid tới RSA	236
21.9	Liên hệ ngang	237
22	Hình học tính toán (Computational Geometry)	242
22.1	Vị từ định hướng: viên gạch của mọi thứ	243
22.2	Bao lồi: thuật toán đầu tiên nên thuộc	243
22.3	Quét đường: biến hai chiều thành một chiều	244
22.4	Bốn bài nền và các bẫy của chúng	245
22.5	Vì sao sai số nhỏ lại gây hậu quả lớn	245
22.6	Vị từ thích nghi: trả giá chính xác đúng chỗ cần	245
22.7	Bộ hình suy biến phải thử trước dữ liệu ngẫu nhiên	246
22.8	Gốc rễ: một luận án năm 1978 và cuộc khủng hoảng về độ tin cậy	246
22.9	Liên hệ ngang	247
23	Độ khó và quy dẫn (Hardness and Reductions)	252
23.1	Tìm và kiểm: sự phân biệt sinh ra cả lý thuyết	253
23.2	Quy dẫn: công cụ để so độ khó	253
23.3	Danh mục cần thuộc	254
23.4	Bốn cách sống chung	254
23.5	Ngoài NP: hai điều nên biết là có	255
23.6	Gốc rễ: từ câu hỏi của Hilbert tới danh mục của Karp	255
23.7	Liên hệ ngang	256
24	Cận dưới và các mô hình tính toán khác (Lower Bounds and Other Models)	261
24.1	Ba cách chứng minh cận dưới	262
24.2	Vì sao cận dưới khó	263
24.3	Độ phức tạp tinh: cận dưới có điều kiện, và cách dùng chúng	263
24.4	Mô hình bộ nhớ ngoài: khi đơn vị chi phí là lần đọc khối	264
24.5	Ba mô hình bỏ giả định khác	264
24.6	Gốc rễ: từ cân đo tới các hệ thống dữ liệu lớn	265
24.7	Liên hệ ngang	266
E	HÀNH NGHỀ VÀ LUYỆN TẬP	271
25	Cài đặt, gỡ lỗi và kiểm thử (Implementation, Debugging, Testing)	274
25.1	Phòng bệnh: viết code ít lỗi ngay từ đầu	275
25.2	Danh mục lỗi kinh điển	275

25.3	Stress test: kỹ thuật đánh giá nhất chương	276
25.4	Khi không có lời giải trọng tài	277
25.5	Gỡ lỗi như một bài toán tìm kiếm	277
25.6	Ba việc riêng cho chương trình có ngẫu nhiên	278
25.7	Biến mỗi lỗi thành một phép kiểm có thể chạy lại	278
25.8	Gốc rễ: từ khẳng định của Turing tới thu nhỏ tự động	279
25.9	Liên hệ ngang	280
26	Thuật toán trong hệ thống thật (Algorithms in Production)	284
26.1	Đo trước khi tối ưu	285
26.2	Hằng số, bộ nhớ và những gì tiệm cận không nói	285
26.3	Những tiêu chí ngoài tốc độ	286
26.4	Thuật toán của quyển sách này đang chạy ở đâu	286
26.5	Một phiếu đo trước khi quyết định tối ưu	287
26.6	Gốc rễ: từ “đừng tối ưu sớm” tới “đuôi mới là thứ quan trọng”	288
26.7	Liên hệ ngang	288
27	Phỏng vấn và thi đấu (Interviews and Contests)	293
27.1	Phỏng vấn: quy trình bảy bước	294
27.2	Thi đấu: chiến thuật quan trọng ngang kiến thức	295
27.3	Luyện hai kỹ năng này thế nào	295
27.4	Gốc rễ: từ một cuộc thi ở Texas tới quy trình phỏng vấn hiện đại	296
27.5	Liên hệ ngang	297
28	Lộ trình luyện tập và ngân hàng bài tập (Training Plan and Problem Bank)	301
28.1	Cấu trúc một buổi và một tuần	302
28.2	Bốn công cụ theo dõi	302
28.3	Ba lộ trình theo mục tiêu	303
28.4	Ngân hàng bài tập	304
28.5	Gốc rễ: vì sao lộ trình lại có hình dạng như vậy	307
28.6	Liên hệ ngang	308
	Tài liệu tham khảo có chú giải	312
	Bảng thuật ngữ	316
	Kết luận	320

TƯ DUY VÀ NỀN TẢNG



TÓM TẮT

Phần A không dạy một thuật toán nào. Nó dựng bốn thứ mà mọi chương sau đều dùng: một quy trình để đối mặt với bài lạ (chương 1), kinh nghiệm luyện tập đã được kiểm chứng của những người giỏi nhất (chương 2), thước đo chi phí để biết lời giải có đủ nhanh không (chương 3), công cụ chứng minh để biết lời giải có đúng không (chương 4), và bộ toán tối thiểu quay lại nhiều lần nhất (chương 5). Đọc xong phần này, bạn có một quy trình để chạy khi không biết làm gì, và hai thước đo để tự chấm lời giải của mình trước khi gõ code. Nếu chỉ nhớ một điều: người giải giỏi không khác người giải kém ở lượng thuật toán thuộc lòng, mà ở khả năng tự giám sát — biết mình đang đi hướng nào, đi bao lâu rồi, và khi nào phải bỏ hướng đó.

Có một lý do rất cụ thể để phần này đứng trước tất cả. Khi bạn bí một bài, cái thiếu hầu như không bao giờ là một thuật toán bạn chưa từng nghe tên. Cái thiếu là một trong ba thứ: bạn chưa hiểu đúng đề, bạn chưa thử đủ ví dụ nhỏ để thấy cấu trúc, hoặc bạn đang đi một hướng sai mà không chịu bỏ. Cả ba đều thuộc về quy trình chứ không thuộc về kiến thức, và cả ba đều luyện được — đó chính là nội dung của chương 1 và chương 2.

Chương 3 và chương 4 là hai cái thước. Không có thước đo chi phí, bạn không biết ý tưởng vừa nghĩ ra có đáng gõ hay không, và bạn sẽ mất hàng giờ cài một lời giải $O(n^2)$ cho $n = 10^6$. Không có thước đo tính đúng, bạn viết code rồi tin vào vài ví dụ tự bịa, và cái sai chỉ lộ ra ở test 47. Hai chương này dạy cách trả lời hai câu hỏi *trước khi gõ dòng code đầu tiên*: bài này cho phép độ phức tạp nào, và lời giải của tôi giữ bất biến gì.

Chương 5 khác ba chương kia: nó là kho tra cứu chứ không phải mạch đọc. Ở đó có tổng, hệ thức truy hồi, tổ hợp, xác suất và một ít đại số — đúng những mảnh toán quay lại nhiều nhất trong phần B, C và D. Đừng đọc một mạch; đọc lướt để biết trong đó có gì, rồi quay lại đúng mục khi một chương sau cần đến.

Bản đồ phần A — Tư duy và nền tảng

TÓM TẮT

Phần A gồm năm chương chia theo *câu hỏi chúng trả lời*, không theo môn học. Chương 1 trả lời “bí thì làm gì”; chương 2 trả lời “người giỏi luyện thế nào”; chương 3 trả lời “lời giải này có đủ nhanh không”; chương 4 trả lời “lời giải này có đúng không”; chương 5 là kho toán tra khi cần. Chuỗi bắt buộc chỉ là $1 \rightarrow 3 \rightarrow 4$. Chương 2 đọc được bất cứ lúc nào và nên đọc sớm, chương 5 không đọc tuần tự.

A. Phần này có gì

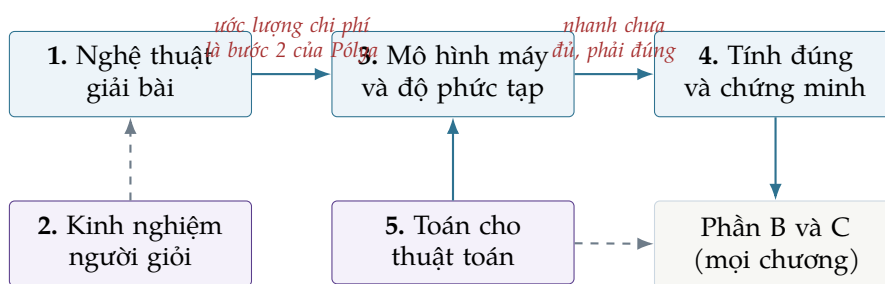
- **01-nghe-thuat-giai-bai** — bốn bước Pólya áp vào lập trình, bộ heuristic sinh ý tưởng, mô hình Schoenfeld về tự giám sát, cách hỏi đúng câu hỏi khi bí.
- **02-kinh-nghiem-nguoi-gioi** — khảo sát cách luyện của Knuth, Dijkstra, Tarjan, Bentley, Skiena, Erik Demaine, các kỳ thủ lập trình hàng đầu; bản đồ sách kinh điển; luyện tập có chủ đích và nhật ký lỗi.
- **03-mo-hinh-may-va-do-phuc-tap** — mô hình RAM, ký hiệu tiệm cận, hệ thức truy hồi, phân tích khâu hao, hằng số thật, phân cấp bộ nhớ, cách đọc giới hạn đề bài ra độ phức tạp cho phép.
- **04-tinh-dung-va-chung-minh** — bất biến vòng lặp, quy nạp, tính dừng, lập luận đổi chỗ, phản chứng, phản ví dụ, và kỷ luật viết chương trình cùng lúc với chứng minh của Dijkstra và Gries.

- **05-toan-cho-thuat-toan** — tổng và chặn, tổ hợp đếm, xác suất và kỳ vọng, số học modulo, đại số tuyến tính rời rạc, lý thuyết trò chơi nhỏ.

B. Vì sao chia như vậy

Một giáo trình thường gộp “toán rời rạc” và “phân tích thuật toán” thành một khối lớn đặt ở đầu sách, và người đọc bỏ dở ở khối đó. Cây này tách thành ba vai trò khác nhau, vì chúng được dùng vào ba thời điểm khác nhau trong quá trình giải. Mô hình chi phí (chương 3) dùng *trước* khi gõ code, để loại các hướng chắc chắn quá chậm. Công cụ chứng minh (chương 4) dùng *trong lúc* nghĩ, vì bất biến vừa là công cụ chứng minh vừa là công cụ sinh ý tưởng. Toán nền (chương 5) dùng *khi cần*, rải rác. Gộp cả ba vào một chương sẽ che mất sự khác nhau đó.

C. Phụ thuộc và thứ tự đọc



Hình 2. Phụ thuộc giữa năm chương của phần A. Chuỗi ngang $1 \rightarrow 3 \rightarrow 4$ là bắt buộc. Hai chương tô tím vào được bất cứ lúc nào: chương 2 là kinh nghiệm luyện tập, chương 5 là kho toán tra cứu.

D. Cốt lõi hay tham khảo

Cốt lõi: toàn bộ chương 1; mục “đọc giới hạn đề bài ra độ phức tạp” và mục phân tích khấu hao của chương 3; mục bất biến vòng lặp và lập luận đổi chỗ của chương 4; mục tổng–chặn và kỳ vọng tuyến tính của chương 5. **Tham khảo:** phần chứng minh hình thức kiểu Hoare ở chương 4, phần giải hệ thức truy hồi bằng hàm sinh ở chương 5.

Nghệ thuật giải bài (The Art of Problem Solving)

PHỤ THUỘC không — đây là chương mở đầu. Mọi chương sau đều giả định bạn đã có quy trình bốn bước và bộ heuristic của chương này trong đầu.

TÓM TẮT

Chương này dựng một quy trình để chạy khi bạn đứng trước một bài chưa biết làm: hiểu đề cho thật kỹ, tìm mối nối với thứ đã biết, thực hiện, rồi nhìn lại. Bốn bước đó là của Pólya, viết cho toán học năm 1945, và chúng áp thẳng vào lập trình mà gần như không phải sửa. Phần thứ hai của chương là bộ heuristic sinh ý tưởng — mười hai câu hỏi cụ thể để hỏi khi bí, mỗi câu kèm ví dụ thuật toán mà nó từng đẻ ra. Phần thứ ba là điều Schoenfeld phát hiện khi quay video người giải bài: cái phân biệt người giỏi với người kém không phải lượng kiến thức mà là *khả năng tự giám sát*. Nếu chỉ nhớ một điều: khi bí, đừng cố nghĩ mạnh hơn — hãy đổi câu hỏi bạn đang tự hỏi.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán (problem)** — một tình huống trong đó bạn biết mình muốn gì nhưng chưa biết cách đi tới đó. Nếu bạn đã biết cách đi thì đó không còn là bài toán, chỉ là bài tập.
- **heuristic** — một mẹo tìm đường, không đảm bảo thành công nhưng làm tăng xác suất tìm ra ý tưởng. Khác với thuật toán ở chỗ nó không hứa hẹn gì cả.
- **vét cạn (brute force)** — cách giải duyệt qua mọi khả năng. Luôn đúng, thường quá chậm, nhưng là điểm xuất phát của gần như mọi lời giải nhanh.
- **bất biến (invariant)** — một mệnh đề luôn đúng tại một điểm nhất định trong thuật toán, chẳng hạn “sau mỗi vòng lặp, đoạn đầu của mảng đã sắp xếp”. Chương [A/04](#) nói kỹ.
- **độ phức tạp thời gian** — cách nói “khi dữ liệu lớn gấp đôi thì thời gian chạy tăng bao nhiêu lần”. Chương [A/03](#) nói kỹ.
- **ví dụ nhỏ nhất** — trường hợp nhỏ nhất mà vẫn còn giữ được cái khó của bài. Công cụ mạnh nhất và bị bỏ qua nhiều nhất trong chương này.

CÂU HỎI MỞ ĐẦU

- Q1. Hai người cùng biết đúng một lượng thuật toán như nhau, vì sao một người giải được bài mới còn người kia thì không?
- Q2. Vì sao lời khuyên “hãy nghĩ kỹ hơn” gần như vô dụng khi bạn đang bí, còn lời khuyên “hãy vẽ ví dụ $n = 4$ ra giấy” thì thường hiệu quả?
- Q3. Vét cạn là cách giải ai cũng nghĩ ra và luôn quá chậm. Vậy vì sao chương này lại bảo bạn phải viết nó ra trước?
- Q4. Bước “nhìn lại” của Pólya là bước bị bỏ qua nhiều nhất. Bỏ nó đi thì bạn mất chính xác cái gì?
- Q5. Khi nào thì việc bỏ hướng đang đi là quyết định đúng, và làm sao biết được điều đó *trước khi* hết giờ?

Hãy bắt đầu bằng một cảnh quen thuộc. Bạn đọc một đề bài, hiểu từng câu, và rồi không biết làm gì tiếp. Không phải bạn thiếu kiến thức — nếu ai đó nói “bài này dùng quy hoạch động” thì bạn làm được ngay. Cái thiếu nằm ở chỗ khác: bạn không có *quy trình* để đi từ đề bài tới cái gọi ý

đó. Trong lúc bí, đầu bạn chạy một vòng lặp vô ích — đọc lại đề, thấy vẫn thế, thử nhớ xem bài này giống bài nào từng gặp, không nhớ ra, đọc lại đề. Vòng lặp đó không hội tụ vì nó không thay đổi thông tin bạn có.

Điều đáng nói là tình trạng này đã được nghiên cứu nghiêm túc, và không phải bởi dân lập trình. Năm 1945, nhà toán học George Pólya viết *How to Solve It*[1], một quyển sách mỏng cố gắng trả lời câu hỏi “khi một người giỏi giải được một bài mới, trong đầu họ thực sự diễn ra chuyện gì”. Bốn mươi năm sau, Alan Schoenfeld làm một việc tàn nhẫn hơn: ông đặt camera quay lại quá trình giải toán của sinh viên và của các nhà toán học, rồi đối chiếu[2]. Kết quả của cả hai người, cộng với nghiên cứu về luyện tập có chủ đích của Ericsson[3], tạo thành nền cho chương này. Chúng ta sẽ dùng chúng theo nghĩa rất thực dụng: đây là quy trình để chạy khi không biết làm gì.

1.1 Bốn bước của Pólya, dịch sang ngôn ngữ lập trình

Pólya chia quá trình giải thành bốn bước: hiểu bài toán, lập kế hoạch, thực hiện kế hoạch, và nhìn lại. Nghe đơn giản đến mức dễ bị coi thường. Nhưng cái giá trị không nằm ở bốn cái tên mà nằm ở *những câu hỏi cụ thể* thuộc về mỗi bước, và ở việc bạn biết mình đang ở bước nào — vì phần lớn thất bại đến từ việc nhảy cóc sang bước 3 khi bước 1 còn chưa xong.

Bước 1 — Hiểu bài toán, kỹ hơn bạn tưởng là cần

Câu hỏi của bước này không phải “đề nói gì” mà là bốn câu rất cụ thể: *cái gì là dữ liệu vào, cái gì là kết quả cần ra, ràng buộc là gì, và tôi có thể phát biểu lại đề bằng lời của mình mà không nhìn vào đề không*. Câu cuối là bài kiểm tra thật sự. Nếu chưa phát biểu lại được, bạn chưa hiểu đề, và mọi phút suy nghĩ tiếp theo đều là phí.

Trong lập trình có ba việc thuộc bước này mà người mới hay bỏ qua. Thứ nhất, *đọc giới hạn*. Dòng “ $1 \leq n \leq 2 \cdot 10^5$ ” không phải chi tiết kỹ thuật, nó là một nửa gợi ý: nó nói lời giải cần cỡ $O(n \log n)$, tức là loại ngay mọi hướng $O(n^2)$. Chương A/03 có bảng chuyển từ giới hạn sang độ phức tạp cho phép; bảng đó nên thuộc lòng.

Thứ hai, *làm tay ví dụ nhỏ*. Không phải ví dụ trong đề — ví dụ của bạn, nhỏ hơn, và bạn tự tính đáp án. Với $n = 1, n = 2, n = 3$. Việc này nghe tầm thường nhưng nó là công cụ sinh ý tưởng mạnh nhất mà chương này có, vì cấu trúc của bài toán thường chỉ lộ ra khi bạn nhìn thấy các trường hợp cạnh nhau.

Thứ ba, *tìm trường hợp biên và trường hợp suy biến*. Mảng rỗng, tất cả phần tử bằng nhau, đồ thị không liên thông, số âm, tràn số. Danh sách này sẽ cứu bạn ở bước 4, nhưng nghĩ ra nó ở bước 1 thì rẻ hơn nhiều.

CẠM BẮY

Người mới thường coi bước 1 là bước “đọc đề”, mất hai phút, rồi lao vào nghĩ thuật toán. Người có kinh nghiệm dành *nhiều hơn* thời gian cho bước 1 chứ không ít hơn — vì đa số bài khó không khó ở thuật toán, mà khó ở chỗ bạn chưa nhìn ra bài toán thật sự đang hỏi gì. Một dấu hiệu chắc chắn rằng bạn bỏ dở bước 1: bạn đang viết code mà vẫn phải cuộn lên đọc lại đề.

Bước 2 – Lập kế hoạch: tìm mối nối

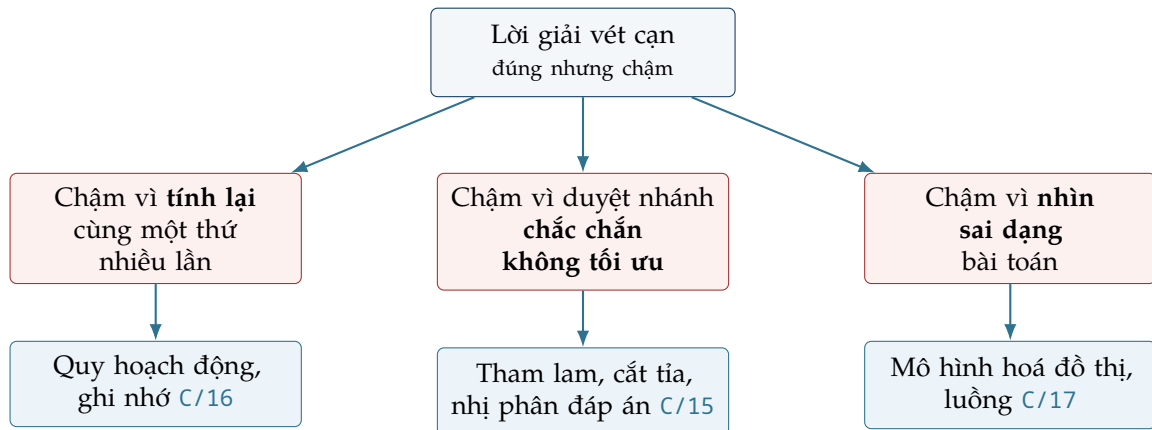
Đây là bước khó nhất và cũng là bước Pólya viết nhiều nhất. Câu hỏi trung tâm của ông rất đơn giản: *bạn đã gặp bài nào giống bài này chưa?* Không phải giống hệt — giống ở một khía cạnh nào đó. Cùng dạng dữ liệu vào, cùng dạng kết quả, cùng cấu trúc ràng buộc.

Trong lập trình, bước 2 có một hình thức rất cụ thể mà tôi sẽ nhắc lại nhiều lần trong quyển sách này: **luôn bắt đầu bằng vết cạn**. Viết ra — bằng lời hoặc bằng mã giả — một lời giải chắc chắn đúng dù chậm. Việc này có ba tác dụng, và cả ba đều lớn.

Tác dụng thứ nhất: nó buộc bạn phát biểu chính xác cái gì đang được tìm. Rất nhiều lúc bí là do bài toán trong đầu bạn vẫn còn mờ, và vết cạn làm nó rõ.

Tác dụng thứ hai: nó cho bạn một *lời giải đúng để đối chiếu*. Sau này, khi có lời giải nhanh, bạn cho hai chương trình chạy trên hàng nghìn test ngẫu nhiên nhỏ và so kết quả — kỹ thuật stress test ở chương E/25, và là kỹ thuật gỡ lỗi hiệu quả nhất trong nghề.

Tác dụng thứ ba, và là tác dụng quan trọng nhất về mặt tư duy: nó cho bạn một câu hỏi tốt hơn để suy nghĩ. Thay vì câu hỏi mù mờ “làm sao giải bài này”, bạn có câu hỏi sắc bén “*vì sao lời giải vết cạn này chậm, và công việc nào trong đó là thừa*”. Hình 1.1 cho thấy vì sao câu hỏi thứ hai dễ trả lời hơn hẳn: ba câu trả lời phổ biến dẫn thẳng tới ba nhóm kỹ thuật lớn của phần C.



Hình 1.1. Cây quyết định đi từ lời giải vết cạn tới nhóm kỹ thuật cần dùng. Câu hỏi “vì sao vết cạn chậm” có ít câu trả lời hơn hẳn câu hỏi “giải bài này thế nào”, nên nó dễ trả lời hơn — đó là toàn bộ giá trị của việc viết vết cạn ra trước.

Bước 3 – Thực hiện, và kiểm tra từng bước

Pólya nhấn mạnh một điều mà dân lập trình hay quên: khi thực hiện kế hoạch, phải kiểm tra từng bước có đúng không, và phải phân biệt được giữa “tôi thấy nó đúng” với “tôi chứng minh được nó đúng”. Trong lập trình, bước này gồm ba việc: chuyển ý tưởng thành mã, viết được các bất biến ra thành lời, và chạy thử trên ví dụ tự làm tay ở bước 1.

Có một quy tắc thực dụng đáng nhớ: *đừng gõ code khi bạn chưa nói được lời giải bằng ba câu*. Nếu bạn không tóm tắt được ý tưởng thành ba câu, tức là kế hoạch chưa xong, và gõ code lúc này chỉ chuyển sự mơ hồ từ đầu bạn vào trình soạn thảo — nơi nó khó sửa hơn nhiều.

Bước 4 – Nhìn lại: bước bị bỏ nhiều nhất, mất mát lớn nhất

Sau khi bài chạy đúng, gần như mọi người đóng bài và làm bài tiếp theo. Đây là lỗi tốn kém nhất trong toàn bộ việc luyện thuật toán, vì bước 4 chính là bước biến một lần giải bài thành một mẫu kiến thức dùng lại được.

Bước 4 gồm ba câu hỏi. Thứ nhất: *lời giải có thể ngắn hơn, nhanh hơn, hay đơn giản hơn không*. Thứ hai: *ý tưởng cốt lõi ở đây là gì, phát biểu bằng một câu, không nhắc tới chi tiết đề bài*. Thứ ba: *tôi đã bí ở đâu, và dấu hiệu nào lẽ ra đã giúp tôi thoát sớm hơn*. Câu thứ hai tạo ra thứ mà người luyện lâu năm gọi là “mẫu” — ví dụ “khi phải trả lời nhiều truy vấn tổng đoạn trên mảng tĩnh, hãy tính trước tổng tiền tố”. Câu thứ ba tạo ra nhật ký lỗi, công cụ được nói kỹ ở chương E/28.

CẦN NHỚ

Bốn bước không phải một đường thẳng mà là một vòng lặp có phản hồi. Bí ở bước 2 thì quay lại bước 1 (thường bạn hiểu sai đề); sai ở bước 3 thì quay lại bước 2 (thường kế hoạch có lỗ hổng, không phải code có lỗi gõ). Chỗ đáng ngờ nhất luôn là bước *ngay trước* chỗ bạn đang mắc.

1.2 Mười hai câu hỏi để hỏi khi bí

Phần này là công cụ thực dụng nhất của chương. Khi bí, đừng cố “nghĩ mạnh hơn” — đó là lời khuyên vô nghĩa vì nó không thay đổi thông tin bạn đang có. Thay vào đó, chạy qua danh sách dưới đây và tự hỏi từng câu. Mỗi câu hỏi có khả năng làm lộ ra một cấu trúc mà cách nhìn hiện tại của bạn đang che mắt. Danh sách này tổng hợp từ Pólya[1], Zeitz[4], Engel[5] và kinh nghiệm lập trình thi đấu.

1. **Làm ví dụ nhỏ nhất ra giấy.** $n = 1, 2, 3, 4$, tự tính đáp án bằng tay, xếp thành bảng, nhìn xem có quy luật gì. Đây là câu hỏi số một vì nó có tỉ lệ thành công cao nhất và chi phí thấp nhất.
2. **Bài này giống bài nào tôi từng giải?** Giống ở dữ liệu vào, ở kết quả ra, hay ở cấu trúc? Nếu giống một phần, phần khác biệt nằm ở đâu?
3. **Bỏ bớt một ràng buộc thì bài có dễ đi không?** Nếu mọi số đều dương thì sao? Nếu đồ thị là cây thì sao? Giải bài dễ hơn trước, rồi hỏi cách bù lại ràng buộc đã bỏ. Pólya gọi đây là “bài toán phụ”.
4. **Đảo ngược hướng nhìn.** Thay vì tìm cách tốt nhất, hãy đếm cách xấu và trừ đi. Thay vì đi từ đầu tới cuối, đi từ cuối về đầu. Thay vì hỏi “đáp án là bao nhiêu”, hỏi “với giá trị x cho trước, đáp án có $\geq x$ không” — câu này chính là cửa vào kỹ thuật nhị phân trên đáp án ở chương C/19.
5. **Sắp xếp lại dữ liệu thì sao?** Rất nhiều bài trở nên hiển nhiên sau khi sắp xếp, vì sắp xếp áp một thứ tự lên dữ liệu và thứ tự là thứ mà thuật toán tham lam cần.
6. **Có đại lượng nào không đổi không?** Tổng, chẵn lẻ, tổng theo modulo, số nghịch thế. Bất biến là công cụ mạnh nhất để chứng minh *không thể* làm được điều gì đó.
7. **Trường hợp cực đoan nói gì?** Cái nhỏ nhất, cái lớn nhất, cái ở biên. Lời giải tham lam thường sống ở đó.
8. **Có thể chia bài thành hai nửa độc lập không?** Nếu có thì chia để trị hoặc gặp nhau ở giữa. Nếu không, chỗ “đính” giữa hai nửa chính là trạng thái của quy hoạch động.
9. **Phát biểu lại bằng ngôn ngữ khác.** Bài mảng phát biểu lại thành bài đồ thị, bài đồ thị thành bài ma trận, bài đếm thành bài xác suất. Đổi ngôn ngữ hay làm hiện ra thuật toán có sẵn.
10. **Cái gì thực sự thay đổi giữa hai bước liên tiếp?** Nếu đáp án của bước $i + 1$ chỉ khác đáp án bước i một ít, bạn có lời giải tăng dần thay vì tính lại từ đầu — đó là ý tưởng nền của hai con trỏ, cửa sổ trượt và cấu trúc dữ liệu động.
11. **Nếu tôi được biết trước một đại lượng thì bài có dễ không?** Nếu “giá mà tôi biết phần tử lớn nhất nằm ở đâu” làm bài trở nên dễ, thì hãy duyệt qua mọi vị trí có thể của nó — kỹ thuật cố định một biến.

12. **Giới hạn đang mách gì?** $n \leq 20$ mách vét cạn 2^n ; $n \leq 10^5$ mách $O(n \log n)$; $n \leq 500$ mách $O(n^3)$; tổng n qua các test được chặn thì mách lời giải tuyến tính theo tổng. Chương A/03 có bảng đầy đủ.

TRỰC GIÁC

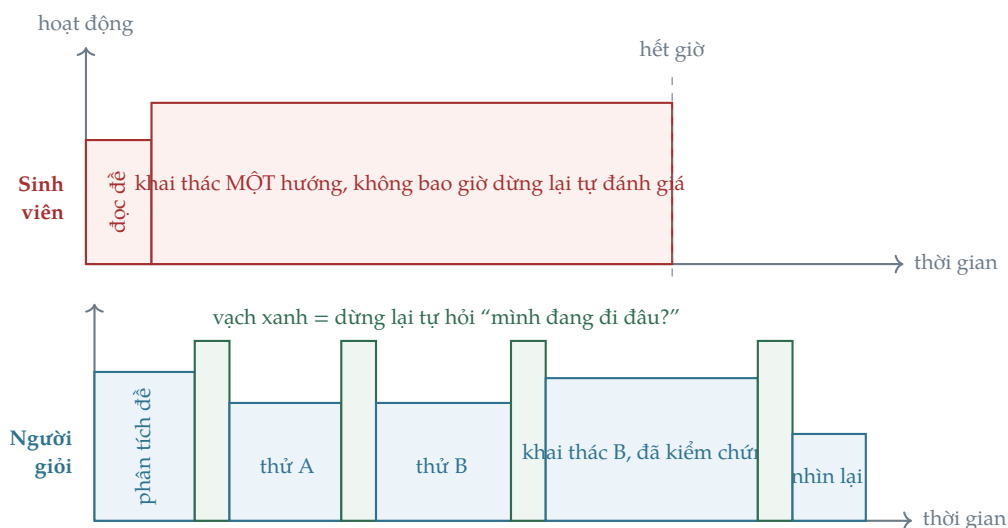
Vì sao một danh sách câu hỏi lại hiệu quả hơn “suy nghĩ tự do”? Vì bí không phải trạng thái thiếu năng lực tư duy, mà là trạng thái *lập lại một cách nhìn*. Đầu bạn cứ chạy lại đúng biểu diễn cũ của bài toán và tất nhiên thu được đúng kết luận cũ. Mỗi câu hỏi trong danh sách buộc bạn thay đổi biểu diễn — vẽ ra, đảo chiều, bỏ ràng buộc, đổi ngôn ngữ. Ý tưởng mới gần như luôn đến từ biểu diễn mới, chứ không từ việc nhìn chăm chăm biểu diễn cũ lâu hơn.

1.3 Điều Schoenfeld quay được trên video

Đến đây có một câu hỏi đáng đặt: nếu bốn bước và mười hai câu hỏi đơn giản như vậy, vì sao không phải ai đọc xong cũng giải bài giỏi lên? Alan Schoenfeld có câu trả lời dựa trên dữ liệu, và nó thay đổi cách nhìn của tôi về việc luyện tập.

Ông quay video các nhóm sinh viên giải toán và so với video các nhà toán học chuyên nghiệp giải cùng loại bài[2]. Điểm khác biệt lớn nhất không nằm ở kiến thức, cũng không nằm ở việc ai biết nhiều heuristic hơn. Nó nằm ở *cách phân bổ thời gian*. Sinh viên điển hình đọc đề, chọn một hướng trong vòng vài phút, rồi đi theo hướng đó cho tới hết giờ — kể cả khi hướng đó không dẫn tới đâu. Nhà toán học thì liên tục dừng lại tự hỏi: “mình đang làm gì đây? cái này có dẫn tới đâu không? đã bao lâu rồi? có nên thử hướng khác không?”.

Schoenfeld gọi tầng đó là *control* — tạm dịch là tự giám sát. Ông tổng kết rằng năng lực giải bài gồm bốn thành phần: kiến thức nền, các heuristic, khả năng tự giám sát, và hệ niềm tin về việc giải toán là gì. Thành phần thứ ba là thứ tạo ra khác biệt lớn nhất giữa hai nhóm, và cũng là thành phần dễ luyện nhất — vì nó chỉ đòi hỏi bạn đặt đồng hồ và tự hỏi vài câu.



Hình 1.2. Hai kiểu phân bổ thời gian mà Schoenfeld quan sát được. Người giải kém dành gần như toàn bộ thời gian để khai thác hướng đầu tiên nghĩ ra. Người giải giỏi cắt quá trình bằng những lần dừng ngắn để tự đánh giá, nhờ đó bỏ được hướng sai sớm. Điểm mấu chốt: các vạch xanh gần như không tốn thời gian, nhưng chúng quyết định phần thời gian còn lại được dùng vào việc gì.

Bài học thực hành rất cụ thể. Khi luyện, hãy đặt một mốc thời gian — ví dụ 10 phút — và mỗi khi tới mốc thì dừng lại 20 giây để trả lời ba câu: *tôi đang thử hướng gì, hướng này đã cho tôi tiến bộ nào chưa, nếu 10 phút nữa vẫn thế thì tôi sẽ chuyển sang hướng nào*. Ba câu này nghe máy móc, nhưng chúng chính là thứ mà người giỏi làm tự động và người mới không làm bao giờ.

1.4 Một ví dụ đi hết bốn bước

Lý thuyết trên sẽ mờ nếu không có một ví dụ chạy hết. Hãy lấy một bài kinh điển, đủ nhỏ để làm tay và đủ giàu để lộ ra cách nghĩ: *cho một mảng số nguyên có cả âm lẫn dương, tìm đoạn con liên tiếp có tổng lớn nhất*. Đây là bài mà Jon Bentley kể lại trong *Programming Pearls*[6] như một ví dụ mẫu mực về việc một bài toán có thể được giải lại nhiều lần, mỗi lần tốt hơn.

Bước 1 — hiểu. Dữ liệu vào là mảng $a_1, \dots, a_n$, kết quả là số lớn nhất trong các tổng $a_i + a_{i+1} + \dots + a_j$. Phát biểu lại bằng lời của mình: “chọn một đoạn liên nhau sao cho tổng lớn nhất”. Trường hợp biên đáng ngờ ngay: nếu mọi phần tử đều âm thì sao? Tuỳ quy ước — hoặc lấy phần tử lớn nhất, hoặc cho phép đoạn rỗng với tổng 0. Đây chính là loại câu hỏi mà bước 1 phải làm rõ, vì nó quyết định code.

Bước 2 — kế hoạch. Vét cạn trước: duyệt mọi cặp (i, j) , tính tổng, lấy lớn nhất. Chi phí $O(n^3)$ nếu tính tổng lại từ đầu mỗi lần, $O(n^2)$ nếu cộng dồn. Bây giờ dùng câu hỏi trung tâm: *công việc nào đang bị lặp?* Rõ ràng khi ta cố định j và thử mọi i , phần lớn công sức lặp lại giữa j và $j + 1$. Đó là dấu hiệu của câu hỏi số 10 trong danh sách: *cái gì thay đổi giữa hai bước liên tiếp?*

Hãy gọi B_j là tổng lớn nhất của một đoạn *kết thúc đúng tại j* . Câu hỏi trở thành: biết B_j , tính B_{j+1} thế nào? Đoạn tốt nhất kết thúc tại $j + 1$ hoặc là chỉ gồm mình a_{j+1} , hoặc là đoạn tốt nhất kết thúc tại j nối thêm a_{j+1} . Vậy

$$B_{j+1} = \max(a_{j+1}, B_j + a_{j+1}) = a_{j+1} + \max(0, B_j),$$

và đáp án là $\max_j B_j$. Công thức nhỏ này nói một điều rất trực giác: *nếu phần đuôi tích lũy được cho tới giờ đang âm, hãy vứt nó đi và bắt đầu lại*. Chi phí xuống $O(n)$, bộ nhớ $O(1)$.

Bước 3 — thực hiện. Bất biến cần giữ khi viết vòng lặp: sau khi xử lý xong vị trí j , biến cur bằng B_j và biến best bằng $\max_{k \leq j} B_k$. Viết được bất biến ra thành lời như vậy thì code gần như tự viết, và mọi câu hỏi kiểu “khởi tạo bằng 0 hay bằng $-\infty$ ” đều có câu trả lời từ bất biến chứ không phải từ việc thử.

Bước 4 — nhìn lại. Ý tưởng cốt lõi, phát biểu không nhắc đề bài: *khi tìm cực trị trên mọi đoạn, hãy cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó*. Câu này là một mẫu dùng lại được — nó giải luôn các bài tích lớn nhất, đoạn có tổng chia hết cho k , và là bước đầu tiên của rất nhiều bài quy hoạch động một chiều. So sánh: nếu bạn chỉ nhớ “bài tổng đoạn lớn nhất dùng thuật toán Kadane” thì bạn có một cái tên; nếu bạn nhớ câu vừa rồi thì bạn có một công cụ.

CẦN NHỚ

Bước 4 là chỗ biến một lần giải bài thành một mẫu kiến thức. Cách làm cụ thể: viết ra *một câu* nói ý tưởng cốt lõi mà không được nhắc tới chi tiết đề bài. Nếu không viết được câu đó, bạn chưa thực sự hiểu vì sao lời giải chạy được.

1.5 Vòng lặp bí — thoát bằng cách nào

Còn một tình huống chưa nói: bạn đã chạy qua danh sách câu hỏi, vẫn bí. Lúc này có ba chiến lược, xếp theo thứ tự nên thử.

Thứ nhất, giải một bài dễ hơn. Không phải bỏ cuộc mà là đổi mục tiêu tạm thời: giải bài với n nhỏ, hoặc với một ràng buộc thêm vào cho dễ, hoặc chỉ giải một trường hợp riêng. Rất thường xuyên, lời giải của bài dễ hơn chứa đựng ý tưởng cần cho bài gốc.

Thứ hai, viết ra chính xác chỗ bạn kẹt. Một câu, dạng “tôi biết cách làm nếu như ... nhưng tôi không biết cách xử lý khi ...”. Việc viết ra buộc sự mơ hồ phải thành hình, và một nửa số lần thì chính lúc viết bạn nhìn ra hướng đi.

Thứ ba, rời bài ra. Đây không phải lời khuyên tinh thần suông — nhiều người giải bài chuyên nghiệp nói về hiện tượng lời giải xuất hiện khi đang đi bộ hoặc tắm. Điều kiện để nó xảy ra là bạn phải đã nạp đủ vào đầu ở giai đoạn trước; rời bài mà chưa hiểu đề thì không có gì để áp cả.

Và một điều cuối, nói thẳng vì nó bị hiểu sai nhiều: *xem lời giải không phải điều xấu, xem lời giải quá sớm mới xấu.* Ngưỡng hợp lý khi luyện là 30–60 phút bí thật sự — bí sau khi đã chạy hết danh sách câu hỏi, chứ không phải bí sau khi đọc đề năm phút. Và khi đã xem, quy tắc bắt buộc là đóng lời giải lại rồi tự cài từ đầu, sau đó ghi vào nhật ký lỗi: *ý tưởng nào tôi đã không nghĩ ra, và dấu hiệu nào trên đề lẽ ra đã gợi ra nó.* Chương E/28 biến việc này thành một quy trình cụ thể.

1.6 Gốc rễ: quy trình này từ đâu ra

Bốn bước của Pólya trông như một danh sách hiển nhiên, nhưng nó là kết tinh của một truyền thống dài, và biết gốc rễ giúp bạn dùng nó đúng chỗ hơn.

Ý tưởng “có phương pháp để tìm ra lời giải, không chỉ để trình bày lời giải” xuất hiện từ Descartes trong *Quy tắc hướng dẫn tư duy* thế kỷ 17, và chính Pólya nhắc tới Descartes cùng Leibniz như những người đi trước. Điều thúc ép Pólya viết *How to Solve It* vào năm 1945 là một quan sát về dạy học: toán học được dạy như một *sản phẩm đã hoàn thành* — định lý, chứng minh, xong — trong khi quá trình thật sự tạo ra nó, gồm mò mẫm, thử ví dụ, đoán sai rồi sửa, thì bị giấu đi hoàn toàn. Người học vì thế tưởng rằng người giỏi “nhìn phát ra ngay”, và khi bản thân không nhìn ra ngay thì kết luận mình không có năng khiếu. Pólya viết sách để phơi bày phần bị giấu đó[1].

Đến thập niên 1970, ngành khoa học máy tính tiếp cận cùng câu hỏi từ hướng khác. Allen Newell và Herbert Simon nghiên cứu quá trình giải bài của con người bằng cách yêu cầu người ta *nói to suy nghĩ* khi giải, rồi mô hình hoá thành chương trình. Kết quả là khái niệm **không gian bài toán** (*problem space*) — giải bài tức là tìm đường trong một đồ thị các trạng thái — và **phân tích phương tiện–mục đích** (*means-ends analysis*): đo khoảng cách giữa trạng thái hiện tại và đích, rồi chọn hành động làm khoảng cách giảm. Đây là chỗ nổi rất đáng chú ý: cái mà ta gọi là “heuristic để nghĩ” trong chương này, và cái mà ta gọi là “hàm heuristic $h(n)$ ” trong thuật toán tìm kiếm A^* ở chương C/17, là *cùng một khái niệm*, ra đời từ cùng một dòng nghiên cứu. Tư duy của người và tìm kiếm của máy được mô hình hoá bằng cùng một ngôn ngữ.

Nhánh thứ ba là tâm lý học về sáng tạo. Graham Wallas mô tả bốn giai đoạn — chuẩn bị, ấp ủ, loé sáng, kiểm chứng — và giai đoạn “áp ủ” chính là cơ sở cho lời khuyên “rời bài ra” ở mục trước, kèm điều kiện quan trọng: phải nạp đủ ở giai đoạn chuẩn bị thì mới có gì để áp. Cuối cùng, Schoenfeld làm việc mà Pólya không làm được vì thiếu công cụ: đo thực nghiệm quá trình giải, và phát hiện ra rằng heuristic thôi chưa đủ, còn phải có tầng tự giám sát[2].

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thế kỷ 17 — Descartes, Leibniz	Tìm một “phương pháp phổ quát” để dẫn dắt tư duy, thay vì mỗi bài mỗi mẹo.	Đặt ra tham vọng có phương pháp; nhưng quá tổng quát nên không dùng trực tiếp được.
1945 — Pólya	Toán được dạy như sản phẩm hoàn chỉnh; người học không thấy quá trình mò mẫm nên tưởng mình thiếu năng khiếu.	Bốn bước và hệ thống câu hỏi; lần đầu quá trình giải bài trở thành thứ dạy được.
1972 — Newell & Simon	Muốn mô hình hoá tư duy đủ chính xác để viết thành chương trình.	Không gian bài toán và phân tích phương tiện–mục đích; heuristic trở thành khái niệm hình thức, dùng chung cho cả người lẫn máy ($\rightarrow A^*$, C/17).
1985 — Schoenfeld	Dạy heuristic Pólya cho sinh viên mà họ vẫn không giải được bài mới.	Phát hiện tăng tự giám sát; giải thích vì sao “biết heuristic” không đủ.
1993 — Ericsson	Vì sao cùng số giờ luyện mà người tiến người không.	Bốn điều kiện của luyện tập có chủ đích; nối chương này với chương A/02.

1.7 Liên hệ ngang: cùng một quy trình ở những nơi khác

Điều đáng để ý là quy trình bốn bước không phải đặc sản của toán hay của lập trình. Nó xuất hiện gần như y hệt ở mọi nghề mà công việc chính là *chẩn đoán trong điều kiện thiếu thông tin*, và nhìn thấy sự trùng khớp đó giúp bạn mượn được kỹ thuật từ nghề khác.

Ví dụ rõ nhất là gỡ lỗi phần mềm. Khi một chương trình sai, người có kinh nghiệm không đọc code từ đầu tới cuối; họ *chia đôi vùng nghi ngờ*: đặt một điểm kiểm tra ở giữa, xem trạng thái ở đó đúng hay sai, và loại bỏ một nửa. Đó chính là tìm kiếm nhị phân, áp lên không gian nguyên nhân thay vì lên mảng — và chương E/25 sẽ chỉ ra rằng git *bisect* là đúng thuật toán đó, đóng gói thành công cụ.

Ví dụ thứ hai đến từ chính công việc của người viết dòng này: gỡ một hệ thống *SLAM* chạy sai. Quy trình hiệu quả không phải đọc lại toàn bộ mã ước lượng, mà là: dựng *ví dụ nhỏ nhất* (một đoạn dữ liệu vài giây thay vì cả chuỗi), tìm một *bất biến* phải luôn đúng (hướng trọng lực không được trôi, hiệp phương sai phải xác định dương), rồi tìm bước đầu tiên mà bất biến bị vi phạm. Ba từ khoá — ví dụ nhỏ nhất, bất biến, bước hỏng đầu tiên — là đúng ba công cụ của chương này và chương A/04, chỉ đổi miền áp dụng.

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Khoa học thực nghiệm	Giả thuyết $\rightarrow$ thí nghiệm có khả năng bác bỏ $\rightarrow$ sửa giả thuyết	Giống: đều tiến bằng cách loại trừ, và một phản ví dụ mạnh hơn nhiều ví dụ ủng hộ. Khác: nhà khoa học không bao giờ có “đáp án đúng” để đối chiếu, còn bạn thì có — nên bạn được phép dùng vết cặn làm trọng tài.
Chẩn đoán y khoa	Chẩn đoán phân biệt: liệt kê mọi khả năng rồi loại bằng xét nghiệm rẻ nhất trước	Giống hệ heuristic “bỏ bớt ràng buộc” và “trường hợp cực đoan”. Khác: chi phí sai lệch bất đối xứng — bỏ sót bệnh nặng đắt hơn nhiều so với dương tính giả, nên thứ tự xét nghiệm không tối ưu theo thông tin thuần túy.
Gỡ lỗi phần mềm	Chia đôi vùng nghi ngờ, git <i>bisect</i>	Cùng thuật toán với tìm kiếm nhị phân, áp lên không gian nguyên nhân. Điều kiện áp dụng cũng giống: cần tính đơn điệu — trước điểm đó thì đúng, sau điểm đó thì sai (E/25).

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Ước lượng trạng thái, SLAM	Ví dụ nhỏ nhất + bất biến (trọng lực, hiệp phương sai) + tìm bước hỏng đầu tiên	Cùng bộ ba công cụ với chương này và A/04. Khác: hệ thực có nhiều, nên “bất biến” trở thành “nằm trong khoảng tin cậy”, và phải phân biệt sai số hợp lệ với lỗi thật.
Trí tuệ nhân tạo cổ điển	Không gian trạng thái, hàm heuristic $h(n)$, A^*	Không phải chỉ giống — cùng một khái niệm, cùng nguồn gốc từ Newell & Simon. Bạn dùng heuristic để cắt bớt nhánh trong đầu, A^* dùng nó để cắt bớt nhánh trong hàng đợi ưu tiên (C/17).
Cờ vua, cờ vây	Tính toán biến (calculation) xen kẽ với đánh giá thể (evaluation)	Giống tăng tự giám sát của Schoenfeld: kỹ thủ mạnh không tính sâu hơn nhiều, họ biết <i>khi nào nên ngừng tính một biến</i> . Xem thêm A/02.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì kiến thức chỉ là một trong bốn thành phần của năng lực giải bài. Người giải được bài mới có thêm ba thứ: một quy trình để chạy khi chưa biết làm gì, một bộ heuristic để đổi cách nhìn bài toán, và quan trọng nhất là khả năng tự giám sát — biết mình đang đi hướng nào, đi bao lâu rồi, và khi nào phải bỏ. Schoenfeld chỉ ra thành phần thứ ba tạo ra khác biệt lớn nhất trong dữ liệu thực nghiệm của ông.
- A2. Vì bí không phải trạng thái thiếu sức nghĩ, mà là trạng thái lặp lại một biểu diễn. “Nghĩ kỹ hơn” giữ nguyên biểu diễn nên cho ra đúng kết luận cũ. “Vẽ ví dụ $n = 4$ ” thì đổi biểu diễn từ ký hiệu trừu tượng sang hình ảnh cụ thể, và cấu trúc thường chỉ lộ ra ở biểu diễn mới.
- A3. Vì vết cạn cho ba thứ mà bạn không có nếu bỏ qua nó: một phát biểu chính xác về cái đang tìm, một lời giải đúng để đối chiếu khi stress test, và quan trọng nhất là một câu hỏi tốt hơn để suy nghĩ — “vì sao nó chậm” thay cho “giải thể nào”. Hình 1.1 cho thấy câu hỏi đó chỉ có vài câu trả lời khả dĩ, mỗi câu dẫn thẳng tới một nhóm kỹ thuật.
- A4. Bạn mất khả năng dừng lại. Bước 4 là chỗ ý tưởng được tách khỏi đề bài cụ thể và phát biểu thành một câu tổng quát — “cố định điểm cuối, duy trì đáp án tốt nhất kết thúc tại đó”. Không có bước 4, bạn tích lũy được những lần giải bài chứ không tích lũy được công cụ, nên bài sau vẫn phải nghĩ lại từ đầu.
- A5. Khi hướng đó đã chạy đủ lâu mà không tạo ra *tiến bộ kiểm chứng được* — không có bỏ đề mới, không có ví dụ nhỏ nào được giải, không có ràng buộc nào bị loại. Cách biết trước khi hết giờ là đặt mốc kiểm tra định kỳ, ví dụ 10 phút một lần, và trả lời ba câu ở mục trước. Không có mốc kiểm tra thì bạn chỉ phát hiện ra hướng sai khi đồng hồ đã hết.

NÓI LẠI THẬT ĐƠN GIẢN

Giải một bài lạ giống như tìm đường trong một thành phố lạ mà không có bản đồ. Việc đầu tiên không phải là chạy nhanh, mà là đứng lại xem mình đang ở đâu và cần tới đâu — đó là bước hiểu đề. Việc thứ hai là tìm một con đường bạn từng đi qua có nối tới đó không — đó là bước tìm mối nối, và mẹo hiệu quả nhất là đi bộ thử một đoạn ngắn bằng cách làm tay ví dụ nhỏ. Cách chậm nhưng chắc chắn tới nơi là đi thử hết mọi ngõ, tức vết cạn; bạn luôn nên hình dung cách đó trước, vì khi biết nó chậm ở chỗ nào thì bạn biết cần rẽ tắt ở đâu. Khi đã tới nơi rồi, hãy dành một phút nhìn lại con đường và ghi nhớ khúc quan trọng nhất — lần sau bạn sẽ không phải dò lại. Còn khi lạc, đừng đi nhanh hơn theo hướng cũ; hãy dừng lại và tự hỏi một câu hỏi khác.

Một câu: Khi bí, đừng nghĩ mạnh hơn — hãy đổi câu hỏi mà bạn đang tự hỏi.

Chỗ không rút gọn được: Không có cách nào biết chắc một hướng là sai trước khi thử; mọi thứ ta làm được chỉ là đặt mốc kiểm tra để phát hiện sớm và trả giá rẻ hơn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Bốn bước của Pólya	Hiểu bài toán → lập kế hoạch → thực hiện → nhìn lại. Không phải đường thẳng mà là vòng lặp: mắc ở bước sau thì quay lại bước ngay trước nó.
●●● Kiểm tra “đã hiểu đề”	Phát biểu lại đề bằng lời của mình, không nhìn vào đề. Chưa làm được thì mọi phút suy nghĩ tiếp theo đều phí.
●●● Vì sao luôn viết vết cạnh trước	Nó cho phát biểu chính xác bài toán, cho lời giải đúng để stress test, và đổi câu hỏi từ “giải thế nào” thành “vì sao chậm” — câu sau dễ trả lời hơn nhiều.
●●● Ba lý do vết cạnh chậm	Tính lại thứ đã tính (→ quy hoạch động); duyệt nhánh chắc chắn tẻ (→ tham lam, cắt tỉa); nhìn sai dạng bài (→ mô hình hoá lại).
●●● Tầng “control” của Schoenfeld	Khả năng tự giám sát: đang làm gì, có tiến bộ không, khi nào bỏ hướng. Đây là khác biệt lớn nhất giữa người giải giỏi và kém, và là thành phần dễ luyện nhất.
●● Heuristic hiệu quả nhất	Làm tay ví dụ nhỏ $n = 1, 2, 3, 4$ và xếp kết quả thành bảng. Chi phí thấp nhất, tỉ lệ ra ý tưởng cao nhất.
●● Heuristic “đảo ngược”	Đếm cái xấu rồi trừ; đi từ cuối về đầu; đổi “đáp án là bao nhiêu” thành “đáp án có $\geq x$ không” — câu cuối là cửa vào nhị phân trên đáp án.
●● Heuristic “bỏ bớt ràng buộc”	Giải bài dễ hơn trước (mọi số dương, đồ thị là cây, n nhỏ), rồi tìm cách bù lại ràng buộc đã bỏ.
●● Câu hỏi “cái gì đổi giữa hai bước”	Nếu đáp án bước $i + 1$ chỉ khác bước i một chút thì có lời giải tăng dần: hai con trỏ, cửa sổ trượt, cấu trúc dữ liệu động.
●● Quy tắc trước khi gõ code	Nói được lời giải bằng ba câu. Không nói được nghĩa là kế hoạch chưa xong, và gõ code chỉ chuyển sự mơ hồ vào trình soạn thảo.
●● Nội dung bước 4	Có ngắn/nhanh/đơn giản hơn không; ý tưởng cốt lõi phát biểu một câu không nhắc đề bài; tôi bí ở đâu và dấu hiệu nào lẽ ra giúp tôi thoát sớm.
●● Ba cách thoát khi hết heuristic	Giải một bài dễ hơn; viết ra chính xác chỗ kẹt dạng “tôi biết làm nếu... nhưng không biết khi...”; rời bài ra sau khi đã nạp đủ.
● Ngưỡng xem lời giải	30–60 phút bí thật sự, sau khi đã chạy hết danh sách câu hỏi. Xem xong bắt buộc đóng lại, tự cài từ đầu, và ghi nhật ký lỗi.
● Bốn thành phần của Schoenfeld	Kiến thức nền, heuristic, tự giám sát, và hệ niềm tin về việc giải toán là gì.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

- Vì sao Pólya nói “nếu bạn không giải được bài toán này, hãy tìm một bài dễ hơn liên quan tới nó”? Nghe như lời khuyên né tránh.

| Vì bài dễ hơn thường chứa *cùng cấu trúc* với bài gốc, chỉ bớt đi phần gây nhiễu. Khi giải được bài dễ, bạn có một lời giải cụ thể để soi, và câu hỏi trở thành “ràng buộc tôi đã bỏ đi làm hỏng chỗ nào của lời giải này” — một câu hỏi hẹp và cụ thể, dễ hơn hẳn câu hỏi mở ban đầu. Đây không phải né tránh mà là chia bài toán khó thành hai bước dễ hơn.

- Tại sao bảo “đọc giới hạn n là một nửa gợi ý”? Chẳng phải giới hạn chỉ là chi tiết kỹ thuật?

| Vì giới hạn loại bỏ phần lớn không gian tìm kiếm ý tưởng. Với $n \leq 20$, người ra đề gần như chắc chắn muốn một lời giải mũ, thường là duyệt 2^n tập con hoặc quy hoạch động trên bitmask — và bạn không cần phí thời gian tìm lời giải đa thức thông minh. Với $n \leq 2 \cdot 10^5$, lời giải phải cỡ $O(n)$ hoặc $O(n \log n)$, nên mọi ý tưởng $O(n^2)$ bị loại ngay và bạn tiết kiệm được nửa giờ.

- Nếu tôi đã biết bài này là quy hoạch động, tôi có cần bước 1 và bước 2 không?

| Có, và bước 1 lúc này còn quan trọng hơn. Biết “là quy hoạch động” chưa nói được trạng thái là gì — mà chọn sai trạng thái là cách hỏng phổ biến nhất của quy hoạch động. Trạng thái đúng thường lộ ra từ việc làm tay ví dụ nhỏ và từ câu hỏi “cái gì thực sự thay đổi giữa hai bước liên tiếp”, tức là từ bước 1 và bước 2.

4. Trong một kỳ thi có giới hạn thời gian, bốn bước của Pólya có xa xỉ quá không?

| Ngược lại, dưới sức ép thời gian thì bốn bước lại càng đáng giá, chỉ là mỗi bước ngắn lại. Kinh nghiệm thi đấu cho thấy chi phí lớn nhất không phải nghĩ chậm mà là *cài xong rồi mới phát hiện hiểu sai đề*, hoặc *đi một hướng sai 40 phút*. Bước 1 rút gọn còn hai phút và bước “dừng lại tự đánh giá” chỉ tốn 20 giây, nhưng chúng chặn đúng hai lỗi tốn kém nhất đó.

5. Làm sao phân biệt “bí lành mạnh” (nên tiếp tục) với “bí vô ích” (nên xem lời giải)?

| Bằng dấu hiệu tiến bộ, không bằng cảm giác. Bí lành mạnh là khi mỗi 10–15 phút bạn vẫn thu được thứ mới: một ví dụ nhỏ giải được, một hướng bị loại có lý do, một bất biến được phát hiện. Bí vô ích là khi bạn đọc lại đề lần thứ năm và nghĩ đúng những ý đã nghĩ. Dấu hiệu thứ hai xuất hiện thì hoặc đổi heuristic, hoặc chuyển bài, hoặc xem gợi ý — gợi ý một dòng, không phải toàn bộ lời giải.

6. Vì sao chương này nhấn mạnh “phát biểu ý tưởng bằng một câu không nhắc tới đề bài”?

| Vì đó là cách kiểm tra xem bạn đã trừu tượng hoá được chưa. Câu nói kèm chi tiết đề bài (“với mảng số trong bài này, ta duyệt từ trái sang”) chỉ dùng được cho đúng bài đó. Câu nói tổng quát (“khi tìm cực trị trên mọi đoạn, cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó”) dùng được cho hàng chục bài khác. Kiến thức chỉ dùng lại được ở dạng thứ hai.

7. Người ta hay nói “giải nhiều bài thì tự khắc giỏi”. Chương này có phản đối điều đó không?

| Có, một phần. Số lượng chỉ có tác dụng nếu mỗi bài đều đi qua bước 4 và nếu độ khó được chọn hơi cao hơn mức hiện tại của bạn — đúng định nghĩa luyện tập có chủ đích của Ericsson[3]. Giải một trăm bài vừa sức, không nhìn lại, không ghi lỗi, thì bạn luyện tốc độ gõ chứ không luyện năng lực giải bài. Chương E/28 biến nhận xét này thành một lịch luyện cụ thể.

8. Bất biến xuất hiện ở đây làm gì, chẳng phải nó thuộc về phần chứng minh?

| Bất biến có hai vai trò, và vai trò sinh ý tưởng thường bị bỏ quên. Khi bạn hỏi “đại lượng nào không đổi trong quá trình này”, bạn vừa đang tìm công cụ chứng minh, vừa đang tìm chính cấu trúc để thiết kế thuật toán — ví dụ bất biến “cur là tổng lớn nhất của đoạn kết thúc tại j ” vừa là lời giải vừa là chứng minh của bài tổng đoạn lớn nhất. Chương A/04 khai thác cả hai vai trò.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn con lớn nhất	CSES “Maximum Subarray Sum”	Đi lại đúng bốn bước của mục 4; viết bất biến ra thành lời trước khi gõ code.
● Đảo ngược một số nguyên, xử lý tràn	LeetCode “Reverse Integer”	Bước 1 quyết định tất cả: liệt kê trường hợp biên (số âm, tận cùng bằng 0, tràn 32 bit) trước khi viết.
● Hai số có tổng bằng x	CSES “Sum of Two Values”	Heuristic “sắp xếp lại dữ liệu” hoặc “đổi sang tra cứu”; so sánh hai lời giải và giải thích khi nào chọn cái nào.
●● Bài toán tám quân hậu	CSES “Chessboard and Queens”	Vết cạn có cắt tỉa; luyện thói quen ước lượng kích thước không gian tìm kiếm <i>trước</i> khi cài.
●● Tìm phần tử xuất hiện quá nửa	LeetCode “Majority Element”	Heuristic “đại lượng bất biến”: thuật toán bỏ phiếu Boyer–Moore. Hãy tự tìm bất biến trước khi tra.
●● Tích đoạn con lớn nhất	LeetCode “Maximum Product Subarray”	Cùng mẫu với bài mở đầu nhưng số âm phá cấu trúc: cần giữ đồng thời cực đại và cực tiểu. Ví dụ nhỏ lộ ra ngay.
●● Trò chơi Nim đơn giản	Bài tập kinh điển	Heuristic “bất biến” và “trường hợp cực đoan”: làm tay các thế $n \leq 3$ trước khi tìm quy luật XOR.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Số cách leo cầu thang với bước 1 và 2	AtCoder DP “Frog 1”	Bài mẫu để tập bước 4: phát biểu ý tưởng bằng một câu không nhắc tới cầu thang.
●●● Bài toán 3-SUM	Bài tập kinh điển	Từ $O(n^3)$ xuống $O(n^2)$: cố định một biến rồi dùng hai con trỏ. Luyện heuristic “giá mà tôi biết trước một đại lượng”.
●●● Xếp lịch cuộc họp tối đa	Bài tập kinh điển	Bí thì hỏi “trường hợp cực đoan nói gì”: chọn theo thời điểm kết thúc sớm nhất. Thử tự chứng minh bằng lập luận đối chỗ (chương A/04).
●●● Bài toán “nước mưa mắc kẹt”	LeetCode “Trapping Rain Water”	Ba lời giải khác nhau (tiền tố cực đại, ngăn xếp, hai con trỏ). Giải cả ba rồi so, đây là bài tập bước 4 tốt nhất trong danh sách này.
●●● Một bài bạn từng bí và đã xem lời giải	Nhật ký của chính bạn	Tự cài lại không nhìn, rồi viết một câu về ý tưởng cốt lõi và một câu về dấu hiệu lẽ ra đã gợi ra nó.

Kết chương và đọc tiếp

Chương này cố ý không chứa thuật toán nào. Nó dựng một quy trình bốn bước, một danh sách mười hai câu hỏi để phá thế bí, và một thói quen tự giám sát bằng mốc thời gian. Ba thứ đó là khung để treo mọi kiến thức kỹ thuật của 27 chương còn lại; thiếu chúng, kiến thức chỉ nằm thành danh mục và không tự tìm được đường ra khi cần.

Còn bỏ ngỏ một câu hỏi lớn: quy trình thì có rồi, nhưng *luyện* thế nào để nó thành phản xạ, và những người giỏi nhất thực sự luyện ra sao? Đó là nội dung chương [A/02](#). Sau đó, chương [A/03](#) và [A/04](#) cấp hai cái thước — nhanh không, đúng không — mà bước 2 và bước 3 của Pólya luôn cần tới.

Kinh nghiệm của những người giỏi nhất (What the Masters Actually Do)

PHỤ THUỘC A/01 — chương này nói cách luyện để quy trình ở chương 1 thành phản xạ. Đọc được độc lập, nhưng nhiều lời khuyên ở đây chỉ có nghĩa khi bạn đã biết bốn bước.

TÓM TẮT

Chương này khảo sát cách làm việc thật của những người đã định hình ngành thuật toán — Knuth, Dijkstra, Hoare, Tarjan, Bentley, Skiena, Sedgewick — cùng cách luyện của giới lập trình thi đấu hiện đại và của giới huấn luyện olympic toán. Mục đích không phải kể chuyện danh nhân mà là rút ra những thói quen *bắt buộc được*: viết chương trình cùng lúc với chứng minh, ước lượng trước khi cài, luyện ở vùng hơi quá sức, upsolving sau mỗi trận, giữ nhật ký lỗi, tự xây thư viện mẫu. Chương cũng cho một bản đồ sách: quyển nào đọc lúc nào và để làm gì. Nếu chỉ nhớ một điều: điểm chung của tất cả những người này không phải năng khiếu mà là *họ có một vòng phản hồi chặt* — làm, đo, phát hiện sai, sửa cách làm, lặp lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **luyện tập có chủ đích (deliberate practice)** — luyện có mục tiêu cụ thể, ở mức hơi quá sức hiện tại, có phản hồi ngay và có sửa sai — khác hẳn với “làm nhiều giờ”. Khái niệm của Ericsson[3].
- **upsolving** — giải lại những bài bạn không giải được trong một kỳ thi, *sau khi* thi xong, tự làm tới cùng. Đây là nguồn tiên bộ chính của dân thi đấu.
- **editorial** — bài phân tích lời giải chính thức của một đề thi lập trình.
- **chunk (khối tri thức)** — một cụm thông tin đã được ghép thành một đơn vị nhớ duy nhất, ví dụ “thế cờ này” thay vì hai mươi quân rời. Chuyên gia nhớ nhiều hơn không phải vì trí nhớ tốt hơn mà vì họ có nhiều khối lớn hơn.
- **thư viện mẫu (template library)** — tập hợp các đoạn mã đã kiểm chứng — DSU, segment tree, Dijkstra — được chuẩn bị sẵn để dán vào khi thi. KACTL của KTH là ví dụ nổi tiếng[7].
- **nhật ký lỗi** — sổ ghi lại mỗi lần sai: sai gì, vì sao, dấu hiệu nào lẽ ra đã phát hiện sớm. Công cụ đơn giản nhất và bị bỏ qua nhiều nhất.

CÂU HỎI MỞ ĐẦU

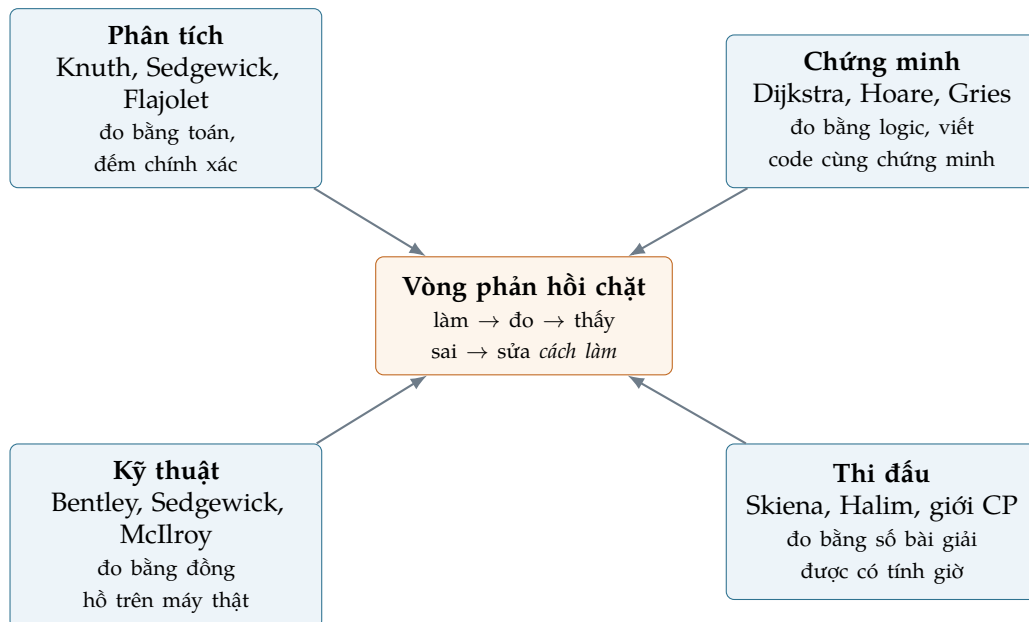
- Q1. Nếu tài năng là thứ quyết định, vì sao gần như mọi người giỏi thuật toán đều mô tả cùng một kiểu thói quen làm việc?
- Q2. Knuth ra bài tập có mức khó từ 0 đến 50 và bảo người đọc phải làm. Vì sao đọc mà không làm bài tập lại gần như vô ích?
- Q3. Dijkstra viết chương trình và chứng minh cùng lúc, không phải viết xong rồi kiểm thử. Điều đó thay đổi cái gì trong cách bạn ngồi trước bàn phím?
- Q4. Vì sao “giải thật nhiều bài” lại có thể khiến bạn dừng tiến bộ sau vài tháng, trong khi giải ít bài hơn nhưng chọn kỹ thì tiếp tục lên?
- Q5. Người thi đấu hàng đầu nói phần lớn tiến bộ đến từ upsolving chứ không từ chính kỳ thi. Cơ chế nào giải thích điều đó?

Có một cách đọc chương này sai ngay từ đầu, và đáng nói trước: đọc như đọc tiểu sử danh nhân, thấy hay, rồi không thay đổi gì. Vì vậy mỗi mục dưới đây được viết theo một khuôn cố định — người này làm gì cụ thể, vì sao nó hiệu quả, và bạn bắt chước được phần nào ngay tuần này. Những gì không bắt chước được, tôi sẽ nói thẳng là không bắt chước được.

Cũng cần nói rõ giới hạn: những phát biểu dưới đây dựa trên sách và tài liệu công khai của chính họ, không phải suy đoán về đời tư. Khi tôi mô tả thói quen của giới thi đấu, đó là những điều lặp đi lặp lại trong lời khuyên công khai của nhiều người giỏi, chứ không phải câu trích nguyên văn của một cá nhân cụ thể.

2.1 Bốn trường phái, một vòng phản hồi

Nếu xếp những người có ảnh hưởng nhất lên một sơ đồ, họ rơi vào bốn trường phái với bốn niềm tin khác nhau về việc “thế nào là làm thuật toán cho tốt”. Điều thú vị là bốn trường phái này không mâu thuẫn; chúng bổ sung nhau, và người mạnh thường lấy từ cả bốn.



Hình 2.1. Bốn trường phái khác nhau ở thứ họ dùng làm thước đo, nhưng giống nhau ở chỗ đều có một thước đo khách quan và đều sửa cách làm dựa trên kết quả đo. Người luyện một mình mà không có thước đo nào là người duy nhất không có vòng phản hồi — và đó là kiểu luyện phổ biến nhất.

Trường phái phân tích: Knuth và việc đếm cho chính xác

Donald Knuth xây cả sự nghiệp trên một niềm tin: muốn hiểu một thuật toán thì phải *đếm* được nó làm bao nhiêu phép toán, chứ không chỉ nói “cỡ $n \log n$ ”. Trong *The Art of Computer Programming* [8], [9], các thuật toán được phân tích tới hằng số, trên một máy ảo mà ông tự thiết kế để không phải phụ thuộc vào máy thật nào.

Hai thói quen của Knuth đáng bắt chước ngay. Thứ nhất là *bài tập có xếp hạng độ khó*: mỗi bài trong TAOCP có một số từ 0 đến 50, trong đó khoảng 20 là bài tập bình thường, còn 50 là bài toán mở nghiên cứu. Ý nghĩa của cách xếp hạng đó không phải để khoe độ khó mà để người đọc *chọn đúng vùng luyện* — ý tưởng mà mục sau sẽ nói kỹ. Thứ hai là thói quen coi việc phát hiện lỗi là có giá trị: Knuth nổi tiếng vì trả tiền thưởng cho mỗi lỗi được tìm thấy trong sách của ông. Thông điệp ẩn sau đó rất thực dụng: một người muốn đúng thì phải *tạo động lực cho việc tìm ra mình sai*, chứ không phòng thủ với nó.

Cũng của Knuth là câu bị trích sai nhiều nhất trong ngành: “tối ưu hoá sớm là gốc rễ của mọi tội lỗi”[10]. Nguyên văn nói rõ hơn nhiều — ý ông là trong phần lớn thời gian ta nên bỏ qua những cải thiện nhỏ, nhưng *không được* bỏ qua cơ hội ở phần mã thực sự quan trọng, và cách biết phần nào quan trọng là đo chứ không đoán. Đây chính là cầu nối sang trường phái kỹ thuật.

Trường phái chứng minh: Dijkstra, Hoare và kỷ luật viết code

Edsger Dijkstra có một quan điểm cực đoan và rất hữu ích: chương trình và chứng minh tính đúng của nó phải được phát triển *cùng lúc*, chứ không phải viết code trước rồi kiểm thử sau[11]. Ông cũng để lại câu nói chính xác về giới hạn của kiểm thử: kiểm thử cho thấy sự *hiện diện* của lỗi, không bao giờ cho thấy sự *vắng mặt* của lỗi.

Nghe có vẻ hàn lâm, nhưng hệ quả thực hành rất cụ thể và rất rẻ: trước khi viết một vòng lặp, hãy viết ra — bằng lời, trong một dòng chú thích — *bất biến* mà vòng lặp đó duy trì. Chỉ một dòng đó thôi đã loại được phần lớn lỗi lệch một đơn vị và lỗi khởi tạo sai, vì mọi câu hỏi kiểu “chạy tới n hay n-1” đều có câu trả lời suy ra từ bất biến. Tony Hoare cấp nền hình thức cho cách làm này[12], còn David Gries biến nó thành giáo trình dạy được[13]. Chương A/04 lấy đúng ý này làm trung tâm.

Cần nói cả mặt trái. Chứng minh hình thức đầy đủ cho mọi chương trình là không thực tế trong công việc hằng ngày, và Dijkstra bị phê phán chính ở điểm đó. Cách dùng hợp lý là dùng *ý thức về bất biến* chứ không phải bộ máy hình thức: viết bất biến ra thành lời, và chỉ hình thức hoá ở những chỗ thật sự tinh vi.

Trường phái kỹ thuật: Bentley và thói quen đo

Jon Bentley viết *Programming Pearls*[6] như những chuyên mục ngắn, mỗi bài xoay quanh một bài toán thật gặp trong công việc. Ba thói quen của ông dùng được ngay.

Thứ nhất là *ước lượng phong bì* — tính nhằm quy mô trước khi cài: bao nhiêu phần tử, mỗi phần tử bao nhiêu byte, một giây làm được cỡ bao nhiêu phép toán. Ước lượng thô loại được những hướng sai hằn với chi phí gần bằng không, và nó là kỹ năng dùng suốt đời chứ không chỉ trong thi đấu.

Thứ hai là quan sát nổi tiếng của ông về *tìm kiếm nhị phân*: ai cũng nghĩ mình viết đúng, còn thực tế thì rất nhiều lập trình viên chuyên nghiệp viết sai khi được yêu cầu viết từ đầu không tra cứu. Bài học không phải “tìm kiếm nhị phân khó” mà là: một thuật toán *đơn giản về ý tưởng* vẫn có thể rất khó viết đúng, nên phần cài đặt xứng đáng được luyện riêng như một kỹ năng độc lập.

Thứ ba là kỷ luật *đo trước khi tối ưu*. Trong bài viết cùng McIlroy về việc kỹ thuật hoá hàm sắp xếp của thư viện C[14], hai người mô tả cách một thuật toán sách vở phải chỉnh sửa nhiều lần mới sống được trong môi trường thật — với dữ liệu thù địch, với mảng đã sắp sẵn, với nhiều phần tử bằng nhau. Đây là ví dụ mẫu mực cho chương E/26.

Trường phái thi đấu: nhận diện bài toán quan trọng hơn thuộc thuật toán

Steven Skiena tổ chức *The Algorithm Design Manual*[15] thành hai nửa rất khác nhau: nửa đầu dạy kỹ thuật, nửa sau là một *danh mục bài toán* — bạn tra “tôi đang có bài toán trông như thế này” và sách chỉ cho bạn nó tên là gì, đã được giải chưa, và nên dùng thư viện nào. Cách tổ chức đó phản ánh một quan sát cốt lõi: trong thực tế, phần khó nhất thường không phải nghĩ ra thuật toán mà là *nhận ra bài toán của mình chính là bài toán kinh điển nào*.

Giới lập trình thi đấu đẩy quan sát này tới cùng. Bộ sách *Competitive Programming*[16] và *Guide to Competitive Programming*[17] tổ chức kiến thức thành các mẫu nhận dạng kèm bài luyện; các nguồn cộng đồng như cp-algorithms[18] và thư viện KACTL[7] cấp phần cài đặt đã kiểm chứng. Điểm chung trong lời khuyên luyện tập của những người ở nhóm dẫn đầu, lặp lại rất nhất quán

qua nhiều nguồn công khai, gồm bốn điểm: giải bài *hơi quá sức*, upsolving sau mỗi trận, tự cài lại thay vì đọc code người khác, và luyện có tính giờ.

I Kinh nghiệm người đi trước

Người / trường phái	Điều họ thực sự làm	Nguồn
Donald Knuth	Phân tích tới hằng số, không chỉ tới bậc tiệm cận. Bài tập xếp hạng 0–50 để người đọc chọn đúng vùng luyện. Thường tiền cho ai tìm ra lỗi trong sách của mình.	TAOCP[8], [9]
Edsger Dijkstra	Phát triển chương trình và chứng minh cùng lúc; viết bất biến trước khi viết vòng lặp; coi kiểm thử là công cụ phát hiện lỗi chứ không phải bằng chứng đúng.	<i>A Discipline of Programming</i> [11]
Tony Hoare	Đặt nền logic cho lập luận về chương trình (tiền điều kiện, hậu điều kiện); Quicksort ra đời từ việc tìm cách sắp xếp trước khi tra từ điển — ý tưởng đến từ ràng buộc thực tế.	[12], [19]
Robert Tarjan	Thiết kế cấu trúc dữ liệu song song với công cụ phân tích của nó: khẩu hao và hàm thể năng ra đời để phân tích chính những cấu trúc ông thiết kế.	[20], [21]
Jon Bentley	Ước lượng phong bì trước khi cài; đo trước khi tối ưu; coi việc viết đúng một thuật toán đơn giản là kỹ năng phải luyện riêng.	<i>Programming Pearls</i> [6]
Robert Sedgewick	Kết hợp phân tích toán học với thực nghiệm: cài đặt, đo, rồi đối chiếu với dự đoán lý thuyết; nếu lệch thì một trong hai phía sai.	[22], [23]
Steven Skiena	Xây danh mục bài toán để tra ngược từ “bài của tôi trông thế này”; nhấn mạnh nhận diện bài toán hơn là sáng tạo thuật toán mới.	<i>The Algorithm Design Manual</i> [15]
George Pólya	Bốn bước và một hệ thống câu hỏi để tự hỏi khi bí; dạy <i>quá trình</i> chứ không chỉ kết quả.	<i>How to Solve It</i> [1]
Alan Schoenfeld	Đo thực nghiệm quá trình giải bài; chỉ ra rằng tự giám sát là khác biệt lớn nhất giữa người giỏi và người kém.	[2]
Anders Ericsson	Luyện tập có chủ đích: mục tiêu hẹp, mức hơi quá sức, phản hồi ngay, sửa sai lập tức. Số giờ không tự sinh ra năng lực.	[3], [24]
Richard Hamming	Chọn bài toán quan trọng để làm; giữ thói quen suy nghĩ có hệ thống về việc mình đang làm gì và vì sao.	<i>The Art of Doing Science and Engineering</i> [25]
Peter Norvig	Không có đường tắt mười ngày; năng lực đến từ nhiều năm luyện có phản hồi, đọc code người khác, và làm dự án thật.	[26]
Giới lập trình thi đấu	Bốn thói quen lặp lại nhất quán: giải bài hơi quá sức, upsolving sau mỗi trận, tự cài lại thay vì chép, luyện có tính giờ; kèm một thư viện mẫu tự viết và đã kiểm chứng.	[7], [16], [17]

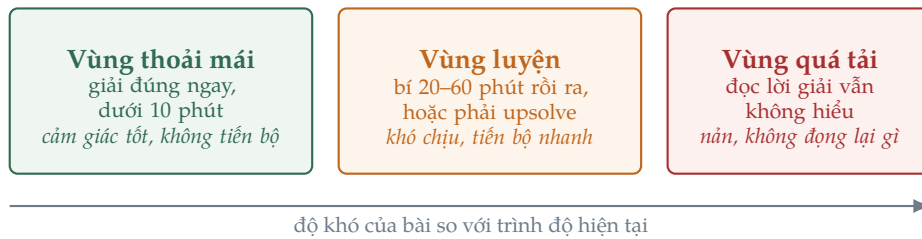
I 2.2 Luyện tập có chủ đích, dịch sang việc giải thuật

Nghiên cứu của Ericsson và cộng sự[3] đưa ra một kết luận đơn giản nhưng khó nuốt: số giờ luyện tập chỉ dự báo được năng lực khi việc luyện có bốn tính chất — mục tiêu hẹp và cụ thể, mức khó hơi vượt khả năng hiện tại, phản hồi gần như tức thì, và có sửa sai chứ không chỉ làm lại. Thiếu bất kỳ tính chất nào thì thời gian bỏ ra vẫn trôi, nhưng năng lực đứng yên.

Dịch sang việc luyện thuật toán, bốn tính chất trở thành bốn quy tắc rất cụ thể.

Mục tiêu hẹp. Không luyện kiểu “hôm nay học quy hoạch động”. Luyện kiểu “hôm nay làm năm bài quy hoạch động mà trạng thái là hai chiều, và với mỗi bài tôi phải viết ra định nghĩa trạng thái trước khi gõ code”. Mục tiêu hẹp cho phép biết mình có đạt hay không.

Vùng khó hơi vượt. Đây là quy tắc bị vi phạm nhiều nhất. Giải bài dễ cho cảm giác tốt — giải nhanh, đúng ngay, tự tin. Nhưng nếu bạn giải đúng ngay trong mười phút thì bài đó không dạy bạn gì. Vùng luyện đúng là vùng mà bạn giải được khoảng một nửa số bài, và nửa còn lại phải vật lộn rồi mới ra hoặc phải upsolve. Hình 2.2 vẽ ba vùng.



Hình 2.2. Ba vùng độ khó. Sai lầm phổ biến nhất khi tự luyện là ở lì trong vùng thoải mái vì nó cho cảm giác thành công; sai lầm ngược lại là nhảy thẳng vào vùng quá tải vì nghĩ rằng làm bài khó thì tiến bộ nhanh. Vùng luyện nằm giữa: bạn giải được khoảng một nửa, và luôn hơi khó chịu.

Phản hồi ngay. Đây là lý do các trang chấm bài tự động có giá trị lớn hơn vẻ ngoài của chúng: bạn biết đúng hay sai trong vài giây. Nhưng phản hồi “sai ở test 12” vẫn còn nghèo; phản hồi tốt hơn là stress test tự viết (chương E/25), vì nó chỉ thẳng ra *ca dữ liệu* nào làm hỏng.

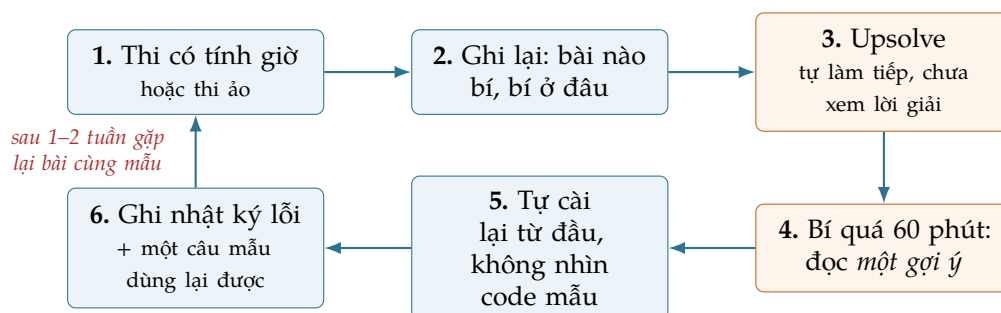
Sửa sai, không chỉ làm lại. Đây là chỗ nhật ký lỗi vào cuộc. Sau mỗi lần sai, ghi ba dòng: tôi đã hiểu sai điều gì, dấu hiệu nào trên đề lẽ ra đã gợi ý đúng, lần sau tôi sẽ kiểm tra gì. Sau vài tháng, nhật ký này sẽ cho bạn thấy một sự thật khó chịu và hữu ích: bạn sai đi sai lại đúng vài kiểu.

TRỰC GIÁC

Vì sao vùng thoải mái không tạo tiến bộ? Vì học xảy ra khi *dự đoán của bạn bị sai*. Giải một bài mà bạn biết chắc cách làm thì không có dự đoán nào bị vi phạm, nên không có gì để cập nhật. Cảm giác khó chịu khi bí chính là dấu hiệu của việc mô hình trong đầu bạn đang được sửa. Đó là lý do luyện đúng cách gần như luôn thấy không thoải mái, và vì sao cảm giác “buổi luyện hôm nay trôi chảy quá” thường là tin xấu.

2.3 Vòng upsolving — cỗ máy tiến bộ của dân thi đấu

Người mới thường nghĩ tiến bộ đến từ việc dự thi. Thực tế thì kỳ thi chỉ là phép đo; phần tạo ra tiến bộ nằm ở việc bạn làm gì trong vài ngày sau đó. Vòng lặp mà giới thi đấu dùng gần như thống nhất được vẽ ở hình 2.3.



Hình 2.3. Vòng upsolving. Hai ô tô hổ phách là phần tạo ra phần lớn tiến bộ, và cũng là phần hay bị cắt nhất vì nó không vui. Bước 5 — tự cài lại thay vì chép — là bước phân biệt “hiểu lời giải” với “làm được”; rất nhiều người tưởng mình đã học xong sau bước 4.

Cơ chế đằng sau vòng này khá rõ. Trong lúc thi, bạn ở trạng thái sức ép và đã tự mình đào rất sâu vào bài — tức là đã nạp đủ ngữ cảnh. Ngay sau đó, mỗi mẫu thông tin mới (một gợi ý, một ý tưởng) sẽ dính vào đúng chỗ trong đầu bạn, vì bạn biết chính xác nó lắp vào lỗ hổng nào. Đọc lời giải khi *chưa* vật lộn thì không có lỗ hổng nào để lắp, và thông tin trôi qua.

Một biến thể đáng dùng khi không có kỳ thi nào: *thi ảo*. Lấy một đề cũ, đặt đồng hồ đúng thời lượng thật, và làm nghiêm túc. Điểm quan trọng là tính giờ — vì kỹ năng phân bổ thời gian và kỹ năng chịu sức ép chỉ luyện được khi có sức ép.

CẠM BẮY

Cạm bẫy phổ biến nhất trong upsolving: đọc lời giải, gật gù “à hoá ra thế, dễ mà”, rồi chuyển bài. Cảm giác hiểu khi đọc là *cảm giác quen thuộc*, không phải năng lực tái tạo. Bài kiểm tra duy nhất đáng tin: đóng lời giải lại và tự cài từ trang trắng. Rất nhiều người phát hiện ra mình không làm được ở đúng bước này — và đó là thông tin quý, không phải điều đáng xấu hổ.

2.4 Đọc sách thế nào cho có tác dụng

Sách thuật toán bị đọc sai cách nhiều hơn bất kỳ loại sách kỹ thuật nào. Có ba cách đọc, và chỉ một cách tạo ra năng lực.

Đọc như tiểu thuyết — đọc từ đầu tới cuối, thấy hiểu. Cách này gần như không đọng lại gì sau ba tuần, vì không có lần nào bạn phải tự truy xuất kiến thức từ trí nhớ.

Đọc như từ điển — chỉ mở khi cần tra. Cách này hợp với nửa sau sách của Skiena hoặc với cp-algorithms, nhưng không xây được nền.

Đọc như phòng tập — đọc một mục, gấp sách, tự viết lại ý chính, rồi làm bài tập của mục đó *trước* khi đọc mục sau. Đây là cách duy nhất tạo ra năng lực, và nó chậm hơn hẳn: một chương có thể mất cả tuần. Knuth thiết kế TAOCP theo đúng giả định này — bài tập là một phần của nội dung, không phải phụ lục.

Bảng dưới đây là bản đồ sách: quyển nào cho vai trò nào. Đừng đọc song song quá hai quyển; chọn một quyển làm trục chính và dùng phần còn lại để tra.

Sách	Vai trò	Dùng khi nào
Pólya, <i>How to Solve It</i> [1]	Quy trình nghĩ	Đọc trước tất cả; mỏng, đọc trong một buổi
Zeitz[4], Engel[5]	Heuristic giải toán	Khi muốn luyện tư duy chứ không luyện thuật toán
Skiena[15]	Nhận diện bài toán	Trực chính tốt nhất cho người tự học; nửa sau là danh mục tra
CLRS[27]	Tham chiếu chuẩn	Tra định nghĩa, chứng minh chuẩn; <i>không</i> nên đọc tuần tự
Kleinberg–Tardos[28]	Dạy cách thiết kế	Khi muốn học cách <i>nghĩ ra</i> thuật toán, đặc biệt tham lam và luồng
Erickson[29]	Đệ quy và chứng minh	Miễn phí; phần quay lui và quy hoạch động rất tốt
Sedgewick–Wayne[22]	Cài đặt và đo	Khi muốn code chạy được, có thực nghiệm
Bentley[6]	Kỹ thuật thực dụng	Đọc rải rác, mỗi lần một chương ngắn
Laaksonen[17], Halim[16]	Thi đấu	Trực chính nếu mục tiêu là contest
Dijkstra[11], Gries[13]	Tính đúng	Khi bạn liên tục viết code gần đúng
Knuth[8]	Phân tích sâu	Tra một chủ đề cụ thể; không đọc tuần tự trừ khi có nhiều năm
Graham–Knuth–Patashnik[30]	Toán rời rạc	Khi phân tích độ phức tạp đệ quy tổng và truy hồi khó
Cormen <i>et al.</i> , Garey–Johnson[31]	Độ khó	Khi nghi ngờ bài của mình là NP-khó
McDowell[32]	Phỏng vấn	Bốn tới sáu tuần trước phỏng vấn, không sớm hơn

2.5 Những gì không bắt buộc được, và những gì bắt buộc được ngay

Trung thực mà nói, có những thứ trong danh sách trên bạn không nên cố bắt buộc. Knuth dành hàng chục năm cho một bộ sách; kiểu đầu tư đó chỉ hợp lý với một người có đúng mục tiêu của ông. Dijkstra làm việc phần lớn bằng bút và giấy, gần như không dùng máy tính — cách đó phù hợp với thời của ông và với tính chất công việc của ông, không phù hợp với người cần giao sản phẩm hàng tuần. Một số kỳ thủ lập trình bắt đầu từ rất sớm và luyện với cường độ mà người đi làm không thể có; so sánh bản thân với họ theo thang thời gian là cách nhanh nhất để nản vô ích.

Nhưng có sáu thứ bắt buộc được ngay trong tuần này, và chúng chiếm phần lớn giá trị:

1. Trước mỗi vòng lặp, viết một dòng bất biến bằng lời (Dijkstra).
2. Trước khi cài, ước lượng phong bì: n bao nhiêu, mỗi thao tác bao nhiêu, tổng cỡ bao nhiêu (Bentley).
3. Chọn bài ở vùng bạn giải được khoảng một nửa, không dễ hơn (Ericsson).
4. Sau mỗi trận hoặc mỗi buổi luyện, upsolve ít nhất một bài tới nơi tới chốn, tự cài lại từ trang trắng.
5. Giữ nhật ký lỗi ba dòng cho mỗi lần sai; đọc lại mỗi tháng.
6. Sau mỗi bài giải xong, viết một câu về ý tưởng cốt lõi không nhắc tới đề bài (Pólya, bước 4).

Sáu việc này cộng lại tốn chừng mười phút mỗi buổi luyện. Điểm chung của chúng: cả sáu đều là cách *đóng vòng phản hồi* — đúng thứ mà hình 2.1 nói là mẫu số chung của cả bốn trường phái.

2.6 Gốc rễ: vì sao ta biết những điều này về luyện tập

Những lời khuyên ở trên nghe như kinh nghiệm dân gian, nhưng phần lớn chúng có nguồn thực nghiệm, và nguồn ấy không đến từ ngành máy tính mà từ nghiên cứu về cờ vua.

Năm 1946, Adriaan de Groot — vừa là nhà tâm lý học vừa là kỳ thủ — đặt các kỳ thủ mạnh và yếu trước cùng một thế cờ và yêu cầu họ nói to suy nghĩ. Kết quả trái với kỳ vọng: kiện tướng *không* tính sâu hơn người trung bình bao nhiêu. Cái họ hơn là ngay từ vài giây đầu, họ đã nhìn ra một nhóm nhỏ nước đi đáng xét, trong khi người yếu hơn phải rà nhiều nước vô nghĩa. Nói cách khác, ưu thế nằm ở *nhận dạng*, không ở sức tính.

Đến 1973, William Chase và Herbert Simon — chính Simon của *Human Problem Solving* nhắc ở chương A/01 — làm thí nghiệm quyết định. Họ cho kỳ thủ nhìn một thế cờ trong vài giây rồi dựng lại. Kiện tướng dựng lại gần như hoàn hảo, người mới thì không. Nhưng khi các quân được đặt *ngẫu nhiên* thay vì theo thế cờ thật, ưu thế biến mất gần hết. Kết luận: chuyên gia không có trí nhớ tốt hơn; họ nhìn thấy *đơn vị lớn hơn* — những cụm quân có nghĩa, gọi là chunk. Trí nhớ của họ chỉ chứa vài chục đơn vị như mọi người, nhưng mỗi đơn vị đóng gói nhiều thông tin hơn.

Hai kết quả đó giải thích trực tiếp vì sao chương A/01 nhấn mạnh việc “viết một câu về ý tưởng cốt lõi không nhắc tới đề bài”. Câu đó chính là một chunk đang được tạo ra. Người nói “bài này dùng hai con trỏ” và người nói “khi cả hai đầu đều đơn điệu thì quét từ hai phía” đang lưu hai thứ rất khác nhau: một cái tên, và một điều kiện kích hoạt. Chỉ cái thứ hai mới tự nhận ra chính nó khi gặp bài mới.

Nhánh nghiên cứu thứ hai bắt đầu từ Benjamin Bloom, người phỏng vấn những cá nhân đạt đỉnh cao trong nhiều lĩnh vực và thấy một mẫu chung: nhiều năm luyện có hướng dẫn, tăng dần độ khó, với người dạy đóng vai trò cấp phản hồi. Ericsson và cộng sự hình thức hoá mẫu đó thành khái niệm luyện tập có chủ đích[3], [24]. Cần nói thẳng phần bị tranh cãi: con số “10 000 giờ” là cách phổ biến hoá của người khác, không phải kết luận của nghiên cứu, và mức độ mà luyện tập giải thích được khác biệt năng lực vẫn còn đang tranh luận trong giới nghiên cứu. Phần lỗi thì vững: luyện có mục tiêu, có phản hồi, ở mức hơi quá sức hiệu quả hơn hẳn lặp lại thụ động — và đó là toàn bộ phần ta dùng.

Còn bản thân môi trường luyện thuật toán hiện đại thì trẻ hơn nhiều. Các kỳ thi lập trình đại học bắt đầu từ thập niên 1970 và lớn dần thành ICPC; kỳ thi tin học quốc tế dành cho học sinh phổ thông (IOI) bắt đầu năm 1989; các nền tảng chấm bài trực tuyến xuất hiện từ cuối thập niên 1990 và bùng nổ trong thập niên 2010. Điều mà những nền tảng này thay đổi không phải nội dung kiến thức mà là *độ trễ phản hồi*: từ vài tuần chờ thầy chấm xuống còn vài giây. Nếu tin vào Ericsson, thì đó chính là biến số quan trọng nhất, và nó giải thích vì sao trình độ trung bình của người luyện nghiêm túc ngày nay cao hơn hẳn hai mươi năm trước.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1946 — de Groot	Muốn biết kiện tướng cờ nghĩ khác người thường ở đâu.	Kiến tướng không tính sâu hơn; họ nhận dạng nhanh hơn. Ưu thế nằm ở nhận dạng chứ không ở sức tính.
1973 — Chase & Simon	Kiểm chứng: đó là trí nhớ tốt hơn hay cách tổ chức tri thức khác?	Ưu thế biến mất với thế cờ ngẫu nhiên ⇒ chuyên gia lưu các <i>khối có nghĩa</i> . Giải thích vì sao phải phát biểu ý tưởng ở dạng tổng quát.
1985 — Bloom	Phỏng vấn người đạt đỉnh cao ở nhiều ngành để tìm mẫu chung.	Nhiều năm luyện có hướng dẫn và tăng dần độ khó; vai trò của người cấp phản hồi.
1993 — Ericsson và cộng sự	Vì sao cùng số giờ mà kết quả khác nhau.	Bốn điều kiện của luyện tập có chủ đích. Phần “10 000 giờ” là diễn giải của người khác và còn tranh cãi.
1989 và sau đó — IOI, chấm bài trực tuyến	Muốn đánh giá được kỹ năng lập trình ở quy mô lớn.	Độ trễ phản hồi tụt từ vài tuần xuống vài giây — thay đổi lớn nhất với người tự học trong 30 năm qua.

2.7 Liên hệ ngang: nghề nào cũng phải giải bài toán luyện tập này

Nếu bốn điều kiện của luyện tập có chủ đích là đúng, thì mọi nghề đòi hỏi kỹ năng cao đều phải tự phát minh ra cách thoả chúng. Và quả thật là vậy — nhìn sang các nghề khác, bạn sẽ thấy những cơ chế trùng khớp đến bất ngờ với upsolving và nhật ký lỗi, chỉ khác tên gọi. Hai chỗ đáng học nhất là y khoa và hàng không, vì họ đã *thể chế hoá* việc học từ lỗi thay vì phó mặc cho ý chí cá nhân.

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Âm nhạc	Tập chậm từng đoạn khó, tách tay, dùng máy đếm nhịp tăng dần tốc độ	Giống: cô lập đúng phần yếu thay vì chơi lại cả bản — tương đương việc luyện riêng một dạng bài. Khác: nhạc công có phản hồi tức thì bằng tai, còn bạn phải tự tạo phản hồi bằng chấm bài và stress test.
Thể thao	Chu kỳ hoá: xen kẽ giai đoạn tăng khối lượng, tăng cường độ và phục hồi	Giống: cường độ cao liên tục không bền, cần xen kẽ. Gợi ý cho lộ trình ở E/28 : xen tuần thi đấu với tuần học kiến thức mới. Khác: giải thuật không có giới hạn sinh lý, giới hạn là sự chú ý và động lực.
Y khoa	Hội chẩn tử vong và biến chứng (M&M): mỗi ca xấu được mổ xẻ công khai, không quy tội cá nhân	Đây chính là nhật ký lỗi được thể chế hoá, và nó hiệu quả hơn nhật ký cá nhân vì bài học được chia sẻ. Bắt buộc được: viết công khai bài phân tích lỗi của mình.
Hàng không	Bảng kiểm (checklist), báo cáo sự cố phi trường phạt, huấn luyện trên buồng lái mô phỏng	Bảng kiểm ứng với danh sách trường hợp biên trước khi nộp bài (E/25). Buồng lái mô phỏng ứng với thi ảo: rủi ro bằng không, lặp lại được, có thể cố tình tạo tình huống xấu.
Quân sự	Rà soát sau hành động (after-action review): điều dự định, điều đã xảy ra, vì sao khác, lần sau làm gì	Bốn câu này gần như trùng khít với bước 4 của Pólya và nội dung nhật ký lỗi. Điểm mạnh: nó là quy trình bắt buộc, không phụ thuộc tâm trạng.
Nghiên cứu khoa học	Số thí nghiệm, tái lập kết quả, bình duyệt	Số thí nghiệm chính là nhật ký lỗi. Bình duyệt là phản hồi từ người ngoài — thứ mà người tự học thiếu nhất; thay thế gần nhất là nhờ người khác đọc code hoặc viết editorial công khai.
Học máy	Vòng huấn luyện: mất mát trên tập kiểm tra, dừng sớm, học theo chương trình (curriculum learning)	Rất giống về cấu trúc: mẫu quá dễ cho gradient nhỏ, mẫu quá khó cho gradient nhiều; vùng học hiệu quả nằm giữa — đúng hình 2.2 . Khác: mô hình không tự chọn được bài, còn bạn thì có.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì thói quen của họ không phải sở thích cá nhân mà là cách đóng vòng phản hồi, và vòng phản hồi thì chỉ có vài cách đóng. Bốn trường phái ở hình [2.1](#) khác nhau ở thước đo — toán, logic, đồng hồ, số bài giải được — nhưng ai cũng phải có một thước đo khách quan và phải sửa cách làm theo kết quả đo. Người luyện không có thước đo nào là người duy nhất không tiến bộ.
- A2. Vì đọc tạo ra *cảm giác quen thuộc*, còn làm bài tập tạo ra *khả năng truy xuất*. Hai thứ đó khác nhau: bạn có thể thấy mọi dòng chứng minh đều hiển nhiên khi đọc mà vẫn không tái tạo được nó khi gấp sách. Bài tập ép bạn truy xuất, và chính lúc truy xuất khó khăn thì trí nhớ mới được củng cố. Cách xếp hạng 0–50 của Knuth còn giúp bạn chọn đúng vùng khó thay vì làm hết mọi bài.
- A3. Nó đổi thứ tự công việc. Thay vì gõ code rồi tìm ca dữ liệu làm nó sai, bạn viết ra điều kiện cần giữ đúng *trước*, và code trở thành cách thực hiện điều kiện đó. Cụ thể nhất là một dòng bắt biến trước mỗi vòng lặp: khi có nó, các câu hỏi “bắt đầu từ 0 hay 1”, “chạy tới n hay n-1”, “khởi tạo bằng 0 hay $-\infty$ ” đều được suy ra thay vì đoán rồi thử.

- A4. Vì “nhiều bài” thường có nghĩa là nhiều bài *vừa sức*, mà vừa sức thì không tạo ra sai lệch dự đoán để học. Sau vài tháng bạn thành thạo hơn ở những gì đã biết và không mở rộng được gì mới. Điều kiện Ericsson đòi hỏi là mức hơi quá sức cộng phản hồi cộng sửa sai; giải ít bài hơn nhưng chọn ở vùng luyện, có upsolve và có nhật ký lỗi thì thoả cả ba.
- A5. Vì trong lúc thi bạn đã tự đào rất sâu vào bài, tức là đã tạo sẵn những “lỗ hổng” rất cụ thể trong hiểu biết. Ngay sau đó, một gợi ý nhỏ sẽ rơi đúng vào lỗ hổng và dính lại. Nếu đọc lời giải khi chưa vật lộn thì không có lỗ hổng nào, thông tin trôi qua và bạn chỉ thu được cảm giác đã hiểu. Đó cũng là lý do bước “tự cài lại từ trang trắng” không được bỏ.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn học chơi đàn. Có bốn kiểu thầy: người bắt bạn đếm nhịp thật chính xác, người bắt bạn hiểu vì sao hợp âm này đi với hợp âm kia, người bắt bạn thu âm lại rồi nghe xem chỗ nào sai, và người bắt bạn lên sân khấu đúng giờ. Bốn kiểu ấy tương ứng với bốn nhóm người trong chương này. Điều họ giống nhau là ai cũng có một cách để *biết mình đang sai ở đâu* — đếm nhịp, lý thuyết, bản thu, khán giả. Người tập đàn một mình trong phòng, không thu âm, không ai nghe, thì tập bao nhiêu năm cũng chỉ giỏi lên chút ít, vì không có gì chỉ cho họ chỗ sai.

Việc luyện thuật toán y hệt. Bạn cần một cái “bản thu”: chấm bài tự động, stress test, đồng hồ bấm giờ, hay đơn giản là quyển sổ ghi mỗi lần bạn sai. Và bạn cần chọn bài ở mức hơi khó — dễ quá thì như tập lại bài đã thuộc, khó quá thì như ngồi trước bản nhạc không đọc nổi nốt nào. Cuối cùng, sau mỗi lần tập, hãy quay lại đúng chỗ bạn vấp và tự chơi lại từ đầu, chứ đừng nghe người khác chơi rồi tưởng mình chơi được.

Một câu: Người giỏi không luyện nhiều hơn bạn nhiều đến thế; họ luyện ở mức khó hơn một chút và luôn biết mình vừa sai ở đâu.

Chỗ không rút gọn được: Có phần thật sự không rút gọn được: thời gian. Vòng phản hồi tốt làm mỗi giờ luyện đáng giá hơn, nhưng không có cách nào bỏ qua vài trăm giờ đầu tiên.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn điều kiện của luyện tập có chủ đích	Mục tiêu hẹp; mức hơi quá sức; phản hồi gần như tức thì; có sửa sai chứ không chỉ làm lại. Thiếu một điều thì thời gian trôi mà năng lực đứng.
••• Vùng luyện đúng	Vùng bạn giải được khoảng một nửa số bài; luôn hơi khó chịu. Dễ quá thì không có dự đoán nào bị sai để mà học.
••• Vòng upsolving	Thi tính giờ → ghi chỗ bí → tự làm tiếp → chỉ đọc <i>một</i> gợi ý khi bí quá 60 phút → tự cài lại từ trang trắng → ghi nhật ký lỗi.
••• Kiểm tra “đã học được” đáng tin duy nhất	Đóng lời giải, tự cài từ trang trắng. Cảm giác hiểu khi đọc chỉ là cảm giác quen thuộc.
••• Câu của Dijkstra về kiểm thử	Kiểm thử cho thấy sự hiện diện của lỗi, không bao giờ cho thấy sự vắng mặt của lỗi — nên tính đúng phải được xây vào lúc viết, không thêm vào lúc kiểm.
•• Câu Knuth bị trích sai	“Tối ưu hoá sớm là gốc rễ mọi tội lỗi” — nguyên ý là bỏ qua cái thiện nhỏ trong phần lớn thời gian, nhưng phải đo để tìm ra phần thực sự quan trọng và tối ưu ở đó.
•• Đóng góp phương pháp của Tarjan	Thiết kế cấu trúc dữ liệu cùng với công cụ phân tích của nó; khẩu hao và hàm thể năng ra đời từ nhu cầu đó.
•• Ba thói quen của Bentley	Ước lượng phong bì trước khi cài; đo trước khi tối ưu; coi việc viết đúng thuật toán đơn giản là kỹ năng phải luyện riêng.
•• Vì sao Skiena tổ chức sách thành danh mục	Vì trong thực tế, phần khó nhất thường là nhận ra bài của mình chính là bài kinh điển nào, chứ không phải nghĩ ra thuật toán mới.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba cách đọc sách và cách nào có tác dụng	Đọc như tiểu thuyết (trôi), như từ điển (tra được, không xây nền), như phòng tập (gấp sách, viết lại, làm bài tập trước khi sang mục sau) — chỉ cách thứ ba tạo năng lực.
●● Nội dung ba dòng của nhật ký lỗi	Tôi hiểu sai điều gì; dấu hiệu nào lẽ ra đã gợi ý đúng; lần sau tôi sẽ kiểm tra gì.
● Thi ảo	Lấy đề cũ, đặt đồng hồ đúng thời lượng thật, làm nghiêm túc — dùng khi không có kỳ thi thật, để luyện phân bổ thời gian dưới sức ép.
● Bốn trường phái và thước đo của họ	Phân tích (toán), chứng minh (logic), kỹ thuật (đồng hồ trên máy thật), thi đấu (số bài giải được có tính giờ).

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi đi làm, mỗi ngày chỉ có một giờ. Nên phân bổ thế nào cho hiệu quả nhất?

| Một cách phân bổ dùng được: 10 phút ôn lại (đọc nhật ký lỗi và bảng active recall của chương đang học), 40 phút giải một bài ở vùng luyện có bầm giờ, 10 phút cho bước 4 — viết một câu về ý tưởng cốt lõi và ghi nhật ký nếu sai. Điều quan trọng hơn tỉ lệ là tính đều đặn: một giờ mỗi ngày trong sáu tháng hơn hẳn tám giờ mỗi cuối tuần, vì trí nhớ củng cố theo số lần truy xuất chứ không theo tổng thời gian.

2. Tôi nên đọc hết CLRS không?

| Gần như chắc chắn là không, trừ khi bạn đang học một khoá có giáo trình đó. CLRS được thiết kế làm tham chiếu chuẩn: định nghĩa chính xác, chứng minh đầy đủ, bao phủ rộng. Đọc tuần tự thì rất chậm và bạn sẽ dừng ở khoảng chương 8. Cách dùng đúng: lấy một quyển khác làm trục chính (Skiena nếu tự học, Laaksonen nếu thi đấu), và mở CLRS khi cần một chứng minh chuẩn hoặc một chi tiết mà trục chính nói lướt.

3. Đọc code của người giỏi có ích không, hay chỉ nên tự viết?

| Cả hai, nhưng đúng thứ tự. Đọc code trước khi tự vật lộn thì bạn chép được kiểu viết mà không hiểu vì sao; đọc sau khi đã tự cài xong thì rất có giá trị, vì bạn thấy ngay chỗ họ ngắn hơn, ít trường hợp riêng hơn, hoặc chọn biểu diễn dữ liệu khôn hơn. Một bài tập tốt: cài xong rồi so với thư viện tham chiếu như KACTL[7] và tự hỏi vì sao họ chọn cách đó.

4. Xây thư viện mẫu cho riêng mình có đáng không, khi đã có thư viện công khai?

| Đáng, vì lợi ích chính không nằm ở đoạn code mà ở quá trình viết. Khi tự cài DSU hay segment tree tới mức đúng và gọn, bạn buộc phải hiểu từng chi tiết, và về sau bạn sửa được nó cho biến thể lạ — điều không làm được với code chép về. Nguyên tắc: mỗi đoạn trong thư viện riêng của bạn phải do bạn viết và đã qua stress test.

5. Làm sao biết mình đang ở vùng luyện chứ không phải vùng thoải mái?

| Bằng tỉ lệ giải được. Nếu bạn giải đúng ngay gần như mọi bài trong dưới 15 phút, bạn đang ở vùng thoải mái — nâng độ khó. Nếu bạn không giải được gì trong hai buổi liên tiếp và đọc lời giải vẫn thấy mù mờ, bạn đang ở vùng quá tải — hạ xuống. Điểm cân bằng là khoảng một nửa số bài giải được sau khi vật lộn, nửa còn lại phải upsolve.

6. Nghiên cứu về luyện tập có chủ đích có bị thổi phồng không?

| Có phần. Kết luận mạnh kiểu “chỉ cần 10 000 giờ là thành chuyên gia” là cách diễn giải phổ biến hoá, và bị tranh luận nhiều trong giới nghiên cứu; các yếu tố khác như điểm xuất phát và điều kiện học tập đều có vai trò. Nhưng phần lõi thì vững và cũng là phần ta dùng ở đây: luyện có mục tiêu, có phản hồi, ở mức hơi quá sức thì hiệu quả hơn hẳn luyện lặp lại thụ động. Ta không cần phần bị tranh cãi để dùng được phần lõi.

7. Chuyên gia cờ vua nhớ thể cờ giỏi hơn người thường — điều đó liên quan gì tới thuật toán?

| Liên quan trực tiếp, và đây là một trong những kết quả kinh điển của tâm lý học nhận thức: ưu thế trí nhớ đồ biến mất khi các quân được đặt ngẫu nhiên. Nghĩa là chuyên gia không nhớ tốt hơn, họ *nhìn thấy đơn vị lớn hơn* — những cụm có nghĩa. Với thuật toán cũng vậy: người có kinh nghiệm không nhớ nhiều dòng code hơn, họ nhìn một đề bài và thấy “đây là bài ghép đôi trá hình”. Xây các khối có nghĩa đó chính là việc mà bước 4 của Pólya và câu “phát biểu ý tưởng không nhắc đề bài” đang làm.

8. Tôi hay bỏ dở lộ trình sau vài tuần. Có mẹo nào không?

| Ba điều chỉnh có hiệu quả rõ. Thứ nhất, hạ mục tiêu hằng ngày xuống mức bạn chắc chắn làm được kể cả ngày bận — một bài, hoặc thậm chí 20 phút; tính liên tục quan trọng hơn khối lượng. Thứ hai, làm cho tiến bộ nhìn thấy được: một bảng đánh dấu ngày, hoặc danh sách bài đã upsolve. Thứ ba, ghép việc luyện vào một khung giờ cố định thay vì “khi nào rảnh”. Chương E/28 có lộ trình cụ thể theo tuần đã tính tới ba điều này.

| Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tự viết lại tìm kiếm nhị phân từ trang trắng, không tra cứu	Bài tập của Bentley[6]	Viết bất biến trước; sau đó stress test với mảng rỗng, một phần tử, nhiều phần tử trùng nhau.
● Đọc mục 1 của <i>How to Solve It</i> và áp bốn bước vào một bài bạn đang bí	[1]	Mục tiêu là tập nhận ra mình đang ở bước nào, đặc biệt phát hiện lúc bạn nhảy cóc sang bước 3.
● Lập nhật ký lỗi và ghi ba dòng cho năm lần sai gần nhất	Bài tập tự tổ chức	Sau năm mục, tự phân loại: sai do hiểu đề, sai do ý tưởng, sai do cài đặt. Tỷ lệ này quyết định bạn nên luyện gì.
●● Chọn một đề contest cũ và làm thi ảo đúng thời lượng	Codeforces / AtCoder / CSES[33]	Ghi lại mốc thời gian mỗi lần đổi bài; sau đó đối chiếu với hình 1.2 ở chương A/01.
●● Upsolve một bài trong đề vừa làm, tự cài lại từ trang trắng	Cùng đề trên	Bước 5 của vòng upsolving; đây là bài tập quan trọng nhất trong cả danh sách này.
●● Cài DSU và segment tree cho thư viện riêng, có stress test	[7] để đối chiếu	So sánh code của bạn với thư viện tham chiếu và giải thích từng khác biệt — đừng chép, hãy giải thích.
●● Chọn một chương của Skiena, làm hết bài tập trước khi đọc chương sau	[15]	Luyện cách đọc “như phòng tập”; ghi lại thời gian thật để biết tốc độ đọc bền vững của bạn.
●●● Viết editorial cho một bài bạn vừa giải, như thể dạy người khác	Bài tập kiểu Feynman	Chỗ bạn viết không trôi chảy là chỗ bạn chưa hiểu; đó là mục đích duy nhất của bài tập này.
●●● Lấy một thuật toán bạn “đã biết” và tự chứng minh tính đúng bằng bất biến	Chương A/04	Chọn Dijkstra hoặc thuật toán tham lam xếp lịch; nếu không viết được chứng minh, bạn mới chỉ nhớ chứ chưa hiểu.
●●● Thiết kế lộ trình 12 tuần cho riêng bạn, có mục tiêu đo được mỗi tuần	Chương E/28 làm mẫu	Mỗi tuần phải có một tiêu chí kiểm tra khách quan, không phải “học xong chương X”.

Kết chương và đọc tiếp

Chương này rút gọn được thành một câu: những người giỏi nhất khác nhau ở thước đo họ chọn, nhưng giống nhau ở chỗ ai cũng có một thước đo và ai cũng sửa cách làm theo kết quả đo. Bạn không sao chép được hoàn cảnh của họ, nhưng sáu thói quen ở mục cuối thì sao chép được ngay, và chúng chính là phần tạo ra phần lớn giá trị.

Còn bỏ ngỏ hai câu hỏi thực dụng mà chương này liên tục giả định đã có câu trả lời: thước đo “đủ nhanh” là gì, và thước đo “chắc chắn đúng” là gì. Hai chương tiếp theo cấp đúng hai thước đo đó — [A/03](#) cho chi phí, [A/04](#) cho tính đúng. Sau đó, [E/28](#) sẽ biến toàn bộ chương này thành một lịch luyện cụ thể theo tuần.

Mô hình máy và độ phức tạp (Cost Models and Complexity)

PHỤ THUỘC A/01 cho vai trò của chương này trong bước 2 của Pólya. Phần giải hệ thức truy hồi dùng vài công cụ tổng ở A/05, nhưng đọc được trước.

TÓM TẮT

Chương này cấp cái thước thứ nhất: *ý tưởng của tôi có đủ nhanh không*. Nó bắt đầu từ mô hình máy mà mọi phân tích ngầm giả định, đi qua ký hiệu tiệm cận, rồi tới ba công cụ dùng hằng ngày — bảng chuyển từ giới hạn đề bài sang độ phức tạp cho phép, cách giải hệ thức truy hồi, và phân tích khấu hao. Phần cuối phá bỏ sự ngây thơ của mô hình: hằng số, bộ nhớ và cache có thể lật ngược kết quả tiệm cận ở kích thước thật. Nếu chỉ nhớ một điều: độ phức tạp không phải để khoe mà để *loại bỏ ý tưởng trước khi gõ code*, và bảng ở mục 3 là công cụ đáng thuộc nhất chương.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình RAM (random access machine)** — mô hình máy giả định: mỗi phép toán số học, so sánh, gán, và truy cập một ô nhớ bất kỳ đều mất một đơn vị thời gian. Mọi ký hiệu $O(\cdot)$ trong sách vở đều ngầm nói về mô hình này.
- **kích thước dữ liệu vào n** — đại lượng mà ta đo chi phí theo nó. Phải nói rõ n là gì — số phần tử, số đỉnh, số bit — vì cùng một thuật toán có thể là đa thức theo cách đo này và mũ theo cách đo kia.
- **trường hợp xấu nhất / trung bình** — chi phí lớn nhất trên mọi dữ liệu vào cỡ n , so với chi phí trung bình theo một phân phối giả định. Nói “trung bình” mà không nói phân phối là nói vô nghĩa.
- **hệ thức truy hồi (recurrence)** — phương trình kiểu $T(n) = 2T(n/2) + n$, mô tả chi phí của thuật toán đệ quy theo chi phí của chính nó trên dữ liệu nhỏ hơn.
- **khấu hao (amortized)** — chi phí trung bình mỗi thao tác trong một *dãy* thao tác xấu nhất. Khác với trung bình xác suất: ở đây không có ngẫu nhiên nào cả.
- **phân cấp bộ nhớ** — thanh ghi, L1, L2, L3, RAM, SSD, đĩa — mỗi tầng chậm hơn tầng trên hàng chục tới hàng nghìn lần. Mô hình RAM giả vờ chúng như nhau, và đó là chỗ nó sai nhiều nhất.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao ta phân tích thuật toán trên một mô hình máy mà không máy thật nào giống, và vì sao điều đó vẫn hữu ích?
- Q2. Hai thuật toán cùng $O(n \log n)$. Trong thực tế một cái nhanh gấp năm lần cái kia. Ký hiệu tiệm cận đã che mất cái gì?
- Q3. Vì sao mảng động tăng gấp đôi kích thước lại cho chi phí thêm phần tử là $O(1)$ khấu hao, trong khi có những lần thêm tốn tới $O(n)$?
- Q4. Đề cho $n \leq 2 \cdot 10^5$ và thời gian 1 giây. Vì sao từ hai con số đó bạn suy ra được gần như toàn bộ dạng lời giải cần tìm?
- Q5. Khi nào một thuật toán $O(n^2)$ là lựa chọn *đúng* thay vì $O(n \log n)$?

Trước khi gõ dòng code đầu tiên, có một câu hỏi cần trả lời và nó chỉ tốn ba mươi giây: *ý tưởng này chạy trong bao lâu với dữ liệu lớn nhất mà đề cho phép?* Người mới bỏ qua câu hỏi này và trả giá bằng cách cài xong một lời giải $O(n^2)$ rồi mới phát hiện n lên tới một triệu. Người có kinh nghiệm trả lời nó trước, và nhờ vậy loại được ba phần tư số ý tưởng trong đầu mà không mất một phút gõ phím nào.

Chương này dựng công cụ cho câu hỏi đó. Nhưng cũng cần nói ngay giới hạn của nó: độ phức tạp tiệm cận là một mô hình, và mô hình nào cũng có chỗ nói dối. Nửa cuối chương dành cho những chỗ nói dối ấy — hằng số, bộ nhớ, cache — vì trong công việc thật, chúng thường quyết định nhiều hơn cả bậc tiệm cận.

3.1 Mô hình máy: giả định mà mọi phân tích đều ngầm dùng

Khi ta nói “thuật toán này chạy $O(n \log n)$ ”, ta đang nói về một cái máy tưởng tượng gọi là *mô hình RAM*: bộ nhớ là một dãy ô đánh số, truy cập ô nào cũng mất một đơn vị thời gian, và mỗi phép toán cơ bản trên một số cũng mất một đơn vị.

Mô hình này sai ở ít nhất ba chỗ. Truy cập bộ nhớ *không* đồng giá — đọc một ô đang nằm trong cache nhanh hơn đọc một ô phải lấy từ RAM cỡ hai bậc. Phép toán *không* đồng giá — phép chia chậm hơn phép cộng nhiều lần, và phép nhân số lớn thì chi phí phụ thuộc số bit. Và số nguyên trong máy thật có kích thước hữu hạn, nên “một phép cộng” chỉ đúng khi số vừa trong một từ máy.

Vậy vì sao vẫn dùng? Vì mô hình này giữ được thứ ta cần và bỏ đi thứ ta không kiểm soát được. Nó cho phép so sánh hai thuật toán mà không phụ thuộc ngôn ngữ, trình biên dịch hay đời máy, và nó dự đoán đúng điều quan trọng nhất: *khi dữ liệu lớn gấp đôi thì thời gian tăng bao nhiêu lần*. Với dữ liệu đủ lớn, bậc tiệm cận luôn thắng hằng số. Sai lầm không nằm ở việc dùng mô hình, mà ở việc quên rằng mình đang dùng mô hình — mục 6 sẽ nói về đúng những lúc phải nhớ ra.

CẠM BẤY

Một chỗ mà mô hình RAM đánh lừa thường xuyên trong lập trình thi đấu: coi số nguyên lớn tùy ý là “một đơn vị”. Nếu bài toán làm việc với số có hàng nghìn chữ số, mỗi phép cộng đã tốn $O(\text{số chữ số})$, và độ phức tạp thật khác hẳn con số bạn vừa tính. Tương tự với chuỗi: so sánh hai chuỗi *không* phải $O(1)$ mà là $O(\text{độ dài})$ — đây là lý do một lời giải dùng `map<string,int>` có thể chậm hơn dự đoán hẳn một bậc.

3.2 Ký hiệu tiệm cận, nói cho đúng

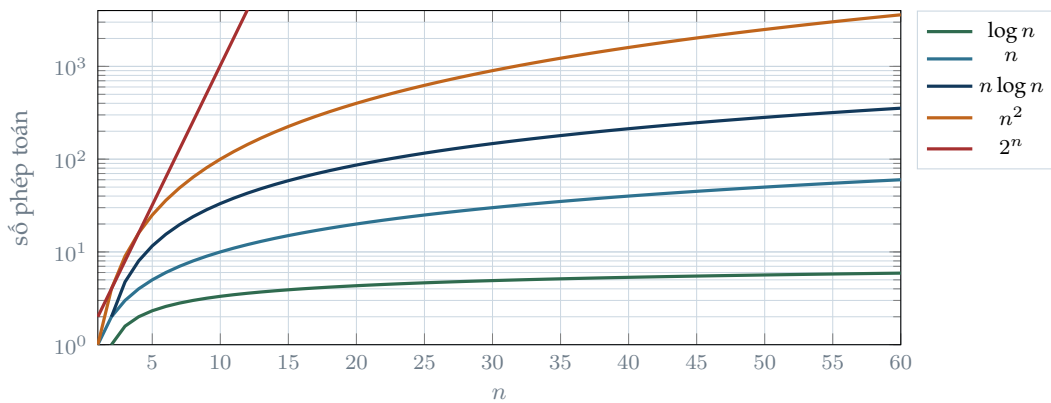
Ba ký hiệu cần phân biệt, và cách nhớ đơn giản nhất là coi chúng như dấu so sánh cho hàm số khi n lớn.

$O(f(n))$ là chặn trên: thuật toán chạy *không chậm hơn* $c \cdot f(n)$ với n đủ lớn. $\Omega(f(n))$ là chặn dưới: *không nhanh hơn*. $\Theta(f(n))$ là cả hai: đúng bậc đó. Trong thực tế người ta nói “thuật toán này $O(n \log n)$ ” và ý là $\Theta(n \log n)$; sự lỏng lẻo này vô hại, trừ khi bạn đang nói về *cận dưới của một bài toán* — lúc đó phải chính xác, vì “mọi thuật toán so sánh cần $\Omega(n \log n)$ ” là một phát biểu hoàn toàn khác với “thuật toán này chạy $O(n \log n)$ ”. Chương D/24 bàn kỹ.

Bậc tăng của các hàm hay gặp, xếp từ chậm tới nhanh:

$$1 \prec \log^* n \prec \log \log n \prec \log n \prec \sqrt{n} \prec n \prec n \log n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n.$$

Điều đáng nhớ hơn danh sách này là *khoảng cách* giữa các bậc, và hình 3.1 cho thấy nó bằng hình.



Hình 3.1. Các bậc tăng trên trục dọc lôgarit. Hai điều đáng nhìn kỹ: n và $n \log n$ gần như song song — khoảng cách giữa chúng nhỏ tới mức trong thực tế hiếm khi đáng đánh đổi tính đơn giản để bỏ thừa số log; còn 2^n cất lên phía trên mọi đường khác trước khi n kịp đạt 20. Đó là toàn bộ lý do vì sao giới hạn $n \leq 20$ trên đề bài lại là một gợi ý mạnh.

Ba quy tắc rút gọn dùng hằng ngày: bỏ hằng số nhân ($3n^2 \rightarrow n^2$); chỉ giữ số hạng lớn nhất ($n^2 + n \log n \rightarrow n^2$); và với vòng lặp lồng nhau thì nhân, với các đoạn nối tiếp thì lấy cái lớn nhất. Riêng cơ sở của lôgarit thì bỏ được, vì đối cơ sở chỉ là nhân với hằng số.

3.3 Từ giới hạn đề bài suy ra độ phức tạp cho phép

Đây là mục thực dụng nhất chương. Một máy chấm hiện đại thực hiện cỡ 10^8 tới 10^9 phép toán đơn giản mỗi giây — con số thô, phụ thuộc ngôn ngữ và loại phép toán, nhưng đủ tốt để ước lượng. Lấy 10^8 làm mốc an toàn cho giới hạn một giây, ta có bảng sau. Bảng này nên thuộc lòng; nó là thứ biến dòng “ $n \leq \dots$ ” trên đề thành một gợi ý về dạng lời giải.

Giới hạn n	Độ phức tạp cho phép	Dạng lời giải thường gặp
$n \leq 11$	$O(n!)$	duyệt mọi hoán vị
$n \leq 20\text{--}25$	$O(2^n \cdot n)$	duyệt tập con, quy hoạch động bitmask, ghép nhau ở giữa
$n \leq 100$	$O(n^4)$	bốn vòng lặp, quy hoạch động bốn chiều
$n \leq 500$	$O(n^3)$	Floyd–Warshall, nhân ma trận, quy hoạch động khoảng
$n \leq 5000$	$O(n^2)$	quy hoạch động hai chiều, mọi cặp
$n \leq 10^5\text{--}10^6$	$O(n \log n)$	sắp xếp, cây chỉ số, chia để trị, tập hợp
$n \leq 10^7\text{--}10^8$	$O(n)$	một lượt quét, hai con trỏ, sàng
$n \leq 10^{18}$	$O(\log n)$ hoặc $O(1)$	nhị phân, lũy thừa nhanh, công thức đóng

Cách dùng bảng này ngược với cách người ta hay nghĩ. Bạn không dùng nó để kiểm tra lời giải *sau khi* nghĩ ra, mà để *định hướng* việc nghĩ. Thấy $n \leq 20$ thì đừng phí thời gian tìm lời giải đa thức tình vi — người ra đề đã nói với bạn rằng lời giải là mũ. Thấy $n \leq 5000$ mà bạn đang cố nghĩ ra $O(n \log n)$ thì bạn đang làm khó mình vô ích; $O(n^2)$ được chấp nhận và thường đơn giản hơn nhiều.

Ba lưu ý làm bảng chính xác hơn. Thứ nhất, để ý *tổng* n qua các bộ test: đề ghi “tổng n không vượt quá $2 \cdot 10^5$ ” nghĩa là bạn được phép làm tuyến tính trên mỗi test mà không lo số test lớn. Thứ hai, hằng số của thao tác quan trọng: 10^8 phép cộng số nguyên thì nhanh, nhưng 10^8 lần chèn vào map thì chắc chắn quá giờ, vì mỗi thao tác đó tốn hàng chục tới hàng trăm lần hơn. Thứ

ba, bộ nhớ cũng có giới hạn: một mảng long long kích thước 10^8 cần 800 MB, vượt xa mức cho phép thông thường — nên đôi khi ràng buộc bộ nhớ mới là ràng buộc quyết định.

CẦN NHỚ

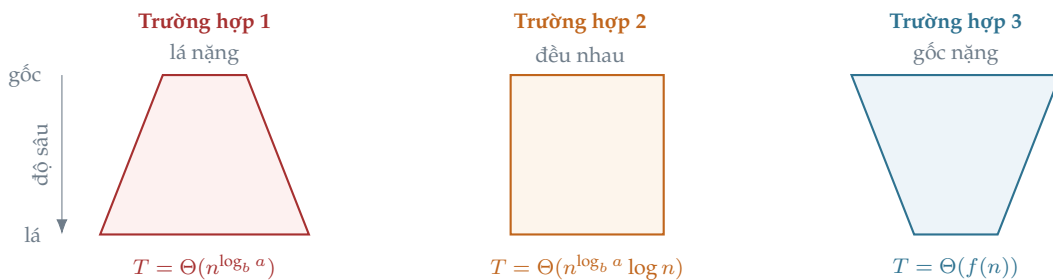
Đọc giới hạn *trước* khi nghĩ, không phải *sau*. Hai con số — n lớn nhất và giới hạn thời gian — thu hẹp không gian ý tưởng xuống còn một hoặc hai dạng. Đây là kỹ thuật rẻ nhất trong cả quyển sách này: tốn ba mươi giây, tiết kiệm hàng giờ.

3.4 Hệ thức truy hồi: chi phí của thuật toán đệ quy

Khi thuật toán gọi lại chính nó trên dữ liệu nhỏ hơn, chi phí được mô tả bằng hệ thức truy hồi. Có ba cách giải, xếp theo thứ tự nên thử.

Cách thứ nhất — vẽ cây đệ quy. Đây là cách trực giác nhất và hầu như luôn đủ. Vẽ cây các lời gọi, tính tổng chi phí ở mỗi tầng, rồi cộng các tầng lại. Với $T(n) = 2T(n/2) + n$: tầng gốc tốn n ; tầng hai có hai nút, mỗi nút tốn $n/2$, tổng vẫn là n ; mọi tầng đều tốn n ; cây sâu $\log_2 n$ tầng; vậy $T(n) = \Theta(n \log n)$. Toàn bộ merge sort nằm trong ba câu đó.

Cách thứ hai — định lý thợ (master theorem). Với dạng $T(n) = aT(n/b) + f(n)$, so sánh $f(n)$ với $n^{\log_b a}$: nếu f nhỏ hơn hẳn thì lá chiếm ưu thế và $T = \Theta(n^{\log_b a})$; nếu bằng nhau thì $T = \Theta(n^{\log_b a} \log n)$; nếu f lớn hơn hẳn (và thoả điều kiện đều đặn) thì gốc chiếm ưu thế và $T = \Theta(f(n))$. Hình 3.2 minh hoạ ba trường hợp bằng hình dáng cây.



Hình 3.2. Ba trường hợp của định lý thợ, nhìn theo “tầng nào tốn nhiều nhất”. Bề rộng mỗi tầng biểu diễn tổng chi phí của tầng đó. Nếu chi phí dồn về lá thì tổng do lá quyết định; nếu mọi tầng bằng nhau thì tổng bằng chi phí một tầng nhân số tầng; nếu dồn về gốc thì tổng do gốc quyết định. Nhớ hình này thì không cần nhớ công thức.

Cách thứ ba — đoán và chứng minh bằng quy nạp. Dùng khi hai cách trên không áp được, ví dụ với truy hồi không đều như $T(n) = T(n/5) + T(7n/10) + n$ của thuật toán chọn trung vị [34]. Đoán $T(n) \leq cn$, thay vào, và tìm c làm bất đẳng thức đúng: $cn/5 + 7cn/10 + n = 9cn/10 + n \leq cn$ khi $c \geq 10$. Vậy $T(n) = O(n)$, và cái đẹp của lời giải là ở chỗ tổng hai phân số nhỏ hơn 1.

Ba truy hồi đáng thuộc: $T(n) = T(n/2) + O(1)$ cho $\Theta(\log n)$ (tìm kiếm nhị phân); $T(n) = 2T(n/2) + O(n)$ cho $\Theta(n \log n)$ (merge sort); $T(n) = T(n-1) + O(n)$ cho $\Theta(n^2)$ (quicksort trường hợp xấu nhất).

3.5 Phân tích khấu hao: khi một thao tác đắt trả cho nhiều thao tác rẻ

Xét mảng động: khi đầy thì cấp phát mảng mới gấp đôi và sao chép toàn bộ. Một lần thêm phần tử có thể tốn $O(n)$. Nhưng nói “thêm phần tử là $O(n)$ ” thì sai lệch nghiêm trọng, vì lần đắt đó

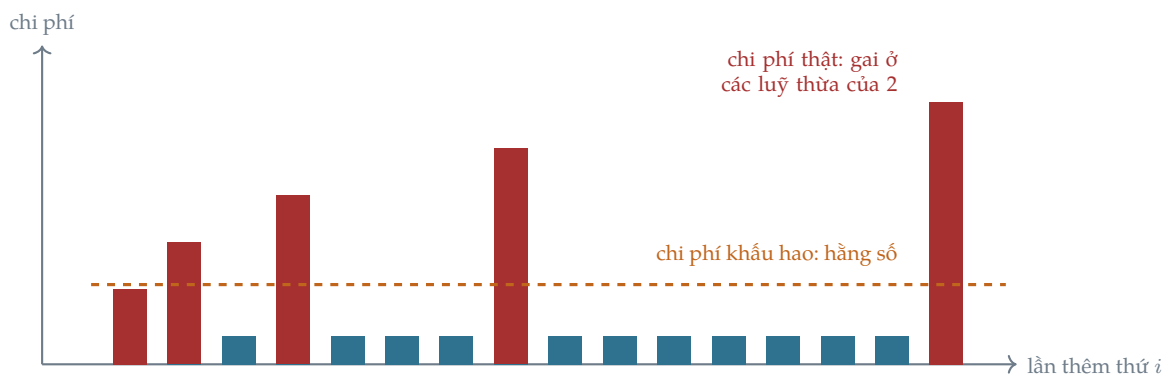
rất hiếm. Cách nói đúng: *chi phí khấu hao* của mỗi lần thêm là $O(1)$ — nghĩa là m lần thêm liên tiếp tốn tổng cộng $O(m)$, dù có vài lần lẻ tẻ tốn nhiều.

Chú ý một điểm bị nhầm rất nhiều: khấu hao *không* phải trung bình xác suất. Không có ngẫu nhiên nào ở đây; phát biểu đúng cho mọi dãy thao tác, kể cả dãy tệ nhất. Đây là công cụ Tarjan phát triển thành hệ thống[21], và nó có ba cách dùng.

Phương pháp tổng gộp: tính tổng chi phí của cả dãy rồi chia cho số thao tác. Với mảng động, tổng chi phí sao chép khi lớn tới n là $1 + 2 + 4 + \dots + n < 2n$, nên trung bình mỗi lần thêm là $O(1)$. Chuỗi hình học ấy là toàn bộ lý do vì sao phải nhân đôi chứ không phải cộng thêm một hằng số — nếu mỗi lần chỉ nới thêm 10 ô, tổng chi phí sao chép thành $\Theta(n^2)$ và chi phí khấu hao thành $\Theta(n)$.

Phương pháp kế toán: gán cho mỗi thao tác một “giá vé” cố định cao hơn chi phí thật, phần dư gửi vào tài khoản, và những thao tác đắt rút từ tài khoản ra. Với mảng động: mỗi lần thêm trả 3 đơn vị — 1 cho việc ghi, 2 gửi tiết kiệm để sau này trả cho việc sao chép chính nó và một phần tử cũ. Nếu chứng minh được tài khoản không bao giờ âm thì giá vé chính là chi phí khấu hao.

Phương pháp thế năng: tổng quát hơn, gán cho cấu trúc một hàm thế năng Φ đo “độ lộn xộn tích lũy”; chi phí khấu hao của một thao tác bằng chi phí thật cộng độ tăng của Φ . Thao tác đắt được phép vì nó làm Φ giảm mạnh. Đây là cách dùng cho những cấu trúc tinh vi như splay tree[35] và Fibonacci heap[36].



Hình 3.3. Chi phí thật của thao tác thêm vào mảng động (cột) so với chi phí khấu hao (đường ngang). Các gai xuất hiện đúng ở lũy thừa của 2 và cao gấp đôi lần trước, nhưng cũng thừa gấp đôi — hai hiệu ứng triệt tiêu nhau, để lại tổng tuyến tính. Đây là hình đáng nhớ nhất về khấu hao: *gai cao hơn thì phải thưa hơn tương ứng*.

CẠM BÃY

Khấu hao chỉ hợp lệ khi bạn quan tâm tới *tổng* thời gian. Trong hệ thống thời gian thực — điều khiển robot, xử lý âm thanh — một thao tác đột ngột tốn $O(n)$ có thể làm hỏng hạn kỳ, dù chi phí khấu hao rất đẹp. Ở đó người ta chọn cấu trúc có đảm bảo *trường hợp xấu nhất* cho từng thao tác, chấp nhận hằng số lớn hơn. Chương E/26 nói kỹ tình huống này.

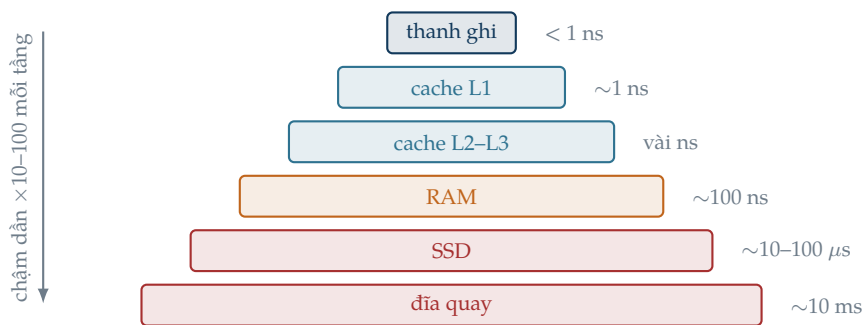
3.6 Chỗ mô hình nói dối: hằng số, bộ nhớ và cache

Đến đây ta có công cụ tiệm cận. Bây giờ là phần mà giáo trình hay nói lướt còn thực tế thì phạt rất nặng nếu bỏ qua.

Hằng số không nhỏ như ký hiệu gợi ý. Sắp xếp $O(n \log n)$ bằng `std::sort` nhanh hơn hẳn một segment tree $O(n \log n)$ tự viết, vì hằng số khác nhau cả chục lần. Ở kích thước nhỏ, thuật toán

“kém” thường thắng: đó là lý do các thư viện sắp xếp thật đều chuyển sang sắp xếp chèn khi đoạn còn dưới vài chục phần tử[14], [37].

Truy cập bộ nhớ mới là chi phí thật. Trên máy hiện đại, đọc một ô trong cache L1 nhanh hơn đọc từ RAM khoảng hai bậc độ lớn. Hệ quả rất cụ thể: duyệt một mảng liên tục nhanh hơn nhiều lần so với duyệt một danh sách liên kết cùng số phần tử, dù cả hai đều $O(n)$. Vì vậy mảng phẳng thường thắng cấu trúc con trỏ ở kích thước thật, và một segment tree cài bằng mảng nhanh hơn hẳn cài bằng con trỏ.



Hình 3.4. Phân cấp bộ nhớ với độ trễ theo bậc độ lớn (con số chỉ mang tính bậc, khác nhau theo máy). Mô hình RAM coi mọi tầng như nhau. Trong thực tế, một thuật toán “chậm hơn” về tiệm cận nhưng truy cập tuần tự có thể thắng một thuật toán “nhanh hơn” nhưng nhảy ngẫu nhiên khắp bộ nhớ — chương D/24 và E/26 khai thác đúng hiện tượng này.

Bộ nhớ cũng là một ngân sách. Một mảng hai chiều $10^4 \times 10^4$ kiểu int cần 400 MB — thường là vượt giới hạn. Khi gặp tình huống đó, hai lối thoát quen thuộc: giảm chiều của trạng thái quy hoạch động bằng cách chỉ giữ lại hàng trước (chương C/16), hoặc đổi sang cấu trúc thưa nếu dữ liệu thưa.

Và cuối cùng: đo, đừng đoán. Trong công việc thật, khoảng cách giữa dự đoán và thực tế đủ lớn để đáng bỏ hai phút chạy thử. Bài học của McSherry và cộng sự[38] rất đáng nhớ: nhiều hệ thống song song “có khả năng mở rộng” bị một chương trình đơn luồng viết cẩn thận đánh bại — vì chi phí điều phối lớn hơn phần được tăng tốc. Bậc tiệm cận không nói gì về những chi phí đó.

3.7 Đo bậc tăng trưởng thay vì đoán: phép thử nhân đôi

Cho tới đây ta luôn đi một chiều: từ mã nguồn suy ra chi phí. Chiều ngược lại — chạy chương trình rồi *đọc ngược* ra bậc tăng trưởng — rẻ hơn nhiều so với vẽ ngoài của nó và gần như không ai làm. Sedgewick và Wayne gọi kỹ thuật này là *phép thử tỉ lệ nhân đôi* (doubling ratio test), và khuyên chạy nó cho mọi chương trình mà hiệu năng có ý nghĩa[22].

Cách làm gồm ba bước. Một, viết một bộ sinh dữ liệu vào giống dữ liệu bạn thật sự gặp. Hai, chạy chương trình với n , rồi $2n$, rồi $4n$, ghi lại thời gian mỗi lần. Ba, lấy tỉ số giữa thời gian của hai lần liên tiếp. Nếu tỉ số ấy tiến tới một hằng số r , thì thời gian chạy xấp xỉ $a n^b$ với $b = \log_2 r$ — bậc của thuật toán hiện ra từ số đo, không cần đọc lại một dòng mã nào.

Vì sao tỉ số lại hội tụ? Chỉ cần một dòng tính. Nếu $T(n) \sim a n^b \lg n$ thì

$$\frac{T(2n)}{T(n)} = \frac{a (2n)^b \lg(2n)}{a n^b \lg n} = 2^b \left(1 + \frac{\lg 2}{\lg n}\right) \sim 2^b.$$

Nói cách khác, thừa số lôgarit — thứ luôn gây phiền khi phân tích bằng tay — gần như biến mất khỏi *tỉ số*. Đó là lý do phép đo này ổn định hơn ta tưởng, và cũng chính là lý do nó *không* phân biệt được $O(n)$ với $O(n \log n)$: cả hai đều cho tỉ số tiến về 2.

Ví dụ trong chính quyển sách của Sedgewick và Wayne đáng nhớ vì nó cho thấy sức mạnh dự đoán. Chương trình ThreeSum có ba vòng lặp lồng nhau; tỉ số đo được hội tụ về 8, tức $b = 3$, đúng bằng n^3 . Từ số đo 51,1 giây tại $n = 8000$, họ dự đoán 408,8 giây cho $n = 16\,000$ và 26 163 giây cho $n = 64\,000$ — chỉ bằng cách nhân liên tiếp với 8, không cần chạy thật.

Phép thử này trả lời ba câu hỏi mà lập luận trên giấy hay trả lời sai.

Chương trình có đúng như tôi nghĩ không? Nếu bạn tin mình viết $O(n \log n)$ mà tỉ số đo được là 4, thì ở đâu đó có một vòng lặp ẩn — thường là một phép sao chép chuỗi trong vòng lặp, một lần chèn vào giữa mảng, hoặc một lần tìm tuyến tính trong danh sách. Đây là cách rẻ nhất để bắt *lỗi hiệu năng*: loại lỗi không làm sai kết quả nên không phép kiểm tính đúng nào phát hiện được (E/25).

Chạy tới kích thước thật thì mất bao lâu? Nhân liên tiếp tỉ số cho một dự đoán đủ tốt, và nó biến câu hỏi “có kịp không” ở mục 3 từ ước lượng trong đầu thành một con số có cơ sở.

Ngưỡng hoà vốn nằm ở đâu? Khi hai lời giải khác bậc nhưng cũng khác hằng số, hai đường thời gian cắt nhau tại một điểm, và chỉ có đo mới tìm được điểm đó. Đây đúng là con số mà mọi thư viện sắp xếp phải chọn khi quyết định lúc nào chuyển sang sắp xếp chèn (C/14, E/26).

Ba điều kiện phải nhớ kèm, nếu không phép đo sẽ nói dối. Thứ nhất, tỉ số chỉ hội tụ với các bậc dạng lũy thừa; với thuật toán mũ nó tăng mãi — và chính việc *không* hội tụ đã là một câu trả lời. Thứ hai, bậc đo được là bậc *trên phân phối dữ liệu bạn sinh ra*: một bảng băm cho tỉ số 2 rất đẹp trên dữ liệu ngẫu nhiên vẫn có thể tệ đi n lần trên dữ liệu do đối thủ dựng (B/08). Thứ ba, ở kích thước nhỏ, thời gian bị chi phối bởi khởi động, cache lạnh và nhiễu đo; hãy bỏ vài điểm đầu và chỉ tin phần đuôi của bảng.

3.8 Gốc rễ: ký hiệu này mượn từ đâu và vì sao lại cần

Ký hiệu $O(\cdot)$ không sinh ra trong ngành máy tính. Nó xuất hiện cuối thế kỷ 19 trong lý thuyết số giải tích — Paul Bachmann dùng nó năm 1894, Edmund Landau phổ biến nó — để nói “sai số của xấp xỉ này không lớn hơn cỡ này” khi nghiên cứu phân bố số nguyên tố. Ở đó, biến chạy ra vô cùng là một đại lượng toán học thuần túy, và mối quan tâm là đáng điệu tiệm cận của hàm.

Ngành máy tính mượn ký hiệu ấy vì gặp đúng nhu cầu đó dưới hình thức khác: muốn so sánh hai thuật toán mà không phải nói “trên máy IBM 7090 với trình biên dịch này thì...”. Knuth là người chuẩn hoá cách dùng cho thuật toán và cũng là người kéo $\Omega(\cdot)$ và $\Theta(\cdot)$ vào để phân biệt chặn trên với chặn dưới — một phân biệt mà giới toán trước đó không cần tới, nhưng ngành thuật toán thì rất cần, vì ở đây ta phải nói được cả “thuật toán của tôi nhanh cỡ này” lẫn “mọi thuật toán đều không thể nhanh hơn cỡ này”.

Điều lý thú là mô hình máy cũng có gốc rễ tương tự: nó ra đời vì một mô hình cũ không dùng được cho mục đích mới. Máy Turing (1936) là mô hình chuẩn của lý thuyết tính toán, nhưng nó tính chi phí theo số bước trên băng, nên ngay cả việc đọc phần tử thứ i của mảng cũng tốn thời gian tỉ lệ i . Với câu hỏi “bài toán này có giải được không” thì chi tiết đó không quan trọng; với câu hỏi “thuật toán nào nhanh hơn” thì nó bóp méo mọi thứ. Mô hình RAM ra đời trong thập niên 1970 chính để lấp chỗ đó: cho phép truy cập ngẫu nhiên với giá một đơn vị, gần với máy thật hơn nhiều.

Còn từ “khấu hao” thì mượn thẳng từ kế toán, và nghĩa gốc soi sáng nghĩa kỹ thuật rất rõ. Trong kế toán, khi mua một cái máy đắt tiền dùng được mười năm, người ta không ghi toàn bộ chi phí vào tháng mua mà *phân bổ* nó ra suốt vòng đời. Tarjan mượn đúng ý đó: lần cấp phát lại tốn kém của mảng động không nên bị tính hết cho một thao tác thêm, mà nên phân bổ cho tất cả các thao tác đã hưởng lợi từ nó[21]. Khi đã thấy nghĩa gốc, phương pháp kế toán ở mục 5 không còn là mẹo mà là chính bút toán trả trước.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1894 — Bachmann, Landau	Cần nói gọn về sai số xấp xỉ trong lý thuyết số giải tích.	Ra đời ký hiệu O ; ban đầu không liên quan gì tới thuật toán.
1936 — Turing	Định nghĩa chính xác “tính được”.	Máy Turing: hoàn hảo cho câu hỏi <i>giải được hay không</i> , nhưng bóp méo chi phí truy cập nên không hợp để so tốc độ.
Thập niên 1960–70 — mô hình RAM	Cần một mô hình chi phí gần máy thật để phân tích và so sánh thuật toán.	Truy cập ngẫu nhiên giá một đơn vị; trở thành mô hình ngầm định của mọi ký hiệu $O(\cdot)$ trong sách vở.
1976 — Knuth	Trong thuật toán cần phân biệt “nhanh cỡ này” với “không thể nhanh hơn cỡ này”.	Chuẩn hoá bộ ba O, Ω, Θ ; mở đường cho lý thuyết cận dưới (D/24).
1985 — Tarjan	Các cấu trúc dữ liệu mới (splay tree, Fibonacci heap) có thao tác lẻ tẻ rất đắt nhưng tổng thể rất rẻ.	Hệ thống hoá phân tích khấu hao, mượn khái niệm phân bổ chi phí của kế toán.
1999 — Frigo và cộng sự	Khoảng cách tốc độ giữa CPU và bộ nhớ ngày càng lớn, mô hình RAM ngày càng nói dối.	Mô hình cache-oblivious: tính chi phí theo số lần chuyển khối bộ nhớ thay vì số phép toán (D/24).

I 3.9 Liên hệ ngang: so sánh với các cách đo chi phí khác

Mô hình RAM chỉ là một trong nhiều mô hình chi phí, và biết các mô hình khác giúp bạn nhận ra lúc nào mô hình quen thuộc đang nói dối. Bảng dưới đặt chúng cạnh nhau theo cùng một tiêu chí: đơn vị đếm, giả định chính, và chỗ hỏng.

Mô hình	Đếm cái gì	Hợp khi nào, hỏng khi nào
Máy Turing	Bước trên băng	Hợp cho câu hỏi giải được / độ khó lớp bài toán; hỏng khi so tốc độ vì truy cập không phải $O(1)$
RAM (mặc định)	Phép toán và truy cập	Hợp cho hầu hết phân tích; hỏng khi số rất lớn, khi chuỗi dài, khi cache quyết định
Độ phức tạp bit	Phép toán trên từng bit	Bắt buộc với số học số lớn, mật mã (D/21); nặng nề khi không cần
Bộ nhớ ngoài	Số lần đọc/ghi khối đĩa	Hợp với cơ sở dữ liệu, dữ liệu lớn hơn RAM; thừa khi mọi thứ nằm gọn trong bộ nhớ
Cache-oblivious	Số lần chuyển khối, không cần biết kích thước khối	Giải thích vì sao mảng thẳng con trở; khó phân tích hơn (D/24)
Song song (PRAM)	Số bước với p bộ xử lý	Hợp để tìm giới hạn song song hoá; bỏ qua chi phí đồng bộ và truyền tin, vốn thường mới là nút cổ chai
Luồng dữ liệu	Bộ nhớ dùng, số lượt quét	Hợp khi dữ liệu chảy qua một lần và không lưu được (D/24)
Năng lượng	Joule mỗi thao tác	Quan trọng trên thiết bị nhúng và robot; gần như vắng mặt trong sách giáo khoa thuật toán

Ngoài các mô hình tính toán, ý tưởng “chỉ quan tâm tốc độ tăng, bỏ hằng số” còn là một công cụ tư duy dùng khắp nơi ngoài ngành máy tính — và biết điều đó làm cho việc phân tích tiệm cận bớt giống một nghi thức học thuật.

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Vật lý	Phân tích thứ nguyên và luật tỉ lệ: bỏ hằng số, giữ số mũ	Cùng một động tác tư duy: muốn biết cái gì chi phối khi hệ lớn lên thì bỏ chi tiết và giữ số mũ. Khác: trong vật lý số mũ đến từ định luật bảo toàn; trong thuật toán nó đến từ cấu trúc đệ quy của lời giải.
Sinh học	Luật tỉ lệ trao đổi chất theo khối lượng cơ thể	Cùng dạng phát biểu “đại lượng này tăng theo lũy thừa của kích thước”. Bài học chung: khi hệ lớn gấp mười, thứ chi phối thường đổi hẳn — đúng như $O(n^2)$ thua $O(n \log n)$ ở quy mô lớn.
Kế toán, tài chính	Khấu hao tài sản, trả góp	Không phải chỉ giống mà là nguồn gốc trực tiếp của thuật ngữ: phân bổ một chi phí lớn cho nhiều kỳ hưởng lợi. Phương pháp kế toán ở mục 5 chính là bút toán trả trước.
Kinh tế	Chi phí biên, lợi thế quy mô	Chi phí khấu hao chính là chi phí biên trung bình của một thao tác. Cả hai đều nhắc: đừng đánh giá một hệ thống bằng chi phí của một sự kiện đơn lẻ.
Kỹ thuật hệ thống	Đo hiệu năng bằng hồ sơ chạy thật (profiling)	Bổ sung chứ không thay thế: tiệm cận nói đáng điều khi n lớn, profiling nói sự thật ở n hiện tại. Người chỉ dùng một trong hai đều sai — chi tiết ở E/26.
Ước lượng trạng thái, robot	Ngân sách tính toán theo chu kỳ thời gian thực	Ở đây cái quan trọng là <i>trường hợp xấu nhất mỗi khung hình</i> , không phải trung bình — nên cấu trúc có chi phí khấu hao đẹp có thể lại là lựa chọn sai. Xem cạm bẫy ở mục 5.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mô hình giữ lại thứ ta cần và bỏ đi thứ ta không kiểm soát. Điều ta cần là *tốc độ tăng khi dữ liệu lớn lên* — thứ quyết định lời giải có sống nổi ở giới hạn của đề hay không — và điều đó không phụ thuộc máy, ngôn ngữ hay trình biên dịch. Cái giá phải trả là mô hình mù với hằng số và với phân cấp bộ nhớ, nên nó chỉ đúng khi n đủ lớn; ở n nhỏ nó có thể sai hoàn toàn.
- A2. Ký hiệu tiệm cận cố ý bỏ hằng số nhân và các số hạng bậc thấp. Chênh lệch năm lần thường đến từ ba nguồn: hằng số của thao tác cơ bản (so sánh số nguyên rẻ hơn nhiều so với so sánh chuỗi hoặc phép chia), kiểu truy cập bộ nhớ (tuần tự so với nhảy ngẫu nhiên), và chi phí cấp phát. Với dữ liệu đủ lớn thì bậc luôn thắng; nhưng “đủ lớn” đôi khi lớn hơn dữ liệu thật của bạn.
- A3. Vì các lần đắt vừa cao hơn vừa thưa hơn theo đúng cùng một tỉ lệ, và hai hiệu ứng triệt tiêu nhau. Chi phí sao chép khi mảng lớn tới n là $1 + 2 + 4 + \dots + n < 2n$, một cấp số nhân bị chặn bởi hai lần số hạng cuối. Chia cho n lần thêm ta được hằng số. Điều then chốt là nhân đôi: nếu chỉ nói thêm một lượng cố định thì tổng chi phí thành $\Theta(n^2)$ và khấu hao thành $\Theta(n)$.
- A4. Vì hai con số đó xác định số phép toán bạn được tiêu — cỡ 10^8 — và từ đó ra bậc cho phép, còn bậc cho phép thì gần như xác định họ kỹ thuật. Với $n \leq 2 \cdot 10^5$ và một giây, bậc cho phép là $O(n \log n)$: điều này loại mọi ý tưởng “xét mọi cặp”, và trở thẳng tới sắp xếp, hai con trỏ, cây chỉ số, chia để trị hoặc tập hợp có thứ tự. Bạn vừa cắt bỏ phần lớn không gian tìm kiếm ý tưởng chỉ bằng cách đọc một dòng của đề.
- A5. Ít nhất bốn tình huống. Khi n nhỏ và luôn nhỏ — hằng số nhỏ và code đơn giản thắng. Khi $O(n^2)$ truy cập tuần tự còn $O(n \log n)$ nhảy ngẫu nhiên — cache có thể lật ngược kết quả. Khi thuật toán phức tạp có nguy cơ sai — một lời giải đúng chậm hơn vẫn hơn một lời giải nhanh mà sai. Và khi thời gian của bạn là tài nguyên khan hiếm hơn thời gian máy, ví dụ trong kỳ thi còn 15 phút hoặc trong đoạn mã chạy mỗi tháng một lần.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ tới việc dọn một thư viện. Nếu có gấp đôi số sách thì bạn mất gấp đôi thời gian để xếp chúng lên kệ, nhưng chỉ mất thêm một chút thời gian để *tìm* một quyển nếu kệ đã sắp theo thứ tự — tìm kiếm nhị phân là thế. Còn nếu bạn phải so từng cặp sách với nhau thì số việc tăng gấp bốn

khi số sách tăng gấp đôi; đó là n^2 , và nó là lý do những việc kiểu “xét mọi cặp” hỏng rất nhanh khi dữ liệu lớn.

Ký hiệu $O(\cdot)$ chỉ là cách nói “khi mọi thứ lớn gấp đôi thì công việc tăng bao nhiêu lần”. Nó cố tình không quan tâm bạn xếp sách nhanh hay chậm, vì cái đó đổi theo người, còn tỉ lệ tăng thì không. Khấu hao là chuyện khác và cũng rất đời thường. Bạn không đi chợ mỗi lần cần một quả trứng; bạn đi một chuyến và mua cả vỉ. Chuyến đi đó tốn thời gian, nhưng chia cho số quả trứng thì rẻ. Mảng động cũng vậy: thỉnh thoảng nó dọn nhà một lần thật tốn, nhưng vì mỗi lần dọn nó tăng gấp đôi sức chứa nên các lần dọn thừa dần đúng bằng mức chúng đất lên.

Cuối cùng, cái bảng “ n tới đâu thì được phép chậm tới đâu” chỉ là câu hỏi: tôi có ngân sách khoảng một trăm triệu việc nhỏ, dữ liệu chừng này, vậy mỗi phần tử tôi được làm bao nhiêu việc?

Một câu: Độ phức tạp là để loại bỏ ý tưởng trước khi gõ code, không phải để trang trí lời giải sau khi xong.

Chỗ không rút gọn được: Ranh giới “ n đủ lớn để bậc tiệm cận thắng hằng số” không có công thức chung; chỉ đo trên máy thật mới biết.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Mô hình RAM giả định gì	Mỗi phép toán cơ bản và mỗi truy cập ô nhớ bất kỳ tốn một đơn vị thời gian. Sai ở ba chỗ: cache, chi phí phép toán khác nhau, và số nguyên có kích thước hữu hạn.
••• Bảng giới hạn $\rightarrow$ độ phức tạp	$n \leq 20$: 2^n ; $n \leq 500$: n^3 ; $n \leq 5000$: n^2 ; $n \leq 10^5$: $n \log n$; $n \leq 10^7$: n ; $n \leq 10^{18}$: $\log n$. Ngân sách khoảng 10^8 phép toán mỗi giây.
••• $O()$, $\Omega()$, $\Theta()$ khác nhau	Chặn trên, chặn dưới, và cả hai. Nói lỏng $O()$ thay cho $\Theta()$ thì vô hại, trừ khi đang phát biểu cận dưới của một <i>bài toán</i> .
••• Ba truy hồi phải thuộc	$T(n) = T(n/2) + O(1) \Rightarrow \Theta(\log n)$; $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$; $T(n) = T(n-1) + O(n) \Rightarrow \Theta(n^2)$.
••• Khấu hao khác trung bình xác suất thế nào	Khấu hao không có ngẫu nhiên: nó là trung bình trên một <i>dãy</i> thao tác xấu nhất, đúng cho mọi dãy. Trung bình xác suất cần một phân phối giả định.
••• Vì sao mảng động phải nhân đôi	Vì tổng chi phí sao chép $1 + 2 + 4 + \dots + n < 2n$ là cấp số nhân, cho khấu hao $O(1)$. Nếu nói thêm hằng số thì tổng thành $\Theta(n^2)$.
•• Ba trường hợp của định lý thợ	So $f(n)$ với $n^{\log_b a}$: lá nặng $\Rightarrow \Theta(n^{\log_b a})$; đều nhau $\Rightarrow$ nhân thêm $\log n$; gốc nặng $\Rightarrow \Theta(f(n))$.
•• Ba cách phân tích khấu hao	Tổng gộp (cộng cả dây rồi chia); kế toán (giá vé cố định, gửi phần dư vào tài khoản); thế năng (hàm Φ đo độ lộn xộn tích lũy).
•• Khi nào khấu hao không dùng được	Hệ thời gian thực có hạn kỳ cho <i>từng</i> thao tác; ở đó cần đảm bảo trường hợp xấu nhất, chấp nhận hằng số lớn hơn.
•• Bấy “một phép toán là $O(1)$ ”	Sai với số lớn tùy ý (chi phí theo số chữ số) và với chuỗi (so sánh tốn theo độ dài) — <code>map<string, int></code> chậm hơn dự đoán một bậc vì lý do này.
•• Vì sao mảng thẳng danh sách liên kết	Truy cập tuần tự thân thiện với cache; RAM chậm hơn L1 khoảng hai bậc. Cùng $O(n)$ nhưng khác nhau nhiều lần trên máy thật.
•• “Tổng n qua các test bị chặn” nghĩa là gì	Bạn được phép chạy tuyến tính trên mỗi test mà không lo số test lớn — đây là gợi ý rất mạnh về dạng lời giải.
• Bài học COST[38]	Nhiều hệ thống song song bị một chương trình đơn luồng viết tốt đánh bại, vì chi phí điều phối không xuất hiện trong phân tích tiệm cận.
•• Phép thử tỉ lệ nhân đôi	Chạy với $n, 2n, 4n, \dots$; nếu tỉ số thời gian tiến tới r thì bậc là $n^{\log_2 r}$. Không phân biệt được $O(n)$ với $O(n \log n)$, và chỉ đúng cho phân phối dữ liệu bạn tự sinh.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao nói “thuật toán này có độ phức tạp trung bình $O(n \log n)$ ” mà không nói gì thêm là chưa đủ?

| Vì trung bình luôn phải kèm phân phối. Quicksort có trung bình $\Theta(n \log n)$ với giả định thứ tự đầu vào ngẫu nhiên đều; nếu dữ liệu đến đã sắp sẵn và bạn chọn chốt là phần tử đầu, nó tụt xuống $\Theta(n^2)$. Trong hệ thống thật, dữ liệu hiếm khi ngẫu nhiên và đôi khi do kẻ tấn công chọn. Cách sửa đúng là tự tạo ra ngẫu nhiên — chọn chốt ngẫu nhiên — để đảm bảo không phụ thuộc phân phối đầu vào; chương C/20 bàn kỹ.

2. Tôi có lời giải $O(n\sqrt{n})$ và một lời giải $O(n \log^2 n)$. Chọn cái nào?

| Tính thử ở n thật. Với $n = 10^5$: $n\sqrt{n} \approx 3 \cdot 10^7$, còn $n \log^2 n \approx 10^5 \cdot 17^2 \approx 3 \cdot 10^7$ — gần như bằng nhau, nên yếu tố quyết định là hằng số và độ khó cài đặt. Với $n = 10^6$ thì $n\sqrt{n} = 10^9$ đã quá lớn còn $n \log^2 n \approx 4 \cdot 10^8$ vẫn có cửa. Bài học chung: khi hai bậc gần nhau, đừng so ký hiệu — thay số thật vào.

3. Định lý thợ không áp được cho truy hồi của tôi. Làm sao?

| Vẽ cây đệ quy, đó là công cụ tổng quát hơn và gần như luôn cho câu trả lời. Tính chi phí mỗi tầng, xem chi phí tăng, giảm hay giữ nguyên theo tầng. Tăng theo cấp số nhân thì tổng do lá quyết định; giảm theo cấp số nhân thì do gốc quyết định; giữ nguyên thì nhân với số tầng. Nếu cây không đều (như truy hồi của thuật toán chọn trung vị), đoán nghiệm rồi chứng minh bằng quy nạp.

4. Vì sao log trong độ phức tạp không cần ghi cơ số?

| Vì đổi cơ số chỉ nhân với hằng số: $\log_2 n = \log_{10} n / \log_{10} 2$. Ký hiệu tiệm cận bỏ hằng số nhân, nên $\Theta(\log_2 n) = \Theta(\log_{10} n)$. Nhưng khi ước lượng số phép toán thật thì cơ số rất quan trọng: $\log_2 10^6 \approx 20$ còn $\log_{10} 10^6 = 6$, lệch hơn ba lần.

5. “Độ phức tạp giả đa thức” nghĩa là gì và vì sao nó quan trọng?

| Là độ phức tạp đa thức theo giá trị của số trong dữ liệu vào, chứ không theo số bit biểu diễn chúng. Quy hoạch động cho bài xếp ba lô chạy $O(nW)$ với W là sức chứa — nhìn thì đa thức, nhưng W được ghi bằng $\log W$ bit, nên theo kích thước dữ liệu vào thật thì đó là mũ. Đây chính là lý do bài ba lô vẫn là NP-khó dù có lời giải quy hoạch động; chương D/23 nói kỹ.

6. Tôi nên tính độ phức tạp bộ nhớ thế nào cho nhanh?

| Đếm mảng lớn nhất rồi nhân với kích thước phần tử: int 4 byte, long long và con trỏ 8 byte. Nhớ rằng các cấu trúc có con trỏ tồn thêm nhiều: mỗi nút của map hoặc set thường tốn cỡ 48 byte cho một phần tử vài byte. Đó là lý do một set<int> với 10^7 phần tử là không khả thi trong khi mảng int cùng kích thước chỉ tốn 40 MB.

7. Nếu thuật toán của tôi vượt giới hạn thời gian, tôi nên tìm thuật toán tốt hơn hay tối ưu hằng số?

| Trả lời bằng khoảng cách. Nếu bạn chậm gấp 2–3 lần, tối ưu hằng số thường đủ: bỏ cấu trúc có con trỏ, đổi map sang mảng, tránh cấp phát trong vòng lặp, đọc dữ liệu nhanh hơn. Nếu chậm gấp 50 lần trở lên, không có tối ưu hằng số nào cứu được — bậc của bạn sai và phải đổi ý tưởng. Ước lượng bằng bảng ở mục 3 trước khi quyết định, để khỏi tối ưu vi mô một lời giải chết từ trong ý tưởng.

8. Phân tích trường hợp xấu nhất có bị quan quá không, khi dữ liệu thật hiếm khi tệ như vậy?

| Có và không. Trong lập trình thi đấu, dữ liệu được cố ý làm tệ nhất có thể, nên phân tích xấu nhất là phân tích duy nhất đáng tin. Trong hệ thống thật, phân tích xấu nhất đôi khi quá bi quan — nhưng nó lại là điều bạn quan tâm nhất khi có hạn kỳ hoặc khi kẻ tấn công có thể chọn dữ liệu (băm là ví dụ kinh điển, xem chương B/08). Cách dùng thực dụng: dùng trường hợp xấu nhất để đảm bảo an toàn, và dùng đo đạc thực nghiệm để tối ưu.

I Bài tập (mức: ● khởi động ● ● chuẩn ● ● ● khó)

Bài	Nguồn	Điều cần nhận ra
• Với mỗi giới hạn $n = 10, 20, 500, 5000, 10^5, 10^7$, viết ra bậc cho phép và một thuật toán mẫu	Tự luyện	Mục tiêu là thuộc bảng mục 3 tới mức phản xạ; kiểm tra lại sau một tuần.
• Tính độ phức tạp của ba đoạn code có vòng lặp lồng nhau mà bước nhảy không đều	Bài tập kinh điển	Vòng lặp <code>for(i=1; i<=n; i*=2)</code> cho $\log n$; vòng lặp trong chạy tới i cho tổng $n^2/2$.
• Giải ba truy hồi cơ bản bằng cây đệ quy, không dùng định lý thợ	[27]	Vẽ cây thật ra giấy; mục tiêu là thấy được “tầng nào nặng” chứ không phải ra đáp số.
•• Chứng minh chi phí khấu hao của mảng động bằng cả ba phương pháp	[21]	Ba phương pháp cho cùng kết quả; điều cần rút ra là mỗi phương pháp dễ dùng ở tình huống khác nhau.
•• Phân tích chi phí khấu hao của bộ đếm nhị phân tăng dần	Bài tập kinh điển	Mỗi lần tăng lật trung bình 2 bit; dùng phương pháp kế toán với “một đơn vị gửi kèm mỗi bit 1”.
•• Đo thời gian thật: sắp xếp 10^6 số bằng <code>std::sort</code> và bằng cây tìm kiếm	Thực nghiệm	Cùng $O(n \log n)$ nhưng khác nhau nhiều lần; giải thích khác biệt bằng cache và cấp phát.
•• Duyệt mảng 10^7 phần tử theo thứ tự và theo hoán vị ngẫu nhiên, đo hai thời gian	Thực nghiệm	Đây là cách rẻ nhất để thấy phân cấp bộ nhớ bằng số liệu của chính máy bạn.
••• Giải truy hồi $T(n) = T(n/5) + T(7n/10) + n$ bằng quy nạp	[34]	Đoán $T(n) \leq cn$ và tìm c ; mấu chốt là $1/5 + 7/10 < 1$.
••• Tìm một trường hợp n nhỏ mà thuật toán $O(n^2)$ nhanh hơn $O(n \log n)$ trên máy bạn	Thực nghiệm	Sắp xếp chèn so với sắp xếp trộn; tìm điểm giao và giải thích vì sao thư viện thật cũng chuyển ngưỡng ở quanh đó.
••• Ước lượng bộ nhớ của quy hoạch động hai chiều $10^4 \times 10^4$ rồi tìm cách giảm chiều	Chương C/16	Chỉ giữ hàng trước; đây là kỹ thuật bạn sẽ dùng lại rất nhiều.
••• Đọc lại một lời giải cũ của bạn bị quá thời gian và phân loại: sai bậc hay sai hằng số	Nhật ký của bạn	Quy tắc ở mục hỏi đáp: chậm 2–3 lần thì sửa hằng số, chậm 50 lần thì phải đổi ý tưởng.

Kết chương và đọc tiếp

Chương này cho cái thước thứ nhất. Ba thứ đáng mang theo: bảng chuyển từ giới hạn đề bài sang độ phức tạp cho phép, hình ảnh “tầng nào nặng” để giải truy hồi mà không cần nhớ công thức, và ý niệm khấu hao — gai cao thì phải thưa tương ứng. Cùng với đó là một sự dè dặt lành mạnh: tiệm cận là mô hình, và ở kích thước thật thì hằng số cùng cache có tiếng nói riêng.

Nhưng nhanh chưa đủ. Một lời giải $O(n \log n)$ sai thì tệ hơn một lời giải $O(n^2)$ đúng. Chương A/04 cấp cái thước thứ hai — công cụ để biết lời giải *đúng* — và bạn sẽ thấy công cụ ấy, bất biến, không chỉ dùng để chứng minh mà còn dùng để *nghĩ ra* thuật toán.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Sedgewick & Wayne, <i>Algorithms</i> (4th ed.), chương 1.4	Addison-Wesley, 2011	Nguồn của phép thử nhân đôi và của cách viết $\sim$ cho hằng số. Đọc mục “Doubling ratio experiments” — ba trang, dùng được ngay.
nền tảng	Tarjan, “Amortized Computational Complexity”	SIAM Journal on Algebraic and Discrete Methods, 1985	Bài báo hệ thống hoá ba phương pháp khấu hao. Đọc phần thể năng nếu muốn hiểu vì sao splay tree và Fibonacci heap phải phân tích kiểu đó.
tra khi cần	Bentley & McIlroy, “Engineering a Sort Function”	Software: Practice and Experience, 1993	Tài liệu tốt nhất về khoảng cách giữa “thuật toán trong sách” và “hàm dùng được”: ngưỡng hoà vốn, chọn chốt, dữ liệu ác ý.
tra khi cần	Frigo, Leiserson, Prokop, Ramachandran, “Cache-Oblivious Algorithms”	FOCS, 1999	Đọc khi muốn một mô hình chi phí có tính tới phân cấp bộ nhớ mà không phải biết kích thước cache (D/24).
tra khi cần	McSherry, Isard, Murray, “Scalability! But at What COST?”	HotOS, 2015	Sáu trang, đổi cách bạn đọc mọi báo cáo hiệu năng song song. Đọc bảng so sánh với chương trình đơn luồng.

Tính đúng và chứng minh (Correctness and Proof)

PHỤ THUỘC A/01 cho vai trò của chứng minh trong bước 3; A/03 cho ngôn ngữ chỉ phí. Chương này là điều kiện cần cho C/15 (tham lam) và C/16 (quy hoạch động), nơi “có vẻ đúng” hỏng nhiều nhất.

TÓM TẮT

Chương này cấp cái thước thứ hai: *lời giải của tôi có đúng không*. Công cụ trung tâm là **bất biến** (*invariant*) — một mệnh đề luôn đúng tại một điểm cố định trong thuật toán. Bất biến có hai vai trò và vai trò thứ hai hay bị bỏ quên: nó vừa để chứng minh, vừa để *nghe ra* thuật toán. Chương đi qua bốn kiểu lập luận dùng nhiều nhất — bất biến vòng lặp, quy nạp, lập luận đổi chỗ cho tham lam, lập luận cắt-dán cho quy hoạch động — cộng với cách tìm phản ví dụ và cách chứng minh thuật toán *dùng*. Nếu chỉ nhớ một điều: viết một dòng bất biến trước mỗi vòng lặp là việc rẻ nhất bạn có thể làm để loại bỏ phần lớn lỗi lệch một đơn vị và lỗi khởi tạo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bất biến vòng lặp (loop invariant)** — mệnh đề đúng ngay trước mỗi lần lặp và vẫn đúng sau mỗi lần lặp. Ba việc phải kiểm: khởi tạo đúng, được duy trì, và khi thoát thì cho ra kết luận cần.
- **quy nạp (induction)** — chứng minh mệnh đề đúng cho mọi n bằng cách chứng minh nó đúng cho trường hợp nhỏ nhất, rồi chứng minh “đúng tới $n - 1$ kéo theo đúng ở n ”.
- **đại lượng giảm (variant / measure)** — một số nguyên không âm giảm nghiêm ngặt sau mỗi vòng lặp. Dùng để chứng minh thuật toán *dùng*, thứ mà bất biến không đảm bảo.
- **lập luận đổi chỗ (exchange argument)** — kỹ thuật chứng minh tham lam: lấy một lời giải tối ưu bất kỳ, biến đổi nó từng bước cho giống lựa chọn tham lam mà không làm nó tệ đi.
- **tiền điều kiện / hậu điều kiện** — điều kiện phải đúng trước khi chạy một đoạn mã, và điều kiện được đảm bảo sau khi chạy xong. Ngôn ngữ của Hoare[12].
- **phản ví dụ** — một dữ liệu vào cụ thể làm thuật toán cho kết quả sai. Một phản ví dụ đủ để bác bỏ; một triệu ví dụ đúng không đủ để chứng minh.

CÂU HỎI MỞ ĐẦU

- Q1. Chạy đúng trên một nghìn bộ test ngẫu nhiên — vì sao vẫn chưa phải bằng chứng đúng, và loại lỗi nào sống sót qua kiểm thử ngẫu nhiên?
- Q2. Bất biến là công cụ chứng minh. Vì sao chương này lại nói nó cũng là công cụ *sinh* thuật toán?
- Q3. Vì sao chứng minh một thuật toán tham lam khó hơn hẳn chứng minh một thuật toán quy hoạch động, dù code tham lam ngắn hơn nhiều?
- Q4. Một thuật toán có thể luôn giữ đúng bất biến mà vẫn sai. Sai ở đâu, và cần thêm gì?
- Q5. Khi nào thì bỏ việc chứng minh là quyết định hợp lý, chứ không phải lười?

Có một trải nghiệm mà ai luyện thuật toán cũng gặp: bạn viết xong, chạy thử vài ví dụ, thấy đúng, nộp bài, và nhận về “sai ở test 47”. Bạn không biết test 47 là gì. Bạn nhìn code, mọi dòng đều hợp lý. Và bạn bắt đầu sửa mò — đổi $<$ thành $\leq$, đổi khởi tạo, nộp lại. Đó là trạng thái tệ nhất trong việc lập trình: sửa mà không hiểu, và không biết khi nào thì xong.

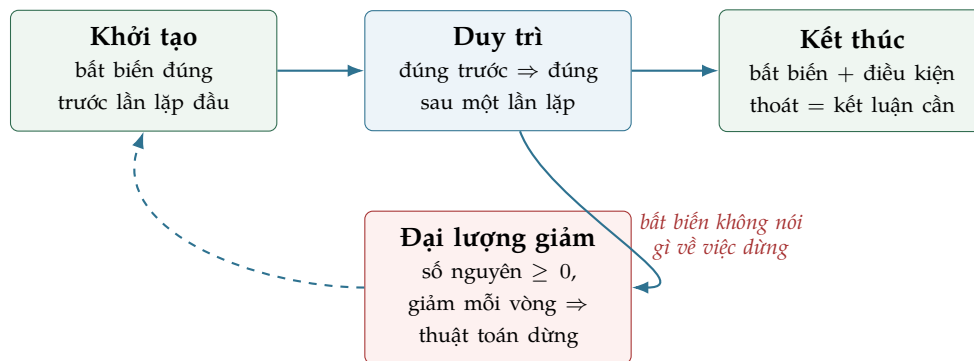
Chương này nói về cách thoát khỏi trạng thái đó, và câu trả lời không phải “kiểm thử nhiều hơn”. Dijkstra đã nói câu chính xác nhất về giới hạn của kiểm thử: nó cho thấy sự *hiện diện* của lỗi, không bao giờ cho thấy sự *vắng mặt* [11]. Muốn biết chương trình đúng, phải có một lập luận — và lập luận đó không nhất thiết phải hình thức, phần lớn thời gian nó chỉ là một hai câu viết ra cho rõ ràng.

Điều bất ngờ, và là luận điểm chính của chương: lập luận đó không phải việc làm *sau* khi có thuật toán. Làm đúng cách, nó chính là việc giúp bạn *tìm ra* thuật toán.

4.1 Bất biến vòng lặp: ba việc phải kiểm

Bất biến là một mệnh đề đúng tại một điểm cố định trong vòng lặp — thường là ngay trước mỗi lần lặp. Để dùng nó chứng minh một vòng lặp, cần kiểm ba việc, đúng như cấu trúc của phép quy nạp.

Khởi tạo: bất biến đúng trước lần lặp đầu tiên. *Duy trì*: nếu đúng trước một lần lặp thì vẫn đúng trước lần lặp kế tiếp. *Kết thúc*: khi vòng lặp thoát, bất biến cộng với điều kiện thoát cho ra đúng điều ta cần.



Hình 4.1. Bộ khung chứng minh một vòng lặp. Ba ô trên là bất biến; ô dưới là thứ thường bị quên: bất biến chỉ nói “nếu dừng thì đúng”, nó không hứa vòng lặp sẽ dừng. Muốn có tính dừng cần thêm một đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng.

Hãy làm một ví dụ đủ nhỏ để thấy hết. Tìm kiếm nhị phân trên mảng đã sắp tăng $a[0..n-1]$, tìm vị trí đầu tiên có $a[i] \geq x$. Bất biến chọn như sau: *mọi phần tử ở vị trí $< lo$ đều nhỏ hơn x , và mọi phần tử ở vị trí $\geq hi$ đều $\geq x$* . Nói cách khác, đáp án luôn nằm trong đoạn $[lo, hi]$.

Khởi tạo: đặt $lo = 0, hi = n$. Hai mệnh đề đúng một cách rỗng vì không có vị trí nào < 0 hay $\geq n$. **Duy trì**: lấy mid ở giữa; nếu $a[mid] < x$ thì mọi vị trí $\leq mid$ cũng nhỏ hơn x (vì mảng đã sắp), nên đặt $lo = mid + 1$ vẫn giữ bất biến; ngược lại đặt $hi = mid$. **Kết thúc**: khi $lo = hi$, bất biến nói mọi thứ bên trái nhỏ hơn x và mọi thứ từ lo trở đi $\geq x$ — đúng định nghĩa của vị trí cần tìm.

Điểm đáng chú ý: mọi câu hỏi cài đặt gây bối rối đều có câu trả lời rút ra từ bất biến, chứ không phải từ việc thử. hi khởi tạo bằng n hay $n - 1$? Bằng n , vì bất biến nói hi là vị trí đầu tiên đã biết là $\geq x$, mà có thể không có vị trí nào như vậy. Cập nhật $lo = mid + 1$ hay $lo = mid$? Bằng $mid + 1$, vì ta vừa chứng minh $a[mid] < x$ nên mid không thể là đáp án. Vòng lặp chạy khi $lo < hi$ hay $lo \leq hi$? Khi $lo < hi$, vì $lo = hi$ đã là kết luận.

CẦN NHỚ

Viết bất biến *trước* khi viết vòng lặp, dưới dạng một dòng chú thích bằng tiếng Việt. Đây là thói quen rẻ nhất và hiệu quả nhất trong cả quyển sách: nó biến các quyết định “< hay

<=" từ chỗ đoán mò thành chỗ suy ra được. Bentley[6] nhận xét rằng rất nhiều lập trình viên chuyên nghiệp viết sai tìm kiếm nhị phân khi phải viết từ đầu — gần như luôn vì họ viết code trước rồi mới nghĩ về ý nghĩa của các biến.

4.2 Tính dừng: thứ bất biến không hứa

Một điểm tinh tế mà người mới hay bỏ qua: bất biến chỉ chứng minh *nếu vòng lặp dừng thì kết quả đúng*. Nó hoàn toàn không nói vòng lặp sẽ dừng. Vòng lặp `while (true) { }` giữ mọi bất biến một cách hoàn hảo.

Công cụ cho tính dừng là *đại lượng giảm*: một số nguyên không âm, giảm ít nhất một đơn vị sau mỗi vòng. Vì số nguyên không âm không giảm được vô hạn lần, vòng lặp phải dừng. Với tìm kiếm nhị phân, đại lượng đó là $hi - lo$. Với thuật toán Euclid tính ước chung lớn nhất, là giá trị của số nhỏ hơn. Với thuật toán Ford–Fulkerson trên đồ thị có trọng số nguyên (chương 18), là tổng luồng còn thiếu — và chính chỗ này lộ ra vì sao thuật toán đó có thể không dừng khi trọng số là số vô tỉ.

Trong đệ quy, đại lượng giảm chính là “kích thước bài toán con nhỏ hơn kích thước bài toán gốc”. Nghe hiển nhiên, nhưng đây đúng là chỗ hỏng của rất nhiều lời giải đệ quy sai: một nhánh nào đó gọi lại chính nó với cùng kích thước.

4.3 Quy nạp: khuôn của mọi chứng minh về cấu trúc đệ quy

Quy nạp và đệ quy là hai mặt của một đồng xu: đệ quy giải bài lớn bằng bài nhỏ, quy nạp chứng minh mệnh đề cho n bằng mệnh đề cho các giá trị nhỏ hơn. Vì vậy mọi thuật toán đệ quy đều được chứng minh tự nhiên bằng quy nạp, và hai phần khớp nhau từng dòng.

Jeff Erickson[29] có một cách diễn đạt rất hữu ích cho việc viết đệ quy: hãy giả định rằng mọi lời gọi đệ quy *đã* cho kết quả đúng — ông gọi đó là “bà tiên đệ quy” — và bạn chỉ cần lo đúng một việc: từ kết quả của các bài toán con, ghép ra kết quả của bài toán gốc. Đây không phải mẹo tâm lý mà chính là bước quy nạp, viết dưới dạng dễ dùng. Người mới hay tự làm khổ mình bằng cách cố hình dung toàn bộ cây đệ quy; điều đó vừa bất khả thi vừa không cần thiết.

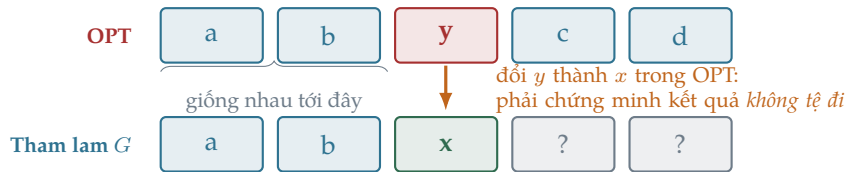
Hai biến thể đáng biết. *Quy nạp mạnh* giả định mệnh đề đúng cho *mọi* giá trị nhỏ hơn n , không chỉ $n - 1$ — cần thiết cho các thuật toán chia bài toán thành nhiều mảnh không đều. *Quy nạp cấu trúc* làm việc trên cây, đồ thị hoặc biểu thức thay vì trên số: chứng minh mệnh đề đúng ở lá, và đúng ở một nút nếu đúng ở mọi con của nó. Gần như mọi chứng minh về cây trong phần B đều theo khuôn này.

4.4 Lập luận đổi chỗ: vũ khí để chứng minh tham lam

Thuật toán tham lam là loại có khoảng cách lớn nhất giữa độ ngắn của code và độ khó của chứng minh. Code thường bốn dòng; chứng minh thì đòi hỏi thật sự, và nếu bỏ qua thì bạn sẽ nộp một lời giải sai với vẻ ngoài rất thuyết phục.

Khuôn chứng minh chuẩn là *lập luận đổi chỗ*: giả sử tồn tại một lời giải tối ưu OPT khác với lời giải tham lam G . Xét chỗ khác nhau *đầu tiên*. Chứng minh rằng ta có thể sửa OPT tại chỗ đó cho

giống G mà không làm nó tệ đi. Lặp lại, ta biến OPT thành G qua hữu hạn bước mà chất lượng không giảm — vậy G cũng tối ưu.



Hình 4.2. Sơ đồ lập luận đổi chỗ. Ta không cố chứng minh trực tiếp rằng tham lam tối ưu; ta chứng minh mọi lời giải tối ưu đều *biến đổi được* về lời giải tham lam mà không mất chất lượng. Toàn bộ công việc nằm ở mũi tên: chứng minh phép đổi tại vị trí khác nhau đầu tiên là hợp lệ và không làm tệ đi.

Ví dụ chuẩn: chọn nhiều nhất các cuộc họp không chồng lấn. Tham lam chọn cuộc họp *kết thúc sớm nhất*. Chứng minh: giả sử OPT chọn cuộc họp đầu tiên là y trong khi tham lam chọn x với thời điểm kết thúc $f(x) \leq f(y)$. Thay y bằng x trong OPT : mọi cuộc họp còn lại của OPT bắt đầu sau $f(y) \geq f(x)$, nên chúng vẫn không chồng lấn với x . Số cuộc họp không đổi, vậy lời giải mới vẫn tối ưu. Lặp lại cho vị trí thứ hai, thứ ba, và ta xong.

Một khuôn họ hàng là *tham lam luôn đi trước* (greedy stays ahead): chứng minh bằng quy nạp rằng sau k bước, lời giải tham lam tốt ít nhất bằng bất kỳ lời giải nào khác sau k bước. Với bài xếp lịch trên, sau k cuộc họp, thời điểm kết thúc của tham lam luôn sớm nhất có thể — và đó chính là lý do nó không bao giờ bị kẹt.

CẠM BÃY

Cạm bẫy chết người của tham lam: dùng “tôi thử vài ví dụ thấy đúng” làm chứng minh. Nhiều tiêu chí tham lam sai chỉ hổng ở những ví dụ khá đặc biệt. Ví dụ bài đổi tiền với các mệnh giá 1, 3, 4 và số tiền 6: tham lam lấy 4 rồi 1 rồi 1 (ba tờ), trong khi tối ưu là 3 + 3 (hai tờ). Nếu bạn chỉ thử với mệnh giá quen thuộc như 1, 5, 10 thì không bao giờ thấy lỗi. Quy tắc: hoặc chứng minh được bằng đổi chỗ, hoặc stress test với lời giải vét cạn (chương E/25); “thấy đúng” không nằm trong hai lựa chọn đó.

4.5 Lập luận cắt-dán: nền của quy hoạch động

Quy hoạch động đúng khi bài toán có *cấu trúc con tối ưu*: lời giải tối ưu của bài lớn chứa trong nó lời giải tối ưu của các bài con. Chứng minh chuẩn cho tính chất này là *cắt-dán*: giả sử lời giải tối ưu của bài lớn chứa một lời giải *không* tối ưu cho bài con nào đó; cắt phần đó ra, dán lời giải tối ưu của bài con vào, và ta được lời giải tốt hơn cho bài lớn — mâu thuẫn.

Nghe hình thức, nhưng nó có tác dụng thực hành rất cụ thể: khi cấu trúc con tối ưu *không* đúng, lập luận cắt-dán sẽ hổng ở một chỗ nhìn thấy được, và chỗ hổng đó cho bạn biết trạng thái đang thiếu chiều nào. Ví dụ kinh điển: đường đi *dài nhất* không có chu trình trong đồ thị tổng quát không có cấu trúc con tối ưu — ghép hai đường dài nhất có thể tạo ra đỉnh lặp. Nhận ra điều đó bằng cắt-dán nhanh hơn nhiều so với cài xong rồi mới thấy sai.

4.6 Tìm phản ví dụ: kỹ năng đối ngẫu với chứng minh

Chứng minh và phản ví dụ là hai mặt của cùng một việc, và người giải bài giỏi luôn làm cả hai xen kẽ: thử chứng minh vài phút, nếu chỗ nào không chứng minh được thì soi đúng chỗ đó để tìm phản ví dụ.

Ba nơi phản ví dụ hay ẩn nấp. *Trường hợp suy biến*: rỗng, một phần tử, mọi phần tử bằng nhau, đồ thị không liên thông, cạnh trọng số 0 hoặc âm. *Ranh giới của tiêu chí*: khi hai lựa chọn hoà nhau theo tiêu chí tham lam, cách phá hoà quyết định đúng sai. *Chỗ mà lập luận của bạn phải nói “hiển nhiên”*: mỗi lần bạn thấy mình viết chữ đó, hãy dừng lại kiểm tra — đó là nơi có mật độ lỗi cao nhất.

Và khi tìm bằng tay không ra, hãy để máy tìm: sinh dữ liệu nhỏ ngẫu nhiên, chạy song song lời giải nhanh và lời giải vét cạn, so kết quả. Kỹ thuật này — stress test — được nói kỹ ở chương E/25, và nó là cách tìm phản ví dụ hiệu quả nhất mà cũng ít tốn công nhất.

4.7 Chứng minh tới mức nào là đủ

Cần trung thực về liều lượng. Chứng minh hình thức đầy đủ cho mọi dòng code là điều không ai làm trong thực tế, kể cả người viết sách về chứng minh. Câu hỏi đúng không phải “có chứng minh hay không” mà là “chứng minh chặt tới mức nào cho phần nào”.

Có ba mức, và chọn mức là một quyết định kỹ thuật. *Mức một* — *một câu bằng lời*: dùng cho mọi vòng lặp, mọi lúc. Ví dụ “sau mỗi bước, best là đáp án tốt nhất trong $a[0..i]$ ”. Rẻ tới mức không có lý do gì để bỏ. *Mức hai* — *lập luận đầy đủ nhưng không hình thức*: dùng cho mỗi tiêu chí tham lam, mỗi công thức truy hồi quy hoạch động, mỗi chỗ bạn thấy “hình như đúng”. Đây là mức mà chương này chủ yếu dạy. *Mức ba* — *hình thức hoá*: dành cho những chỗ thật sự tinh vi, hiếm, và cho phần mềm mà lỗi có giá quá đắt.

Ngược lại, có những lúc bỏ chứng minh là hợp lý: trong kỳ thi khi bạn còn 10 phút và có thể stress test thay thế; khi thuật toán là một biến thể rất gần với một thuật toán đã được chứng minh; hoặc khi hậu quả của sai là nhỏ và phát hiện được ngay. Điều không hợp lý là bỏ chứng minh và bỏ luôn stress test, rồi tin vào cảm giác.

TRỰC GIÁC

Vì sao viết bất biến lại giúp *nghĩ ra* thuật toán chứ không chỉ kiểm chứng nó? Vì bất biến buộc bạn phát biểu “sau khi xử lý xong i phần tử đầu, tôi đang giữ đúng thông tin gì”. Câu hỏi đó chính là câu hỏi thiết kế trung tâm: nếu thông tin bạn giữ đủ để tính bước tiếp theo thì bạn có thuật toán; nếu chưa đủ thì bạn vừa phát hiện chính xác cái còn thiếu, và thứ còn thiếu đó thường trở thành một chiều mới của trạng thái. Đây là lý do người quen viết bất biến ít khi phải viết lại code từ đầu: họ phát hiện lỗ hổng ở giai đoạn suy nghĩ, chỗ mà sửa còn rẻ.

4.8 Máy trạng thái: một khung duy nhất cho ba loại bất biến

Chương này đã dùng bất biến ở ba vai trò trông rất khác nhau: bất biến vòng lặp để chứng minh một thuật toán đúng (mục 1), bất biến kiểu olympiad — chắn lẻ, tô màu — để chứng minh một cấu hình *không thể* đạt tới, và ở chương C/17 sẽ có thêm đồ thị trạng thái mà ta duyệt bằng tìm kiếm. Ba vai trò ấy là một, và Lehman, Leighton cùng Meyer[39] trình bày khung gộp chúng gọn hơn cả: **máy trạng thái** (state machine).

Máy trạng thái gồm ba thứ: một tập trạng thái, một trạng thái đầu, và một quan hệ chuyển. Một **bất biến bảo toàn** (*preserved invariant*) là một vị từ P trên trạng thái sao cho hễ $P(q)$ đúng và có bước chuyển $q \rightarrow r$ thì $P(r)$ cũng đúng. *Nguyên lý bất biến* phát biểu: nếu P đúng ở trạng thái đầu và được bảo toàn qua mọi bước chuyển, thì P đúng ở *mọi* trạng thái tới được. Nói cho đủ, đây chỉ là quy nạp theo số bước — nhưng viết ở dạng này thì dùng được ngay mà không phải dựng lại phép quy nạp mỗi lần.

Ví dụ nhỏ nhất trong sách của họ đáng nhớ vì nó gói cả ý tưởng trong ba dòng. Một robot xuất phát tại $(0, 0)$ trên lưới nguyên, mỗi bước đi chéo: $(m, n) \rightarrow (m \pm 1, n \pm 1)$. Nó có tới được $(1, 0)$ không? Xét vị từ “ $m + n$ chẵn”. Nó đúng ở trạng thái đầu, và mỗi bước làm tổng đổi đi $0, +2$ hoặc -2 nên tính chẵn được giữ. Mà $(1, 0)$ có tổng lẻ. Vậy robot *không bao giờ* tới được $(1, 0)$ — và chú ý loại kết luận: đây là điều mà không lượng thời gian chạy tìm kiếm nào chứng minh được, vì tìm kiếm chỉ có thể chạy mãi mà không tìm thấy. Bài Fifteen Puzzle trong cùng chương của họ là bản lớn hơn của đúng lập luận này.

Khung này đáng nhớ vì nó gộp ba việc thành một, và mỗi lần gộp là một lần bạn phải nhớ ít đi.

Vòng lặp là một máy trạng thái mà trạng thái là bộ giá trị các biến và bước chuyển là một lần chạy thân vòng lặp. Bất biến vòng lặp chính là bất biến bảo toàn, và “ba việc phải kiểm” ở mục 1 — đúng lúc đầu, được giữ qua mỗi vòng, kết luận được điều cần khi thoát — đúng là ba phần của nguyên lý trên.

Bài toán tìm kiếm trên không gian trạng thái cũng vậy, chỉ khác là ta duyệt thay vì lập luận. Và đây là chỗ khung này trả công nhiều nhất trong lúc thi: trước khi viết duyệt rộng trên hàng triệu trạng thái, hãy dành một phút hỏi có bất biến nào không. Nếu trạng thái đích vi phạm một bất biến thì khỏi chạy — đáp án là “không thể”. Nếu bất biến chia không gian thành nhiều lớp thì bạn chỉ phải duyệt lớp chứa trạng thái đầu, và không gian bài toán nhỏ đi theo đúng số lớp.

Còn tính dừng thì bất biến không nói gì, đúng như mục 2 đã cảnh báo. Máy trạng thái làm rõ chỗ khác nhau: bất biến là phát biểu về *tập trạng thái tới được*, còn đại lượng giảm là phát biểu về *độ dài của đường đi*. Hai câu hỏi khác nhau thì cần hai công cụ khác nhau, và lần chúng là lỗi phổ biến khi người ta tưởng đã chứng minh xong.

Một mẹo thực hành khấp lại mục: khi đã phát biểu được bất biến thành một câu, hãy cài luôn nó thành một phép kiểm assert chạy sau mỗi bước trong bản gỡ lỗi. Bất biến bị vi phạm ở đúng bước đầu tiên là thông tin đắt nhất mà bạn có thể có khi đi tìm lỗi (E/25).

4.9 Gốc rễ: chứng minh chương trình bắt đầu từ đâu

Ý tưởng lập luận về chương trình thay vì chỉ chạy thử nó xuất hiện sớm hơn nhiều người tưởng. Năm 1949, trong một bài viết ngắn về việc kiểm tra một chương trình lớn, Alan Turing đã trình bày gần như đầy đủ bộ khung của chương này: gắn vào từng điểm của chương trình một khẳng định về trạng thái, kiểm tra rằng mỗi bước bảo toàn các khẳng định đó, và — điểm đáng chú ý nhất — chứng minh riêng tính dừng bằng một đại lượng giảm. Nghĩa là ngay từ những năm đầu tiên của máy tính, người ta đã thấy *bất biến* và *đại lượng giảm* là hai thứ tách biệt và cùng cần thiết.

Ý tưởng đó bị bỏ quên gần hai mươi năm, cho tới khi Robert Floyd đưa ra cách gán ý nghĩa cho sơ đồ khối bằng các khẳng định (1967), và Tony Hoare biến nó thành hệ tiên đề với bộ ba tiên đề điều kiện–lệnh–hậu điều kiện (1969) [12]. Bối cảnh lịch sử giải thích vì sao chuyện này xảy ra vào đúng lúc đó: cuối thập niên 1960 là thời kỳ mà giới công nghiệp bắt đầu nói về “khủng hoảng phần mềm” — phần cứng mạnh lên nhanh, chương trình lớn lên theo, và số lỗi vượt quá khả năng kiểm soát bằng cách chạy thử. Dijkstra đẩy phản ứng ấy tới cực đoan với luận điểm rằng chương trình phải được *dẫn xuất* cùng với chứng minh của nó [11], và Gries biến nó thành sách dạy được [13].

Nhánh này về sau chia làm hai. Nhánh thực dụng thấm vào thói quen hằng ngày: khẳng định trong code, kiểm thử dựa trên tính chất, và chính lối viết bất biến bằng lời mà chương này khuyến nghị. Nhánh hình thức đi tới các công cụ kiểm chứng tự động — kiểm tra mô hình và trợ lý chứng minh — và tới thập niên 2000 đã đủ mạnh để chứng minh trọn vẹn những phần mềm thật: một trình biên dịch C có chứng minh và một nhân hệ điều hành có chứng minh là hai ví dụ được nhắc nhiều nhất. Cái giá là công sức lớn hơn viết code thông thường nhiều lần, nên nó chỉ dừng ở nơi mà một lỗi đắt tới mức đáng trả cái giá đó.

Với người luyện thuật toán, bài học từ lịch sử này rất cụ thể: bạn không cần nhánh hình thức, nhưng bạn cần đúng cái mà Turing đã dùng năm 1949 — một khẳng định ở mỗi điểm quan trọng, và một đại lượng giảm.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1949 — Turing	Kiểm tra một chương trình dài mà chạy thử không xuể.	Khẳng định tại từng điểm + đại lượng giảm cho tính dừng; bộ khung của cả chương này đã có mặt.
1967 — Floyd	Cần cách gán ý nghĩa chính xác cho sơ đồ khối.	Khẳng định gắn vào cạnh của sơ đồ; chính thức hoá ý tưởng bất biến.
1969 — Hoare	Muốn lập luận về chương trình như lập luận trong toán học.	Bộ ba $\{P\} S \{Q\}$; ngôn ngữ tiền điều kiện–hậu điều kiện dùng tới ngày nay.
Thập niên 1970 — Dijkstra, Gries	“Khủng hoảng phần mềm”: chương trình lớn hơn khả năng kiểm soát bằng chạy thử.	Chương trình và chứng minh phát triển cùng nhau; kiểm thử bị hạ xuống vai trò phát hiện lỗi, không phải bảo chứng.
Thập niên 1980–2000 — kiểm tra mô hình, trợ lý chứng minh	Hệ thống tương tranh và hệ thống trọng yếu mà lỗi có giá quá đắt.	Kiểm chứng tự động; đủ sức chứng minh trọn vẹn trình biên dịch và nhân hệ điều hành, với chi phí công sức rất lớn.

■ 4.10 Liên hệ ngang: bất biến là ý tưởng dùng chung của nhiều ngành

Đây có lẽ là chỗ liên hệ đẹp nhất trong cả phần A. Khái niệm “một đại lượng không đổi trong khi mọi thứ khác thay đổi” không phải phát minh của khoa học máy tính; nó là một trong những công cụ tư duy phổ quát nhất mà con người có, và nó xuất hiện dưới những cái tên khác nhau ở khắp nơi.

Trong vật lý, đó là các định luật bảo toàn: năng lượng, động lượng, điện tích. Ta không giải phương trình chuyển động cho từng hạt trong một vụ va chạm — ta viết ra đại lượng được bảo toàn và suy ngay ra kết quả. Đó chính xác là cách bất biến được dùng để chứng minh một thuật toán mà không phải theo dõi từng vòng lặp.

Trong lý thuyết điều khiển, đó là hàm Lyapunov: một hàm không âm của trạng thái, luôn giảm theo thời gian, dùng để chứng minh hệ hội tụ về điểm cân bằng. So sánh với mục 2 của chương này thì thấy ngay: hàm Lyapunov *chính là* đại lượng giảm, chỉ khác ở chỗ nó nhận giá trị thực và giảm liên tục thay vì nguyên và giảm rời rạc. Cùng một ý: gán một con số cho trạng thái, chứng minh con số đó đi đúng một chiều, và kết luận về hành vi dài hạn mà không phải mô phỏng.

Trong toán olympiad, bất biến và bán bất biến là một chủ đề riêng: tô màu bàn cờ, xét tính chẵn lẻ, xét tổng theo modulo — tất cả để chứng minh một cấu hình *không thể* đạt tới. Đây cũng là cách dùng mà lập trình viên hay quên: bất biến không chỉ để chứng minh thuật toán đúng, mà còn để chứng minh một việc là bất khả thi, tức là để *ngừng tìm* một lời giải không tồn tại.

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Vật lý	Định luật bảo toàn năng lượng, động lượng	Cùng một chiến lược: thay vì theo dõi diễn biến chi tiết, tìm đại lượng không đổi rồi suy trực tiếp ra kết luận. Khác: bất biến vật lý do tự nhiên quy định, còn bất biến thuật toán do bạn <i>chọn</i> — chọn khéo hay vụng quyết định chứng minh dễ hay khó.
Lý thuyết điều khiển	Hàm Lyapunov: hàm không âm, giảm dọc quỹ đạo $\Rightarrow$ hệ ổn định	Chính là đại lượng giảm ở mục 2, phiên bản liên tục. Cả hai đều chứng minh “rồi sẽ tới đích” mà không cần giải ra nghiệm. Khác: hệ liên tục cần thêm điều kiện để loại trường hợp giảm mãi mà không tới nơi.
Toán olympiad	Bất biến và bán bất biến: chẵn lẻ, tô màu, tổng theo modulo	Dùng để chứng minh <i>bất khả thi</i> — công dụng hay bị bỏ quên. Trong lập trình, nó tiết kiệm hàng giờ: nhận ra cấu hình đích vi phạm một bất biến thì bạn ngừng tìm thuật toán ngay.
Kế toán	Ghi sổ kép: tổng nợ luôn bằng tổng có	Một bất biến toàn hệ thống được kiểm tra liên tục; lệch một đồng là dấu hiệu có lỗi ở đâu đó. Tương đương assert chạy ở mọi bước.
Cơ sở dữ liệu	Ràng buộc toàn vẹn và tính chất giao dịch	Giao dịch được định nghĩa đúng bằng ngôn ngữ bất biến: hệ đúng trước, có thể tạm sai ở giữa, phải đúng lại khi kết thúc — y hệt cấu trúc khởi tạo / duy trì / kết thúc.
Ước lượng trạng thái, SLAM	Ràng buộc khả quan sát: bộ lọc không được tự sinh thông tin về những hướng mà dữ liệu không quan sát được	Đây là một bất biến thật của bộ ước lượng, và vi phạm nó gây ra sai lệch có hệ thống chứ không phải nhiễu ngẫu nhiên. Cùng bài học với mục 6: bất biến vi phạm thâm lạng nguy hiểm hơn lỗi làm chương trình sập.
Trình biên dịch	Bất biến kiểu (type safety): chương trình đã qua kiểm kiểu thì không kẹt ở trạng thái vô nghĩa	Cùng khuôn ba bước, áp lên toàn ngôn ngữ thay vì một vòng lặp. Đây là ví dụ cho thấy bất biến mở rộng được từ một hàm lên cả một hệ thống.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì kiểm thử chỉ chứng minh cho những dữ liệu đã thử. Ba loại lỗi sống sót qua kiểm thử ngẫu nhiên: lỗi ở trường hợp suy biến mà bộ sinh ngẫu nhiên hầu như không tạo ra (mảng rỗng, mọi phần tử bằng nhau, $n = 1$); lỗi chỉ xuất hiện ở kích thước lớn (tràn số, quá thời gian); và lỗi ở những cấu hình xác suất thấp (một tiêu chí tham lam hỏng đúng khi có hoà). Đó là ý câu của Dijkstra: kiểm thử cho thấy sự hiện diện của lỗi, không cho thấy sự vắng mặt.
- A2. Vì viết bất biến chính là trả lời câu hỏi “sau i bước tôi giữ đúng thông tin gì”, mà đó là câu hỏi thiết kế chứ không phải câu hỏi kiểm chứng. Nếu thông tin đang giữ đủ để bước tiếp thì bạn đã có thuật toán; nếu chưa đủ thì phần còn thiếu chỉ đúng ra thứ phải bổ sung — rất thường xuyên nó trở thành một chiều mới trong trạng thái quy hoạch động. Ví dụ mở đầu ở chương A/01 là minh họa: bất biến “cur là tổng lớn nhất của đoạn kết thúc tại j ” vừa là lời giải vừa là chứng minh.
- A3. Vì tham lam đưa ra một khẳng định mạnh hơn nhiều. Quy hoạch động chỉ khẳng định “tôi đã xét mọi khả năng phân rã”, và chứng minh chỉ cần kiểm rằng công thức truy hồi bao phủ hết các trường hợp. Tham lam khẳng định “lựa chọn cục bộ này an toàn cho *mọi* dữ liệu” — một mệnh đề toàn thể, cần lập luận đối chỗ để chứng minh, và sai chỉ với một phản ví dụ. Code ngắn hơn chính vì nó bỏ qua việc xét các khả năng, và cái giá là gánh nặng chứng minh chuyển sang bạn.
- A4. Sai ở tính dừng. Bất biến chỉ nói “nếu vòng lặp kết thúc thì kết quả đúng”; một vòng lặp vô hạn giữ mọi bất biến hoàn hảo. Cần thêm một đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng — *hi-lo* trong tìm kiếm nhị phân, số nhỏ hơn trong thuật toán Euclid. Trong đệ quy, đại lượng đó là kích thước bài toán con, và chỗ hỏng phổ biến là một nhánh gọi lại chính nó với cùng kích thước.
- A5. Khi có một cơ chế kiểm chứng khác đủ mạnh và rẻ hơn — điển hình là stress test đối chiếu với lời giải vét cạn, vốn bắt được gần hết lỗi logic trên dữ liệu nhỏ. Cũng hợp lý khi thuật toán chỉ

là biến thể rất sát một kết quả đã biết, hoặc khi chi phí của một lần sai là nhỏ và phát hiện được ngay. Không hợp lý là bỏ cả hai — không chứng minh và cũng không stress test — rồi dựa vào cảm giác “nhìn có vẻ đúng”.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn leo cầu thang trong bóng tối và muốn chắc chắn rằng cuối cùng mình sẽ lên tới sân thượng. Bạn cần hai điều. Điều thứ nhất: mỗi bước chân đều đặt lên một bậc thật, không hụt — đó là bất biến, thứ luôn đúng sau mỗi bước. Điều thứ hai: số bậc còn lại giảm đi sau mỗi bước — đó là đại lượng giảm, thứ bảo đảm bạn không đi mãi. Rất nhiều người chỉ nghĩ tới điều thứ nhất và ngạc nhiên khi chương trình chạy mãi không dừng.

Chứng minh một thuật toán tham lam thì giống chuyện khác. Bạn nói “lúc nào cũng cứ chọn cái kết thúc sớm nhất là ổn”. Người ta hỏi “sao biết?”. Cách trả lời thuyết phục nhất không phải kể vài ví dụ, mà là: đưa tôi bất kỳ cách xếp tốt nhất nào của bạn, tôi sẽ sửa nó từng chút một cho giống cách của tôi, và mỗi lần sửa tôi chứng minh nó không xấu đi. Sửa xong thì hai cách giống hệt nhau, vậy cách của tôi cũng tốt nhất.

Cuối cùng, có một điều nên nhớ về kiểm thử: chạy thử giống như kiểm tra vài viên gạch trong một bức tường. Thấy chục viên tốt không có nghĩa cả tường tốt. Nhưng chỉ cần tìm ra một viên vỡ là đủ biết tường có vấn đề — nên khi nghi ngờ, hãy tìm viên vỡ ở đúng những chỗ dễ vỡ nhất: góc tường, mép, chỗ mới xây.

Một câu: Bất biến trả lời “tôi đang giữ gì”; đại lượng giảm trả lời “tôi có dừng không”; đối chỗ trả lời “vì sao chọn thế là an toàn”.

Chỗ không rút gọn được: Không có quy trình máy móc nào tìm ra bất biến đúng; nó đến từ việc nhìn ví dụ nhỏ và hỏi “sau bước này tôi thực sự biết điều gì”.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba việc phải kiểm của bất biến	Khởi tạo (đúng trước lần lặp đầu), duy trì (đúng trước $\Rightarrow$ đúng sau), kết thúc (bất biến + điều kiện thoát = kết luận cần).
••• Bất biến không hứa điều gì	Không hứa vòng lặp dừng. Cần thêm đại lượng nguyên không âm giảm nghiêm ngặt mỗi vòng.
••• Bất biến của tìm kiếm nhị phân (lower bound)	Mọi vị trí $< lo$ có giá trị $< x$; mọi vị trí $\geq hi$ có giá trị $\geq x$. Từ đó suy ra mọi chi tiết cài đặt.
••• Lập luận đối chỗ	Lấy lời giải tối ưu bất kỳ, xét chỗ khác đầu tiên so với tham lam, chứng minh sửa cho giống tham lam không làm tệ đi; lặp lại tới khi trùng nhau.
••• Lập luận cắt-dán	Nếu lời giải tối ưu chứa một lời giải bài con không tối ưu, thay bằng lời giải tối ưu của bài con sẽ được kết quả tốt hơn — mâu thuẫn. Đây là chứng minh cấu trúc con tối ưu.
••• Vì sao tham lam khó chứng minh hơn quy hoạch động	Tham lam khẳng định “lựa chọn cục bộ an toàn cho mọi dữ liệu” — mệnh đề toàn thể; quy hoạch động chỉ khẳng định “đã xét mọi phân rã”.
•• Bà tiên đệ quy (Erickson)	Giả định mọi lời gọi đệ quy đã trả kết quả đúng; chỉ lo việc ghép kết quả bài con thành kết quả bài gốc. Đây chính là bước quy nạp.
•• Ba nơi phản ví dụ hay ẩn	Trường hợp suy biến; ranh giới hoà nhau của tiêu chí; và mọi chỗ lập luận của bạn phải nói “hiển nhiên”.
•• Phản ví dụ kinh điển của tham lam đối tiền	Mệnh giá 1, 3, 4 và số tiền 6: tham lam cho $4+1+1$, tối ưu là $3+3$.
•• Vì sao đường đi dài nhất không có cấu trúc con tối ưu	Ghép hai đường dài nhất của hai bài con có thể tạo đỉnh lặp, nên phép cắt-dán không hợp lệ.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Ba mức chứng minh	Một câu bằng lời cho mọi vòng lặp; lập luận đầy đủ không hình thức cho tham lam và truy hồi; hình thức hoá chỉ cho chỗ thật tinh vi.
●● Quy nạp mạnh dùng khi nào	Khi bài toán chia thành nhiều mảnh không đều, cần giả định đúng với mọi giá trị nhỏ hơn n chứ không chỉ $n - 1$.
● Tham lam luôn đi trước	Chứng minh bằng quy nạp rằng sau k bước, tham lam tốt ít nhất bằng mọi lời giải khác sau k bước.
● Khi nào bỏ chứng minh là hợp lý	Khi có stress test thay thế, khi thuật toán là biến thể sát một kết quả đã biết, hoặc khi sai thì rẻ và phát hiện ngay. Không hợp lý là bỏ cả hai.
●● Máy trạng thái và bất biến bảo toàn	Trạng thái đầu + quan hệ chuyển; vị từ đúng lúc đầu và được giữ qua mỗi bước thì đúng ở mọi trạng thái tới được. Vòng lặp, bài duyệt không gian trạng thái và bất biến olympiad là cùng một khung.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao tìm ra bất biến đúng? Nhìn vòng lặp rồi tôi vẫn không biết viết gì.

| Đi ngược: hỏi “khi vòng lặp kết thúc, tôi muốn có kết luận gì?” rồi hỏi “mệnh đề nào, cộng với điều kiện thoát, cho ra kết luận đó?”. Với tìm kiếm nhị phân, kết luận muốn có là “ lo là vị trí đầu tiên $\geq x$ ”, điều kiện thoát là $lo = hi$; mệnh đề khớp là “đáp án nằm trong $[lo, hi]$ ”. Một cách khác cũng hiệu quả: viết ra trạng thái các biến sau mỗi vòng cho một ví dụ $n = 5$ cụ thể, rồi tìm câu mô tả đúng mọi dòng của bảng đó.

2. Chứng minh thuật toán Dijkstra bằng bất biến như thế nào?

| Bất biến: với mọi đỉnh đã “chốt”, nhãn khoảng cách của nó bằng khoảng cách ngắn nhất thật. Duy trì: khi chốt đỉnh u có nhãn nhỏ nhất trong các đỉnh chưa chốt, giả sử tồn tại đường ngắn hơn tới u ; đường đó phải rời tập đã chốt tại một cạnh nào đó, tới một đỉnh v chưa chốt, và vì mọi trọng số không âm nên phần đầu của đường đó đã dài ít nhất bằng nhãn của v , mà nhãn của v không nhỏ hơn nhãn của u — mâu thuẫn. Chú ý chỗ dùng giả thiết trọng số không âm: đó chính xác là chỗ thuật toán hỏng với cạnh âm.

3. Tôi chứng minh tham lam đúng nhưng vẫn sai khi nộp. Chuyện gì xảy ra?

| Ba khả năng, theo thứ tự tần suất. Thứ nhất, chứng minh có một bước bạn nói “rõ ràng” mà thực ra không đúng — hãy đọc lại đúng những chỗ đó. Thứ hai, tiêu chí bạn cài không hoàn toàn là tiêu chí bạn chứng minh, thường ở cách phá hoà khi hai phần tử bằng nhau. Thứ ba, thuật toán đúng nhưng cài sai. Cách phân biệt nhanh: stress test với vết cạn trên $n \leq 8$. Nếu sai trên dữ liệu nhỏ thì là lỗi một hoặc ba; nếu chỉ sai ở dữ liệu lớn thì thường là tràn số hoặc quá thời gian, không phải lỗi ý tưởng.

4. Bất biến trong code (assert) có thay được chứng minh không?

| Không thay được, nhưng bổ sung rất tốt. assert kiểm bất biến trên những dữ liệu thực sự chạy qua, nên nó bắt lỗi ở đúng chỗ phát sinh thay vì để lỗi lan ra và chỉ lộ ở kết quả cuối — điều này rút ngắn thời gian gỡ lỗi rất nhiều. Cách dùng tốt: mỗi bất biến bạn viết thành chú thích, nếu kiểm được rẽ thì viết luôn thành assert. Nhớ tắt trong bản nộp nếu nó tốn thời gian.

5. Chứng minh quy hoạch động gồm những gì?

| Ba phần. Một, định nghĩa trạng thái phải rõ ràng và đủ: từ trạng thái, mọi quyết định sau này không cần biết thêm gì về quá khứ. Hai, công thức truy hồi phải bao phủ: mọi cách tạo ra trạng thái này đều nằm trong một nhánh của công thức. Ba, thứ tự tính phải hợp lệ: khi tính một trạng thái, mọi trạng thái nó phụ thuộc đã được tính — tức đồ thị phụ thuộc không có chu trình. Phần lớn lỗi quy hoạch động nằm ở điểm một (trạng thái thiếu chiều) hoặc điểm hai (quên một nhánh).

6. Có cần chứng minh cả những thuật toán kinh điển tôi chỉ dùng lại?

| Không cần chứng minh lại, nhưng cần biết *giả thiết* của chúng, vì đó là chỗ chúng hỏng. Dijkstra cần trọng số không âm. Tham lam Huffman cần chi phí là tổng có trọng số. Sắp xếp hai giai đoạn cần quan hệ so sánh có tính bắc cầu — một hàm so sánh không nhất quán sẽ làm `std::sort` hành xử không xác định, kể cả sắp chương trình. Quy tắc: thuộc giả thiết, không cần thuộc chứng minh.

7. Chứng minh bằng phản chứng và bằng đối chỗ khác nhau chỗ nào?

| Đối chỗ là một dạng chứng minh có tính xây dựng: nó chỉ ra cách biến lời giải tối ưu thành lời giải tham lam, nên đọc xong bạn hiểu *vì sao* tham lam an toàn. Phản chứng chỉ nói “giả sử sai thì mâu thuẫn”, đúng nhưng ít soi sáng hơn. Trong thực hành, đối chỗ có lợi thế lớn: nếu phép đối không thực hiện được, chỗ hỏng thường chính là phản ví dụ đang cần tìm.

8. Phương pháp hàm thế năng ở chương A/03 có liên quan gì tới bất biến không?

| Rất gần nhau. Bất biến là mệnh đề đúng ở mọi thời điểm; hàm thế năng là một *đại lượng số* mà ta phát biểu bất biến về nó — “ Φ luôn không âm” — và ta lập luận qua độ biến thiên của nó. Đại lượng giảm để chứng minh tính dừng cũng cùng họ: cũng là một số nguyên với bất biến “không âm” và tính chất “giảm mỗi bước”. Ba công cụ này là ba biến tấu của cùng một ý: gán một con số cho trạng thái rồi lập luận về cách con số đó thay đổi.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết bất biến và chứng minh tính dừng cho sắp xếp chèn	[27]	Bất biến: đoạn $a[0..i-1]$ đã sắp và gồm đúng các phần tử ban đầu của đoạn đó.
● Viết bất biến cho thuật toán Euclid, chỉ ra đại lượng giảm	Bài tập kinh điển	Bất biến $\gcd(a, b)$ không đổi; đại lượng giảm là $\min(a, b)$.
● Tìm phản ví dụ cho tham lam “luôn lấy đồng tiền lớn nhất” với mệnh giá bất kỳ	Bài tập kinh điển	Mệnh giá 1, 3, 4 với số tiền 6. Tự tìm thêm một bộ mệnh giá khác cũng hỏng.
●● Chứng minh tham lam xếp lịch theo thời điểm kết thúc sớm nhất	[28]	Viết đầy đủ cả hai cách: đối chỗ và “tham lam luôn đi trước”.
●● Chứng minh cấu trúc con tối ưu của bài dây con chung dài nhất bằng cắt-dán	[27]	Xét ký tự cuối của hai chuỗi, chia hai trường hợp; cắt-dán ở từng trường hợp.
●● Chỉ ra chính xác chỗ chứng minh Dijkstra dùng giả thiết trọng số không âm	Chương C/17	Sau đó dựng một đồ thị có cạnh âm làm Dijkstra sai, càng nhỏ càng tốt.
●● Viết bất biến cho thuật toán hai con trỏ tìm cặp có tổng bằng x	Chương B/06	Bất biến: mọi cặp có chỉ số ngoài đoạn $[l, r]$ hiện tại đều đã bị loại một cách hợp lệ.
●●● Chứng minh thuật toán bỏ phiếu Boyer–Moore tìm phần tử quá bán	Bài tập kinh điển	Bất biến khó tìm; gợi ý: nghĩ theo kiểu “triệt tiêu từng cặp phần tử khác nhau”.
●●● Chứng minh tham lam của Kruskal đúng bằng lập luận đối chỗ	[28]	Tính chất cắt (cut property); đây là bài tập tiêu biểu cho cả họ chứng minh matroid ở chương C/15.
●●● Tìm một bài mà tham lam “có vẻ đúng” và dùng stress test để bác bỏ	Chương E/25	Mục tiêu kép: luyện viết bộ sinh test và luyện thói quen không tin cảm giác.
●●● Chứng minh tính dừng của Ford–Fulkerson với trọng số nguyên, và chỉ ra vì sao có thể không dừng với trọng số vô tỉ	Chương C/18	Đại lượng giảm chỉ tồn tại khi các bước tăng luồng bị chặn dưới bởi một hằng số dương.

Kết chương và đọc tiếp

Với chương này, phần A đã cấp đủ hai cái thước: [A/03](#) cho “đủ nhanh không”, chương này cho “có đúng không”. Ba công cụ đáng mang theo là bất biến (tôi đang giữ gì), đại lượng giảm (tôi có dừng không) và lập luận đối chỗ (vì sao lựa chọn này an toàn). Điểm quan trọng nhất, và cũng là điểm dễ quên nhất: cả ba đều là công cụ *thiết kế*, không chỉ công cụ kiểm chứng.

Còn một mảnh nữa của nền tảng: bộ toán mà các chương sau sẽ dùng rải rác — tổng, truy hồi, tổ hợp, xác suất. Đó là chương [A/05](#), và nó là chương duy nhất trong quyển sách này không nên đọc tuần tự. Sau đó, phần B bắt đầu với câu hỏi mới: dữ liệu nên nằm ở dạng nào để thao tác tôi cần trở nên rẻ.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Lehman, Leighton, Meyer, <i>Mathematics for Computer Science</i> , chương “State Machines”	sách giáo trình MIT, bản 2018	Nguồn của khung máy trạng thái và nguyên lý bất biến. Đọc mục 6.1–6.2 cùng bài tập Fifteen Puzzle — khoảng hai mươi trang, và là cách vào đề tốt nhất cho người viết code.
nền tảng	Hoare, “An Axiomatic Basis for Computer Programming”	Communications of the ACM, 1969	Bài mở đầu của cả ngành đặc tả và chứng minh chương trình. Ngắn, và đọc được ngay cả khi bạn không định dùng phương pháp hình thức.
tra khi cần	Gries, <i>The Science of Programming</i>	Springer, 1981	Sách dạy viết chương trình <i>cùng lúc</i> với chứng minh; đọc khi muốn kỹ năng phát biểu bất biến trở thành phản xạ.
tra khi cần	Erickson, <i>Algorithms</i>	bản điện tử miễn phí, 2019	Chương về đệ quy và quy nạp; nguồn của hình ảnh “bà tiên đệ quy” ở mục 3.

Toán cho thuật toán (The Mathematics You Actually Reuse)

PHỤ THUỘC A/03 cho ký hiệu tiệm cận. Chương này không nên đọc tuần tự: đọc lướt một lượt để biết trong đó có gì, rồi quay lại đúng mục khi một chương sau cần tới.

TÓM TẮT

Đây là kho toán dùng lại nhiều nhất trong 23 chương còn lại, chia theo *câu hỏi nó trả lời* chứ không theo tên môn. Mục 1 là tổng và chặn — dùng mỗi lần bạn phân tích một vòng lặp. Mục 2 là tổ hợp đếm — dùng cho mọi bài đếm và cho việc ước lượng kích thước không gian tìm kiếm. Mục 3 là xác suất và kỳ vọng, với một công cụ mạnh bất thường là kỳ vọng tuyến tính. Mục 4 là số học modulo, mục 5 là đại số rời rạc, mục 6 là lý thuyết trò chơi tối thiểu. Nếu chỉ nhớ một điều: ba thứ quay lại nhiều nhất là chuỗi hình học bị chặn bởi hai lần số hạng lớn nhất, tổng điều hoà xấp xỉ $\ln n$, và kỳ vọng của tổng bằng tổng các kỳ vọng dù các biến phụ thuộc nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chuỗi hình học** — tổng dạng $1 + r + r^2 + \dots$; khi $r > 1$, tổng bị chi phối bởi số hạng cuối, khi $r < 1$ thì bởi số hạng đầu.
- **tổng điều hoà** — $H_n = 1 + 1/2 + \dots + 1/n \approx \ln n + 0,577$. Xuất hiện mỗi khi có “lần thứ i tốn $1/i$ ” hoặc “đếm bội số”.
- **hệ số nhị thức** — $\binom{n}{k}$ — số cách chọn k phần tử từ n phần tử, không kể thứ tự.
- **biến chỉ báo (indicator)** — biến nhận giá trị 1 nếu một sự kiện xảy ra và 0 nếu không. Kỳ vọng của nó bằng đúng xác suất của sự kiện.
- **số học modulo** — làm việc với phần dư khi chia cho m ; phép cộng, trừ, nhân đều bảo toàn phần dư, phép chia thì cần nghịch đảo.
- **nguyên lý chuồng bồ câu** — nhốt $n + 1$ con vào n chuồng thì có chuồng chứa từ hai con trở lên. Công cụ chứng minh tồn tại đơn giản nhất.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chuỗi $1 + 2 + 4 + \dots + n$ lại “chỉ bằng” $O(n)$, trong khi chuỗi $1 + 1/2 + \dots + 1/n$ lại không phải $O(1)$?
- Q2. Kỳ vọng của tổng bằng tổng các kỳ vọng ngay cả khi các biến phụ thuộc chằng chịt. Vì sao điều đó lại đúng, và vì sao nó mạnh đến vậy?
- Q3. Vì sao gần như mọi bài đếm trong lập trình thi đấu đều yêu cầu kết quả theo modulo một số nguyên tố lớn?
- Q4. Nguyên lý chuồng bồ câu tầm thường tới mức trẻ con cũng hiểu. Vì sao nó lại chứng minh được những điều không hiển nhiên chút nào?
- Q5. Khi nào một bài toán “tính giá trị sau 10^{18} bước” lại giải được, và cấu trúc nào làm nó giải được?

Chương này khác mọi chương khác trong quyển sách: nó không có một mạch lập luận từ đầu tới cuối, vì nó là kho chứ không phải là bài. Cách dùng đúng là đọc lướt một lượt ngay bây giờ —

đủ để nhớ “trong đó có gì” — rồi quay lại từng mục khi một chương sau thực sự cần. Mỗi mục đều ghi rõ nó quay lại ở chương nào.

Nguyên tắc chọn nội dung cũng cần nói rõ: tôi chỉ giữ những mảnh toán mà tôi thấy quay lại *ít nhất ba lần* trong các chương sau. Rất nhiều thứ đẹp đẽ đã bị bỏ ra ngoài, không phải vì chúng không hay mà vì học một công cụ không có nhu cầu đi kèm thì hai tuần sau sẽ quên sạch.

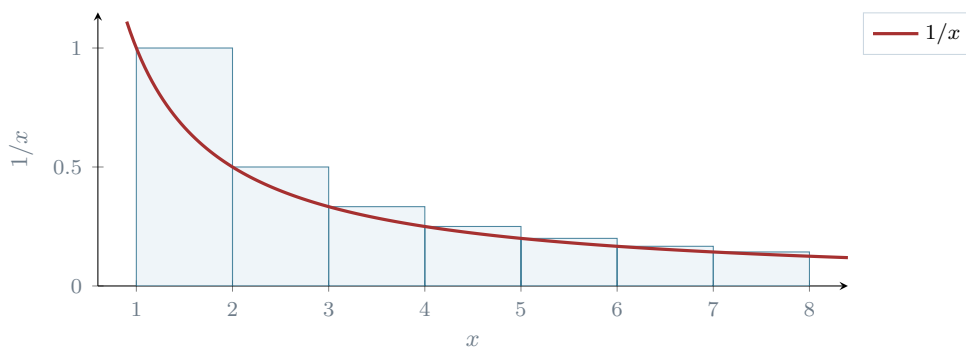
5.1 Tổng và chặn: đọc chi phí của một vòng lặp

Ba dạng tổng chiếm gần hết những gì bạn cần.

Cấp số cộng. $1 + 2 + \dots + n = n(n+1)/2 = \Theta(n^2)$. Xuất hiện khi vòng lặp trong chạy tới chỉ số của vòng lặp ngoài — ví dụ mọi thuật toán “xét mọi cặp”.

Cấp số nhân. $1 + 2 + 4 + \dots + 2^k = 2^{k+1} - 1 < 2 \cdot 2^k$. Điều đáng nhớ không phải công thức mà là câu này: **tổng của một cấp số nhân tăng bị chặn bởi hai lần số hạng lớn nhất**. Câu đó giải thích ngay chi phí khấu hao của mảng động (A/03), chi phí xây dựng cây phân đoạn, và chi phí tổng cộng của kỹ thuật nhân đôi. Ngược lại, với $r < 1$ thì tổng bị chặn bởi hằng số nhân số hạng đầu: $1 + 1/2 + 1/4 + \dots < 2$ — lý do vì sao chia đôi mãi vẫn chỉ tốn tuyến tính.

Tổng điều hoà. $H_n = \sum_{i=1}^n 1/i \approx \ln n$. Đây là tổng bị đánh giá sai nhiều nhất: nó *không* hội tụ, nhưng tăng cực chậm. Hình 5.1 cho thấy vì sao xấp xỉ bằng $\ln n$: tổng các cột chữ nhật kẹp giữa hai tích phân. Tổng này xuất hiện ở ba chỗ rất khác nhau và đáng nhớ cả ba: sàng Eratosthenes tốn $\sum_p n/p \approx n \ln \ln n$ (D/21); quicksort ngẫu nhiên tốn trung bình $2n \ln n$ so sánh (C/14); và bài toán “sưu tập đủ bộ” cần trung bình nH_n lần thử.



Hình 5.1. Vì sao $H_n \approx \ln n$. Mỗi cột có diện tích $1/i$, nên tổng diện tích các cột là H_n ; đường cong $1/x$ có tích phân từ 1 tới n bằng $\ln n$. Các cột kẹp đường cong từ trên, cho $\ln n < H_n < \ln(n+1)$. Bài học tổng quát hơn: khi cần chặn một tổng, hãy so nó với tích phân của hàm tương ứng — đây là mẹo dùng được cho hầu hết các tổng đơn điệu.

Ngoài ba dạng trên, hai xấp xỉ đáng nhớ: $\log_2(n!) = \Theta(n \log n)$ — công thức Stirling, và là lý do cận dưới của sắp xếp so sánh là $\Omega(n \log n)$ (D/24); và $\binom{n}{k} \leq (en/k)^k$ — dùng để chặn kích thước không gian tìm kiếm.

5.2 Tổ hợp đếm: bao nhiêu khả năng, và đếm chúng thế nào

Bốn công thức nền, mỗi công thức ứng với một câu hỏi.

Chọn có thứ tự, không lặp: $n!/(n-k)!$. Chọn không thứ tự, không lặp: $\binom{n}{k}$. Chọn có lặp, có thứ tự: n^k . Chọn có lặp, không thứ tự (bài toán chia kẹo): $\binom{n+k-1}{k}$ — kỹ thuật “sao và vách ngăn”.

Hai đẳng thức đáng thuộc vì chúng biến thành thuật toán. *Đẳng thức Pascal* $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ chính là công thức truy hồi để tính bảng hệ số nhị thức bằng quy hoạch động, và cách chứng minh nó — “phần tử thứ n hoặc được chọn hoặc không” — là khuôn mẫu của mọi lập luận chia trường hợp trong chương C/16. *Nguyên lý bao hàm–loại trừ* cho phép đếm hợp của các tập bằng cách cộng rồi trừ các giao; nó là nền của kỹ thuật đếm bù — đếm cái xấu rồi lấy tổng trừ đi — vốn thường dễ hơn đếm trực tiếp cái tốt.

Số Catalan $C_n = \frac{1}{n+1} \binom{2n}{n}$ đáng biết vì nó đếm một họ bài toán trông rất khác nhau nhưng cùng cấu trúc: số dãy ngoặc đúng độ dài $2n$, số cây nhị phân có n nút, số cách chia đa giác thành tam giác, số đường đi không vượt đường chéo. Khi bạn tính tay các trường hợp nhỏ và ra dãy 1, 2, 5, 14, 42 thì gần như chắc chắn bài của bạn có cấu trúc Catalan — một ví dụ đẹp cho heuristic “làm ví dụ nhỏ rồi tra dãy số” ở chương A/01.

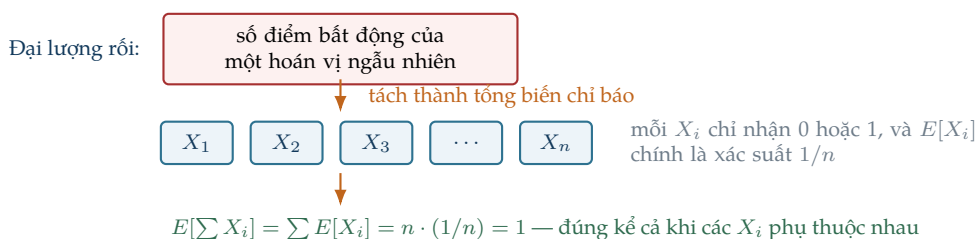
Nguyên lý chuồng bồ câu tầm thường về phát biểu nhưng mạnh về hệ quả, vì nó chứng minh sự tồn tại mà không cần chỉ ra vật thể. Ví dụ dùng trong thuật toán: trong $n+1$ số bất kỳ từ 1 tới $2n$, luôn có hai số mà số này chia hết số kia; hoặc trong một dãy n^2+1 số phân biệt luôn có dãy con đơn điệu độ dài $n+1$. Cách dùng thực dụng nhất: chứng minh rằng một cấu trúc lặp lại phải xuất hiện sau một số bước nhất định — đó chính là cơ sở của thuật toán tìm chu trình ở chương C/20.

5.3 Xác suất và kỳ vọng: công cụ mạnh nhất là công cụ đơn giản nhất

Nếu chỉ được giữ một công cụ xác suất cho thuật toán, đó là *kỳ vọng tuyến tính*: kỳ vọng của tổng bằng tổng các kỳ vọng, $E[X+Y] = E[X] + E[Y]$, **kể cả khi X và Y phụ thuộc nhau**. Câu cuối là chỗ làm nó mạnh, và cũng là chỗ người mới không tin.

Cách dùng chuẩn gồm ba bước: viết đại lượng cần tính thành *tổng của các biến chỉ báo*; tính kỳ vọng của từng biến chỉ báo, mà kỳ vọng đó chỉ là xác suất; rồi cộng lại. Ví dụ: số điểm bất động trung bình của một hoán vị ngẫu nhiên. Đặt $X_i = 1$ nếu phần tử i đứng đúng chỗ. Ta có $E[X_i] = 1/n$, và các X_i rõ ràng phụ thuộc nhau — nhưng không sao: $E[\sum X_i] = n \cdot (1/n) = 1$. Trung bình đúng một điểm bất động, bất kể n lớn thế nào. Tính trực tiếp bằng cách xét phân phối thì rắc rối hơn nhiều bậc.

Cùng kỹ thuật này giải thích chi phí trung bình của quicksort ngẫu nhiên: đặt biến chỉ báo cho sự kiện “phần tử thứ i và thứ j từng được so sánh với nhau”, tính xác suất của nó là $2/(|i-j|+1)$, rồi cộng — và tổng điều hoà xuất hiện, cho $\Theta(n \log n)$.



Hình 5.2. Quy trình ba bước của kỳ vọng tuyến tính. Điểm làm nó mạnh bất thường: phép cộng kỳ vọng không cần bất kỳ giả thiết độc lập nào, nên một đại lượng có phân phối rất rối vẫn tính được bằng cách chẻ thành nhiều câu hỏi có/không rất dễ.

Ba bất đẳng thức đuôi, xếp theo thứ tự cần thêm giả thiết và cho kết luận mạnh hơn: *Markov* chỉ cần biến không âm, cho chặn yếu; *Chebyshev* cần phương sai, cho chặn tốt hơn; *Chernoff* cần tổng các biến độc lập, và cho chặn giảm theo hàm mũ. Chernoff là lý do vì sao “lặp lại thuật toán ngẫu nhiên k lần rồi lấy đa số” làm xác suất sai giảm cực nhanh — nền của cả chương C/20.

CẦN NHỚ

Ba mẫu toán quay lại nhiều nhất trong 23 chương còn lại: (i) tổng cấp số nhân tăng $\leq$ hai lần số hạng lớn nhất; (ii) $H_n \approx \ln n$, tăng nhưng cực chậm; (iii) kỳ vọng tuyến tính đúng kể cả khi các biến phụ thuộc. Nếu chỉ mang theo ba thứ từ chương này, hãy mang ba thứ đó.

5.4 Số học modulo: vì sao mọi bài đếm đều lấy dư

Bài đếm hay cho kết quả khổng lồ — số cách xếp có thể là 2^{10^5} — nên đề bài yêu cầu kết quả theo modulo một số nguyên tố lớn, thường là $10^9 + 7$ hoặc 998 244 353. Đây không phải để làm khó mà để bài toán vẫn có ý nghĩa mà không cần số nguyên lớn.

Bốn điều cần nhớ. Cộng, trừ, nhân đều “đi xuyên qua” phép lấy dư: $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$. Phép chia thì không, và phải thay bằng nhân với *nghịch đảo modulo*, chỉ tồn tại khi số chia nguyên tố cùng nhau với m — đây là lý do người ta chọn m nguyên tố. Lũy thừa tính bằng bình phương liên tiếp, $O(\log n)$ phép nhân. Và cẩn thận tràn số: với $m \approx 10^9$, tích hai số dư đã cỡ 10^{18} , vừa sát giới hạn của số nguyên 64 bit — một chỗ sai kinh điển. Chi tiết ở chương [D/21](#).

5.5 Đại số rời rạc và trò chơi: hai mảnh nhỏ nhưng hay dùng

Lũy thừa ma trận biến các bài “tính trạng thái sau 10^{18} bước” thành khả thi, với điều kiện quan hệ truy hồi là *tuyến tính*: nếu $v_{k+1} = Av_k$ thì $v_n = A^n v_0$, và A^n tính được bằng bình phương liên tiếp trong $O(d^3 \log n)$. Đây là câu trả lời cho câu hỏi mở đầu thứ năm: cấu trúc làm bài giải được chính là tính tuyến tính. Cùng ý tưởng áp cho đếm đường đi độ dài k trong đồ thị, vì lũy thừa ma trận kể đếm đúng thứ đó.

Đại số trên $\mathbb{F}_2$ — cộng là phép XOR — cho phép giải hệ phương trình tuyến tính bằng các thao tác bit, rất nhanh. Ứng dụng hay gặp: tìm tập con có XOR lớn nhất, kiểm tra một số có biểu diễn được thành XOR của tập con hay không (dùng “cơ sở tuyến tính” của tập số).

Trò chơi tổ hợp. Với các trò chơi hai người, thông tin đầy đủ, không hoà, người không đi được thì thua: mỗi thế cờ là *thua* nếu mọi nước đi dẫn tới thế *thắng* cho đối thủ, và ngược lại. Với trò chơi tách được thành nhiều thành phần độc lập, định lý Sprague–Grundy nói giá trị của cả trò chơi là XOR các giá trị Grundy của từng phần — đó là lý do trò Nim giải bằng XOR các đồng. Điều đáng nhớ về mặt phương pháp: cách tìm ra quy luật này luôn là tính tay các thế nhỏ rồi tìm mẫu, đúng heuristic số một ở chương [A/01](#).

5.6 Gốc rễ: những mẫu toán này ra đời để giải bài gì

Điều thú vị là gần như mọi mảnh toán trong chương này đều ra đời từ một bài toán rất cụ thể, thường là rất trần tục, chứ không từ nhu cầu trừu tượng.

Xác suất bắt đầu từ cờ bạc: trao đổi thư giữa Pascal và Fermat năm 1654 về cách chia tiền cược khi một ván bị bỏ dở giữa chừng. Khái niệm kỳ vọng ra đời chính từ câu hỏi đó — “ván này đáng giá bao nhiêu nếu dừng bây giờ” — và đó vẫn là cách hiểu đúng nhất về kỳ vọng khi bạn dùng nó để phân tích thuật toán ngẫu nhiên.

Tổng điều hoà và hằng số γ đi kèm đến từ Euler, trong bối cảnh tính toán các chuỗi vô hạn thế kỷ 18. Hệ số nhị thức thì cổ hơn nhiều, xuất hiện độc lập ở nhiều nền toán học — Ấn Độ, Ba Tư, Trung Quốc — trước khi mang tên Pascal ở châu Âu; tam giác Pascal được biết tới ở Trung Quốc dưới tên tam giác Dương Huy từ thế kỷ 13.

Bất đẳng thức Chernoff, thứ hiện đại nhất trong danh sách, ra đời năm 1952 trong thống kê — bài toán là kiểm định giả thuyết, cần biết xác suất một tổng lớn các quan sát lệch xa kỳ vọng nhỏ tới đâu. Ngành thuật toán mượn nó vào thập niên 1980 khi thuật toán ngẫu nhiên trở nên phổ biến, và ngày nay nó có mặt ở khắp nơi từ băm (B/08) tới học máy.

Cuối cùng, một gốc rễ đáng nhớ vì nó nói lên cách toán và thuật toán tương tác: *Concrete Mathematics*[30] của Graham, Knuth và Patashnik ra đời từ chính chương “Toán học chuẩn bị” trong TAOCP. Knuth thấy rằng để phân tích thuật toán tới hằng số, ông cần một bộ công cụ toán mà các giáo trình toán thời đó không dạy theo cách dùng được — thiên về tổng, truy hồi và tiệm cận. Nói cách khác, nhu cầu phân tích thuật toán đã tạo ra một môn toán riêng, chứ không chỉ mượn.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1654 — Pascal, Fermat	Chia tiền cược thế nào khi ván bạc bị bỏ dở.	Khái niệm kỳ vọng; nền của toàn bộ phân tích thuật toán ngẫu nhiên (C/20).
Thế kỷ 13–17 — tam giác Dương Huy / Pascal	Đếm số cách chọn trong tổ hợp và trong khai triển nhị thức.	Đẳng thức Pascal — cũng chính là công thức truy hồi quy hoạch động đầu tiên trong lịch sử (C/16).
Thế kỷ 18 — Euler	Tính các chuỗi vô hạn và xấp xỉ tổng bằng tích phân.	$H_n \approx \ln n + \gamma$; kỹ thuật chặn tổng bằng tích phân, dùng suốt trong phân tích vòng lặp.
1730 — Stirling, de Moivre	Xấp xỉ $n!$ cho các bài toán xác suất với n lớn.	$\log(n!) = \Theta(n \log n)$ — cận dưới của sắp xếp so sánh dựa trực tiếp vào đây (D/24).
1952 — Chernoff	Kiểm định giả thuyết: xác suất một tổng lệch xa kỳ vọng.	Chặn đuôi giảm theo hàm mũ; lý do “lặp lại rồi lấy đa số” hiệu quả đến thế trong thuật toán ngẫu nhiên.
1989 — Graham, Knuth, Patashnik	Phân tích thuật toán tới hằng số cần bộ công cụ toán mà giáo trình thời đó không dạy theo cách dùng được.	<i>Concrete Mathematics</i> : tổng, truy hồi, tiệm cận trở thành một môn riêng sinh ra từ nhu cầu của ngành thuật toán.

5.7 Liên hệ ngang: cùng công cụ, khác miền

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thống kê và học máy	Bất đẳng thức tập trung, cận tổng quát hoá, biến chỉ báo	Cùng bộ công cụ đuôi (Markov, Chebyshev, Chernoff), chỉ khác câu hỏi: ở đây ta chặn xác suất thuật toán chạy chậm, ở đó chặn xác suất mô hình sai trên dữ liệu mới.
Vật lý thống kê	Hàm phân hoạch, đếm trạng thái vi mô	Cả hai đều đếm số cấu hình rồi lấy tổng có trọng số. Kỹ thuật “đếm bù” và bao hàm–loại trừ xuất hiện ở cả hai. Khác: vật lý quan tâm giới hạn nhiệt động, thuật toán quan tâm chi phí tính chính xác con số đó.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Lý thuyết thông tin	Entropy, cận dưới cho mã hoá	$\log_2(\text{số khả năng})$ là số bit tối thiểu; đây chính là dạng thông tin của cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh (D/24). Cùng một phép đếm, hai cách phát biểu.
Mật mã	Số học modulo, lũy thừa nhanh, nghịch đảo	Cùng công cụ ở D/21, nhưng đảo ngược mục tiêu: ở đây ta muốn tính nhanh, ở đó người ta dựa vào việc <i>bài toán ngược</i> không tính nhanh được.
Tài chính định lượng	Kỳ vọng và định giá theo giá trị kỳ vọng	Cùng khái niệm ra đời năm 1654 cho cùng loại câu hỏi “ván này đáng giá bao nhiêu”. Khác: tài chính phải quan tâm cả phương sai và rủi ro đuôi, còn phân tích thuật toán trung bình thường chỉ cần kỳ vọng — trừ khi có hạn kỳ, lúc đó đuôi mới là thứ quyết định.
Lý thuyết trò chơi kinh tế	Chiến lược tối ưu, cân bằng	Trò chơi tổ hợp ở mục 6 là trường hợp rất hẹp: hai người, thông tin đầy đủ, tổng bằng không. Kinh tế học tổng quát hơn nhiều nhưng mất đi lời giải sạch kiểu Sprague–Grundy.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì chúng có bản chất khác hẳn. $1 + 2 + 4 + \dots + n$ là cấp số nhân tăng, và mỗi số hạng lớn hơn tổng của tất cả số hạng trước cộng lại, nên tổng bị chặn bởi hai lần số hạng cuối — vẫn là $O(n)$. Còn $1 + 1/2 + \dots + 1/n$ giảm quá chậm: các số hạng không giảm theo cấp số nhân mà theo tỉ lệ nghịch, và tổng của chúng phân kỳ, chỉ có điều phân kỳ chậm như $\ln n$. Hình 5.1 cho thấy điều đó bằng diện tích.
- A2. Đúng vì kỳ vọng là một tổng có trọng số theo xác suất, và phép lấy tổng thì hoán đổi được với phép lấy tổng — không cần bất kỳ giả thiết độc lập nào. Mạnh vì nó tách một bài toán rối thành nhiều bài rất dễ: bạn không cần biết phân phối của tổng, chỉ cần biết xác suất của từng sự kiện nhỏ. Ví dụ số điểm bất động của hoán vị ngẫu nhiên: các biến chỉ báo phụ thuộc chẳng chặt, nhưng mỗi cái có kỳ vọng $1/n$, nên tổng bằng 1.
- A3. Vì kết quả thật thường lớn hơn mọi kiểu số nguyên có sẵn, mà bài toán lại không muốn ép bạn cài số lớn. Lấy dư giữ được cấu trúc cần thiết: cộng, trừ, nhân đều đi xuyên qua phép lấy dư. Chọn số nguyên tố vì khi đó mọi số khác 0 đều có nghịch đảo, nên phép chia — cần cho công thức có giai thừa — vẫn dùng được. Riêng 998 244 353 còn được chọn vì nó có căn nguyên thủy với bậc là lũy thừa lớn của 2, điều kiện để chạy biến đổi Fourier trên số nguyên (D/21).
- A4. Vì nó là một trong số ít công cụ chứng minh sự tồn tại mà không cần chỉ ra vật thể. Sức mạnh nằm ở việc *chọn chuồng* — bước sáng tạo thật sự. Với bài “trong $n + 1$ số từ 1 tới $2n$ luôn có hai số chia hết nhau”, chuồng là “phần lẻ lớn nhất của mỗi số”, và chỉ có n phần lẻ khả dĩ. Chọn được chuồng đúng thì kết luận thành hiển nhiên; toàn bộ độ khó nằm ở bước chọn đó.
- A5. Khi phép chuyển trạng thái là *tuyến tính*, tức trạng thái mới là tổ hợp tuyến tính của trạng thái cũ. Lúc đó n bước là phép nhân ma trận lũy thừa n , tính được bằng bình phương liên tiếp trong $O(\log n)$ phép nhân ma trận. Đó là lý do dãy Fibonacci sau 10^{18} bước tính được còn phần lớn quy hoạch động thì không — và cũng là gợi ý ngược: gấp n lên tới 10^{18} , hãy tìm cách viết truy hồi ở dạng tuyến tính.

NÓI LẠI THẬT ĐƠN GIẢN

Toán trong chương này không phải để chứng minh định lý, mà để trả lời bốn câu hỏi rất đời thường khi bạn viết chương trình.

Câu thứ nhất: “vòng lặp này chạy bao nhiêu lần?” Nếu mỗi lần bạn làm gấp đôi thì tổng công việc chỉ nhiều gấp đôi lần cuối cùng — giống như bạn gấp đôi số tiền tiết kiệm mỗi tháng, tổng tiền cả năm cũng chỉ hơn tháng cuối một chút. Còn nếu mỗi lần bạn chỉ làm bớt đi một phần rất nhỏ, tổng có thể lớn lên chậm mà không bao giờ dừng — đó là tổng điều hoà.

Câu thứ hai: “có bao nhiêu khả năng?” Bốn cách đếm ở mục 2 trả lời gần hết, và điều quan trọng nhất là biết mình đang đếm có thứ tự hay không, có lặp hay không.

Câu thứ ba: “trung bình thì mất bao lâu?” Mẹo duy nhất bạn thực sự cần là: đừng cố tính trung bình của cả cục. Hãy chế nó thành nhiều câu hỏi có/không nhỏ xíu, tính xác suất từng cái, rồi cộng. Cộng được kể cả khi chúng ảnh hưởng lẫn nhau — đó là điều bất ngờ và tiện nhất trong cả chương. Câu thứ tư: “số quá lớn thì làm sao?” Lấy dư. Cộng, trừ, nhân vẫn chạy bình thường sau khi lấy dư; chỉ có phép chia là phải làm kiểu khác.

Một câu: Toán ở đây không phải để chứng minh định lý, mà để trả lời “bao nhiêu lần”, “bao nhiêu cách”, “trung bình bao lâu” và “số lớn quá thì sao”.

Chỗ không rút gọn được: Chọn đúng “chuồng” cho nguyên lý chuồng bồ câu, hay chọn đúng biến chỉ báo, vẫn là bước sáng tạo không có công thức.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Chặn cấp số nhân tăng	Tổng $\leq$ hai lần số hạng lớn nhất. Giải thích khẩu hao của mảng động, chi phí xây cây phân đoạn, kỹ thuật nhân đôi.
••• Tổng điều hoà	$H_n \approx \ln n$: phân kỳ nhưng cực chậm. Xuất hiện ở sàng nguyên tố, quicksort ngẫu nhiên, bài sưu tập đủ bộ.
••• Kỳ vọng tuyến tính	$E[\sum X_i] = \sum E[X_i]$ kể cả khi phụ thuộc. Quy trình: viết thành tổng biến chỉ báo $\rightarrow$ mỗi kỳ vọng là một xác suất $\rightarrow$ cộng.
••• Bốn công thức đếm nền	Có thứ tự không lặp $n!/(n-k)!$; không thứ tự không lặp $\binom{n}{k}$; có lặp có thứ tự n^k ; có lặp không thứ tự $\binom{n+k-1}{k}$.
••• Bốn quy tắc modulo	Cộng/trừ/nhân đi xuyên qua phép lấy dư; chia phải dùng nghịch đảo; lũy thừa bằng bình phương liên tiếp; coi chừng tràn khi nhân hai số cỡ 10^9 .
•• Đẳng thức Pascal	$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ — “phần tử thứ n được chọn hoặc không”; khuôn mẫu của mọi lập luận chia trường hợp trong quy hoạch động.
•• Dãy 1, 2, 5, 14, 42	Số Catalan — dãy ngoặc đúng, cây nhị phân, chia đa giác thành tam giác. Gặp dãy này khi tính tay thì bài có cấu trúc Catalan.
•• Ba bất đẳng thức đuôi	Markov (chỉ cần không âm, yếu nhất); Chebyshev (cần phương sai); Chernoff (cần độc lập, chặn giảm theo hàm mũ).
•• Khi nào lũy thừa ma trận dùng được	Khi truy hồi tuyến tính; cho $O(d^3 \log n)$ với n tới 10^{18} . Cũng dùng để đếm đường đi độ dài k trong đồ thị.
•• Stirling và hệ quả	$\log_2(n!) = \Theta(n \log n)$ — cơ sở trực tiếp của cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh.
•• Nguyên lý chuồng bồ câu	Chứng minh tồn tại mà không cần chỉ ra vật thể; toàn bộ độ khó nằm ở bước chọn chuồng.
• Sprague–Grundy	Trò chơi tách thành phần độc lập: giá trị chung là XOR các giá trị Grundy; Nim là trường hợp riêng.
• Cơ sở tuyến tính trên $\mathbb{F}_2$	Cho phép hỏi “số này có là XOR của một tập con không” và “XOR lớn nhất là bao nhiêu” trong $O(n \log C)$.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao phân tích quicksort ngẫu nhiên lại ra tổng điều hoà?

| Vì ta đếm bằng biến chỉ báo cho sự kiện “hai phần tử ở vị trí i và j trong thứ tự đã sắp có được so sánh với nhau hay không”. Chúng chỉ được so sánh nếu một trong hai được chọn làm chốt trước mọi phần tử nằm giữa chúng, xác suất là $2/(j-i+1)$. Cộng theo mọi cặp thì mỗi khoảng cách d đóng

góp cỡ $n \cdot 2/(d+1)$, và tổng theo d chính là tổng điều hoà — cho $2n \ln n$. Đây là ví dụ mẫu mực cho quy trình ba bước ở mục 3.

2. Khi nào tôi nên đếm bù thay vì đếm trực tiếp?

| Khi điều kiện “tốt” có dạng “không có cái nào vi phạm” còn điều kiện “xấu” dễ mô tả. Ví dụ đếm hoán vị không có điểm bất động thì khó, nhưng đếm hoán vị có ít nhất một điểm bất động lại làm được bằng bao hàm–loại trừ. Dấu hiệu nhận biết trên đề: có chữ “không”, “ít nhất một”, “khác nhau đôi một”.

3. Tôi luôn nhầm giữa $\binom{n+k-1}{k}$ và $\binom{n}{k}$. Có cách nhớ nào không?

| Hãy nhớ mô hình vật lý thay vì công thức. Chia k viên kẹo giống nhau cho n đứa trẻ: xếp k viên kẹo thành hàng và cắm $n-1$ vách ngăn vào; mỗi cách sắp xếp k viên và $n-1$ vách là một cách chia, tổng cộng $\binom{n+k-1}{k}$ cách. Còn $\binom{n}{k}$ là chọn k đứa trẻ trong n đứa. Hai bài khác nhau ở chỗ vật được chia có phân biệt hay không.

4. Chernoff và Chebyshev khác nhau trên thực tế bao nhiêu?

| Rất nhiều, và khoảng cách lớn dần theo số phép thử. Chebyshev cho xác suất lệch giảm theo $1/t^2$; Chernoff cho giảm theo e^{-t} . Với 100 lần tung đồng xu, hỏi xác suất được từ 70 mặt ngửa trở lên: Chebyshev cho một chặn lỏng cỡ vài phần trăm, Chernoff cho chặn nhỏ hơn nhiều bậc. Cái giá là Chernoff đòi các biến độc lập, còn Chebyshev chỉ cần phương sai — nên khi có phụ thuộc thì Chebyshev vẫn dùng được còn Chernoff thì không.

5. Tại sao $10^9 + 7$ lại phổ biến đến thế?

| Ba lý do thực dụng. Nó là số nguyên tố, nên mọi số khác 0 có nghịch đảo. Nó nhỏ hơn 2^{30} , nên tích hai phần dư nhỏ hơn 2^{60} , vừa trong số nguyên 64 bit có dấu. Và nó đủ lớn để hai kết quả khác nhau hiếm khi trùng phần dư một cách tình cờ. Còn 998 244 353 được chọn khi bài cần biến đổi Fourier trên số nguyên, vì $998\,244\,353 = 119 \cdot 2^{23} + 1$ có căn nguyên thủy bậc 2^{23} .

6. Nếu tôi kém toán, tôi có bị chặn trần trong việc luyện thuật toán không?

| Không bị chặn ở phần lớn nội dung. Cấu trúc dữ liệu, đồ thị, quy hoạch động, tham lam — chiếm phần lớn bài tập và phỏng vấn — chỉ cần đúng những gì trong chương này, mà phần lõi chỉ có ba mẫu ở khối *Cần nhớ*. Toán trở thành rào cản thật sự ở ba nơi hẹp: số học nâng cao (D/21), một số bài đếm tổ hợp khó, và phân tích thuật toán ngẫu nhiên tinh vi. Chiến lược hợp lý là học ba mẫu lõi thật chắc rồi bổ sung theo nhu cầu, chứ không học toán trước rồi mới bắt đầu.

7. Cách nhanh nhất để nhận ra một dãy số quen thuộc khi tính tay ví dụ nhỏ?

| Tính năm tới sáu số hạng đầu rồi so với vài dãy thường gặp: $2^n; n!; \text{Fibonacci } 1,1,2,3,5,8; \text{Catalan } 1,1,2,5,14,42; \text{số Stirling}; \text{số Bell } 1,1,2,5,15,52$ (chú ý dễ nhầm với Catalan ở ba số đầu). Nếu không khớp, hãy thử tra trong từ điển dãy số nguyên trực tuyến. Nhưng nhớ: nhận ra dãy mới là gợi ý, vẫn phải chứng minh vì sao bài của bạn cho ra dãy đó.

8. Có nên học hàm sinh không?

| Chỉ khi bạn đi sâu vào các bài đếm. Hàm sinh là công cụ đẹp và mạnh cho việc giải truy hồi và đếm cấu trúc, nhưng trong phần lớn bài tập thực tế thì cây đệ quy và quy hoạch động đã đủ. Xếp nó vào loại “biết là có, học khi gặp bài thật cần” — và khi đó *Concrete Mathematics*[30] là nguồn đúng.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Tính tổng $\sum_{i=1}^n i, \sum 2^i, \sum 1/i$ và xác định bậc của từng cái	Tự luyện	Mục tiêu là phản xạ: nhìn vòng lặp ra ngay bậc, không phải nhớ công thức.
• Tính $\binom{n}{k} \bmod p$ cho n tới 10^6 bằng bảng giai thừa và nghịch đảo	Chương D/21	Tiền xử lý giai thừa và nghịch đảo giai thừa một lần; mỗi truy vấn $O(1)$.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tính số điểm bất động trung bình của hoán vị ngẫu nhiên bằng mô phỏng, so với lý thuyết	Thực nghiệm	Kết quả bằng 1 với mọi n — hãy tự kiểm chứng rồi giải thích bằng kỳ vọng tuyến tính.
•• Đếm số dãy ngoặc đúng độ dài $2n$ bằng hai cách: quy hoạch động và công thức Catalan	AtCoder DP, CSES[33]	Hai cách phải khớp; nếu lệch, gần như luôn do quên trường hợp biên $n = 0$.
•• Tính Fibonacci thứ 10^{18} theo modulo bằng lũy thừa ma trận	Bài tập kinh điển	Chú ý thứ tự nhân ma trận và tràn số khi nhân hai phần tử.
•• Chứng minh trong $n + 1$ số bất kỳ từ 1 tới $2n$ luôn có hai số mà số này chia hết số kia	Bài tập kinh điển	Chuồng là “phần lẻ lớn nhất”; toàn bộ bài nằm ở bước chọn chuồng.
•• Bài sưu tập đủ bộ: cần trung bình bao nhiêu lần mở gói để có đủ n loại thẻ	Bài tập kinh điển	Chia thành n giai đoạn, mỗi giai đoạn là biến hình học; tổng cho nH_n .
••• Chứng minh quicksort ngẫu nhiên có kỳ vọng $2n \ln n$ so sánh	[40]	Biến chỉ báo cho từng cặp, xác suất $2/(j - i + 1)$; đây là bài tập kinh điển nhất về kỳ vọng tuyến tính.
••• Dựng cơ sở tuyến tính trên $\mathbb{F}_2$ và tìm XOR lớn nhất của một tập con	Bài tập thi đấu kinh điển	Khử Gauss trên bit; sau khi có cơ sở, thuật toán tham lam từ bit cao xuống bit thấp.
••• Tính giá trị Grundy cho một trò chơi lấy sỏi có luật lạ, tìm quy luật	Bài tập kinh điển	Tính tay các thế nhỏ, lập bảng, tìm chu kỳ — đúng quy trình heuristic số một ở A/01.
••• Dùng bao hàm–loại trừ đếm số hoán vị không có điểm bất động (số mất trật tự)	Bài tập kinh điển	Kết quả xấp xỉ $n!/e$; một kết quả đẹp và bất ngờ đáng tự dẫn ra.

Kết chương và đọc tiếp

Chương này khép lại phần A. Nếu ba chương trước cho bạn quy trình, thước chi phí và thước tính đúng, thì chương này là cái hộp dụng cụ đặt cạnh bàn — không cần thuộc hết, chỉ cần biết trong đó có gì và mở đúng ngăn khi cần. Ba mẫu quay lại nhiều nhất, nhắc lại lần cuối: cấp số nhân bị chặn bởi hai lần số hạng lớn nhất, tổng điều hoà xấp xỉ $\ln n$, và kỳ vọng tuyến tính không cần độc lập.

Từ đây, phần B đổi câu hỏi. Bốn chương vừa rồi hỏi “tôi nghĩ thế nào, và làm sao biết mình đúng”. Bảy chương tiếp theo hỏi một câu rất khác: *dữ liệu nên nằm ở dạng nào để thao tác mà tôi lặp lại nhiều nhất trở nên rẻ*. Bắt đầu bằng thứ đơn giản nhất và cũng bị đánh giá thấp nhất — mảng, cùng những kỹ thuật hai con trở biến $O(n^2)$ thành $O(n)$ (B/06).

CẤU TRÚC DỮ LIỆU

TÓM TẮT

Phần B đi qua bảy nhóm cấu trúc dữ liệu, nhưng chia theo *thao tác được làm rõ đi* chứ không theo hình dạng. Mảng và hai con trỏ làm rõ việc quét có cửa sổ; ngăn xếp, hàng đợi và heap làm rõ việc lấy phần tử “đúng lúc”; bảng băm làm rõ tra cứu theo khoá; cây tìm kiếm làm rõ tra cứu theo thứ tự; cây chỉ số làm rõ truy vấn trên đoạn kèm cập nhật; DSU làm rõ việc hợp nhất; trie và các cấu trúc chuỗi làm rõ việc so khớp. Đọc xong phần này, khi nhìn một đề bài bạn sẽ hỏi “thao tác nào lặp lại nhiều nhất” thay vì “bài này dùng cây gì”. Nếu chỉ nhớ một điều: cấu trúc dữ liệu là một cách trả trước chi phí — bạn bỏ công sắp đặt trước để mỗi truy vấn sau rõ đi, và mọi cấu trúc đều có một thao tác mà nó làm *tệ hơn* mảng thường.

Có một cách nhìn thống nhất cho cả phần này, và nó đáng nói ngay từ đầu. Mọi cấu trúc dữ liệu đều là một sự đánh đổi giữa *trật tự bạn duy trì* và *giá bạn trả để duy trì nó*. Mảng thô không duy trì trật tự nào nên chèn rõ mà tìm đất. Cây tìm kiếm duy trì trật tự toàn phần nên tìm rõ mà mỗi lần chèn phải trả tiền giữ cân bằng. Heap duy trì một trật tự yếu hơn — chỉ cần biết ai nhỏ nhất — nên rõ hơn cây, đổi lại không trả lời được câu “phần tử thứ k là ai”. Khi bạn đọc mỗi chương, hãy tự hỏi cấu trúc đó duy trì *đúng bao nhiêu* trật tự, vì đó là thứ quyết định nó nhanh ở đâu và mù ở đâu.

Thứ tự bảy chương không phải ngẫu nhiên. Chương 6 và 7 là những cấu trúc mà bạn dùng gần như mọi ngày, và chúng cũng là nơi các kỹ thuật “ngây thơ nhưng đúng” như hai con trỏ hay ngăn xếp đơn điệu biến $O(n^2)$ thành $O(n)$. Chương 8 và 9 là hai cách tra cứu đối lập — băm cho tra cứu chính xác, cây cho tra cứu theo thứ tự — và ranh giới giữa chúng là câu hỏi “bài có cần thứ tự không”. Chương 10 và 11 là hai cấu trúc mà người luyện thi đấu dùng nhiều nhất vì tỉ lệ sức mạnh trên độ dài code rất cao. Chương 12 khép lại bằng chuỗi, miễn mà cấu trúc dữ liệu chuyên biệt tạo ra khác biệt lớn nhất.

Bản đồ phần B — Cấu trúc dữ liệu

TÓM TẮT

Bảy chương của phần B được xếp theo thao tác mà chúng làm rõ đi, và bản đồ này cho bạn bảng tra ngược: gặp thao tác nào thì mở chương nào. Chuỗi phụ thuộc bắt buộc là $6 \rightarrow 7 \rightarrow 9 \rightarrow 10$; chương 8 (băm) và 11 (DSU) độc lập, đọc lúc nào cũng được; chương 12 cần chương 8 cho hashing chuỗi. Nếu chỉ nhớ một điều: chọn cấu trúc bằng cách liệt kê thao tác và tần suất, không bằng cách nhớ tên.

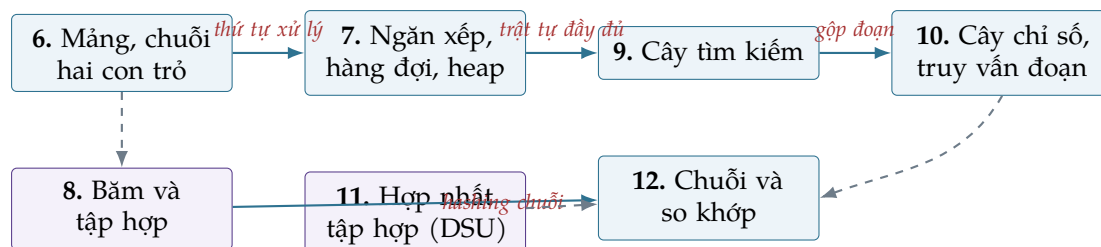
A. Bảng tra ngược: thao tác $\rightarrow$ chương

Thao tác lặp lại nhiều	Chương	Cấu trúc điển hình
Quét cửa sổ, tổng đoạn tĩnh	6	hai con trỏ, tổng tiền tố
Lấy phần tử nhỏ nhất liên tục	7	heap / hàng đợi ưu tiên
“Phần tử lớn hơn kế tiếp”	7	ngăn xếp đơn điệu
Tra cứu chính xác theo khoá	8	bảng băm
Tra cứu theo thứ tự, cận trên/dưới	9	cây tìm kiếm cân bằng
Truy vấn đoạn kèm cập nhật điểm	10	Fenwick / segment tree
Truy vấn đoạn trên mảng tĩnh	10	sparse table
Hợp nhất hai nhóm, hỏi cùng nhóm	11	DSU
So khớp mẫu, tiền tố chung	12	trie, KMP, Z, Aho–Corasick

B. Vì sao chia như vậy

Chia theo hình dạng — “mảng, danh sách liên kết, cây, đồ thị” — là cách chia của người đã biết đáp án. Người đang giải bài không nhìn thấy hình dạng, họ nhìn thấy thao tác: “tôi phải hỏi tổng đoạn này một triệu lần và thỉnh thoảng sửa một phần tử”. Bảng tra ở trên chính là cầu nối từ thao tác về cấu trúc, và nó là thứ đáng thuộc nhất của cả phần B.

C. Phụ thuộc và thứ tự đọc



Hình 5.3. Phụ thuộc giữa bảy chương của phần B. Hai chương tô tím độc lập với mạch chính; chương 12 là nơi hội tụ của băm, DSU và cây chỉ số.

D. Cốt lõi hay tham khảo

Cốt lõi: hai con trỏ và tổng tiền tố (6); ngăn xếp đơn điệu và heap (7); băm với khoá ngẫu nhiên hoá (8); Fenwick và segment tree cơ bản kèm lazy (10); DSU với hai tối ưu (11); KMP, Z và hashing chuỗi (12). **Tham khảo:** cài đặt chi tiết cây đỏ–đen và treap (9); segment tree beats và cây bền vững (10); suffix automaton (12).

Đọc bằng ví dụ: chương B/06 có vết chạy cửa sổ và phân ví dụ số âm; chương B/10 vẽ các khối lowbit và kiểm thử tự thành lazy.

Mảng, chuỗi và hai con trỏ (Arrays, Strings, Two Pointers)

PHỤ THUỘC A/03 cho ngân sách chỉ phí và phân cấp bộ nhớ; A/04 cho cách viết bất biến — chương này là nơi bất biến trả công nhiều nhất.

TÓM TẮT

Mảng là cấu trúc dữ liệu bị đánh giá thấp nhất: nó không có “thuật toán” nào hào nhoáng, nhưng nó thắng gần như mọi cấu trúc khác về hằng số và về tính thân thiện với cache. Chương này đi qua bốn kỹ thuật biến $O(n^2)$ thành $O(n)$ mà không cần thêm cấu trúc nào: tổng tiền tố, mảng hiệu, hai con trỏ, và cửa sổ trượt. Điểm chung của cả bốn là một ý duy nhất — *đừng tính lại từ đầu, hãy cập nhật phần thay đổi*. Nếu chỉ nhớ một điều: khi thấy hai vòng lặp lồng nhau, hãy hỏi “khi vòng ngoài tiến một bước, vòng trong có thật sự phải quay lại từ đầu không”; câu trả lời thường là không, và đó là toàn bộ chương này.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mảng (array)** — dãy phần tử cùng kiểu nằm liên tiếp trong bộ nhớ; truy cập phần tử thứ i tốn $O(1)$ bằng phép tính địa chỉ.
- **tổng tiền tố (prefix sum)** — mảng P với $P_i = a_1 + \dots + a_i$; cho phép lấy tổng đoạn $[l, r]$ bằng $P_r - P_{l-1}$ trong $O(1)$.
- **mảng hiệu (difference array)** — mảng D với $D_i = a_i - a_{i-1}$; cho phép cộng một hằng số vào cả đoạn trong $O(1)$, rồi khôi phục mảng gốc bằng tổng tiền tố.
- **hai con trỏ (two pointers)** — kỹ thuật duy trì hai chỉ số cùng quét qua dữ liệu, mỗi chỉ số chỉ đi tiến, cho tổng chỉ phí $O(n)$ thay vì $O(n^2)$.
- **cửa sổ trượt (sliding window)** — trường hợp riêng của hai con trỏ: duy trì một đoạn liên tiếp thoả điều kiện, mở rộng bên phải và co bên trái.
- **tính đơn điệu** — tính chất “nếu đoạn $[l, r]$ thoả thì mọi đoạn con của nó cũng thoả” (hoặc chiều ngược lại). Đây là điều kiện sống còn để hai con trỏ đúng.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao mảng — cấu trúc thô sơ nhất — lại thường thắng những cấu trúc tinh vi hơn trên máy thật, dù độ phức tạp tiệm cận tệ hơn?
- Q2. Hai con trỏ chạy $O(n)$ dù trông như hai vòng lặp lồng nhau. Lập luận nào cho ta biết nó thật sự tuyến tính?
- Q3. Kỹ thuật hai con trỏ hỏng khi nào? Cho một bài mà áp dụng nó sẽ cho kết quả sai.
- Q4. Tổng tiền tố và mảng hiệu là hai phép toán ngược nhau. Việc nhận ra điều đó cho bạn thêm kỹ thuật gì?
- Q5. Vì sao “sắp xếp trước” lại là nước đi mở đầu hiệu quả đến vậy, dù bản thân nó tốn $O(n \log n)$?

Có một hiện tượng đáng chú ý trong lập trình thi đấu: rất nhiều bài được gắn nhãn khó lại có lời giải chỉ gồm một vòng lặp và hai biến chỉ số. Không cấu trúc dữ liệu, không đệ quy, không công

thức. Người chưa quen nhìn lời giải đó thấy nó như trò ảo thuật — rõ ràng bài đòi xét mọi đoạn con, mà đây chỉ quét một lượt.

Không có ảo thuật nào. Cả chương này chỉ khai thác một quan sát duy nhất, chính là câu hỏi số 10 trong danh sách heuristic ở chương A/01: *cái gì thực sự thay đổi giữa hai bước liên tiếp?* Nếu đáp án của bước $i + 1$ chỉ khác đáp án của bước i một chút, thì tính lại từ đầu là lãng phí, và toàn bộ chi phí sụp từ $O(n^2)$ xuống $O(n)$. Bốn kỹ thuật trong chương — tổng tiền tố, mảng hiệu, hai con trỏ, cửa sổ trượt — là bốn cách hiện thực hoá quan sát đó cho bốn tình huống khác nhau.

6.1 Vì sao bắt đầu bằng mảng

Trước khi vào kỹ thuật, cần một lời biện hộ cho mảng, vì người mới thường vội bỏ qua nó để đi học cấu trúc “xịn” hơn.

Mảng có ba ưu thế mà không cấu trúc nào lấy lại được. Thứ nhất, truy cập phần tử là một phép tính địa chỉ, không có con trỏ để lần theo. Thứ hai, các phần tử nằm liên tiếp trong bộ nhớ, nên khi bạn đọc phần tử i thì phần cứng đã kéo sẵn cả một khối chứa $i + 1, i + 2, \dots$ vào cache — duyệt tuần tự gần như miễn phí sau lần đọc đầu. Thứ ba, không có chi phí cấp phát cho từng phần tử.

Hệ quả thực hành mạnh hơn nhiều người nghĩ, và nó nổi thẳng với hình phân cấp bộ nhớ ở chương A/03: một thuật toán $O(n \log n)$ nhảy ngẫu nhiên khắp bộ nhớ có thể thua một thuật toán $O(n^2)$ quét tuần tự, ở những kích thước n rất thực tế. Đây là lý do các cấu trúc dữ liệu “cây” trong thực chiến thường được cài bằng mảng phẳng thay vì bằng con trỏ (B/10), và là lý do khi tối ưu hàng số thì việc đầu tiên nên thử là đổi cấu trúc có con trỏ sang mảng.

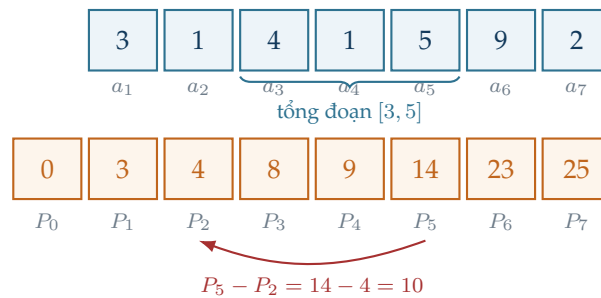
CẠM BÃY

Cạm bẫy ngược lại cũng có thật: mảng đắt ở phép chèn và xoá giữa mảng, vì phải dịch chuyển toàn bộ phần đuôi. Nếu bài của bạn chèn và xoá liên tục ở giữa, mảng là lựa chọn sai và bạn cần cấu trúc ở chương B/09 hoặc B/10. Quy tắc: mảng thắng khi *cấu trúc dữ liệu tĩnh, truy vấn nhiều*; nó thua khi *cấu trúc thay đổi liên tục ở giữa*.

6.2 Tổng tiền tố: trả trước một lượt quét, sau đó mọi truy vấn miễn phí

Bài toán mở đầu: cho mảng và q truy vấn, mỗi truy vấn hỏi tổng của một đoạn $[l, r]$. Cách ngây thơ tốn $O(n)$ mỗi truy vấn, tổng $O(nq)$. Với $n = q = 10^5$ thì đó là 10^{10} phép toán — quá xa ngân sách ở chương A/03.

Ý tưởng: tính trước mảng P với $P_0 = 0$ và $P_i = P_{i-1} + a_i$. Khi đó tổng đoạn $[l, r]$ bằng $P_r - P_{l-1}$. Chi phí: $O(n)$ một lần, rồi $O(1)$ mỗi truy vấn.



Hình 6.1. Tổng tiền tố. Hàng trên là mảng gốc, hàng dưới là tổng tiền tố với một ô đệm $P_0 = 0$ — ô đệm này loại bỏ mọi trường hợp riêng khi $l = 1$, và bỏ nó đi là nguồn lỗi lệch một đơn vị phổ biến nhất của kỹ thuật này.

Ý tưởng mở rộng theo ba hướng và cả ba đều hay gặp. *Hai chiều*: với ma trận, $P_{i,j}$ là tổng của hình chữ nhật từ góc trên trái tới (i, j) ; tổng của một hình chữ nhật bất kỳ tính bằng bốn giá trị theo nguyên lý bao hàm–loại trừ (A/05). Trong thị giác máy tính, cấu trúc này có tên riêng là *ảnh tích phân* và nó là thứ khiến bộ phát hiện khuôn mặt Viola–Jones chạy được thời gian thực năm 2001. *Phép toán khác tổng*: XOR dùng được vì có phép ngược; tích thì phải cẩn thận với số 0; còn cực đại thì không dùng được vì không có phép ngược — đó chính là lúc bạn cần sparse table hoặc cây phân đoạn (B/10). *Tổng tiền tố theo điều kiện*: đếm số phần tử thoả tính chất nào đó trong đoạn, bằng cách xây tổng tiền tố trên mảng chỉ báo 0/1.

Một mẹo hệ quả đáng nhớ: bài “đếm số đoạn con có tổng bằng k ” giải bằng tổng tiền tố cộng bảng băm — vì tổng đoạn $[l, r] = k$ tương đương $P_r - P_{l-1} = k$, tức là với mỗi r ta chỉ cần đếm xem có bao nhiêu P trước đó bằng $P_r - k$. Cách nghĩ này — *biến điều kiện trên đoạn thành điều kiện trên hai điểm đầu mút* — là một mẫu dùng lại rất nhiều lần.

6.3 Mảng hiệu: phép ngược của tổng tiền tố

Nếu tổng tiền tố giải bài “cập nhật ít, truy vấn đoạn nhiều”, thì mảng hiệu giải bài đối xứng: “cập nhật đoạn nhiều, đọc kết quả một lần ở cuối”.

Cách làm: để cộng v vào mọi phần tử trong đoạn $[l, r]$, ta chỉ cần $D_l += v$ và $D_{r+1} -= v$. Mỗi cập nhật tốn $O(1)$ thay vì $O(r - l + 1)$. Cuối cùng, lấy tổng tiền tố của D để khôi phục mảng thật.

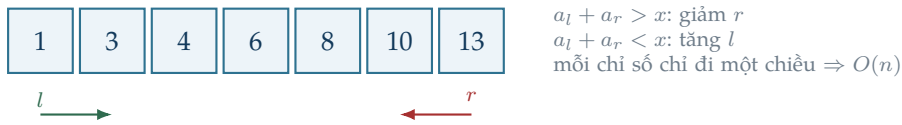
Nhận ra hai kỹ thuật này là nghịch đảo của nhau cho bạn một mẫu tư duy tổng quát: nếu một phép toán tốn kém khi làm trực tiếp, hãy tìm một biểu diễn khác của cùng dữ liệu mà trong đó phép toán ấy rẻ, rồi chuyển đổi một lần ở cuối. Cùng ý tưởng xuất hiện lại ở phép biến đổi Fourier (D/21), nơi phép nhân chập tốn $O(n^2)$ trong miền thời gian trở thành phép nhân từng điểm $O(n)$ trong miền tần số.

6.4 Hai con trỏ: vì sao hai vòng lặp lồng nhau lại tuyến tính

Bài toán mẫu: mảng đã sắp tăng, tìm cặp có tổng bằng x . Vét cạn xét mọi cặp, $O(n^2)$. Kỹ thuật hai con trỏ: đặt l ở đầu, r ở cuối. Nếu $a_l + a_r > x$ thì giảm r ; nếu nhỏ hơn thì tăng l ; nếu bằng thì tìm thấy.

Vì sao đúng? Bất biến là mẫu chốt, và đây là một bất biến đẹp: mọi cặp đã bị loại đều chắc chắn không phải đáp án. Khi $a_l + a_r > x$, mọi cặp gồm a_r với một phần tử ở vị trí $\geq l$ đều có tổng $\geq a_l + a_r > x$, nên loại r là hợp lệ. Vì sao tuyến tính? Vì l chỉ tăng và r chỉ giảm, mỗi chỉ số đi tối đa n bước, tổng

cộng tối đa $2n$ bước — lập luận này được gọi là *tính chi phí bằng tổng quãng đường của con trỏ*, và nó dùng lại cho mọi biến thể.



Hình 6.2. Hai con trỏ trên mảng đã sắp. Điều kiện sống còn là *tính đơn điệu*: khi r giảm thì tổng chỉ có thể giảm, khi l tăng thì tổng chỉ có thể tăng. Mất tính đơn điệu — ví dụ khi mảng chưa sắp, hoặc khi có số âm trong bài cửa sổ trượt về tổng — thì kỹ thuật này cho kết quả sai chứ không phải chậm.

Biến thể quan trọng nhất là *cửa sổ trượt*: tìm đoạn liên tiếp dài nhất (hoặc ngắn nhất) thỏa một điều kiện. Khuôn chung: mở rộng r từng bước; sau mỗi lần mở rộng, nếu điều kiện bị vi phạm thì co l cho tới khi hợp lệ trở lại; cập nhật đáp án. Điều kiện để khuôn này đúng là tính đơn điệu theo nghĩa: nếu đoạn $[l, r]$ vi phạm thì mọi đoạn chứa nó cũng vi phạm.

Đây chính là chỗ hỏng thường gặp. Bài “đoạn con dài nhất có tổng $\leq S$ ” giải được bằng cửa sổ trượt nếu mọi số đều không âm; có số âm thì tổng không còn đơn điệu theo độ dài, và cửa sổ trượt cho đáp án sai. Bài đó phải chuyển sang tổng tiền tố cộng cấu trúc tìm kiếm. Khi gặp một bài cửa sổ trượt, câu hỏi đầu tiên luôn phải là: *tính chất này có đơn điệu không?*

CẦN NHỚ

Ba câu hỏi trước khi dùng hai con trỏ: (i) dữ liệu có thứ tự cần thiết chưa — nếu chưa, có sắp được không; (ii) tính chất có đơn điệu không — có lại thì luôn “dễ thỏa hơn”, mở ra thì luôn “khó thỏa hơn”; (iii) mỗi con trỏ có chỉ đi một chiều không. Thiếu bất kỳ điều nào thì kết quả sai, và cái sai đó *im lặng* — chương trình vẫn chạy, chỉ ra số khác.

6.5 Sắp xếp như một nước đi mở đầu

Một mẫu xuất hiện liên tục: rất nhiều bài trở nên dễ sau khi sắp xếp. Điều này đáng được nói thành nguyên tắc, vì nó là heuristic có tỉ lệ thành công cao và chi phí thấp — $O(n \log n)$ gần như luôn nằm trong ngân sách.

Sắp xếp có tác dụng vì nó tạo ra thứ mà ba kỹ thuật trên cần: *tính đơn điệu*. Sau khi sắp, hai con trỏ dùng được, tìm kiếm nhị phân dùng được, tham lam thường lộ ra, và các phần tử giống nhau nằm cạnh nhau nên đếm bội số thành chuyện tầm thường.

Một biến thể đáng nhớ là *sắp xếp theo tiêu chí do bạn thiết kế*. Ví dụ với các đoạn thẳng, sắp theo điểm kết thúc cho bài xếp lịch tham lam (C/15); sắp theo điểm bắt đầu cho bài gộp đoạn giao nhau; sắp theo hiệu của hai đại lượng cho các bài đánh đổi. Việc chọn khóa sắp xếp thường chính là toàn bộ lời giải, và cách kiểm tra nó đúng là lập luận đổi chỗ ở chương A/04.

6.6 Chuỗi: mảng có tính chất riêng

Chuỗi là mảng ký tự, nên mọi kỹ thuật trên đều áp dụng được, nhưng có ba điểm riêng đáng nhớ ngay từ đây.

Thứ nhất, *so sánh hai chuỗi không phải $O(1)$ mà là $O(\text{độ dài})$* — điều mà chương A/03 đã cảnh báo. Dùng chuỗi làm khóa của cấu trúc tìm kiếm thì nhân thêm một thừa số vào độ phức tạp.

Thứ hai, bảng chữ cái thường nhỏ, và điều đó mở ra kỹ thuật đếm bằng mảng cố định 26 hoặc 128 phần tử thay vì bảng băm — nhanh hơn nhiều lần. Rất nhiều bài chuỗi trong phỏng vấn (đếm anagram, cửa sổ chứa đủ ký tự) là bài cửa sổ trượt với một mảng đếm 26 phần tử.

Thứ ba, các bài so khớp mẫu có cấu trúc đặc thù và cần công cụ riêng — hàm tiền tố, hàm Z, băm chuỗi — toàn bộ là nội dung chương B/12.

6.7 Nhìn cửa sổ di chuyển, rồi chủ động làm nó hỏng

Với $a = [2, 1, 3, 2]$ và $S = 4$, ta tìm đoạn dài nhất có tổng không vượt S . Hình 6.3 cho thấy vì sao một bước mở rộng có thể kéo theo nhiều bước co. Chỉ sau khi co xong mới cập nhật đáp án. Tổng số bước co vẫn không quá n , vì đầu trái không quay lại.

[1, 1]	2	1	3	2	tổng = 2	
[1, 2]	2	1	3	2	tổng = 3	
[1, 3]	2	1	3	2	tổng = 6	vi phạm: bỏ 2 rồi bỏ 1
[3, 3]	2	1	3	2	tổng = 3	
[3, 4]	2	1	3	2	tổng = 5	vi phạm: bỏ 3
[4, 4]	2	1	3	2	tổng = 2	

Hình 6.3. Vết chạy cửa sổ trượt với chỉ số từ 1. Ô xanh là cửa sổ hiện tại, ô trắng nằm ngoài. Hai hàng tổng 6 và 5 là trạng thái tạm thời chưa hợp lệ; đáp án tốt nhất là độ dài 2 ở đoạn [1, 2].

Để kiểm tra giả thiết, đổi dữ liệu thành $[5, -2]$, vẫn lấy $S = 4$. Khuôn vừa dùng loại 5 ngay ở bước đầu, nên không thể tìm lại đoạn dài 2 có tổng 3. Đây là phản ví dụ đủ nhỏ để nhớ: số âm có thể cứu một cửa sổ từng vi phạm. Đừng sắp xếp để chữa bài này vì sắp xếp phá tính liên tiếp của đoạn gốc.

Bảng 6.1. Chọn kỹ thuật trên mảng theo thao tác cần làm. Chi phí giả sử phép toán trên một giá trị tồn hằng số.

Kỹ thuật	Giả thiết / cơ chế	Chi phí	Chỗ dễ sai
Tổng tiền tố	Mảng tĩnh, lấy hiệu hai tiền tố	Xây $O(n)$, hỏi $O(1)$	Thiếu P_0 ; tràn tổng
Mảng hiệu	Cộng đoạn, đọc kết quả sau cùng	Sửa $O(1)$, khôi phục $O(n)$	Thiếu ô $r + 1$; hỏi giữa chừng
Cửa sổ trượt	Bỏ một đầu nút không mất nghiệm tương lai	Tổng $O(n)$ nếu hai đầu chỉ tiến	Tổng có số âm mất đơn điệu
Tiền tố + băm	Đếm tổng đoạn bằng k , cho phép số âm	$O(n)$ kỳ vọng, bộ nhớ $O(n)$	Quên tần suất tiền tố 0 bằng 1

Bảng 6.1 phân biệt bốn bài toán thường bị nhận nhầm thành cùng một mẫu. Khi đếm tổng bằng k , hãy tra số tiền tố bằng $P_r - k$ trước khi thêm P_r ; đảo thứ tự sẽ đếm cả đoạn rỗng nếu $k = 0$.

6.8 Phản biện, nhược điểm và rủi ro triển khai

Mảng, tổng tiền tố và hai con trỏ là những công cụ cơ bản nhất, nhưng chính sự đơn giản đó tạo ra cảm giác an toàn giả tạo. Trong thực tế, đây là nơi phát sinh nhiều lỗi im lặng và lạm dụng phương pháp nhất.

Phản biện: sự lạm dụng hai con trỏ và giới hạn của mảng hiệu. Hai con trỏ chỉ chạy đúng khi bài toán có tính đơn điệu (monotonicity) rõ ràng: mở rộng khoảng làm điều kiện vi phạm nhiều hơn, và thu hẹp khoảng làm điều kiện dễ thoả mãn hơn. Khi mảng xuất hiện số âm, tổng đoạn con dao động hỗn loạn; việc co/dãn cửa sổ tham lam sẽ bỏ sót nghiệm tối ưu toàn cục. Cố chấp ép hai con trỏ vào mảng có số âm là sai lầm kinh điển của người mới. Tương tự, mảng hiệu chỉ là giải pháp xử lý ngoại tuyến (offline batch): cập nhật toàn bộ rồi đọc kết quả một lần duy nhất. Nếu bài toán có truy vấn xen kẽ cập nhật (online dynamic), mảng hiệu hoàn toàn vô dụng; việc khôi phục mảng sau mỗi truy vấn làm độ phức tạp thoái hoá thành $O(q \cdot n)$, chậm hơn hàng nghìn lần so với cây Fenwick hay Segment Tree (B/10).

Nhược điểm cốt lõi về cấu trúc và bộ nhớ:

- *Tính tĩnh tuyệt đối (Rigid Immutability):* Cả tổng tiền tố và mảng hiệu đều phụ thuộc vào chỉ số cố định của mảng. Chỉ cần chen thêm hoặc xoá bớt một phần tử ở đầu mảng, toàn bộ bảng tính trước bị vô hiệu hoá và buộc phải xây lại từ đầu với chi phí $O(n)$.
- *Sự phình to bộ nhớ của tiền tố đa chiều:* Trên mảng hai chiều hoặc ba chiều (như voxel 3D trong thị giác máy tính), bảng tổng tiền tố đòi hỏi bộ nhớ $O(n^d)$. Với ảnh phân giải cao hoặc lưới thể tích 3D, dung lượng RAM bùng nổ nhanh chóng; đồng thời công thức bao hàm–loại trừ (Inclusion-Exclusion) nhiều chiều rất dễ nhầm lẫn dấu tại các góc và gây trượt cache L1 liên tục.

Rủi ro vận hành và các lỗi im lặng nguy hiểm:

- *Tràn số nguyên trong tổng tiền tố (Overflow):* Với $n = 10^5$ phần tử và giá trị phần tử tới 10^9 , tổng tiền tố vượt quá 10^{14} , vượt xa giới hạn số nguyên 32-bit có dấu. Dùng `int` thay vì `long` (hoặc `int64_t`) là lỗi im lặng phổ biến nhất: chương trình chạy mượt mà nhưng trả về kết quả âm vô lý. Trong bài toán lấy modulo, hiệu $P[r] - P[l - 1]$ có thể âm, nếu quên cộng thêm m trước khi lấy dư tiếp theo $((P[r] - P[l - 1] \% m + m) \% m)$, kết quả sẽ bị sai hoàn toàn.
- *Lỗi lệch một đơn vị (Off-by-One) và truy cập ngoài biên:* Quên khởi tạo $P[0] = 0$ làm sai lệch truy vấn đoạn bắt đầu từ đầu mảng $[1, r]$. Trong mảng hiệu, việc cập nhật $D[r+1] -= x$ đòi hỏi mảng hiệu phải có kích thước tối thiểu $n + 2$; quên điều này sẽ ghi đè vào vùng nhớ ngoài biên (Out-of-Bounds Memory Write), làm hỏng dữ liệu của các biến lân cận.
- *Trượt cửa sổ vô hạn hoặc vượt biên:* Trong thuật toán hai con trỏ, nếu vòng lặp co cửa sổ không kiểm tra điều kiện $l \leq r$, con trỏ l có thể vượt qua r , tạo ra cửa sổ có độ dài âm và rơi vào vòng lặp vô tận (hang/infinite loop) trên các trường hợp biên mảng rỗng hoặc toàn phần tử không thoả mãn.

CẠM BÃY

Không bao giờ dùng kiểu `int` 32-bit cho mảng tổng tiền tố trừ khi đề bài bảo đảm rõ ràng tổng toàn mảng nằm gọn trong khoảng $[-2 \cdot 10^9, 2 \cdot 10^9]$. Một thói quen tốt là luôn khai báo mảng tiền tố bằng `long` ngay từ đầu để loại bỏ hoàn toàn một lớp lỗi im lặng.

6.9 Gốc rễ: từ băng từ tới ảnh tích phân

Mảng là cấu trúc dữ liệu cổ nhất trong ngành và lý do rất vật lý: bộ nhớ máy tính *vốn* là một dãy ô đánh số, nên mảng không phải một phát minh mà là hình dạng tự nhiên của phần cứng. Mọi cấu trúc khác trong phần B đều được xây *trên* mảng, và cái giá của chúng là chi phí gián tiếp so với truy cập trực tiếp.

Tổng tiền tố thì cổ hơn máy tính. Ý tưởng “lập bảng tổng lũy tích để tra nhanh” đã có trong các bảng thống kê và bảng thiên văn từ trước; trong toán, đó chính là mối quan hệ giữa tổng và sai phân, tương ứng rời rạc của tích phân và đạo hàm — và mối quan hệ đó giải thích vì sao mảng hiệu là nghịch đảo của tổng tiền tố, đúng như đạo hàm là nghịch đảo của tích phân.

Điều thú vị là kỹ thuật này được phát minh lại nhiều lần ở các ngành khác nhau. Trong tính toán song song, phép *scan* (tổng tiền tố) được nhận ra là một nguyên thủy cơ bản mà rất nhiều thuật toán song song quy về được, và nó song song hoá được trong $O(\log n)$ bước — điều không hiển nhiên chút nào vì định nghĩa của nó có vẻ hoàn toàn tuần tự. Trong thị giác máy tính, cùng cấu trúc mang tên *ảnh tích phân*: Viola và Jones dùng nó năm 2001 để tính tổng cường độ trên hình chữ nhật bất kỳ trong bốn phép truy cập, và chính điều đó biến bộ phát hiện khuôn mặt của họ từ ý tưởng thành sản phẩm chạy thời gian thực trên máy ảnh số.

Hai con trỏ thì không có “người phát minh” — nó thuộc loại kỹ thuật hiển nhiên khi nhìn đúng góc, được dùng rải rác từ lâu, và chỉ được đặt tên và hệ thống hoá khi lập trình thi đấu cần một từ vựng chung để nói về nó.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1940 — máy tính có bộ nhớ đánh địa chỉ	Cần một cách tổ chức dữ liệu khớp với phần cứng.	Mảng: không phải phát minh mà là hình dạng tự nhiên của bộ nhớ; mọi cấu trúc khác đều xây trên nó.
Trước cả máy tính — bảng lũy tích trong thống kê, thiên văn	Tra nhanh tổng của một khoảng trong bảng số.	Tổng tiền tố như một dạng tích phân rời rạc; mảng hiệu là sai phân, tức phép ngược.
Thập niên 1980–90 — tính toán song song	Nhiều thuật toán song song cần cùng một nguyên thủy tích lũy.	Phép <i>scan</i> song song hoá được trong $O(\log n)$ bước dù định nghĩa trông thuần tuần tự; trở thành khối xây dựng chuẩn của GPU (D/24).
2001 — Viola & Jones	Phát hiện khuôn mặt thời gian thực: cần tổng cường độ trên hàng nghìn hình chữ nhật mỗi khung hình.	Ảnh tích phân — tổng tiền tố hai chiều — cho mỗi hình chữ nhật trong bốn phép truy cập; biến thuật toán từ không khả thi thành sản phẩm thương mại.
Thập niên 2000 — lập trình thi đấu	Cần từ vựng chung để dạy các mẫu lặp lại.	“Hai con trỏ”, “cửa sổ trượt” được đặt tên và hệ thống hoá; kỹ thuật cũ nhưng cái tên mới làm chúng dạy được.

■ 6.10 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính	Ảnh tích phân, bộ lọc hộp, tính đặc trưng Haar	Cùng một cấu trúc với tổng tiền tố hai chiều. Bài học chung: khi cần rất nhiều truy vấn trên vùng chữ nhật, hãy trả trước một lượt quét. Ứng dụng gần: tính tổng cường độ trên các ô lưới khi lọc điểm đặc trưng trong SLAM.
Giải tích	Tích phân và đạo hàm là hai phép ngược nhau	Tổng tiền tố và mảng hiệu là bản rời rạc chính xác của cặp này. Nhận ra sự tương ứng giúp nhớ được cả hai và biết khi nào đổi qua lại.
Xử lý tín hiệu	Trung bình trượt, bộ lọc chạy	Cùng khuôn cửa sổ trượt, chỉ khác là ở đó cửa sổ có độ dài cố định nên còn đơn giản hơn. Cả hai đều dựa vào việc cập nhật gia tăng thay vì tính lại.
Mạng máy tính	Cửa sổ trượt trong TCP	Cùng tên và cùng ý tưởng cơ bản: duy trì một khoảng hợp lệ, tiến mép phải khi nhận được xác nhận, kéo mép trái theo. Khác: ở đó cửa sổ điều tiết tốc độ, không phải để tìm cực trị.
Tính toán song song, GPU	Nguyên thủy <i>scan</i> và <i>reduce</i>	Tổng tiền tố là nguyên thủy song song cơ bản. Điểm bất ngờ đáng nhớ: một phép định nghĩa hoàn toàn tuần tự vẫn có thể song song hoá trong $O(\log n)$ bước bằng cây tổng.
Cơ sở dữ liệu	Chỉ mục tổng hợp, bảng tổng lũy tích	Cùng đánh đổi: trả trước chi phí xây dựng để truy vấn rẻ. Cùng nhược điểm: dữ liệu thay đổi thì phải xây lại — và đó chính là lý do tồn tại của cây chỉ số ở B/10.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì ba lý do vật lý mà ký hiệu tiệm cận không nhìn thấy: truy cập là phép tính địa chỉ chứ không phải lần theo con trỏ; các phần tử nằm liên tiếp nên phần cứng nạp sẵn phần tử kế tiếp vào cache; và không có chi phí cấp phát cho từng phần tử. Với dữ liệu vừa cỡ vài triệu phần tử, ba yếu tố này cộng lại có thể tạo chênh lệch cả chục lần — đủ để lật ngược một khác biệt tiệm cận nhỏ như thừa số log.
- A2. Lập luận “tổng quãng đường của con trỏ”. Vòng lặp ngoài chỉ tăng l , vòng lặp trong chỉ giảm r , và không chỉ số nào quay lui. Mỗi chỉ số vì thế đi tối đa n bước trong suốt cả thuật toán, nên tổng số bước tối đa là $2n$. Cấu trúc lồng nhau của code đánh lừa: cái quyết định chi phí không phải độ sâu lồng nhau mà là tổng số lần các biến chỉ số thay đổi.
- A3. Nó hỏng khi mất tính đơn điệu. Ví dụ: tìm đoạn con dài nhất có tổng $\leq S$ trong mảng có số âm. Với mảng $[5, -6, 5]$ và $S = 4$: cửa sổ trượt sẽ co bỏ phần tử đầu ngay khi tổng vượt S và bỏ sót đoạn $[5, -6, 5]$ có tổng 4. Nguyên nhân: khi có số âm, việc mở rộng cửa sổ có thể làm tổng giảm, nên điều kiện không còn đơn điệu theo độ dài. Bài đó phải giải bằng tổng tiền tố cộng cấu trúc tìm kiếm.
- A4. Nó cho một mẫu tổng quát: nếu phép toán A rẻ trên biểu diễn gốc còn phép toán B đắt, hãy tìm biểu diễn mà trong đó B rẻ, làm hết các thao tác B rồi chuyển đổi ngược một lần. Cụ thể: tổng tiền tố cho “truy vấn đoạn rẻ, cập nhật đắt”; mảng hiệu cho “cập nhật đoạn rẻ, đọc kết quả một lần”. Khi bài cần cả hai đều nhiều thì không biểu diễn tĩnh nào đủ, và đó chính là lúc cần cây chỉ số ở B/10.
- A5. Vì sắp xếp tạo ra tính đơn điệu, và tính đơn điệu là điều kiện đầu vào của hầu hết kỹ thuật rẻ: hai con trỏ, tìm kiếm nhị phân, tham lam, gộp các phần tử bằng nhau. Chi phí $O(n \log n)$ gần như luôn nằm trong ngân sách, nên đây là một trong những nước đi có tỉ lệ lợi ích trên rủi ro cao nhất khi bí. Điểm cần cẩn thận: đôi khi thứ tự gốc mang thông tin (bài về đoạn liên tiếp), và lúc đó sắp xếp làm mất bài toán.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng bạn quản lý sổ thu chi của một quán ăn và mỗi ngày có người hỏi “từ ngày 5 tới ngày 20 quán thu bao nhiêu?”. Cách vụng là mỗi lần hỏi lại cộng lại từ đầu. Cách khôn là ngay từ đầu bạn ghi thêm một cột: tổng thu từ đầu tháng tới hết ngày này. Khi có người hỏi, bạn lấy con số ở ngày 20 trừ con số ở ngày 4. Đó là tổng tiền tố, và toàn bộ mẹo chỉ là *trả trước một lần để mọi câu hỏi sau chỉ tốn một phép trừ*.

Ngược lại, nếu sếp nói “cộng thêm 10 nghìn vào mọi ngày từ 5 tới 20” rất nhiều lần trong tháng, thì sửa từng ngày là phí. Bạn chỉ cần ghi “từ ngày 5 tăng thêm 10” và “từ ngày 21 giảm đi 10”, rồi cuối tháng cộng dồn một lượt. Đó là mảng hiệu.

Còn hai con trỏ thì giống hai người cùng đi trên một hàng ghế từ hai đầu, và không ai được quay lui. Vì không ai quay lui nên dù họ đi lâu tới đâu, tổng số bước cũng chỉ bằng chiều dài hàng ghế. Điều kiện để cách này đúng là hàng ghế phải được sắp có trật tự: bước sang phải thì giá trị luôn tăng, bước sang trái thì luôn giảm. Nếu trật tự đó không có — ví dụ trong dãy có số âm làm tổng lúc tăng lúc giảm — thì hai người sẽ bỏ sót chỗ cần tìm, và tệ hơn là chương trình vẫn chạy ngon lành, chỉ đưa ra kết quả sai.

Một câu: *Đừng tính lại từ đầu; hãy cập nhật đúng phần vừa thay đổi.*

Chỗ không rút gọn được: Việc kiểm tra “tính chất này có đơn điệu không” vẫn phải làm bằng lập luận cho từng bài; không có dấu hiệu bề ngoài nào chắc chắn.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý chung của cả chương	Khi vòng ngoài tiến một bước, vòng trong đừng quay lại từ đầu — hãy cập nhật phần thay đổi. Biến $O(n^2)$ thành $O(n)$.
••• Tổng tiền tố	$P_i = P_{i-1} + a_i$ với $P_0 = 0$; tổng đoạn $[l, r] = P_r - P_{l-1}$. Luôn để ô đệm P_0 để không có trường hợp riêng.
••• Điều kiện dùng được tổng tiền tố	Phép toán phải có phép ngược: tổng và XOR được, cực đại thì không — cực đại cần sparse table hoặc cây phân đoạn (B/10).
••• Mảng hiệu	Cộng v vào $[l, r]$: $D_l += v$, $D_{r+1} -= v$; cuối cùng lấy tổng tiền tố. Dừng khi cập nhật đoạn nhiều, đọc kết quả một lần.
••• Vì sao hai con trỏ tuyến tính	Mỗi chỉ số chỉ đi một chiều và không quay lui, nên tổng quãng đường $\leq 2n$. Độ sâu lồng nhau của code không quyết định chi phí.
••• Điều kiện sống còn của cửa sổ trượt	Tính đơn điệu: đoạn vi phạm thì mọi đoạn chứa nó cũng vi phạm. Mất điều kiện này thì kết quả sai một cách im lặng.
••• Phân ví dụ kinh điển	“Đoạn dài nhất có tổng $\leq S$ ” với mảng có số âm: $[5, -6, 5]$, $S = 4$ — cửa sổ trượt bỏ sót đáp án.
•• Khuôn cửa sổ trượt	Mở rộng r một bước; trong khi vi phạm thì co l ; cập nhật đáp án. Ba dòng, dùng cho hàng trăm bài.
•• Đếm đoạn con có tổng bằng k	$P_r - P_{l-1} = k$: với mỗi r , đếm số P trước đó bằng $P_r - k$ bằng bảng băm. Mẫu “biến điều kiện đoạn thành điều kiện hai điểm nút”.
•• Tổng tiền tố hai chiều	Tổng hình chữ nhật bằng bốn giá trị theo bao hàm–loại trừ; trong thị giác máy tính gọi là ảnh tích phân.
•• Vì sao mảng thẳng con trỏ	Truy cập trực tiếp, dữ liệu liên tiếp thân thiện cache, không cấp phát từng phần tử. Thua khi phải chèn/xóa giữa mảng nhiều.
•• Sắp xếp như nước mở đầu	Nó tạo ra tính đơn điệu — điều kiện đầu vào của hai con trỏ, nhị phân và tham lam. $O(n \log n)$ hầu như luôn trong ngân sách.
• Đặc thù của chuỗi	So sánh chuỗi tốn theo độ dài chứ không $O(1)$; bảng chữ cái nhỏ nên đếm bằng mảng 26 phần tử nhanh hơn bảng băm.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào dùng tổng tiền tố, khi nào phải dùng cây phân đoạn?

| Quyết định bằng hai câu hỏi. Dữ liệu có thay đổi giữa các truy vấn không? Nếu không, tổng tiền tố là đủ và luôn tốt hơn vì hằng số nhỏ hơn nhiều. Nếu có thay đổi, tổng tiền tố phải xây lại $O(n)$ mỗi lần nên hỏng, và bạn cần Fenwick hoặc cây phân đoạn (B/10). Câu hỏi thứ hai: phép toán có phép ngược không? Tổng và XOR có nên tổng tiền tố dùng được; cực đại và cực tiểu thì không, phải dùng sparse table cho dữ liệu tĩnh hoặc cây phân đoạn cho dữ liệu động.

2. Làm sao biết chắc một bài giải được bằng cửa sổ trượt trước khi cài?

| Kiểm tính đơn điệu bằng cách hỏi hai câu cụ thể. Một: nếu đoạn $[l, r]$ đã thỏa điều kiện, mọi đoạn con của nó có chắc chắn thỏa không? Hai: nếu đoạn $[l, r]$ vi phạm, mọi đoạn chứa nó có chắc chắn vi phạm không? Cả hai câu đều “có” thì cửa sổ trượt dùng được. Nếu bạn không chắc, hãy thử ngay ba dữ liệu đặc biệt: có số âm, có số 0, và mọi phần tử bằng nhau — ba ca này bắt được phần lớn lỗi đơn điệu.

3. Hai con trỏ và tìm kiếm nhị phân đều cần dữ liệu đã sắp. Chọn cái nào?

| Chọn theo số lượng truy vấn. Nếu bạn cần trả lời một câu hỏi cho mỗi vị trí trái (kiểu “với mỗi l , tìm r lớn nhất thỏa điều kiện”) thì hai con trỏ cho $O(n)$ tổng cộng, tốt hơn $O(n \log n)$ của việc nhị phân từng vị trí. Nhưng hai con trỏ chỉ dùng được khi r là hàm *không giảm* theo l ; nếu không, phải nhị phân. Trong thực tế, viết nhị phân dễ hơn và ít lỗi hơn, nên khi giới hạn cho phép thì đó là lựa chọn an toàn.

4. Tổng tiền tố có bị tràn số không?

| Rất hay bị, và đây là lỗi im lặng kinh điển. Với $n = 10^5$ phần tử, mỗi phần tử tới 10^9 , tổng có thể tới 10^{14} — vượt xa số nguyên 32 bit. Quy tắc: mảng tổng tiền tố gần như luôn phải dùng kiểu 64 bit ngay cả khi mảng gốc là 32 bit. Trong bài lấy modulo, còn phải cẩn thận thêm: hiệu $P_r - P_{l-1}$ có thể âm sau khi lấy dư, nên phải cộng thêm m rồi lấy dư lần nữa.

5. Kỹ thuật này còn dùng ở đâu ngoài bài tập luyện?

| Rất nhiều nơi trong hệ thống thật. Đếm số sự kiện trong một khoảng thời gian trong hệ thống giám sát chính là tổng tiền tố trên trục thời gian. Giới hạn tốc độ theo cửa sổ trượt trong máy chủ web là đúng khuôn ở mục 4. Trong thị giác máy tính, ảnh tích phân dùng để tính nhanh thống kê kê trên vùng ảnh. Trong xử lý dữ liệu cảm biến, trung bình trượt để lọc nhiễu chính là cửa sổ trượt với độ dài cố định.

6. Có mẹo nào tránh lỗi lệch một đơn vị với tổng tiền tố không?

| Ba quy ước cụ thể giúp gần như hết hẳn. Một: luôn để $P_0 = 0$ và đánh số mảng từ 1, khi đó công thức $P_r - P_{l-1}$ đúng cho mọi l kể cả $l = 1$. Hai: viết bất biến ra thành chú thích — “ P_i = tổng của i phần tử đầu tiên” — và mọi chỉ số suy ra từ đó. Ba: kiểm tra bằng một ví dụ ba phần tử đã tính tay trước khi chạy dữ liệu lớn. Đây là ứng dụng trực tiếp của kỷ luật ở A/04.

7. Bài yêu cầu cả cập nhật đoạn lẫn truy vấn đoạn thì sao?

| Không cấu trúc tính nào trong chương này giải được, vì tổng tiền tố mạnh ở truy vấn còn mảng hiệu mạnh ở cập nhật, và ghép trực tiếp thì cái này phá cái kia. Đó chính xác là lý do tồn tại của cây phân đoạn có lazy propagation ở B/10: nó chấp nhận $O(\log n)$ cho *cả hai* phía thay vì $O(1)$ cho một phía và $O(n)$ cho phía kia.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn với truy vấn tĩnh	CSES “Range Sum Queries I”[33]	Tổng tiền tố với ô đệm; chú ý kiểu 64 bit.
● Cập có tổng bằng x trong mảng đã sắp	CSES “Sum of Two Values”[33]	Hai con trỏ; viết bất biến “mọi cặp đã loại đều không thể là đáp án”.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tổng đoạn con lớn nhất	CSES “Maximum Subarray Sum”[33]	So sánh cách Kadane với cách tổng tiền tố cộng cực tiểu chạy — hai góc nhìn cho cùng bài.
•• Đoạn con dài nhất không có ký tự lặp	LeetCode “Longest Substring Without Repeating Characters”	Cửa sổ trượt với mảng đếm; kiểm tính đơn điệu trước khi cài.
•• Đếm số đoạn con có tổng bằng x	CSES “Subarray Sums II”[33]	Tổng tiền tố cộng bằng băm; mẫu “điều kiện đoạn thành điều kiện hai điểm nút”.
•• Cập nhật đoạn, truy vấn điểm	CSES “Range Update Queries”[33]	Mảng hiệu; sau đó thử lại bằng Fenwick để thấy khi nào cần cấu trúc mạnh hơn.
•• Tổng hình chữ nhật trong ma trận tĩnh	CSES “Forest Queries”[33]	Tổng tiền tố hai chiều; vẽ hình bao hàm—loại trừ ra giấy trước khi viết công thức.
•• Gộp các đoạn giao nhau	LeetCode “Merge Intervals”	Sắp theo điểm bắt đầu; đây là bài mẫu cho “chọn khoá sắp xếp chính là lời giải”.
••• Cửa sổ nhỏ nhất chứa đủ các ký tự cần thiết	LeetCode “Minimum Window Substring”	Cửa sổ trượt hai chiều với bộ đếm; bài kinh điển để luyện việc co cửa sổ đúng lúc.
••• Nước mưa mắc kẹt	LeetCode “Trapping Rain Water”	Giải cả ba cách (tiền tố cực đại, ngăn xếp B/07, hai con trỏ) và so sánh — bài tập bước 4 tốt nhất của chương.
••• Số đoạn con có đúng k phần tử phân biệt	Bài tập kinh điển	Mẹo “đúng k ” = “nhiều nhất k ” trừ “nhiều nhất $k - 1$ ” — một mẫu biến bài không đơn điệu thành hai bài đơn điệu.
••• Đoạn con dài nhất có tổng $\leq S$ khi mảng có số âm	Bài tập kinh điển	Bài mà cửa sổ trượt sai; cần tổng tiền tố cộng cấu trúc tìm kiếm — hãy tự chỉ ra chính xác chỗ tính đơn điệu gây.

Kết chương và đọc tiếp

Chương này chỉ có một ý và bốn cách hiện thực hoá nó: đừng tính lại thứ vừa tính. Tổng tiền tố trả trước cho truy vấn; mảng hiệu trả trước cho cập nhật; hai con trỏ và cửa sổ trượt cập nhật gia tăng khi biên di chuyển. Điều kiện dùng được của cả bốn đều quy về tính đơn điệu, và mất tính đơn điệu thì lỗi xảy ra một cách im lặng — nên câu hỏi kiểm tra ở khối *Cần nhớ* đáng thành phần xạ.

Chương tiếp theo mở rộng đúng ý đó cho tình huống mà thứ tự xử lý mới là cái quan trọng: khi bạn cần lấy ra “phần tử đúng lúc” — mới nhất, nhỏ nhất, hay lớn hơn kế tiếp — thay vì quét theo chỉ số. Đó là ngăn xếp, hàng đợi và heap (B/07), và bạn sẽ thấy ngăn xếp đơn điệu là họ hàng gần của hai con trỏ, với cùng lập luận “mỗi phần tử vào ra đúng một lần”.

Ngăn xếp, hàng đợi và heap (Stacks, Queues, Heaps)

PHỤ THUỘC B/06 cho kỹ thuật quét tuyến tính; A/03 cho phân tích khẩu hao — ngăn xếp đơn điệu là ví dụ khẩu hao đẹp nhất trong cả quyển sách.

TÓM TẮT

Ba cấu trúc trong chương này trả lời cùng một câu hỏi với ba câu trả lời khác nhau: *trong đồng dư liệu đang chờ, phần tử nào nên xử lý tiếp?* Ngăn xếp nói “cái mới nhất”, hàng đợi nói “cái cũ nhất”, heap nói “cái nhỏ nhất”. Ba lựa chọn ấy sinh ra ba họ thuật toán rất khác nhau: đệ quy và duyệt sâu, duyệt rộng và mô phỏng theo thời gian, và các thuật toán tham lam kiểu Dijkstra hay Huffman. Chương cũng dựng hai kỹ thuật có tỉ lệ sức mạnh trên độ dài code rất cao: ngăn xếp đơn điệu và hàng đợi hai đầu đơn điệu. Nếu chỉ nhớ một điều: heap cổ tình duy trì *ít trật tự hơn* mảng đã sắp — nó chỉ biết ai nhỏ nhất — và chính sự tiết chế đó làm nó rẻ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ngăn xếp (stack)** — cấu trúc vào sau ra trước (LIFO): thêm và lấy đều ở một đầu, cả hai $O(1)$.
- **hàng đợi (queue)** — vào trước ra trước (FIFO): thêm ở một đầu, lấy ở đầu kia.
- **hàng đợi hai đầu (deque)** — thêm và lấy được ở cả hai đầu, mỗi thao tác $O(1)$.
- **heap nhị phân** — cây nhị phân gần đầy, mỗi nút không lớn hơn các con của nó, lưu bằng mảng: con của i là $2i$ và $2i + 1$. Lấy nhỏ nhất $O(1)$, thêm và xóa $O(\log n)$.
- **hàng đợi ưu tiên (priority queue)** — giao diện trừu tượng “thêm phần tử, lấy phần tử ưu tiên nhất”; heap là cách cài phổ biến nhất nhưng không phải duy nhất.
- **ngăn xếp đơn điệu** — ngăn xếp mà các phần tử trong nó luôn giữ thứ tự tăng (hoặc giảm); dùng để tìm “phần tử lớn hơn kế tiếp” cho mọi vị trí trong $O(n)$.

CÂU HỎI MỞ ĐẦU

- Q1. Ngăn xếp đơn điệu có vòng lặp lồng nhau và trông như $O(n^2)$. Vì sao nó thật sự là $O(n)$?
- Q2. Vì sao heap chỉ cho biết phần tử nhỏ nhất mà không cho biết phần tử thứ hai một cách trực tiếp — và vì sao đó lại là *ưu điểm* chứ không phải khuyết điểm?
- Q3. Xây heap từ n phần tử tốn $O(n)$ chứ không phải $O(n \log n)$, dù mỗi lần chèn tốn $O(\log n)$. Chỗ nào trong lập luận “ n lần chèn” bị sai?
- Q4. Duyệt sâu dùng ngăn xếp, duyệt rộng dùng hàng đợi — chỉ khác một dòng code. Vì sao hai thuật toán lại có tính chất khác nhau đến vậy?
- Q5. Khi nào một mảng đã sắp xếp tốt hơn heap, và ngược lại?

Chương trước quét dữ liệu theo thứ tự chỉ số. Nhưng rất nhiều bài toán không cho phép làm vậy: bạn xử lý một phần tử, việc đó sinh ra công việc mới, và công việc mới phải xếp vào một chỗ nào đó để xử lý sau. Câu hỏi trung tâm trở thành: *trong đồng công việc đang chờ, cái nào làm tiếp?*

Ba cấu trúc trong chương này là ba câu trả lời, và điều đáng chú ý là chúng chỉ khác nhau ở đúng chỗ đó — cùng một khung “lấy ra, xử lý, đẩy vào” nhưng đổi quy tắc lấy ra thì được ba thuật toán khác hẳn nhau về hành vi. Duyệt sâu, duyệt rộng và Dijkstra là cùng một chương trình với ba cấu trúc chờ khác nhau, và nhận ra điều đó làm ba thuật toán ấy sụp thành một.

I 7.1 Ngăn xếp: cái mới nhất trước

Ngăn xếp là cấu trúc gắn liền với *sự lồng nhau*. Mỗi khi bài toán có cấu trúc “mở rồi phải đóng theo đúng thứ tự ngược lại” thì ngăn xếp là công cụ tự nhiên: dấu ngoặc trong biểu thức, thẻ trong tài liệu đánh dấu, lời gọi hàm trong chương trình, thao tác hoàn tác trong trình soạn thảo.

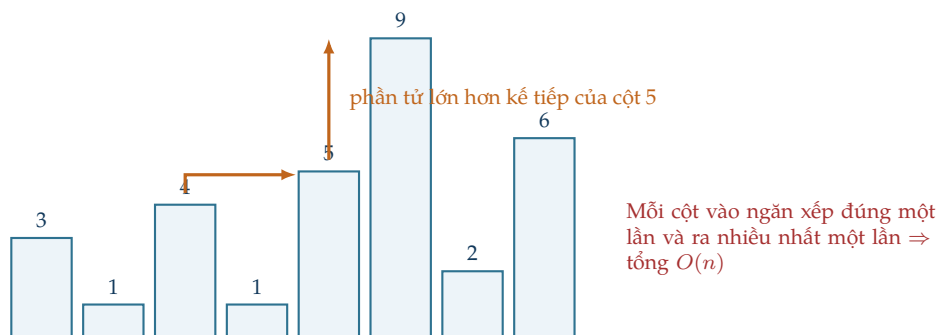
Điều đáng để ý về mặt khái niệm: đệ quy và ngăn xếp là cùng một thứ. Máy tính thực hiện đệ quy bằng cách đẩy trạng thái lời gọi vào ngăn xếp phân cứng. Vì vậy mọi thuật toán đệ quy đều viết lại được thành vòng lặp với ngăn xếp tường minh — một kỹ thuật đáng biết vì đôi khi nó là cách duy nhất tránh tràn ngăn xếp khi độ sâu lên tới hàng trăm nghìn (E/25).

Ngăn xếp đơn điệu: kỹ thuật đáng giá nhất của chương

Bài toán: với mỗi vị trí i trong mảng, tìm vị trí đầu tiên bên phải có giá trị lớn hơn a_i . Vết cạn $O(n^2)$. Ngăn xếp đơn điệu cho $O(n)$.

Ý tưởng: quét từ trái sang phải, duy trì một ngăn xếp chứa các vị trí mà *chưa tìm được phần tử lớn hơn*. Khi tới vị trí i , lấy ra khỏi ngăn xếp mọi vị trí có giá trị nhỏ hơn a_i — với mỗi vị trí lấy ra, đáp án của nó chính là i . Rồi đẩy i vào.

Bất biến: *ngăn xếp luôn chứa các giá trị giảm dần từ đáy lên đỉnh, và đó đúng là những vị trí còn đang chờ đáp án*. Vì sao $O(n)$? Mỗi vị trí được đẩy vào đúng một lần và lấy ra nhiều nhất một lần, nên tổng số thao tác không quá $2n$ — lập luận khấu hao y hệt lập luận “tổng quãng đường con trỏ” ở chương B/06. Vòng lặp trong có thể chạy nhiều bước ở một số lần lặp, nhưng khi đó nó vừa xoá đi nhiều phần tử mà ai đó đã trả tiền lúc đẩy vào.



Hình 7.1. Ngăn xếp đơn điệu tìm “phần tử lớn hơn kế tiếp”. Các cột đang nằm trong ngăn xếp luôn giảm dần từ trái sang phải; khi gặp một cột cao hơn, tất cả các cột thấp hơn nó bị lấy ra và nhận cột này làm đáp án. Cùng cấu trúc giải bài “hình chữ nhật lớn nhất trong biểu đồ cột” — ở đó mỗi cột bị lấy ra chính là lúc ta biết được cả biên trái lẫn biên phải của hình chữ nhật cao bằng nó.

Họ bài giải bằng ngăn xếp đơn điệu rộng hơn tương ứng: phần tử lớn hơn/nhỏ hơn kế tiếp hai phía, hình chữ nhật lớn nhất trong biểu đồ cột, nước mưa mắc kẹt, số đoạn con mà một phần tử là cực đại, và cả việc dựng cây Cartesian. Dấu hiệu nhận biết trên đề: *với mỗi phần tử, cần biết phần tử gần nhất về một phía thoả một quan hệ so sánh*.

I 7.2 Hàng đợi và hàng đợi hai đầu

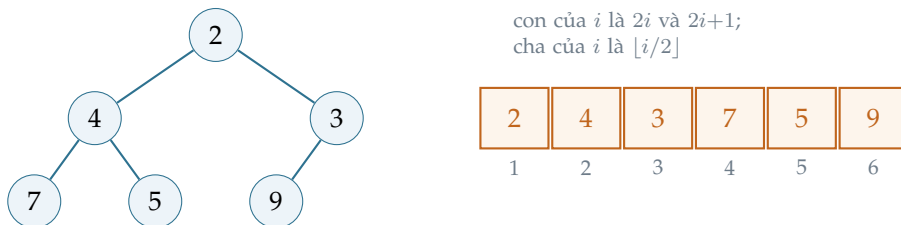
Hàng đợi gắn với việc xử lý theo *lớp*: duyệt rộng trên đồ thị xử lý hết các đỉnh cách 1 bước rồi mới tới các đỉnh cách 2 bước, và chính tính chất đó làm duyệt rộng cho đường đi ngắn nhất trên đồ thị không trọng số (C/17). Trong hệ thống thật, hàng đợi là cấu trúc của mọi thứ có “xếp hàng chờ”: hàng đợi tin nhắn, bộ đệm, lịch tiến trình.

Hàng đợi hai đầu cho một kỹ thuật quan trọng: *cực đại trên cửa sổ trượt* trong $O(n)$. Duy trì một deque chứa các chỉ số có giá trị giảm dần; khi cửa sổ tiến, bỏ ở đầu những chỉ số đã rơi ra ngoài cửa sổ, và bỏ ở cuối những chỉ số có giá trị nhỏ hơn phần tử mới — vì chúng không bao giờ còn cơ hội làm cực đại nữa. Đây là kết hợp của cửa sổ trượt (B/06) với ý tưởng đơn điệu của mục trước, và nó cũng là bước tăng tốc chuẩn cho một họ bài quy hoạch động (C/16).

7.3 Heap: chỉ biết ai nhỏ nhất, và chỉ cần thế

Heap nhị phân là một cây nhị phân gần đầy lưu trong mảng, với tính chất: mỗi nút không lớn hơn các con của nó. Chú ý điều *không* được đảm bảo: hai anh em không có quan hệ gì với nhau, và mảng biểu diễn heap hoàn toàn không sắp xếp. Heap chỉ biết đúng một điều — gốc là nhỏ nhất.

Sự tiết chế đó chính là nguồn sức mạnh. Duy trì trật tự toàn phần (mảng đã sắp) tốn $O(n)$ cho mỗi lần chèn; duy trì trật tự bộ phận của heap chỉ tốn $O(\log n)$. Nếu bài toán chỉ cần biết ai nhỏ nhất — và rất nhiều bài chỉ cần thế — thì trả tiền cho trật tự toàn phần là lãng phí.



Hình 7.2. Heap nhị phân và biểu diễn mảng của nó. Không có con trỏ nào — quan hệ cha con là phép tính chỉ số, nên heap vừa tiết kiệm bộ nhớ vừa thân thiện với cache. Chú ý mảng *không* sắp xếp: 4 đứng trước 3. Heap chỉ hứa mỗi nút nhỏ hơn các con của nó, không hứa gì về quan hệ giữa các nhánh.

Ba thao tác và chi phí: lấy nhỏ nhất $O(1)$; thêm phần tử $O(\log n)$ bằng cách đặt vào cuối rồi đẩy lên; xoá nhỏ nhất $O(\log n)$ bằng cách đưa phần tử cuối lên gốc rồi đẩy xuống. Còn một thao tác đáng nhớ vì nó phản trực giác: *xây heap từ n phần tử có sẵn tốn $O(n)$, không phải $O(n \log n)$* . Lý do: nếu đi từ dưới lên và đẩy xuống, thì các nút ở tầng đáy — chiếm một nửa số nút — gần như không phải làm gì, các nút ở tầng trên thì ít. Tổng chi phí là $\sum_h (n/2^h) \cdot h$, một chuỗi hội tụ về $O(n)$. Đây là ứng dụng trực tiếp của “cấp số nhân giảm bị chặn bởi hằng số” ở chương A/05.

Ba mẫu dùng heap đáng thuộc

Chọn k phần tử lớn nhất trong n phần tử. Giữ một heap nhỏ nhất kích thước k ; mỗi phần tử mới, nếu lớn hơn gốc thì thay gốc. Chi phí $O(n \log k)$ và bộ nhớ $O(k)$ — tốt hơn hẳn việc sắp toàn bộ khi $k \ll n$, và quan trọng hơn, nó chạy được cả khi dữ liệu là luồng không lưu hết được (D/24).

Trộn k danh sách đã sắp. Heap chứa phần tử đầu của mỗi danh sách; mỗi lần lấy ra cái nhỏ nhất rồi nạp phần tử kế tiếp của danh sách đó. Chi phí $O(N \log k)$. Đây là thuật toán nền của việc sắp xếp ngoài (dữ liệu lớn hơn bộ nhớ) và của phép trộn trong nhiều hệ cơ sở dữ liệu.

Trung vị của luồng dữ liệu. Hai heap: một heap lớn nhất giữ nửa nhỏ, một heap nhỏ nhất giữ nửa lớn, giữ cho kích thước chênh nhau tối đa 1. Trung vị luôn ở đỉnh của một trong hai. Mẫu này — tách dữ liệu thành hai nửa, mỗi nửa một heap quay ngược nhau — còn dùng cho nhiều bài “duy trì phân vị”.

Cấu trúc	Lấy nhỏ nhất	Chèn	Tìm phần tử thứ k	Nên chọn khi
Mảng chưa sắp	$O(n)$	$O(1)$	$O(n)$	Dữ liệu tĩnh, ít truy vấn
Mảng đã sắp	$O(1)$	$O(n)$	$O(1)$	Xây một lần, truy vấn nhiều, không chèn
Heap nhị phân	$O(1)$	$O(\log n)$	<i>không hỗ trợ</i>	Chỉ cần cực trị, có chèn liên tục
Cây cân bằng (B/09)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Cần cả thứ tự lẫn cực trị

Bảng trên trả lời câu hỏi mở đầu thứ năm: heap thắng khi có chèn liên tục và chỉ cần cực trị; mảng đã sắp thắng khi dữ liệu tĩnh và cần truy vấn theo hạng; cây cân bằng đắt hơn cả hai nhưng làm được cả hai việc.

CẠM BẤY

Heap nhị phân chuẩn *không* hỗ trợ hai thao tác mà người ta hay tưởng là có: xoá một phần tử bất kỳ, và giảm khoá của một phần tử. Muốn có chúng cần lưu thêm chỉ số vị trí của từng phần tử, tức là một heap có đánh dấu. Trong lập trình thi đấu, mẹo thông dụng hơn là *xoá lười*: cứ đẩy bản ghi mới vào và khi lấy ra thì bỏ qua những bản ghi đã lỗi thời. Đây chính là lý do bản Dijkstra viết bằng hàng đợi ưu tiên thường có dòng kiểm tra “nếu khoảng cách lấy ra lớn hơn khoảng cách đã biết thì bỏ qua” (C/17).

7.4 Cùng một khung, ba cấu trúc, ba thuật toán

Đây là chỗ chương này trả công nhiều nhất về mặt tư duy. Xét khung chương trình sau: lấy một phần tử từ kho chờ, xử lý nó, đẩy các phần tử liên quan vào kho chờ, lặp lại. Đổi kho chờ, bạn được ba thuật toán khác nhau:

- **Ngăn xếp** → duyệt sâu. Đi tới cùng một nhánh rồi mới quay lại. Dùng để tìm thành phần liên thông, phát hiện chu trình, sắp thứ tự tô-pô.
- **Hàng đợi** → duyệt rộng. Xử lý theo lớp khoảng cách. Cho đường đi ngắn nhất trên đồ thị không trọng số.
- **Hàng đợi ưu tiên** → Dijkstra và A*. Xử lý theo thứ tự chi phí tăng dần. Cho đường đi ngắn nhất với trọng số không âm.

Nhận ra ba thuật toán này là một giúp bạn nhớ ít hơn và suy được nhiều hơn. Ví dụ, câu hỏi “vì sao Dijkstra cần trọng số không âm” trở thành hiển nhiên khi nhìn theo khung này: thuật toán chốt một đỉnh ngay khi lấy nó ra khỏi hàng đợi ưu tiên, tức là nó tin rằng không có đường nào rẻ hơn sẽ tới sau — và niềm tin ấy chỉ đúng nếu mọi cạnh đều làm chi phí tăng lên. Chương C/17 khai thác đầy đủ cách nhìn này.

7.5 Giảm khoá và bài học Fibonacci heap: khi cận và máy thật không cùng kết luận

Khối *Cạm bẫy* ở trên nói heap nhị phân không có thao tác giảm khoá. Câu hỏi tự nhiên là: có cấu trúc nào có không, và nó đáng giá bao nhiêu? Câu trả lời dẫn tới một trong những câu chuyện đáng học nhất của ngành cấu trúc dữ liệu.

Bắt đầu từ chỗ thao tác này thực sự tốn tiền. Dijkstra (C/17) mỗi lần thư giãn một cạnh là một lần muốn hạ nhãn của một đỉnh. Với heap nhị phân, cách làm phổ biến là đẩy thêm một bản ghi

mới rồi bỏ qua bản ghi lỗi thời khi lấy ra — nghĩa là hàng đợi chứa tới m phần tử và tổng chi phí là $O(m \log n)$. Fredman và Tarjan[36] đặt câu hỏi: nếu giảm khoá rẻ đi thì Dijkstra rẻ đi bao nhiêu? Với $O(1)$ cho giảm khoá và $O(\log n)$ cho lấy nhỏ nhất, tổng thành $O(m + n \log n)$ — tốt hơn hẳn khi đồ thị dày, vì lúc đó m lớn hơn $n \log n$ nhiều lần.

Fibonacci heap đạt hai cận đó theo nghĩa *khấu hao*, và cơ chế của nó là một minh hoạ đẹp cho phương pháp thể năng ở chương A/03: hoãn mọi việc dọn dẹp tới lúc bắt buộc. Chèn chỉ là thêm một cây vào danh sách gốc, không so sánh gì cả. Giảm khoá chỉ là *cắt* nút đó ra khỏi cha và biến nó thành một gốc mới — cũng không so sánh gì. Toàn bộ việc gộp các cây cùng bậc bị dồn sang lần lấy nhỏ nhất kế tiếp, và để cây không bị cắt tới mức xơ xác, mỗi nút bị mất con lần thứ hai sẽ bị cắt lên theo dây chuyền. Hàm thể năng đếm số cây gốc cùng số nút đã bị đánh dấu, và chính nó trả tiền cho những lần dọn dẹp đất đỏ.

Đến đây lý thuyết rất đẹp. Nhưng câu hỏi thực dụng — *có nên dùng không* — lại được trả lời bằng máy đo chứ không bằng chứng minh, và câu trả lời không đơn giản như hai phe vẫn nói. Larkin, Sen và Tarjan[41] cài mười biến thể heap theo cùng một chuẩn “không tối ưu hoá lắt léo” rồi đo trên ba loại tải: dãy thao tác nhân tạo, Dijkstra trên bản đồ đường thật, và một bộ mô phỏng sự kiện rời rạc. Bốn kết luận của họ đáng nhớ hơn bất kỳ bảng độ phức tạp nào.

Thứ nhất, thời gian chạy tương quan mạnh với *tỉ lệ trượt cache*, đặc biệt là cache L1 — nghĩa là quyết định thiết kế ở tầng cao nhất, “mảng hay con trỏ”, chi phối kết quả nhiều hơn cận khấu hao. Thứ hai, pairing heap thắng Fibonacci heap trên mọi tải mà họ thử: cùng ý tưởng hoãn dọn dẹp nhưng ít siêu dữ liệu hơn nên ít trượt cache hơn. Thứ ba, cả hai điều mà “ai cũng biết” đều sai ở một số tải: heap d phân lưu trong mảng *không* phải luôn tốt nhất, và Fibonacci heap *không* phải luôn chậm thảm hại — có tải nó vượt heap mảng. Thứ tư, và mỉa mai nhất, những biến thể được đề xuất về sau nhằm *đơn giản hoá* Fibonacci heap thường chạy chậm hơn chính bản gốc.

Bài học rút ra không phải “đừng tin lý thuyết” mà là một câu chính xác hơn: cận khấu hao đếm *số thao tác trên cấu trúc*, còn máy tính tiền theo *cách bố trí bộ nhớ*. Hai thước đo ấy chỉ trùng nhau khi mỗi thao tác có giá như nhau, mà điều đó không đúng từ khi có phân cấp cache (A/03). Vì vậy quy tắc thực hành gọn lại thành ba dòng: trong thi đấu, dùng heap nhị phân của thư viện kèm mẹo xoá lười; trong hệ thống thật mà hàng đợi ưu tiên nằm trên đường nóng, hãy đo với đúng tải của mình (E/26); và chỉ nghĩ tới Fibonacci heap khi đồ thị dày tới mức số lần giảm khoá lớn hơn hẳn số lần lấy nhỏ nhất — một điều kiện hiếm gặp hơn nhiều so với ấn tượng mà giáo trình để lại.

7.6 Gốc rễ: từ máy tính không có đệ quy tới lịch trình hệ điều hành

Ngăn xếp có một lịch sử gắn liền với việc làm cho đệ quy trở nên khả thi. Những máy tính đầu tiên không có cơ chế gọi hàm lồng nhau: địa chỉ trở về được ghi đè vào một ô cố định, nên hàm không thể gọi lại chính nó. Cuối thập niên 1950, khi ngành trình biên dịch cần đánh giá biểu thức toán học có dấu ngoặc lồng nhau, hai nhà nghiên cứu người Đức là Friedrich Bauer và Klaus Samelson đề xuất dùng một cấu trúc vào sau ra trước để lưu các phép toán đang chờ — và cùng thời gian đó, việc thiết kế ngôn ngữ Algol 60 với đệ quy đã biến ngăn xếp thành một phần của kiến trúc máy. Ngày nay “ngăn xếp lời gọi” là khái niệm mà mọi lập trình viên gặp mỗi lần đọc thông báo lỗi, nhưng nó là một phát minh có ngày tháng, không phải điều hiển nhiên.

Heap ra đời năm 1964 từ một nhu cầu rất cụ thể: J. W. J. Williams muốn một thuật toán sắp xếp tại chỗ với đảm bảo $O(n \log n)$ ở trường hợp xấu nhất — thứ mà quicksort không có — và ông thiết kế cấu trúc heap để phục vụ heapsort. Ngay sau đó, Robert Floyd đưa ra cách xây heap từ dưới lên trong $O(n)$. Điều thú vị là cấu trúc sinh ra để phục vụ sắp xếp lại trở nên nổi tiếng hơn ở vai trò khác: hàng đợi ưu tiên cho mô phỏng sự kiện rời rạc, cho lịch trình hệ điều hành, và cho các thuật toán tham lam trên đồ thị.

Câu chuyện đó lặp lại một lần nữa hai mươi năm sau. Fredman và Tarjan thiết kế Fibonacci heap [36] với mục tiêu rất hẹp: giảm độ phức tạp của Dijkstra từ $O(m \log n)$ xuống $O(m + n \log n)$, bằng cách làm cho thao tác giảm khoá tốn $O(1)$ khấu hao. Họ đạt được mục tiêu lý thuyết, nhưng hằng số lớn tới mức trong thực tế heap nhị phân thường vẫn nhanh hơn — một ví dụ kinh điển cho bài học “tiệm cận không phải tất cả” ở chương A/03, và cũng là ví dụ cho thấy một kết quả lý thuyết vẫn có giá trị dù ít khi được cài đặt: nó xác lập đâu là giới hạn.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Cuối thập niên 1950 — Bauer, Samelson và Algol 60	Đánh giá biểu thức có ngoặc lồng nhau; và làm cho hàm gọi được chính nó.	Ngăn xếp trở thành cấu trúc nền của trình biên dịch và của kiến trúc máy; “ngăn xếp lời gọi” ra đời.
1964 — Williams	Cần thuật toán sắp xếp tại chỗ có đảm bảo $O(n \log n)$ ở trường hợp xấu nhất.	Heap nhị phân và heapsort; cấu trúc sau đó nổi tiếng hơn ở vai trò hàng đợi ưu tiên.
1964 — Floyd	Xây heap bằng n lần chèn tốn $O(n \log n)$, có cách rẻ hơn không?	Xây từ dưới lên trong $O(n)$; ví dụ mẫu mực cho việc tổng của cấp số nhân giảm là hằng số.
Thập niên 1960–70 — mô phỏng sự kiện rời rạc, hệ điều hành	Cần lấy sự kiện sắp xảy ra sớm nhất trong hàng triệu sự kiện đang chờ.	Hàng đợi ưu tiên trở thành cấu trúc lõi của mô phỏng và của bộ lập lịch.
1984–87 — Fredman & Tarjan	Dijkstra bị chặn bởi chi phí giảm khoá.	Fibonacci heap: giảm khoá $O(1)$ khấu hao, cho $O(m + n \log n)$; nhưng hằng số lớn nên hiếm khi thắng trong thực tế.

7.7 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kiến trúc máy tính	Ngăn xếp lời gọi, khung ngăn xếp, tràn ngăn xếp	Không phải chỉ tương tự — đệ quy <i>được cài đặt</i> bằng đúng cấu trúc này. Hệ quả thực dụng: đệ quy sâu quá thì sập, và cách chữa là viết lại bằng ngăn xếp tường minh (E/25).
Trình biên dịch	Phân tích biểu thức, khớp ngoặc, ký pháp Ba Lan ngược	Ứng dụng lịch sử đầu tiên của ngăn xếp. Mọi bài “kiểm tra dãy ngoặc hợp lệ” trong phòng vẫn là phiên bản thu nhỏ của bộ phân tích cú pháp thật.
Hệ điều hành	Lịch trình tiến trình theo độ ưu tiên, hàng đợi sẵn sàng	Cùng cấu trúc hàng đợi ưu tiên. Khác ở ràng buộc: bộ lập lịch phải tránh bỏ đói tiến trình, nên độ ưu tiên thay đổi theo thời gian chờ — một yêu cầu không có trong bài toán thuật toán thuần túy.
Mô phỏng sự kiện rời rạc	Lịch sự kiện: luôn lấy sự kiện có thời điểm sớm nhất	Đây là ứng dụng “đúng nghĩa đen” của hàng đợi ưu tiên, và là lý do cấu trúc này quan trọng ngoài phạm vi thuật toán đồ thị.
Lập kế hoạch đường đi cho robot	A^* với biên là hàng đợi ưu tiên	Cùng khung “lấy ra, mở rộng, đẩy vào” ở mục 4, chỉ đổi khoá ưu tiên thành $g + h$. Bài học chuyển thẳng: chọn cấu trúc kho chờ chính là chọn thuật toán.
Xử lý dòng dữ liệu	Giữ k phần tử lớn nhất bằng heap kích thước k	Cùng mẫu ở mục 3.1. Ưu điểm quyết định trong bối cảnh luồng: bộ nhớ $O(k)$ không phụ thuộc độ dài luồng (D/24).

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Nén dữ liệu	Thuật toán Huffman: luôn gộp hai tần suất nhỏ nhất	Một thuật toán tham lam mà hàng đợi ưu tiên là công cụ thực thi trực tiếp. Xem C/15 cho phần chứng minh vì sao tham lam ở đây đúng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mỗi phần tử được đẩy vào ngăn xếp đúng một lần và bị lấy ra nhiều nhất một lần trong suốt cả thuật toán, nên tổng số thao tác không quá $2n$ bất kể vòng lặp trong chạy dài tới đâu ở một số lần lặp cụ thể. Đây là lập luận khấu hao kiểu tổng gộp (A/03): một lần lấy ra nhiều phần tử chỉ xảy ra được sau khi đã có nhiều lần đẩy vào tương ứng. Cùng dạng lập luận với “tổng quãng đường con trỏ” ở B/06.
- A2. Vì duy trì ít trật tự hơn thì rẻ hơn. Sắp xếp toàn phần đòi $O(n \log n)$ và mỗi lần chèn vào mảng đã sắp tồn $O(n)$; heap chỉ duy trì quan hệ cha-con nên chèn chỉ $O(\log n)$. Nếu bài toán chỉ hỏi “ai nhỏ nhất” — và rất nhiều thuật toán tham lam chỉ hỏi đúng thế — thì trả tiền cho trật tự toàn phần là lãng phí. Đây là ví dụ tiêu biểu của nguyên tắc chung ở phần B: chọn cấu trúc duy trì *đúng* lượng trật tự mà bài cần, không hơn.
- A3. Sai ở chỗ giả định mọi phần tử đều phải đi hết $\log n$ tầng. Khi xây từ dưới lên, một nửa số nút nằm ở tầng đáy và không phải đẩy xuống bước nào; một phần tư ở tầng kế và đi tối đa một bước; cứ thế. Tổng chi phí là $\sum_h (n/2^h) \cdot h$, một chuỗi hội tụ, cho $O(n)$. Còn cách chèn lần lượt thì ngược lại: phần tử vào sau đi từ đáy lên nên đúng là tốn $O(\log n)$ mỗi cái.
- A4. Vì thứ tự lấy ra quyết định thứ tự khám phá. Ngăn xếp lấy cái mới nhất nên thuật toán lao sâu theo một nhánh trước khi quay lại — hợp cho việc phát hiện chu trình và tính thứ tự tô-pô, vì thời điểm vào/ra của mỗi đỉnh mang thông tin cấu trúc. Hàng đợi lấy cái cũ nhất nên khám phá theo lớp khoảng cách tăng dần — đó chính xác là lý do duyệt rộng cho đường đi ngắn nhất trên đồ thị không trọng số. Cùng chương trình, đổi một dòng, đổi hẳn tính chất được đảm bảo.
- A5. Mảng đã sắp thẳng khi dữ liệu tĩnh: nó cho cả cực trị lẫn truy vấn phần tử thứ k trong $O(1)$, còn heap không trả lời được câu thứ hai. Heap thẳng khi có chèn xen kẽ liên tục, vì chèn vào mảng đã sắp tồn $O(n)$ còn heap chỉ $O(\log n)$; và heap còn thẳng khi bạn chỉ cần k phần tử đầu của một luồng rất dài, vì bộ nhớ chỉ $O(k)$. Cần cả hai khả năng thì phải trả giá cao hơn bằng cây cân bằng ở B/09.

NÓI LẠI THẬT ĐƠN GIẢN

Ba cấu trúc này chỉ khác nhau ở một câu hỏi: trong đồng việc đang chờ, làm việc nào trước? Ngăn xếp giống chồng giấy tờ trên bàn: bạn luôn xử lý tờ trên cùng, tức là tờ mới đặt xuống gần nhất. Kiểu này hợp khi công việc lồng nhau — đang làm việc A thì phát sinh việc B phải xong trước, xong B mới quay lại A. Đó cũng đúng là cách máy tính chạy hàm gọi hàm.

Hàng đợi giống hàng người chờ mua vé: ai tới trước phục vụ trước. Kiểu này hợp khi bạn muốn lan toả đều ra xung quanh, ví dụ “tìm những chỗ cách đây đúng ba bước” — vì mọi chỗ cách hai bước sẽ được xử lý xong trước khi chạm tới chỗ cách ba bước.

Heap giống phòng cấp cứu: không xét ai tới trước, mà xét ai nặng nhất. Điều đáng nói là phòng cấp cứu không cần xếp hạng toàn bộ bệnh nhân từ nặng tới nhẹ; nó chỉ cần biết *ai nặng nhất ngay lúc này*. Chính vì đòi hỏi ít hơn nên nó làm nhanh hơn — và đó là bài học chung: đừng sắp xếp mọi thứ khi bạn chỉ cần biết cái đứng đầu.

Còn ngăn xếp đơn điệu thì giống việc bạn xếp các cột chờ “ai sẽ cao hơn tôi”. Mỗi cột mới cao lên sẽ giải quyết xong một loạt cột thấp đang chờ, và những cột đó rời hàng vĩnh viễn. Vì mỗi cột vào hàng một lần và rời hàng một lần, tổng công việc chỉ tỉ lệ với số cột, dù có lúc một cột mới giải quyết hết cả chục cột cũ.

Một câu: Chọn cấu trúc kho chờ chính là chọn thuật toán: mới nhất, cũ nhất, hay nhỏ nhất.

Chỗ không rút gọn được: Việc heap không hỗ trợ “xoá phần tử bất kỳ” là hạn chế thật, không phải chi tiết cài đặt; muốn có nó thì phải trả bằng cấu trúc phức tạp hơn hoặc bằng mẹo xoá lười.

Thuật ngữ / câu hỏi	Trả lời
●●● Ba cấu trúc, ba quy tắc lấy ra	Ngăn xếp: mới nhất (LIFO). Hàng đợi: cũ nhất (FIFO). Heap: nhỏ nhất. Đối kho chờ trong cùng một khung thì được duyệt sâu, duyệt rộng, Dijkstra.
●●● Ngăn xếp đơn điệu — ý tưởng	Giữ các vị trí chưa tìm được đáp án theo thứ tự đơn điệu; phần tử mới giải quyết và loại bỏ mọi vị trí nhỏ hơn nó.
●●● Ngăn xếp đơn điệu — vì sao $O(n)$	Mỗi phần tử vào một lần, ra tối đa một lần; tổng $\leq 2n$ thao tác. Khẩu hao kiểu tổng gộp.
●●● Dấu hiệu dùng ngăn xếp đơn điệu	“Với mỗi phần tử, tìm phần tử gần nhất về một phía thoả quan hệ so sánh” — gồm cả biểu đồ cột, nước mưa, số đoạn mà phần tử là cực đại.
●●● Tính chất heap	Mỗi nút $\leq$ các con; lưu bằng mảng, con của i là $2i, 2i+1$. Không đảm bảo gì về quan hệ giữa hai nhánh; mảng không sắp xếp.
●●● Chi phí heap	Lấy nhỏ nhất $O(1)$; chèn và xoá gốc $O(\log n)$; xây từ n phần tử có sẵn $O(n)$.
●●● Vì sao xây heap là $O(n)$	Xây từ dưới lên: nửa số nút ở đáy không phải làm gì; tổng $\sum_h (n/2^h)h$ hội tụ. Cấp số nhân giảm bị chặn bởi hằng số (A/05).
●● Cực đại trên cửa sổ trượt	Deque chứa chỉ số có giá trị giảm dần; bỏ đầu khi rơi khỏi cửa sổ, bỏ cuối khi nhỏ hơn phần tử mới. $O(n)$ tổng cộng.
●● Ba mẫu dùng heap	k lớn nhất bằng heap kích thước k ($O(n \log k)$), bộ nhớ $O(k)$; trộn k danh sách đã sắp; trung vị luồng bằng hai heap ngược nhau.
●● Heap không hỗ trợ gì	Xoá phần tử bất kỳ và giảm khoá — cần heap có đánh dấu vị trí, hoặc dùng mẹo xoá lười (kiểm tra bản ghi lỗi thời khi lấy ra).
●● Vì sao Dijkstra cần trọng số không âm	Vì nó chốt đỉnh ngay khi lấy ra khỏi hàng đợi ưu tiên; niềm tin “không có đường rẻ hơn tới sau” chỉ đúng khi mọi cạnh làm chi phí tăng.
●● Heap so với mảng đã sắp	Mảng đã sắp: dữ liệu tĩnh, cần truy vấn theo hạng. Heap: có chèn liên tục, chỉ cần cực trị. Cần cả hai thì dùng cây cân bằng (B/09).
● Fibonacci heap	Giảm khoá $O(1)$ khẩu hao, cho Dijkstra $O(m + n \log n)$; hằng số lớn nên hiếm khi thắng trong thực tế.
●● Fibonacci heap có đáng dùng không	Cận $O(m + n \log n)$ cho Dijkstra là thật, nhưng đo thực nghiệm cho thấy pairing heap thắng nó trên mọi tải và thời gian chạy bám theo tỉ lệ trượt cache L1. Thi đấu: heap nhị phân kèm xoá lười.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi nên viết lại đệ quy thành vòng lặp với ngăn xếp tường minh?

| Chủ yếu khi độ sâu đệ quy có thể lớn — duyệt sâu trên đồ thị 10^5 đỉnh dạng đường thẳng sẽ làm tràn ngăn xếp trong nhiều môi trường. Lý do phụ nhưng cũng quan trọng: khi bạn cần lưu trạng thái tùy biến ở mỗi mức, hoặc cần tạm dừng và tiếp tục việc duyệt. Cái giá là code dài hơn và dễ sai hơn, nên đừng làm khi không cần: đệ quy dễ đọc hơn và khớp với chứng minh quy nạp (A/04).

2. Ngăn xếp đơn điệu tăng hay giảm — làm sao khỏi nhầm?

| Đừng nhớ theo tên, hãy suy từ mục tiêu. Nếu bạn tìm “phần tử lớn hơn kế tiếp”, thì những phần tử còn chờ đáp án phải là những phần tử chưa gặp ai lớn hơn — và nếu $a_i \leq a_j$ với $i < j$ thì a_i không thể còn chờ khi a_j đã vào, nên ngăn xếp giữ giá trị giảm dần từ đáy lên đỉnh. Cách kiểm tra nhanh: viết mảng bốn phần tử ra giấy và chạy tay — nhanh hơn nhớ quy tắc.

3. Tôi dùng hàng đợi ưu tiên cho Dijkstra và bị quá thời gian. Nguyên nhân hay gặp?

| Ba nguyên nhân, theo thứ tự tần suất. Một, quên bỏ qua bản ghi lỗi thời khi lấy ra — không có dòng kiểm tra đó thì mỗi đỉnh bị xử lý nhiều lần và số cạnh duyệt tăng vọt. Hai, đẩy vào hàng đợi mọi cạnh kể cả khi không cải thiện được khoảng cách. Ba, dùng cấu trúc nặng làm khoá (chuỗi, cặp lớn) khiến hàng số so sánh cao. Kiểm điểm một trước vì nó là lỗi phổ biến nhất.

4. Deque và hai heap — khi nào dùng cái nào cho bài cửa sổ trượt?

| Nếu bạn chỉ cần cực trị của cửa sổ, deque đơn điệu cho $O(n)$ và hằng số rất nhỏ — luôn chọn nó. Nếu bạn cần trung vị hoặc phần tử thứ k của cửa sổ thì deque không làm được, và bạn cần hai heap có xoá lười hoặc một cây cân bằng (B/09), chấp nhận $O(n \log n)$. Nguyên tắc chung: dùng cấu trúc yếu nhất còn trả lời được câu hỏi của bạn.

5. Vì sao heapsort ít được dùng dù có đảm bảo $O(n \log n)$ ở trường hợp xấu nhất?

| Vì nó truy cập bộ nhớ theo kiểu nhảy cóc giữa các tầng của cây, rất không thân thiện với cache, trong khi quicksort quét gần như tuần tự. Kết quả là quicksort thường nhanh hơn đáng kể trên máy thật dù tệ hơn trên giấy. Giải pháp thực tế của các thư viện là kết hợp: bắt đầu bằng quicksort, chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng — đúng ý tưởng introsort[37], một ví dụ đẹp cho chương E/26.

6. Có cấu trúc nào thay heap được không?

| Có vài lựa chọn tùy bối cảnh. Nếu khoá là số nguyên nhỏ và tăng dần theo thời gian — ví dụ Dijkstra với trọng số nhỏ — thì hàng đợi theo nhóm (bucket queue) cho $O(1)$ mỗi thao tác. Nếu chỉ có hai loại trọng số 0 và 1, deque thay được hoàn toàn và cho thuật toán 0–1 BFS (C/17). Nếu cần hợp nhất hai hàng đợi thường xuyên, các cấu trúc heap có thể hợp nhất như pairing heap sẽ hợp hơn. Câu hỏi luôn là: cấu trúc yếu nhất còn đủ dùng ở đây là gì.

7. Trong hệ thống thật, hàng đợi ưu tiên có nhược điểm gì mà bài tập không nhắc?

| Có, và nó quan trọng: sự bỏ đói. Nếu luôn phục vụ phần tử ưu tiên cao nhất, các phần tử ưu tiên thấp có thể chờ vô hạn. Hệ điều hành và bộ lập lịch tác vụ giải quyết bằng cách tăng dần độ ưu tiên theo thời gian chờ, tức là biến khoá thành hàm của cả nội dung lẫn thời gian. Đây là ví dụ điển hình của việc bài toán thật có thêm ràng buộc công bằng mà mô hình thuật toán thuần túy bỏ qua — chủ đề của E/26.

| Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Kiểm tra dãy ngoặc hợp lệ	LeetCode “Valid Parentheses”	Ứng dụng ngăn xếp cổ điển nhất; chú ý trường hợp còn dư ngoặc mở khi hết chuỗi.
● Trộn k danh sách đã sắp	LeetCode “Merge k Sorted Lists”	Heap kích thước k ; so sánh với cách trộn đôi một để thấy $O(N \log k)$ từ đâu ra.
● k phần tử lớn nhất trong luồng	LeetCode “Kth Largest Element in a Stream”	Heap nhỏ nhất kích thước k ; giải thích vì sao không dùng heap lớn nhất kích thước n .
●● Phần tử lớn hơn kế tiếp cho mọi vị trí	CSES “Nearest Smaller Values”[33]	Ngăn xếp đơn điệu; viết bất biến ra trước khi cài.
●● Cực đại trên mọi cửa sổ độ dài k	CSES “Sliding Window Cost”-loại; LeetCode “Sliding Window Maximum”	Deque đơn điệu; chú ý lưu chỉ số chứ không lưu giá trị.
●● Trung vị của luồng dữ liệu	LeetCode “Find Median from Data Stream”	Hai heap cân bằng; bất biến về chênh lệch kích thước là chỗ dễ sai nhất.
●● Nhiệm vụ và hạn chót, tối đa số việc làm được	Bài tập kinh điển	Tham lam kết hợp heap: nhận hết rồi bỏ việc tệ nhất khi quá tải — mẫu “tham lam có hối tiếc”.
●●● Hình chữ nhật lớn nhất trong biểu đồ cột	LeetCode “Largest Rectangle in Histogram”	Ngăn xếp đơn điệu; mẫu chót là hiểu rằng lúc một cột bị lấy ra thì cả hai biên của nó đã xác định.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Hình chữ nhật toàn số 1 lớn nhất trong ma trận nhị phân	LeetCode “Maximal Rectangle”	Quy về bài biểu đồ cột trên từng hàng — ví dụ đẹp về việc quy bài lạ về bài đã giải.
●●● Nước mưa mắc kẹt bằng ngăn xếp	LeetCode “Trapping Rain Water”	Đã gặp ở B/06 bằng hai con trỏ; lần này bắt buộc dùng ngăn xếp để thấy hai góc nhìn.
●●● Tổng của cực tiểu trên mọi đoạn con	LeetCode “Sum of Subarray Minimums”	Với mỗi phần tử, đếm số đoạn mà nó là cực tiểu bằng ngăn xếp đơn điệu; cẩn thận với phần tử bằng nhau để khỏi đếm trùng.
●●● Sắp xếp ngoài: trộn nhiều tệp đã sắp lớn hơn bộ nhớ	Bài tập hệ thống	Heap k đường; nối trực tiếp với mô hình bộ nhớ ngoài ở D/24.

Kết chương và đọc tiếp

Ba cấu trúc, ba quy tắc chọn phần tử tiếp theo, và một bài học chung đáng mang theo cả phần B: hãy dùng cấu trúc *yếu nhất mà vẫn trả lời được câu hỏi của bạn*. Heap rẻ hơn cây cân bằng vì nó biết ít hơn; deque rẻ hơn heap vì nó biết còn ít hơn nữa. Mỗi lần bạn đòi hỏi thêm một khả năng, bạn trả thêm tiền — và rất nhiều lời giải chậm là do đòi hỏi những khả năng không dùng tới.

Hai chương tiếp theo là hai câu trả lời đối lập cho bài toán tra cứu. Chương B/08 hỏi “làm sao tìm được phần tử theo khoá thật nhanh nếu tôi *không* cần thứ tự”, và câu trả lời là băm — kèm một cam bẫy an ninh mà rất ít giáo trình nói tới. Chương B/09 hỏi câu ngược lại: nếu tôi *cần* thứ tự thì phải trả thêm bao nhiêu.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Fredman & Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms”	Journal of the ACM, 1987	Nguồn của cận $O(m + n \log n)$ cho Dijkstra. Đọc mục mô tả cấu trúc và hàm thế năng; phần ứng dụng cho cây khung nhỏ nhất đọc sau cũng được.
nền tảng	Larkin, Sen, Tarjan, “A Back-to-Basics Empirical Study of Priority Queues”	ALENEX (SIAM), 2014	Số đo thật cho mười biến thể heap. Đọc phần kết luận và bảng Dijkstra trên bản đồ đường — đây là tài liệu chữa bệnh “chọn cấu trúc theo bảng tiệm cận”.
tra khi cần	Tarjan, “Amortized Computational Complexity”	SIAM Journal on Algebraic and Discrete Methods, 1985	Đọc khi muốn hiểu phương pháp thế năng đủ kỹ để tự phân tích một cấu trúc hoãn việc.
tra khi cần	Sedgewick & Wayne, <i>Algorithms</i> (4th ed.), chương 2.4	Addison-Wesley, 2011	Bản trình bày heap nhị phân kèm số đo và mã chạy được; hợp để đối chiếu cài đặt của bạn.

Băm và tập hợp (Hashing and Sets)

PHỤ THUỘC A/05 cho xác suất và nghịch lý ngày sinh; B/06 cho mẫu "tổng tiền tố cộng bảng băm". Chuẩn bị cho B/12 (băm chuỗi) và C/20 (ngẫu nhiên hoá).

TÓM TẮT

Băm là cách đổi một khoá bất kỳ thành một chỉ số mảng, để tra cứu trở thành truy cập trực tiếp. Nó cho $O(1)$ trung bình cho thêm, xoá và tìm — rẻ hơn hẳn cây tìm kiếm — nhưng đánh đổi ba thứ: không có thứ tự, không có đảm bảo trường hợp xấu nhất, và dễ bị tấn công có chủ đích. Chương này đi qua cách chọn hàm băm, hai cách xử lý va chạm, ý nghĩa của hệ số tải, nghịch lý ngày sinh và hệ quả của nó với băm chuỗi, cùng hai cấu trúc xác suất là Bloom filter và bộ đếm số phần tử phân biệt. Nếu chỉ nhớ một điều: bảng băm nhanh vì nó *giả định dữ liệu không thù địch*, và trong lập trình thi đấu cũng như trong hệ thống công khai, giả định đó bị phá bằng cách trộn một hạt giống ngẫu nhiên vào hàm băm.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hàm băm (hash function)** — hàm biến một khoá thành một số nguyên trong khoảng $[0, m)$. Yêu cầu duy nhất về mặt đúng đắn: khoá bằng nhau phải cho cùng giá trị; yêu cầu về hiệu năng: các khoá khác nhau nên rải đều.
- **va chạm (collision)** — hai khoá khác nhau cho cùng giá trị băm. Không tránh được vì số khoá khả dĩ luôn lớn hơn số ô.
- **hệ số tải (load factor)** — tỉ số giữa số phần tử và số ô của bảng, $\alpha = n/m$. Quyết định chi phí trung bình.
- **ngịch lý ngày sinh** — trong k giá trị lấy ngẫu nhiên từ N khả năng, xác suất có hai giá trị trùng nhau trở nên đáng kể khi $k \approx \sqrt{N}$.
- **băm đa thức cho chuỗi** — coi chuỗi như đa thức theo cơ số b , lấy giá trị theo modulo m : $H(s) = \sum s_i b^i \bmod m$. Cho phép tính băm của mọi đoạn con trong $O(1)$ sau tiền xử lý.
- **Bloom filter** — cấu trúc trả lời "phần tử này có trong tập không" với bộ nhớ rất nhỏ, không bao giờ báo sai "không có", nhưng đôi khi báo nhầm "có".

CÂU HỎI MỞ ĐẦU

- Q1. Bảng băm cho $O(1)$ trung bình còn cây cân bằng cho $O(\log n)$. Vì sao không phải lúc nào cũng dùng bảng băm?
- Q2. Vì sao một lời giải dùng `unordered_map` có thể bị đánh trượt trên Codeforces trong khi cùng lời giải đó dùng `map` thì qua?
- Q3. Băm chuỗi bằng một modulo $10^9 + 7$ nghe rất an toàn vì xác suất trùng chỉ 10^{-9} . Vì sao trong bài có 10^5 chuỗi thì lập luận đó sai?
- Q4. Bloom filter có thể báo nhầm "có" nhưng không bao giờ báo nhầm "không có". Cấu trúc nào của nó tạo ra sự bất đối xứng đó, và vì sao bất đối xứng lại hữu ích?
- Q5. Trong hệ phân tán, thêm một máy chủ vào cụm làm hầu hết khoá phải chuyển chỗ nếu dùng `hash mod n`. Cách sửa là gì?

Bài toán tra cứu là bài toán phổ biến nhất trong lập trình: cho một khoá, tìm giá trị đi kèm. Nếu khoá là số nguyên nhỏ, lời giải hiển nhiên là dùng mảng — khoá chính là chỉ số, tra cứu tốn một phép truy cập. Toàn bộ ý tưởng băm là mở rộng lời giải hiển nhiên ấy cho khoá bất kỳ: nếu khoá không phải chỉ số, hãy ép nó thành chỉ số bằng một hàm.

Cái giá của việc ép đó là va chạm, vì số khoá khả dĩ luôn nhiều hơn số ô. Toàn bộ kỹ thuật băm — chọn hàm, xử lý va chạm, chọn kích thước bảng — chỉ là cách sống chung với va chạm sao cho chi phí trung bình vẫn gần $O(1)$. Và có một tình tiết mà giáo trình hay bỏ qua nhưng lại quyết định trong thực tế: mọi phân tích “trung bình” ở đây đều giả định dữ liệu không được chọn bởi người muốn hại bạn.

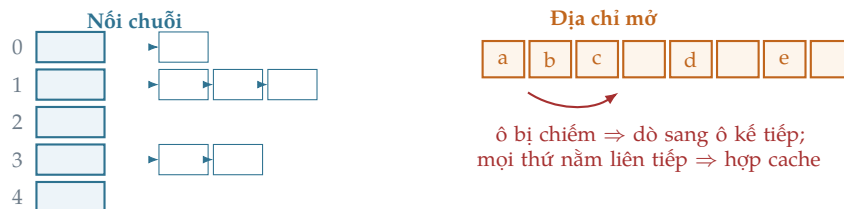
8.1 Hàm băm và hai cách sống chung với va chạm

Một hàm băm tốt cần hai tính chất: tính được nhanh, và rải các khoá thực tế đều trên các ô. Tính chất thứ hai quan trọng hơn về ngoài của nó, vì các khoá thực tế thường có cấu trúc — toạ độ trên lưới, chuỗi có tiền tố chung, số bội của một hằng — và một hàm băm ngây thơ sẽ dồn chúng vào ít ô.

Khi va chạm xảy ra, có hai cách xử lý.

Nối chuỗi: mỗi ô giữ một danh sách các phần tử băm vào đó. Đơn giản, chịu được hệ số tải lớn hơn 1, nhưng mỗi phần tử tốn thêm một con trỏ và việc lần theo danh sách không thân thiện với cache.

Địa chỉ mở: mọi phần tử nằm ngay trong mảng; nếu ô đã bị chiếm thì dò sang ô kế tiếp theo một quy tắc. Bộ nhớ gọn, truy cập tuần tự nên rất hợp cache — các bảng băm nhanh nhất trong thực tế đều dùng cách này. Cái giá: hệ số tải phải giữ dưới khoảng 0,7 nếu không chi phí bùng nổ, và việc xoá phần tử phức tạp hơn (phải đánh dấu “đã xoá” thay vì để trống).



Hình 8.1. Hai cách xử lý va chạm. Nối chuỗi đơn giản và chịu được tải cao, nhưng mỗi lần lần theo con trỏ là một lần có nguy cơ trượt cache. Địa chỉ mở giữ mọi thứ trong một mảng liên tiếp nên nhanh hơn đáng kể trên máy thật, đổi lại phải giữ hệ số tải thấp và xử lý xoá bằng cách đánh dấu.

Về chi phí: với nối chuỗi, độ dài danh sách trung bình là α , nên chi phí tìm là $O(1 + \alpha)$. Với địa chỉ mở và dò tuyến tính, chi phí trung bình tăng vọt khi α tiến gần 1 — đó là lý do bảng băm phải *tự mở rộng*: khi vượt ngưỡng tải, cấp phát bảng lớn gấp đôi và băm lại toàn bộ. Chi phí băm lại là $O(n)$ nhưng khấu hao thành $O(1)$ mỗi thao tác, đúng lập luận nhân đôi ở chương [A/03](#).

8.2 Chỗ mà giáo trình nói “trung bình” và thực tế nói “kẻ tấn công”

Đây là mục quan trọng nhất chương, và cũng là mục hay bị bỏ.

Mọi phân tích trên đều giả định các khoá rải đều trên các ô. Nhưng hàm băm là một hàm *tất định*: nếu ai đó biết hàm băm của bạn, họ có thể chuẩn bị một tập khoá mà tất cả cùng băm vào

một ô. Khi đó bảng băm thoái hoá thành một danh sách liên kết, và mọi thao tác thành $O(n)$. Với $n = 10^5$ phần tử, lời giải $O(n)$ trên mỗi thao tác trở thành 10^{10} — quá thời gian chắc chắn.

Điều này không phải chuyện lý thuyết. Trong lập trình thi đấu, các bộ test chống `unordered_map` của C++ được viết ra chính bằng cách này và rất phổ biến. Trong hệ thống thật, hiện tượng tương tự từng được dùng để tấn công máy chủ web: gửi một yêu cầu chứa hàng nghìn tham số cùng băm vào một ô làm máy chủ tiêu tốn thời gian gấp hàng nghìn lần bình thường.

Cách chữa giống nhau ở cả hai bối cảnh: *làm cho hàm băm không đoán trước được*, bằng cách trộn vào một hạt giống ngẫu nhiên chọn lúc chạy. Về mặt lý thuyết, ý tưởng này có tên là *băm phổ dụng* — chọn ngẫu nhiên một hàm từ một họ hàm có tính chất “với hai khoá bất kỳ, xác suất va chạm nhỏ” — và điểm mấu chốt là ngẫu nhiên nằm ở *lựa chọn của bạn*, không ở dữ liệu. Nhờ vậy đảm bảo vẫn đúng ngay cả khi dữ liệu do đối thủ chọn, miễn là họ không biết hạt giống. Đây là một ví dụ mẫu mực cho toàn bộ chương C/20: ngẫu nhiên hoá không phải để hy vọng gặp may, mà để *tước bỏ khả năng dự đoán của đối thủ*.

CẠM BẤY

Trong C++, `unordered_map` với khoá số nguyên dùng một hàm băm rất đơn giản và có thể đoán được, nên nó bị đánh trượt thường xuyên trên các kỳ thi. Cách chữa tiêu chuẩn là cung cấp hàm băm riêng có trộn hạt giống lấy từ đồng hồ hệ thống. Trong ngôn ngữ khác, hãy kiểm tra xem thư viện chuẩn có ngẫu nhiên hoá hạt giống theo tiến trình hay không — nhiều ngôn ngữ hiện đại đã làm sẵn việc này sau các đợt tấn công vào máy chủ web.

8.3 Bảo đảm mạnh hơn: băm cuckoo và băm bảng tra

Băm phổ dụng cứu được trường hợp xấu *kỳ vọng*, nhưng vẫn không hứa gì về một thao tác cụ thể: một lần tra vẫn có thể phải lần qua một chuỗi dài. Với hệ thống có hạn kỳ thời gian — bộ định tuyến, bộ điều khiển, phần cứng — kỳ vọng là không đủ. Hai kết quả sau cho thấy có thể đòi hỏi nhiều hơn, và cả hai đều đáng biết vì chúng đổi cách nghĩ chứ không chỉ thêm một cấu trúc.

Băm cuckoo[42] lật ngược câu hỏi: thay vì cho phép một khoá nằm ở nhiều chỗ tùy theo lịch sử va chạm, hãy quy định mỗi khoá chỉ được nằm ở *đúng hai* vị trí, cho bởi hai hàm băm. Khi đó tra cứu là hai lần đọc bộ nhớ, không hơn — $O(1)$ trong *trường hợp xấu nhất*, và xoá cũng đơn giản vì không có ô “đã xoá” nào phải đánh dấu. Cái giá đền hết sang phép chèn: nếu cả hai vị trí đều bị chiếm, ta *đá* kẻ đang ngồi ở một trong hai ô sang vị trí thay thế của nó, và kẻ bị đá lại đá tiếp người khác. Chuỗi đá này kết thúc nhanh nếu bảng còn đủ rộng — điều kiện quen thuộc là hệ số tải dưới $1/2$ — còn nếu nó dài quá ngưỡng thì ta bốc lại hai hàm băm mới và xây lại bảng. Kết quả: chèn tốn $O(1)$ kỳ vọng theo nghĩa khấu hao, đổi lấy bộ nhớ rộng gấp đôi và một cái đuôi hiếm hoi rất đắt.

Điểm đáng học nằm ở chỗ đánh đổi được *dời* chứ không biến mất: cùng một bài toán, nhưng chi phí bị đẩy hết từ phía đọc sang phía ghi. Đó là lựa chọn đúng cho các bảng viết một lần đọc rất nhiều lần, và là lựa chọn sai cho tài ghi liên tục.

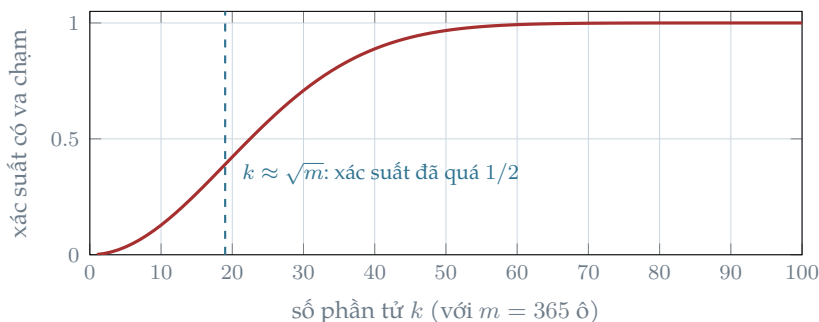
Băm bảng tra[43] trả lời một câu hỏi khác: hàm băm phải “ngẫu nhiên” tới mức nào thì các phân tích lý thuyết mới còn đúng? Cách làm đơn giản tới mức đáng ngờ — cất khoá thành vài ký tự, mỗi ký tự tra một bảng số ngẫu nhiên đã dựng sẵn, rồi XOR tất cả lại. Hàm như vậy chỉ đọc lập bậc ba, thấp hơn nhiều so với mức mà các chứng minh cổ điển đòi hỏi, nên theo lý thuyết cũ thì nó không đủ tốt. Pătraşcu và Thorup chứng minh rằng nó vẫn cho các chặn tập trung kiểu Chernoff, và nhờ đó dò tuyến tính lẫn băm cuckoo chạy đúng như phân tích với một hàm băm chỉ tốn vài phép tra bảng và một phép XOR.

Ba dòng đáng mang theo từ mục này. Một, “ngẫu nhiên hoá” không phải một mức duy nhất mà là cả một thang, và câu hỏi kỹ thuật luôn là *cần độc lập tới bậc mấy*. Hai, hàm băm rẻ kèm một phân tích tinh vi thường thắng hàm băm đắt kèm một phân tích thô. Ba, khi bài toán đòi bảo đảm cho *từng* thao tác chứ không cho trung bình, hãy hỏi xem có cấu trúc nào chuyển chi phí sang phía còn lại không — đây là mẫu tư duy dùng lại được, không riêng gì bảng băm.

8.4 Nghịch lý ngày sinh và băm chuỗi

Băm chuỗi là công cụ mạnh: coi chuỗi như một đa thức, ta tính được giá trị băm của *mọi* đoạn con trong $O(1)$ sau khi tiền xử lý $O(n)$, nên so sánh hai đoạn con bất kỳ trở thành $O(1)$. Rất nhiều bài chuỗi khó sụp xuống thành dễ nhờ điều đó (B/12).

Nhưng có một cái bẫy xác suất mà nhiều người tính sai. Nếu modulo là $10^9 + 7$, xác suất hai chuỗi cụ thể trùng băm là khoảng 10^{-9} — nghe rất an toàn. Sai lầm là ở chỗ bài toán thường so sánh *rất nhiều* cặp. Với k chuỗi, số cặp là $k^2/2$, nên xác suất có ít nhất một va chạm xấp xỉ $k^2/(2m)$. Với $k = 10^5$ và $m = 10^9$, con số đó là khoảng 5, tức là gần như chắc chắn có va chạm. Đây chính là nghịch lý ngày sinh: va chạm trở nên đáng kể khi số phần tử đạt cỡ $\sqrt{m}$, chứ không phải cỡ m .



Hình 8.2. Nghịch lý ngày sinh với $m = 365$. Trực giác nói phải cần cỡ $m/2$ phần tử mới có va chạm; thực tế chỉ cần cỡ $\sqrt{m}$, tức khoảng 23 người. Hệ quả trực tiếp cho băm chuỗi: modulo 10^9 chỉ “an toàn” tới cỡ $3 \cdot 10^4$ chuỗi, nên bài có 10^5 chuỗi cần hai modulo hoặc một modulo cỡ 10^{18} .

Hai cách chữa. Dùng *hai modulo* và coi cặp giá trị là mã băm — xác suất va chạm trở thành tích của hai xác suất, đủ nhỏ cho mọi bài thực tế. Hoặc dùng *một modulo* cỡ 10^{18} với phép nhân cẩn thận để không tràn. Và như mục trước: cơ sở của đa thức phải chọn *ngẫu nhiên lúc chạy*, vì băm chuỗi với cơ sở cố định cũng bị dựng test phá.

8.5 Cấu trúc xác suất: chấp nhận sai một chiều để tiết kiệm bộ nhớ

Khi tập dữ liệu quá lớn để lưu, có một hướng đi rất khác: chấp nhận trả lời sai với xác suất nhỏ, đổi lấy bộ nhớ nhỏ hơn nhiều bậc.

Bloom filter là ví dụ kinh điển. Ta giữ một mảng bit và k hàm băm. Thêm phần tử: bật k bit tương ứng. Hỏi phần tử: nếu có bit nào bằng 0 thì chắc chắn phần tử chưa từng được thêm; nếu mọi bit đều bằng 1 thì *có lẽ* đã thêm — có thể các bit đó do những phần tử khác bật lên. Sự bất đối xứng ấy chính là điều làm nó hữu ích: nó dùng ở vị trí bộ lọc đứng trước một phép tra cứu đắt tiền, nơi “chắc chắn không có” cho phép bỏ qua ngay, còn “có lẽ có” chỉ tốn thêm một lần kiểm tra thật. Cơ sở dữ liệu dùng nó để tránh đọc đĩa vô ích; hệ thống mạng dùng nó để lọc yêu cầu.

Đếm số phần tử phân biệt là bài toán họ hàng. Đếm chính xác cần bộ nhớ tỉ lệ với số phần tử phân biệt; nhưng nếu chấp nhận sai số vài phần trăm thì có thể dùng vài kilobyte cho hàng tỉ phần tử.

Ý tưởng gốc rất đẹp: nếu các giá trị băm rải đều, thì giá trị băm nhỏ nhất mà bạn nhìn thấy nói cho bạn biết bạn đã nhìn bao nhiêu phần tử phân biệt — càng nhiều phần tử thì cực tiểu càng nhỏ. Cấu trúc HyperLogLog[44] là bản tinh chỉnh của ý tưởng đó và được dùng rộng rãi trong các hệ thống đo lường.

8.6 Phản biện, nhược điểm và rủi ro triển khai

Bảng băm thường được ca tụng là cấu trúc thần thánh cho phép truy cập $O(1)$. Nhưng con số $O(1)$ đó đi kèm một hợp đồng ngầm có rất nhiều điều khoản bất lợi về phần cứng, bảo mật và tính ổn định.

Phản biện: ảo tưởng về $O(1)$ của thư viện chuẩn. Trong C++, `std::unordered_map` là một trong những container gây thất vọng nhất về hiệu năng thực tế. Thư viện chuẩn bắt buộc dùng phương pháp tách chuỗi (chaining) để đảm bảo iterator không bị vô hiệu hoá khi băm lại. Hệ quả là mỗi phần tử được bọc trong một nút danh sách liên kết cấp phát động riêng biệt trên heap. Trên các kích thước dữ liệu vừa và nhỏ ($N \leq 100$), chi phí cấp phát bộ nhớ và hiện tượng trượt cache liên tục (pointer chasing) khiến `std::unordered_map` chạy chậm hơn đáng kể so với `std::map` (cây đỏ-đen), dù về mặt tiệm cận cây đỏ-đen tốn $O(\log N)$. Chỉ các bảng băm dùng địa chỉ mở hiện đại với mảng phẳng (như `absl::flat_hash_map` của Google hay `robin_hood::unordered_map`) mới thực sự đạt được tốc độ của phần cứng hiện đại.

Nhược điểm cốt lõi về tài nguyên và kiến trúc:

- *Không có thứ tự và tính địa phương:* Bảng băm phá huỷ hoàn toàn thứ tự của các khoá. Nó bất lực trước các truy vấn cơ bản như “tìm khoá nhỏ nhất lớn hơn x ”, “duyệt các phần tử theo thứ tự tăng dần”, hay “truy vấn đoạn $[l, r]$ ”. Bất kỳ bài toán nào có yêu cầu về khoảng cách hay lân cận đều buộc phải dùng cây nhị phân tìm kiếm (B/09) hoặc cây chỉ số (B/10).
- *Chi phí ẩn của khoá phức tạp:* Thời gian tính băm tỉ lệ thuận với độ dài của khoá. Băm một chuỗi dài L ký tự tốn chi phí $O(L)$. Do đó, $O(1)$ của bảng băm thực chất là $O(L)$; coi thao tác này là một phép tính hằng số trên chuỗi dài là sai lầm làm chương trình chậm hơn dự tính hàng chục lần.
- *Lựa chọn sai lầm khi miền giá trị nhỏ:* Khi tập khoá là các số nguyên nằm trong một khoảng cố định (ví dụ $0 \dots 10^6$), dùng bảng băm là một quyết định kỹ thuật tồi. Một mảng tĩnh `int a[1000005]` chỉ tốn 4 MB, không bao giờ có va chạm, không tốn chi phí tính toán hàm băm và tận dụng tuyệt đối bộ nhớ đệm CPU.

Rủi ro vận hành và các lỗi sập hệ thống nghiêm trọng:

- *Tấn công suy biến (Hash Collision DoS / Test phá):* Trong các thư viện chuẩn (như `libstdc++` của GNU), hàm băm mặc định cho số nguyên chỉ là hàm đồng nhất (`hash(x) = x`), và kích thước bảng thường là các số nguyên tố cố định có thể đoán trước. Đối thủ hoặc người ra đề chỉ cần đưa vào các số nguyên cách nhau đúng một chu kỳ bảng, toàn bộ dữ liệu sẽ rơi vào cùng một bucket duy nhất. Bảng băm thoái hoá thành danh sách liên kết, chi phí mỗi thao tác từ $O(1)$ nhảy vọt lên $O(n)$, đẩy tổng thời gian lên $O(n^2)$ và làm chương trình bị quá thời gian (TLE) hoặc sập máy chủ vì nghẽn CPU (HashDoS). Cách phòng thủ bắt buộc: dùng hàm băm tùy biến kết hợp với hạt giống ngẫu nhiên lấy từ đồng hồ hệ thống (`chrono::steady_clock`) lúc chạy chương trình.
- *Gai độ trễ khi băm lại (Rehashing Latency Spike):* Khi số phần tử vượt ngưỡng hệ số tải (load factor, mặc định 1.0), bảng băm buộc phải cấp phát mảng mới lớn gấp đôi và phân bổ lại toàn bộ phần tử cũ. Bước băm lại này tốn chi phí $O(n)$. Trong các hệ thống điều khiển thời gian thực (như robot, xe tự hành, giao dịch tài chính), một thao tác đột ngột tốn vài chục mili-giây sẽ làm vỡ hạn kỳ điều khiển chu kỳ kín (deadline miss) và gây tai nạn nghiêm trọng.

- *Hiện tượng rò rỉ mộ phần (Tombstone Accumulation)*: Trong các bảng băm dùng địa chỉ mở (open addressing), khi xoá một phần tử ta không thể xoá sạch ô nhớ vì sẽ làm đứt chuỗi dò tìm của các phần tử chen sau; ta buộc phải đánh dấu ô đó là “đã xoá” (tombstone). Nếu hệ thống thực hiện hàng triệu thao tác chen và xoá xen kẽ, các tombstone sẽ lấp kín bảng, làm chuỗi dò tìm dài ra vô tận và hiệu năng tra cứu tụt dốc thê thảm dù số phần tử thực tế trong bảng rất ít.

CẠM BẦY

Không bao giờ dùng `std::unordered_map<long long, int>` mặc định trong các kỳ thi có hack (như Codeforces) hoặc trên các API công khai tiếp nhận đầu vào của người dùng mà không có custom hash. Việc tạo dữ liệu đối kháng để ép bảng băm chạy $O(n^2)$ là cực kỳ dễ dàng và được tự động hoá hoàn toàn bởi đối thủ.

8.7 Gốc rễ: từ bảng ký hiệu của trình dịch hợp ngữ tới hệ phân tán

Băm ra đời cùng lúc với những trình biên dịch đầu tiên, và vì một lý do rất cụ thể. Trình dịch hợp ngữ cần một *bảng ký hiệu*: ánh xạ tên biến sang địa chỉ. Tên biến là chuỗi, bộ nhớ thì đánh số, nên cần một câu nối — và câu nối đó là hàm băm. Ý tưởng được ghi nhận trong một ghi chú nội bộ của Hans Peter Luhn tại IBM năm 1953, và kỹ thuật địa chỉ mở xuất hiện ngay sau đó trong các trình dịch hợp ngữ giữa thập niên 1950. Đây là một ví dụ điển hình cho mẫu lặp đi lặp lại trong lịch sử ngành: cấu trúc dữ liệu quan trọng thường sinh ra từ nhu cầu của công cụ hệ thống, không từ lý thuyết.

Phần lý thuyết đến sau và giải một vấn đề thật. Suốt hai thập niên, người ta biết băm nhanh “trung bình” nhưng không có cách nào đảm bảo điều đó khi dữ liệu không ngẫu nhiên. Năm 1979, Carter và Wegman đề xuất băm phổ dụng: thay vì hy vọng dữ liệu ngẫu nhiên, hãy *chọn hàm băm một cách ngẫu nhiên*. Đây là một chuyển dịch tư duy quan trọng và nó lặp lại ở nhiều nơi khác trong quyển sách này — quicksort với chốt ngẫu nhiên (C/14) là cùng một ý.

Bloom filter (1970) sinh ra từ ràng buộc bộ nhớ của thời đó: kiểm tra một từ có nằm trong từ điển gạch nổi hay không, khi từ điển không nạp hết vào bộ nhớ được. Bài toán cũ đó biến mất, nhưng cấu trúc thì sống rất khoẻ trong các hệ thống hiện đại, ở đúng vai trò “lọc trước một phép tra cứu đắt tiền”.

Cuối cùng, một gốc rễ hiện đại hơn và có ảnh hưởng lớn: *băm nhất quán*, do nhóm của Karger đề xuất năm 1997 cho bài toán bộ nhớ đệm web phân tán. Vấn đề rất thực tế: nếu ánh xạ khoá tới máy chủ bằng hash mod n thì thêm hay bớt một máy làm gần như *toàn bộ* khoá đổi chỗ. Băm nhất quán xếp cả khoá lẫn máy chủ lên một vòng tròn và giao mỗi khoá cho máy chủ kế tiếp theo chiều kim đồng hồ; khi thêm một máy, chỉ các khoá thuộc một cung bị ảnh hưởng. Kỹ thuật này về sau trở thành nền của nhiều hệ cơ sở dữ liệu phân tán, và nó là câu trả lời cho câu hỏi mở đầu thứ năm.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1953 — Luhn (IBM)	Bảng ký hiệu của trình dịch: ánh xạ tên biến (chuỗi) sang địa chỉ (số).	Ý tưởng băm; ra đời từ nhu cầu công cụ hệ thống chứ không từ lý thuyết.
Giữa thập niên 1950 — các trình dịch hợp ngữ	Bộ nhớ rất hẹp, không đủ chỗ cho danh sách liên kết mỗi ô.	Địa chỉ mở với dò tuyến tính; cách cài vẫn nhanh nhất trên phần cứng hiện đại vì hợp cache.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1970 — Bloom	Kiểm tra từ có trong từ điển không, khi từ điển không nạp hết vào bộ nhớ.	Bloom filter: chấp nhận sai một chiều để giảm bộ nhớ nhiều bậc; nay dùng làm bộ lọc trước truy cập đĩa.
1979 — Carter & Wegman	“Trung bình nhanh” không còn đúng khi dữ liệu do người khác chọn.	Băm phổ dụng: ngẫu nhiên hoá <i>lựa chọn hàm</i> , không phụ thuộc dữ liệu; nền cho cách chống test phá ngày nay.
1997 — Karger và cộng sự	Bộ nhớ đệm web phân tán: thêm một máy chủ làm mọi khoá đổi chỗ.	Băm nhất quán; trở thành cơ sở phân mảnh dữ liệu của nhiều hệ phân tán hiện đại.
2007 — Flajolet và cộng sự	Đếm số phần tử phân biệt trong luồng hàng tỉ bản ghi với vài kilobyte.	HyperLogLog; hoàn thiện dòng nghiên cứu ước lượng lực lượng bắt đầu từ thập niên 1980.

8.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Hệ phân tán	Băm nhất quán, phân mảnh dữ liệu	Cùng hàm băm, khác mục tiêu: ở đây không phải để tra cứu nhanh mà để <i>ổn định ánh xạ</i> khi số máy thay đổi. Bài học: khi tập ô có thể thay đổi, $\text{mod } n$ là lựa chọn tệ.
Hệ thống kiểm soát phiên bản	Git định danh mọi đối tượng bằng mã băm nội dung	Băm dùng làm <i>địa chỉ</i> chứ không làm chỉ số: nội dung giống nhau thì tự động chung một tên. Điều kiện để dùng được là hàm băm mật mã, tức va chạm phải khó tạo ra có chủ đích — yêu cầu mạnh hơn hẳn băm thông thường.
Mật mã	Hàm băm mật mã, cây Merkle, chữ ký	Cùng tên gọi nhưng yêu cầu khác hẳn: ở đây bắt buộc <i>không ai tạo được va chạm</i> , kể cả khi cố tình và có nhiều tài nguyên. Vì vậy chúng chậm hơn nhiều và không nên dùng làm hàm băm cho bảng băm.
Cơ sở dữ liệu	Nối bằng băm (hash join), chỉ mục băm, Bloom filter chặn đọc đĩa	Cùng đánh đổi với chương này: chỉ mục băm nhanh hơn chỉ mục cây cho truy vấn bằng, nhưng vô dụng cho truy vấn khoảng — đó là lý do B-tree vẫn là mặc định (B/09).
Nhận dạng ảnh, tìm gần đúng	Băm nhảy cảm cục bộ (LSH): khoá gần nhau thì băm giống nhau	Đảo ngược yêu cầu của chương này. Băm thường muốn rải đều để tránh va chạm; LSH <i>cố ý</i> tạo va chạm cho các phần tử tương tự, để tìm hàng xóm gần trong không gian nhiều chiều.
SLAM và nhận dạng địa điểm	Túi từ thị giác, cây từ vừng, mô tả nhị phân	Cùng ý tưởng LSH: gán mỗi đặc trưng ảnh vào một “ô” để so khớp nhanh giữa hàng nghìn khung hình. Điểm giống quan trọng: cả hai đều chấp nhận sai sót và bù lại bằng bước kiểm chứng hình học phía sau — kiến trúc “lọc rẻ trước, kiểm đất sau” giống hệt Bloom filter.
Chống trùng lặp dữ liệu	Lưu trữ theo nội dung, phát hiện tệp trùng	Băm để nhận diện thay vì để tra cứu. Cùng lưu ý về nghịch lý ngày sinh: số đối tượng càng lớn thì độ dài mã băm cần thiết càng phải tăng theo bình phương.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì $O(1)$ của bảng băm là trung bình chứ không phải đảm bảo, và vì nó mất thứ tự. Ba tình huống buộc phải dùng cây: bài cần truy vấn “khoá gần nhất”, “lớn hơn hoặc bằng x ” hay duyệt theo thứ tự; hệ thống có hạn kỳ thời gian thực không chịu được một thao tác đột ngột $O(n)$; và trường hợp khoá do đối thủ chọn mà bạn không ngẫu nhiên hoá được hàm băm. Ngoài ra, khi khoá là số nguyên trong khoảng nhỏ thì mảng thường thắng cả hai.
- A2. Vì hàm băm mặc định cho số nguyên trong nhiều cài đặt là hàm đồng nhất hoặc gần như vậy, và kích thước bảng có thể đoán được. Người viết test chọn một tập khoá cùng số dư theo kích thước bảng, làm mọi khoá dồn vào một ô; bảng băm thoái hoá thành danh sách và mỗi thao tác thành $O(n)$. Cây cân bằng không có điểm yếu này vì $O(\log n)$ của nó là đảm bảo cho mọi dữ liệu. Cách chữa là trộn hạt giống ngẫu nhiên lấy lúc chạy vào hàm băm.
- A3. Vì cái đáng lo không phải một cặp mà là *mọi cặp*. Với k chuỗi có $k^2/2$ cặp, nên xác suất có ít nhất một va chạm xấp xỉ $k^2/(2m)$. Với $k = 10^5$ và $m = 10^9$ thì con số này lớn hơn 1, tức gần như chắc chắn có va chạm. Đây là nghịch lý ngày sinh: ngưỡng nguy hiểm là $\sqrt{m}$, không phải m . Chữa bằng hai modulo hoặc một modulo cỡ 10^{18} .
- A4. Bất đối xứng đến từ chỗ Bloom filter chỉ *bật* bit chứ không bao giờ tắt: nếu một bit cần thiết đang bằng 0 thì chắc chắn phần tử chưa từng được thêm, còn mọi bit bằng 1 thì có thể do các phần tử khác bật. Điều đó hữu ích vì nó đặt cấu trúc vào đúng vai trò bộ lọc: câu trả lời “chắc chắn không” cho phép bỏ qua ngay một phép tra cứu đắt tiền, còn câu trả lời “có lẽ có” chỉ tốn thêm một lần kiểm tra thật — sai lầm không gây kết quả sai, chỉ gây thêm việc.
- A5. Băm nhất quán. Thay vì hash mod n , xếp cả khoá lẫn máy chủ lên một vòng tròn giá trị băm, và giao mỗi khoá cho máy chủ đầu tiên gặp theo chiều kim đồng hồ. Thêm một máy chủ làm các khoá trong một cung phải chuyển, tức trung bình $1/n$ số khoá thay vì gần như toàn bộ. Trong thực tế người ta còn đặt nhiều “nút ảo” cho mỗi máy để tải rải đều hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Băm chỉ là cách biến một cái tên thành một số ngắn tử. Nếu tử của bạn có 100 ngắn và bạn có quy tắc “lấy tên, tính ra một số từ 0 tới 99”, thì cất và tìm đồ đều chỉ mất một bước: tính số rồi mở đúng ngắn. Nhanh hơn hẳn việc mở lần lượt từng ngắn.

Rắc rối duy nhất là hai cái tên khác nhau có thể ra cùng một số ngắn. Có hai cách sống chung. Cách thứ nhất: mỗi ngắn treo một cái túi, ai vào cùng ngắn thì bỏ chung túi, lúc tìm thì lục túi. Cách thứ hai: nếu ngắn đã có người, đi sang ngắn kế bên cho tới khi gặp ngắn trống. Cách thứ hai nhanh hơn trong thực tế vì bạn đi bộ dọc dãy tử, không phải lục túi.

Có hai điều dễ nhầm và đáng nhớ. Thứ nhất, đụng ngắn xảy ra sớm hơn ta tưởng rất nhiều: trong một lớp 23 người thì đã có khoảng một nửa cơ hội có hai người trùng ngày sinh, dù một năm có 365 ngày. Nói cách khác, muốn an toàn với k món đồ thì số ngắn phải cỡ k^2 , chứ không phải cỡ k . Thứ hai, nếu có người biết quy tắc đánh số ngắn của bạn, họ có thể cố ý mang tới toàn những cái tên rơi vào cùng một ngắn, và tử của bạn thành một cái túi khổng lồ phải lục từng món. Cách chống rất đơn giản: mỗi lần mở tử buổi sáng, bạn trộn thêm một con số bí mật ngẫu nhiên vào quy tắc. Họ không đoán được nữa.

Một câu: Băm biến tên thành ngắn tử; mọi kỹ thuật còn lại chỉ để sống chung với chuyện hai tên rơi vào một ngắn.

Chỗ không rút gọn được: Không có hàm băm nào tránh được va chạm — số tên luôn nhiều hơn số ngắn; ta chỉ chọn được cách phân bố và cách phản ứng.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng cốt lõi của băm	Ép khoá bất kỳ thành chỉ số mảng bằng một hàm, để tra cứu thành truy cập trực tiếp. Va chạm là không tránh được.
••• Hai cách xử lý va chạm	Nói chuỗi (mỗi ô một danh sách, chịu tải > 1) và địa chỉ mở (dò sang ô kế tiếp, hợp cache, phải giữ tải $< 0,7$).

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao bảng băm bị đánh trượt	Hàm băm tắt định $\Rightarrow$ đối thủ dựng được tập khoá cùng ô $\Rightarrow$ mọi thao tác thành $O(n)$. Chứa bằng hạt giống ngẫu nhiên lúc chạy.
●●● Băm phổ dụng — ý chính	Ngẫu nhiên nằm ở <i>lựa chọn hàm băm của bạn</i> , không ở dữ liệu; nhờ đó đảm bảo vẫn đúng với dữ liệu thù địch.
●●● Nghịch lý ngày sinh	Va chạm đáng kể khi số phần tử đạt cỡ $\sqrt{m}$, không phải m . Xác suất có va chạm $\approx k^2/(2m)$.
●●● Hệ quả cho băm chuỗi	Modulo 10^9 chỉ an toàn tới cỡ $3 \cdot 10^4$ chuỗi; bài lớn cần hai modulo hoặc modulo cỡ 10^{18} , và cơ sở phải ngẫu nhiên.
●● Hệ số tải và mở rộng bảng	$\alpha = n/m$; vượt ngưỡng thì cấp phát bảng gấp đôi và băm lại, chi phí $O(n)$ nhưng khấu hao $O(1)$.
●● Bloom filter	Mảng bit + k hàm băm; “chắc chắn không có” hoặc “có lẽ có”. Dùng làm bộ lọc rẻ trước một phép tra cứu đắt.
●● Ba thứ băm không làm được	Không có thứ tự (không hỏi cận trên/dưới, không duyệt tăng dần); không đảm bảo trường hợp xấu nhất; $O(1)$ chỉ tính theo số phép băm, khoá dài vẫn tồn theo độ dài.
●● Khi nào băm là lựa chọn sai	Khi khoá là số nguyên trong khoảng nhỏ — mảng thường luôn thắng: nhanh hơn, đơn giản hơn, không va chạm.
●● Băm nhất quán	Xếp khoá và máy chủ lên vòng tròn, khoá thuộc về máy kế tiếp theo chiều kim đồng hồ; thêm máy chỉ làm $1/n$ số khoá đổi chỗ.
● Đếm phần tử phân biệt	Giá trị băm nhỏ nhất nhìn thấy được nói lên số phần tử phân biệt; HyperLogLog tinh chỉnh ý tưởng đó, vài KB cho hàng tỉ bản ghi.
● Băm mật mã khác băm thường	Yêu cầu không ai tạo được va chạm kể cả khi cố tình; chậm hơn nhiều nên không dùng cho bảng băm, nhưng cần cho định danh nội dung như trong git.
●● Băm cuckoo và băm bảng tra	Cuckoo: mỗi khoá chỉ nằm ở một trong hai ô nên tra cứu là $O(1)$ <i>trường hợp xấu nhất</i> , giá dồn sang phép chen và đòi tải dưới $1/2$. Bảng tra: hàm băm rẻ, chỉ đọc lập bậc ba, nhưng vẫn có chặn tập trung đã chứng minh.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên chọn kích thước bảng băm là số nguyên tố hay lũy thừa của 2?

| Lũy thừa của 2 cho phép thay phép chia lấy dư bằng phép AND bit, nhanh hơn đáng kể — nhưng nó chỉ giữ lại các bit thấp của mã băm, nên đòi hỏi hàm băm phải trộn tốt các bit. Số nguyên tố tha thứ hơn cho hàm băm kém vì phép lấy dư trộn mọi bit, nhưng phép chia chậm. Thực hành hiện đại nghiêng về lũy thừa của 2 kèm một bước trộn bit sau khi băm. Trong thi đấu, dùng lũy thừa của 2 kèm trộn hạt giống là lựa chọn an toàn.

2. Khi nào nên dùng map thay unordered_map trong C++?

| Bốn tình huống. Khi cần thứ tự hoặc truy vấn cận (lower_bound). Khi số phần tử nhỏ — dưới vài trăm thì hằng số của map không đáng kể và code an toàn hơn. Khi bài thi có nguy cơ bị dựng test phá và bạn không muốn viết hàm băm riêng. Và khi cần đảm bảo trường hợp xấu nhất. Ngược lại, với hàng triệu thao tác tra cứu thuần bằng khoá và có hàm băm tự viết, unordered_map hoặc một bảng địa chỉ mở tự cài sẽ nhanh hơn nhiều lần.

3. Băm chuỗi có thể thay hoàn toàn KMP và suffix array không?

| Thay được rất nhiều bài và thường ngắn hơn hẳn, nhưng có ba giới hạn thật. Một, nó là thuật toán xác suất — xác suất sai nhỏ nhưng khác không, và trong bài yêu cầu kết quả chắc chắn đúng thì đó là điều cần cân nhắc. Hai, một số bài cần cấu trúc bên trong của chuỗi (biên tuần hoàn, số chuỗi con phân biệt) mà băm không cho, phải dùng công cụ ở B/12. Ba, băm chuỗi so sánh được “bằng

nhau” nhưng không so sánh được “lớn hơn” theo thứ tự từ điển, trừ khi kết hợp thêm tìm kiếm nhị phân trên độ dài tiền tố chung.

4. Bloom filter chọn bao nhiêu hàm băm là tốt nhất?

| Có sự đánh đổi rõ: ít hàm băm thì mỗi phần tử bật ít bit nên mảng thưa hơn, nhưng mỗi truy vấn kiểm tra ít điều kiện nên dễ báo nhầm; nhiều hàm băm thì ngược lại. Với m bit và n phần tử, số hàm băm tối ưu tỉ lệ với m/n — cụ thể là $(m/n) \ln 2$, và tại điểm tối ưu thì khoảng một nửa số bit được bật. Điều đáng nhớ về mặt trực giác: mảng đầy quá nửa thì cấu trúc bắt đầu mất tác dụng.

5. Vì sao không dùng luôn hàm băm mật mã cho bảng băm để khỏi lo bị tấn công?

| Vì nó chậm hơn nhiều bậc — được thiết kế để chống lại đối thủ có nhiều tài nguyên, nên có tính tốn nhiều phép toán. Với bảng băm, thứ bạn cần chỉ là đối thủ *không đoán trước được*, mà điều đó đạt được rẻ hơn nhiều bằng một hàm băm nhanh cộng một hạt giống ngẫu nhiên bí mật. Dùng đúng công cụ cho đúng mức đe dọa là nguyên tắc chung, và dùng quá mức cũng là một dạng sai.

6. Trong hệ thống thật, lỗi liên quan tới băm hay gặp là gì?

| Ba lỗi phổ biến. Một, dùng mã băm làm định danh lâu dài rồi gặp va chạm khi dữ liệu lớn lên — đúng bấy ngày sinh, và nó xuất hiện muộn nên rất khó truy. Hai, giả định thứ tự duyệt của cấu trúc băm là ổn định giữa các lần chạy; nhiều thư viện hiện đại cố tình xáo thứ tự đó, và code phụ thuộc vào nó sẽ hỏng không đều. Ba, dùng $\text{hash} \bmod n$ cho phân mảnh rồi phải mở rộng cụm máy — lúc đó gần như toàn bộ dữ liệu phải di chuyển, và cách sửa là băm nhất quán.

7. Có cấu trúc nào cho $O(1)$ trường hợp xấu nhất chứ không chỉ trung bình không?

| Có: băm hoàn hảo. Với tập khoá *tĩnh* biết trước, người ta xây được một hàm băm không có va chạm nào, cho tra cứu $O(1)$ đảm bảo; kỹ thuật hai tầng cổ điển đạt được điều đó với bộ nhớ tuyến tính. Hạn chế là tập khoá phải cố định, nên nó dùng cho từ khoá ngôn ngữ lập trình, bảng tra cố định trong hệ thống nhúng — những nơi dữ liệu không đổi sau khi biên dịch.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Đếm số phần tử phân biệt trong mảng	CSES “Distinct Numbers”[33]	So sánh ba cách: sắp xếp, tập hợp có thứ tự, tập hợp băm — và đo thời gian thật.
● Hai số có tổng bằng x , mảng chưa sắp	LeetCode “Two Sum”	Bảng băm một lượt; đối chiếu với cách hai con trỏ ở B/06 và nêu điều kiện chọn mỗi cách.
● Nhóm các chuỗi đảo chữ với nhau	LeetCode “Group Anagrams”	Thiết kế khoá băm là bước quyết định — chuỗi đã sắp hay vector đếm 26 phần tử.
●● Đếm số đoạn con có tổng bằng x	CSES “Subarray Sums II”[33]	Tổng tiền tố cộng bảng băm; chú ý tổng có thể âm và phải dùng kiểu 64 bit.
●● Đoạn con dài nhất có tổng bằng 0	Bài tập kinh điển	Cùng mẫu: lưu lần xuất hiện <i>đầu tiên</i> của mỗi giá trị tổng tiền tố.
●● Cài một bảng băm địa chỉ mở của riêng bạn	Bài tập cài đặt	So thời gian với thư viện chuẩn trên 10^7 thao tác; đây là bài tập cho thấy rõ nhất tác động của cache.
●● Dựng bộ test làm chậm <code>unordered_map</code>	Bài tập an ninh	Chọn khoá cùng số dư theo kích thước bảng; sau đó chứa bằng hàm băm có hạt giống và đo lại.
●●● So sánh mọi cặp đoạn con bằng băm chuỗi	CSES nhóm bài chuỗi[33]	Hai modulo; tính xác suất va chạm cho k chuỗi trước khi cài, đừng đoán.
●●● Tìm chuỗi con lặp lại dài nhất bằng băm và nhị phân trên độ dài	Bài tập kinh điển	Kết hợp băm với nhị phân trên đáp án (C/19); chú ý tính đơn điệu của điều kiện.
●●● Cài Bloom filter và đo tỉ lệ báo nhầm theo số hàm băm	Bài tập thực nghiệm	Vẽ đường cong tỉ lệ báo nhầm theo k ; so với dự đoán lý thuyết và giải thích chỗ lệch.
●●● Cài băm nhất quán với nút ảo, mô phỏng thêm/bớt máy chủ	Bài tập hệ thống	Đo tỉ lệ khoá phải di chuyển; so với $\text{mod } n$ để thấy khác biệt bằng số liệu.

Kết chương và đọc tiếp

Băm đối ba thứ — thứ tự, đảm bảo xấu nhất, và tính không đoán trước — để lấy $O(1)$ trung bình. Hai điều đáng mang theo: nghịch lý ngày sinh nói ngưỡng an toàn là $\sqrt{m}$ chứ không phải m ; và mọi phân tích “trung bình” đều sụp nếu dữ liệu do đối thủ chọn, nên hạt giống ngẫu nhiên không phải tùy chọn mà là một phần của lời giải.

Chương tiếp theo trả lời câu hỏi ngược lại: nếu bạn *cần* thứ tự thì phải trả thêm bao nhiêu, và các cấu trúc cây tìm kiếm giữ cân bằng bằng cách nào (B/09). Bạn sẽ thấy một mẫu quen thuộc lặp lại: các cấu trúc thắng cuộc trong thực tế thường là các cấu trúc *ngẫu nhiên hoá*, vì lý do giống hệt lý do trong chương này.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Pagh & Rodler, “Cuckoo Hashing”	Journal of Algorithms, 2004	Bản viết rất gọn của một ý tưởng đẹp. Đọc phần mô tả thuật toán chèn và điều kiện hệ số tải; phần chứng minh đọc sau.
nền tảng	Pătraşcu & Thorup, “The Power of Simple Tabulation Hashing”	Journal of the ACM, 2012	Đọc phần mở đầu và phát biểu các định lý — đủ để hiểu vì sao một hàm băm chỉ đọc lập bậc ba lại dùng được ở nơi lý thuyết cũ đòi bậc cao.
tra khi cần	Bloom, “Space/Time Trade-offs in Hash Coding with Allowable Errors”	Communications of the ACM, 1970	Bốn trang, bài gốc của Bloom filter; đọc để thấy động cơ ban đầu là tiết kiệm bộ nhớ cho từ điển chính tả.
tra khi cần	Flajolet, Fusy, Gandouet, Meunier, “HyperLogLog”	Conference on Analysis of Algorithms, 2007	Đọc khi cần đếm phần tử phân biệt ở quy mô lớn; phần phân tích là ví dụ mẫu mực về xác suất trong thiết kế cấu trúc.

Cây tìm kiếm và cân bằng (Search Trees and Balance)

PHỤ THUỘC B/08 cho phía đối lập — băm không thứ tự; A/04 cho bất biến, vì cân bằng chính là một bất biến được duy trì sau mỗi thao tác.

TÓM TẮT

Cây tìm kiếm là câu trả lời cho câu hỏi mà băm không trả lời được: *giữ dữ liệu có thứ tự với chi phí cập nhật rẻ*. Ý tưởng gốc rất đơn giản — mỗi nút chia dữ liệu thành nhỏ hơn và lớn hơn — nhưng nó thoái hoá thành danh sách nếu dữ liệu vào theo thứ tự sắp sẵn, và toàn bộ chương này là các cách khác nhau để chống lại sự thoái hoá đó: quay để giữ chiều cao (AVL, đỏ-đen), làm nút to ra cho hợp với đĩa (B-cây), hoặc dùng ngẫu nhiên để cân bằng gần như miễn phí (treap, danh sách bỏ qua). Chương cũng nói về *tăng cường* — gắn thêm thông tin vào mỗi nút để cây trả lời được câu hỏi mới, kỹ thuật đáng giá nhất ở đây. Nếu chỉ nhớ một điều: chiều cao cây là bất biến cần duy trì, và mọi cấu trúc chỉ khác nhau ở cách trả tiền cho bất biến đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **cây tìm kiếm nhị phân (BST)** — cây nhị phân trong đó mọi nút ở cây con trái nhỏ hơn nút gốc, mọi nút ở cây con phải lớn hơn. Duyệt giữa cho dãy tăng dần.
- **chiều cao cây** — số cạnh trên đường dài nhất từ gốc tới lá; quyết định chi phí mọi thao tác. Cây cân bằng có chiều cao $O(\log n)$, cây thoái hoá có chiều cao $O(n)$.
- **phép quay (rotation)** — thao tác cục bộ đổi chỗ một nút với con của nó, giữ nguyên thứ tự nhưng đổi chiều cao. Đây là công cụ cơ bản của mọi cây tự cân bằng.
- **tăng cường (augmentation)** — lưu thêm thông tin ở mỗi nút — kích thước cây con, tổng, cực đại — để trả lời được truy vấn mới mà vẫn giữ $O(\log n)$.
- **B-cây** — cây tìm kiếm nhiều nhánh, mỗi nút chứa nhiều khoá, thiết kế để mỗi nút vừa một khối đĩa. Cơ sở của mọi chỉ mục cơ sở dữ liệu.
- **treap** — cây kết hợp tính chất BST theo khoá và tính chất heap theo một độ ưu tiên *ngẫu nhiên*; cân bằng theo nghĩa kỳ vọng mà không cần luật quay phức tạp.

CÂU HỎI MỞ ĐẦU

- Q1. Cây tìm kiếm nhị phân đơn giản chạy $O(\log n)$ “trung bình”. Vì sao trong thực tế nó lại nguy hiểm, và tình huống nào làm nó tệ nhất?
- Q2. Cây đỏ-đen và AVL cùng cho $O(\log n)$. Vì sao thư viện chuẩn thường chọn đỏ-đen còn cơ sở dữ liệu lại chọn B-cây?
- Q3. Treap dùng số ngẫu nhiên để cân bằng và không có luật quay phức tạp nào. Vì sao nó vẫn đảm bảo được chiều cao $O(\log n)$?
- Q4. Vì sao “tăng cường” là kỹ thuật đáng giá nhất của chương này, hơn cả việc thuộc luật cân bằng?
- Q5. Khi nào một cây cân bằng $O(\log n)$ lại thua một mảng đã sắp $O(n)$ trên máy thật?

Chương trước cho ta tra cứu $O(1)$ nhưng không có thứ tự. Rất nhiều bài toán cần thứ tự: tìm phần tử gần nhất, liệt kê mọi phần tử trong một khoảng, tìm phần tử nhỏ nhất lớn hơn x , hỏi phần tử đứng thứ k . Nếu dữ liệu tĩnh thì mảng đã sắp cộng tìm kiếm nhị phân giải hết — rẻ và

nhANH. Bài toán chỉ trở nên khó khi dữ liệu *vừa cần thứ tự vừa thay đổi liên tục*, vì chèn vào mảng đã sắp tốn $O(n)$.

Cây tìm kiếm là câu trả lời cho đúng tình huống đó, và ý tưởng thì hiển nhiên: thay vì giữ dữ liệu thành một dãy, hãy giữ nó thành một cấu trúc phân nhánh, để việc chèn chỉ động tới một đường từ gốc xuống lá. Cái không hiển nhiên — và là toàn bộ nội dung của chương — là làm sao giữ cho đường đó ngắn.

9.1 Bài toán thoái hoá và ý nghĩa của cân bằng

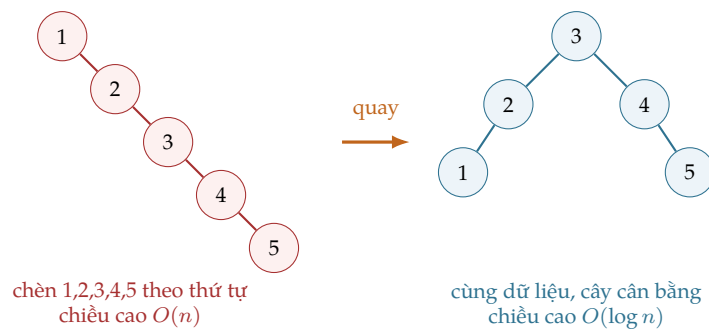
Cây tìm kiếm nhị phân thô sơ chèn phần tử mới bằng cách đi từ gốc, rẽ trái nếu nhỏ hơn, rẽ phải nếu lớn hơn, cho tới khi gặp chỗ trống. Nếu dữ liệu tới theo thứ tự ngẫu nhiên, chiều cao kỳ vọng là $O(\log n)$ và mọi thứ đều tốt.

Nhưng nếu dữ liệu tới theo thứ tự *đã sắp* — tình huống cực kỳ phổ biến trong thực tế, vì dữ liệu thật thường có trật tự sẵn — thì mọi phần tử đều rẽ phải và cây thành một danh sách liên kết dài n . Mọi thao tác thành $O(n)$, và cấu trúc mà bạn chọn để tăng tốc lại làm chậm chương trình so với mảng thường.

Đây là cùng một câu chuyện với chương B/08: một cấu trúc nhanh “trung bình” nhưng có trường hợp xấu mà dữ liệu thực tế lại rất dễ rơi vào. Và có hai hướng chữa, hết như ở đó.

Hướng tất định: thêm một bất biến về hình dạng và duy trì nó sau mỗi thao tác. AVL yêu cầu chiều cao hai cây con của mọi nút chênh nhau không quá 1 — rất chặt, nên cây thấp và tìm kiếm nhanh, nhưng phải quay nhiều khi chèn/xoá. Cây đỏ-đen yêu cầu lỏng hơn — không đường nào từ gốc tới lá dài quá hai lần đường ngắn nhất — nên cây cao hơn một chút nhưng số phép quay ít hơn. Sự khác nhau đó giải thích lựa chọn của các thư viện: khi tỉ lệ chèn/xoá cao, đỏ-đen tổng thể rẻ hơn.

Hướng ngẫu nhiên: không ép hình dạng, mà làm cho hình dạng *không phụ thuộc thứ tự dữ liệu vào*. Đây là ý tưởng của treap và danh sách bỏ qua, và mục 3 sẽ nói kỹ vì nó rất đẹp.



Hình 9.1. Cùng năm khoá, hai hình dạng. Cây trái là kết quả của việc chèn dữ liệu đã sắp sẵn — trường hợp rất hay gặp trong thực tế, không phải trường hợp hiếm. Mọi cấu trúc trong chương này chỉ làm một việc: ngăn hình dạng bên trái xuất hiện, bằng phép quay hoặc bằng ngẫu nhiên hoá.

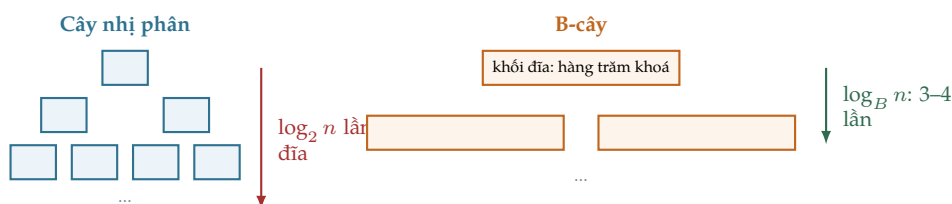
9.2 Phép quay: thao tác cơ bản duy nhất

Mọi cây tự cân bằng đều dựa trên một thao tác duy nhất là phép quay: đổi chỗ một nút với con của nó, nối lại ba cây con cho đúng. Điều quan trọng nhất về phép quay — và là chỗ đáng dừng lại một phút — là nó *bảo toàn thứ tự duyệt giữa* nhưng *thay đổi chiều cao*. Nói theo ngôn ngữ chương A/04: phép quay giữ nguyên bất biến ngữ nghĩa (tính chất BST) trong khi sửa bất biến cấu trúc

(chiều cao). Đó chính là lý do nó là công cụ đúng: bạn cần sửa hình dạng mà không được làm hỏng nội dung.

Từ đó, các cấu trúc chỉ khác nhau ở *khi nào quay* và *quay bao nhiêu*: AVL quay khi chênh lệch chiều cao vượt 1; đỏ–đen quay và đổi màu theo một bộ luật dài hơn nhưng ít quay hơn; treap quay để khôi phục tính chất heap theo độ ưu tiên ngẫu nhiên; splay quay đưa nút vừa truy cập lên gốc.

Cây splay đáng nhắc riêng vì nó đại diện cho một triết lý khác[35]: không đảm bảo gì cho từng thao tác, chỉ đảm bảo $O(\log n)$ *khấu hao*, đổi lại có tính chất rất hay là các phần tử hay truy cập tự động nổi lên gần gốc. Trong hệ thống có tính cục bộ truy cập cao, điều đó làm nó nhanh hơn cây cân bằng chặt. Nhược điểm: mỗi lần *đọc* cũng làm thay đổi cấu trúc, nên nó khó dùng trong môi trường nhiều luồng.



Hình 9.2. Đổi đơn vị chi phí thì đổi hình dạng cấu trúc. Khi chi phí thật là *lần đọc khối* chứ không phải phép so sánh, việc đăng nào cũng phải trả tiền cho một lần đọc khiến ta muốn đọc thật nhiều khoá một lúc — và cây từ nhị phân biến thành nhiều nhánh với nút vừa một khối đĩa.

9.3 Ngẫu nhiên hoá: cách rẻ nhất để cân bằng

Treap là một trong những ý tưởng đẹp nhất trong cấu trúc dữ liệu, và nó chỉ cần một câu để phát biểu: mỗi nút có khoá (do dữ liệu quyết định) và một *độ ưu tiên ngẫu nhiên* (do bạn sinh ra); cây được giữ sao cho theo khoá thì nó là BST, còn theo độ ưu tiên thì nó là heap.

Vì sao điều đó cho cân bằng? Vì hình dạng của một treap chỉ phụ thuộc vào *thứ tự các độ ưu tiên*, không phụ thuộc thứ tự chèn. Mà độ ưu tiên do bạn sinh ngẫu nhiên, nên hình dạng của treap giống hệt hình dạng của một BST được chèn theo thứ tự ngẫu nhiên — và chiều cao kỳ vọng của cây đó là $O(\log n)$. Nói cách khác: bạn không thể kiểm soát thứ tự dữ liệu tới, nhưng bạn có thể tự tạo ra một thứ tự ngẫu nhiên khác và để hình dạng phụ thuộc vào thứ tự đó.

Đây chính xác là ý tưởng của băm phổ dụng ở chương B/08 và của chốt ngẫu nhiên trong quick-sort ở chương C/14, xuất hiện lần thứ ba trong quyển sách này. Nếu bạn thấy mẫu ấy lặp lại thì đó không phải trùng hợp mà là một nguyên lý: *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc đó sang một nguồn ngẫu nhiên mà bạn kiểm soát*. Chương C/20 dành trọn để khai thác nguyên lý này.

Danh sách bỏ qua (skip list) là một hiện thực hoá khác của cùng ý tưởng, dùng nhiều tầng danh sách liên kết với xác suất lên tầng là $1/2$. Nó không phải cây, nhưng cho cùng đảm bảo kỳ vọng, dễ cài hơn cây cân bằng và dễ làm phiên bản nhiều luồng — đó là lý do một số hệ cơ sở dữ liệu chọn nó cho phần dữ liệu nằm trong bộ nhớ.

9.4 Tăng cường: kỹ thuật đáng giá nhất của chương

Đây là mục mà nếu chỉ đọc một mục thì nên đọc mục này. Ý tưởng: gắn thêm thông tin vào mỗi nút, miễn là thông tin đó *tính được từ hai con*, thì bạn giữ được nó qua mọi thao tác với chi phí không đổi, và cây trả lời được câu hỏi hoàn toàn mới.

Ví dụ quan trọng nhất: lưu ở mỗi nút *kích thước cây con* của nó. Với thông tin đó, cây trả lời được hai câu hỏi mà băm và heap đều chịu: “phần tử đứng thứ k là ai” và “có bao nhiêu phần tử nhỏ hơn x ” — cả hai trong $O(\log n)$, bằng cách đi xuống từ gốc và trừ dần. Cấu trúc này gọi là cây thống kê thứ tự, và nó là công cụ chuẩn cho các bài đếm nghịch thế hoặc đếm phần tử nhỏ hơn trong đoạn.

Ví dụ thứ hai: lưu *giá trị lớn nhất của điểm kết thúc* trong cây con, ta được cây khoảng — trả lời “có đoạn nào chứa điểm x không” hoặc “liệt kê mọi đoạn giao với $[l, r]$ ”. Đây là cấu trúc dùng trong hình học tính toán (D/22) và trong các hệ thống lập lịch.

Quy tắc chung đáng nhớ: *thông tin tăng cường phải tổng hợp được từ các con trong $O(1)$* . Kích thước, tổng, cực đại, cực tiểu đều thỏa; “phần tử phổ biến nhất trong cây con” thì không, vì không suy ra được từ hai con. Cùng điều kiện này sẽ xuất hiện lại ở chương B/10 với cây phân đoạn, và ở đó ta sẽ thấy nó chính là điều kiện “phép toán có tính kết hợp”.

9.5 B-cây: khi chi phí thật không phải là số phép so sánh

Mọi phân tích trên đếm số phép so sánh. Nhưng nếu dữ liệu nằm trên đĩa, chi phí thật là số lần đọc khối — và một lần đọc đĩa đắt hơn hàng triệu phép so sánh trong bộ nhớ. Với mô hình chi phí đó, cây nhị phân là lựa chọn tệ: chiều cao $\log_2 n$ nghĩa là $\log_2 n$ lần đọc đĩa, mà với một tỉ bản ghi thì đó là 30 lần đọc cho một truy vấn.

B-cây giải quyết bằng cách làm nút to ra: mỗi nút chứa hàng trăm khoá và vừa đúng một khối đĩa. Chiều cao trở thành $\log_{100} n$, tức là ba tới bốn tầng cho một tỉ bản ghi. Trong nút thì tìm kiếm bằng so sánh — rẻ, vì nút đã nằm trong bộ nhớ sau khi đọc.

Đây là ví dụ rõ nhất trong cả quyển sách cho luận điểm ở chương A/03: *đổi mô hình chi phí thì đổi luôn cấu trúc tối ưu*. Cùng bài toán, cùng yêu cầu, nhưng khi đơn vị chi phí chuyển từ “phép so sánh” sang “lần chuyển khối” thì lời giải tốt nhất đổi hẳn hình dạng. Và điều đó vẫn đúng khi dữ liệu nằm trong RAM: vì bộ nhớ cũng có phân cấp cache, các cấu trúc “B-cây thu nhỏ” với nút vừa một dòng cache thường nhanh hơn cây nhị phân thuần túy.

Cấu trúc	Đảm bảo	Cài đặt	Chọn khi
AVL	$O(\log n)$ chặt, cây thấp	Khó	Đọc nhiều hơn ghi rất nhiều
Đỏ-đen	$O(\log n)$, ít quay hơn	Khó	Thư viện chuẩn, ghi và đọc cân bằng
Treap	$O(\log n)$ kỳ vọng	Dễ	Tự cài trong thi đấu; cần tách/ghép cây
Danh sách bỏ qua	$O(\log n)$ kỳ vọng	Dễ	Cần phiên bản nhiều luồng
Splay	$O(\log n)$ khẩu hao	Trung bình	Truy cập có tính cục bộ cao
B-cây / B ⁺ -cây	$O(\log_B n)$ lần đọc khối	Khó	Dữ liệu trên đĩa; chỉ mục cơ sở dữ liệu

CẠM BẮY

Trong lập trình thi đấu, đừng cài AVL hay đỏ-đen. Nếu cần cây cân bằng tự viết — thường là để có thao tác tách và ghép, thứ mà thư viện chuẩn không cho — hãy dùng treap: ngắn hơn nhiều lần, ít lỗi hơn hẳn, và đảm bảo kỳ vọng là đủ. Còn nếu chỉ cần tập hợp có thứ tự thông thường thì dùng thẳng thư viện chuẩn; viết lại chỉ tăng nguy cơ sai.

9.6 Splay và câu hỏi mở nổi tiếng nhất của ngành cấu trúc dữ liệu

Bảng ở mục trước xếp splay tree vào ô “ $O(\log n)$ khấu hao, hợp khi truy cập có tính cục bộ”. Dòng đó đúng nhưng giấu mất phần thú vị nhất, và phần ấy đáng kể lại vì nó dạy một cách nghĩ chứ không chỉ một cấu trúc.

Splay tree không lưu *bất kỳ* thông tin cân bằng nào — không chiều cao, không màu, không độ ưu tiên. Luật duy nhất: mỗi lần chạm vào một nút, hãy quay nó lên tận gốc theo một khuôn cố định gồm các bước hai tầng. Sleator và Tarjan[35] chứng minh rằng chỉ luật đó đã đủ cho $O(\log n)$ khấu hao — nhưng kết quả đáng nhớ hơn là hai tính chất mạnh hơn cận ấy.

Tối ưu tĩnh: trên bất kỳ dãy truy cập nào, tổng chi phí của splay chỉ hơn một hằng số lần so với *cây tĩnh tốt nhất* dựng bởi một người biết trước tần suất truy cập của mọi khoá. Nói cách khác, splay đạt được cận entropy của phân phối mà *không hề biết* phân phối đó — nó học từ chính dãy thao tác. *Tính chất tập làm việc*: chi phí chạm vào một khoá phụ thuộc số khoá phân biệt đã được chạm kể từ lần trước chạm nó, chứ không phụ thuộc n . Vì vậy một chương trình liên tục quay lại vài trăm khoá quen sẽ trả giá theo vài trăm chứ không theo hàng triệu — đúng hành vi mà bộ nhớ đệm mong muốn.

Câu hỏi mở nằm ở bước tiếp theo. Sleator và Tarjan phỏng đoán rằng splay không chỉ tối ưu so với cây tĩnh tốt nhất, mà tối ưu trong vòng một hằng số so với *mọi* thuật toán trên cây tìm kiếm nhị phân — kể cả thuật toán được phép quay tùy ý và được biết trước toàn bộ dãy truy cập. Đó là *phỏng đoán tối ưu động*, và sau bốn thập kỷ nó vẫn mở. Tiến bộ lớn đầu tiên là Tango tree của Demaine, Harmon, Iacono và Patrascu[45]: một cây trực tuyến có tỉ lệ cạnh tranh $O(\log \log n)$, dựng trên một cận dưới cho *mọi* cây nhị phân. Trước đó, tỉ lệ tốt nhất người ta biết chỉ là $O(\log n)$ — tức là “không tệ hơn việc chẳng thích nghi gì cả”.

Có hai điều đáng mang theo, và cả hai đều lớn hơn bản thân splay tree. Thứ nhất là cách đặt câu hỏi: thay vì hỏi “cấu trúc này tốn bao nhiêu”, ta hỏi “nó tốn bao nhiêu so với lời giải tốt nhất có thể trên cùng dãy thao tác”. Đó chính là phân tích cạnh tranh, thước đo chuẩn của các bài toán trực tuyến ở chương D/24, và ở đây nó được áp cho một cấu trúc dữ liệu. Thứ hai là lời nhắc rằng chữ “tối ưu” luôn kèm một lớp đối thủ và một mô hình: tối ưu so với cây tĩnh, so với mọi cây nhị phân, hay so với mọi cấu trúc bất kỳ — ba câu hỏi khác nhau với ba câu trả lời khác nhau.

Còn phần thực dụng thì ngắn. Đừng cài splay trong kỳ thi: hằng số lớn, một thao tác đơn lẻ có thể tốn $O(n)$, và mọi lần *đọc* đều *ghi* vào cây — điều làm nó khó dùng trong môi trường nhiều luồng và bất lợi với bộ nhớ đệm chỉ đọc. Giá trị của nó với người học nằm ở chỗ khác: nó là bằng chứng rằng một cấu trúc không giữ chút siêu dữ liệu cân bằng nào vẫn có thể tự điều chỉnh theo dữ liệu, và nó là cửa vào cho cả một nhánh nghiên cứu vẫn còn sống.

9.7 Gốc rễ: từ bài toán chiều cao tới bài toán đĩa quay

Cây tìm kiếm nhị phân là ý tưởng cổ và không có tác giả rõ ràng — nó xuất hiện tự nhiên ở nhiều nơi trong thập niên 1950. Điều *có* tác giả rõ ràng là bài toán cân bằng, và cây tự cân bằng đầu tiên ra đời năm 1962 do hai nhà toán học Liên Xô, Adelson-Velsky và Landis, đề xuất — tên AVL là chữ đầu của họ. Đóng góp thực sự của họ không phải cấu trúc mà là *ý niệm*: rằng có thể duy trì một bất biến hình dạng sau mỗi thao tác với chi phí không đổi, và nhờ đó biến một đảm bảo “trung bình” thành đảm bảo tuyệt đối.

B-cây ra đời năm 1970 tại phòng nghiên cứu của Boeing, từ một bài toán hoàn toàn thực tế: chỉ mục cho các tệp lớn nằm trên đĩa quay. Rudolf Bayer và Edward McCreight nhận ra điều mà mô hình đếm phép so sánh che mất — rằng chi phí đọc một khối đĩa gần như không phụ thuộc kích thước khối, nên nếu đăng nào cũng phải trả tiền cho một lần đọc thì hãy đọc thật nhiều khoá một

lúc. Từ nhận xét đó ra cả một họ cấu trúc, và ngày nay gần như mọi hệ cơ sở dữ liệu quan hệ và mọi hệ tệp hiện đại đều dùng một biến thể của nó. Đây là ví dụ đẹp cho việc *đặt lại câu hỏi về đơn vị chi phí* có thể sinh ra cấu trúc mới.

Cây đỏ–đen có nguồn gốc thú vị: nó khởi đầu như một cách biểu diễn B-cây bậc 4 bằng cây nhị phân, rồi được Guibas và Sedgewick trình bày lại vào năm 1978 dưới dạng luật màu mà ta biết ngày nay. Nghĩa là hai cấu trúc trông rất khác nhau thực ra là hai cách nhìn cùng một thứ — một liên hệ đáng nhớ vì nó giải thích vì sao luật đỏ–đen trông tuy tiện: chúng là dấu vết của cấu trúc nhiều nhánh gốc.

Cuối cùng, hai cấu trúc ngẫu nhiên hoá ra đời muộn hơn nhiều và vì một lý do rất người: cây cân bằng tất định quá khó cài đúng. Pugh đề xuất danh sách bỏ qua năm 1990 với luận điểm thẳng thắn rằng nó đơn giản hơn hẳn cây cân bằng mà vẫn cho cùng đảm bảo kỳ vọng; Seidel và Aragon đưa ra treap năm 1996 với cùng tinh thần. Sự chuyển dịch này — chấp nhận đảm bảo kỳ vọng để đổi lấy code ngắn và đúng — là một chủ đề lặp lại trong ngành, và nó gắn với chương [C/20](#).

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — BST	Cần cấu trúc vừa tra cứu nhanh vừa chèn được, thay cho mảng đã sắp.	Ý tưởng phân nhánh theo khoá; nhưng chưa ai lo chuyện thoái hoá.
1962 — Adelson-Velsky & Landis	BST thoái hoá thành danh sách khi dữ liệu vào có trật tự.	Cây AVL — cây tự cân bằng đầu tiên; xác lập ý niệm “duy trì bất biến hình dạng sau mỗi thao tác”.
1970 — Bayer & McCreight (Boeing)	Chỉ mục tệp lớn trên đĩa quay; mỗi lần đọc đĩa đắt hơn hàng triệu phép so sánh.	B-cây: đổi đơn vị chi phí từ phép so sánh sang lần đọc khối $\Rightarrow$ nút to, cây thấp. Nền của mọi chỉ mục cơ sở dữ liệu.
1972–78 — Bayer, Guibas & Sedgewick	Muốn ưu điểm của B-cây bậc 4 nhưng ở dạng cây nhị phân.	Cây đỏ–đen; luật màu trông tuy tiện chính là dấu vết của cấu trúc nhiều nhánh gốc.
1985 — Sleator & Tarjan	Có cần đảm bảo cho <i>từng</i> thao tác không, hay tổng thể là đủ?	Splay tree: $O(\log n)$ khẩu hao, tự thích nghi với mẫu truy cập; kèm theo là cả lý thuyết phân tích khẩu hao.
1990–96 — Pugh; Seidel & Aragon	Cây cân bằng tất định quá khó cài đúng.	Danh sách bỏ qua và treap: dùng ngẫu nhiên để đạt cùng đảm bảo kỳ vọng với code ngắn hơn nhiều lần.

9.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Cơ sở dữ liệu	Chỉ mục B ⁺ -cây, quét khoảng, khoá chính	Cùng cấu trúc, và lý do chọn nó chính là mục 6. Điểm đáng nhớ: chỉ mục bám nhanh hơn cho truy vấn bằng nhưng vô dụng cho BETWEEN — đó là vì sao B-cây là mặc định.
Hệ tệp	Chỉ mục thư mục, ánh xạ khối	Cùng ràng buộc đĩa nên cùng lời giải. Ví dụ về việc một mô hình chi phí (chuyển khối) sinh ra một họ cấu trúc riêng — xem thêm D/24 .

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Hình học tính toán	Cây khoảng, cây phân đoạn, cây k -d	Đều là cây tìm kiếm được <i>tăng cường</i> : thêm thông tin ở mỗi nút để trả lời câu hỏi hình học. Cùng điều kiện: thông tin phải tổng hợp được từ các con (D/22).
SLAM và xử lý đám mây điểm	Cây k -d cho tìm điểm gần nhất trong ICP	Cùng ý tưởng chia không gian theo ngưỡng, chỉ khác là chia luân phiên theo từng chiều toạ độ. Cùng điểm yếu: dữ liệu phân bố lệch làm cây mất cân bằng, nên trong thực tế phải xây lại định kỳ.
Đồ hoạ, dò tia	Cây bao khối (BVH), cây phân vùng không gian	Cùng mục tiêu: giảm số phép kiểm tra từ tuyến tính xuống lôgarit bằng cấu trúc phân cấp. Tiêu chí xây cây khác nhau (diện tích bề mặt thay vì cân bằng số nút).
Lập trình hàm	Cấu trúc bền vững, cây bất biến	Cây rất hợp với biểu diễn bền vững vì một thao tác chỉ dùng $O(\log n)$ nút, phần còn lại chia sẻ được giữa các phiên bản [46]. Đây là nền của cây bền vững ở B/10.
Hệ thống nhiều luồng	Danh sách bỏ qua trong bộ nhớ đệm ghi của cơ sở dữ liệu	Lý do chọn không phải tốc độ mà là <i>để làm phiên bản đồng thời</i> : các thao tác chỉ động vào con trỏ cục bộ, không cần quay cả cây.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì trường hợp xấu nhất của nó không hiếm mà lại rất phổ biến: dữ liệu thật thường tới theo trật tự nào đó — theo thời gian, theo mã tăng dần, theo thứ tự đã sắp từ nguồn khác. Khi đó mọi phần tử rẽ cùng một hướng và cây thành danh sách với chiều cao n . Đây là cùng dạng vấn đề với bảng băm ở B/08: “trung bình nhanh” vô nghĩa nếu phân phối thực tế nằm đúng vào vùng xấu.
- A2. Vì hai bối cảnh có đơn vị chi phí khác nhau. Trong bộ nhớ, chi phí là số phép so sánh và số lần lần theo con trỏ, nên cây nhị phân là hợp lý; giữa AVL và đỏ-đen thì đỏ-đen quay ít hơn nên tốt hơn khi có nhiều chèn/xoá, và đó là lựa chọn của thư viện chuẩn. Trên đĩa, chi phí là số lần đọc khối và một lần đọc đắt hơn hàng triệu phép so sánh; lúc đó phải làm nút thật to để cây thật thấp — B-cây. Cùng bài toán, đổi mô hình chi phí, đổi cấu trúc tối ưu.
- A3. Vì hình dạng của treap chỉ phụ thuộc vào thứ tự tương đối của các độ ưu tiên, chứ không phụ thuộc thứ tự chèn. Mà độ ưu tiên do bạn sinh ngẫu nhiên, nên treap có phân phối hình dạng giống hệt một BST được chèn theo thứ tự ngẫu nhiên đều — và chiều cao kỳ vọng của cây đó là $O(\log n)$. Điểm mấu chốt: đổi thủ chọn được dữ liệu nhưng không chọn được độ ưu tiên, nên không dựng được trường hợp xấu.
- A4. Vì luật cân bằng thì bạn hầu như luôn dùng lại từ thư viện, còn tăng cường là thứ bạn phải tự nghĩ cho từng bài. Nó biến một cấu trúc chỉ biết “có hay không” thành cấu trúc trả lời được “phần tử thứ k ”, “bao nhiêu phần tử nhỏ hơn x ”, “đoạn nào chứa điểm này”. Điều kiện áp dụng cũng rất dễ kiểm: thông tin thêm phải tổng hợp được từ hai con trong $O(1)$ — và chính điều kiện đó sẽ trở lại ở B/10 dưới tên “phép toán có tính kết hợp”.
- A5. Khi dữ liệu tĩnh hoặc gần như tĩnh. Mảng đã sắp cho tìm kiếm nhị phân với truy cập gần như tuần tự, rất hợp cache, không có con trỏ và không có chi phí cấp phát; cây cân bằng lần theo con trỏ nên mỗi tầng có thể là một lần trượt cache. Với vài triệu phần tử, khác biệt hằng số này lớn hơn khác biệt giữa $O(\log n)$ và $O(n)$ chèn — miễn là số lần chèn nhỏ. Quy tắc thực dụng: ít cập nhật thì mảng đã sắp; cập nhật liên tục thì cây.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy tưởng tượng một cuốn danh bạ. Nếu tên được ghi lộn xộn, muốn tìm ai bạn phải đọc từ đầu. Nếu ghi theo thứ tự chữ cái, bạn mở giữa quyển rồi lật qua lật lại — đó là tìm kiếm nhị phân, rất nhanh, nhưng thêm một cái tên mới vào giữa thì phải chép lại cả quyển.

Cây tìm kiếm là cách sắp xếp danh bạ thành nhiều tờ rồi có mũi tên chỉ nhau, để thêm một tên chỉ phải sửa vài tờ. Vấn đề duy nhất: nếu bạn thêm tên theo đúng thứ tự chữ cái từ A tới Z, các mũi tên sẽ nối thành một hàng dài và bạn quay lại đúng tình cảnh đọc từ đầu.

Có hai cách chống. Cách thứ nhất là cứ vài lần thêm lại chỉnh cho cây khỏi lệch — giống việc sắp lại giá sách cho hai bên đều nhau. Cách thứ hai thông minh hơn và rất đáng nhớ: gán cho mỗi tờ một con số ngẫu nhiên và xếp cây theo con số đó thay vì theo thứ tự bạn thêm vào. Vì con số là do bạn tung ra, người khác không thể cố tình đưa dữ liệu theo thứ tự xấu để hại bạn.

Cuối cùng là một chuyện thực tế thú vị. Nếu quyn danh bạ để trên bàn, việc lật một trang gần như miễn phí. Nhưng nếu nó nằm trong kho và mỗi lần cần một trang thì phải chạy xuống kho lấy, thì bạn sẽ không muốn mỗi trang chỉ có một cái tên — bạn sẽ muốn mỗi lần chạy xuống kho mang lên một tập dày. Đó chính là lý do cơ sở dữ liệu không dùng cây nhị phân mà dùng cây có nút rất to.

Một câu: *Chiều cao là thứ phải giữ; cách trả tiền cho nó — quay tay hay tung ngẫu nhiên — là chỗ các cấu trúc khác nhau.*

Chỗ không rút gọn được: Việc chọn thông tin tăng cường cho một bài cụ thể vẫn là bước sáng tạo; điều kiện “tổng hợp được từ hai con” chỉ nói cái gì không dùng được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vấn đề gốc của BST	Dữ liệu vào theo thứ tự đã sắp làm cây thoái hoá thành danh sách, chiều cao $O(n)$. Đây là trường hợp phổ biến, không hiếm.
••• Phép quay làm gì	Bảo toàn thứ tự duyệt giữa (bất biến ngữ nghĩa) trong khi thay đổi chiều cao (bất biến cấu trúc). Là thao tác cơ bản duy nhất của mọi cây tự cân bằng.
••• Vì sao treap cân bằng	Hình dạng chỉ phụ thuộc thứ tự các độ ưu tiên ngẫu nhiên, không phụ thuộc thứ tự chèn $\Rightarrow$ tương đương BST chèn ngẫu nhiên $\Rightarrow$ chiều cao kỳ vọng $O(\log n)$.
••• Tăng cường — điều kiện	Thông tin thêm phải tổng hợp được từ hai con trong $O(1)$: kích thước, tổng, cực trị được; “phần tử phổ biến nhất” thì không.
••• Cây thống kê thứ tự	Lưu kích thước cây con $\Rightarrow$ trả lời “phần tử thứ k ” và “bao nhiêu phần tử nhỏ hơn x ” trong $O(\log n)$.
••• Vì sao B-cây tồn tại	Chi phí thật trên đĩa là số lần đọc khối, không phải số phép so sánh. Nút to $\Rightarrow$ chiều cao $\log_B n \Rightarrow 3-4$ lần đọc cho một tỉ bản ghi.
•• AVL so với đỏ-đen	AVL chặt hơn nên cây thấp hơn, tìm nhanh hơn, nhưng quay nhiều hơn khi cập nhật. Đỏ-đen lỏng hơn, ít quay hơn — lựa chọn của thư viện chuẩn.
•• Splay tree	Không đảm bảo từng thao tác, chỉ $O(\log n)$ khấu hao; phần tử hay truy cập tự nổi lên gần gốc. Nhược điểm: thao tác đọc cũng làm đổi cấu trúc.
•• Danh sách bỏ qua	Nhiều tầng danh sách liên kết, xác suất lên tầng $1/2$; cùng đảm bảo kỳ vọng với cây, dễ cài và dễ làm phiên bản nhiều luồng.
•• Mẫu ngẫu nhiên hoá lặp lại	Băm phổ dụng (B/08), treap (chương này), chốt quicksort (C/14) — cùng một nguyên lý: chuyển sự phụ thuộc từ dữ liệu sang nguồn ngẫu nhiên bạn kiểm soát.
•• Cây khoảng	Lưu cực đại của điểm kết thúc trong cây con $\Rightarrow$ hỏi “đoạn nào chứa x ”, “đoạn nào giao $[l, r]$ ”.
• Khi nào mảng đã sắp thẳng cây	Dữ liệu tĩnh hoặc ít cập nhật: không con trỏ, hợp cache, không cấp phát. Cây chỉ thẳng khi có cập nhật liên tục.
• Lời khuyên thi đấu	Đừng cài AVL hay đỏ-đen; cần cây tự viết thì dùng treap (đặc biệt khi cần tách/ghép), còn lại dùng thư viện chuẩn.
•• Splay và phỏng đoán tối ưu động	Splay không lưu thông tin cân bằng nào, vẫn đạt tối ưu tĩnh và tính chất tập làm việc. Phỏng đoán “cạnh tranh hằng số với mọi cây nhị phân” vẫn mở; tiến bộ lớn nhất là Tango tree, $O(\log \log n)$ -cạnh tranh.

● ● ● phải thuộc ● ● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi thực sự cần tự cài cây cân bằng thay vì dùng thư viện?

| Chủ yếu khi cần hai thao tác mà thư viện chuẩn không có: *tách* cây thành hai theo khoá và *ghép* hai cây, hoặc cần truy vấn “phần tử thứ k ”. Với nhu cầu thứ nhất, treap là lựa chọn gần như bắt buộc vì tách/ghép trên treap chỉ vài dòng. Với nhu cầu thứ hai, nếu khoá là số nguyên trong khoảng vừa phải thì Fenwick (B/10) đơn giản hơn nhiều và nhanh hơn — hãy kiểm tra khả năng đó trước khi viết cây.

2. Tôi nghe nói cây đỏ-đen “khó cài”. Khó ở chỗ nào?

| Ở phần xoá. Chèn có bốn trường hợp và còn dễ nhớ; xoá có nhiều trường hợp hơn, phải xử lý cả tình huống “nút thiếu màu đen” lan ngược lên gốc, và mỗi trường hợp lại chia theo màu của anh em và của các cháu. Sai một nhánh thì cây vẫn chạy đúng về mặt thứ tự nhưng mất cân bằng dần — một lỗi im lặng rất khó phát hiện. Đây là lý do thực tế khiến treap và danh sách bỏ qua được ưa chuộng ngoài các thư viện chuẩn.

3. Cây k -d có phải là cây tìm kiếm không, và vì sao nó hay mất cân bằng?

| Nó là cây tìm kiếm nhị phân trên không gian nhiều chiều: mỗi tầng chia theo một chiều toạ độ khác nhau. Nó mất cân bằng vì chiều được chọn luân phiên chứ không theo phân bố dữ liệu; nếu điểm dồn về một mặt phẳng hoặc một trục thì nhiều lần chia gần như không tách được gì. Trong thực tế — ví dụ tìm điểm gần nhất trên đám mây điểm của cảm biến — cách xử lý phổ biến là chọn chiều có phương sai lớn nhất thay vì luân phiên, và xây lại cây định kỳ thay vì cập nhật từng điểm.

4. Cây cân bằng có dùng được trong môi trường nhiều luồng không?

| Được nhưng khó, vì phép quay động tới nhiều nút cùng lúc nên phải khoá một vùng lớn. Có ba hướng thực tế: dùng danh sách bỏ qua, vốn chỉ cần sửa vài con trỏ cục bộ và có bản không khoá; dùng cấu trúc bền vững kiểu lập trình hàm, ở đó người đọc luôn nhìn một ảnh chụp bất biến[46]; hoặc dùng B-cây với kỹ thuật khoá theo nút và tách trước khi đi xuống. Splay tree là lựa chọn tệ nhất ở đây vì ngay cả thao tác đọc cũng làm thay đổi cấu trúc.

5. Vì sao đếm nghịch thể lại là bài tiêu biểu cho chương này?

| Vì nó có ba lời giải bằng ba công cụ khác nhau của phần B, và so sánh chúng dạy nhiều hơn bản thân bài. Cách một: cây thống kê thứ tự, chèn dần từ trái sang phải và mỗi lần hỏi “bao nhiêu phần tử đã chèn lớn hơn giá trị này”. Cách hai: Fenwick trên giá trị đã nén (B/10) — ngắn hơn và nhanh hơn. Cách ba: sắp xếp trộn, đếm trong lúc trộn (C/14). Ba cách cùng $O(n \log n)$ nhưng hằng số và độ dài code rất khác.

6. “Bền vững” nghĩa là gì và vì sao cây lại hợp với nó?

| Bền vững là khả năng giữ lại mọi phiên bản cũ sau khi cập nhật, để truy vấn được quá khứ. Cây rất hợp vì một thao tác chỉ đụng tới các nút trên một đường từ gốc xuống lá — chỉ $O(\log n)$ nút. Thay vì sửa tại chỗ, ta tạo bản sao của đúng đường đó và cho các nhánh còn lại trở chung sang cây cũ; chi phí thêm là $O(\log n)$ bộ nhớ mỗi phiên bản. Ý tưởng này sẽ quay lại với cây phân đoạn bền vững ở B/10.

7. Nếu khoá là số nguyên trong khoảng hẹp, có gì tốt hơn cây không?

| Có, và chênh lệch lớn. Với khoá trong $[0, U)$, cấu trúc van Emde Boas cho mọi thao tác trong $O(\log \log U)$ — nhanh hơn hẳn $O(\log n)$ khi U không quá lớn, đổi lại tốn bộ nhớ $O(U)$ và cài đặt phức tạp. Trong thực hành thi đấu, thường chỉ cần một Fenwick trên mảng bit hoặc một cấu trúc phân tầng đơn giản là đủ nhanh. Bài học chung vẫn thế: biết thêm về dữ liệu — ở đây là “khoá là số nguyên nhỏ” — thì luôn mua được tốc độ.

I Bài tập (mức: ● khởi động ● ● chuẩn ● ● ● khó)

Bài	Nguồn	Điều cần nhận ra
• Cài BST cơ bản, rồi chèn $1..n$ theo thứ tự và đo chiều cao	Bài tập cài đặt	Thấy tận mắt sự thoái hoá; sau đó chèn theo thứ tự ngẫu nhiên và đo lại.
• Dùng tập hợp có thứ tự để tìm phần tử gần nhất với mỗi truy vấn	CSES “Concert Tickets”[33]	Bài mẫu cho việc dùng <code>lower_bound</code> — thứ mà bảng băm không làm được.
•• Đếm số nghịch thế	CSES “Inversions”-loại; bài kinh điển	Giải bằng cả ba cách (cây thống kê thứ tự, Fenwick, sắp xếp trộn) và so hằng số.
•• Cài treap với chèn, xoá, tìm phần tử thứ k	Bài tập cài đặt	Bắt đầu bằng tách/ghép; mọi thao tác khác viết từ hai thao tác đó.
•• Duy trì trung vị của một tập thay đổi bằng cây thống kê thứ tự	Bài tập kinh điển	So sánh với cách hai heap ở B/07 — cây tổng quát hơn, heap nhanh hơn cho đúng bài này.
•• Lịch đặt phòng: kiểm tra chồng lấn khi thêm mỗi lịch	LeetCode “My Calendar I/II”	Cây khoảng hoặc tập hợp có thứ tự; chú ý xử lý biên đóng/mở cho đúng.
••• Cài danh sách bỏ qua và so tốc độ với cây cân bằng của thư viện	Bài tập thực nghiệm	Đo cả thời gian lẫn bộ nhớ; giải thích chênh lệch bằng cache chứ không bằng độ phức tạp.
••• Dùng treap để chèn/xoá cả đoạn của một dãy	Bài tập thi đấu kinh điển	Treap theo chỉ số ngẫu định — ứng dụng mạnh nhất của treap và là lý do chính để tự cài nó.
••• Cài B-cây bậc lớn và đo số lần truy cập nút so với cây nhị phân	Bài tập hệ thống	Mô phỏng chi phí đọc khối; đây là cách rẻ nhất để thấy vì sao cơ sở dữ liệu chọn B-cây.
••• Cây k -d cho tìm điểm gần nhất trên đám mây điểm 3D	Bài tập ứng dụng	So với vét cạn về thời gian; thử phân bố lệch để thấy cây mất cân bằng.

Kết chương và đọc tiếp

Cây tìm kiếm là câu chuyện về một bất biến: chiều cao phải là $O(\log n)$. Mọi cấu trúc trong chương chỉ khác nhau ở cách trả tiền cho bất biến đó — quay theo luật chặt, quay theo luật lỏng, quay theo độ ưu tiên ngẫu nhiên, hay làm nút to ra cho hợp với đơn vị chi phí của đĩa. Kỹ thuật đáng mang theo nhất không phải luật cân bằng mà là *tăng cường*, cùng điều kiện của nó: thông tin thêm phải tổng hợp được từ các con.

Chương tiếp theo lấy đúng ý tưởng tăng cường ấy và đẩy tới cùng cho một bài toán rất cụ thể: truy vấn trên đoạn kèm cập nhật. Bạn sẽ gặp lại điều kiện “tổng hợp được từ các con” dưới cái tên chính xác của nó — tính kết hợp — và sẽ thấy hai cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả phần B là Fenwick và cây phân đoạn (B/10).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Sleator & Tarjan, “Self-Adjusting Binary Search Trees”	Journal of the ACM, 1985	Bài gốc của splay tree. Đọc phần phát biểu tối ưu tĩnh và tập làm việc; phần chứng minh là ví dụ mẫu mực cho phương pháp thế năng (A/03).
nền tảng	Demaine, Harmon, Iacono, Pătraşcu, “Dynamic Optimality—Almost”	SIAM Journal on Computing, 2007	Tango tree và cận dưới đàn xen. Đọc mục giới thiệu để hiểu phát biểu của phỏng đoán tối ưu động, kể cả khi bỏ qua chi tiết kỹ thuật.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Knuth, “Optimum Binary Search Trees”	Acta Informatica, 1971	Cây tìm tối ưu tính được bằng quy hoạch động $O(n^2)$ nhờ tính đơn điệu của điểm chia — cùng kết quả sẽ trở lại ở C/16.
tra khi cần	Demaine, khoá học MIT 6.851 <i>Advanced Data Structures</i>	tài liệu khoá học công khai, MIT	Bài giảng về cây bền vững, tối ưu động và các cấu trúc gần biên; hợp để đi tiếp sau chương này.

Cây chỉ số và truy vấn đoạn (Fenwick, Segment Trees, Range Queries)

PHỤ THUỘC B/06 cho tổng tiền tố — chương này là câu trả lời khi mảng thay đổi; B/09 cho ý tưởng tăng cường; A/05 cho tính kết hợp.

TÓM TẮT

Chương B/06 cho truy vấn đoạn $O(1)$ với điều kiện mảng đứng yên. Chương này bỏ điều kiện đó, và cái giá phải trả là $O(\log n)$ cho cả truy vấn lẫn cập nhật. Bốn cấu trúc: Fenwick — gọn cho tổng/XOR với cập nhật điểm; cây phân đoạn — tổng quát nhất, thêm lazy propagation thì cập nhật cả đoạn cũng $O(\log n)$; sparse table — $O(1)$ truy vấn nhưng mảng phải tĩnh; và phân rã căn bậc hai — chậm hơn nhưng dùng được cho những phép toán mà ba cấu trúc kia chịu. Các cấu trúc gộp đoạn cần trạng thái đủ thông tin và phép gộp kết hợp; Mo cần thao tác thêm/bớt ở biên đủ rẻ. Nếu chỉ nhớ một điều: chọn cấu trúc bằng cách trả lời hai câu — phép toán có kết hợp không, và mảng có thay đổi không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phép toán kết hợp (associative)** — phép $\oplus$ thoả $(a \oplus b) \oplus c = a \oplus (b \oplus c)$. Điều kiện để gộp kết quả của hai nửa thành kết quả của cả đoạn — nền của mọi cấu trúc trong chương.
- **phần tử trung hoà** — giá trị e với $a \oplus e = e \oplus a = a$; cần để trả lời truy vấn trên đoạn rỗng mà không phải viết trường hợp riêng. Cùng với phép kết hợp, ta có một *vị nhóm* (monoid).
- **Fenwick (binary indexed tree)** — mảng phụ trong đó ô i giữ tổng của một đoạn có độ dài bằng bit thấp nhất của i ; cập nhật điểm và tổng tiền tố đều $O(\log n)$, code chỉ vài dòng.
- **cây phân đoạn (segment tree)** — cây nhị phân trong đó mỗi nút giữ kết quả gộp của một đoạn; lá là phần tử đơn, gốc là cả mảng.
- **lazy propagation** — kỹ thuật hoãn việc đẩy cập nhật xuống các nút con cho tới khi thật sự cần, để cập nhật cả đoạn cũng chỉ $O(\log n)$.
- **sparse table** — bảng tiền xử lý $f[i][k] =$ kết quả gộp của đoạn dài 2^k bắt đầu tại i ; truy vấn $O(1)$ cho phép toán *luỹ đẳng* như cực trị, nhưng mảng phải tĩnh.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao Fenwick không dùng được cho truy vấn cực đại, trong khi cây phân đoạn thì được?
- Q2. Sparse table trả lời cực đại trong $O(1)$ còn cây phân đoạn cần $O(\log n)$. Vì sao sparse table không thay thế được cây phân đoạn?
- Q3. Lazy propagation hoãn việc cập nhật lại. Vì sao việc hoãn đó không làm sai kết quả, và lập luận nào bảo đảm điều đó?
- Q4. Cùng $O(\log n)$ nhưng Fenwick thường nhanh hơn cây phân đoạn vài lần. Khác biệt nằm ở đâu?
- Q5. Có phép toán nào mà cả Fenwick lẫn cây phân đoạn đều chịu, nhưng phân rã căn bậc hai vẫn giải được không?

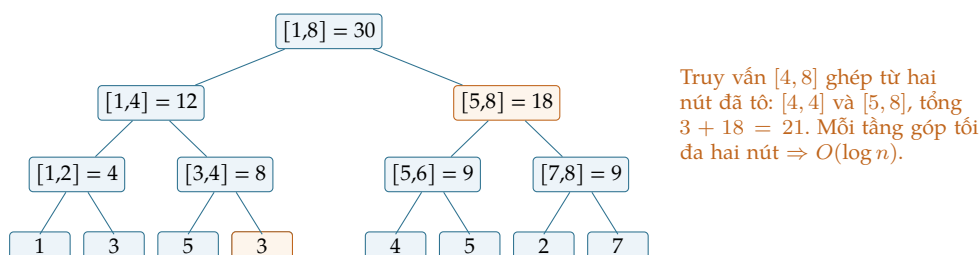
Cuối chương B/06 còn một câu hỏi bỏ ngỏ. Tổng tiền tố cho truy vấn đoạn trong $O(1)$, nhưng chỉ khi mảng đứng yên; mảng hiệu cho cập nhật đoạn trong $O(1)$, nhưng chỉ khi ta đọc kết quả một lần ở cuối. Bài toán thật thường trộn cả hai: cập nhật và truy vấn xen kẽ, mỗi loại hàng trăm nghìn lần.

Không có biểu diễn tĩnh nào làm được cả hai, và lý do khá sâu: tổng tiền tố nhanh vì mỗi ô của nó tóm tắt *rất nhiều* phần tử, nên sửa một phần tử làm hỏng rất nhiều ô. Đây là một đánh đổi thật sự, không phải thiếu sót của kỹ thuật. Lời giải của chương này là một sự thỏa hiệp có cấu trúc: thay vì mỗi ô tóm tắt một tiền tố dài, hãy để mỗi ô tóm tắt một đoạn có độ dài *lũy thừa của 2*, xếp thành cây. Khi đó mỗi phần tử chỉ nằm trong $O(\log n)$ đoạn, và mỗi đoạn truy vấn ghép được từ $O(\log n)$ mảnh.

10.1 Cây phân đoạn: ý tưởng tổng quát nhất

Chia mảng làm đôi, mỗi nửa lại chia đôi, cho tới khi còn một phần tử. Mỗi nút của cây giữ kết quả gộp của đoạn tương ứng. Với tổng, nút cha bằng tổng hai con; với cực đại, nút cha bằng cực đại hai con.

Truy vấn đoạn $[l, r]$: đi từ gốc xuống, nếu đoạn của nút nằm gọn trong $[l, r]$ thì trả kết quả đã lưu, nếu không giao thì bỏ qua, nếu giao một phần thì đi tiếp cả hai con. Có thể chứng minh mỗi tầng đóng góp tối đa hai nút được “dùng nguyên”, nên chi phí là $O(\log n)$. Cập nhật một phần tử: đi từ lá lên gốc, cập nhật $O(\log n)$ nút trên đường.



Hình 10.1. Cây phân đoạn cho mảng tám phần tử. Điều đáng chú ý: truy vấn không đi tới lá mà dừng lại ngay khi gặp một nút nằm gọn trong đoạn hỏi. Đây cũng là chỗ giải thích vì sao lazy propagation hoạt động — nếu ta dừng ở một nút thì mọi thứ bên dưới nó không cần chính xác ngay lúc này.

Điều kiện để mọi thứ trên đúng là *tính kết hợp*: kết quả của cả đoạn phải ghép được từ kết quả hai nửa. Tổng, tích, cực đại, cực tiểu, GCD, XOR, số lượng, “đoạn con có tổng lớn nhất” đều thỏa. Ngược lại, “phần tử xuất hiện nhiều nhất trong đoạn” thì *không* — biết đáp án của hai nửa không đủ để suy ra đáp án của cả đoạn — và đó là lý do bài đó cần công cụ khác, ví dụ thuật toán Mo ở mục 5.

Nhận xét này là cùng một điều kiện với “thông tin tăng cường phải tổng hợp được từ hai con” ở chương B/09, nay được gọi bằng tên toán học của nó. Trong ngôn ngữ đại số, ta cần một *vị nhóm*: một phép kết hợp cộng một phần tử trung hoà. Phần tử trung hoà không phải chi tiết hình thức — nó chính là giá trị bạn trả về cho đoạn rỗng, và quên nó là nguồn lỗi cài đặt phổ biến (trả về 0 cho truy vấn cực đại là sai nếu mảng có số âm; phải trả về $-\infty$).

Lazy propagation: hoãn cho tới khi thật cần

Cập nhật một phần tử là $O(\log n)$, nhưng cập nhật cả một đoạn — “cộng 5 vào mọi phần tử trong $[l, r]$ ” — nếu làm từng phần tử thì thành $O(n \log n)$. Lazy propagation giải quyết bằng một ý rất giống ý của mảng hiệu: *ghi nợ*.

Khi một cập nhật phủ trọn đoạn của một nút, ta cập nhật giá trị tổng hợp của nút đó và ghi một “khoản nợ” vào nút, rồi *dừng lại* — không đi xuống các con. Chỉ khi nào một truy vấn hay cập nhật sau này cần đi xuống dưới nút đó, ta mới đẩy khoản nợ xuống hai con.

Vì sao không sai? Vì bất biến được duy trì là: *giá trị lưu tại mỗi nút đã đúng, còn khoản nợ tại nút mô tả phần chưa được áp cho các con*. Mọi đường đi xuống đều đẩy nợ trước khi đọc con, nên không ai đọc dữ liệu cũ. Đây lại là một ví dụ của mẫu “trì hoãn công việc cho tới khi có người thật sự cần” — cùng ý với xoá lười ở chương B/07.

10.2 Fenwick: ngắn hơn, nhanh hơn, hẹp hơn

Fenwick làm cùng việc với cây phân đoạn cho trường hợp tổng, nhưng bằng một mẹo chỉ số rất gọn: ô i giữ tổng của đoạn kết thúc tại i có độ dài bằng *bit thấp nhất* của i . Nhờ vậy tổng tiền tố tới i ghép từ các ô i , $i - \text{lowbit}(i)$, $\dots$ — số ô bằng số bit 1 của i , tối đa $\log n$.

Ưu điểm là rất thực dụng: code khoảng năm dòng, bộ nhớ đúng n số, và thường có hằng số nhỏ. Các bước truy cập nhảy theo lowbit, không phải quét tuần tự; mức nhanh hơn phải đo trên cách cài và dữ liệu cụ thể.

Cái giá của khuôn Fenwick tổng/XOR: truy vấn đoạn dùng nghịch đảo để loại tiền tố trái, còn cập nhật điểm kiểu cộng dồn dùng tính giao hoán. Không nên suy rộng rằng mọi biến thể Fenwick đều cần nghịch đảo: cực tiểu tiền tố với cập nhật chỉ giảm vẫn làm được, nhưng không hỗ trợ gán tùy ý hay suy ra cực tiểu đoạn bằng hai tiền tố. Tổng và XOR có; cực đại thì không — biết cực đại của $[1, r]$ và của $[1, l - 1]$ không suy ra được cực đại của $[l, r]$. Đó là câu trả lời cho câu hỏi mở đầu thứ nhất, và cũng là ranh giới rõ ràng giữa hai cấu trúc.

Hai mở rộng đáng biết: Fenwick cũng làm được “cập nhật đoạn, truy vấn điểm” bằng cách chạy trên mảng hiệu; và làm được cả “cập nhật đoạn, truy vấn đoạn” bằng hai Fenwick với một chút đại số. Ngoài ra, có thể tìm “vị trí nhỏ nhất mà tổng tiền tố vượt x ” trong $O(\log n)$ bằng cách đi xuống theo bit, với điều kiện các phần tử không âm để tổng tiền tố đơn điệu — kỹ thuật thay thế cho cây thống kê thứ tự ở B/09 và thường nhanh hơn nhiều.

10.3 Sparse table: trả trước tất cả, với điều kiện mảng tĩnh

Nếu mảng không bao giờ thay đổi và phép toán *luỹ đẳng* — tức $a \oplus a = a$, đúng với cực đại, cực tiểu, GCD — thì ta có thể làm tốt hơn $O(\log n)$. Tiền xử lý $f[i][k] =$ kết quả gộp của đoạn dài 2^k bắt đầu tại i , tốn $O(n \log n)$ thời gian và bộ nhớ. Truy vấn $[l, r]$: lấy $k = \lfloor \log_2(r - l + 1) \rfloor$ và gộp hai đoạn dài 2^k , một bắt đầu tại l , một kết thúc tại r . Hai đoạn này *chồng lên nhau*, và đó chính là chỗ cần tính lũy đẳng — với tổng thì cách này sai vì phần chồng bị đếm hai lần.

Vậy nên bảng so sánh dưới đây tóm tắt lựa chọn, và nó đáng thuộc vì đây là quyết định bạn phải ra rất nhiều lần.

Cấu trúc	Truy vấn	Cập nhật	Điều kiện	Chọn khi
Tổng tiền tố (B/06)	$O(1)$	$O(n)$	có nghịch đảo	Mảng tĩnh, phép có nghịch đảo
Sparse table	$O(1)$	$O(n)$ sửa ô bị ảnh hưởng	kết hợp, lũy đẳng	Mảng tĩnh, hỏi cực trị rất nhiều
Fenwick	$O(\log n)$	$O(\log n)$	tổng/XOR, cộng dồn	Tổng có cập nhật; ưu tiên vì ngắn và nhanh
Cây phân đoạn	$O(\log n)$	$O(\log n)$	kết hợp	Phép toán bất kỳ có kết hợp; cần cập nhật đoạn (lazy)
Phân rã $\sqrt{n}$	$O(\sqrt{n})$	$O(1)$ hoặc $O(\sqrt{n})$	gộp khối đủ rẻ	Tổng/cực trị theo khối; kiểm tra chi phí gộp

10.4 Phân rã căn bậc hai và thuật toán Mo

Khi không tìm được trạng thái gọn để gộp hai đoạn, hãy xét xử lý theo khối; đây không phải lời giải tự động cho mọi phép toán không kết hợp. Lúc đó vẫn còn một hướng: chia mảng thành $\sqrt{n}$ khối, mỗi khối $\sqrt{n}$ phần tử, và giữ thông tin tổng hợp cho từng khối. Truy vấn đoạn xử lý các khối nằm trọn bằng thông tin đã lưu, và duyệt tay các phần tử ở hai đầu — tổng cộng $O(\sqrt{n})$ nếu mỗi khối được gộp trong $O(1)$.

Cần thiết kế và phân tích thao tác gộp/cập nhật khối; đổi lại cấu trúc khá linh hoạt, và code thường ngắn hơn cây phân đoạn có lazy. Trong nhiều bài, $O(n\sqrt{n})$ vẫn nằm trong ngân sách của chương A/03 và đó là lựa chọn hợp lý — đặc biệt khi bạn còn ít thời gian trong kỳ thi.

Một biến thể rất đẹp là *thuật toán Mo*: khi mọi truy vấn được biết trước (truy vấn ngoại tuyến), hãy sắp xếp chúng theo khối của đầu trái rồi theo đầu phải, và di chuyển hai con trỏ dần dần từ truy vấn này sang truy vấn kế tiếp. Tổng quãng đường của hai con trỏ là $O((n+q)\sqrt{n})$, và ta giải được những bài mà cây phân đoạn không đựng tới được — ví dụ “có bao nhiêu giá trị phân biệt trong đoạn” hay “phần tử xuất hiện nhiều nhất trong đoạn”. Chú ý điều kiện đánh đổi: kỹ thuật này đòi truy vấn phải biết trước, tức là bạn mua tốc độ bằng cách từ bỏ tính trực tuyến.

10.5 Ba mở rộng nên biết là có

Cây phân đoạn bền vững. Mỗi cập nhật chỉ đụng $O(\log n)$ nút, nên thay vì sửa tại chỗ, ta tạo bản sao của đúng đường đó và chia sẻ phần còn lại với phiên bản cũ. Kết quả: giữ được mọi phiên bản với chi phí $O(\log n)$ mỗi lần. Ứng dụng kinh điển là truy vấn “phần tử nhỏ thứ k trong đoạn $[l, r]$ ” — giải bằng hiệu của hai phiên bản.

Hai chiều. Cây phân đoạn của các cây phân đoạn cho truy vấn trên hình chữ nhật với $O(\log^2 n)$. Tốn bộ nhớ, và trong thực tế thường thay bằng Fenwick hai chiều nếu chỉ cần tổng.

Cây phân đoạn “beats”. Xử lý được các cập nhật lạ như “gán mọi phần tử trong đoạn bằng $\min(a_i, x)$ ” với phân tích khấu hao tinh vi. Xếp vào loại biết là có, tra khi gặp.

CẠM BẮY

Lỗi cài đặt phổ biến nhất với cây phân đoạn không nằm ở logic cây mà ở *phần tử trung hoà*. Truy vấn cực đại trên đoạn rỗng phải trả về $-\infty$, không phải 0; truy vấn GCD trên đoạn rỗng phải trả về 0, không phải 1. Lỗi này ẩn kỹ vì nó chỉ lộ ra khi mảng có số âm hoặc khi truy vấn rơi vào nhánh không giao. Cách phòng: viết rõ phần tử trung hoà thành một hằng số có tên, và tự hỏi “nếu đoạn rỗng thì hàm này trả về gì”.

10.6 Biến cấu trúc tĩnh thành cấu trúc động: phép biến đổi Bentley–Saxe

Sparse table ở mục 3 trả lời truy vấn trong $O(1)$ nhưng đòi hỏi mảng đứng yên. Đó không phải trường hợp cá biệt: rất nhiều cấu trúc mạnh đều là cấu trúc *tĩnh* — mảng đã sắp cộng tìm kiếm nhị phân, bao lồi dựng sẵn, cây phân hoạch không gian. Câu hỏi tự nhiên là có cách chung nào thêm phép chèn vào một cấu trúc tĩnh bất kỳ không, thay vì phải thiết kế lại từ đầu cho từng bài. Bentley và Saxe[47] trả lời là có, với một điều kiện rõ ràng, và lời giải của họ đơn giản tới mức nên thuộc.

Điều kiện: bài toán truy vấn phải *phân rã được*, nghĩa là khi tập dữ liệu tách thành hai phần rời nhau thì đáp án trên cả tập gộp được từ hai đáp án thành phần bằng một phép toán rẻ. Cụ thể, cực tiểu, cực đại, tổng, “có tồn tại phần tử thoả tính chất này không”, “điểm gần nhất tới q ” đều phân rã được. “Phần tử nhỏ thứ k ” thì không — biết phần tử thứ k của mỗi nửa không cho biết phần tử thứ k của cả tập.

Cách dựng đọc như một phép đếm nhị phân, và đó chính là trực giác nên giữ. Ta không nuôi một cấu trúc lớn mà nuôi một *bộ* cấu trúc tĩnh có kích thước 1, 2, 4, 8, ..., mỗi lũy thừa của hai có nhiều nhất một cái — đúng như biểu diễn nhị phân của số phần tử hiện có. Chèn một phần tử là dựng một cấu trúc kích thước 1; để có hai cấu trúc cùng kích thước thì huỷ cả hai và dựng lại một cấu trúc tĩnh kích thước gấp đôi, đúng như việc nhó 1 khi cộng nhị phân. Truy vấn thì hỏi cả $O(\log n)$ cấu trúc rồi gộp các đáp án lại.

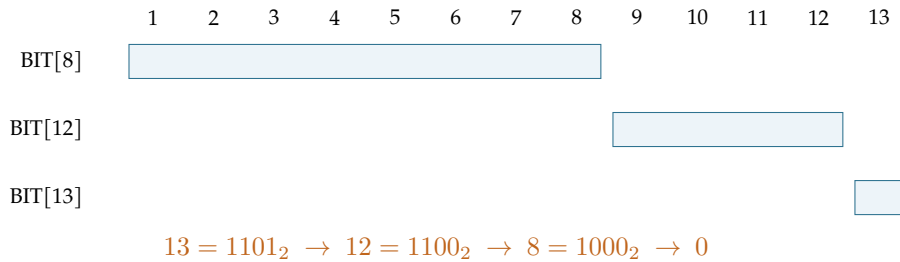
Chi phí suy ra trực tiếp từ hình ảnh đó. Mỗi phần tử chỉ bị dựng lại một lần cho mỗi tầng nó leo qua, tức là $O(\log n)$ lần trong cả đời; nên nếu dựng cấu trúc tĩnh cho n phần tử tốn $P(n)$ thì chèn tốn $O(\frac{P(n)}{n} \log n)$ khấu hao, còn truy vấn tốn thêm một thừa số $O(\log n)$ so với bản tĩnh. Lập luận này chính là lập luận bộ đếm nhị phân ở chương A/03, xuất hiện lại ở một vai trò hoàn toàn khác.

Ba hệ quả thực dụng. Một, khi bạn có một cấu trúc tĩnh viết sẵn và bài lại đòi thêm phần tử *trực tuyến*, đừng vội đi tìm cấu trúc động tương ứng — hãy hỏi bài toán có phân rã được không đã. Hai, phép biến đổi này không cho phép xoá; muốn xoá phải dùng các biến thể khác của cùng dòng nghiên cứu, và trong bài thi thì thường rẻ hơn nhiều nếu chuyển sang xử lý ngoại tuyến bằng chia để trị trên trục thời gian (C/14). Ba, nếu đã có một cấu trúc động tự nhiên — cây phân đoạn cho các phép toán kết hợp là ví dụ ngay trong chương này — thì dùng nó, vì phép biến đổi tổng quát luôn phải trả thêm một thừa số lôgarit cho tính tổng quát.

Hình dạng “nhiều tầng, mỗi tầng gấp đôi, hợp nhất khi đầy, truy vấn phải hỏi mọi tầng” còn xuất hiện ở nơi khác hẳn: các động cơ lưu trữ ghi nhiều đọc ít trong cơ sở dữ liệu hiện đại tổ chức dữ liệu đúng theo kiểu đó. Sự trùng hình dạng ấy không phải quan hệ vay mượn có chứng cứ lịch sử, nhưng nó cho thấy cùng một ràng buộc — ghi phải rẻ, và cấu trúc tĩnh thì rẻ hơn cấu trúc động — dẫn tới cùng một lời giải ở hai ngành cách nhau ba chục năm.

10.7 Nhìn lowbit và kiểm hợp thành lazy bằng tay

Hình 10.2 triển khai trực tiếp truy vấn tiền tố 13. Mỗi bước xoá bit 1 thấp nhất nên các khối vừa khít, không chồng lấn. Để tự kiểm cài đặt, thử thêm Δ tại vị trí 11: với $n = 16$, các ô bị cập nhật phải là 11, 12, 16.



Hình 10.2. Tiền tố $[1, 13]$ là hợp rời của $[13, 13]$, $[9, 12]$, $[1, 8]$. Vì thế tổng cần đúng ba ô Fenwick: $\text{BIT}[13] + \text{BIT}[12] + \text{BIT}[8]$.

Lazy propagation còn đòi cập nhật tác động được lên trạng thái tổng hợp và hợp thành được trong thời gian hằng số[48]. Với cộng đoạn và hỏi tổng, lưu (sum, len) ; cộng d biến nó thành $(sum + d \cdot len, len)$. Không lưu độ dài thì không biết phải tăng tổng bao nhiêu. Với cả gán và cộng, thứ tự là phần của thuật toán: gán 5 rồi cộng 3 cho kết quả 8, còn cộng 3 rồi gán 5 cho kết quả 5. Hãy kiểm hai chuỗi thao tác này trên mảng hai phần tử trước khi thử ngẫu nhiên.

CẠM BẤY

Không phải mọi phép gộp kết hợp đều tự động hỗ trợ mọi cập nhật lazy. Ví dụ chỉ lưu tổng thì không đủ cập nhật $a_i \leftarrow \min(a_i, x)$: hai đoạn $[0, 10]$ và $[5, 5]$ cùng tổng 10, nhưng chặn trên ở 5 cho tổng 5 và 10. Muốn xử lý phải thêm thông tin và chứng minh chi phí, như segment tree beats.

10.8 Phản biện, nhược điểm và rủi ro triển khai

Cây phân đoạn và Fenwick thường tạo ra ảo tưởng rằng mọi bài toán truy vấn đoạn đều có thể giải quyết nhanh chóng bằng cách gõ một khung cây quen thuộc. Tuy nhiên, cái giá về tài nguyên và các cạm bẫy logic khi cài đặt lazy propagation là những rào cản rất thực tế.

Phản biện: sự lạm dụng cây phân đoạn (Segment Tree Overkill). Cây phân đoạn là cấu trúc bị lạm dụng nhiều nhất trong lập trình thi đấu. Khi thấy truy vấn đoạn, người mới thường lập tức gõ cây phân đoạn dài cả trăm dòng mã. Nhưng nếu dữ liệu tĩnh, Sparse Table cho phép trả lời RMQ trong thời gian $O(1)$ tuyệt đối — nhanh hơn $O(\log n)$ từ 5 tới 10 lần. Nếu bài toán chỉ cần cập nhật điểm và truy vấn tổng/ngịch đảo, cây Fenwick viết chỉ trong 10 dòng, chạy nhanh gấp 3–5 lần nhờ tận dụng hoàn hảo cache L1 và chỉ tốn đúng $1N$ bộ nhớ thay vì $4N$. Lạm dụng cây phân đoạn không chỉ làm code cồng kềnh, khó gỡ lỗi mà còn khiến hệ số hằng số thời gian phình to, dễ dẫn tới lỗi quá thời gian (TLE) một cách đáng tiếc.

Nhược điểm cốt lõi về tài nguyên và giới hạn cấu trúc:

- **Gánh nặng bộ nhớ $4N$ và phân mảnh heap:** Cây phân đoạn dạng mảng cần mảng kích thước tối thiểu $4N$. Nếu mỗi nút phải lưu các cấu trúc phức tạp (như mảng vector hay ma trận kích thước nhỏ), dung lượng RAM sẽ bùng nổ hàng trăm megabyte. Nếu cài đặt bằng con trỏ động (`new Node`), chi phí lưu con trỏ 64-bit và sự phân mảnh bộ nhớ do cấp phát rải rác trên heap sẽ làm tốn thêm gấp đôi dung lượng và phá hủy hoàn toàn tính liên tục của cache phần cứng.
- **Giới hạn cố hữu của cây Fenwick:** Fenwick hoàn toàn không hỗ trợ truy vấn đoạn $[l, r]$ tùy ý cho các phép toán thiếu tính nghịch đảo (như $\min$, $\max$, $\gcd$) khi mảng có cập nhật điểm tùy ý. Ta không thể suy ra cực tiểu đoạn $[l, r]$ chỉ từ tiền tố $[1, r]$ và $[1, l - 1]$ vì thông tin đã bị xoá sổ.

- *Tính bất động tuyệt đối của Sparse Table*: Sparse Table đổi thời gian tiền xử lý $O(n \log n)$ và bộ nhớ $O(n \log n)$ để lấy truy vấn $O(1)$. Nhưng chỉ cần một giá trị trong mảng thay đổi, có tới $O(n)$ ô bảng bị ảnh hưởng. Việc cập nhật động trên Sparse Table là hoàn toàn bất khả thi.

Rủi ro vận hành và các lỗi im lặng nguy hiểm:

- *Xung đột thứ tự của các nhãn Lazy (Tag Composition Pitfall)*: Khi bài toán có nhiều loại thao tác cập nhật đồng thời (như vừa gán đề giá trị vừa cộng dồn, hoặc vừa nhân vừa cộng), các phép toán lazy không có tính giao hoán. Nếu không xác định rõ thứ tự đại số và cập nhật đồng thời cả hai nhãn (ví dụ phép gán đề phải xoá sạch nhãn cộng trước đó), cây sẽ âm thầm trả về kết quả sai lệch hoàn toàn.
- *Quên đẩy nợ (Missing Push-Down)*: Trong cây phân đoạn có lazy propagation, chỉ cần quên gọi hàm push_down ở đúng một nhánh rẽ (ví dụ khi đoạn truy vấn chỉ chạm một phần nút con), nút con sẽ tiếp tục mang giá trị lỗi thời. Cây vẫn chạy và trả về số, nhưng kết quả sai một cách im lặng và cực kỳ khó tái hiện trên các bộ test nhỏ.
- *Tràn số nguyên trong phép nhân độ dài đoạn*: Khi truyền nhãn lazy cộng giá trị v xuống một đoạn có độ dài len , giá trị cộng vào tổng là $v \cdot len$. Với $v \approx 10^9$ và $len \approx 10^5$, tích này vượt ngưỡng 10^{14} . Nếu viết `tree[node] += v * len` mà không ép kiểu sang long long, phép nhân sẽ bị tràn số 32-bit thành số âm trước khi cộng vào tổng.

CẠM BÃY

Mảng cây phân đoạn 1-indexed phải có kích thước tối thiểu $4N$, không phải $2N$. Kích thước $2^{\lceil \log_2 N \rceil + 1} - 1 < 4N$ là cận trên chặt chẽ; khai báo thiếu kích thước (ví dụ khai báo $2N$ hay $3N$) sẽ gây tràn bộ nhớ (Out-of-Bounds Write) ở các tầng lá sâu nhất, dẫn tới lỗi bộ nhớ ngầm cực kỳ khó truy vết.

10.9 Gốc rễ: từ nén dữ liệu và hình học tới bài toán truy vấn đoạn

Điều thú vị là hai cấu trúc chính của chương này ra đời ở hai ngành hoàn toàn khác nhau và không phải để giải bài “truy vấn đoạn” như ta biết ngày nay.

Cây phân đoạn xuất hiện trong *hình học tính toán* cuối thập niên 1970, gắn với tên Jon Bentley. Bài toán gốc là bài toán đâm xuyên: cho một tập đoạn thẳng trên đường thẳng, hỏi một điểm bị bao nhiêu đoạn phủ. Cấu trúc phân cấp theo đoạn ra đời từ đó, và mãi về sau cộng đồng thuật toán mới dùng nó cho bài toán truy vấn tổng/cực trị trên mảng — ứng dụng mà ngày nay hầu như là ứng dụng duy nhất người ta nhắc tới.

Fenwick ra đời năm 1994 từ một bài toán *nén dữ liệu*. Trong mã hoá số học, bộ mã cần liên tục cập nhật tần suất của từng ký hiệu và liên tục hỏi tổng tần suất lũy tích — đúng bài toán “cập nhật điểm, truy vấn tiền tố”. Peter Fenwick tìm ra cách biểu diễn bằng bit thấp nhất và công bố nó như một giải pháp cho bài toán nén; cấu trúc gọn tới mức ngày nay nó là thứ đầu tiên người ta viết ra khi cần tổng đoạn có cập nhật.

Cả hai câu chuyện minh hoạ cùng một điều đáng nhớ về cách kiến thức lan truyền: cấu trúc dữ liệu hay được sinh ra để phục vụ một ứng dụng rất hẹp, rồi được cộng đồng khác nhận ra là công cụ tổng quát. Với người học, hệ quả thực dụng là: khi bạn gặp một bài lạ, rất có thể công cụ cần dùng đã tồn tại ở một ngành khác dưới một cái tên khác — và đó cũng là lý do phần *Liên hệ ngang* có mặt trong mọi chương của quyển sách này.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Cuối thập niên 1970 — Bentley và hình học tính toán	Bài toán đâm xuyên: một điểm bị bao nhiêu đoạn phủ.	Cây phân đoạn ra đời cho hình học; ứng dụng “truy vấn tổng trên mảng” đến sau và trở thành ứng dụng chính.
1994 — Fenwick	Mã hoá số học cần cập nhật tần suất và hỏi tổng lũy tích liên tục.	Cấu trúc dựa trên bit thấp nhất; ngắn tới mức thành lựa chọn mặc định cho tổng đoạn có cập nhật.
Thập niên 1980–2000 — bài toán RMQ và LCA	Truy vấn cực tiểu trên đoạn xuất hiện khắp nơi, cần lời giải $O(1)$.	Sparse table và các lời giải tiền xử lý tuyến tính; nối RMQ với bài toán tổ tiên chung gần nhất trên cây (C/17).
Thập niên 2000 — cộng đồng thi đấu	Nhiều bài truy vấn không có tính kết hợp nên cây chịu.	Phân rã căn bậc hai và thuật toán Mo: đổi tính trực tuyến lấy khả năng xử lý phép toán bất kỳ.
Thập niên 2000 — cấu trúc bền vững	Cần truy vấn trạng thái ở các thời điểm quá khứ.	Cây phân đoạn bền vững bằng chia sẻ đường đi; giải được “phần tử nhỏ thứ k trong đoạn”.

10.10 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Dại số trừu tượng	Vị nhóm: phép kết hợp cộng phần tử trung hoà	Không phải chỉ tương tự — đây đúng là điều kiện toán học để cây phân đoạn hoạt động. Biết khái niệm này giúp trả lời ngay câu “bài này dùng cây phân đoạn được không”: chỉ cần kiểm tính kết hợp.
Thị giác máy tính, đồ hoạ	Kim tự tháp ảnh, mipmap, ảnh tích phân	Cùng ý tưởng tổng hợp phân cấp: mỗi tầng lưu bản tóm tắt của tầng dưới với độ phân giải giảm nửa. Khác: ở đó mục tiêu là lọc và lấy mẫu theo tỉ lệ, ở đây là trả lời truy vấn nhanh.
Cơ sở dữ liệu phân tích	Khối dữ liệu OLAP, bảng tổng hợp trước, chuỗi thời gian có tổng theo cấp	Cùng đánh đổi trả trước để truy vấn rẻ, và cùng vấn đề: dữ liệu thay đổi thì phải cập nhật tổng hợp. Cây phân đoạn chính là bản trong bộ nhớ của ý tưởng đó.
Xử lý tín hiệu	Biến đổi wavelet, phân tích đa phân giải	Cùng cấu trúc phân cấp nhị phân trên miền chỉ số. Bài học chung: biểu diễn đa mức cho phép mô tả cả thay đổi cục bộ lẫn tổng thể với chi phí lôgarit.
Tính toán song song	Cây rút gọn (reduce) và scan trên GPU	Cây phân đoạn chính là cây rút gọn được lưu lại thay vì tính xong rồi bỏ. Điều kiện dùng được cũng giống nhau: phép toán phải kết hợp — đó là lý do reduce trong mọi thư viện song song đều yêu cầu điều kiện này.
Lập trình hàm	Cấu trúc bền vững, chia sẻ cấu trúc	Cây phân đoạn bền vững là ứng dụng trực tiếp của kỹ thuật chia sẻ đường đi[46]; cùng lý do vì sao cây hợp với bất biến còn mảng thì không.
Hệ thống giám sát	Cửa sổ thời gian, tổng hợp theo cấp phút/giờ/ngày	Cùng cấu trúc phân cấp theo thời gian. Điểm giống thú vị: các hệ này cũng chỉ hỗ trợ được các phép tổng hợp có tính kết hợp, và “số giá trị phân biệt” cũng chính là phép khó ở đó — nên họ dùng cấu trúc xấp xỉ như HyperLogLog (B/08).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì Fenwick tính đoạn $[l, r]$ bằng *hiệu* của hai tiền tố, nên nó cần phép toán có nghịch đảo. Với cực đại thì biết max của $[1, r]$ và của $[1, l - 1]$ không suy ra được max của $[l, r]$ — thông tin đã mất. Cây phân đoạn không dùng phép trừ mà *ghép* kết quả của các đoạn con rời nhau, nên chỉ cần tính kết hợp chứ không cần nghịch đảo. Đây là ranh giới rõ ràng: có nghịch đảo thì Fenwick (ngắn hơn, nhanh hơn); chỉ có kết hợp thì cây phân đoạn.
- A2. Vì sparse table đòi mảng *tĩnh*: cập nhật một phần tử có thể làm hỏng $O(n)$ ô (ở tầng k , tối đa 2^k ô chứa điểm đó). Xây lại toàn bảng tốn $O(n \log n)$, nhưng sửa đúng các ô bị ảnh hưởng tốn $O(n)$. Nó cũng đòi phép toán lũy đẳng, vì hai đoạn dài 2^k dùng để phủ truy vấn có phần chồng lên nhau — với tổng thì phần chồng bị đếm hai lần. Vậy sparse table thắng đúng một tình huống: mảng không đổi, phép lũy đẳng, và số truy vấn rất lớn nên $O(1)$ đáng giá.
- A3. Vì bất biến được duy trì là: giá trị tổng hợp lưu tại mỗi nút *đã đúng*, còn khoản nợ tại nút chỉ mô tả phần chưa được truyền xuống các con. Mọi thao tác muốn đọc thông tin của con đều phải đi xuống qua nút cha, và ta luôn đẩy nợ trước khi đi xuống — nên không ai từng đọc dữ liệu chưa cập nhật. Đây là lập luận bất biến chuẩn của A/04, và nó cũng chỉ ra chỗ dễ sai nhất khi cài: quên đẩy nợ ở đúng một nhánh.
- A4. Ở hằng số, không ở bậc. Fenwick là một vòng lặp trên mảng với vài phép toán bit, không đệ quy, không cấp phát, truy cập gần như tuần tự nên rất hợp cache. Cây phân đoạn thường viết đệ quy, mỗi nút truy cập hai con ở vị trí xa nhau, và dùng gấp bốn lần bộ nhớ. Với cùng $\log n$ bước, chênh lệch thực tế thường là ba tới năm lần — đủ để quyết định giữa đạt và quá thời gian.
- A5. Có, và đó là lý do chính để biết phân rẽ căn bậc hai. Ví dụ “có bao nhiêu giá trị phân biệt trong đoạn $[l, r]$ ” không kết hợp được: biết số giá trị phân biệt của hai nửa không cho biết số giá trị phân biệt của cả đoạn, vì không biết phần chung. Thuật toán Mo giải được bằng cách di chuyển hai đầu mút dần dần và duy trì một bảng đếm — với cái giá là mọi truy vấn phải biết trước, tức là mất tính trực tuyến.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một dãy phố với một trăm cửa hàng, và bạn thường xuyên phải trả lời câu “tổng doanh thu từ cửa hàng số 20 tới số 57 là bao nhiêu?”. Nếu doanh thu không bao giờ đổi, bạn chỉ cần một bảng cộng dồn từ đầu phố — lấy số ở 57 trừ số ở 19 là xong.

Nhưng nếu mỗi ngày có vài cửa hàng báo lại doanh thu mới thì bảng cộng dồn hỏng: sửa một cửa hàng làm sai mọi con số phía sau nó.

Cách chữa là chia phố thành nhóm lồng nhau: hai nửa phố, mỗi nửa lại chia đôi, cứ thế xuống từng cửa hàng. Mỗi nhóm ghi sẵn tổng của nhóm mình. Khi một cửa hàng đổi số, chỉ những nhóm chứa nó — khoảng bảy nhóm với một trăm cửa hàng — phải sửa. Khi có người hỏi một khoảng, bạn ghép khoảng đó từ vài nhóm có sẵn thay vì cộng từng cửa hàng.

Còn nếu sếp nói “tăng thuế thêm 3% cho cả đoạn từ 20 tới 57” thì bạn không cần đi sửa từng cửa hàng. Bạn dán một mẫu giấy nợ lên các nhóm nằm trọn trong đoạn đó, và chỉ khi nào có ai hỏi chi tiết bên trong một nhóm thì mới truyền mẫu giấy ấy xuống các nhóm con. Đó là lazy propagation, và nó đúng vì bất kỳ ai muốn nhìn vào bên trong đều phải đi qua chỗ dán giấy.

Một điều cuối cùng đáng nhớ: cách này chỉ chạy được với những câu hỏi mà *biết đáp án của hai nửa thì suy ra được đáp án của cả đoạn*. Tổng thì được, lớn nhất thì được, nhưng “mặt hàng bán chạy nhất trong đoạn” thì không — và đó là ranh giới giữa những bài dùng được cây và những bài phải dùng cách khác.

Một câu: Chia đoạn thành các mảnh lồng nhau: sửa một chỗ chỉ đụng $\log n$ mảnh, hỏi một đoạn chỉ ghép $\log n$ mảnh.

Chỗ không rút gọn được: Việc phép toán có kết hợp hay không là tính chất của bài toán, không phải của cấu trúc — không có mẹo nào làm cho “giá trị phổ biến nhất” trở nên kết hợp được.

Thuật ngữ / câu hỏi	Trả lời
●●● Điều kiện chung của cả chương	Phép toán phải <i>kết hợp</i> : kết quả cả đoạn ghép được từ hai nửa. Cộng thêm phần tử trung hoà thì có một vị nhóm.
●●● Fenwick so với cây phân đoạn	Fenwick cần phép có <i>ngịch đảo</i> (dùng hiệu hai tiền tố), thường gọn hơn; tốc độ cần đo. Cây phân đoạn chỉ cần kết hợp, tổng quát hơn, làm được lazy.
●●● Vì sao truy vấn cây phân đoạn là $O(\log n)$	Mỗi tầng đóng góp tối đa hai nút được dùng nguyên vẹn; các nhánh còn lại hoặc nằm gọn (dùng) hoặc không giao (bỏ).
●●● Lazy propagation — bất biến	Giá trị tại nút đã đúng; khoản nợ mô tả phần chưa áp cho con. Mọi đường đi xuống đều đẩy nợ trước khi đọc con.
●●● Sparse table — điều kiện	Mảng tĩnh và phép <i>luỹ đẳng</i> ($a \oplus a = a$), vì hai đoạn phủ truy vấn chồng lên nhau. Tiền xử lý $O(n \log n)$, truy vấn $O(1)$.
●●● Bảng chọn cấu trúc	Tĩnh + có nghịch đảo: tổng tiền tố. Tĩnh + lũy đẳng: sparse table. Động + nghịch đảo: Fenwick. Động + chỉ kết hợp: cây phân đoạn. Không có trạng thái gộp gọn: xét chia khối hoặc Mo, rồi kiểm chi phí các thao tác.
●● Lỗi cài đặt phổ biến nhất	Phần tử trung hoà sai: trả 0 cho truy vấn cực đại trên đoạn rỗng khi mảng có số âm. Hãy đặt nó thành hằng số có tên.
●● Thuật toán Mo	Truy vấn ngoại tuyến, sắp theo khối của đầu trái rồi đầu phải, di chuyển hai con trỏ; $O((n + q)\sqrt{n})$. Giải được bài không kết hợp như đếm giá trị phân biệt.
●● Phân rã căn bậc hai	$\sqrt{n}$ khối, mỗi khối giữ tổng hợp; truy vấn $O(\sqrt{n})$. Cần gộp mỗi khối đủ rẻ; không tự động xử lý mọi phép toán.
●● Cây phân đoạn bền vững	Mỗi cập nhật sao chép đúng một đường $O(\log n)$ nút, chia sẻ phần còn lại; giải “phần tử nhỏ thứ k trong đoạn”.
●● Fenwick tìm vị trí theo tổng	Đi xuống theo bit để tìm vị trí nhỏ nhất có tổng tiền tố vượt x , $O(\log n)$ — thay được cây thống kê thứ tự ở B/09.
● Hai chiều	Fenwick 2D cho tổng trên hình chữ nhật; cây phân đoạn của cây phân đoạn cho $O(\log^2 n)$ với phép tổng quát hơn, tốn bộ nhớ.
●● Phép biến đổi Bentley–Saxe	Bài toán phân rã được cộng một bộ cấu trúc tính kích thước 1, 2, 4, ..., hợp nhất như phép cộng nhị phân: thêm được phần tử trực tuyến với giá một thừa số $O(\log n)$. Không hỗ trợ xoá.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên học Fenwick hay cây phân đoạn trước?

| Fenwick trước, vì nó ngắn, cho kết quả nhanh và dạy đúng ý tưởng cốt lõi “mỗi phần tử nằm trong $\log n$ đoạn tóm tắt”. Nhưng đừng dừng ở đó: cây phân đoạn mới là cấu trúc tổng quát, và một khi đã quen thì bạn viết nó gần như tự động. Lộ trình hợp lý là Fenwick cho tổng, rồi cây phân đoạn không lazy, rồi lazy, rồi mới tới các mở rộng.

2. Cây phân đoạn nên cài đệ quy hay lặp?

| Bản đệ quy dài hơn nhưng dễ mở rộng và bắt buộc nếu cần lazy; bản lặp (cây phân đoạn không đệ quy) ngắn hơn và nhanh hơn nhưng khó thêm lazy và khó xử lý các phép không giao hoán. Lời khuyên thực dụng: học kỹ bản đệ quy trước vì nó là bản dùng được cho mọi bài, rồi mới thêm bản lặp vào thư viện riêng cho những bài chỉ cần cập nhật điểm.

3. Bài yêu cầu “gán mọi phần tử trong đoạn bằng x ” thay vì cộng — lazy có khác gì không?

| Có một khác biệt quan trọng: phép gán *không giao hoán* với phép cộng, nên khi hai loại cập nhật cùng xuất hiện, bạn phải định nghĩa rõ cách gộp hai khoản nợ và thứ tự áp dụng. Quy tắc an toàn: biểu diễn khoản nợ dưới dạng một phép biến đổi affine ($a \mapsto ka + b$) và định nghĩa phép hợp thành của hai biến đổi. Khi đó mọi thứ trở lại đúng khuôn “vị nhóm”, chỉ là vị nhóm của các phép biến đổi thay vì của các giá trị.

4. Khi nào tôi nên dùng thuật toán Mo thay vì cố tìm cấu trúc dữ liệu?

| Khi cả ba điều kiện cùng đúng: mọi truy vấn biết trước; thao tác thêm/bớt một phần tử ở biên là $O(1)$; và $O((n+q)\sqrt{n})$ nằm trong ngân sách. Điều kiện thứ hai hay bị bỏ qua — nếu mỗi bước di chuyển con trỏ tốn $O(\log n)$ thì tổng thành $O((n+q)\sqrt{n} \log n)$ và thường quá giờ. Trước khi dùng Mo, hãy luôn tự hỏi “phép này có kết hợp không” một lần nữa, vì cây phân đoạn luôn tốt hơn khi dùng được.

5. Nén toạ độ là gì và vì sao nó hay đi kèm chương này?

| Là việc ánh xạ các giá trị thật — có thể tới 10^9 — về các chỉ số liên tiếp $0, 1, 2, \dots$ bằng cách sắp xếp và loại trùng. Nó cần thiết vì Fenwick và cây phân đoạn cấp phát bộ nhớ theo kích thước miền giá trị, chứ không theo số phần tử. Với 10^5 giá trị nằm rải rác trong $[0, 10^9]$, nén xuống còn 10^5 chỉ số làm cấu trúc khả thi. Đây là bước tiền xử lý gần như bắt buộc cho các bài đếm nghịch thế hay truy vấn theo giá trị.

6. Có cách nào cập nhật đoạn và truy vấn đoạn chỉ bằng Fenwick không?

| Có, dùng hai mảng Fenwick. Ý tưởng: viết tổng tiền tố sau các cập nhật đoạn dưới dạng một biểu thức tuyến tính theo chỉ số, rồi tách thành phần hệ số và phần hằng, mỗi phần một Fenwick. Kết quả là code vẫn ngắn hơn cây phân đoạn có lazy và nhanh hơn đáng kể — đáng để có sẵn trong thư viện riêng. Hạn chế: chỉ áp dụng cho tổng, không mở rộng sang cực trị.

7. Truy vấn cực tiểu trên đoạn có liên quan gì tới bài toán tổ tiên chung gần nhất trên cây?

| Hai bài quy dẫn được về nhau và đây là một liên hệ đẹp. Tổ tiên chung gần nhất của hai đỉnh chính là đỉnh có độ sâu nhỏ nhất trong đoạn tương ứng của dãy Euler khi duyệt cây — tức là một truy vấn cực tiểu trên đoạn. Ngược lại, mọi bài cực tiểu trên đoạn cũng dựng được thành bài tổ tiên chung trên cây Cartesian của mảng. Nhờ quy dẫn đó, một lời giải tốt cho bài này lập tức là lời giải tốt cho bài kia — chi tiết ở [C/17](#).

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn có cập nhật điểm	CSES “Dynamic Range Sum Queries”[33]	Fenwick; viết lại từ trí nhớ tới khi không cần tra.
● Cực tiểu đoạn với mảng tĩnh	CSES “Static Range Minimum Queries”[33]	Sparse table; giải thích vì sao cách này sai nếu đổi sang tổng.
●● Cực tiểu đoạn có cập nhật điểm	CSES “Dynamic Range Minimum Queries”[33]	Cây phân đoạn; chú ý phần tử trung hoà.
●● Đếm số nghịch thế bằng Fenwick sau khi nén toạ độ	Bài kinh điển	Hai kỹ thuật ghép lại; so với cách sắp xếp trộn ở C/14 .
●● Cộng một giá trị vào đoạn, hỏi giá trị tại một điểm	CSES “Range Update Queries”[33]	Fenwick trên mảng hiệu; đối chiếu với cách mảng hiệu thuần ở B/06 .
●● Cộng vào đoạn và hỏi tổng đoạn	Bài kinh điển	Cây phân đoạn có lazy; hoặc hai Fenwick — hãy cài cả hai và so thời gian.
●●● Đoạn con có tổng lớn nhất, có cập nhật điểm	Bài thi đấu kinh điển	Mỗi nút lưu bốn giá trị (tổng, tiền tố tốt nhất, hậu tố tốt nhất, đáp án); ví dụ đẹp nhất về việc thiết kế phép gộp.
●●● Số giá trị phân biệt trong đoạn	Bài kinh điển	Thuật toán Mo hoặc cách ngoại tuyến với Fenwick; giải thích vì sao cây phân đoạn không dùng được trực tiếp.
●●● Phần tử nhỏ thứ k trong đoạn	Bài thi đấu kinh điển	Cây phân đoạn bền vững; hiệu của hai phiên bản chính là “đếm trong đoạn”.
●●● Tổng trên hình chữ nhật có cập nhật điểm	CSES “Forest Queries II”[33]	Fenwick hai chiều; so bộ nhớ với cây phân đoạn hai chiều.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Cài phân rã căn bậc hai cho một truy vấn không kết hợp tự chọn	Bài tập thiết kế	Mục tiêu là tự nhận ra ranh giới “kết hợp được hay không”, không phải cài cho xong.

Kết chương và đọc tiếp

Chương này khép lại câu chuyện bắt đầu từ B/06: từ tổng tiền tố tĩnh, qua Fenwick và cây phân đoạn cho dữ liệu động, tới phân rã căn bậc hai cho những phép toán mà cây chịu. Quyết định luôn chỉ dựa trên hai câu hỏi — phép toán có kết hợp không, và mảng có thay đổi không — và bảng ở mục 4 là thứ đáng thuộc nhất.

Chương tiếp theo là cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả quyển sách: hợp nhất tập hợp, chỉ hai chục dòng nhưng đứng sau thuật toán cây khung nhỏ nhất, các bài liên thông ngoại tuyến, và một phân tích khấu hao đẹp tới mức hàm chi phí của nó tăng chậm hơn mọi hàm bạn từng gặp (B/11).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Bentley & Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation”	Journal of Algorithms, 1980	Nguồn của phép biến đổi ở mục 6. Đọc phần định nghĩa bài toán phân rã được và bảng các phép biến đổi; phần tối ưu tổ hợp đọc sau.
nền tảng	Fenwick, “A New Data Structure for Cumulative Frequency Tables”	Software: Practice and Experience, 1994	Bài gốc của cây chỉ số nhị phân, viết cho bài toán nén số học. Mười trang, và cách trình bày lowbit ở đây rõ hơn phần lớn tài liệu về sau.
tra khi cần	Bentley, “Programming Pearls”	Addison-Wesley, tái bản 2000	Đọc các chương về tổng tiền tố và về thiết kế cấu trúc dữ liệu nhỏ; văn phong ngắn, hợp đọc rải rác.
tra khi cần	Demaine, khoá học MIT 6.851 <i>Advanced Data Structures</i>	tài liệu khoá học công khai, MIT	Bài giảng về cấu trúc bền vững và về “động hoá” cấu trúc tĩnh, đi tiếp đúng hướng của mục này.

Hợp nhất tập hợp — DSU (Disjoint Set Union)

PHỤ THUỘC A/03 cho phân tích khâu hao; A/04 cho bất biến. Là điều kiện cần cho C/15 (Kruskal) và nhiều bài ở C/17.

TÓM TẮT

DSU trả lời đúng hai câu hỏi: “hai phần tử này có cùng nhóm không” và “gộp hai nhóm lại”. Chỉ hai thao tác, khoảng hai chục dòng code, nhưng đó là cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả quyển sách — nó đứng sau thuật toán Kruskal, các bài liên thông xử lý ngoại tuyến, kỹ thuật gộp nhỏ vào lớn, và nhiều mẹo đếm. Hai tối ưu đơn giản — gộp theo kích thước và nén đường — đưa chi phí xuống gần như hằng số, với một phân tích khâu hao thuộc loại đẹp nhất ngành. Điểm yếu duy nhất, nhưng rất quan trọng: DSU *không xoá được*. Nếu chỉ nhớ một điều: DSU là cấu trúc một chiều — nhóm chỉ có thể to ra — và mọi kỹ thuật nâng cao trong chương chỉ là cách né hạn chế đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **quan hệ tương đương** — quan hệ có tính phản xạ, đối xứng, bắc cầu; nó chia tập thành các lớp rời nhau. DSU chính là cấu trúc duy trì các lớp đó khi quan hệ được bổ sung dần.
- **đại diện (representative)** — một phần tử được chọn làm “tên” của cả nhóm; hai phần tử cùng nhóm khi và chỉ khi có cùng đại diện.
- **gộp theo kích thước / hạng** — luôn treo cây nhỏ vào gốc cây lớn, để chiều cao không vượt $O(\log n)$.
- **nén đường (path compression)** — sau mỗi lần tìm đại diện, nối thẳng mọi nút trên đường lên gốc, làm các lần tìm sau rẻ hơn.
- **hàm Ackermann ngược $\alpha(n)$** — hàm tăng cực chậm; với mọi n trong vũ trụ thực tế thì $\alpha(n) \leq 4$. Chi phí khâu hao của DSU là $O(\alpha(n))$.
- **gộp nhỏ vào lớn (small to large)** — kỹ thuật tổng quát hơn: khi gộp hai tập hợp thật sự, luôn duyệt tập nhỏ và đổ vào tập lớn; tổng chi phí $O(n \log n)$ vì mỗi phần tử chỉ bị chuyển $O(\log n)$ lần.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chỉ cần hai mẹo đơn giản — gộp theo kích thước và nén đường — đã đưa chi phí từ $O(n)$ xuống gần như $O(1)$?
- Q2. Vì sao DSU không hỗ trợ thao tác tách nhóm, và trong bài toán bắt buộc phải tách thì làm thế nào?
- Q3. Kỹ thuật “gộp nhỏ vào lớn” cho $O(n \log n)$ bằng một lập luận rất ngắn. Lập luận đó là gì?
- Q4. DSU chỉ biết “cùng nhóm hay không”. Làm sao nó trả lời được câu hỏi tinh vi hơn như “hai phần tử này khác phe nhau”?
- Q5. Trong bài toán mà các cạnh bị xoá dần theo thời gian, vì sao xử lý ngược thời gian lại cứu được tình hình?

Có một dạng bài toán rất phổ biến: các phần tử dần dần được nối lại với nhau, và ta cần biết ai đang cùng nhóm với ai. Máy tính trong mạng được nối thêm dây; các pixel cùng màu được gộp

thành vùng; các đỉnh của đồ thị được nối thêm cạnh. Nếu mỗi lần hỏi ta lại chạy một lượt duyệt đồ thị thì chi phí là $O(n)$ cho mỗi câu hỏi — quá đắt khi có hàng trăm nghìn câu.

DSU là câu trả lời, và điều làm nó đặc biệt không phải sự tinh vi mà là sự *tiết chế*. Nó không lưu danh sách thành viên của mỗi nhóm, không lưu cấu trúc của các liên kết, không trả lời được “nhóm này gồm những ai”. Nó chỉ lưu đúng một con trỏ cha cho mỗi phần tử, và chỉ trả lời đúng câu “ai là đại diện của nhóm chứa phần tử này”. Chính vì hứa ít nên nó rẻ — lại một lần nữa nguyên tắc xuyên suốt phần B.

11.1 Từ ngây thơ tới gần như hằng số

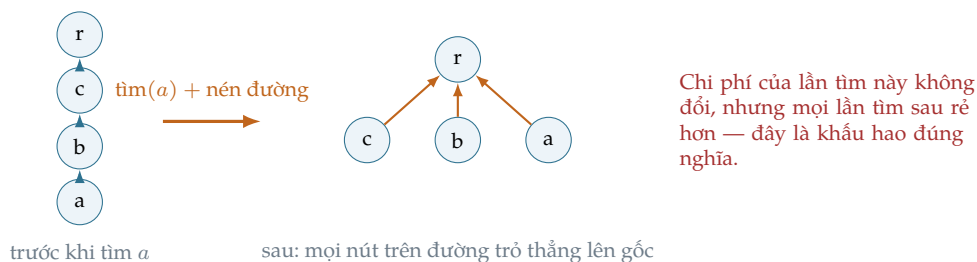
Biểu diễn: mỗi phần tử có một con trỏ cha; đi ngược theo cha tới khi gặp nút trỏ vào chính nó, đó là đại diện. Gộp hai nhóm: cho gốc này trỏ vào gốc kia.

Cách ngây thơ này có thể tạo ra một chuỗi dài n , và mỗi lần tìm tốn $O(n)$. Hai tối ưu chứa được điều đó, và điều đáng chú ý là chúng độc lập nhau nhưng bổ sung nhau.

Gộp theo kích thước. Luôn treo cây có ít phần tử hơn vào gốc của cây lớn hơn. Bất biến sinh ra: một cây có chiều cao h phải chứa ít nhất 2^h phần tử — chứng minh bằng quy nạp, vì chiều cao chỉ tăng khi hai cây cùng chiều cao gộp lại, và khi đó kích thước ít nhất gấp đôi. Suy ra chiều cao tối đa $\log_2 n$, và mọi thao tác là $O(\log n)$.

Nén đường. Trong lúc đi tìm đại diện, ta đã đi qua cả một đường; hãy nối thẳng mọi nút trên đường đó lên gốc. Việc này không tốn thêm chi phí tiệm cận nào ở lần tìm hiện tại, nhưng làm mọi lần tìm sau rẻ đi. Đây là một ví dụ mẫu mực cho ý tưởng khấu hao ở chương A/03: trả thêm một chút bây giờ, hưởng lợi nhiều lần về sau.

Dùng cả hai, chi phí khấu hao mỗi thao tác là $O(\alpha(n))$ với α là hàm Ackermann ngược — một hàm tăng chậm tới mức với n nhỏ hơn số nguyên tử trong vũ trụ quan sát được thì $\alpha(n) \leq 4$. Kết quả này do Tarjan chứng minh năm 1975[49], và nó nổi tiếng vì kết luận rất bất ngờ: một cấu trúc chỉ dùng một mảng số nguyên lại có chi phí không phải hằng số nhưng gần tới mức không phân biệt được. Về sau người ta còn chứng minh được rằng $O(\alpha(n))$ là *tối ưu* — không cấu trúc nào làm tốt hơn cho bài toán này.



Hình 11.1. Nén đường. Điều đáng nhớ là nó không làm lần tìm hiện tại nhanh hơn — ta vẫn phải đi hết đường — mà làm cấu trúc phẳng đi cho tương lai. Kết hợp với gộp theo kích thước, chi phí khấu hao trở thành $O(\alpha(n))$, thực tế là hằng số.

11.2 Hạn chế cốt lõi: không xoá được

DSU là cấu trúc *một chiều*: nhóm chỉ có thể to ra, không bao giờ nhỏ đi. Lý do rất trực tiếp: nén đường đã phá hủy thông tin về cấu trúc gốc, nên không có cách nào biết phải tách ở đâu.

Đây không phải khiếm khuyết cài đặt mà là đánh đổi cố ý — chính việc vứt bỏ thông tin làm cấu trúc rẻ như vậy. Nhưng rất nhiều bài toán thật lại có cả xoá: cạnh mạng bị đứt, kết nối bị huỷ. Có ba cách né, và cả ba đều đáng biết vì chúng là mẫu tư duy dùng lại được.

Cách thứ nhất — đảo ngược thời gian. Nếu bài toán chỉ có xoá chứ không có thêm, hãy xử lý các truy vấn theo thứ tự ngược: xoá theo chiều xuôi trở thành thêm theo chiều ngược. Đây là một mẹo đơn giản tới mức dễ bị bỏ qua, nhưng nó giải quyết trọn vẹn cả một họ bài. Điều kiện: truy vấn phải biết trước, tức là bài toán ngoại tuyến.

Cách thứ hai — DSU có hoàn tác. Bỏ nén đường, chỉ giữ gộp theo kích thước (chi phí thành $O(\log n)$), và ghi lại mọi thay đổi vào một ngăn xếp để có thể hoàn tác. Kết hợp với chia để trị trên trục thời gian, ta giải được bài liên thông động ngoại tuyến tổng quát: mỗi cạnh tồn tại trong một khoảng thời gian, và ta duyệt cây phân đoạn trên thời gian, thêm cạnh khi đi xuống và hoàn tác khi đi lên.

Cách thứ ba — cấu trúc khác hẳn. Nếu bài buộc phải trực tuyến và có cả thêm lẫn xoá, DSU không đủ và cần cấu trúc như link-cut tree. Xếp vào loại biết là có.

CẠM BÃY

Sự kết hợp giữa nén đường và hoàn tác là không tương thích — đây là lỗi thường gặp khi người ta cố ghép hai kỹ thuật. Nén đường thay đổi nhiều con trỏ trong một lần tìm, và những thay đổi đó không được ghi lại đúng cách nếu bạn chỉ ghi các thao tác gộp. Muốn hoàn tác thì phải bỏ nén đường và chấp nhận $O(\log n)$.

11.3 Tăng cường DSU: đếm, phe phái, khoảng cách

Giống như cây tìm kiếm ở chương B/09, DSU mạnh lên rất nhiều khi được gắn thêm thông tin — và điều kiện cũng tương tự: thông tin phải *gộp được* khi hai nhóm hợp nhất.

Đếm. Lưu kích thước mỗi nhóm ở gốc: gộp thì cộng lại. Cho phép trả lời “nhóm chứa x có bao nhiêu phần tử” và “hiện có bao nhiêu nhóm”. Đây là dạng dùng nhiều nhất.

Phe phái, hay DSU có trọng số. Lưu thêm khoảng cách tương đối từ mỗi nút tới cha nó theo một nhóm cộng — thường là tính chẵn lẻ. Khi đó DSU trả lời được “ x và y cùng phe hay khác phe”, và phát hiện được mâu thuẫn khi có ràng buộc trái ngược. Ứng dụng: kiểm tra đồ thị hai phía trực tuyến, và các bài ràng buộc kiểu “ x và y khác loại”. Chú ý khi nén đường phải cộng dồn trọng số dọc đường — đây là chỗ dễ sai nhất.

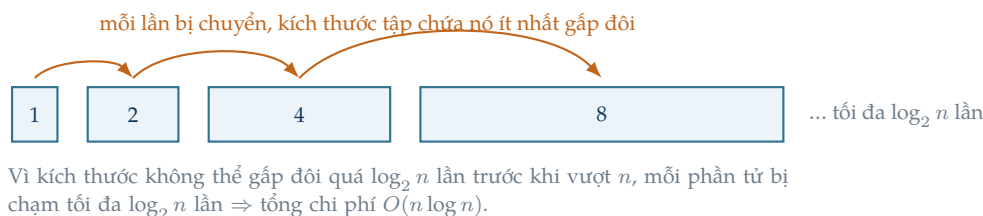
DSU trên miền giá trị — mẹo “vị trí trống kế tiếp”. Một cách dùng rất khéo: cho DSU trỏ mỗi vị trí đã bị chiếm sang vị trí trống kế tiếp bên phải. Khi cần “tìm ô trống đầu tiên từ i trở đi”, chỉ việc tìm đại diện; khi chiếm ô đó thì gộp nó với ô kế tiếp. Kỹ thuật này biến nhiều bài “tô màu đoạn, mỗi ô chỉ tô một lần” từ $O(n^2)$ thành gần tuyến tính — và nó là ví dụ điển hình cho việc dùng DSU ở nơi mà bài toán không hề nói tới “nhóm” nào cả.

11.4 Gộp nhỏ vào lớn: người anh em tổng quát hơn

DSU không lưu danh sách thành viên. Khi bài toán *cần* danh sách — ví dụ phải duyệt mọi phần tử của nhóm khi gộp — thì ta dùng kỹ thuật gộp nhỏ vào lớn: giữ tập hợp thật cho mỗi nhóm, và khi gộp thì luôn duyệt tập nhỏ hơn rồi đổ vào tập lớn hơn.

Chi phí tổng cộng là $O(n \log n)$, và lập luận cực ngắn: mỗi lần một phần tử bị chuyển, nó chuyển từ tập nhỏ sang tập lớn, nên kích thước tập chứa nó ít nhất gấp đôi. Kích thước không thể gấp đôi quá $\log_2 n$ lần, nên mỗi phần tử bị chuyển tối đa $\log_2 n$ lần. Đây là một trong những lập luận khẩu hao đẹp và dễ nhớ nhất trong quyển sách, và nó dùng lại ở rất nhiều nơi.

Biến thể quan trọng là *DSU trên cây*: khi giải các bài truy vấn về cây con — “trong cây con của u có bao nhiêu màu phân biệt” — ta duyệt cây, giữ lại cấu trúc đếm của con nặng nhất và gộp các con nhẹ vào. Chi phí $O(n \log n)$ nhờ đúng lập luận trên.



Hình 11.2. Lập luận “gộp nhỏ vào lớn”. Điều đáng học không phải kết quả mà là cách đếm: ta không tính chi phí theo từng phép gộp — có phép gộp rất đắt — mà theo *tổng số lần các phần tử bị chạm*. Cùng kiểu lập luận với khẩu hao ở A/03.

11.5 “Tối ưu” nghĩa là gì ở đây: hai cận dưới cho hợp nhất tập hợp

Mục 1 kết thúc bằng một câu đáng ngờ nếu đọc kỹ: $O(\alpha(n))$ là *tối ưu*. Tối ưu so với cái gì? Không ai chứng minh được cận dưới cho “mọi thuật toán khả dĩ” — chương D/24 giải thích vì sao điều đó gần như bất khả thi. Cái người ta chứng minh được là hai phát biểu hẹp hơn nhưng mạnh một cách bất ngờ, và chúng đáng biết vì chúng cho thấy hàm α không phải vết tích của một phép phân tích vụng về, mà là một tính chất của chính bài toán.

Kết quả thứ nhất là của Tarjan[50]. Ông xét một lớp thuật toán được mô tả rõ: những thuật toán chỉ thao tác với các phần tử qua con trỏ, đúng như cách DSU đi từ nút lên cha, không được phép làm số học trên nhãn để đi tắt. Trong lớp đó, mọi thuật toán trả lời trực tuyến m thao tác trên n phần tử đều cần $\Omega(m \alpha(m, n))$ trong trường hợp xấu nhất. Vậy trong thế giới con trỏ, nén đường cộng gộp theo kích thước không chỉ tốt: nó chạm cận.

Kết quả thứ hai mạnh hơn nhiều vì nó bỏ luôn giả thiết con trỏ. Fredman và Saks[51] làm việc trong mô hình *cell-probe*, ở đó ta chỉ đếm số ô nhớ mà thuật toán chạm vào và cho phép nó tính bất cứ thứ gì nó muốn trên các ô ấy — kể cả các phép bit và song song mức từ máy. Ngay trong mô hình rộng rãi đó, chi phí khẩu hao mỗi thao tác vẫn là $\Omega(\alpha)$. Nói cách khác, không có mẹo cài đặt nào đưa bài toán này xuống hằng số thật sự.

Ba điều đáng mang theo. Thứ nhất, đây là một trong số ít trường hợp mà cận trên và cận dưới khớp nhau ở một hàm kỳ dị như α ; sự khớp đó là bằng chứng rằng hàm ấy thuộc về bài toán chứ không thuộc về cách ta phân tích. Thứ hai, và tổng quát hơn: mọi phát biểu “tối ưu” đều kèm một mô hình, và khi đọc một kết quả như vậy, câu hỏi đầu tiên phải là *tối ưu trong lớp thuật toán nào* (D/24). Thứ ba, về thực hành thì kết luận vẫn giản dị: đừng đi tìm DSU hằng số, và cũng đừng bỏ bớt một trong hai tối ưu — dùng riêng gộp theo kích thước hay riêng nén đường đều chỉ cho $O(\log n)$, phải có cả hai mới xuống tới α .

11.6 Gốc rễ: từ câu lệnh EQUIVALENCE của Fortran

Bài toán mà DSU giải ra đời cùng với những trình biên dịch đầu tiên. Ngôn ngữ Fortran có câu lệnh EQUIVALENCE cho phép lập trình viên khai báo rằng hai biến dùng chung một vùng nhớ. Trình biên dịch phải xử lý một loạt khai báo như vậy và tính ra các *lớp tương đương* — nhóm các biến thực sự chia sẻ bộ nhớ. Đó chính xác là bài toán “hợp nhất và hỏi cùng nhóm”.

Năm 1964, Bernard Galler và Michael Fischer công bố thuật toán cho bài toán đó, với cấu trúc rừng cây và ý tưởng đại diện mà ta dùng tới ngày nay. Điều đáng chú ý về mặt phương pháp: bài toán được phát hiện từ một nhu cầu công cụ rất cụ thể, giống hệt câu chuyện bảng băm ở chương B/08.

Phần lý thuyết mất thêm mười năm. Nén đường và gộp theo hạng được dùng khá sớm như những mẹo hiển nhiên, nhưng không ai chứng minh được chúng nhanh tới mức nào. Năm 1975 Tarjan cho lời giải: chi phí khấu hao là $O(\alpha(n))$ [49] — một kết quả gây bất ngờ vì hàm Ackermann ngược trước đó thuộc về logic và lý thuyết đệ quy, không ai chờ nó xuất hiện trong phân tích một cấu trúc dữ liệu thực dụng. Về sau người ta chứng minh thêm rằng cận này không cải thiện được, tức DSU với hai tối ưu là tối ưu về bản chất, không chỉ tối ưu trong các cách cài đã biết.

Còn một liên hệ ít được nhắc nhưng rất sâu: cùng cấu trúc này là hạt nhân của thuật toán *hợp nhất* trong suy diễn kiểu của trình biên dịch. Khi trình biên dịch suy ra kiểu của một biểu thức, nó liên tục phát biểu “kiểu này phải bằng kiểu kia” và phải duy trì các lớp tương đương của các biến kiểu — đúng bài toán của Galler và Fischer, chỉ khác ngữ cảnh. Nghĩa là DSU vẫn đang chạy trong trình biên dịch mỗi lần bạn dịch một chương trình có suy diễn kiểu.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1964 — Galler & Fischer	Trình biên dịch Fortran phải xử lý câu lệnh EQUIVALENCE: các biến dùng chung vùng nhớ.	Cấu trúc rừng với đại diện; bài toán lớp tương đương được hình thức hoá.
Cuối thập niên 1960 — các cài đặt thực tế	Chuỗi cha dài làm thao tác tìm chậm.	Gộp theo hạng và nén đường xuất hiện như mẹo hiển nhiên, nhưng chưa ai biết chúng nhanh tới đâu.
1975 — Tarjan	Cần biết chính xác chi phí của hai mẹo đó.	Chi phí khấu hao $O(\alpha(n))$; hàm Ackermann ngược lần đầu xuất hiện trong phân tích một cấu trúc thực dụng.
Sau đó — các kết quả cận dưới	Có làm tốt hơn α được không?	Không: $O(\alpha(n))$ là tối ưu cho bài toán này trong mô hình con trỏ; một trong số ít cấu trúc dữ liệu đạt tối ưu chứng minh được.
Thập niên 1970–80 — suy diễn kiểu	Trình biên dịch phải duy trì các ràng buộc “kiểu này bằng kiểu kia”.	Thuật toán hợp nhất dùng đúng cấu trúc DSU; nó chạy mỗi lần bạn biên dịch một chương trình có suy diễn kiểu.

I 11.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Trình biên dịch	Hợp nhất trong suy diễn kiểu	Cùng cấu trúc, cùng thao tác: gộp hai lớp và hỏi cùng lớp. Điểm khác đáng chú ý: ở đó việc phát hiện mâu thuẫn (hai kiểu không hợp nhất được) chính là thông báo lỗi kiểu mà bạn nhìn thấy.
Thị giác máy tính	Gán nhãn thành phần liên thông trên ảnh nhị phân	Thuật toán hai lượt kinh điển dùng DSU để gộp các nhãn tạm thời. Ứng dụng gần: gộp các vùng ảnh hoặc các điểm đặc trưng thuộc cùng vật thể.
Vật lý thống kê	Mô phỏng thấm (percolation): các ô mở dần, hỏi khi nào hai bờ nối được với nhau	Đây là ứng dụng lịch sử nổi tiếng và trùng khít về hình thức: các ô mở dần chính là các phép gộp, câu hỏi “đã thấm chưa” chính là truy vấn cùng nhóm.
Mạng và hạ tầng	Kiểm tra liên thông mạng, thiết kế mạng chi phí nhỏ nhất	Kruskal (C/15) chạy trực tiếp trên DSU. Điểm khác trong thực tế: mạng thật có cả sự cố đứt kết nối, tức là có xoá — và đó chính là chỗ DSU thuận tuy không đủ.
Cơ sở dữ liệu, làm sạch dữ liệu	Gộp bản ghi trùng lặp thành một thực thể	“Hai bản ghi này là cùng một người” là một quan hệ tương đương được bổ sung dần — đúng mô hình DSU. Vấn đề thực tế: quyết định sai thì không rút lại được, đúng như hạn chế một chiều ở mục 2.
Toán học	Quan hệ tương đương và tập thương	DSU là hiện thực hoá tính toán của khái niệm tập thương: duy trì các lớp khi quan hệ sinh được bổ sung dần. Nhìn theo góc này thì “không tách được” là hệ quả tự nhiên: bỏ bớt một quan hệ sinh có thể làm cả cấu trúc lớp thay đổi.
SLAM	Gộp các quan sát về cùng một điểm mốc	Khi hai đường quan sát được xác định là cùng một điểm mốc thì hai tập quan sát phải gộp lại — đúng thao tác của DSU. Cùng cạm bẫy: nếu sau đó phát hiện gộp nhầm thì không tách ra được, nên trong thực tế người ta trì hoãn việc gộp cho tới khi đủ bằng chứng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai mẹo tấn công hai nguyên nhân khác nhau của chuỗi dài. Gộp theo kích thước ngăn không cho cây cao lên ngay từ đầu: một cây cao h buộc phải chứa ít nhất 2^h nút, nên chiều cao bị chặn bởi $\log_2 n$. Nén đường thì làm phẳng những đường đã đi qua, nên chi phí đã trả một lần không phải trả lại. Kết hợp lại, mọi cấu hình xấu đều tự triệt tiêu sau vài thao tác, và phân tích chặt cho ra $O(\alpha(n))$ khâu hao.
- A2. Vì nén đường đã phá huỷ thông tin về cấu trúc liên kết gốc: sau khi mọi nút trở thẳng lên gốc, không còn dấu vết của việc ai đã nối với ai. Đây là đánh đổi cố ý, và ba cách né là: xử lý ngược thời gian nếu bài chỉ có xoá; DSU có hoàn tác (bỏ nén đường, ghi nhật ký thao tác) kết hợp chia để trị trên trục thời gian nếu bài ngoại tuyến; hoặc dùng cấu trúc mạnh hơn như link-cut tree nếu buộc phải trực tuyến.
- A3. Mỗi khi một phần tử bị chuyển, nó đi từ tập nhỏ sang tập lớn, nên kích thước tập chứa nó ít nhất tăng gấp đôi. Kích thước không thể gấp đôi quá $\log_2 n$ lần trước khi vượt n , nên mỗi phần tử bị chuyển tối đa $\log_2 n$ lần, tổng $O(n \log n)$. Điều đáng chú ý về mặt phương pháp: ta không đếm chi phí theo từng thao tác gộp — có phép gộp rất đắt — mà đếm theo *tổng số lần các phần tử bị chạm*, đúng tinh thần phân tích khâu hao ở A/03.
- A4. Bằng cách lưu thêm một “khoảng cách” tương đối từ mỗi nút tới cha của nó, thường là bit chẵn lẻ. Khi đó “cùng phe” hay “khác phe” được suy ra từ tổng chẵn lẻ dọc đường lên gốc, và một ràng buộc mới mâu thuẫn với các ràng buộc cũ sẽ bị phát hiện ngay khi hai đầu đã cùng nhóm mà chẵn lẻ không khớp. Chỗ dễ sai nhất là phải cộng dồn trọng số trong lúc nén đường; quên bước đó thì cấu trúc vẫn chạy nhưng trả lời sai.
- A5. Vì DSU chỉ làm được một chiều — gộp — nên bài toán “xoá dần” là bài toán ngược của thứ nó làm được. Chạy các truy vấn theo thứ tự ngược thời gian biến mọi phép xoá thành phép thêm,

và lúc đó DSU dùng được nguyên vẹn. Cái giá là ta phải biết toàn bộ truy vấn từ trước, tức là chấp nhận lời giải ngoại tuyến — đây là cùng loại đánh đổi với thuật toán Mo ở B/10.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một lớp học mà học sinh dần dần kết bạn. Bạn cần trả lời hai câu hỏi: “hai bạn này có chung một nhóm bạn không?” và “từ giờ hai nhóm này nhập làm một”.

Cách làm rất đơn giản: mỗi nhóm cử một bạn làm nhóm trưởng, và mỗi người chỉ cần nhớ mình theo ai. Muốn biết mình thuộc nhóm nào thì cứ hỏi ngược lên tới khi gặp người tự làm trưởng. Hai người cùng nhóm khi và chỉ khi họ ra cùng một trưởng.

Có hai mẹo làm việc này nhanh hẳn lên. Mẹo thứ nhất: khi nhập hai nhóm, luôn để nhóm đồng giữ trưởng và nhóm ít theo về — như vậy chuỗi hỏi ngược không bao giờ dài. Mẹo thứ hai: mỗi lần bạn phải hỏi ngược qua một chuỗi dài, hãy dặn tất cả những người trên đường “từ giờ cứ theo trưởng trưởng luôn” — lần sau ai hỏi cũng chỉ mất một bước. Hai mẹo tầm thường ấy làm chi phí gần như bằng hằng số, và điều đó phải mất mười năm mới được chứng minh chặt chẽ.

Nhưng có một điều DSU không làm được, và cần nhớ kỹ: *không tách nhóm ra được*. Sau khi mọi người đã theo trưởng trưởng, không ai còn nhớ ai đã kết bạn với ai lúc đầu, nên không có cách nào biết phải cắt ở đâu. Nếu bài toán của bạn có chuyện “huỷ kết bạn”, thì mẹo thường dùng là chạy ngược thời gian: xem cuốn phim từ cuối về đầu, lúc đó mọi việc huỷ trở thành việc kết.

Một câu: DSU chỉ hứa hai điều — cùng nhóm hay không, và gộp hai nhóm — và chính vì hứa ít nên nó gần như miễn phí.

Chỗ không rút gọn được: Chi phí $O(\alpha(n))$ thực sự không phải hằng số về mặt toán học; nó chỉ là hằng số về mặt thực tiễn vì hàm đó tăng quá chậm.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Hai thao tác của DSU	“Tìm đại diện” và “gộp hai nhóm”. Không lưu danh sách thành viên, không trả lời được “nhóm gồm những ai”.
••• Hai tối ưu và tác dụng	Gộp theo kích thước (cây cao h phải có $\geq 2^h$ nút $\Rightarrow$ chiều cao $\leq \log_2 n$); nén đường (làm phẳng đường vừa đi cho tương lai).
••• Chi phí khấu hao	$O(\alpha(n))$ với α là Ackermann ngược, thực tế ≤ 4 ; và cận này đã được chứng minh là tối ưu.
••• Hạn chế cốt lõi	Không xoá, không tách — nén đường đã phá huỷ cấu trúc liên kết gốc. Đây là đánh đổi cố ý, không phải thiếu sót.
••• Ba cách né hạn chế đó	Xử lý ngược thời gian (bài chỉ có xoá); DSU có hoàn tác + chia để trị trên trục thời gian (ngoại tuyến); link-cut tree (trực tuyến).
••• Gộp nhỏ vào lớn	Luôn đổ tập nhỏ vào tập lớn; mỗi phần tử bị chuyển $\leq \log_2 n$ lần vì mỗi lần chuyển thì kích thước tập chứa nó ít nhất gấp đôi. Tổng $O(n \log n)$.
•• DSU có trọng số / chặn lẻ	Lưu quan hệ tương đối tới cha; trả lời “cùng phe hay khác phe”, phát hiện mâu thuẫn. Phải cộng dồn trọng số khi nén đường.
•• Mẹo “vị trí trống kế tiếp”	Cho mỗi ô đã dùng trở sang ô trống kế tiếp; biến bài “tô đoạn, mỗi ô tô một lần” từ $O(n^2)$ thành gần tuyến tính.
•• Nén đường và hoàn tác	Không tương thích: muốn hoàn tác thì phải bỏ nén đường và chấp nhận $O(\log n)$.
•• DSU trên cây	Giữ cấu trúc đếm của con nặng nhất, gộp các con nhẹ vào; $O(n \log n)$ theo đúng lập luận gộp nhỏ vào lớn.
•• Kiểm tra đồ thị hai phía trực tuyến	DSU chặn lẻ: mâu thuẫn xuất hiện khi hai đầu một cạnh đã cùng nhóm mà cùng chặn lẻ.
• Gốc rễ	Ra đời 1964 cho câu lệnh EQUIVALENCE của Fortran; ngày nay vẫn chạy trong suy diễn kiểu của trình biên dịch.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● “ α là tối ưu” nghĩa là gì	Tối ưu trong mô hình con trỏ (Tarjan, 1979) và vẫn $\Omega(\alpha)$ khấu hao ngay trong mô hình cell-probe (Fredman–Saks, 1989). Mọi phát biểu tối ưu đều kèm một mô hình.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Chỉ dùng một trong hai tối ưu thì chi phí là bao nhiêu?

| Chỉ gộp theo kích thước cho $O(\log n)$ trường hợp xấu nhất — đây là đảm bảo thật, không phải khấu hao, nên nó là lựa chọn khi cần hoàn tác. Chỉ nén đường cho $O(\log n)$ khấu hao. Dùng cả hai cho $O(\alpha(n))$ khấu hao. Trong thực hành, nếu không cần hoàn tác thì luôn dùng cả hai vì chúng chỉ tốn vài ký tự code.

2. Tôi có nên cài DSU bằng đệ quy không?

| Bản đệ quy cho hàm tìm đại diện rất ngắn và tự nhiên, nhưng có rủi ro tràn ngăn xếp nếu bạn chưa nén đường và cây sâu — tình huống này xảy ra trong các bài xây dựng đối nghịch. Bản lặp hai lượt (đi lên tìm gốc, đi lại lần nữa để nén) an toàn hơn và cũng nhanh hơn một chút. Với n lớn, nên dùng bản lặp; đây là ứng dụng cụ thể của lời khuyên ở E/25.

3. Làm sao dùng DSU để đếm số thành phần liên thông sau mỗi lần thêm cạnh?

| Khởi tạo số nhóm bằng n , và mỗi lần gộp thành công — tức hai đầu cạnh chưa cùng nhóm — thì giảm đi một. Cạnh nối hai đỉnh đã cùng nhóm không đổi gì, và đó cũng chính là dấu hiệu phát hiện chu trình, tức phần lõi của Kruskal ở C/15.

4. Khi nào tôi phải nén toạ độ trước khi dùng DSU?

| Khi các phần tử không phải số nguyên trong khoảng nhỏ. DSU cấp phát mảng theo số phần tử, nên với các khoá là số lớn, chuỗi hoặc cặp giá trị, bạn cần ánh xạ chúng về $0..n-1$ trước bằng cách sắp xếp và loại trùng, hoặc bằng bảng băm. Đây là bước tiền xử lý giống hệt như với Fenwick ở B/10.

5. Bài toán “liên thông động” đầy đủ — vừa thêm vừa xoá, trực tuyến — giải thế nào?

| Đây là bài toán khó thật sự, không phải bài tập thông thường. Với cây (không có chu trình), link-cut tree cho $O(\log n)$ khấu hao. Với đồ thị tổng quát, các cấu trúc liên thông động có chi phí *poly-logarit* nhưng cài đặt rất nặng. Trong thực hành thi đấu, nếu bài cho phép ngoại tuyến thì hãy dùng chia để trị trên trục thời gian với DSU có hoàn tác — ngắn hơn nhiều lần và đủ nhanh.

6. Gộp nhỏ vào lớn khác gì DSU thông thường?

| Khác ở chỗ nó giữ *dữ liệu thật* của mỗi nhóm — một tập hợp, một bảng đếm, một danh sách — chứ không chỉ giữ đại diện. Vì vậy nó tốn $O(n \log n)$ thay vì gần như tuyến tính, nhưng đổi lại trả lời được những câu mà DSU chịu, ví dụ “trong nhóm này có bao nhiêu giá trị phân biệt”. Quy tắc chọn: nếu chỉ cần biết cùng nhóm hay không thì DSU; nếu cần thống kê bên trong nhóm thì gộp nhỏ vào lớn.

7. Có bài nào trông không liên quan gì tới “nhóm” mà vẫn dùng DSU không?

| Có, và đó là dấu hiệu bạn đã thật sự nắm cấu trúc này. Mẹo “vị trí trống kế tiếp” ở mục 3 là ví dụ rõ nhất: bài toán nói về việc tô màu các ô, không nói gì về nhóm, nhưng quan hệ “ô này đã đầy, hãy hỏi ô bên phải” lại có đúng cấu trúc hợp nhất một chiều. Một ví dụ khác là các bài ràng buộc bằng nhau kiểu “ $x_i = x_j$ ”, nơi DSU dùng để phát hiện mâu thuẫn với các ràng buộc khác nhau.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Đếm số thành phần liên thông khi thêm cạnh dần	CSES “Road Construction”[33]	DSU cơ bản kèm đếm kích thước nhóm lớn nhất.
• Kiểm tra đồ thị có chu trình khi thêm cạnh	Bài kinh điển	Cạnh nối hai đỉnh đã cùng nhóm chính là cạnh tạo chu trình.
•• Cây khung nhỏ nhất bằng Kruskal	CSES “Road Reparation”[33]	Sắp cạnh rồi DSU; chứng minh tính đúng ở C/15 .
•• Kiểm tra đồ thị hai phía khi thêm cạnh dần	Bài kinh điển	DSU chặn lẻ; chú ý cộng dồn trọng số lúc nén đường.
•• Tô màu các đoạn, mỗi ô chỉ tô một lần	Bài thi đấu kinh điển	Mẹo “vị trí trống kế tiếp”; so với cây phân đoạn dễ thấy DSU ngắn hơn nhiều.
•• Số đảo trên lưới sau mỗi lần thêm đất	LeetCode “Number of Islands II”	DSU trên lưới; ánh xạ tọa độ hai chiều về chỉ số một chiều.
••• Liên thông khi các cạnh bị xoá dần	Bài kinh điển	Đảo ngược thời gian; đây là bài mẫu để hiểu mẹo đó.
••• Liên thông động ngoại tuyến với cạnh có khoảng thời gian tồn tại	Bài thi đấu kinh điển	DSU có hoàn tác + cây phân đoạn trên trục thời gian; kết hợp B/10 với chương này.
••• Với mỗi cây con, đếm số màu phân biệt	Bài kinh điển	DSU trên cây (gộp nhỏ vào lớn); tự chứng minh cận $O(n \log n)$ trước khi cài.
••• Ràng buộc “ x_i và x_j bằng nhau / khác nhau”, kiểm tra mâu thuẫn	Bài kinh điển	DSU chặn lẻ tổng quát; liên hệ với hợp nhất trong suy diễn kiểu ở mục 5.
••• Mô phỏng thăm trên lưới $n \times n$, tìm ngưỡng thăm	Bài tập thực nghiệm	Ứng dụng vật lý cổ điển; thêm hai nút ảo cho hai bờ là mẹo cài đặt cần thiết.

Kết chương và đọc tiếp

DSU là ví dụ rõ nhất trong quyển sách này cho luận điểm “hứa ít thì rẻ”: hai thao tác, một mảng số nguyên, và chi phí gần như hằng số với một chứng minh mà phải mất mười năm mới có. Hai điều đáng mang theo: lập luận gộp nhỏ vào lớn — mỗi phần tử chỉ bị chuyển $\log n$ lần — vì nó dùng lại ở nhiều nơi ngoài DSU; và ý thức về hạn chế một chiều, cùng ba cách né nó.

Chương cuối của phần B chuyển sang một miền dữ liệu riêng, nơi các cấu trúc chuyên biệt tạo ra khác biệt lớn nhất: chuỗi ký tự. Bạn sẽ thấy cùng một ý tưởng “đừng tính lại thứ đã biết” của chương [B/06](#) xuất hiện lại dưới dạng hàm tiền tố của KMP, và sẽ thấy vì sao một cấu trúc như suffix array biến cả một họ bài chuỗi thành bài mảng thông thường ([B/12](#)).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm”	Journal of the ACM, 1975	Bài chứng minh cận $O(\alpha)$. Đọc phần phát biểu kết quả và định nghĩa α ; phần chứng minh dài và có thể để sau.
nền tảng	Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets”	Journal of Computer and System Sciences, 1979	Cận dưới trong mô hình con trỏ — tài liệu cho thấy “tối ưu” ở đây nghĩa là gì.
tra khi cần	Fredman & Saks, “The Cell Probe Complexity of Dynamic Data Structures”	STOC, 1989	Cận dưới $\Omega(\alpha)$ trong mô hình cell-probe. Đọc phần mở đầu; đây cũng là cửa vào cho các cận dưới cấu trúc dữ liệu nói chung.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Cormen, Leiserson, Rivest, Stein, <i>Introduction to Algorithms</i> (4th ed.), chương về tập rời nhau	MIT Press, 2022	Tra khi cần một chứng minh đầy đủ, viết theo chuẩn giáo trình, cho cận α .

Chuỗi và so khớp (Strings and Pattern Matching)

PHỤ THUỘC B/06 cho ý tưởng không tính lại; B/08 cho băm và nghịch lý ngày sinh; B/09 cho cây. Đây là chương cuối của phần B.

TÓM TẮT

Chuỗi là mảng, nhưng có một tính chất riêng làm nó đáng có chương riêng: các đoạn con của nó *chồng lấn* nhau, và sự chồng lấn ấy vừa là nguồn của mọi khó khăn vừa là nguồn của mọi lời giải nhanh. Chương đi từ bài toán so khớp một mẫu — KMP và hàm Z, cả hai đều dựa trên ý “thất bại ở đây cho ta thông tin, đừng vứt đi” — tới nhiều mẫu cùng lúc với Aho–Corasick, tới các cấu trúc nhìn được toàn bộ chuỗi con là trie và mảng hậu tố. Băm chuỗi được đặt cạnh chúng như một lựa chọn ngắn và mạnh với cái giá xác suất. Nếu chỉ nhớ một điều: mọi thuật toán chuỗi nhanh đều trả lời cùng một câu hỏi — *khi so khớp thất bại tại vị trí i , tôi đã biết được gì mà không phải quay lại từ đầu?*

KIẾN THỨC CẦN CÓ TRƯỚC

- **tiền tố, hậu tố** — đoạn đầu và đoạn cuối của một chuỗi. Chuỗi abc có tiền tố a, ab, abc.
- **biên (border)** — một chuỗi vừa là tiền tố vừa là hậu tố thực sự của chuỗi đã cho; ví dụ aba có biên a. Khái niệm trung tâm của KMP.
- **hàm tiền tố π** — $\pi[i]$ là độ dài biên dài nhất của tiền tố kết thúc tại i . Đây là toàn bộ nội dung “tiền xử lý” của KMP.
- **hàm Z** — $z[i]$ là độ dài tiền tố chung dài nhất giữa chuỗi và hậu tố bắt đầu tại i . Cùng sức mạnh với π , thường dễ nghĩ hơn.
- **trie** — cây trong đó mỗi cạnh mang một ký tự, nên mỗi đường từ gốc biểu diễn một tiền tố; dùng để lưu một tập chuỗi và tra theo tiền tố.
- **mảng hậu tố (suffix array)** — mảng các vị trí hậu tố, sắp theo thứ tự từ điển của các hậu tố; cùng với mảng LCP, nó biến nhiều bài chuỗi thành bài mảng thông thường.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán so khớp ngây thơ chạy $O(nm)$. KMP chỉ dùng thêm một mảng số nguyên mà xuống $O(n + m)$. Thông tin nào đã bị lãng phí trong cách ngây thơ?
- Q2. Nếu băm chuỗi làm được gần hết những gì KMP làm và code ngắn hơn nhiều, vì sao vẫn phải học KMP?
- Q3. Vì sao mảng hậu tố lại được coi là “giải quyết một họ bài” chứ không chỉ là một cấu trúc?
- Q4. Aho–Corasick tìm đồng thời hàng nghìn mẫu trong một lượt quét. Nó khác gì so với chạy KMP nhiều lần, và cái gì làm cho khác biệt đó có thể xảy ra?
- Q5. Bài toán tìm chuỗi con đối xứng dài nhất có lời giải $O(n)$. Ý tưởng nào cho phép bỏ qua phần lớn công việc?

Chuỗi ký tự là mảng, và mọi kỹ thuật ở chương B/06 đều dùng được. Nhưng có một điều làm chuỗi khác hẳn: một chuỗi độ dài n có $n(n+1)/2$ đoạn con, và các đoạn ấy chồng lên nhau chằng chịt. Điều này vừa là tin xấu — không thể liệt kê hết — vừa là tin tốt, vì sự chồng lấn nghĩa là có

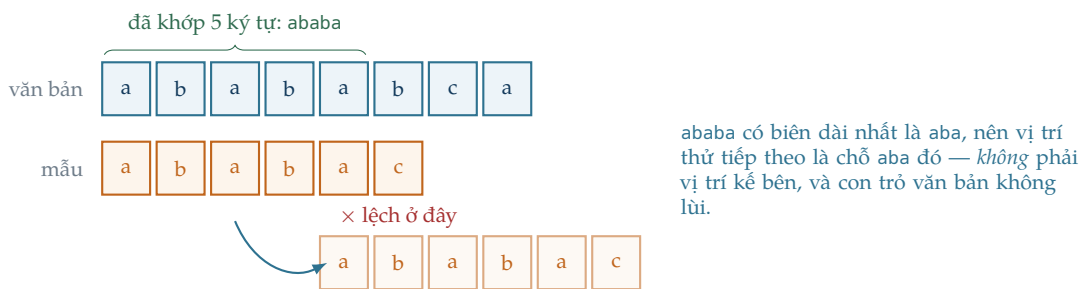
rất nhiều thông tin lặp lại, và mọi thuật toán chuỗi nhanh đều sống nhờ việc khai thác chỗ lặp lại đó.

Bài toán mở đầu là bài toán so khớp: tìm mọi vị trí xuất hiện của mẫu P độ dài m trong văn bản T độ dài n . Cách ngây thơ thử mọi vị trí bắt đầu và so từng ký tự, cho $O(nm)$. Cái lãng phí nằm ở chỗ: khi so khớp thất bại sau khi đã khớp được k ký tự, ta *đã biết* k ký tự đó của văn bản là gì — nhưng cách ngây thơ vứt bỏ thông tin ấy và bắt đầu lại từ vị trí kế tiếp. Toàn bộ chương này là các cách khác nhau để không vứt.

12.1 KMP: dùng lại thông tin của lần thất bại

Giả sử ta đang so mẫu P với văn bản và đã khớp được k ký tự thì gặp ký tự không khớp. Ta biết rằng k ký tự vừa qua của văn bản đúng bằng $P[1..k]$. Câu hỏi đúng là: *vị trí bắt đầu tiếp theo nào còn có cơ hội?* Một vị trí như vậy chỉ khả dĩ nếu phần mẫu tính từ đó khớp với đuôi của $P[1..k]$ — nghĩa là ta cần một chuỗi vừa là tiền tố vừa là hậu tố của $P[1..k]$. Đó chính là khái niệm *biên*.

Vậy toàn bộ việc tiền xử lý của KMP là tính hàm π : với mỗi tiền tố của mẫu, độ dài biên dài nhất của nó. Khi thất bại sau k ký tự khớp, ta không lùi con trỏ văn bản chút nào; ta chỉ nhảy con trỏ mẫu về $\pi[k]$ và so tiếp.



Hình 12.1. Ý tưởng của KMP. Khi lệch sau 5 ký tự khớp, ta biết văn bản vừa qua là ababa. Vị trí bắt đầu tiếp theo có cơ hội phải là chỗ mà mẫu khớp với một hậu tố của ababa đồng thời là tiền tố của mẫu — tức một biên. Biên dài nhất là aba, nên ta trượt mẫu đúng hai bước, và không ký tự nào của văn bản bị đọc lại.

Chi phí $O(n + m)$, và lập luận lại chính là lập luận khấu hao quen thuộc: con trỏ văn bản chỉ tiến, còn con trỏ mẫu tăng tối đa n lần tổng cộng nên cũng chỉ giảm tối đa n lần. Cùng dạng lập luận với hai con trỏ (B/06) và ngăn xếp đơn điệu (B/07) — đây là lần thứ ba nó xuất hiện, và đó không phải trùng hợp.

Hàm Z là công cụ tương đương về sức mạnh nhưng thường dễ nghĩ hơn: $z[i]$ là độ dài đoạn khớp dài nhất giữa chuỗi và hậu tố bắt đầu tại i . Tính được trong $O(n)$ bằng cách duy trì đoạn khớp phải nhất đã biết và tái sử dụng nó. Mẹo dùng chung cho cả hai: để tìm P trong T , hãy tính hàm trên chuỗi ghép $P + \# + T$ với $\#$ là ký tự không xuất hiện ở đâu cả; mọi vị trí có giá trị bằng m chính là một lần xuất hiện.

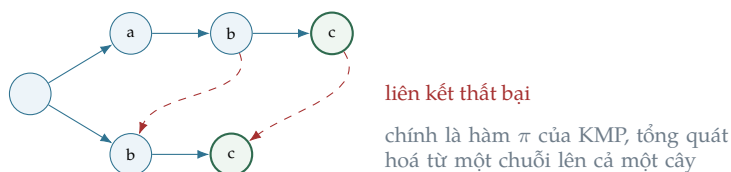
Hàm π còn cho thêm hai kết quả đáng nhớ, và chúng thường là mục đích thật sự của bài chứ không phải việc so khớp: *chu kỳ nhỏ nhất* của chuỗi bằng $n - \pi[n]$ khi n chia hết cho hiệu đó, và *mọi biên* của chuỗi tìm được bằng cách lặp $\pi[n], \pi[\pi[n]], \dots$

12.2 Aho–Corasick: KMP cho nhiều mẫu cùng lúc

Nếu có k mẫu, chạy KMP k lần cho $O(k \cdot n)$. Aho–Corasick làm trong một lượt quét văn bản.

Ý tưởng ghép hai thứ đã biết: xếp mọi mẫu vào một *trie*, rồi thêm vào mỗi nút một “liên kết thất bại” trở tới nút ứng với hậu tố dài nhất của chuỗi hiện tại mà cũng là một tiền tố của mẫu nào đó. Nói cách khác: liên kết thất bại là hàm π của KMP, tổng quát hoá từ một chuỗi lên cả một cây. Khi quét văn bản, ta đi trên *trie*; gặp ký tự không có cạnh thì theo liên kết thất bại, hết như KMP nhảy về $\pi[k]$.

Kết quả là $O(n + \text{tổng độ dài mẫu} + \text{số lần khớp})$. Đây là thuật toán chạy phía sau các công cụ tìm nhiều từ khoá trong một lượt — lệnh tìm nhiều mẫu của Unix, các hệ phát hiện xâm nhập quét gói tin theo hàng nghìn chữ ký, các bộ lọc nội dung.



Hình 12.2. Automaton Aho–Corasick cho tập mẫu $\{abc, bc\}$. Các mẫu chia sẻ tiền tố nên đi chung nhánh; liên kết thất bại (nét đứt) cho phép chuyển trạng thái khi lệch mà không đọc lại văn bản. Nhờ vậy chi phí quét không phụ thuộc số mẫu.

12.3 Băm chuỗi: lựa chọn ngắn với cái giá xác suất

Băm đa thức (B/08) cho phép so sánh hai đoạn con bất kỳ trong $O(1)$ sau tiền xử lý $O(n)$. Với công cụ đó, rất nhiều bài chuỗi sụp xuống thành dễ: tìm mẫu, tìm chuỗi con lặp lại dài nhất (kết hợp nhị phân trên độ dài, C/19), so sánh hai đoạn, đếm chuỗi con phân biệt.

Ba điều phải nhớ mỗi lần dùng, đã nói ở chương B/08 nhưng đáng nhắc lại vì đây là nơi chúng gây hại nhiều nhất: chọn cơ số *ngẫu nhiên lúc chạy*; dùng hai modulo khi số chuỗi cần so lớn hơn cỡ $3 \cdot 10^4$; và nhớ rằng đây là thuật toán xác suất.

Vậy vì sao vẫn học KMP? Ba lý do. Thứ nhất, KMP cho kết quả *chắc chắn đúng*. Thứ hai, hàm π cho thông tin về cấu trúc chuỗi — biên, chu kỳ — mà băm không cho. Thứ ba, và quan trọng nhất về mặt học tập: ý tưởng của KMP — dùng lại thông tin của lần thất bại — là ý tưởng tổng quát, và bạn sẽ gặp lại nó ở Aho–Corasick, ở các thuật toán tự động hoá, và ở nhiều nơi ngoài chuỗi.

12.4 Nhìn được mọi chuỗi con: trie, mảng hậu tố, automaton

Các công cụ trên trả lời câu hỏi về *một mẫu cho trước*. Có một lớp câu hỏi khác đòi hỏi nhiều hơn: “chuỗi con lặp lại dài nhất là gì”, “có bao nhiêu chuỗi con phân biệt”, “hai chuỗi có đoạn chung dài nhất là bao nhiêu”. Những câu này cần một cấu trúc *nhìn được toàn bộ tập chuỗi con* cùng lúc.

Trie là bước đầu: lưu một tập chuỗi thành cây theo ký tự. Nó cho tra cứu theo tiền tố, tự động hoàn thành, và là nền của Aho–Corasick. Nhược điểm: bộ nhớ lớn nếu bảng chữ cái rộng.

Mảng hậu tố là công cụ chủ lực. Ý tưởng: lấy toàn bộ n hậu tố của chuỗi và sắp chúng theo thứ tự từ điển; mảng hậu tố lưu vị trí bắt đầu của chúng theo thứ tự đó. Kèm theo là mảng LCP, ghi độ dài tiền tố chung của hai hậu tố liền nhau trong thứ tự đã sắp. Xây được trong $O(n \log n)$ hoặc $O(n)$.

Vì sao nó “giải quyết một họ bài”? Vì sau khi có mảng hậu tố và LCP, các câu hỏi về chuỗi con trở thành các câu hỏi về *đoạn trên một mảng số* — thứ mà cả phần B trước đó đã trang bị đầy đủ công cụ. Chuỗi con lặp lại dài nhất chính là giá trị lớn nhất trong mảng LCP. Số chuỗi con phân biệt là tổng độ dài các hậu tố trừ tổng LCP. Chuỗi con chung dài nhất của hai chuỗi giải bằng cách nối

hai chuỗi và tìm cực đại LCP giữa hai hậu tố khác nguồn gốc. Nghĩa là mảng hậu tố đóng vai trò một *phép biến đổi*: nó chuyển bài toán từ miền chuỗi sang miền mảng, nơi ta đã mạnh.

Suffix automaton và *suffix tree* là hai cấu trúc mạnh hơn, biểu diễn được mọi chuỗi con trong $O(n)$ trạng thái và cho nhiều bài lời giải tuyến tính đẹp. Chúng cũng khó cài và khó nhớ hơn hẳn. Lời khuyên thực dụng: nắm chắc mảng hậu tố trước; *suffix automaton* xếp vào loại tham khảo, học khi gặp bài thật cần.

12.5 Manacher và một mẫu tư duy đáng nhớ

Bài tìm chuỗi con đối xứng dài nhất có lời giải $O(n)$ của Manacher, và ý tưởng của nó là một mẫu dùng lại được: *tái sử dụng đối xứng đã biết*. Khi ta đã biết một chuỗi đối xứng lớn quanh tâm c , thì với một vị trí i nằm trong nó, ta biết trước phần lớn thông tin về bán kính đối xứng tại i — vì nó phải giống với bán kính tại vị trí đối xứng của i qua c , ít nhất cho tới khi chạm biên.

Mẫu này giống hệt cách tính hàm Z (tái sử dụng đoạn khớp phải nhất) và giống cách Aho–Corasick tái dùng liên kết thất bại. Nếu bạn muốn nhớ một điều duy nhất về nhóm thuật toán chuỗi tuyến tính, hãy nhớ: *luôn duy trì cấu trúc phải nhất đã biết, và dùng nó để bắt đầu chỗ mới không phải từ số không*.

Công cụ	Xây	Trả lời	Chọn khi
Băm chuỗi	$O(n)$	so sánh đoạn $O(1)$	Cần nhanh và ngắn; chấp nhận xác suất
KMP / hàm Z	$O(n + m)$	so khớp một mẫu	Cần chắc chắn đúng; cần biên và chu kỳ
Aho–Corasick	$O(\sum P_i)$	nhiều mẫu, một lượt	Hàng nghìn mẫu cố định
Trie	$O(\sum S_i)$	tra theo tiền tố	Tự động hoàn thành, từ điển
Mảng hậu tố + LCP	$O(n \log n)$	mọi câu hỏi về chuỗi con	Bài về chuỗi con lặp/chung/phân biệt
Suffix automaton	$O(n)$	mọi chuỗi con, đếm	Khi mảng hậu tố không đủ; tham khảo

12.6 Gốc rễ: từ tìm kiếm thư mục tới giải mã bộ gen

Bài toán so khớp chuỗi có một lịch sử gắn liền với những nhu cầu rất thực tế của từng thời kỳ, và mỗi thời kỳ lại sinh ra một cấu trúc.

Thuật toán KMP được công bố năm 1977[52], nhưng câu chuyện đằng sau đáng nhớ: Morris và Pratt tìm ra thuật toán trước, còn Knuth đến với nó từ hướng lý thuyết — ông đang nghiên cứu một kết quả về automaton hai chiều và nhận ra nó kéo theo một thuật toán so khớp tuyến tính. Ba người ghép công trình lại. Điều đáng học ở đây là hai con đường rất khác nhau — kỹ thuật và lý thuyết — dẫn tới cùng một thuật toán, và bài báo chung mạnh hơn cả hai.

Aho và Corasick công bố thuật toán của họ năm 1975[53] với một bài toán rất cụ thể trong tiêu đề: tra cứu thư mục thư viện. Nhu cầu là quét một khối văn bản để tìm hàng nghìn từ khoá cùng lúc — và cấu trúc họ tạo ra sau đó trở thành công cụ chuẩn cho việc quét mẫu trong các hệ thống an ninh mạng, nơi mỗi gói tin phải được đối chiếu với hàng nghìn chữ ký tấn công.

Nhánh cấu trúc hậu tố có một câu chuyện về đánh đổi bộ nhớ rất đáng kể. Cây hậu tố được Weiner đưa ra năm 1973 và cho lời giải tuyến tính cho rất nhiều bài, nhưng nó tốn bộ nhớ lớn — hằng số cao và phụ thuộc bảng chữ cái. Năm 1990, Manber và Myers[54] đề xuất mảng hậu tố

với động cơ rõ ràng là *tiết kiệm bộ nhớ*: thay cây bằng một mảng số nguyên. Đây là ví dụ mẫu mực cho việc một cấu trúc “yếu hơn” về lý thuyết lại thắng trong thực tế nhờ hằng số.

Và cú hích lớn nhất cho cả nhánh này đến từ ngoài ngành: dự án giải mã bộ gen người. Việc căn chỉnh hàng tỉ đoạn đọc DNA lên một bộ gen tham chiếu là bài toán so khớp chuỗi ở quy mô chưa từng có, với ràng buộc bộ nhớ khắc nghiệt. Điều đó thúc đẩy các chỉ mục nén dựa trên phép biến đổi Burrows–Wheeler, cho phép tìm kiếm trên bộ gen ba tỉ ký tự trong vài gigabyte bộ nhớ — và các công cụ căn chỉnh phổ biến trong tin sinh học ngày nay đều dựa trên đó.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1970–77 — Morris, Pratt, Knuth	So khớp một mẫu trong văn bản mà không lùi con trỏ văn bản.	KMP; hai con đường (kỹ thuật và lý thuyết automaton) gặp nhau ở cùng một thuật toán.
1975 — Aho & Corasick	Tra cứu thư mục thư viện: tìm hàng nghìn từ khoá trong một lượt quét.	Trie + liên kết thất bại; nay là lõi của quét chữ ký trong hệ thống an ninh mạng.
1973 — Weiner	Cần trả lời mọi câu hỏi về chuỗi con trong thời gian tuyến tính.	Cây hậu tố: mạnh nhưng tốn bộ nhớ, hằng số cao, phụ thuộc bảng chữ cái.
1990 — Manber & Myers	Cây hậu tố quá tốn bộ nhớ cho dữ liệu lớn thời đó.	Mảng hậu tố: đổi cấu trúc cây lấy một mảng số nguyên; ví dụ điển hình cho việc hằng số thắng lý thuyết.
1994 — Burrows & Wheeler	Nén văn bản tốt hơn bằng cách sắp xếp lại ký tự cho dễ nén.	Phép biến đổi BWT; sau này thành nền cho chỉ mục nén tìm kiếm được.
Thập niên 2000 — tin sinh học	Căn chỉnh hàng tỉ đoạn đọc DNA lên bộ gen ba tỉ ký tự với bộ nhớ hạn chế.	Chỉ mục FM dựa trên BWT; các công cụ căn chỉnh chuẩn của ngành đều dùng, đưa thuật toán chuỗi vào ứng dụng quy mô lớn nhất của nó.

I 12.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Tin sinh học	Căn chỉnh đoạn đọc DNA, tìm mẫu trong bộ gen	Cùng bài toán, khác quy mô và khác ràng buộc: bảng chữ cái chỉ bốn ký tự, nhưng chuỗi dài ba tỉ và phải chịu được sai khác. Đây là lý do ngành này đẩy mạnh chỉ mục nén, thứ mà lập trình thi đấu ít gặp.
Nén dữ liệu	LZ77, BWT	Nén tìm các đoạn lặp lại — đúng thông tin mà mảng hậu tố và LCP mô tả. Liên hệ hai chiều: cấu trúc chuỗi giúp nén, và phép biến đổi phục vụ nén (BWT) quay lại thành chỉ mục tìm kiếm.
An ninh mạng	Hệ phát hiện xâm nhập quét gói tin theo hàng nghìn chữ ký	Aho–Corasick nguyên bản. Ràng buộc thêm so với bài tập: phải chạy ở tốc độ đường truyền, nên bộ nhớ của automaton và tính thân thiện cache là yếu tố quyết định.
Trình biên dịch	Phân tích từ vựng bằng automaton hữu hạn	Aho–Corasick là một automaton nhận biết tập mẫu; bộ phân tích từ vựng cũng là automaton. Cùng ý tưởng: biên dịch tập mẫu thành một máy trạng thái rồi quét văn bản một lượt.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Công cụ tìm kiếm	Chỉ mục ngược, gợi ý tự động, sửa lỗi chính tả	Trie cho gợi ý theo tiền tố; khoảng cách chỉnh sửa (C/16) cho sửa lỗi. Khác quy mô: chỉ mục thật phân tán và nén, nhưng ý tưởng gốc vẫn là các cấu trúc ở đây.
Xử lý ngôn ngữ tự nhiên	Tách từ con (subword tokenization) bằng từ điển và trie	Bộ tách từ của mô hình ngôn ngữ dùng cấu trúc kiểu trie để khớp tham lam đoạn dài nhất. Cùng cấu trúc, khác mục tiêu: ở đây là biểu diễn đầu vào chứ không phải tìm kiếm.
Thị giác máy tính, SLAM	So khớp mô tả nhị phân, phát hiện lặp lại chuỗi ảnh	Không dùng KMP, nhưng chung tinh thần: chuyển bài so khớp về một biểu diễn cho phép so sánh $O(1)$ (băm, mã nhị phân) rồi kiểm chứng lại bằng bước đắt hơn. Kiến trúc “lọc rẻ rồi kiểm đắt” giống hệt mục 3.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Thông tin về *phần đã khớp*. Khi lệch sau k ký tự khớp, ta biết chính xác k ký tự vừa qua của văn bản bằng $P[1..k]$; cách này vớt bỏ điều đó và đọc lại từ vị trí kế tiếp. KMP dùng nó để xác định vị trí thử tiếp theo duy nhất còn cơ hội — vị trí ứng với biên dài nhất của $P[1..k]$ — nên con trỏ văn bản không bao giờ lùi. Đây là lần thứ ba trong phần B xuất hiện lập luận “mỗi chỉ số chỉ đi một chiều”.
- A2. Ba lý do. Một, KMP đúng đắn chắc chắn, còn băm có xác suất sai — nhỏ nhưng khác không, và nó tăng theo bình phương số phép so sánh. Hai, hàm π cho thông tin cấu trúc mà băm không cho: mọi biên, chu kỳ nhỏ nhất — và nhiều bài thật ra hỏi đúng những thứ này chứ không hỏi việc so khớp. Ba, ý tưởng “dùng lại thông tin của lần thất bại” là ý tưởng tổng quát, quay lại ở Aho–Corasick và ở các thuật toán tự động hoá khác.
- A3. Vì nó là một *phép biến đổi* miền bài toán: sau khi có mảng hậu tố và mảng LCP, các câu hỏi về chuỗi con trở thành câu hỏi về đoạn trên một mảng số nguyên — miền mà cả phần B đã trang bị đầy đủ công cụ. Chuỗi con lặp lại dài nhất thành cực đại của LCP; số chuỗi con phân biệt thành một hiệu của hai tổng; chuỗi chung dài nhất thành truy vấn trên đoạn. Một cấu trúc mở khoá cả một họ bài chính là vì lý do đó.
- A4. Khác ở chỗ nó *biên dịch toàn bộ tập mẫu thành một máy trạng thái* trước, rồi quét văn bản đúng một lần với chi phí không phụ thuộc số mẫu. Điều làm việc đó khả thi là các mẫu chia sẻ tiền tố với nhau — trie khai thác đúng phần chia sẻ ấy — và liên kết thất bại cho phép chuyển trạng thái khi lệch mà không cần quay lui. Chạy KMP k lần thì mỗi lần đọc lại toàn bộ văn bản, tức là bỏ qua chính phần chia sẻ đó.
- A5. Tái sử dụng đối xứng đã biết. Nếu đã biết một chuỗi đối xứng lớn quanh tâm c , thì với vị trí i nằm bên trong nó, bán kính đối xứng tại i ít nhất bằng bán kính tại điểm đối xứng của i qua c , cho tới khi chạm biên của chuỗi lớn. Nhờ đó phần lớn vị trí được khởi tạo sẵn thay vì bắt đầu từ số không, và tổng chi phí mở rộng là tuyến tính. Cùng mẫu với cách tính hàm Z: luôn duy trì cấu trúc phải nhất đã biết.

NÓI LẠI THẬT ĐƠN GIẢN

Giả sử bạn đang dò xem từ “ababac” có xuất hiện trong một trang sách hay không. Cách vụng là đặt từ đó lên đầu dòng, so từng chữ; hễ sai một chữ thì dịch sang phải một ô rồi so lại từ đầu. Cách này phí ở chỗ: khi bạn đã so đúng năm chữ rồi mới sai, bạn *đã biết* năm chữ đó của trang sách là gì — vậy mà bạn vứt hết và dò lại.

KMP chỉ làm một việc: trước khi dò, nó đọc kỹ chính cái từ cần tìm và ghi lại một bảng nhỏ trả lời câu “nếu tôi khớp được k chữ rồi sai, thì phải trượt sang bao nhiêu ô là hợp lý”. Câu trả lời phụ thuộc vào chỗ từ đó tự lặp lại chính nó ở đầu và cuối — ví dụ “ababa” có phần “aba” vừa là đầu vừa là đuôi, nên trượt hai ô là đủ, không cần trượt một ô. Nhờ bảng đó, mắt bạn không bao giờ phải đọc lại một chữ nào của trang sách.

Nếu cần tìm hàng nghìn từ cùng lúc thì đừng dò từng từ một. Hãy xếp tất cả các từ vào một cái cây theo chữ cái đầu — những từ có chung phần đầu thì đi chung một nhánh — rồi đọc trang sách một lượt và đi trên cây đó. Đó là Aho–Corasick.

Cuối cùng, có một cách nhìn khác cho các câu hỏi kiểu “đoạn nào lặp lại nhiều nhất”: hãy liệt kê mọi phần đuôi của chuỗi rồi xếp chúng theo thứ tự từ điển, như xếp từ trong từ điển. Khi đó những đoạn giống nhau tự nằm cạnh nhau, và câu hỏi “đoạn lặp dài nhất” chỉ còn là “hai hàng xóm nào giống nhau nhiều chữ nhất”. Đó là mảng hậu tố, và giá trị của nó là biến một bài về chuỗi thành một bài về dãy số thông thường.

Một câu: Mọi thuật toán chuỗi nhanh đều trả lời cùng một câu: khi thất bại, tôi đã biết được gì để không phải làm lại từ đầu?

Chỗ không rút gọn được: Cấu trúc hậu tố (mảng hậu tố, automaton) không có cách giải thích nào thật sự ngắn gọn; phải cài một lần và chạy tay trên ví dụ nhỏ mới thấy được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý chung của mọi thuật toán chuỗi tuyến tính	Khi thất bại tại vị trí i , khai thác thông tin đã biết thay vì bắt đầu lại; duy trì “cấu trúc phải nhất đã biết” và dùng nó khởi tạo cho vị trí mới.
••• Biên và hàm π	Biên là chuỗi vừa là tiền tố vừa là hậu tố thực sự; $\pi[i]$ là biên dài nhất của tiền tố kết thúc tại i . Toàn bộ tiền xử lý của KMP.
••• Vì sao KMP là $O(n + m)$	Con trỏ văn bản chỉ tiến; con trỏ mẫu tăng tối đa n lần nên giảm tối đa n lần. Cùng lập luận với hai con trỏ và ngăn xếp đơn điệu.
••• Chu kỳ nhỏ nhất từ π	Bằng $n - \pi[n]$ khi n chia hết cho hiệu đó; mọi biên tìm bằng cách lặp $\pi[n], \pi[\pi[n]], \dots$
••• Mẹo dùng chung của KMP và hàm Z	Tính hàm trên chuỗi ghép $P + \# + T$; vị trí có giá trị bằng m là một lần xuất hiện.
••• Mảng hậu tố giải quyết gì	Biến câu hỏi về chuỗi con thành câu hỏi về đoạn trên mảng số: chuỗi lặp dài nhất = $\max \text{LCP}$; số chuỗi con phân biệt = tổng độ dài trừ tổng LCP.
•• Aho–Corasick	Trie của mọi mẫu + liên kết thất bại (chính là π tổng quát hoá lên cây); quét văn bản một lượt, chi phí không phụ thuộc số mẫu.
•• Ba lưu ý khi băm chuỗi	Cơ số ngẫu nhiên lúc chạy; hai modulo khi số chuỗi vượt cỡ $3 \cdot 10^4$; luôn nhớ đây là thuật toán xác suất.
•• Vì sao vẫn học KMP dù có băm	Chắc chắn đúng; cho thông tin cấu trúc (biên, chu kỳ); và ý tưởng “dùng lại lần thất bại” tổng quát hơn nhiều so với một thuật toán.
•• Manacher	Tái sử dụng đối xứng: bán kính tại i khởi tạo từ bán kính tại điểm đối xứng qua tâm của chuỗi đối xứng phải nhất đã biết.
•• Trie	Cây theo ký tự, mỗi đường là một tiền tố; dùng cho từ điển, gợi ý tự động, và làm nền cho Aho–Corasick.
• Suffix automaton / suffix tree	Biểu diễn mọi chuỗi con trong $O(n)$ trạng thái; mạnh hơn mảng hậu tố nhưng khó cài — học sau khi đã chắc mảng hậu tố.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên học KMP hay hàm Z trước?

| Hàm Z, vì định nghĩa của nó trực tiếp hơn — “đoạn khớp với tiền tố bắt đầu tại i dài bao nhiêu” — và cách tính dễ hình dung bằng hình. KMP có định nghĩa gián tiếp hơn qua khái niệm biên, nhưng

lại cần thiết khi bài hỏi về chu kỳ, về biên, hoặc khi cần xây automaton. Trong thực hành thi đấu, nhiều người dùng hàm Z cho việc so khớp và chỉ dùng π khi bài hỏi cấu trúc.

2. Khi nào tôi thực sự cần mảng hậu tố thay vì băm chuỗi?

| Khi bài hỏi về *tồn bộ tập chuỗi con* chứ không về một mẫu cụ thể: đếm số chuỗi con phân biệt, sắp xếp các chuỗi con, tìm chuỗi con thứ k theo thứ tự từ điển. Băm giải tốt các bài “so sánh hai đoạn” và nhiều bài tìm kiếm có kết hợp nhị phân, nhưng nó không cho bạn một thứ tự trên tập chuỗi con. Quy tắc thực dụng: đề có chữ “phân biệt”, “thứ k theo từ điển”, “sắp xếp các chuỗi con” thì nghĩ tới mảng hậu tố.

3. Xây mảng hậu tố $O(n \log^2 n)$ có đủ dùng không?

| Hầu như luôn đủ với n tới vài trăm nghìn, và nó ngắn hơn hẳn bản $O(n \log n)$ dùng sắp xếp đếm. Trong bài mà n tới 10^6 , hãy dùng bản $O(n \log n)$. Bản tuyến tính đẹp về lý thuyết nhưng dài và ít khi cần — đúng tinh thần “chọn cấu trúc yếu nhất còn đủ dùng”.

4. Aho–Corasick tốn bộ nhớ thế nào khi bảng chữ cái lớn?

| Nếu mỗi nút lưu một mảng chuyển trạng thái đầy đủ thì bộ nhớ là số nút nhân kích thước bảng chữ cái — với bảng Unicode thì không khả thi. Ba cách xử lý: dùng bảng băm cho các cạnh thay vì mảng; chỉ lưu cạnh thật và tra liên kết thất bại khi thiếu (chậm hơn một chút nhưng gọn hơn nhiều); hoặc ánh xạ bảng chữ cái về tập ký tự thực sự xuất hiện. Trong hệ thống thật, lựa chọn này quyết định việc automaton có nằm vừa trong cache hay không, và đó mới là yếu tố chi phối tốc độ.

5. Bài so khớp có cho phép sai khác vài ký tự thì dùng gì?

| Đó là bài toán khác hẳn và các công cụ trong chương này không giải trực tiếp. Với sai khác nhỏ, hướng chuẩn là quy hoạch động khoảng cách chỉnh sửa (C/16), có thể tăng tốc bằng cách chỉ tính một dải quanh đường chéo khi số sai khác bị chặn. Với dữ liệu rất lớn như bộ gen, người ta dùng chiến lược “hạt giống rồi mở rộng”: tìm các đoạn khớp chính xác ngắn bằng chỉ mục, rồi mở rộng bằng căn chỉnh có chi phí — lại đúng kiến trúc lọc rẻ trước, kiểm đất sau.

6. Vì sao ký tự phân cách # trong mẹo $P + \# + T$ lại quan trọng?

| Vì nếu không có nó, một đoạn khớp có thể vắt qua ranh giới giữa mẫu và văn bản, tạo ra kết quả sai. Ký tự phân cách phải *không xuất hiện* trong cả mẫu lẫn văn bản, và trong bài dùng bảng chữ cái đầy đủ thì bạn phải chọn một giá trị ngoài bảng — đây là lỗi hay gặp khi bài cho phép mọi byte làm ký tự.

7. Có mối liên hệ nào giữa chương này và automaton trong lý thuyết tính toán không?

| Có, và nó khá trực tiếp. Hàm π của KMP chính là hàm chuyển trạng thái của một automaton hữu hạn nhận biết mẫu; Aho–Corasick là automaton nhận biết một tập mẫu; suffix automaton là automaton nhận biết mọi hậu tố của chuỗi. Nhìn theo góc này, cả chương là một chuỗi các cách xây automaton nhỏ gọn cho những ngôn ngữ rất cụ thể — và đó cũng là con đường mà Knuth đã đi tới KMP.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tìm mọi vị trí xuất hiện của một mẫu	CSES “String Matching”[33]	Làm bằng cả ba cách: băm, hàm Z, KMP; so độ dài code và thời gian chạy.
● Tính hàm tiền tố và liệt kê mọi biên của chuỗi	Bài kinh điển	Lập $\pi[n], \pi[\pi[n]], \dots$; kiểm chứng bằng tay trên chuỗi ngắn.
● Kiểm tra một chuỗi có phải là phép quay của chuỗi kia	Bài kinh điển	Nối chuỗi với chính nó rồi so khớp — một mẹo đáng nhớ.
●● Chu kỳ nhỏ nhất của chuỗi	CSES nhóm bài chuỗi[33]	$n - \pi[n]$ khi chia hết; hãy tự chứng minh điều kiện chia hết là cần thiết.
●● Đếm số lần xuất hiện của mỗi mẫu trong một tập mẫu	Bài kinh điển	Aho–Corasick; cộng dồn theo cây liên kết thất bại.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Chuỗi con đối xứng dài nhất	LeetCode “Longest Palindromic Substring”	Manacher; hoặc băm cộng nhị phân trên bán kính — so hai cách.
•• Tiền tố chung dài nhất của một tập chuỗi	Bài kinh điển	Trie hoặc so trực tiếp; bài đơn giản nhưng luyện thói quen xét trường hợp tập rỗng.
••• Chuỗi con lặp lại dài nhất	Bài kinh điển	Mảng hậu tố + LCP, hoặc băm + nhị phân; hai cách cùng $O(n \log n)$ với hằng số rất khác.
••• Đếm số chuỗi con phân biệt	CSES “Distinct Substrings”-loại[33]	Tổng độ dài hậu tố trừ tổng LCP; tự dẫn ra công thức trước khi tra.
••• Chuỗi con chung dài nhất của hai chuỗi	Bài kinh điển	Nối hai chuỗi với ký tự phân cách; cực đại LCP giữa hai hậu tố khác nguồn.
••• Chuỗi con thứ k theo thứ tự từ điển	Bài thi đấu kinh điển	Bài mà băm không giải được — lý do rõ ràng nhất để học mảng hậu tố hoặc suffix automaton.
••• Cài Aho–Corasick và dùng nó lọc một tập từ khoá trên văn bản lớn	Bài tập ứng dụng	Đo thời gian so với chạy KMP nhiều lần; số liệu sẽ cho thấy khác biệt rõ hơn mọi lập luận.

Kết chương và đọc tiếp

Phần B khép lại ở đây. Bảy chương vừa qua đều trả lời cùng một câu hỏi — *dữ liệu nên nằm ở dạng nào để thao tác tôi lập lại nhiều nhất trở nên rẻ* — và đều tuân theo cùng một nguyên tắc: chọn cấu trúc duy trì *đúng* lượng trật tự mà bài cần, không hơn. Heap biết ít hơn cây nên rẻ hơn; DSU biết ít hơn đồ thị nên gần như miễn phí; băm bỏ thứ tự nên nhanh hơn cây. Mỗi khả năng bạn đòi thêm đều có giá.

Phần C đổi câu hỏi lần nữa, và đây là câu hỏi trung tâm của cả quyển sách: *dùng khuôn tư duy nào để sinh ra lời giải*. Nó bắt đầu từ chỗ mọi lời giải bắt đầu — vết cạn — và chỉ ra rằng phần lớn thuật toán hay chỉ là vết cạn cộng thêm một quan sát cho phép bỏ bớt nhánh (C/13).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Knuth, Morris, Pratt, “Fast Pattern Matching in Strings”	SIAM Journal on Computing, 1977	Bài gốc của KMP. Đọc phần xây hàm thất bại — chỗ mà mọi tài liệu về sau đều rút gọn quá tay.
nền tảng	Aho & Corasick, “Efficient String Matching: An Aid to Bibliographic Search”	Communications of the ACM, 1975	Động cơ ban đầu là tra cứu thư mục; đọc để thấy vì sao cấu trúc phải là trie cộng liên kết thất bại.
tra khi cần	Manber & Myers, “Suffix Arrays: A New Method for On-line String Searches”	SIAM Journal on Computing, 1993	Bài đưa mảng hậu tố thành lựa chọn thay cây hậu tố; đọc phần so sánh bộ nhớ.
tra khi cần	Kärkkäinen, Sanders, Burkhardt, “Linear Work Suffix Array Construction”	Journal of the ACM, 2006	Thuật toán skew: dựng mảng hậu tố trong $O(n)$ bằng chia để trị theo phần dư của chỉ số. Đọc khi cần bản tuyến tính thay cho bản $O(n \log n)$.
tra khi cần	Ukkonen, “On-line Construction of Suffix Trees”	Algorithmica, 1995	Bản dựng cây hậu tố trực tuyến; tra khi cần chứ không cần thuộc.

CHIẾN LƯỢC THIẾT KẾ

TÓM TẮT

Phần C là phần dày nhất và cũng là phần sinh ra nhiều lời giải nhất trong thực tế. Tám chương ứng với tám khuôn tư duy: vết cặn có cất tĩa, chia để trị, tham lam, quy hoạch động, đồ thị, luồng và ghép đôi, tìm kiếm trên đáp án, và ngẫu nhiên hoá. Điểm chung của cả tám: mỗi khuôn khai thác một loại cấu trúc riêng — tính con-bài-toán-lập-lại, tính chọn-được-cục-bộ, tính đơn điệu, tính đối ngẫu — và câu hỏi cốt lõi của phần này là “bài toán trước mặt để lộ ra cấu trúc nào”. Nếu chỉ nhớ một điều: mọi lời giải nhanh đều bắt đầu từ một lời giải vết cặn đúng, rồi tìm cách bỏ đi phần công việc lặp lại hoặc chằng chịt vô ích.

Trình tự tám chương phản ánh một mạch phát triển ý tưởng chứ không phải một danh sách. Chương 13 bắt đầu từ vết cặn, cách giải luôn tồn tại và luôn đúng — và chỉ ra rằng phần lớn thuật toán hay chỉ là vết cặn cộng thêm một quan sát giúp bỏ bớt nhánh. Chương 14 lấy ý tưởng “bỏ bớt” đó theo hướng cấu trúc đệ quy: nếu bài chia được thành các phần độc lập thì chi phí sụp xuống theo hệ thức truy hồi. Chương 15 và 16 là hai cách khai thác con bài toán chồng lặp: tham lam khi lựa chọn cục bộ chứng minh được là an toàn, quy hoạch động khi không.

Chương 17 và 18 chuyển sang một ngôn ngữ khác: khi bạn phát biểu lại bài toán thành đồ thị, rất nhiều bài tưởng khác nhau hoá ra là cùng một bài. Đây là kỹ năng *mô hình hoá*, và nó đáng giá hơn việc thuộc thêm mười thuật toán. Chương 19 và 20 khép lại bằng hai công cụ có tỉ lệ hiệu quả trên độ khó cao bất thường: nhị phân trên đáp án biến bài tối ưu thành bài kiểm tra, và ngẫu nhiên hoá cho lời giải đơn giản đến bất ngờ cho những bài mà lời giải tất định rất rắc rối.

Bản đồ phần C — Chiến lược thiết kế**TÓM TẮT**

Tám chương, tám khuôn tư duy, và mỗi khuôn có một *dấu hiệu nhận biết* trên đề bài. Bản đồ này liệt kê tám dấu hiệu đó thành bảng để dùng lúc đang bí. Chuỗi phụ thuộc: 13 → 14 → 16 là mạch chính; 15 nên đọc ngay trước 16 để thấy ranh giới giữa tham lam và quy hoạch động; 17 là điều kiện tiên quyết của 18; 19 và 20 độc lập, đọc sớm được và nên đọc sớm.

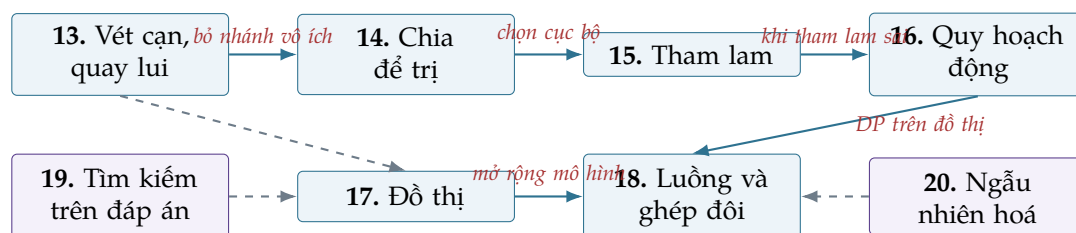
A. Bảng dấu hiệu: đề bài nói gì → thử khuôn nào

Dấu hiệu trên đề	Chương	Vì sao
$n \leq 20$ –25, hỏi “có tồn tại cách nào”	13	không gian 2^n còn duyệt nổi
Bài chia đôi mà hai nửa không ảnh hưởng nhau	14	truy hồi $T(n) = 2T(n/2) + f(n)$
Xếp lịch, chọn nhiều nhất, luôn có “cái tốt nhất hiện tại”	15	thử đổi chỗ để chứng minh an toàn
Đếm số cách, giá trị tối ưu, có trạng thái nhỏ	16	con bài toán lặp lại
Quan hệ đôi một, đường đi, phụ thuộc	17	phát biểu lại thành đồ thị
Phân công, sức chứa, “mỗi bên dùng một lần”	18	luồng cực đại hoặc ghép đôi
“Nhỏ nhất của giá trị lớn nhất”, đáp án đơn điệu	19	nhị phân trên đáp án
Bài tất định rắc rối, chỉ cần đúng với xác suất cao	20	ngẫu nhiên hoá

B. Vì sao chia như vậy

Tám khuôn này không phải tám chủ đề rời rạc mà là một cây quyết định. Gốc cây luôn là vết cạn: viết ra lời giải chậm nhưng chắc chắn đúng, rồi hỏi *tại sao nó chậm*. Nếu chậm vì tính lại cùng một thứ nhiều lần, đi hướng quy hoạch động. Nếu chậm vì duyệt những nhánh chắc chắn không tối ưu, đi hướng tham lam hoặc cắt tỉa. Nếu chậm vì bài toán bị nhìn sai dạng, đi hướng mô hình hoá thành đồ thị hoặc luồng. Cách chia này giữ nguyên mạch suy nghĩ thật khi giải bài, thay vì trình bày tám thuật toán như tám hòn đảo.

C. Phụ thuộc và thứ tự đọc



Hình 12.3. Phụ thuộc giữa tám chương của phần C. Mạch trên là mạch “khai thác cấu trúc đệ quy”; mạch dưới là mạch “mô hình hoá”. Hai chương tô tím là công cụ cắt ngang, dùng được cho cả hai mạch.

D. Cốt lõi hay tham khảo

Cốt lõi: toàn bộ chương 15, 16, 17 và 19 — bốn chương này chiếm tỉ lệ áp đảo trong mọi kỳ thi, phỏng vấn và cả công việc thật. Trong chương 13, phần cắt tỉa và gặp nhau ở giữa. Trong chương 14, phần đọc hệ thức truy hồi. **Tham khảo:** luồng chi phí nhỏ nhất và Hungarian ở chương 18; các tối ưu quy hoạch động nâng cao (divide and conquer optimization, Knuth optimization) ở chương 16; các thuật toán ngẫu nhiên chuyên sâu ở chương 20.

Để đối chiếu cách chọn, xem hình đối chỗ ở [C/15](#), chiều duyệt ba lô ở [C/16](#), phản ví dụ BFS/Dijkstra ở [C/17](#), cạnh dư ngược ở [C/18](#) và vết chạy tìm kiếm đáp án ở [C/19](#).

Vét cạn, quay lui và cắt tỉa (Brute Force, Backtracking, Pruning)

PHỤ THUỘC A/01 cho vai trò của vét cạn trong bước 2; A/03 cho việc ước lượng kích thước không gian tìm kiếm; B/07 cho ngăn xếp và đệ quy.

TÓM TẮT

Vét cạn là lời giải luôn tồn tại và luôn đúng, nên nó là điểm xuất phát của mọi chương trong phần C. Chương này dựng nó thành một kỹ thuật đẳng hoàng: khuôn quay lui chuẩn, cách ước lượng kích thước không gian trước khi cài, và bốn nhóm cắt tỉa — cắt theo cận, cắt theo đối xứng, cắt theo ràng buộc lan truyền, và sắp thứ tự biến để thất bại sớm. Chương cũng nói về gặp nhau ở giữa, kỹ thuật biến 2^n thành $2^{n/2}$, và về ranh giới quan trọng: khi nào vét cạn có cắt tỉa *chính là* lời giải đúng chứ không phải giải pháp tạm. Nếu chỉ nhớ một điều: đừng hỏi “làm sao tránh vét cạn”, hãy hỏi “vét cạn của tôi đang lãng phí ở đâu” — câu thứ hai dẫn thẳng tới mọi kỹ thuật của bảy chương còn lại.

KIẾN THỨC CẦN CÓ TRƯỚC

- **không gian tìm kiếm** — tập mọi cấu hình mà thuật toán có thể phải xét; kích thước của nó quyết định vét cạn có khả thi hay không.
- **quay lui (backtracking)** — xây lời giải từng bước; khi một bước dẫn tới ngõ cụt thì huỷ bước đó và thử lựa chọn khác. Về bản chất là duyệt sâu trên cây các lời giải bộ phận.
- **cắt tỉa (pruning)** — loại bỏ cả một nhánh của cây tìm kiếm khi chứng minh được nhánh đó không chứa lời giải tốt hơn cái đã có.
- **nhánh cận (branch and bound)** — quay lui có kèm chặn: nếu cận tốt nhất có thể của một nhánh còn tệ hơn lời giải đã tìm được thì bỏ nhánh.
- **gặp nhau ở giữa (meet in the middle)** — chia bài toán thành hai nửa độc lập, liệt kê đủ mỗi nửa ($2^{n/2}$), rồi ghép hai danh sách bằng sắp xếp hoặc bảng băm.
- **phá đối xứng** — loại bỏ những cấu hình chỉ khác nhau bởi một phép hoán vị hay phản chiếu vô nghĩa, để không đếm và không duyệt trùng.

CÂU HỎI MỞ ĐẦU

- Q1. Vét cạn luôn đúng và luôn chậm. Vậy vì sao chương đầu tiên của phần thiết kế lại dành cho nó?
- Q2. Bài toán tám quân hậu có $64^8 \approx 2,8 \cdot 10^{14}$ cách đặt thô, nhưng chương trình quay lui giải trong tích tắc. Ba tầng cắt giảm nào đã xảy ra?
- Q3. Cắt tỉa không đối độ phức tạp trường hợp xấu nhất. Vậy tại sao nó lại tạo khác biệt lớn tới vậy trong thực tế?
- Q4. Kỹ thuật gặp nhau ở giữa đưa 2^n xuống $2^{n/2}$ — vẫn là hàm mũ. Vì sao điều đó vẫn thay đổi được kết cục của bài toán?
- Q5. Khi nào một lời giải vét cạn có cắt tỉa là câu trả lời *đúng* cho bài toán, chứ không phải là giải pháp tạm bợ?

Phần A kết thúc bằng một lời khuyên lặp lại nhiều lần: khi bí, hãy viết ra lời giải vét cạn trước. Chương này giữ đúng lời khuyên đó nhưng đi xa hơn — nó chỉ ra rằng vét cạn không chỉ là bước đệm mà là một kỹ thuật có kỹ thuật riêng, và trong một số bài, vét cạn được cắt tỉa tốt *chính là* lời giải tốt nhất mà loài người biết.

Điều này đáng nhấn mạnh vì người mới thường coi vét cạn là thứ đáng xấu hổ. Thực tế thì các bộ giải SAT hiện đại — công cụ giải những bài logic công nghiệp với hàng triệu biến — về bản chất là quay lui có học từ thất bại. Các máy chơi cờ mạnh nhất trong nhiều thập kỷ là tìm kiếm có cắt tỉa. Đó không phải giải pháp tạm; đó là trạng thái nghệ thuật cho những bài toán mà không ai biết lời giải đa thức.

13.1 Ước lượng trước khi cài

Việc đầu tiên, và nó tốn ba mươi giây: *ước lượng kích thước không gian tìm kiếm*. Ba dạng thường gặp và cách nhận ra chúng trên đề bài:

Tập con — 2^n cấu hình. Dấu hiệu: mỗi phần tử chỉ có hai lựa chọn, chọn hoặc không. Khả thi tới $n \approx 25$.

Hoán vị — $n!$ cấu hình. Dấu hiệu: sắp thứ tự, gán một-một. Khả thi tới $n \approx 11$.

Gán k giá trị cho n biến — k^n . Dấu hiệu: tô màu, điền số. Khả thi khi k^n dưới cỡ 10^8 .

Bảng ở chương A/03 biến các con số này thành quyết định. Nhưng có một điều quan trọng hơn con số thô: ước lượng đó là ước lượng *trước khi cắt tỉa*. Với cắt tỉa tốt, số nhánh thực sự được duyệt có thể nhỏ hơn hàng chục bậc — và đó chính là lý do bài tám quân hậu giải được. Vì vậy quy trình đúng là: ước lượng thô để biết mình đang ở đâu, rồi hỏi “ràng buộc nào của đề cho phép tôi cắt sớm”.

13.2 Khuôn quay lui và bốn tầng cắt

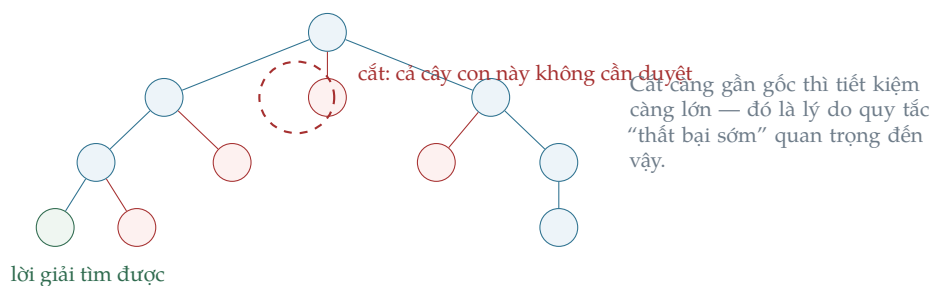
Mọi thuật toán quay lui đều có cùng khuôn: chọn biến tiếp theo, thử từng giá trị khả dĩ, kiểm tra tính hợp lệ, đệ quy, rồi huỷ lựa chọn. Phần khác nhau — và phần tạo ra toàn bộ hiệu năng — nằm ở bốn chỗ cắt.

Tầng 1 — cắt theo ràng buộc ngay khi đặt. Đừng đặt xong cả cấu hình rồi mới kiểm tra. Với bài tám quân hậu, ngay khi đặt quân thứ hai đã kiểm tra xung đột với quân thứ nhất; nhánh nào vi phạm thì cắt luôn. Chỉ tầng này thôi đã đưa 64^8 xuống cỡ vài nghìn nút.

Tầng 2 — phá đối xứng. Nhiều cấu hình chỉ khác nhau bởi một phép hoán vị vô nghĩa. Nếu bài không phân biệt thứ tự các phần tử được chọn, hãy ép chỉ số tăng dần; nếu bàn cờ đối xứng, hãy cố định quân đầu tiên ở nửa trái. Cắt này thường cho hệ số 2 tới $n!$ tùy bài, và nó cũng chữa luôn lỗi đếm trùng — một lỗi rất hay gặp trong các bài đếm.

Tầng 3 — cắt theo cận (nhánh cận). Nếu đang tìm cực tiểu và đã có lời giải giá B , thì với một nhánh mà ngay cả trong trường hợp tốt nhất vẫn không thể dưới B , hãy bỏ nhánh. Chất lượng của cắt này phụ thuộc vào chất lượng của hàm cận: cận càng chặt thì cắt càng nhiều, nhưng tính cận cũng tốn. Đây là một sự đánh đổi phải cân bằng bằng thực nghiệm.

Tầng 4 — thứ tự thử. Hai quy tắc kinh nghiệm mạnh. Thứ nhất, *chọn biến bị ràng buộc nhiều nhất trước* — ô sudoku còn ít khả năng nhất, quân cờ có ít nước đi nhất — vì như vậy thất bại xảy ra sớm, gần gốc, cắt được cây to. Thứ hai, *thử giá trị hứa hẹn nhất trước* khi tìm cực trị, vì tìm được lời giải tốt sớm làm cận ở tầng 3 chặt hơn cho toàn bộ phần còn lại.



Hình 13.1. Cây quay lui với các nhánh bị cắt (đỏ). Điều đáng nhớ: giá trị của một lần cắt tỉ lệ với kích thước cây con bị loại, mà kích thước đó giảm theo hàm mũ theo độ sâu. Vì vậy một cắt ở tầng 2 đáng giá hơn hàng nghìn cắt ở tầng 10, và đó là toàn bộ lý do của quy tắc “chọn biến bị ràng buộc nhiều nhất trước”.

CẦN NHỚ

Cắt tỉa *không* cải thiện độ phức tạp trường hợp xấu nhất — bài toán vẫn là hàm mũ. Nó chỉ thay đổi trường hợp trung bình trên dữ liệu thực tế, nhưng thay đổi đó thường là vài bậc độ lớn, đủ để biến “không chạy nổi” thành “xong trong một giây”. Đây là một trong số ít chỗ trong quyển sách này mà lý thuyết và thực hành nói hai câu chuyện khác nhau, và cả hai đều đúng.

13.3 Gặp nhau ở giữa: đổi bộ nhớ lấy thời gian

Khi n khoảng 40, $2^n \approx 10^{12}$ là quá lớn, nhưng $2^{n/2} \approx 10^6$ thì rất thoải mái. Kỹ thuật gặp nhau ở giữa khai thác đúng khe đó: chia tập thành hai nửa, liệt kê đủ mọi tập con của mỗi nửa, rồi ghép hai danh sách.

Ví dụ chuẩn: cho $n \leq 40$ số, hỏi có tập con nào có tổng bằng S không. Liệt kê mọi tổng của nửa đầu (2^{20} giá trị) và mọi tổng của nửa sau; sắp xếp danh sách thứ hai, rồi với mỗi tổng x của nửa đầu, tìm nhị phân $S - x$. Chi phí $O(2^{n/2} \cdot n)$ thời gian và $O(2^{n/2})$ bộ nhớ.



Với mỗi tổng x của nửa trái, tìm $S - x$ trong nửa phải đã sắp: $O(2^{n/2} \log)$ thay vì $O(2^n)$.
 Với $n = 40$: $2^{40} \approx 10^{12}$ không chạy nổi, còn $2^{20} \approx 10^6$ thì tức thì. Sáu bậc độ lớn là ranh giới giữa hai kết cục.

Hình 13.2. Gặp nhau ở giữa. Điều cần nhận ra trên đề bài chính là giới hạn: n trong khoảng 30–45 nằm đúng giữa hai vùng — quá lớn cho 2^n , quá nhỏ để đòi hỏi lời giải đa thức — nên nó gần như luôn là lời mời dùng kỹ thuật này.

Điều đáng nhớ về mặt tư duy không phải bản thân kỹ thuật mà là *dấu hiệu* nhận biết: giới hạn n khoảng 30–45 gần như luôn là lời mời gặp nhau ở giữa, vì nó nằm đúng giữa hai vùng — quá lớn cho 2^n , quá nhỏ để đòi hỏi lời giải đa thức. Đây là ví dụ rõ nhất cho luận điểm ở chương A/03: giới hạn trên đề là một nửa gợi ý.

Điều kiện áp dụng cũng cần nói rõ: hai nửa phải *độc lập* và phải *ghép được bằng một phép tra cứu*. Nếu quyết định ở nửa sau phụ thuộc chi tiết vào cấu hình cụ thể của nửa đầu chứ không chỉ vào một giá trị tổng hợp, thì kỹ thuật này không áp dụng — và lúc đó bạn đang ở địa hạt của quy hoạch động (C/16).

13.4 Khi vét cạn là câu trả lời đúng

Có ba tình huống mà vét cạn có cắt tỉa không phải giải pháp tạm mà là lời giải tốt nhất đã biết.

Thứ nhất: bài toán NP-khó với dữ liệu nhỏ. Không ai biết thuật toán đa thức, và n đủ nhỏ. Lúc đó câu hỏi không còn là “có tránh được hàm mũ không” mà là “hằng số của hàm mũ nhỏ tới đâu”. Chương D/23 bàn kỹ.

Thứ hai: bài toán công nghiệp có cấu trúc. Các bài logic, lập lịch và kiểm chứng phần cứng trong thực tế có rất nhiều cấu trúc ẩn, và các bộ giải hiện đại khai thác được. Một bộ giải SAT hiện nay có thể xử lý những bài hàng triệu biến — không phải vì nó tránh được hàm mũ mà vì nó *học từ mỗi lần thất bại*: khi một nhánh mâu thuẫn, nó suy ra một mệnh đề mới ngăn cả một lớp cấu hình tương tự. Đây là bước tiến khái niệm quan trọng nhất của quay lui trong ba chục năm qua.

Thứ ba: khi bạn cần một lời giải chắc chắn đúng để đối chiếu. Trong quy trình stress test ở chương E/25, lời giải vét cạn đóng vai trò trọng tài. Nó chậm nhưng chạy trên dữ liệu nhỏ, và nó là thứ duy nhất bạn tin tuyệt đối.

13.5 Gốc rễ: từ trò chơi ô chữ tới bộ giải công nghiệp

Quay lui là kỹ thuật cổ, và nó có tên trước khi có máy tính hiện đại: nhà toán học D. H. Lehmer đặt ra thuật ngữ *backtrack* trong thập niên 1950 khi mô tả cách giải các bài toán tổ hợp bằng máy. Đến năm 1965, Golomb và Baumert viết một khảo sát hệ thống hoá kỹ thuật này, và nhiều mẹo trong mục 2 — phá đối xứng, chọn biên ràng buộc nhiều nhất — đã có mặt ở đó.

Có một đóng góp của Knuth đáng nhắc riêng vì nó rất thực dụng và ít người biết. Năm 1975, ông công bố một cách *ước lượng* kích thước cây quay lui mà không phải duyệt hết: chạy thử một đường ngẫu nhiên từ gốc xuống lá, mỗi bước nhân số nhánh khả dĩ vào một tích, lặp lại vài trăm lần rồi lấy trung bình. Kết quả là ước lượng không chệch cho tổng số nút. Ý nghĩa thực hành rất lớn: trước khi bỏ hai giờ chờ một chương trình quay lui, bạn tốn vài giây để biết nó sẽ chạy vài giây hay vài năm. Đây là một ví dụ đẹp cho tinh thần “đo trước khi làm” ở chương A/02.

Nhánh nhánh cận đến từ nghiên cứu vận trù học: Land và Doig đề xuất nó năm 1960 cho quy hoạch nguyên, tức là để giải các bài tối ưu công nghiệp có biến nguyên. Nhánh SAT bắt đầu năm 1962 với thuật toán DPLL — vẫn là quay lui, cộng thêm hai quy tắc lan truyền ràng buộc. Bước nhảy vọt thật sự đến vào cuối thập niên 1990 với ý tưởng *học mệnh đề từ xung đột*: mỗi lần bế tắc, bộ giải phân tích nguyên nhân và ghi lại một ràng buộc mới. Chính bước này biến SAT từ bài toán học thuật thành công cụ công nghiệp, dùng để kiểm chứng thiết kế vi mạch và phân tích chương trình.

Cuối cùng, một liên hệ đáng nhớ với miền khác: trong thị giác máy tính và robot, khi bài toán ước lượng có quá nhiều tổ hợp dữ liệu ngoại lai, người ta không vét cạn mà *lấy mẫu ngẫu nhiên* — kỹ thuật RANSAC. Đó là một câu trả lời khác cho cùng câu hỏi “không gian quá lớn thì làm gì”, và chương C/20 sẽ khai thác chính hướng đó.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — Lehmer	Giải các bài toán tổ hợp bằng máy tính đời đầu.	Thuật ngữ <i>backtrack</i> ; ý tưởng huỷ lựa chọn và thử nhánh khác được đặt tên.
1960 — Land & Doig	Bài tối ưu công nghiệp có biến nguyên, không giải được bằng quy hoạch tuyến tính.	Nhánh cận: dùng cận nói lòng để cắt nhánh; nền của các bộ giải quy hoạch nguyên ngày nay.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1962 — Davis, Putnam, Logemann, Loveland	Chứng minh tự động các mệnh đề logic.	DPLL: quay lui + lan truyền ràng buộc; khung sườn vẫn dùng tới nay.
1965 — Golomb & Baumert	Nhiều người tự phát minh lại cùng các mẹo quay lui.	Khảo sát hệ thống; phá đối xứng, chọn biến ràng buộc nhất, kiểm tra sớm.
1975 — Knuth	Không biết một chương trình quay lui sẽ chạy vài giây hay vài năm.	Ước lượng kích thước cây bằng cách lấy mẫu đường ngẫu nhiên — vài giây để biết có nên chạy hay không.
Cuối thập niên 1990 — học mệnh đề từ xung đột	Bộ giải SAT bế tắc rồi lặp lại đúng sai lầm ở nhánh khác.	Học từ mỗi thất bại thành một ràng buộc mới; biến SAT thành công cụ công nghiệp cho kiểm chứng vi mạch và phân tích chương trình.

13.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Cờ vua và trò chơi	Tìm kiếm cây trò chơi với cắt tỉa alpha-beta	Alpha-beta <i>chính là</i> nhánh cận cho trò chơi hai người: cận trên và cận dưới thay cho một cận. Cùng quy tắc thứ tự: thử nước hứa hẹn trước để cắt được nhiều hơn.
Kiểm chứng phần cứng, phân tích chương trình	Bộ giải SAT/SMT	Cùng khung quay lui, cộng học từ xung đột. Bài học chuyển được: khi bế tắc, đừng chỉ lùi một bước — hãy hỏi <i>nguyên nhân</i> bế tắc và ghi lại nó để không lặp lại.
Vận trù học	Quy hoạch nguyên, nhánh cận với nới lỏng tuyến tính	Cùng cấu trúc; cận đến từ việc bỏ ràng buộc nguyên và giải bài liên tục. Ý tưởng chung rất mạnh: cận tốt thường đến từ một <i>phiên bản dễ hơn</i> của chính bài toán — đúng heuristic “bỏ bớt ràng buộc” ở A/01.
Trí tuệ nhân tạo cổ điển	Thoả mãn ràng buộc, lan truyền ràng buộc, cung nhất quán	Tầng cắt thứ nhất và thứ tư của mục 2 chính là các kỹ thuật ở đó dưới tên khác. Cùng phát hiện: chi phí lan truyền ràng buộc thường rẻ hơn nhiều so với chi phí duyệt nhánh chết.
Thị giác máy tính, SLAM	RANSAC: lấy mẫu ngẫu nhiên thay vì duyệt mọi tổ hợp	Câu trả lời khác cho cùng vấn đề “không gian quá lớn”. Khác biệt cốt lõi: RANSAC từ bỏ tính đầy đủ để đổi lấy tốc độ, còn quay lui giữ tính đầy đủ và cắt những nhánh <i>chứng minh được</i> là vô ích.
Hoá học, sinh học tính toán	Tìm cấu hình phân tử, gấp protein	Không gian cấu hình khổng lồ, cắt theo năng lượng chính là nhánh cận. Khác: hàm mục tiêu ở đó chỉ là mô hình xấp xỉ, nên “lời giải tối ưu” của bài toán không chắc là lời giải đúng của thực tại.
Mật mã	Tấn công vết cặn khoá, đánh đổi thời gian-bộ nhớ	Gặp nhau ở giữa là một kỹ thuật thám mã kinh điển; nó chính là lý do một số sơ đồ mã hoá kép không an toàn gấp đôi như trực giác gợi ý.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nó là điểm xuất phát chung của cả bảy chương còn lại. Mọi kỹ thuật sau này đều được sinh ra bằng cách hỏi “vết cặn lãng phí ở đâu”: tính lại thứ đã tính dẫn tới quy hoạch động, duyệt nhánh chắc chắn tệ dẫn tới tham lam và cắt tỉa, nhìn sai dạng bài dẫn tới mô hình hoá lại. Ngoài

ra vết cặn còn là trọng tài cho stress test, và trong nhiều bài NP-khó nó là lời giải tốt nhất đã biết.

- A2. Tầng một: mỗi cột chỉ đặt một quân, đưa 64^8 xuống $8^8 \approx 1,7 \cdot 10^7$. Tầng hai: kiểm tra xung đột hàng và đường chéo *ngay khi đặt* thay vì đặt xong mới kiểm, cắt phần lớn nhánh ở gần gốc và đưa số nút xuống cỡ vài nghìn. Tầng ba: phá đối xứng, ví dụ chỉ đặt quân đầu ở nửa trái rồi nhân đôi kết quả. Điều đáng rút ra là tầng hai — kiểm tra sớm — đóng góp lớn nhất, đúng như hình 13.1 chỉ ra.
- A3. Vì trường hợp xấu nhất hiếm khi là trường hợp bạn gặp. Dữ liệu thực tế có cấu trúc: các ràng buộc mâu thuẫn nhau sớm, các cặn chặn chặt, và phần lớn cây tìm kiếm chết ở gần gốc. Cắt tỉa khai thác đúng cấu trúc đó. Nói theo ngôn ngữ chương A/03: độ phức tạp tiệm cận mô tả trường hợp xấu nhất của *mọi* dữ liệu, còn cắt tỉa cải thiện phân phối thực tế — hai câu chuyện khác nhau và cả hai đều đúng.
- A4. Vì hàm mũ với số mũ bằng nửa là một hàm hoàn toàn khác về mặt thực tiễn: 2^{40} là 10^{12} , còn 2^{20} là 10^6 . Sáu bậc độ lớn là ranh giới giữa “không chạy nổi” và “xong tức thì”. Cái giá là bộ nhớ $O(2^{n/2})$, tức là ta đổi thời gian lấy bộ nhớ — một đánh đổi rất đáng khi bộ nhớ còn dư. Dấu hiệu nhận biết trên đề: n khoảng 30–45.
- A5. Ba tình huống. Khi bài toán NP-khó và n nhỏ — không tồn tại lời giải đa thức đã biết, nên câu hỏi chỉ còn là hằng số của hàm mũ. Khi bài toán công nghiệp có cấu trúc mà bộ giải khai thác được — SAT và quy hoạch nguyên giải những bài hàng triệu biến bằng đúng cách này. Và khi bạn cần một lời giải chắc chắn đúng làm trọng tài cho stress test (E/25). Trong cả ba, vết cặn không phải bước lùi mà là lựa chọn có cân nhắc.

NÓI LẠI THẬT ĐƠN GIẢN

Vết cặn là cách giải mà ai cũng nghĩ ra: thử hết mọi khả năng. Vấn đề duy nhất là “mọi khả năng” thường nhiều tới mức không sống nổi để thử hết.

Nhưng hãy để ý cách một người thật sự đi tìm chìa khoá trong nhà. Họ không mở lần lượt từng ngăn kéo trong toàn bộ căn nhà. Họ nghĩ: “chìa khoá không thể ở trong tủ lạnh” — và loại cả căn bếp bằng một câu. Đó chính là cắt tỉa: bỏ cả một vùng vì *chắc chắn* không có gì trong đó, chứ không phải vì lười tìm.

Ba mẹo mà người tìm chìa khoá dùng cũng chính là ba mẹo trong chương này. Một, kiểm tra sớm: vừa mở ngăn kéo đã thấy nó rỗng thì đóng lại ngay, đừng lục từng góc. Hai, đừng tìm hai lần cùng một chỗ chỉ vì bạn đi vào phòng theo hướng khác — đó là phá đối xứng. Ba, tìm chỗ khả nghi nhất trước; nếu tìm thấy sớm thì bạn khỏi phải lục phần còn lại.

Có một mẹo thứ tư rất hay dùng khi số khả năng quá lớn: chia đôi bài toán. Nếu bạn cần tìm hai món ghép lại thành một tổng cho trước, đừng thử mọi cặp. Hãy liệt kê mọi khả năng của nửa trái ra một danh sách, rồi với mỗi khả năng của nửa phải, tra xem cái bù của nó có trong danh sách không. Số việc từ “nhân” thành “cộng”, và đó là toàn bộ ý của ghép nhau ở giữa.

Cuối cùng, đừng coi thường vết cặn. Có những bài mà cả thế giới không biết cách nào tốt hơn ngoài thử hết một cách thông minh — và các phần mềm kiểm tra thiết kế vi mạch ngày nay chính là những cỗ máy vết cặn rất giỏi cắt tỉa và biết rút kinh nghiệm sau mỗi lần đâm vào ngõ cụt.

Một câu: Đừng hỏi làm sao tránh vết cặn; hãy hỏi vết cặn của tôi đang phí công ở đâu.

Chỗ không rút gọn được: Không có cách nào biết trước một hàm cặn có đủ chặt để cắt hiệu quả hay không; phải cài và đo.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vai trò của vết cặn	Điểm xuất phát của mọi kỹ thuật ở phần C; trọng tài cho stress test; và là lời giải tốt nhất đã biết cho nhiều bài NP-khó.
••• Ba dạng không gian và ngưỡng	Tập con 2^n (tới $n \approx 25$); hoán vị $n!$ (tới $n \approx 11$); gán k giá trị k^n . Ước lượng trước khi cài.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Bốn tầng cắt	Kiểm tra ràng buộc ngay khi đặt; phá đối xứng; cắt theo cặn (nhánh cặn); chọn thứ tự biến và giá trị để thất bại sớm.
●●● Vì sao “thất bại sớm” quan trọng	Giá trị của một lần cắt tỉ lệ với kích thước cây con bị loại, mà cây con giảm theo hàm mũ theo độ sâu — cắt gần gốc đáng giá gấp bội.
●●● Cắt tỉa và độ phức tạp	Không cải thiện trường hợp xấu nhất; chỉ cải thiện phân phối thực tế — nhưng thường vài bậc độ lớn.
●●● Gặp nhau ở giữa	Chia đôi, liệt kê $2^{n/2}$ mỗi nửa, ghép bằng sắp xếp hoặc băm. Dấu hiệu: n khoảng 30–45. Đổi bộ nhớ lấy thời gian.
●● Điều kiện của gặp nhau ở giữa	Hai nửa độc lập và ghép được qua một giá trị tổng hợp. Nếu nửa sau cần biết chi tiết cấu hình nửa đầu thì phải chuyển sang quy hoạch động.
●● Hai quy tắc thứ tự	Chọn biến bị ràng buộc nhiều nhất trước; thử giá trị hứa hẹn nhất trước (để cặn sớm chặt hơn).
●● Ước lượng cây quay lui của Knuth	Chạy vài trăm đường ngẫu nhiên từ gốc, nhân số nhánh dọc đường, lấy trung bình — ước lượng không chệch cho tổng số nút.
●● Học từ xung đột	Khi bế tắc, phân tích nguyên nhân và ghi lại thành một ràng buộc mới để không lặp lại ở nhánh khác. Bước tiến lớn nhất của quay lui hiện đại.
●● Alpha-beta là gì	Nhánh cặn cho trò chơi hai người: duy trì đồng thời cặn trên và cặn dưới. Cùng quy tắc thứ tự nước đi.
● Nguồn của một cặn tốt	Thường là lời giải của một <i>phiên bản dễ hơn</i> của chính bài toán (bỏ bớt ràng buộc) — đúng heuristic của A/01.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao biết chương trình quay lui của tôi sẽ chạy bao lâu trước khi chạy?

| Dùng phép ước lượng của Knuth ở mục 5: chạy vài trăm lần một đường đi ngẫu nhiên từ gốc xuống, tại mỗi nút nhân vào tích số nhánh khả dĩ, và lấy trung bình các tích. Con số đó là ước lượng không chệch cho tổng số nút của cây. Cách này tốn vài giây và cho bạn biết ngay thứ tự độ lớn — rất đáng làm trước khi ngồi chờ một chương trình có thể chạy hàng giờ.

2. Quay lui nên viết đệ quy hay dùng ngăn xếp tường minh?

| Đệ quy trong hầu hết trường hợp: nó ngắn hơn, khớp với cấu trúc bài toán, và việc “huỷ lựa chọn” xảy ra tự nhiên khi hàm trả về. Chuyển sang ngăn xếp tường minh chỉ khi độ sâu quá lớn gây tràn, hoặc khi bạn cần tạm dừng và tiếp tục việc tìm kiếm — ví dụ tìm kiếm có giới hạn thời gian trong một hệ thống thật.

3. Tôi nên phá đối xứng thế nào cho đúng, tránh làm mất nghiệm?

| Nguyên tắc: đặt một *thứ tự chuẩn tắc* và chỉ duyệt các cấu hình chuẩn tắc. Ví dụ với tập con thì ép chỉ số tăng dần; với việc chia n phần tử vào các nhóm không phân biệt thì ép phần tử đầu tiên của mỗi nhóm mới phải tăng dần. Cách kiểm tra an toàn: đếm số nghiệm với và không có phá đối xứng trên n nhỏ, rồi đối chiếu với con số kỳ vọng (có thể phải nhân lại hệ số đối xứng).

4. Nhánh cặn cần hàm cặn, mà hàm cặn lấy ở đâu ra?

| Gần như luôn từ một *phiên bản nói lỏng* của bài toán: bỏ bớt một ràng buộc rồi giải bài dễ hơn. Với bài ba lô, cặn là lời giải của phiên bản cho phép lấy phần lẻ của món đồ — giải bằng tham lam. Với bài người bán hàng, cặn có thể là tổng các cạnh nhẹ nhất tại mỗi đỉnh, hoặc trọng số cây khung nhỏ nhất. Đây là ứng dụng trực tiếp của heuristic “bỏ bớt ràng buộc” ở A/01: bài dễ hơn không chỉ giúp bạn hiểu bài, nó còn cho bạn một cặn.

5. Khi nào quay lui thua quy hoạch động và ngược lại?

| Quy hoạch động thắng khi các nhánh khác nhau dẫn tới *cùng một trạng thái* và trạng thái đó mô tả được gọn — lúc đó lưu lại kết quả tránh tính lại. Quay lui thắng khi không gian trạng thái quá lớn hoặc không nên được thành ít chiều, nhưng ràng buộc lại chặt tới mức phần lớn nhánh chết sớm. Dấu hiệu thực dụng: nếu bạn viết được một khoá trạng thái ngắn thì hãy nghĩ tới quy hoạch động (C/16); nếu khoá đó phải chứa cả cấu hình đã chọn thì quay lui có cắt tỉa thường là hướng đúng.

6. Có nên dùng ghi nhớ cho quay lui không?

| Có, và ranh giới giữa hai kỹ thuật mờ hơn người ta tưởng. Nếu nhiều nhánh dẫn tới cùng trạng thái, hãy lưu kết quả — lúc đó bạn đã đi từ quay lui sang quy hoạch động mà không đổi cấu trúc code. Điều cần cẩn thận là khoá trạng thái: nó phải *đủ* để xác định phần còn lại của bài toán. Khoá thiếu thông tin cho kết quả sai một cách im lặng, và đây là lỗi phổ biến nhất khi thêm ghi nhớ vào một hàm đệ quy đã chạy đúng.

7. Bộ giải SAT thực sự làm gì khác so với quay lui trong sách giáo khoa?

| Ba thứ, và cả ba đều là ý tưởng chuyển được. Một, lan truyền ràng buộc rất mạnh: mỗi lần gán một biến, suy ra ngay mọi hệ quả bắt buộc. Hai, học mệnh đề từ xung đột: khi bế tắc, phân tích *nguyên nhân* và ghi lại một ràng buộc mới, nhờ đó không lặp lại sai lầm ở nhánh khác. Ba, quay lui không tuần tự: thay vì lùi một bước, nhảy thẳng về mức thực sự gây ra mâu thuẫn. Ý tưởng thứ hai là thứ đáng mang theo nhất, kể cả khi bạn tự viết quay lui cho một bài cụ thể.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Sinh mọi tập con và mọi hoán vị của n phần tử	CSES “Creating Strings”, “Apple Division”[33]	Viết cả bản đệ quy lẫn bản dùng mặt nạ bit; hiểu rõ cả hai.
● Bài toán tám quân hậu	CSES “Chessboard and Queens”[33]	Đếm số nút cây với và không có kiểm tra sớm, để thấy hiệu quả bằng số liệu.
●● Tô màu đồ thị bằng k màu	Bài kinh điển	Chọn đỉnh có nhiều màu bị cấm nhất trước — quy tắc “thất bại sớm”.
●● Giải sudoku	Bài kinh điển	Kết hợp lan truyền ràng buộc với chọn ô ít khả năng nhất; so thời gian với bản không có hai mẹo đó.
●● Tập con có tổng bằng S với $n \leq 40$	Bài kinh điển	Gặp nhau ở giữa; dấu hiệu là chính giới hạn n .
●● Chia n món đồ thành hai nhóm sao cho chênh lệch nhỏ nhất	CSES “Apple Division”[33]	$n \leq 20$ nên duyệt tập con là đủ; sau đó thử lại với $n = 40$ bằng gặp nhau ở giữa.
●●● Bài toán người bán hàng với $n \leq 20$	Bài kinh điển	Quy hoạch động bitmask (C/16); rồi thử nhánh cận và so hai cách trên cùng dữ liệu.
●●● Đếm số cách đặt n quân hậu, có phá đối xứng	Bài kinh điển	Kiểm chứng bằng cách so với dãy kết quả đã biết cho n nhỏ.
●●● Ước lượng kích thước cây quay lui bằng phương pháp Knuth	Bài tập thực nghiệm	Áp dụng cho bài n quân hậu; so ước lượng với số nút thật.
●●● Cài một bộ giải SAT đơn giản kiểu DPLL	Bài tập cài đặt	Thêm lan truyền đơn vị, rồi đo lại; đây là cách rẻ nhất để hiểu vì sao lan truyền ràng buộc mạnh đến vậy.
●●● Bài toán phủ chính xác (xếp hình pentomino hoặc tương tự)	Bài kinh điển	Cấu trúc “liên kết nhảy” giúp huỷ và khôi phục lựa chọn trong $O(1)$; một bài tập cài đặt đẹp.

Kết chương và đọc tiếp

Chương này mở phần C bằng thứ mà mọi lời giải đều bắt đầu. Ba điều đáng mang theo: ước lượng không gian trước khi cài; bốn tầng cắt, trong đó “thất bại sớm” đáng giá nhất vì cắt gần gốc; và ý thức rằng vét cạn có cắt tỉa đôi khi *chính là* lời giải đúng, không phải bước lùi.

Bảy chương còn lại của phần C đều trả lời câu hỏi “vét cạn của tôi lãng phí ở đâu”. Câu trả lời đầu tiên: nếu bài toán chia được thành các phần *độc lập* thì chi phí không cộng lại mà sụp xuống theo hệ thức truy hồi — đó là chia để trị (C/14), và nó cũng là nơi ta gặp lại một ý tưởng của chương B/09: ngẫu nhiên hoá để không có trường hợp xấu.

Chia để trị (Divide and Conquer)

PHỤ THUỘC A/03 cho hệ thức truy hồi; A/04 cho quy nạp — chia để trị và quy nạp là hai mặt của một đồng xu; C/13 cho điểm xuất phát vết cạn.

TÓM TẮT

Chia để trị khai thác một tính chất rất cụ thể: nếu bài toán tách được thành các phần *độc lập* thì tổng chi phí không cộng lại mà sụp xuống theo hệ thức truy hồi. Chương đi qua các đại diện — sắp xếp trộn, quicksort, chọn phần tử thứ k , cặp điểm gần nhất, nhân nhanh — nhưng trọng tâm không phải danh sách thuật toán mà là ba câu hỏi thiết kế: chia ở đâu, ghép thế nào, và chi phí ghép có nhỏ hơn lợi ích chia không. Chương cũng là nơi ta gặp lại ngẫu nhiên hoá lần thứ tư, ở dạng thuyết phục nhất: quicksort với chốt ngẫu nhiên. Nếu chỉ nhớ một điều: phần khó của chia để trị nằm ở bước *ghép*, không ở bước *chia* — và mọi thuật toán hay trong chương này đều là kết quả của việc tìm ra một cách ghép rẻ bất ngờ.

KIẾN THỨC CẦN CÓ TRƯỚC

- **chia để trị** — giải bài lớn bằng cách chia thành các bài con cùng dạng nhưng nhỏ hơn, giải chúng độc lập, rồi ghép kết quả.
- **bước ghép (combine)** — công đoạn tạo lời giải của bài lớn từ lời giải các bài con; đây là nơi chứa gần như toàn bộ độ khó.
- **hệ thức truy hồi** — $T(n) = aT(n/b) + f(n)$; a là số bài con, n/b là kích thước mỗi bài con, $f(n)$ là chi phí chia và ghép. Xem A/03.
- **sắp xếp trộn (merge sort)** — chia đôi, sắp mỗi nửa, rồi trộn hai nửa đã sắp trong $O(n)$. Ổn định, $O(n \log n)$ đảm bảo, nhưng cần bộ nhớ phụ.
- **quicksort** — chọn một chốt, phân hoạch mảng thành nhỏ hơn và lớn hơn, rồi đệ quy hai phía. Không cần bộ nhớ phụ, nhanh trong thực tế, nhưng có trường hợp xấu $O(n^2)$.
- **tính ổn định của sắp xếp** — các phần tử bằng nhau giữ nguyên thứ tự tương đối ban đầu. Quan trọng khi sắp nhiều lần theo nhiều khoá.

CÂU HỎI MỞ ĐẦU

- Q1. Chia đôi rồi giải hai nửa — nghe như vẫn phải làm hết công việc. Vậy tiết kiệm đến từ đâu?
- Q2. Vì sao quicksort thường nhanh hơn sắp xếp trộn trên máy thật, dù nó có trường hợp xấu $O(n^2)$ còn sắp xếp trộn thì không?
- Q3. Bài toán tìm cặp điểm gần nhất trong mặt phẳng: sau khi chia đôi và giải hai nửa, vì sao bước ghép lại *không* tốn $O(n^2)$?
- Q4. Nhân hai số n chữ số theo cách học ở trường tốn $O(n^2)$. Karatsuba xuống $O(n^{1,585})$ chỉ bằng một mẹo đại số. Mẹo đó thay đổi cái gì trong hệ thức truy hồi?
- Q5. Khi nào chia để trị là lựa chọn sai — tức là chia bài ra làm mọi thứ tệ hơn?

Chương trước kết thúc bằng câu hỏi “vết cạn lãng phí ở đâu”. Một câu trả lời rất phổ biến: vết cạn xét mọi cặp phần tử, trong khi phần lớn các cặp đó không liên quan gì tới nhau. Nếu ta chia

dữ liệu thành hai nửa và chứng minh được rằng phần lớn công việc chỉ xảy ra *bên trong* mỗi nửa, thì số cặp phải xét giảm mạnh, và giảm lặp đi lặp lại ở mỗi mức của đệ quy.

Đó là toàn bộ ý tưởng chia để trị, và nó có một đặc điểm dễ bị hiểu sai. Bước “chia” hầu như luôn tầm thường — cắt mảng làm đôi thì có gì khó. Cái khó, và cái tạo ra mọi thuật toán đẹp trong chương này, nằm ở bước *ghép*: làm sao dựng lời giải của bài lớn từ lời giải hai nửa mà không tốn lại $O(n^2)$. Khi bạn đọc mỗi thuật toán dưới đây, hãy đọc nó như câu trả lời cho câu hỏi ghép.

14.1 Ba câu hỏi thiết kế và hệ thức truy hồi

Khuôn chung: chia bài kích thước n thành a bài con kích thước n/b , giải đệ quy, rồi ghép với chi phí $f(n)$. Chi phí tổng là $T(n) = aT(n/b) + f(n)$, và chương A/03 đã cho cách đọc nó bằng hình ảnh “tầng nào nặng”.

Điều đáng rút ra từ hình đó, và nó định hướng toàn bộ việc thiết kế: *nếu chi phí ghép $f(n)$ đủ nhỏ thì tổng chi phí bị chi phối bởi các lá; nếu $f(n)$ lớn thì bị chi phối bởi gốc*. Với sắp xếp trộn, $a = 2$, $b = 2$, $f(n) = n$ — rơi đúng vào trường hợp cân bằng, cho $\Theta(n \log n)$. Thừa số $\log n$ chính là số tầng, và nó là cái giá phải trả cho việc bước ghép tốn tuyến tính.

Từ đó có ba câu hỏi thiết kế cần đặt mỗi khi định dùng chia để trị:

Chia ở đâu? Chia đôi đều là mặc định, nhưng không phải luôn đúng. Quicksort chia theo giá trị chốt nên hai phần có thể lệch; thuật toán chọn phần tử thứ k chỉ đệ quy *một* nhánh; thuật toán cặp điểm gần nhất chia theo toạ độ.

Các bài con có thật sự độc lập không? Nếu lời giải của nửa này ảnh hưởng tới nửa kia thì chia để trị không áp dụng trực tiếp, và bạn đang ở địa hạt quy hoạch động (C/16).

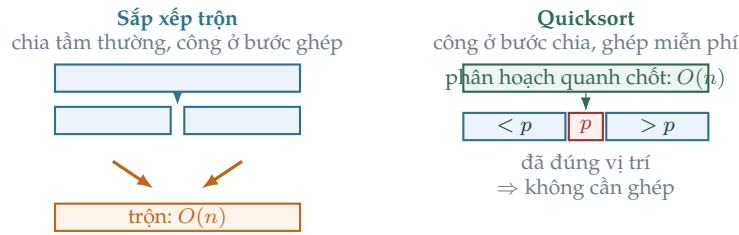
Ghép có rẻ hơn không? Nếu ghép vẫn tốn $O(n^2)$ thì chẳng tiết kiệm gì. Câu hỏi này là chỗ đòi hỏi sáng tạo thật sự, và ba ví dụ dưới đây là ba câu trả lời đẹp.

14.2 Sắp xếp: hai triết lý ngược nhau

Sắp xếp trộn và quicksort đều là chia để trị, nhưng chúng đặt công việc ở hai chỗ ngược nhau — và so sánh đó dạy nhiều hơn bản thân hai thuật toán.

Sắp xếp trộn: chia dễ, ghép khó. Chia là cắt đôi ở giữa, không cần suy nghĩ. Toàn bộ công việc nằm ở bước trộn hai dãy đã sắp, tốn $O(n)$. Kết quả: $O(n \log n)$ *đảm bảo* cho mọi dữ liệu, ổn định, và trộn là thao tác quét tuần tự nên rất hợp cho dữ liệu lớn hơn bộ nhớ (D/24). Cái giá: cần $O(n)$ bộ nhớ phụ.

Quicksort: chia khó, ghép miễn phí. Toàn bộ công việc nằm ở bước phân hoạch — sắp xếp lại mảng sao cho mọi phần tử nhỏ hơn chốt nằm bên trái. Sau khi hai nửa được sắp xong thì không cần ghép gì cả, vì chúng đã ở đúng vị trí. Kết quả: không cần bộ nhớ phụ, truy cập tuần tự nên rất hợp cache, và trong thực tế thường nhanh hơn sắp xếp trộn vài lần. Cái giá: nếu chốt chọn tệ thì hai phần lệch hẳn và chi phí thành $O(n^2)$.



Hình 14.1. Hai cách phân bổ công việc trong chia để trị. Bài học tổng quát: công việc phải nằm ở đâu đó — hoặc lúc chia, hoặc lúc ghép. Khi thiết kế thuật toán chia để trị cho bài mới, hãy thử cả hai hướng: đôi khi chuyển công việc từ ghép sang chia làm mọi thứ đơn giản hẳn.

Chốt ngẫu nhiên: lần thứ tư của cùng một ý tưởng

Trường hợp xấu của quicksort không hiếm: mảng đã sắp sẵn cộng với việc chọn chốt là phần tử đầu tiên cho ngay $O(n^2)$. Và như ở chương B/08 và B/09, cái nguy hiểm là dữ liệu thực tế *rất hay* đã sắp sẵn một phần.

Cách chữa lại là ý tưởng cũ: *chọn chốt ngẫu nhiên*. Khi đó không có dữ liệu vào nào là “xấu”, vì hành vi của thuật toán không còn phụ thuộc thứ tự dữ liệu mà phụ thuộc các lựa chọn ngẫu nhiên của chính nó. Kỳ vọng chi phí là $\Theta(n \log n)$ với *mọi* dữ liệu vào, và xác suất tệ hơn nhiều lần giảm theo hàm mũ.

Đây là lần thứ tư nguyên lý này xuất hiện — băm phổ dụng, treap, cắt tia qua lấy mẫu, và bây giờ là quicksort. Nếu bạn thấy nó lặp lại thì đó là dấu hiệu nên đọc kỹ chương C/20, nơi nó được phát biểu thành nguyên lý chung.

Chọn phần tử thứ k là biến thể quan trọng của quicksort: sau khi phân hoạch, ta biết chốt đứng ở vị trí nào, nên chỉ cần đệ quy *một* nửa. Chi phí kỳ vọng trở thành $\Theta(n)$ chứ không phải $\Theta(n \log n)$, vì $n + n/2 + n/4 + \dots < 2n$ — lại là chuỗi hình học của chương A/05. Có cả phiên bản tất định đảm bảo $O(n)$ trong trường hợp xấu nhất, dựa trên ý tưởng “trung vị của các trung vị”[34], nhưng hằng số lớn nên trong thực tế bản ngẫu nhiên thường thắng.

14.3 Khi bước ghép đòi hỏi một ý tưởng: cặp điểm gần nhất

Cho n điểm trên mặt phẳng, tìm hai điểm gần nhau nhất. Vết cặn $O(n^2)$.

Chia để trị: sắp các điểm theo hoành độ, chia làm hai nửa bằng một đường thẳng đứng, giải đệ quy mỗi nửa, được khoảng cách nhỏ nhất d trong hai nửa. Bước ghép phải xét các cặp *vắt qua* đường chia — và nếu làm ngây thơ thì đó vẫn là $O(n^2)$.

Ý tưởng cứu bài toán: chỉ những điểm nằm trong dải rộng $2d$ quanh đường chia mới có cơ hội, và trong dải đó, nếu ta sắp theo tung độ thì *mỗi điểm chỉ cần so với một số lượng điểm bị chặn bởi hằng số ở phía sau nó*. Lý do hình học: trong một hình chữ nhật $2d \times d$ không thể có quá một số hằng định điểm mà đôi một cách nhau ít nhất d . Vậy bước ghép là $O(n)$, và tổng chi phí là $O(n \log n)$.

Đây là ví dụ mẫu mực cho luận điểm mở đầu: bước chia tầm thường, còn cái làm nên thuật toán là một quan sát hình học cho phép bước ghép rẻ đi. Khi bạn thiết kế một thuật toán chia để trị mới, đây chính là chỗ phải tìm.

14.4 Nhân nhanh: giảm số bài con thay vì giảm chi phí ghép

Nhân hai số n chữ số theo cách trường học tốn $O(n^2)$. Chia để trị ngây thơ: tách mỗi số thành hai nửa, và tích trở thành bốn phép nhân nửa kích thước cộng vài phép cộng. Truy hồi $T(n) = 4T(n/2) + O(n)$ cho... vẫn $O(n^2)$. Không tiết kiệm gì.

Karatsuba nhận ra rằng *bốn* phép nhân là thừa: ba là đủ, vì tích chéo tính được từ ba tích kia bằng phép cộng trừ. Truy hồi thành $T(n) = 3T(n/2) + O(n)$, cho $O(n^{\log_2 3}) = O(n^{1,585})$.

Ngây thơ $a_1 b_1$ $a_1 b_2$ $a_2 b_1$ $a_2 b_2$ $4T(n/2) \Rightarrow \Theta(n^2)$

Karatsuba $a_1 b_1$ $a_2 b_2$ $(a_1 + a_2)(b_1 + b_2)$ $3T(n/2) \Rightarrow \Theta(n^{1,585})$

Tích chéo $a_1 b_2 + a_2 b_1$ suy được từ ba tích kia bằng cộng trừ, nên phép nhân thứ tư là thừa.

Hình 14.2. Hai chiến lược tăng tốc trong chia để trị. Bài cặp điểm gần nhất thắng bằng cách làm *bước ghép* rẻ đi; Karatsuba thắng bằng cách *giảm số bài con*. Vì số bài con nằm ở số mũ của lời giải còn chi phí ghép chỉ cộng thêm, chiến lược thứ hai đổi hẳn bậc.

Điểm đáng học ở đây khác hẳn với mục trước, và đó là lý do tôi đặt hai ví dụ cạnh nhau: ở bài cặp điểm gần nhất, ta thắng bằng cách làm *bước ghép* rẻ đi; ở đây, ta thắng bằng cách *giảm số bài con*. Nhìn vào hệ thức truy hồi thì thấy rõ: một bên giảm $f(n)$, một bên giảm a . Và vì a nằm ở số mũ còn $f(n)$ chỉ là cộng thêm, việc giảm a có tác động sâu hơn — nó đổi *bậc* của lời giải.

Cùng ý tưởng cho phép nhân ma trận: Strassen[55] giảm từ 8 xuống 7 phép nhân ma trận con, cho $O(n^{\log_2 7}) \approx O(n^{2,81})$. Và cùng ý tưởng, đẩy tới cùng, cho biến đổi Fourier nhanh với $O(n \log n)$ — công cụ ở chương D/21.

CẠM BÃY

Các thuật toán nhân nhanh có ngưỡng hoà vốn thật sự. Karatsuba chỉ thắng cách trường học khi số đủ lớn — thường vài trăm tới vài nghìn chữ số, tùy cài đặt — vì nó tốn thêm nhiều phép cộng và nhiều lời gọi đệ quy. Strassen còn tệ hơn về mặt này: hằng số lớn, kém ổn định số học, và không thân thiện cache, nên các thư viện đại số tuyến tính chỉ dùng nó ở kích thước rất lớn nếu có dùng. Đây là ví dụ tiêu biểu cho lời cảnh báo ở chương A/03: bậc tiệm cận nhỏ hơn không tự động nghĩa là nhanh hơn.

14.5 Chia để trị ở những nơi không ngờ

Ba biến thể đáng biết vì chúng mở rộng phạm vi áp dụng.

Đếm trong lúc trộn. Bài đếm số nghịch thế giải được bằng cách sửa sắp xếp trộn: khi trộn, mỗi lần lấy một phần tử từ nửa phải trước một phần tử của nửa trái thì ta vừa phát hiện một loạt nghịch thế và đếm được ngay. Mẫu tổng quát: *nếu bài toán đếm các cặp có quan hệ nào đó, hãy thử đếm chúng trong lúc trộn*. Cùng mẫu giải bài đếm cặp có tổng nhỏ hơn ngưỡng, hay đếm cặp đảo ngược có trọng số.

Chia để trị trên trục thời gian. Nếu các đối tượng chỉ “tồn tại” trong một khoảng thời gian, hãy xây một cây phân đoạn trên trục thời gian, đặt mỗi đối tượng vào $O(\log n)$ nút, rồi duyệt sâu cây đó với một cấu trúc có hoàn tác. Đây chính là kỹ thuật đã nhắc ở B/11 cho bài liên thông động ngoại tuyến.

Tối ưu hoá chia để trị cho quy hoạch động. Khi điểm chuyển tối ưu có tính đơn điệu, ta tính các trạng thái theo kiểu chia để trị và giảm được một thừa số n . Chi tiết ở chương C/16; điều đáng nhớ ở đây là chia để trị không chỉ là cách tổ chức thuật toán mà còn là cách tổ chức việc tính toán bên trong một thuật toán khác.

14.6 Biên giới hiện nay: $O(n \log n)$ cho nhân số, và số mũ của nhân ma trận

Câu chuyện Karatsuba không dừng ở năm 1960, và phần tiếp theo của nó dạy một điều mà chương A/03 mới chỉ nói bằng lời cảnh báo: một cận có thể vừa đúng vừa vô dụng.

Sau Karatsuba, Toom và Cook tổng quát hoá mẹo ấy thành “chia thành k mảnh, dùng $2k - 1$ phép nhân”, đẩy số mũ xuống gần 1 nhưng không bao giờ tới. Bước nhảy thật sự đến khi người ta bỏ hẳn cách nhìn chia để trị thô và chuyển sang miền tần số: nhân hai đa thức là tích chập, mà tích chập trở thành phép nhân từng điểm sau biến đổi Fourier (D/21). Schönhage và Strassen dùng đúng ý đó năm 1971 để đạt $O(n \log n \log \log n)$, và cùng lúc phỏng đoán rằng đáp án cuối cùng phải là $O(n \log n)$.

Phỏng đoán ấy được chứng minh năm 2021 bởi Harvey và van der Hoeven[56], đăng trên *Annals of Mathematics* — một tạp chí toán học chứ không phải kỷ yếu tin học, điều đã nói lên tính chất của kết quả. Nhưng đây mới là phần đáng học: thuật toán của họ chỉ thắng Schönhage–Strassen từ những kích thước lớn hơn mọi thứ có thể xuất hiện trong máy tính thật. Nó là một thuật toán thiên hà: đúng, đẹp, giải quyết một câu hỏi mở nửa thế kỷ, và không bao giờ được cài vào thư viện nào. Giá trị của nó là chốt lại hình dạng của câu trả lời, chứ không phải làm chương trình của ai đó nhanh lên. Và chú ý sự bất đối xứng quen thuộc: chưa ai chứng minh được cận dưới $\Omega(n \log n)$ tương ứng, nên khả năng còn nhanh hơn nữa vẫn chưa bị loại trừ (D/24).

Nhân ma trận có cùng cấu trúc câu chuyện nhưng chưa có hồi kết. Strassen[55] mở màn với $O(n^{2.81})$; nửa thế kỷ tiếp theo là một chuỗi cải tiến bằng những kỹ thuật đại số ngày càng phức tạp, tới $\omega < 2.3729$ của Alman và Vassilevska Williams[57], và các bản tiền ấn tới năm 2026 hạ tiếp xuống quanh 2,3712[58] — những con số này chưa qua phản biện nên hãy đọc chúng như tin tức, không như kiến thức đã ổn định. Cận dưới hiển nhiên là $\omega \geq 2$, vì chỉ riêng việc đọc dữ liệu vào đã tốn n^2 ; không ai biết $\omega = 2$ hay không, và đó là một trong những câu hỏi mở lớn nhất của đại số tính toán.

Phần thực hành thì gọn và ngược hẳn với vẻ hào nhoáng ở trên: mọi thuật toán sau Strassen đều thiên hà, và ngay cả Strassen cũng chỉ được dùng ở kích thước lớn vì hằng số, độ ổn định số học và tính thân thiện cache đều kém hơn thuật toán ba vòng lặp đã tối ưu theo khối. Đây là lời nhắc cụ thể nhất mà tôi biết cho luận điểm của chương A/03: bậc tiệm cận là một trong các thước đo, không phải thước đo duy nhất, và giữa “tồn tại thuật toán với số mũ nhỏ hơn” và “chương trình của tôi chạy nhanh hơn” là cả một khoảng cách có thể đo bằng thập kỷ.

14.7 Gốc rễ: một mẹo đại số phá vỡ một phỏng đoán

Sắp xếp trộn xuất hiện rất sớm — nó được ghi nhận trong một bản mô tả của von Neumann từ năm 1945, thuộc những thuật toán đầu tiên được viết cho máy tính có chương trình lưu trữ. Điều đó không ngẫu nhiên: máy tính thời đó dùng băng từ, và trộn hai dãy đã sắp là thao tác gần như duy nhất mà băng từ làm tốt — đọc tuần tự, không nhảy. Nói cách khác, thuật toán sinh ra từ ràng buộc của phần cứng, đúng như câu chuyện B-cây ở chương B/09.

Quicksort có một nguồn gốc rất cụ thể và rất đời. Tony Hoare nghĩ ra nó khoảng năm 1960 khi đang làm việc về dịch máy Nga–Anh: ông cần sắp xếp các từ của một câu để tra chúng trong

từ điển lưu trên băng từ một cách hiệu quả. Ràng buộc bộ nhớ thời đó rất chặt, và đó là lý do quicksort được thiết kế để sắp xếp *tại chỗ* — một quyết định tình cờ hoá ra lại là lý do nó vẫn thắng trên phần cứng hiện đại, nơi việc tránh cấp phát và truy cập tuần tự quan trọng hơn bao giờ hết.

Câu chuyện đáng nhớ nhất trong chương này thuộc về Karatsuba, và nó là một bài học về giá trị của việc nghi ngờ. Năm 1960, Kolmogorov — một trong những nhà toán học lớn nhất thế kỷ — phát biểu tại một buổi seminar phỏng đoán rằng nhân hai số n chữ số bất buộc cần $\Omega(n^2)$ phép toán. Karatsuba, khi đó là một sinh viên, tìm ra thuật toán ba phép nhân chỉ trong khoảng một tuần, bác bỏ phỏng đoán. Điều đáng học không phải mẹo đại số mà là bối cảnh: một phát biểu “chắc chắn phải cần n^2 ” được đưa ra bởi người có uy quyền lớn nhất, và nó sai — vì trực giác về cận dưới rất khó tin cậy khi chưa có chứng minh. Chương D/24 sẽ nói vì sao chứng minh cận dưới lại khó tới vậy.

Cuối cùng, biến đổi Fourier nhanh có một gốc rễ gây bất ngờ: thuật toán mà Cooley và Tukey công bố năm 1965[59] về sau được phát hiện là đã có mặt trong các ghi chép chưa xuất bản của Gauss từ năm 1805, viết bằng tiếng Latin và bị bỏ quên. Bối cảnh thúc đẩy việc tái phát minh năm 1965 rất cụ thể: nhu cầu phát hiện các vụ thử hạt nhân qua tín hiệu địa chấn, một bài toán xử lý tín hiệu quy mô lớn.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1805 — Gauss (không xuất bản)	Nội suy quỹ đạo thiên thể từ các quan sát rời rạc.	Ý tưởng của biến đổi Fourier nhanh, bị bỏ quên trong ghi chép suốt 160 năm.
1945 — von Neumann	Sắp xếp dữ liệu trên băng từ, nơi chỉ đọc tuần tự được.	Sắp xếp trộn — một trong những thuật toán đầu tiên viết cho máy có chương trình lưu trữ; ràng buộc phần cứng sinh ra thuật toán.
1960 — Hoare	Sắp xếp các từ của câu để tra từ điển trên băng, với bộ nhớ rất hạn chế.	Quicksort: sắp tại chỗ. Ràng buộc bộ nhớ thời đó tình cờ tạo ra ưu thế cache của ngày nay.
1960 — Karatsuba	Kolmogorov phỏng đoán nhân số cần $\Omega(n^2)$.	Ba phép nhân thay vì bốn, cho $O(n^{1,585})$; phỏng đoán bị bác bỏ trong một tuần. Bài học: trực giác về cận dưới rất dễ sai.
1965 — Cooley & Tukey	Xử lý tín hiệu địa chấn để phát hiện thử hạt nhân.	FFT $O(n \log n)$; sau đó phát hiện ra Gauss đã có ý tưởng từ 1805.
1969 — Strassen	Nhân ma trận có bất buộc $\Omega(n^3)$ không?	Bảy phép nhân thay vì tám, cho $O(n^{2,81})$; mở ra cả một nhánh nghiên cứu về số mũ nhân ma trận vẫn đang tiếp diễn.

■ 14.8 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Tính toán song song	MapReduce, phép rút gọn theo cây	Chia để trị là cấu trúc song song lý tưởng vì các bài con độc lập nên chạy đồng thời được. Điều kiện cũng giống hệt: phép ghép phải kết hợp — cùng điều kiện với cây phân đoạn ở B/10.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Xử lý ảnh	Kim tự tháp ảnh, biến đổi wavelet, phương pháp đa lưới	Cùng cấu trúc đệ quy chia đôi độ phân giải. Bài học chung: nhiều bài toán dễ hơn hẳn khi giải ở độ phân giải thô rồi tinh chỉnh dần — một chiến lược mà chia để trị thuần túy không có nhưng rất mạnh trong thực tế.
Đại số tuyến tính số	Phân rã khối, nhân ma trận theo khối	Chia theo khối không phải để giảm bậc mà để <i>hợp cache</i> — mỗi khối vừa trong bộ nhớ nhanh. Đây là chia để trị phục vụ mô hình chi phí bộ nhớ chứ không phục vụ hệ thức truy hồi (D/24).
Xử lý tín hiệu	FFT, tích chập nhanh	Ứng dụng nổi tiếng nhất của chia để trị. Ý tưởng chuyển miền — việc khó ở miền này thành việc dễ ở miền kia — cũng chính là mẫu “tổng tiền tố và mảng hiệu” ở B/06, phóng to lên.
Ước lượng trạng thái, SLAM	Chia bản đồ thành các bản đồ con rồi ghép	Cùng khuôn: giải cục bộ rồi ghép. Điểm khác quyết định là bước ghép ở đây <i>không</i> chính xác — các bản đồ con có sai số tương quan — nên việc ghép ngay thơ tạo ra sai lệch hệ thống. Đây là ví dụ cho câu hỏi thứ hai ở mục 1: các bài con có thật sự độc lập không.
Quản lý dự án	Chia nhỏ công việc	Cùng trực giác, nhưng đáng nêu vì lý do thất bại giống nhau: nếu các phần việc phụ thuộc lẫn nhau thì chi phí “ghép” (phối hợp) tăng nhanh hơn lợi ích của việc chia — đúng trường hợp $f(n)$ lớn trong hệ thức truy hồi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ việc phân lớn công việc trong lời giải vét cạn là so sánh những cặp *vắt qua* ranh giới, và chia để trị làm cho số cặp như vậy nhỏ đi rất nhiều. Nói bằng hệ thức truy hồi: nếu chi phí ghép $f(n)$ nhỏ hơn hẳn $n^{\log_b a}$ thì tổng bị chi phối bởi lá, còn nếu bằng nhau thì ta chỉ trả thêm một thừa số $\log n$. So với $O(n^2)$ thì cả hai kết cục đều tốt hơn nhiều bậc. Nếu bước ghép vẫn tốn $O(n^2)$ thì chia để trị không tiết kiệm gì — đó là lý do bước ghép mới là phần khó.
- A2. Vì bậc tiệm cận không nói hết. Quicksort sắp tại chỗ nên không có chi phí cấp phát và không chép dữ liệu qua lại giữa hai mảng; phân hoạch là một lượt quét gần như tuần tự nên rất hợp cache; và vòng lặp trong của nó rất ngắn. Sắp xếp trộn phải ghi ra mảng phụ rồi chép về, tức là chạm mỗi phần tử nhiều lần hơn. Trường hợp xấu $O(n^2)$ thì được xử lý bằng chốt ngẫu nhiên, hoặc bằng cách chuyển sang heapsort khi độ sâu vượt ngưỡng[37].
- A3. Vì một quan sát hình học: chỉ những điểm nằm trong dải rộng $2d$ quanh đường chia mới có thể tạo cặp gần hơn d , và trong dải đó, khi các điểm được sắp theo tung độ thì mỗi điểm chỉ cần so với một số hằng định điểm kế tiếp. Lý do là trong một hình chữ nhật $2d \times d$ không thể nhét quá một số hằng định điểm đôi một cách nhau ít nhất d . Nhờ vậy bước ghép là $O(n)$ thay vì $O(n^2)$.
- A4. Nó giảm số *bài con* a từ 4 xuống 3, chứ không giảm chi phí ghép $f(n)$. Điều này quan trọng vì a nằm ở số mũ của lời giải — $T = \Theta(n^{\log_b a})$ — nên giảm a làm đổi *bậc*, trong khi giảm $f(n)$ chỉ ảnh hưởng khi f đang là thành phần chi phối. Đây là hai chiến lược khác nhau của chia để trị, và bài cặp điểm gần nhất minh họa chiến lược còn lại.
- A5. Ba tình huống. Khi các bài con *không độc lập* — lời giải nửa này ảnh hưởng nửa kia — thì phải chuyển sang quy hoạch động. Khi bước ghép đặt ngang bài gốc, ví dụ vẫn phải xét mọi cặp *vắt ngang*. Và khi n nhỏ, vì chi phí gọi đệ quy và các hằng số phụ vượt quá lợi ích — đó là lý do mọi thư viện sắp xếp thật đều chuyển sang sắp xếp chèn khi đoạn còn dưới vài chục phần tử[14].

NÓI LẠI THẬT ĐƠN GIẢN

Chia để trị là cách bạn xử lý một chồng bài kiểm tra cần chấm: thay vì tự chấm hết, bạn chia đôi chồng bài cho hai người, mỗi người lại chia đôi cho hai người nữa, tới khi mỗi người chỉ còn vài bài. Việc chấm xong rất nhanh. Nhưng có một khâu dễ bị quên: sau khi chấm xong phải *gộp kết quả lại*, và nếu khâu gộp cũng mệt như chấm thì chia ra chẳng lợi gì.

Đó là ý chính của cả chương: chia thì dễ, gộp mới khó, và mọi thuật toán hay ở đây đều là kết quả của việc tìm ra một cách gộp rẻ bất ngờ.

Hai cách sắp xếp kinh điển cho thấy hai kiểu phân bổ công việc. Sắp xếp trộn giống việc bạn cứ chia bữa đôi chồng bài, rồi khi gộp thì phải so từng cặp để trộn hai chồng đã sắp thành một — công việc dồn vào lúc gộp. Quicksort thì ngược: bạn chọn một bài làm mốc và ngay từ đầu chia thành “kém hơn mốc” và “hơn mốc”; sau khi hai bên được sắp xong thì chỉ việc đặt cạnh nhau, không phải gộp gì cả. Công việc dồn vào lúc chia.

Còn mẹo của Karatsuba thì thuộc loại khác hẳn và rất đáng nhớ. Khi nhân hai số lớn, cách chia tự nhiên đòi bốn phép nhân nhỏ. Ông nhận ra chỉ cần ba, vì cái thứ tư suy được từ ba cái kia bằng cộng trừ. Nghe như một mẹo vặt, nhưng vì nó lặp lại ở mọi tầng của đệ quy nên tiết kiệm nhân lên theo cấp số nhân. Đó cũng là bài học chung: trong chia để trị, giảm *số bài con* có tác động sâu hơn nhiều so với làm cho mỗi bài con nhẹ đi một chút.

Một câu: Chia thì dễ; cả nghệ thuật nằm ở chỗ ghép hai nửa lại mà không phải làm lại từ đầu.

Chỗ không rút gọn được: Không có quy trình chung để tìm ra một bước ghép rẻ; nó thường đến từ một quan sát riêng của bài toán, như quan sát hình học trong bài cặp điểm gần nhất.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Ba câu hỏi thiết kế	Chia ở đâu; các bài con có độc lập không; bước ghép có rẻ hơn bài gốc không. Câu thứ ba chứa gần hết độ khó.
●●● Đọc hệ thức truy hồi	$T(n) = aT(n/b) + f(n)$; so $f(n)$ với $n^{\log_b a}$ để biết tầng nào chi phối (<i>A/03</i>).
●●● Sắp xếp trộn so với quicksort	Trộn: chia tầm thường, ghép $O(n)$, đảm bảo $O(n \log n)$, ổn định, cần bộ nhớ phụ. Quicksort: công ở bước phân hoạch, ghép miễn phí, tại chỗ, hợp cache, xấu nhất $O(n^2)$.
●●● Chốt ngẫu nhiên	Làm hành vi phụ thuộc lựa chọn của thuật toán thay vì thứ tự dữ liệu; kỳ vọng $\Theta(n \log n)$ với mọi dữ liệu vào.
●●● Chọn phần tử thứ k	Chỉ đệ quy một nửa sau khi phân hoạch; $n + n/2 + n/4 + \dots < 2n$ cho kỳ vọng $\Theta(n)$.
●●● Hai chiến lược tăng tốc	Giảm chi phí ghép $f(n)$ (cặp điểm gần nhất) hoặc giảm số bài con a (Karatsuba, Strassen). Giảm a đôi <i>bạc</i> vì a nằm ở số mũ.
●● Bước ghép của cặp điểm gần nhất	Chỉ xét dải rộng $2d$ quanh đường chia, sắp theo tung độ, mỗi điểm so với một số hằng định điểm kế tiếp $\Rightarrow O(n)$.
●● Karatsuba	Ba phép nhân thay vì bốn: $T(n) = 3T(n/2) + O(n)$, cho $O(n^{\log_2 3})$. Có ngưỡng hoà vốn thật, chỉ thắng khi số đủ lớn.
●● Đếm trong lúc trộn	Đếm nghịch thế bằng sắp xếp trộn; mẫu tổng quát cho các bài “đếm cặp có quan hệ”.
●● Chia để trị trên trục thời gian	Cây phân đoạn trên trục thời gian + cấu trúc có hoàn tác; giải bài liên thông động ngoại tuyến (<i>B/11</i>).
●● Khi nào chia để trị sai	Bài con phụ thuộc nhau ($\rightarrow$ quy hoạch động); bước ghép đắt ngang bài gốc; hoặc n nhỏ nên chi phí đệ quy vượt lợi ích.
● Bài học Karatsuba–Kolmogorov	Một phỏng đoán cận dưới của nhà toán học lớn bị bác bỏ trong một tuần — trực giác về cận dưới rất dễ sai (<i>D/24</i>).
● Biên giới của nhân nhanh	Nhân số nguyên đã có $O(n \log n)$ (2021) nhưng là thuật toán thiên hà; nhân ma trận dừng ở $\omega < 2,3729$ và câu hỏi $\omega = 2$ vẫn mở. Thực hành chỉ dùng tới Karatsuba và Strassen.

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Vì sao các thư viện sắp xếp thật không dùng thuần một thuật toán nào?

| Vì mỗi thuật toán mạnh ở một vùng khác nhau. Cách làm chuẩn — introsort — là bắt đầu bằng quicksort vì nó nhanh nhất trong trường hợp thường; chuyển sang sắp xếp chèn khi đoạn còn nhỏ vì lúc đó hằng số quan trọng hơn bậc; và chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng, để chặn trường hợp xấu $O(n^2)$ [37]. Ngoài ra còn phải xử lý dữ liệu có nhiều phần tử bằng nhau bằng phân hoạch ba phần [14]. Đây là một trong những ví dụ rõ nhất cho chương E/26.

2. Sắp xếp ổn định có quan trọng không, hay chỉ là chi tiết lý thuyết?

| Rất quan trọng trong thực tế, vì nó cho phép sắp nhiều lần theo nhiều khoá: sắp theo khoá phụ trước, rồi theo khoá chính, và các phần tử bằng khoá chính vẫn giữ đúng thứ tự theo khoá phụ. Đây là cách các bảng dữ liệu và giao diện người dùng làm việc. Sắp xếp trộn ổn định tự nhiên; quicksort thì không, và làm nó ổn định đòi bộ nhớ phụ — một lý do nữa vì sao cả hai cùng tồn tại.

3. Có phải mọi thuật toán chia để trị đều song song hoá được không?

| Về nguyên tắc thì các bài con độc lập nên chạy song song được, nhưng có ba điều kiện thực tế. Bước ghép phải kết hợp và rẻ, nếu không nó thành nút cổ chai. Kích thước bài con phải đủ lớn để bù chi phí tạo luồng — nên trong thực tế người ta chỉ song song hoá vài mức trên cùng rồi chuyển sang tuần tự. Và dữ liệu phải chia được mà không tranh chấp bộ nhớ. Sắp xếp trộn song song hoá tốt hơn quicksort vì bước phân hoạch của quicksort khó song song.

4. Tôi có nên tự cài quicksort trong thi đấu không?

| Không, trừ khi bài yêu cầu một biến thể đặc biệt như chọn phần tử thứ k hoặc phân hoạch ba phần. Thư viện chuẩn đã được kỹ thuật hoá kỹ hơn nhiều so với những gì bạn viết trong năm phút. Điều cần biết là *cạm bẫy*: trong một số ngôn ngữ, hàm sắp xếp chuẩn có trường hợp xấu bị khai thác được, nên khi nghi ngờ hãy trộn ngẫu nhiên mảng trước khi sắp — một dòng code, và nó áp dụng đúng nguyên lý ngẫu nhiên hoá của chương này.

5. Khi nào nên dùng đếm trong lúc trộn thay vì Fenwick cho bài đếm nghịch thế?

| Cả hai đều $O(n \log n)$. Fenwick ngắn hơn nếu bạn đã có sẵn trong thư viện, nhưng cần nén toạ độ. Đếm trong lúc trộn không cần nén và tổng quát hoá tự nhiên sang các quan hệ phức tạp hơn — ví dụ đếm cặp thoả $a_i > 2a_j$, thứ mà Fenwick phải xử lý khéo hơn. Lời khuyên: biết cả hai, và chọn theo việc quan hệ cần đếm có dễ biểu diễn thành “đếm phần tử đã chèn nhỏ hơn x ” hay không.

6. Chia để trị và quy nạp liên quan với nhau thế nào?

| Chúng là hai mặt của một đồng xu, và nhận ra điều đó làm việc viết code lẫn viết chứng minh dễ hơn hẳn. Thuật toán chia để trị giải bài lớn bằng bài nhỏ; chứng minh quy nạp khẳng định tính đúng cho n dựa vào tính đúng cho các giá trị nhỏ hơn. Chúng khớp nhau từng dòng: trường hợp cơ sở của đệ quy ứng với cơ sở quy nạp, và bước ghép ứng với bước quy nạp. Vì vậy nếu bạn viết được thuật toán mà không viết được chứng minh, thường là do bước ghép có lỗ hổng — xem A/04.

7. Ngưỡng chuyển sang thuật toán đơn giản nên đặt bao nhiêu?

| Không có con số phổ quát — nó phụ thuộc ngôn ngữ, kích thước phần tử và máy. Cách đúng là đo: cài cả hai, chạy trên dữ liệu thật với nhiều kích thước, và tìm điểm giao. Trong thực tế các thư viện thường dùng ngưỡng trong khoảng vài chục phần tử, nhưng đừng chép con số mà hãy chép *phương pháp*: đây là một quyết định thực nghiệm, không phải quyết định lý thuyết (E/26).

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Cài sắp xếp trộn và kiểm chứng bằng stress test	Bài tập cài đặt	Viết bất biến của bước trộn ra trước; kiểm với mảng rỗng và mảng toàn phần tử bằng nhau.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Tìm phần tử thứ k bằng phân hoạch ngẫu nhiên	Bài kinh điển	Chỉ đề quy một nửa; giải thích vì sao kỳ vọng là $\Theta(n)$ chứ không $\Theta(n \log n)$.
•• Đếm số nghịch thế bằng sắp xếp trộn	Bài kinh điển	So với cách Fenwick ở B/10; nêu điều kiện chọn mỗi cách.
•• Lũy thừa nhanh và lũy thừa ma trận	CSES “Exponentiation”[33]	Chia để trị trên số mũ; nối với A/05 và D/21.
•• Tìm cặp điểm gần nhất trong mặt phẳng	CSES “Minimum Euclidean Distance”[33]	Bước ghép là toàn bộ bài; hãy tự chứng minh cận hằng số cho số điểm phải so.
•• Tìm phần tử đa số bằng chia để trị	LeetCode “Majority Element”	So với thuật toán bỏ phiếu $O(n)$ ở A/01 — ví dụ cho thấy chia để trị không phải luôn là cách tốt nhất.
••• Cài Karatsuba cho số lớn và tìm ngưỡng hoà vốn trên máy bạn	Bài tập thực nghiệm	Vẽ đường thời gian theo số chữ số cho cả hai cách; điểm giao là câu trả lời.
••• Bài toán “skyline” — gộp đường viền các toà nhà	LeetCode “The Skyline Problem”	Chia để trị với bước ghép giống trộn; bài mẫu cho việc thiết kế phép ghép.
••• Đếm số cặp (i, j) với $i < j$ và $a_i > 2a_j$	Bài kinh điển	Biến thể của đếm trong lúc trộn; chỗ dễ sai là phải đếm trước khi trộn.
••• Nhân đa thức bằng chia để trị, rồi bằng FFT	Chương D/21	So $O(n^{1,585})$ với $O(n \log n)$ bằng số liệu thật; ngưỡng hoà vốn lại là câu hỏi thực nghiệm.
••• Chia để trị trên trục thời gian cho bài liên thông động ngoại tuyến	Bài thi đấu kinh điển	Kết hợp B/10 và B/11; đây là bài tập tổng hợp tốt nhất của phần B và C.

Kết chương và đọc tiếp

Chia để trị là kỹ thuật đầu tiên trong phần C khai thác cấu trúc *đệ quy* của bài toán. Ba điều đáng mang theo: phần khó nằm ở bước ghép; có hai chiến lược tăng tốc rất khác nhau — làm ghép rẻ đi hay giảm số bài con — và chiến lược thứ hai đôi bậc; và điều kiện tiên quyết luôn là các bài con phải *độc lập*.

Chính điều kiện cuối dẫn tới hai chương tiếp theo. Khi các lựa chọn không độc lập mà nối tiếp nhau, có hai kịch bản. Kịch bản may mắn: tồn tại một lựa chọn cục bộ luôn an toàn, và ta có thuật toán tham lam — ngăn tới bất ngờ nhưng cần chứng minh thật sự (C/15). Kịch bản còn lại: không lựa chọn cục bộ nào an toàn, và ta phải xét mọi khả năng nhưng nhớ lại kết quả đã tính — quy hoạch động (C/16).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Harvey & van der Hoeven, “Integer Multiplication in Time $O(n \log n)$ ”	Annals of Mathematics, 2021	Đọc phần mở đầu: nó kể lại toàn bộ lịch sử cận trên của phép nhân và nói rõ vì sao $n \log n$ là ngưỡng được kỳ vọng.
nền tảng	Strassen, “Gaussian Elimination Is Not Optimal”	Numerische Mathematik, 1969	Ba trang, đọc trong hai mươi phút, và là ví dụ hoàn hảo cho chiến lược “giảm số bài con”.
tra khi cần	Alman & Vassilevska Williams, “A Refined Laser Method and Faster Matrix Multiplication”	SODA, 2021	Tra khi muốn biết các cận ω hiện đại được lấy ra bằng kỹ thuật gì; phần mở đầu đủ cho người đọc không chuyên.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Alman, Duan, Vassilevska Williams, Xu, Xu, Zhou, "Improving the Matrix Multiplication Exponent..."	bản tiền ấn arXiv, 2026 — <i>chưa qua phản biện</i>	Chỉ đọc để biết con số mới nhất; không trích dẫn như kết quả đã ổn định.
tra khi cần	Cooley & Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series"	Mathematics of Computation, 1965	Bài gốc của FFT — công cụ đứng sau mọi thuật toán nhân nhanh hiện đại (D/21).

Tham lam (Greedy Algorithms)

PHỤ THUỘC A/04 cho lập luận đối chỗ — chương này không dùng được nếu thiếu nó; B/07 cho heap; B/11 cho DSU trong Kruskal.

TÓM TẮT

Tham lam là chiến lược có khoảng cách lớn nhất giữa độ ngắn của code và độ khó của chứng minh: bốn dòng chương trình, nhưng nếu không chứng minh thì bạn không biết mình đúng hay sai. Chương này tập trung vào phần chứng minh — hai khuôn chuẩn là lập luận đối chỗ và “tham lam luôn đi trước” — và vào câu hỏi sâu hơn: *cấu trúc nào của bài toán làm cho tham lam đúng?* Câu trả lời có tên riêng là matroid, và biết nó giúp bạn nhận ra ngay một lớp bài mà tham lam chắc chắn tối ưu. Chương cũng nói về những nơi tham lam sai nhưng vẫn hữu ích: xấp xỉ với bảo đảm chất lượng. Nếu chỉ nhớ một điều: tham lam không sai vì “nhìn ngắn”, nó sai vì lựa chọn hôm nay *phá huỷ* khả năng của tương lai — và nhiệm vụ của chứng minh là loại trừ đúng khả năng đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thuật toán tham lam** — xây lời giải bằng cách lặp lại một lựa chọn tốt nhất theo tiêu chí cục bộ, không bao giờ xét lại.
- **tính chất lựa chọn tham lam** — tồn tại một lời giải tối ưu chứa lựa chọn mà tham lam đưa ra ở bước đầu. Đây là điều phải chứng minh.
- **lập luận đối chỗ** — biến một lời giải tối ưu bất kỳ thành lời giải tham lam qua các phép đổi không làm nó tệ đi (A/04).
- **matroid** — cấu trúc gồm một tập nền và một họ “tập độc lập” thỏa hai tính chất; đó chính xác là cấu trúc mà tham lam luôn cho kết quả tối ưu.
- **hàm dưới mô-đun (submodular)** — hàm mà lợi ích biên giảm dần khi tập đã chọn lớn lên. Tham lam trên hàm này cho bảo đảm xấp xỉ $1 - 1/e$.
- **tham lam có hối tiếc** — biến thể: cứ nhận tất cả, rồi khi vi phạm ràng buộc thì bỏ đi phần tử tệ nhất đã nhận — thường cài bằng heap.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao tham lam sai ở bài đổi tiền với mệnh giá $\{1, 3, 4\}$ mà lại đúng với mệnh giá tiền thật của hầu hết các nước?
- Q2. Kruskal và Prim đều là tham lam, đều cho cây khung nhỏ nhất, nhưng chọn cạnh theo hai tiêu chí hoàn toàn khác nhau. Vì sao cả hai cùng đúng?
- Q3. Có cách nào biết *trước* rằng tham lam sẽ đúng cho một bài, thay vì thử rồi hy vọng?
- Q4. Khi tham lam sai, vì sao câu trả lời thường là quy hoạch động chứ không phải một tiêu chí tham lam khôn hơn?
- Q5. Thuật toán tham lam cho bài phủ tập hợp không tối ưu nhưng vẫn được dùng rộng rãi. Điều gì làm nó đáng tin cậy dù không tối ưu?

Có một trải nghiệm phổ biến khi luyện thuật toán: bạn đọc đề, nghĩ ra một quy tắc đơn giản — “cứ chọn cái rẻ nhất”, “cứ chọn cái kết thúc sớm nhất” — cài trong ba phút, nộp, và sai. Bạn thử

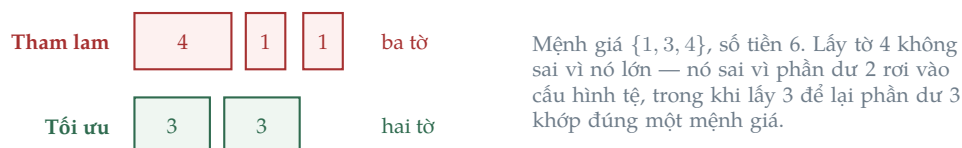
một quy tắc khác, cũng sai. Đến quy tắc thứ ba thì đúng, nhưng bạn không biết vì sao, và bài sau bạn lại phải đoán từ đầu.

Đó là trạng thái mà chương này muốn chấm dứt. Tham lam không phải trò may rủi; nó có lý thuyết, và lý thuyết đó trả lời được câu “bài này tham lam có đúng không” trước khi bạn gõ code. Nhưng phải nói ngay điều khó chịu: cái giá của việc dùng tham lam là bạn *phải chứng minh*, và không có cách nào né. Một lời giải tham lam không có chứng minh chỉ là một phỏng đoán được viết bằng ngôn ngữ lập trình.

15.1 Vì sao tham lam sai, nói cho chính xác

Trực giác thông thường nói tham lam sai vì nó “nhìn ngắn”. Cách nói đó đúng nhưng quá mơ hồ để dùng. Phát biểu chính xác hơn: tham lam sai khi lựa chọn hôm nay *phá huỷ những khả năng* mà một lựa chọn khác giữ lại được.

Hãy xem bài đổi tiền với mệnh giá $\{1, 3, 4\}$ và số tiền 6. Tham lam lấy 4, còn lại 2, phải lấy $1 + 1$ — ba tờ. Tối ưu là $3 + 3$ — hai tờ. Vấn đề không phải ở chỗ 4 lớn hơn 3, mà ở chỗ sau khi lấy 4 thì phần dư 2 rơi vào một cấu hình tệ, trong khi lấy 3 để lại phần dư 3 rơi đúng vào mệnh giá có sẵn.



Hình 15.1. Vì sao tham lam sai, nói cho chính xác. Vấn đề không nằm ở chỗ lựa chọn hiện tại “nhỏ” hay “lớn” mà ở chỗ nó *phá huỷ* cấu hình còn lại. Mọi chứng minh trong chương này đều nhằm loại trừ đúng khả năng đó.

Từ cách phát biểu này rút ra ngay một cách kiểm tra thực dụng: *sau khi tham lam chọn một phần tử, bài toán còn lại có còn cùng dạng và có tệ đi so với phương án khác không?* Nếu bạn chứng minh được rằng chọn theo tiêu chí tham lam để lại một bài toán con “không tệ hơn” mọi lựa chọn khác, thì tham lam đúng — và đó chính là nội dung của lập luận đổi chỗ.

15.2 Hai khuôn chứng minh

Chương [A/04](#) đã giới thiệu hai khuôn; ở đây ta dùng chúng trên các bài thật.

Khuôn 1 — đổi chỗ. Lấy một lời giải tối ưu bất kỳ, xét chỗ khác đầu tiên so với tham lam, chứng minh rằng có thể sửa lời giải tối ưu tại đó cho giống tham lam mà không làm nó tệ đi. Áp dụng cho bài xếp lịch: tham lam chọn cuộc họp kết thúc sớm nhất; nếu lời giải tối ưu chọn cuộc khác thì đổi sang cuộc kết thúc sớm nhất vẫn hợp lệ vì nó kết thúc sớm hơn, nên không cản trở các cuộc sau.

Khuôn 2 — tham lam luôn đi trước. Chứng minh bằng quy nạp rằng sau k bước, lời giải tham lam “không kém” bất kỳ lời giải nào theo một đại lượng đo được. Với bài xếp lịch, đại lượng là thời điểm kết thúc của cuộc họp thứ k : tham lam luôn có thời điểm kết thúc sớm nhất có thể, nên nó không bao giờ bị kẹt trước lời giải khác.

Trong thực hành, khuôn 1 thường dễ áp hơn vì nó chỉ đòi hỏi một phép biến đổi cục bộ. Nhưng khuôn 2 có ưu điểm riêng: khi phép đổi chỗ không thực hiện được, chỗ vướng thường chính là phản ví dụ đang cần tìm.

Có một mẹo phụ rất hữu ích cho các bài mà tiêu chí tham lam là “sắp xếp theo cái gì đó”: nếu bạn nghi ngờ nên sắp theo tiêu chí f , hãy xét *hai phần tử liền kề* trong một lời giải tối ưu và hỏi “đổi chỗ hai phần tử này có làm kết quả tốt lên không”. Điều kiện để không đổi chỗ được sẽ cho bạn chính xác tiêu chí sắp xếp. Đây là cách người ta suy ra các tiêu chí lạ như “sắp theo tỉ số” hay “sắp theo hiệu” mà nhìn đề thì không đoán ra.

15.3 Matroid: cấu trúc mà tham lam luôn đúng

Đến đây có một câu hỏi tự nhiên: thay vì chứng minh lại cho từng bài, có đặc trưng chung nào cho các bài mà tham lam đúng không? Có, và nó là một trong những kết quả đẹp nhất của lý thuyết thuật toán.

Một *matroid* gồm một tập nền E và một họ các tập con được gọi là “độc lập”, thoả hai điều kiện: mọi tập con của một tập độc lập cũng độc lập; và nếu tập độc lập A nhỏ hơn tập độc lập B thì luôn tồn tại một phần tử trong $B \setminus A$ thêm được vào A mà vẫn giữ tính độc lập. Điều kiện thứ hai — gọi là tính chất trao đổi — chính là thứ đảm bảo rằng tham lam không bao giờ tự đưa mình vào ngõ cụt.

Định lý cốt lõi: *thuật toán tham lam cho kết quả tối ưu trên mọi hàm trọng số khi và chỉ khi cấu trúc là một matroid*. Chiều thuận nói tham lam đúng; chiều đảo còn mạnh hơn — nếu cấu trúc không phải matroid thì tồn tại một hàm trọng số làm tham lam sai.

Ví dụ điển hình: tập các tập cạnh không tạo chu trình của một đồ thị tạo thành một matroid. Đó là lý do Kruskal đúng — và cũng là câu trả lời cho câu hỏi mở đầu thứ hai: Kruskal và Prim khác nhau ở tiêu chí nhưng cùng khai thác một cấu trúc, nên cả hai cùng đúng. Một ví dụ khác: bài xếp lịch các công việc có hạn chót, mỗi việc một đơn vị thời gian, tối đa hoá lợi nhuận — tập các việc “xếp được” cũng tạo thành matroid.

Giá trị thực hành của khái niệm này không phải để bạn kiểm tra tiên đề mỗi lần gặp bài, mà là để có một danh mục: nếu bài của bạn quy về “chọn tập con độc lập trọng số lớn nhất” trên một cấu trúc kiểu rừng, kiểu không gian vector, hay kiểu phân hoạch, thì tham lam gần như chắc chắn đúng và bạn không cần đắn đo.

TRỰC GIÁC

Vì sao tính chất trao đổi lại là điều kiện đúng? Hãy nghĩ theo hướng ngược lại. Tham lam thất bại khi nó chọn xong k phần tử và rơi vào tình trạng “không thêm được gì nữa”, trong khi một lời giải khác chọn k phần tử khác lại còn mở rộng được. Tính chất trao đổi loại trừ đúng tình huống này: nếu lời giải kia lớn hơn thì luôn có phần tử của nó thêm được vào tập của tham lam. Nói cách khác, matroid là cách phát biểu hình thức cho điều kiện “tham lam không bao giờ tự bịt đường của chính mình”.

15.4 Danh mục các bài tham lam kinh điển

Bảng dưới đây gom các bài mà tiêu chí tham lam đã được chứng minh, kèm tiêu chí và lý do. Đáng thuộc vì các bài mới thường là biến thể của chúng.

Bài toán	Tiêu chí tham lam	Vì sao đúng
Chọn nhiều cuộc họp nhất	Kết thúc sớm nhất trước	Để lại nhiều thời gian nhất cho phần còn lại; đổi chỗ trực tiếp
Cây khung nhỏ nhất	Cạnh nhẹ nhất không tạo chu trình (Kruskal)	Cấu trúc rừng là matroid; tính chất cắt
Cây khung nhỏ nhất	Cạnh nhẹ nhất rời khỏi cây đang có (Prim)	Cùng tính chất cắt, duyệt theo thứ tự khác
Ba lô phân số	Tỉ số giá trị trên khối lượng	Được lấy phần lẻ nên không có “lãng phí chỗ”
Mã Huffman	Gộp hai tần suất nhỏ nhất	Hai ký hiệu hiếm nhất phải nằm sâu nhất; đổi chỗ
Xếp lịch giảm trễ tối đa	Hạn chót sớm nhất trước	Đổi chỗ hai việc liên kế không làm trễ tăng
Đổi tiền (hệ chuẩn)	Mệnh giá lớn nhất trước	Chỉ đúng với hệ mệnh giá đặc biệt — phải kiểm
Chọn việc có hạn chót, tối đa lợi nhuận	Lợi nhuận lớn trước, xếp muộn nhất có thể	Matroid phân hoạch theo thời điểm
Phủ tập hợp	Chọn tập phủ nhiều phần tử mới nhất	Không tối ưu, nhưng bảo đảm xấp xỉ $\ln n$

Có một mẫu đáng chú ý trong bảng: gần như mọi tiêu chí đều là một *thứ tự sắp xếp*. Điều đó nổi chương này với chương B/06: rất nhiều bài tham lam thực chất là “sắp xếp theo đúng khoá rồi quét một lượt”, và toàn bộ độ khó nằm ở việc tìm ra khoá.

Tham lam có hối tiếc

Một biến thể đáng thuộc vì nó giải một họ bài mà tham lam thuần tuý chịu. Ý tưởng: cứ nhận mọi thứ đến; khi vi phạm ràng buộc thì bỏ đi phần tử tệ nhất trong những cái đã nhận. Cấu trúc thực thi là heap (B/07).

Ví dụ: mỗi công việc có hạn chót và thời lượng, cần chọn nhiều việc nhất làm được. Duyệt theo hạn chót tăng dần, nhận mọi việc; nếu tổng thời gian vượt hạn chót hiện tại thì bỏ việc dài nhất đã nhận. Kết quả tối ưu, và chứng minh vẫn là đổi chỗ.

Điều đáng học ở đây có tính phương pháp: khi tiêu chí tham lam “chọn hay không chọn” quá khó, hãy đổi sang tiêu chí “nhận hết rồi loại dần”. Hai cách nhìn cho cùng bài toán, nhưng cách thứ hai thường có tiêu chí đơn giản hơn nhiều.

15.5 Khi tham lam sai: xấp xỉ có bảo đảm

Không phải cứ tham lam sai là vứt đi. Với nhiều bài NP-khó (D/23), tham lam là thuật toán tốt nhất trong thực tế, và điều làm nó đáng tin không phải là nó “thường đúng” mà là nó có *bảo đảm chất lượng chứng minh được*.

Hai kết quả kinh điển. Với bài phủ tập hợp, tham lam chọn tập phủ được nhiều phần tử mới nhất cho lời giải không tệ hơn $\ln n$ lần tối ưu — và người ta chứng minh được rằng dưới các giả thuyết chuẩn của lý thuyết độ phức tạp thì không thuật toán đa thức nào làm tốt hơn đáng kể. Nghĩa là tham lam ở đây không phải giải pháp tạm mà là gần như tốt nhất có thể.

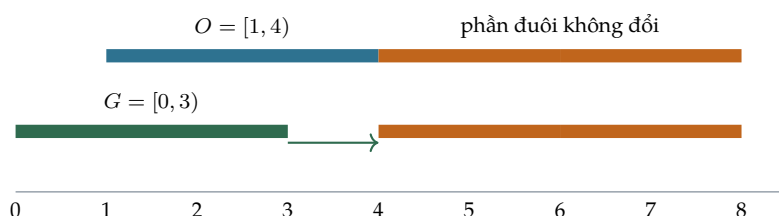
Kết quả thứ hai đẹp hơn và có ứng dụng rộng: khi hàm mục tiêu là *dưới mô-đun* — lợi ích biên giảm dần khi tập đã chọn lớn lên — và ta tối đa hoá dưới ràng buộc số lượng, thì tham lam cho ít nhất $1 - 1/e \approx 63\%$ giá trị tối ưu. Tính chất lợi ích biên giảm dần xuất hiện tự nhiên ở rất nhiều nơi: chọn vị trí đặt cảm biến, chọn tập ảnh đại diện, chọn đặc trưng cho mô hình, chọn khung hình chủ chốt trong một hệ thống bản đồ. Trong tất cả những bài đó, tham lam vừa đơn giản vừa có bảo đảm — một sự kết hợp hiếm.

CẠM BẤY

Bảo đảm xấp xỉ là bảo đảm về *trường hợp xấu nhất*, không phải mô tả hành vi thường gặp. Tham lam cho phủ tập hợp trong thực tế thường cho kết quả rất gần tối ưu, tốt hơn nhiều so với cận $\ln n$. Ngược lại, đừng lấy hành vi thực tế tốt làm cơ sở để bỏ qua việc kiểm tra: có những dữ liệu chạm đúng trường hợp xấu, và trong thi đấu thì dữ liệu *được cố ý* dựng như vậy.

15.6 Vẽ phép đổi chỗ: điều gì được bảo toàn

Trong bài chọn nhiều cuộc họp nhất, dùng đoạn nửa mở $[s, f)$ để hai cuộc chạm đầu mút vẫn tương thích. Hình 15.2 giữ nguyên phần đuôi của một lịch tối ưu, chỉ thay cuộc đầu tiên. Lý do đúng là $f(G) \leq f(O)$, không phải vì độ dài cuộc G ngắn hơn.



Hình 15.2. Đổi cuộc đầu tiên của lịch tối ưu sang cuộc kết thúc sớm nhất không làm mất cuộc nào ở phía sau. Đoạn trống từ 3 tới 4 cho thấy phần thời gian được nới ra sau phép đổi.

Nếu mỗi cuộc có lợi ích khác nhau, phép đổi vẫn giữ tính hợp lệ nhưng có thể giảm tổng lợi ích. Ví dụ $[0, 3)$ lợi ích 2 và $[0, 4)$ lợi ích 100: chọn kết thúc sớm cho giá trị 2 thay vì 100. Khi đó dùng quy hoạch động trên các cuộc đã sắp theo thời điểm kết thúc, với trạng thái có lựa chọn lấy hoặc bỏ cuộc hiện tại. Kinh nghiệm rút ra: mỗi lần đề thêm trọng số, hãy kiểm lại đại lượng được bảo toàn trong chứng minh cũ.

15.7 Gốc rễ: điện khí hoá Moravia và một bài tập về nhà

Bài toán cây khung nhỏ nhất có một nguồn gốc bất ngờ và rất cụ thể. Năm 1926, nhà toán học Séc Otakar Borůvka công bố thuật toán đầu tiên cho bài toán này, và động cơ là một nhu cầu kỹ thuật của thời đó: quy hoạch mạng lưới điện cho vùng Moravia sao cho tổng chiều dài dây dẫn nhỏ nhất. Thuật toán của ông — gộp đồng thời mọi thành phần với cạnh nhẹ nhất rời khỏi chúng — về sau được đánh giá cao vì rất dễ song song hoá, dù suốt nhiều thập kỷ nó ít nổi tiếng hơn hai thuật toán sau. Jarník tìm ra thuật toán kiểu Prim năm 1930, cũng ở Tiệp Khắc; Kruskal và Prim công bố độc lập vào năm 1956 và 1957 tại Mỹ, trong bối cảnh nghiên cứu mạng viễn thông.

Mã Huffman có một câu chuyện thường được kể trong giới: năm 1952, David Huffman là sinh viên cao học ở MIT, và giáo sư cho lớp chọn giữa việc thi cuối kỳ hoặc viết một tiểu luận về việc tìm mã tiền tố tối ưu — một bài toán mà chính giáo sư và các đồng nghiệp chưa giải được. Huffman chọn tiểu luận, gần như bỏ cuộc, rồi tìm ra ý tưởng gộp từ dưới lên. Điều đáng học không phải giai thoại mà là hướng suy nghĩ: mọi người trước đó cố xây cây *từ trên xuống*, còn lời giải nằm ở hướng ngược lại. Đây là một ví dụ trực tiếp cho heuristic “đảo ngược hướng nhìn” ở chương A/01.

Phần lý thuyết đến muộn hơn nhiều và từ một ngành khác. Khái niệm matroid do Hassler Whitney đưa ra năm 1935 trong bối cảnh toán học thuần túy — ông muốn trừu tượng hoá khái niệm

“độc lập tuyến tính” để nó dùng được cho cả đồ thị. Mãi tới đầu thập niên 1970, Jack Edmonds mới nối hai đầu lại và chứng minh rằng matroid *chính xác* là cấu trúc mà tham lam tối ưu. Đây là một mẫu lặp lại trong lịch sử ngành: một khái niệm toán học ra đời không vì ứng dụng, ba mươi năm sau trở thành câu trả lời chính xác cho một câu hỏi thuật toán.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1926 — Borůvka	Quy hoạch mạng điện cho vùng Moravia với tổng chiều dài dây nhỏ nhất.	Thuật toán cây khung nhỏ nhất đầu tiên; ít nổi tiếng nhưng song song hoá tốt nhất trong ba thuật toán.
1930, 1956–57 — Jarník, Kruskal, Prim	Thiết kế mạng viễn thông chi phí thấp.	Hai tiêu chí tham lam khác nhau cho cùng bài toán; cả hai đều đúng vì cùng khai thác tính chất cắt.
1952 — Huffman	Tìm mã tiền tố tối ưu — bài toán mở mà thầy giao thay cho bài thi.	Xây cây từ dưới lên thay vì từ trên xuống; ví dụ trực tiếp của heuristic “đảo ngược hướng nhìn”.
1935 — Whitney	Trừu tượng hoá khái niệm độc lập tuyến tính cho cả đồ thị.	Matroid ra đời trong toán thuần túy, không nhằm ứng dụng nào.
Đầu thập niên 1970 — Edmonds	Vì sao tham lam đúng ở bài này mà sai ở bài kia?	Định lý: tham lam tối ưu trên mọi trọng số khi và chỉ khi cấu trúc là matroid — một điều kiện <i>cần và đủ</i> , hiếm gặp trong lý thuyết thuật toán.
1978 — Nemhauser và cộng sự	Tối đa hoá hàm có lợi ích biên giảm dần dưới ràng buộc số lượng.	Tham lam bảo đảm $1 - 1/e$; nền lý thuyết cho việc chọn cảm biến, chọn đặc trưng, chọn dữ liệu đại diện.

I 15.8 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kinh tế học	Quyết định cận biên, tối ưu hoá thiên cận	Cùng cấu trúc: chọn theo lợi ích tức thời. Cùng chỗ hổng: khi lựa chọn hôm nay thay đổi tập lựa chọn ngày mai. Kinh tế học gọi đó là chi phí cơ hội động; ta gọi là “phá huỷ khả năng tương lai”.
Tiến hoá, tối ưu hoá cục bộ	Leo đồi: luôn đi theo hướng cải thiện	Tham lam là leo đồi trên không gian tổ hợp. Cùng thất bại: kẹt ở cực trị địa phương. Cách chữa cũng tương tự — cho phép bước lùi, tức là mô phỏng luyện kim (C/19).
Hệ điều hành, bộ nhớ đệm	Chính sách thay thế LRU, lập lịch tiến trình	LRU là tham lam trên thông tin quá khứ. Điểm khác quan trọng: ở đây tương lai <i>chưa biết</i> , nên tiêu chuẩn đánh giá không phải tối ưu mà là tỉ lệ cạnh tranh so với lời giải biết trước tương lai (D/24).
Nén dữ liệu	Mã Huffman, mã số học	Ứng dụng tham lam kinh điển nhất, và vẫn đang chạy trong mọi tệp ảnh nén bạn mở hôm nay.
Học máy	Chọn đặc trưng tham lam, cây quyết định, học tích cực	Cây quyết định chọn thuộc tính chia tốt nhất tại mỗi nút — tham lam thuần túy, và nó <i>không</i> tối ưu toàn cục nhưng chạy tốt trong thực tế. Nhiều tiêu chí chọn dữ liệu có tính dưới mô-đun nên tham lam có bảo đảm $1 - 1/e$.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Robot và SLAM	Chọn khung hình chủ chốt, chọn điểm mốc, chọn cảm biến	Bài toán “chọn k quan sát cho nhiều thông tin nhất” thường có hàm mục tiêu dưới mô-đun — lợi ích của một quan sát mới giảm khi đã có nhiều quan sát tương tự. Đó là lý do tham lam ở đây không chỉ tiện mà còn có bảo đảm chất lượng.
Tài chính	Chiến lược thiên cận so với tối ưu động	Cùng bài học: tối ưu từng bước không cho tối ưu cả chuỗi khi có ràng buộc liên thời gian. Đây cũng chính là ranh giới dẫn sang quy hoạch động ở C/16.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì tính đúng của tham lam ở bài đổi tiền là tính chất của *hệ mệnh giá*, không phải của thuật toán. Với $\{1, 3, 4\}$ và số tiền 6, lấy 4 để lại phần dư 2 rơi vào cấu hình tệ, trong khi lấy 3 để lại phần dư 3 khớp đúng một mệnh giá. Các hệ tiền thật thường có cấu trúc bội số $(1, 2, 5, 10, \dots)$ làm cho việc lấy mệnh giá lớn nhất không bao giờ phá huỷ khả năng của phần dư. Có thuật toán kiểm tra một hệ mệnh giá có “tham lam được” hay không, và đó là bằng chứng rõ ràng rằng đây là tính chất của dữ liệu chứ không của phương pháp.
- A2. Vì cả hai đều dựa trên cùng một tính chất, gọi là tính chất cắt: với mọi cách chia đỉnh thành hai nhóm, cạnh nhẹ nhất nối hai nhóm luôn thuộc một cây khung nhỏ nhất nào đó. Kruskal áp dụng nó bằng cách duyệt cạnh theo trọng số tăng; Prim áp dụng bằng cách luôn xét cắt giữa cây đang có và phần còn lại. Nói theo ngôn ngữ mục 3: tập các tập cạnh không tạo chu trình là một matroid, và mọi cách duyệt tham lam hợp lệ trên matroid đều cho kết quả tối ưu.
- A3. Có, ít nhất cho một lớp lớn: nếu bài toán quy về “chọn tập con độc lập có trọng số lớn nhất” trên một cấu trúc thoả hai tiên đề matroid thì tham lam chắc chắn tối ưu, và định lý của Edmonds nói điều ngược lại cũng đúng — không phải matroid thì tồn tại trọng số làm tham lam sai. Trong thực hành, thay vì kiểm tiên đề, người ta nhận diện qua danh mục: cấu trúc rừng, không gian vector, phân hoạch theo nhóm. Ngoài lớp đó thì vẫn phải chứng minh bằng đổi chỗ cho từng bài.
- A4. Vì khi không có lựa chọn cục bộ nào an toàn, vấn đề không nằm ở việc tiêu chí chưa đủ khôn mà nằm ở chỗ *lời giải tối ưu phụ thuộc vào những gì đã chọn trước đó*. Nói cách khác, bài toán có trạng thái. Quy hoạch động là cách xử lý đúng cho tình huống đó: thay vì cam kết một lựa chọn, ta xét mọi lựa chọn nhưng nhớ lại kết quả theo trạng thái để khởi tính lại (C/16). Cố tìm tiêu chí tham lam khôn hơn trong tình huống này thường dẫn tới một tiêu chí phức tạp mà vẫn có phản ví dụ.
- A5. Bảo đảm chất lượng *chứng minh được*: nó không bao giờ tệ hơn tối ưu quá $\ln n$ lần cho bài phủ tập hợp, hoặc không kém $1 - 1/e$ lần tối ưu khi hàm mục tiêu có lợi ích biên giảm dần. Điều này khác hẳn với “thực nghiệm thấy tốt”: bảo đảm đúng cho mọi dữ liệu, kể cả dữ liệu do đối thủ dựng. Và với bài phủ tập hợp còn có kết quả mạnh hơn: dưới các giả thuyết chuẩn về độ phức tạp, không thuật toán đa thức nào cải thiện được cận đó đáng kể — nên tham lam gần như là tốt nhất có thể.

NÓI LẠI THẬT ĐƠN GIẢN

Tham lam là cách giải mà ai cũng thử đầu tiên: cứ chọn cái tốt nhất trước mắt, rồi tính tiếp. Nhiều khi nó đúng, nhiều khi nó sai, và chương này nói về cách phân biệt hai trường hợp đó. Hãy nghĩ tới việc trả tiền lẻ. Nếu có các tờ 1, 2, 5, 10 thì cứ lấy tờ to nhất còn dùng được là ổn. Nhưng nếu tiền chỉ có mệnh giá 1, 3 và 4, mà bạn phải trả 6, thì lấy tờ 4 trước là hỏng — còn 2 và bạn phải dùng hai tờ 1, tổng ba tờ; trong khi hai tờ 3 là đủ. Điều làm hỏng không phải vì tờ 4 nhỏ hay to, mà vì sau khi lấy nó thì phần còn lại rơi vào tình thế xấu. Đó chính là câu hỏi duy nhất cần hỏi mỗi lần định dùng tham lam: *sau khi tôi chọn cái này, phần còn lại có bị kẹt hơn so với chọn cái khác không?* Nếu chứng minh được là không bao giờ kẹt hơn, tham lam đúng. Cách chứng minh chuẩn cũng rất trực quan: đưa tôi bất kỳ cách làm tốt nhất nào của bạn, tôi sẽ sửa dần nó cho giống cách của tôi, mỗi lần sửa đều không làm nó xấu đi.

Có một điều thú vị mà các nhà toán học đã tìm ra: tồn tại một loại cấu trúc mà trên đó tham lam *luôn luôn* đúng, bất kể trọng số là gì. Nếu bài toán của bạn có dạng “chọn một nhóm phần tử sao cho không tạo thành vòng” — như chọn dây điện nối các thành phố mà không tạo vòng lặp — thì bạn ở trong loại cấu trúc đó và có thể yên tâm.

Cuối cùng, khi tham lam sai, đôi khi ta vẫn dùng nó — nhưng chỉ khi biết rõ nó sai tới mức nào. Có những bài mà tham lam bảo đảm cho kết quả ít nhất bằng khoảng hai phần ba mức tốt nhất. Biết con số đó là điều khác biệt giữa “dùng liều” và “dùng có căn cứ”.

Một câu: Tham lam không sai vì nhìn ngắn; nó sai khi lựa chọn hôm nay bịt mắt đường của ngày mai.

Chỗ không rút gọn được: Tìm ra tiêu chí tham lam đúng cho một bài mới vẫn là bước sáng tạo; matroid chỉ nhận diện được một lớp, không phải mọi bài.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Tham lam sai khi nào — phát biểu chính xác	Khi lựa chọn hiện tại phá hủy những khả năng mà lựa chọn khác giữ lại được, chứ không phải vì “nhìn ngắn”.
●●● Hai khuôn chứng minh	Đổi chỗ (biến lời giải tối ưu thành tham lam, từng bước không tệ đi) và “tham lam luôn đi trước” (quy nạp theo một đại lượng đo được).
●●● Mẹo tìm tiêu chí sắp xếp	Xét hai phần tử liền kề trong lời giải tối ưu, hỏi “đổi chỗ có tốt lên không”; điều kiện không đổi được cho ra chính tiêu chí sắp xếp.
●●● Matroid — định lý cốt lõi	Tham lam tối ưu trên mọi hàm trọng số <i>khi và chỉ khi</i> cấu trúc là matroid. Rừng trong đồ thị là matroid, nên Kruskal đúng.
●●● Tính chất cắt	Với mọi cách chia đỉnh thành hai nhóm, cạnh nhẹ nhất nối hai nhóm thuộc một cây khung nhỏ nhất — nền của cả Kruskal lẫn Prim.
●●● Khi tham lam sai thì làm gì	Chuyển sang quy hoạch động, vì bài toán có trạng thái: lời giải tối ưu phụ thuộc những gì đã chọn trước đó (C/16).
●● Tham lam có hối tiếc	Nhận hết, khi vi phạm thì bỏ phần tử tệ nhất đã nhận (dùng heap). Đổi tiêu chí “chọn hay không” thành “nhận rồi loại”, thường đơn giản hơn nhiều.
●● Huffman	Gộp hai tần suất nhỏ nhất; ý tưởng đến từ việc xây cây <i>từ dưới lên</i> thay vì từ trên xuống.
●● Ba lô phân số so với ba lô nguyên	Phân số: tham lam theo tỉ số, tối ưu. Nguyên: tham lam sai, cần quy hoạch động — khác nhau đúng ở chỗ “có lấy phần lẻ được không”.
●● Bảo đảm xấp xỉ của tham lam	Phủ tập hợp: không tệ hơn $\ln n$ lần tối ưu. Hàm dưới mô-đun dưới ràng buộc số lượng: ít nhất $1 - 1/e$ lần tối ưu.
●● Hàm dưới mô-đun	Lợi ích biên giảm dần khi tập đã chọn lớn lên; xuất hiện tự nhiên ở chọn cảm biến, chọn đặc trưng, chọn khung hình chủ chốt.
● Bảo đảm xấp xỉ nghĩa là gì	Bảo đảm cho trường hợp xấu nhất, không mô tả hành vi thường gặp — thực tế thường tốt hơn nhiều, nhưng dữ liệu thi đấu được dựng để chạm đúng trường hợp xấu.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Trong kỳ thi, tôi nên dành bao lâu để chứng minh một tiêu chí tham lam?

| Một quy trình thực dụng: dành hai phút thử đổi chỗ trong đầu; nếu ra thì cài. Nếu không ra sau hai phút, hãy viết bộ sinh test ngẫu nhiên nhỏ và so với vết cặn — năm phút và cho câu trả lời

chắc chắn hơn nhiều so với ngồi nghĩ thêm mười phút. Nếu stress test không tìm được phản ví dụ sau vài nghìn ca nhỏ, xác suất tiêu chí đúng là khá cao. Điều tuyệt đối không nên làm là cài rồi nộp mà không làm cả hai việc trên.

2. Bài của tôi có nhiều tiêu chí tham lam hợp lý. Chọn thế nào?

| Dùng mẹo hai phần tử liền kề ở mục 2: giả sử lời giải tối ưu xếp phần tử a ngay trước b ; viết ra điều kiện để việc đổi chỗ chúng *không* cải thiện kết quả. Bất đẳng thức thu được chính là tiêu chí so sánh cần dùng để sắp xếp, và nó thường có dạng bất ngờ như “sắp theo $a_i + b_i$ ” hoặc “sắp theo tỉ số”. Cách này biến việc đoán tiêu chí thành việc giải một bất đẳng thức — xác định hơn nhiều.

3. Nếu tham lam cho kết quả gần đúng thì có nên nộp không?

| Trong thi đấu thì không, vì chấm là đúng hoặc sai. Trong công việc thật thì tùy, và câu hỏi đúng là: bạn có *bảo đảm* nào không? “Thử vài ca thấy gần đúng” rất khác với “chứng minh được không tệ hơn $1 - 1/e$ lần tối ưu”. Nếu có bảo đảm và sai số chấp nhận được thì tham lam là lựa chọn tốt: đơn giản, nhanh, dễ bảo trì. Nếu không có bảo đảm, ít nhất hãy đo trên dữ liệu thật và biết phân phối sai số.

4. Tham lam và quy hoạch động có thể kết hợp không?

| Có, theo nhiều cách. Cách phổ biến nhất: dùng tham lam để giảm không gian trạng thái — chứng minh rằng một số lựa chọn không bao giờ tối ưu và loại chúng, rồi quy hoạch động trên phần còn lại. Cách thứ hai: dùng lời giải tham lam làm cận trên trong nhánh cận (C/13). Cách thứ ba, thường gặp trong bài khó: sắp xếp bằng tiêu chí tham lam trước, rồi quy hoạch động trên thứ tự đó — vì tiêu chí đúng thu hẹp bài toán từ “mọi thứ tự” xuống “một thứ tự”.

5. Vì sao Dijkstra được xếp vào tham lam?

| Vì nó cam kết một lựa chọn không bao giờ xét lại: mỗi bước chốt đỉnh có nhãn nhỏ nhất và coi khoảng cách đó là cuối cùng. Chứng minh tính đúng chính là chứng minh lựa chọn tham lam an toàn, và nó dùng giả thiết trọng số không âm đúng một lần — ở chỗ khẳng định rằng không có đường nào đi vòng qua các đỉnh chưa chốt rồi quay lại mà ngắn hơn. Đây là ví dụ tốt cho việc “giả thiết nằm ở đâu trong chứng minh” quyết định “thuật toán hỏng ở đâu”.

6. Tính dưới mô-đun kiểm tra thế nào trong thực tế?

| Định nghĩa là: thêm một phần tử vào tập nhỏ cho lợi ích ít nhất bằng thêm chính phần tử đó vào tập lớn hơn chứa nó. Trong thực hành, ba dấu hiệu gợi ý mạnh: hàm đếm số phần tử được phủ (phủ tập hợp); hàm đo lượng thông tin thu được từ các quan sát có phần trùng lặp; và hàm đo độ đa dạng của một tập đã chọn. Ba họ này phủ phần lớn ứng dụng thực tế. Nếu nghi ngờ, hãy kiểm tra bằng số trên các trường hợp nhỏ trước khi dựa vào bảo đảm lý thuyết.

7. Có bài nào mà tham lam đúng nhưng chứng minh rất khó không?

| Có, và một ví dụ nổi tiếng là bài xếp lịch với hạn chót và tiền phạt, hay các biến thể của bài toán ghép đôi ổn định. Thuật toán ghép đôi ổn định — mỗi bên lần lượt cầu hôn theo thứ tự ưu tiên — trông rất giống tham lam và luôn cho kết quả ổn định, nhưng chứng minh đòi hỏi lập luận tinh tế về việc không tồn tại cặp muốn bỏ nhau. Bài học: độ ngắn của code không nói gì về độ khó của chứng minh, và đó là chủ đề của cả chương này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Chọn nhiều cuộc họp nhất không chồng lấn	CSES “Movie Festival”[33]	Sắp theo thời điểm kết thúc; viết chứng minh đổi chỗ ra giấy.
● Ba lô phân số	Bài kinh điển	Sắp theo tỉ số; giải thích vì sao cách này sai với ba lô nguyên.
● Số tiền nhỏ nhất không tạo được từ các mệnh giá đã có	CSES “Coin Piles”-loại[33]	Sắp tăng dần và duy trì tổng đạt được; bất biến rất gọn.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Mã Huffman	Bài kinh điển	Cài bằng heap; sau đó tự chứng minh bằng đổi chỗ hai ký hiệu hiếm nhất.
•• Cây khung nhỏ nhất bằng Kruskal và bằng Prim	CSES “Road Reparation”[33]	Cài cả hai, so thời gian trên đồ thị thưa và dày; giải thích khác biệt.
•• Xếp lịch giảm thiểu độ trễ lớn nhất	Bài kinh điển	Sắp theo hạn chót; chứng minh bằng đổi chỗ hai việc liền kề.
•• Chọn nhiều việc nhất khi mỗi việc có thời lượng và hạn chót	Bài kinh điển	Tham lam có hồi tiếc với heap; ví dụ mẫu cho mẫu “nhận hết rồi loại”.
••• Sắp thứ tự công việc trên hai máy để tổng thời gian nhỏ nhất	Bài kinh điển	Tiêu chí sắp xếp lại; hãy dùng mẹo đổi chỗ hai phần tử liền kề để tự dẫn ra.
••• Phủ tập hợp bằng tham lam, so với tối ưu trên dữ liệu nhỏ	Bài tập thực nghiệm	Đo tỉ số thực tế so với cận $\ln n$; đây là cách tốt nhất để hiểu “bảo đảm xấp xỉ” nghĩa là gì.
••• Chọn k vị trí đặt cảm biến để phủ nhiều nhất	Bài tập ứng dụng	Hàm mục tiêu dưới mô-đun; tham lam có bảo đảm $1 - 1/e$ — kiểm chứng bằng thực nghiệm.
••• Tìm một hệ mệnh giá làm tham lam đổi tiền sai, càng nhỏ càng tốt	Bài tập thiết kế	Cách rẻ nhất là stress test với vết cạn; bài này luyện đúng thói quen chương E/25.
••• Ghép đôi ổn định	Bài kinh điển	Cài thuật toán cầu hôn; thử chứng minh không tồn tại cặp muốn bỏ nhau — một chứng minh khó hơn vẻ ngoài của thuật toán.

Kết chương và đọc tiếp

Tham lam là chiến lược ngắn nhất về code và đắt nhất về nghĩa vụ chứng minh. Ba điều đáng mang theo: phát biểu chính xác về lý do tham lam sai — phá huỷ khả năng tương lai; mẹo đổi chỗ hai phần tử liền kề để *suy ra* tiêu chí sắp xếp thay vì đoán; và ý thức rằng có một lớp cấu trúc (matroid) mà trên đó tham lam chắc chắn đúng.

Khi không lựa chọn cục bộ nào an toàn, ta buộc phải xét nhiều khả năng — nhưng không phải xét lại từ đầu mỗi lần. Chương tiếp theo là chương dài nhất và quan trọng nhất của phần C: quy hoạch động, kỹ thuật biến “xét mọi khả năng” từ hàm mũ thành đa thức bằng cách nhận ra rằng rất nhiều nhánh khác nhau dẫn tới cùng một trạng thái (C/16).

Quy hoạch động (Dynamic Programming)

PHỤ THUỘC C/13 cho lời giải vét cạn làm điểm xuất phát; C/15 cho ranh giới “khi nào tham lam đủ”; A/04 cho lập luận cắt-dán chứng minh cấu trúc con tối ưu.

TÓM TẮT

Quy hoạch động là kỹ thuật quan trọng nhất của phần C và cũng là kỹ thuật bị dạy tệ nhất. Nó không phải một danh sách bài mẫu cần thuộc; nó là một quy trình năm bước — định nghĩa trạng thái, viết công thức chuyển, xác định cơ sở, chọn thứ tự tính, đọc đáp án — trong đó bước một chiếm chín phần mười độ khó. Chương đi qua sáu họ trạng thái hay gặp (một chiều, ba lô, hai chuỗi, khoảng, trên cây, mặt nạ bit), cách giảm bộ nhớ, cách truy vết lời giải, và bốn kỹ thuật tăng tốc khi số trạng thái nhân số chuyển quá lớn. Nếu chỉ nhớ một điều: trạng thái phải *đủ* — biết trạng thái thì phần tương lai không cần biết gì thêm về quá khứ — và mọi lỗi quy hoạch động đều quy về việc vi phạm đúng câu này.

KIẾN THỨC CẦN CÓ TRƯỚC

- **trạng thái (state)** — bản tóm tắt của quá khứ, đủ để quyết định tương lai. Nếu hai lịch sử khác nhau dẫn tới cùng trạng thái thì phần còn lại của bài toán là như nhau.
- **cấu trúc con tối ưu** — lời giải tối ưu của bài lớn chứa lời giải tối ưu của các bài con; chứng minh bằng lập luận cắt-dán (A/04).
- **bài toán con chồng lặp** — các nhánh khác nhau của cây đệ quy gặp lại cùng bài toán con — đây là lý do việc ghi nhớ có tác dụng.
- **ghi nhớ (memoization)** — cách viết từ trên xuống: hàm đệ quy lưu kết quả đã tính vào bảng, lần sau tra thay vì tính lại.
- **bảng từ dưới lên (tabulation)** — cách viết lặp: điền bảng theo một thứ tự bảo đảm mọi phụ thuộc đã có sẵn.
- **mảng cuộn (rolling array)** — kỹ thuật giảm bộ nhớ: nếu tầng i chỉ phụ thuộc tầng $i - 1$ thì chỉ cần giữ hai tầng.

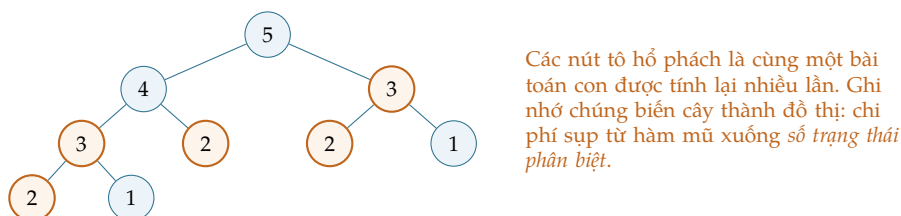
CÂU HỎI MỞ ĐẦU

- Q1. Quy hoạch động chỉ là đệ quy cộng một cái bảng. Vì sao nó lại khó tới vậy với hầu hết người học?
- Q2. Vì sao “chọn sai trạng thái” là lỗi phổ biến nhất, và làm sao biết trạng thái của mình *đủ* trước khi cài?
- Q3. Bài ba lô có lời giải $O(nW)$, nhìn thì đa thức. Vì sao bài ba lô vẫn được xếp vào NP-khó?
- Q4. Viết từ trên xuống (ghi nhớ) và từ dưới lên (điền bảng) cho cùng kết quả. Khi nào phải chọn cách nào?
- Q5. Khi số trạng thái đã tối thiểu mà vẫn quá chậm, còn hướng nào ngoài việc đổi thuật toán?

Người học quy hoạch động thường đi qua ba giai đoạn. Giai đoạn một: đọc lời giải thấy hiển nhiên, tự làm thì không nghĩ ra. Giai đoạn hai: nhận ra một số dạng quen thuộc và giải được các bài gần với chúng, nhưng gặp bài lạ thì lại đứng. Giai đoạn ba: nhìn bài mới và tự đặt ra được

trạng thái. Khoảng cách giữa giai đoạn hai và ba là khoảng cách giữa “thuộc bài mẫu” và “nắm phương pháp”, và chương này viết cho đúng khoảng cách đó.

Điểm khởi đầu phải là chương C/13: quy hoạch động không phải kỹ thuật độc lập mà là *vết cạn cộng trí nhớ*. Bạn viết lời giải đệ quy duyệt mọi khả năng — đúng nhưng chậm — rồi quan sát rằng rất nhiều nhánh khác nhau của cây đệ quy dẫn tới cùng một tình huống. Nếu tình huống đó mô tả được gọn thì lưu kết quả lại, và chi phí sụp từ hàm mũ xuống bằng *số tình huống phân biệt*. Cả chương này chỉ là các cách trả lời câu hỏi “tình huống được mô tả gọn bằng cái gì”.



Các nút tô hổ phách là cùng một bài toán con được tính lại nhiều lần. Ghi nhớ chúng biến cây thành đồ thị: chi phí sụp từ hàm mũ xuống *số trạng thái phân biệt*.

Hình 16.1. Cây đệ quy của một lời giải vết cạn. Quy hoạch động không phải kỹ thuật mới mà chỉ là quan sát rằng cây này có rất nhiều nút lặp lại. Toàn bộ độ khó nằm ở chỗ mô tả “nút nào giống nút nào” cho gọn — tức là ở việc chọn trạng thái.

16.1 Quy trình năm bước

Mọi lời giải quy hoạch động đều được viết theo cùng năm bước. Viết chúng ra thành lời *trước khi gõ code* là thói quen tạo ra khác biệt lớn nhất, đúng tinh thần chương A/04.

Bước 1 — định nghĩa trạng thái. Viết một câu hoàn chỉnh dạng “ $f[i][j]$ là ...”. Câu này phải rõ tới mức người khác đọc xong tính được ví dụ nhỏ bằng tay. Đây là bước chiếm chín phần mười độ khó, và mục 2 dành riêng cho nó.

Bước 2 — công thức chuyển. Từ trạng thái hiện tại, liệt kê *mọi* quyết định khả dĩ và trạng thái mà mỗi quyết định dẫn tới. Yêu cầu quan trọng: các nhánh phải *phủ hết* và tốt nhất là *không chồng nhau* (nếu đang đếm).

Bước 3 — cơ sở. Trạng thái nhỏ nhất và giá trị của nó. Rất nhiều lỗi nằm ở đây, đặc biệt việc phân biệt “không có cách nào” với “có đúng một cách rỗng”.

Bước 4 — thứ tự tính. Phải bảo đảm khi tính một trạng thái thì mọi trạng thái nó phụ thuộc đã xong. Nói cách khác, đồ thị phụ thuộc phải không có chu trình — và khi nó có chu trình thì bạn đang ở bài toán khác, cần Dijkstra hoặc giải hệ phương trình.

Bước 5 — đáp án và truy vết. Đáp án nằm ở trạng thái nào; nếu cần chỉ ra lời giải cụ thể chứ không chỉ giá trị, hãy lưu quyết định tại mỗi trạng thái hoặc dò ngược bằng cách so sánh giá trị.

16.2 Bước khó nhất: trạng thái phải đủ

Tiêu chuẩn duy nhất cho một trạng thái tốt: *biết trạng thái thì phần còn lại của bài toán hoàn toàn xác định, không cần biết thêm bất cứ điều gì về cách ta tới đó*. Trong ngôn ngữ xác suất, đây là tính chất Markov; trong ngôn ngữ chương A/04, đây chính là một bất biến.

Có ba cách phát hiện trạng thái thiếu, và cả ba đều rẻ hơn việc cài rồi mới sai.

Cách thứ nhất — thử kể lại tương lai. Đặt câu hỏi: “tôi đang ở trạng thái $f[i]$, tôi cần quyết định gì tiếp theo, và tôi có đủ thông tin để quyết định không?” Nếu câu trả lời là “còn tùy vào việc trước đó tôi đã chọn gì” thì trạng thái thiếu đúng cái đó, và cái đó phải trở thành một chiều mới.

Cách thứ hai — tìm hai lịch sử cùng trạng thái nhưng khác tương lai. Nếu dựng được hai cách đi tới cùng $f[i]$ mà phần tối ưu còn lại khác nhau thì trạng thái chắc chắn thiếu. Đây là phản ví dụ, và như chương A/04 đã nói, một phản ví dụ đủ để bác bỏ.

Cách thứ ba — kiểm tra cắt-dán. Nếu bạn không chứng minh được rằng thay một phần lời giải bằng lời giải tối ưu của bài con thì kết quả không tệ đi, thì cấu trúc con tối ưu không có — và chỗ chứng minh hỏng chỉ ra chiều còn thiếu. Ví dụ kinh điển: đường đi *dài nhất* không lập đỉnh trong đồ thị tổng quát; ghép hai đường dài nhất có thể trùng đỉnh, nên trạng thái phải mang theo tập đỉnh đã đi — và đó là lý do bài này chỉ giải được bằng mặt nạ bit với n nhỏ.

CẦN NHỚ

Ba câu hỏi để kiểm trạng thái trước khi cài: (i) tôi phát biểu được $f[\cdot]$ thành một câu đầy đủ không; (ii) từ trạng thái, tôi quyết định được bước tiếp theo mà không cần nhìn lại quá khứ chứ; (iii) hai lịch sử khác nhau tới cùng trạng thái có thật sự cho cùng tương lai không. Trả lời được cả ba thì phần còn lại là kỹ thuật.

16.3 Sáu họ trạng thái hay gặp

Thay vì học từng bài, hãy học sáu họ này — phần lớn bài quy hoạch động trong thi đấu và phỏng vấn là biến thể của chúng.

Họ 1 — một chiều theo vị trí. $f[i]$ là đáp án tối ưu xét tới phần tử i , thường kèm điều kiện “có dùng phần tử i ”. Đại diện: dây con tăng dài nhất, chia dây thành các đoạn, bài trộm nhà không lấy hai nhà liền kề. Dấu hiệu: quyết định tại mỗi vị trí chỉ phụ thuộc vài vị trí gần trước đó.

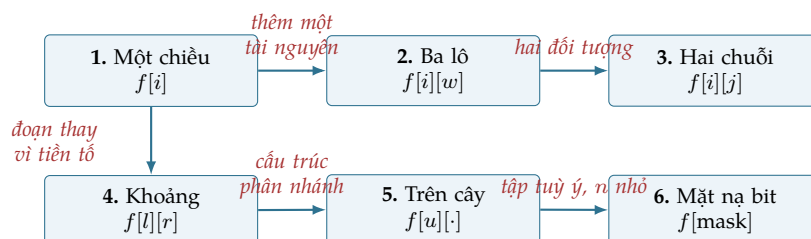
Họ 2 — ba lô: vị trí kèm một tài nguyên. $f[i][w]$ với w là lượng tài nguyên đã dùng. Đại diện: ba lô 0/1, đổi tiền, chia tập thành hai phần cân bằng. Dấu hiệu: có một đại lượng tích lũy bị giới hạn.

Họ 3 — hai chuỗi. $f[i][j]$ xét i ký tự đầu của chuỗi này và j ký tự đầu của chuỗi kia. Đại diện: dây con chung dài nhất, khoảng cách chỉnh sửa, so khớp có ký tự đại diện. Dấu hiệu: bài toán so hai dãy với nhau.

Họ 4 — khoảng. $f[l][r]$ là đáp án cho đoạn $[l, r]$, tính theo độ dài tăng dần. Đại diện: nhân dãy ma trận, cắt gậy, chuỗi đối xứng, trò chơi lấy đồ ở hai đầu. Dấu hiệu: bài toán có cấu trúc lồng nhau, hoặc quyết định chia đoạn tại một điểm giữa.

Họ 5 — trên cây. $f[u][\cdot]$ là đáp án cho cây con gốc u ; tính từ lá lên. Đại diện: tập độc lập lớn nhất trên cây, phủ đỉnh, đường kính, đếm cách tô màu. Dấu hiệu: dữ liệu là cây, và điều kiện chỉ ràng buộc giữa cha và con.

Họ 6 — mặt nạ bit. $f[\text{mask}][\cdot]$ với mask mô tả tập đã dùng. Đại diện: người bán hàng, ghép cặp hoàn hảo, phủ hình. Dấu hiệu: $n \leq 20$ và bài yêu cầu “mỗi phần tử dùng đúng một lần”.



Hình 16.2. Sáu họ trạng thái, xếp theo mức độ thông tin phải mang theo. Đi từ trái sang phải và từ trên xuống dưới, trạng thái ngày càng nặng và số trạng thái ngày càng lớn. Nguyên tắc chọn: dùng họ *nhẹ nhất* còn đủ mô tả tương lai — cùng nguyên tắc “dùng cấu trúc yếu nhất còn đủ dùng” của phần B.

Ngoài sáu họ trên còn ba họ chuyên hơn, nên biết là có: *quy hoạch động trên chữ số* cho các bài đếm số trong khoảng thoả điều kiện; *quy hoạch động kỳ vọng* cho các bài xác suất, ở đó thứ tự tính đôi khi phải đi ngược hoặc phải giải hệ phương trình; và *quy hoạch động trò chơi*, ở đó giá trị của trạng thái được định nghĩa qua cực đại và cực tiểu xen kẽ.

16.4 Ba lô và câu hỏi “vì sao vẫn NP-khó”

Bài ba lô 0/1: n món, món i nặng w_i và có giá trị v_i ; chọn tập con nặng không quá W sao cho tổng giá trị lớn nhất. Trạng thái $f[i][w]$ = giá trị lớn nhất khi xét i món đầu và sức chứa còn w . Chuyển: lấy hoặc không lấy món i . Chi phí $O(nW)$.

Đây là chỗ nên dừng lại vì nó dạy một điều quan trọng về độ phức tạp. $O(nW)$ nhìn thì đa thức, nhưng W được ghi trong dữ liệu vào bằng $\log W$ bit. Nếu $W = 10^{18}$ thì kích thước dữ liệu vào chỉ tăng thêm vài chục bit trong khi chi phí tăng lên 10^{18} . Nói cách khác, thuật toán đa thức theo *giá trị* nhưng mũ theo *kích thước biểu diễn* — người ta gọi đó là **giả đa thức** (*pseudo-polynomial*). Bài ba lô vẫn là NP-khó, và điều đó hoàn toàn nhất quán với sự tồn tại của lời giải $O(nW)$. Chi tiết ở chương D/23.

Hệ quả thực hành: khi thấy trạng thái chứa một *giá trị* thay vì một chỉ số, hãy kiểm tra ngay giá trị đó lớn tới đâu. Nếu W lớn nhưng tổng giá trị nhỏ thì hãy đảo vai trò — đặt trạng thái theo giá trị và tối thiểu hoá khối lượng. Mẹo “đảo vai trò của mục tiêu và ràng buộc” này giải được rất nhiều bài trông bề tặc.

16.5 Từ trên xuống hay từ dưới lên

Hai cách viết cho cùng kết quả, nhưng khác nhau ở bốn điểm thực dụng.

Ghi nhớ (từ trên xuống) gần với cách bạn nghĩ, nên viết nhanh và ít sai logic hơn; nó chỉ tính những trạng thái thực sự cần, điều rất quý khi không gian trạng thái thừa; và nó không đòi bạn nghĩ ra thứ tự tính. Nhược điểm: chi phí gọi hàm, nguy cơ tràn ngăn xếp khi độ sâu lớn, và khó áp dụng các tối ưu bộ nhớ.

Điền bảng (từ dưới lên) nhanh hơn theo hằng số, không tràn ngăn xếp, và cho phép giảm bộ nhớ bằng mảng cuộn. Nhược điểm: bạn phải tự xác định thứ tự tính đúng, và nó tính cả những trạng thái không cần.

Quy tắc thực dụng: nghĩ bằng ghi nhớ, và chuyển sang điền bảng khi cần tốc độ hoặc cần giảm bộ nhớ. Trong kỳ thi, nếu không gian trạng thái dày và thứ tự rõ ràng thì viết thẳng bảng.

Giảm bộ nhớ. Nếu tầng i chỉ phụ thuộc tầng $i - 1$ thì chỉ giữ hai tầng — hoặc thậm chí một tầng nếu duyệt đúng chiều, như trong ba lô 0/1 khi duyệt w giảm dần. Kỹ thuật này biến $O(nW)$ bộ nhớ thành $O(W)$ và thường là thứ quyết định bài có chạy nổi hay không. Cái giá: bạn mất khả năng truy vết lời giải cụ thể, vì các tầng cũ đã bị ghi đè. Khi cần cả hai, dùng kỹ thuật chia để trị của Hirschberg cho họ hai chuỗi, hoặc lưu quyết định dưới dạng bit.

16.6 Bốn kỹ thuật tăng tốc

Chi phí quy hoạch động bằng *số trạng thái nhân chi phí mỗi chuyển*. Khi số trạng thái đã tối thiểu mà vẫn chậm, phải tấn công thừa số thứ hai. Bốn kỹ thuật, xếp theo mức độ hay gặp:

Hàng đợi đơn điệu. Khi chuyển có dạng “lấy cực trị của $f[j]$ với j trong một cửa sổ trượt”, dùng deque đơn điệu (B/07) để mỗi trạng thái tốn $O(1)$ khấu hao thay vì $O(k)$.

Cấu trúc dữ liệu cho truy vấn đoạn. Khi chuyển lấy cực trị hoặc tổng trên một đoạn tùy ý, dùng cây phân đoạn hoặc Fenwick (B/10) để giảm từ $O(n)$ xuống $O(\log n)$. Đây là cách tăng tốc dãy con tăng dài nhất từ $O(n^2)$ xuống $O(n \log n)$.

Bao lồi. Khi chuyển có dạng $f[i] = \min_j (f[j] + a_j \cdot b_i)$ — tức là mỗi j đóng góp một đường thẳng theo biến b_i — thì việc lấy cực tiểu trên tập đường thẳng chính là bài bao lồi, và có cấu trúc trả lời trong $O(\log n)$ hoặc $O(1)$ khi các hệ số đơn điệu.

Tối ưu chia để trị và tối ưu Knuth. Khi điểm chuyển tối ưu đơn điệu theo trạng thái — tức là tối ưu của trạng thái sau không bao giờ nằm bên trái tối ưu của trạng thái trước — ta tính các trạng thái theo kiểu chia để trị và giảm một thừa số n . Điều kiện đơn điệu này thường phải chứng minh hoặc kiểm bằng thực nghiệm.

Có một hướng thứ năm ít được nhắc nhưng rất mạnh: *giảm số trạng thái bằng một quan sát tham lam*. Nếu chứng minh được rằng một lớp lựa chọn không bao giờ tối ưu, hãy loại chúng khỏi không gian trạng thái. Đây là chỗ chương C/15 và chương này gặp nhau, và nó thường hiệu quả hơn mọi tối ưu kỹ thuật.

16.7 Vì sao điểm tối ưu lại đơn điệu: bất đẳng thức tứ giác

Kỹ thuật thứ tư ở mục trên có một câu bỏ lửng: “điều kiện đơn điệu này thường phải chứng minh hoặc kiểm bằng thực nghiệm”. Có một điều kiện đủ, phát biểu được trong một dòng, và biết nó thì bạn không còn phải đoán.

Xét lớp truy hồi rất hay gặp ở họ khoảng:

$$f(i, j) = w(i, j) + \min_{i < k < j} (f(i, k) + f(k, j)),$$

với w là chi phí gộp. Nói w thoả **bất đẳng thức tứ giác** (*quadrangle inequality*) nghĩa là với mọi $a \leq b \leq c \leq d$,

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c).$$

Đọc bằng lời: *hai đoạn lồng nhau không đắt hơn hai đoạn cắt nhau*. Đây đúng là hình ảnh của lập luận đổi chỗ ở chương A/04 — nếu hai lựa chọn tối ưu “bắt chéo” nhau, ta gỡ chéo được mà không làm tệ đi — và chính nó kéo theo tính đơn điệu của điểm chia tối ưu.

Lịch sử của điều kiện này minh hoạ cách một quan sát riêng lẻ trở thành công cụ chung. Knuth[60] nhận ra tính đơn điệu ấy khi tính cây tìm kiếm nhị phân tối ưu và nhờ đó hạ $O(n^3)$ xuống $O(n^2)$; hơn mười năm sau, Yao[61] tách bất đẳng thức tứ giác ra khỏi bài toán cụ thể và chứng minh nó là điều kiện đủ cho cả lớp truy hồi trên. Từ chỗ “một mẹo cho một bài”, nó thành “một định lý cho một họ bài” — và đó là lý do đáng học nó ở dạng tổng quát thay vì học thuộc bài cây tìm kiếm.

Đi xa hơn một bước là thuật toán SMAWK[62], đặt theo chữ đầu của năm tác giả. Khi ma trận chi phí *đơn điệu toàn phần* — một điều kiện họ hàng với bất đẳng thức tứ giác — thì cực tiểu của mọi hàng tìm được trong thời gian *tuyến tính* theo kích thước ma trận, không có cả thừa số lôgarit. Chi tiết đáng chú ý: bài báo này ra đời trong hình học tính toán, cho bài tìm đỉnh xa nhất từ mỗi đỉnh của một đa giác lồi, rồi mới di cư sang quy hoạch động. Lại một lần nữa, cùng cấu trúc toán học xuất hiện ở hai miền không liên quan — đúng điều mà mục liên hệ ngang của mọi chương trong quyển này cố chỉ ra.

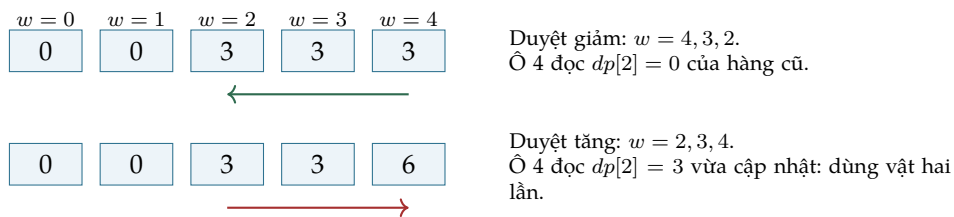
Còn một mảnh nữa hoàn thiện bức tranh, và nó trả lời câu hỏi “đã nén bộ nhớ thì truy vết thế nào” đã hẹn ở mục 5. Hirschberg[63] lấy họ hai chuỗi làm ví dụ: tính bảng từ trái sang tới cột

giữa và từ phải sang tới cột giữa, cộng hai kết quả lại thì biết lời giải tối ưu đi qua ô nào của cột giữa; cắt bài toán tại ô đó và đệ quy trên hai nửa. Tổng chi phí vẫn $O(nm)$ vì mỗi tầng đệ quy làm việc trên một nửa diện tích, còn bộ nhớ chỉ $O(\min(n, m))$. Ý tưởng “không lưu vết, mà tính lại có tổ chức” này dùng được cho nhiều họ trạng thái khác, không riêng hai chuỗi.

Lời khuyên thực hành khép lại mục: trước khi tin vào tính đơn điệu, hãy *kiểm* nó — viết một đoạn vết cận nhỏ tính điểm chia tối ưu cho mọi (i, j) với n cỡ vài chục và kiểm tra trực tiếp bất đẳng thức tứ giác trên hàm w của bạn. Năm phút đó rẻ hơn nhiều so với việc gỡ một lời giải sai chỉ trên vài phần trăm bộ test (E/25).

16.8 Cùng một công thức, đổi chiều duyệt là đổi bài toán

Xét một vật duy nhất có trọng lượng 2, giá trị 3 và sức chứa 4. Với trạng thái “giá trị tốt nhất khi sức chứa không quá w ”, khởi tạo mọi ô bằng 0. Công thức nén là $dp[w] \leftarrow \max(dp[w], dp[w-2]+3)$. Hình 16.3 cho thấy chiều duyệt quyết định ô bên phải đọc giá trị của hàng cũ hay hàng mới.



Hình 16.3. Hai mảng sau khi xử lý cùng một vật. Hàng trên đúng cho ba lô 0/1; hàng dưới đúng cho ba lô không giới hạn. Giá trị 6 không phải lỗi cộng mà là lỗi phụ thuộc nếu đề chỉ cho lấy vật một lần.

Bảng 16.1. Ba dạng ba lô: cùng tài nguyên nhưng khác tập quyết định hợp lệ. Giả sử trọng lượng nguyên dương.

Dạng	Giới hạn mỗi vật	Cách duyệt / xử lý	Test phân biệt
0/1	Tối đa một lần	Duyệt sức chứa giảm, $O(nW)$	Một vật, $W = 2w_i$: không được lấy hai lần
Không giới hạn	Bao nhiêu lần cũng được	Duyệt sức chứa tăng, $O(nW)$	Một vật, $W = 2w_i$: được lấy hai lần
Có số lượng	Tối đa c_i lần	Tách nhị phân thành vật 0/1, $O(W \sum_i \log(c_i + 1))$	Sức chứa đủ cho $c_i + 1$ vật: vẫn chỉ lấy c_i

Bảng 16.1 giúp chọn biến thể trước khi nén bộ nhớ. Nếu trạng thái yêu cầu trọng lượng *đúng bằng* w , chỉ $dp[0] = 0$, các ô còn lại là không đạt được ($-\infty$ trong bài cực đại), và không cộng vào trạng thái không đạt được. Khi chưa chắc, giữ bảng hai chiều làm bản đối chiếu; chỉ nén sau khi xác định rõ mỗi phép đọc thuộc hàng nào.

16.9 Phản biện, nhược điểm và rủi ro triển khai

Dù quy hoạch động là một trong những công cụ đẹp nhất của khoa học máy tính, việc thần thánh hoá nó như một giải pháp vạn năng cho mọi bài toán tối ưu là một sai lầm nguy hiểm trong kỹ thuật. Để dùng nó một cách trưởng thành, cần nhìn thẳng vào ba phương diện: khi nào nó bị các phương pháp khác đánh bại, nó mang nhược điểm cấu trúc gì, và những lỗi im lặng nào chực chờ làm sập hệ thống khi chạy thật.

Phản biện: sự lạm dụng và ranh giới áp dụng. Quy hoạch động thực chất là một phép vét cạn có tổ chức: nó vẫn buộc phải ghé thăm toàn bộ các trạng thái khả dĩ. Nếu bài toán sở hữu cấu trúc matroid (C/15), thuật toán tham lam tìm ra nghiệm tối ưu chỉ bằng một lượt quét đơn giản với bộ nhớ gần như $O(1)$. Nếu hàm mục tiêu có tính lồi hoặc đơn điệu, tìm kiếm nhị phân trên đáp án (C/19) hoặc kỹ thuật hai con trỏ (B/06) cho tốc độ vượt trội gấp nhiều bậc mà không cần lưu trữ bảng trạng thái. Đặc biệt, khi bài toán chứa ràng buộc toàn cục mang tính phi Markov — ví dụ như chi phí bước sau phụ thuộc vào phương sai của toàn bộ chuỗi lựa chọn trước đó — cấu trúc con tối ưu sụp đổ hoàn toàn. Cố tình nhồi nhét toàn bộ lịch sử vào trạng thái (như dùng mặt nạ bit) sẽ đẩy chi phí lên hàm mũ, biến quy hoạch động thành một cách viết rườm rà của thuật toán quay lui thô thiển.

Nhược điểm cốt lõi về tài nguyên và phần cứng. Có ba cái giá cấu trúc luôn đi kèm quy hoạch động:

- *Lời nguyền số chiều (Curse of Dimensionality):* Mỗi ràng buộc phụ của bài toán đòi hỏi mở thêm một chiều trạng thái mới. Số lượng ô nhớ tăng theo tích số các chiều; chỉ cần thêm hai tham số phụ với miền giá trị cỡ 10^3 , bộ nhớ lập tức phình to từ megabyte thành gigabyte, vượt ngưỡng bộ nhớ của mọi máy thi đấu hay thiết bị nhúng.
- *Phá vỡ phân cấp bộ nhớ và tính bất khả thi của SIMD:* Khác với các phép nhân ma trận có thể sắp xếp khối để tối ưu cache L1/L2, truy cập bảng quy hoạch động thường nhảy cóc theo bước nhảy của dữ liệu (như $w - w_i$ trong ba lô) hoặc theo con trỏ cây, gây trượt cache liên tục. Quan trọng hơn, do bản chất của hệ thức truy hồi luôn chứa phụ thuộc dữ liệu theo vòng lặp (loop-carried dependency: bước sau bắt buộc phải đợi bước trước hoàn thành), các compiler hiện đại gần như bất lực trong việc tự động vector hoá (SIMD) hay phân tán tính toán ra hàng nghìn nhân GPU.
- *Mất mát thông tin khi nén bộ nhớ:* Kỹ thuật mảng cuộn cứu được bộ nhớ nhưng triệt tiêu toàn bộ đường dẫn quyết định. Trong công việc thực tế, một hệ thống chỉ đưa ra con số chi phí tối ưu mà không thể chỉ ra phương án hành động cụ thể là một hệ thống vô dụng. Khôi phục lại đường đi buộc phải trả giá bằng việc nhân đôi thời gian chạy qua thuật toán chia để trị kiểu Hirschberg hoặc cài đặt các cấu trúc bit phức tạp.

Rủi ro vận hành và các chế độ hồng im lặng. Trong môi trường sản xuất và thi đấu, các lỗi của quy hoạch động hiếm khi ném ra ngoại lệ rõ ràng, mà chúng âm thầm trả về kết quả sai:

- *Tràn số khi gán vô cùng (∞):* Để tìm cực tiểu, ta thường khởi tạo các trạng thái không hợp lệ bằng một giá trị vô cùng lớn. Nếu chọn giá trị này bất cẩn (ví dụ dùng $0x7fffffff$ trong số nguyên 32-bit có dấu), phép tính $f[j] + cost$ sẽ bị tràn số số học thành một số âm rất lớn. Thuật toán sẽ nhầm tưởng trạng thái cấm đó là một lời giải siêu tối ưu, làm sai lệch hoàn toàn kết quả toàn cục. Quy tắc an toàn: dùng hằng số vừa đủ lớn (như $0x3f3f3f3f$ cho 32-bit hoặc 10^{18} cho 64-bit) để phép cộng hai số vô cùng không bị tràn.
- *Tràn số trong các bài toán đếm cách:* Số cách thực hiện có xu hướng bùng nổ theo cấp số nhân. Quên lấy dư modulo ở chỉ một phép cộng dồn trung gian, hoặc cộng hai số đã lấy modulo mà không kiểm tra xem tổng có vượt giới hạn kiểu dữ liệu hay không, sẽ gây tràn số âm thầm.
- *Tràn ngăn xếp đệ quy (Stack Overflow):* Dùng đệ quy có nhớ (memoization) trên các đồ thị trạng thái có chuỗi dài (như chuỗi $n \geq 10^5$ phần tử) sẽ nhanh chóng vượt quá dung lượng call stack mặc định của hệ điều hành (thường chỉ 8 MB trên Linux), gây sập chương trình đột ngột (Segmentation Fault) ngay trên dữ liệu lớn.
- *Hiện tượng underflow trong quy hoạch động xác suất:* Trong các bài toán xử lý tín hiệu hay thị giác (thuật toán Viterbi, HMM), việc nhân dồn lặp các xác suất $p \in (0, 1)$ qua hàng trăm bước thời gian sẽ đẩy giá trị về số 0 dấu phẩy động (underflow). Lời giải bắt buộc trong thực tế là chuyển toàn bộ sang miền logarit ($\ln p$) và thay phép nhân xác suất bằng phép cộng, kết hợp hàm log-sum-exp để tránh mất mát độ chính xác số học.

CẠM BÃY

Đừng bao giờ tối ưu bộ nhớ bằng mảng cuộn khi chưa kiểm thử xong bản hai chiều hoàn chỉnh. Mảng cuộn nén bộ nhớ nhưng làm mờ ranh giới thời gian giữa trạng thái cũ và trạng thái mới, khiến việc gỡ lỗi logic chuyển trạng thái trở nên khó gấp bội. Hãy luôn giữ một bản đầy đủ để đối chiếu và kiểm chứng tính đúng trước khi tối ưu hoá sớm.

16.10 Gốc rễ: một cái tên được chọn để nguy trang

Quy hoạch động do Richard Bellman phát triển tại RAND Corporation trong thập niên 1950, và câu chuyện về cái tên của nó đáng kể lại vì nó nói nhiều về bối cảnh nghiên cứu thời đó. Trong hồi ký của mình, Bellman kể rằng ông làm việc dưới sự tài trợ của Không quân Mỹ, và vị bộ trưởng quốc phòng khi đó có ác cảm với từ “nghiên cứu” và đặc biệt với “toán học”. Ông cần một cái tên nghe có vẻ thực dụng và không thể phản đối. “Dynamic” nghe tích cực và có nghĩa thời gian; “programming” thời đó mang nghĩa lập kế hoạch, như trong “quy hoạch tuyến tính”. Kết quả là một cái tên gần như không mô tả gì về kỹ thuật — điều gây khó khăn cho người học suốt bảy chục năm sau.

Nội dung thật sự thì nghiêm túc hơn cái tên nhiều. Bellman đưa ra *nguyên lý tối ưu*: một chính sách tối ưu có tính chất là, dù trạng thái ban đầu và quyết định đầu tiên là gì, các quyết định còn lại phải tạo thành chính sách tối ưu cho trạng thái sinh ra bởi quyết định đầu. Phát biểu đó chính là cấu trúc con tối ưu, và phương trình Bellman đi kèm là công thức chuyển ở dạng tổng quát nhất. Điều đáng nói là nó ra đời cho các bài toán *điều khiển liên tục theo thời gian* — quản lý tồn kho, điều khiển tên lửa — chứ không phải cho bài tập tổ hợp; các ứng dụng rời rạc là hệ quả về sau.

Ba nhánh ứng dụng lớn ra đời gần như đồng thời trong thập niên 1960–70, mỗi nhánh từ một ngành khác nhau, và cả ba đều là quy hoạch động dù người phát minh không luôn dùng từ đó. Trong *viễn thông*, Viterbi công bố năm 1967 thuật toán giải mã cho mã chập, về sau trở thành thuật toán chuẩn để tìm chuỗi trạng thái khả dĩ nhất của mô hình Markov ẩn — nền của nhận dạng tiếng nói suốt nhiều thập kỷ. Trong *tin sinh học*, Needleman và Wunsch công bố năm 1970 thuật toán căn chỉnh hai chuỗi protein — chính là bài khoảng cách chỉnh sửa với ma trận điểm số sinh học. Trong *ngôn ngữ học tính toán*, thuật toán phân tích cú pháp CYK dùng quy hoạch động trên khoảng để phân tích câu.

Sự trùng hợp này không phải ngẫu nhiên. Cả ba bài toán đều có cùng cấu trúc: một chuỗi quyết định, mỗi quyết định phụ thuộc trạng thái tóm tắt được, và mục tiêu cộng dồn. Bất cứ khi nào ba điều kiện đó xuất hiện — ở bất kỳ ngành nào — quy hoạch động sẽ được phát minh lại.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Thập niên 1950 — Bellman (RAND)	Điều khiển tối ưu nhiều giai đoạn: tồn kho, phân bổ tài nguyên, dẫn đường.	Nguyên lý tối ưu và phương trình Bellman; cái tên “dynamic programming” được chọn để tránh bị cắt tài trợ vì nghe giống “toán học”.
1962 — Held & Karp	Bài người bán hàng với n vừa phải.	Quy hoạch động trên mặt nạ bit, $O(2^n n^2)$ — vẫn là thuật toán chính xác tốt nhất đã biết cho bài này.
1965–70 — Levenshtein; Needleman & Wunsch	So sánh chuỗi: mã sửa lỗi, rồi căn chỉnh chuỗi protein.	Khoảng cách chỉnh sửa; nền của cả kiểm tra chính tả lẫn tin sinh học hiện đại.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
1967 — Viterbi	Giải mã mã chập trong truyền tin có nhiễu.	Thuật toán Viterbi; về sau thành công cụ chuẩn cho mô hình Markov ẩn trong nhận dạng tiếng nói và xử lý tín hiệu.
Thập niên 1960 — CYK	Phân tích cú pháp câu theo văn phạm phi ngữ cảnh.	Quy hoạch động trên khoảng; cùng khuôn với bài nhân dây ma trận.
Thập niên 1980–nay — học tăng cường	Ra quyết định tuần tự khi mô hình môi trường không biết trước.	Phương trình Bellman trở thành nền của học tăng cường; “hàm giá trị” chính là bảng quy hoạch động, chỉ khác là nó được <i>học</i> thay vì được điền.

16.11 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Học tăng cường	Phương trình Bellman, hàm giá trị, lặp giá trị	Cùng một phương trình. Khác: ở đây bảng được điền bằng cách duyệt hết trạng thái, ở đó hàm giá trị được <i>xấp xỉ</i> bằng mạng nơ-ron vì không gian trạng thái quá lớn để liệt kê.
Lý thuyết điều khiển	Điều khiển tối ưu, bộ điều khiển toàn phương tuyến tính	Bản liên tục của cùng nguyên lý; “trạng thái” ở đây đúng nghĩa trạng thái vật lý. Điều kiện Markov trong điều khiển tương ứng chính xác với yêu cầu “trạng thái phải đủ” ở mục 2.
Tin sinh học	Căn chỉnh chuỗi, gấp RNA	Khoảng cách chỉnh sửa với ma trận điểm số sinh học; bài gấp RNA là quy hoạch động trên khoảng. Đây là nơi quy hoạch động chạy ở quy mô lớn nhất trong khoa học.
Xử lý tiếng nói và tín hiệu	Viterbi trong mô hình Markov ẩn, xoắn thời gian động	Cùng thuật toán, khác cách gọi. Xoắn thời gian động là bài hai chuỗi với chi phí căn chỉnh — dùng để so hai chuỗi tín hiệu có tốc độ khác nhau.
Thị giác máy tính, stereo	So khớp ảnh theo dòng quét bằng quy hoạch động	Bài toán tìm độ lệch cho từng điểm ảnh trên một dòng, với ràng buộc tron giữa các điểm kề — đúng khuôn họ một chiều với chi phí chuyển. Cùng bài toán về sau được giải bằng cắt đồ thị (C/18), một ví dụ đẹp về hai công cụ cho cùng mô hình.
Tài chính	Định giá quyền chọn kiểu Mỹ bằng cây nhị thức	Đi ngược từ thời điểm đáo hạn về hiện tại, mỗi nút lấy cực đại giữa thực hiện ngay và giữ tiếp — đúng quy hoạch động trò chơi với thứ tự tính ngược thời gian.
Vận trù học	Quản lý tồn kho, lập lịch sản xuất	Bài toán gốc mà Bellman nhắm tới. Ràng buộc thực tế thêm vào là bất định về nhu cầu, dẫn tới quy hoạch động ngẫu nhiên — trạng thái mang theo phân phối thay vì giá trị.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

A1. Vì phần khó không nằm ở kỹ thuật mà nằm ở việc *đặt tên cho đúng thứ*. Đệ quy và bảng là chuyện cơ khí; cái khó là nhìn một bài toán và nhận ra “mọi lịch sử dẫn tới đây đều tương đương với nhau nếu ta chỉ nhớ hai con số này”. Đó là một bước trừu tượng hoá, và nó không

có quy trình máy móc. Thêm nữa, cách dạy phổ biến — trình bày công thức đã hoàn chỉnh — che mất chính bước này, khiến người học thấy lời giải hiển nhiên mà không tái tạo được.

- A2. Vì trạng thái thiếu vắng cho code chạy và vẫn ra số, chỉ là số sai — một lỗi im lặng. Ba cách kiểm tra trước khi cài: thử kể lại tương lai từ trạng thái và xem có phải nói “còn tùy vào trước đó” không; tìm hai lịch sử cùng trạng thái nhưng khác tương lai tối ưu; và kiểm lập luận cắt-dán, vì chỗ nó hỏng chỉ ra đúng chiều còn thiếu. Sau khi cài, cách kiểm rẻ nhất vẫn là stress test với vết cặn trên n nhỏ.
- A3. Vì W xuất hiện trong chi phí như một *giá trị*, trong khi kích thước dữ liệu vào chỉ chứa $\log W$ bit để biểu diễn nó. Với $W = 10^{18}$, dữ liệu vào vẫn ngắn nhưng thuật toán cần 10^{18} bước — tức là mũ theo kích thước biểu diễn. Loại thuật toán này gọi là giả đa thức, và sự tồn tại của nó hoàn toàn nhất quán với việc bài toán là NP-khó (D/23). Hệ quả thực hành: khi trạng thái chứa một giá trị chứ không phải chỉ số, hãy kiểm tra ngay biên của giá trị đó.
- A4. Chọn ghi nhớ khi không gian trạng thái *thưa* — chỉ một phần nhỏ trạng thái thực sự được dùng — hoặc khi thứ tự tính khó xác định, hoặc khi bạn đang trong giai đoạn nghĩ và muốn code gần với suy nghĩ. Chọn điền bảng khi cần tốc độ (không có chi phí gọi hàm), khi độ sâu đệ quy có thể gây tràn ngăn xếp, hoặc khi cần giảm bộ nhớ bằng mảng cuộn — kỹ thuật gần như không áp dụng cho bản ghi nhớ.
- A5. Còn ba hướng. Thứ nhất, tấn công chi phí mỗi chuyển bằng cấu trúc dữ liệu hoặc bằng các tối ưu ở mục 6: hàng đợi đơn điệu, cây phân đoạn, bao lồi, tính đơn điệu của điểm tối ưu. Thứ hai, giảm số trạng thái bằng một quan sát tham lam — chứng minh rằng cả một lớp lựa chọn không bao giờ tối ưu và loại chúng; hướng này thường hiệu quả nhất. Thứ ba, đổi vai trò các chiều, ví dụ đặt trạng thái theo giá trị thay vì theo khối lượng khi giá trị nhỏ hơn nhiều.

NÓI LẠI THẬT ĐƠN GIẢN

Quy hoạch động chỉ là hai chuyện gộp lại: thử mọi khả năng, và *ghi lại những gì đã tính để khỏi tính lại*.

Hãy nghĩ tới việc leo cầu thang mỗi bước một hoặc hai bậc, và bạn muốn đếm số cách leo lên bậc thứ mười. Nếu ngồi vẽ cây mọi khả năng thì rất nhiều nhánh trùng nhau: dù bạn tới bậc thứ bảy bằng đường nào, số cách đi tiếp từ đó vẫn y hệt. Vậy thay vì vẽ lại, hãy ghi “từ bậc bảy còn bao nhiêu cách” vào một tờ giấy và dùng lại. Số việc từ chỗ tăng gấp đôi mỗi bậc trở thành mỗi bậc một dòng ghi chép.

Câu quan trọng nhất trong chuyện trên là câu “dù bạn tới bậc thứ bảy bằng đường nào”. Đó chính là điều kiện để cách làm này chạy được: những gì đã xảy ra không còn ảnh hưởng gì tới tương lai, ngoài việc bạn đang đứng ở bậc nào. Nếu bài toán có thêm luật kiểu “không được đi hai bước dài liên tiếp” thì đứng ở bậc bảy chưa đủ — bạn còn phải nhớ bước vừa rồi dài hay ngắn. Lúc đó tờ giấy ghi chép phải có hai cột thay vì một.

Toàn bộ khó khăn của quy hoạch động nằm ở đúng chỗ đó: quyết định *tờ giấy cần ghi những gì*. Ghi thiếu thì kết quả sai một cách im lặng; ghi thừa thì tờ giấy to quá và chương trình chạy chậm. Mọi kỹ thuật còn lại trong chương — cách điền bảng, cách tiết kiệm giấy, cách tăng tốc — đều là chuyện phụ so với câu hỏi này.

Một mẹo kiểm tra rất hiệu quả: hãy tự kể lại phần còn lại của bài toán chỉ dựa vào những gì đã ghi. Nếu bạn thấy mình phải nói “còn tùy trước đó tôi làm gì”, thì đúng cái “tùy” đó là thứ còn thiếu.

Một câu: *Trạng thái phải đủ: biết trạng thái thì tương lai không cần biết gì thêm về quá khứ.*

Chỗ không rút gọn được: Việc nghĩ ra trạng thái đúng cho một bài mới vẫn là bước sáng tạo, không có quy trình máy móc — chỉ có các cách kiểm tra sau khi đã đoán.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Quy hoạch động là gì	Vết cặn cộng trí nhớ: các nhánh khác nhau gặp lại cùng bài toán con, nên lưu kết quả để khỏi tính lại. Chi phí bằng số trạng thái nhân chi phí mỗi chuyển.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Quy trình năm bước	Trạng thái → công thức chuyển → cơ sở → thứ tự tính → đáp án và truy vết. Bước một chiếm chín phần mười độ khó.
●●● Tiêu chuẩn của trạng thái tốt	Biết trạng thái thì tương lai hoàn toàn xác định, không cần biết cách tới đó. Đây là tính chất Markov, cũng là một bất biến.
●●● Ba cách phát hiện trạng thái thiếu	Kể lại tương lai và xem có phải nói “còn tuý”; tìm hai lịch sử cùng trạng thái khác tương lai; kiểm lập luận cắt-dán và xem nó hỏng ở đâu.
●●● Sáu họ trạng thái	Một chiều $f[i]$; ba lô $f[i][w]$; hai chuỗi $f[i][j]$; khoảng $f[l][r]$; trên cây $f[u][\cdot]$; mặt nạ bit $f[\text{mask}]$.
●●● Vì sao ba lô vẫn NP-khó	$O(nW)$ là giả đa thức: đa thức theo giá trị W nhưng mũ theo số bit biểu diễn W . Dấu hiệu: trạng thái chứa một giá trị chứ không phải chỉ số.
●● Ghi nhớ so với điền bảng	Ghi nhớ: gần cách nghĩ, chỉ tính trạng thái cần, không phải tìm thứ tự. Điền bảng: nhanh hơn, không tràn ngăn xếp, cho phép mảng cuộn.
●● Mảng cuộn	Giữ hai tầng (hoặc một, nếu duyệt đúng chiều) khi tầng i chỉ phụ thuộc tầng $i - 1$; cái giá là mất khả năng truy vết.
●● Bốn kỹ thuật tăng tốc chuyển	Deque đơn điệu (cửa sổ trượt); cây phân đoạn/Fenwick (đoạn tùy ý); bao lồi (chuyển dạng đường thẳng); tính đơn điệu điểm tối ưu (chia để trị, Knuth).
●● Hướng tăng tốc mạnh nhất	Giảm số trạng thái bằng một quan sát tham lam: chứng minh cả một lớp lựa chọn không bao giờ tối ưu rồi loại.
●● Mẹo đảo vai trò	Khi W quá lớn nhưng tổng giá trị nhỏ, đặt trạng thái theo giá trị và tối thiểu hoá khối lượng.
●● Ba họ chuyên hơn	Quy hoạch động trên chữ số (đếm số thoả điều kiện); kỳ vọng (có thể phải giải hệ); trò chơi (cực đại và cực tiểu xen kẽ).
● Truy vết lời giải	Lưu quyết định tại mỗi trạng thái, hoặc dò ngược bằng so sánh giá trị; nếu đã dùng mảng cuộn thì cần kỹ thuật chia để trị kiểu Hirschberg.
●● Bất đẳng thức tứ giác	$w(a, c) + w(b, d) \leq w(a, d) + w(b, c)$ với $a \leq b \leq c \leq d$ — “lông nhau không đắt hơn cắt nhau” — kéo theo điểm chia tối ưu đơn điệu, tức giảm được một thừa số n . Kiểm bằng vết cạnh nhỏ trước khi tin.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi luôn bí ở bước đặt trạng thái. Có quy trình nào không?

| Có một quy trình bốn bước dùng được cho phần lớn bài. Một, viết lời giải vết cạn để quy trước, với các tham số tự nhiên. Hai, nhìn danh sách tham số của hàm đệ quy đó — chúng chính là ứng viên trạng thái. Ba, loại các tham số suy ra được từ những tham số khác (ví dụ nếu biết i thì biết luôn tổng đã dùng thì bỏ tham số tổng). Bốn, kiểm tra tính đủ bằng ba câu hỏi ở khối *Cần nhớ*. Quy trình này biến việc “nghĩ ra trạng thái” thành việc “viết vết cạn rồi rút gọn tham số”, dễ hơn nhiều.

2. Làm sao biết bài này nên dùng tham lam hay quy hoạch động?

| Thử tham lam trước vì nó rẻ hơn nhiều: nghĩ một tiêu chí, và stress test với vết cạn trên n nhỏ trong năm phút. Nếu tìm được phản ví dụ thì chuyển sang quy hoạch động, và điều đáng chú ý là chính phản ví dụ đó thường chỉ ra chiều trạng thái cần thêm — nó cho thấy quyết định tối ưu phụ thuộc vào cái gì. Đây là một trong những cách hiệu quả nhất để “đọc ra” trạng thái từ bài toán.

3. Số trạng thái của tôi là 10^7 và mỗi chuyển $O(1)$. Có chạy nổi không?

| Theo bảng ở A/03 thì 10^7 phép toán đơn giản là thoải mái, nhưng hãy kiểm hai điều mà bảng không nói. Thứ nhất là bộ nhớ: 10^7 số nguyên 32 bit là 40 MB, thường vẫn được, nhưng 10^7 số 64 bit là 80 MB và có thể vượt giới hạn. Thứ hai là kiểu truy cập: nếu bạn duyệt bảng hai chiều theo cột thay

vì theo hàng thì mỗi lần truy cập là một lần trượt cache và chương trình có thể chậm gấp năm lần. Thứ tự vòng lặp là tối ưu rẻ nhất và hay bị bỏ qua nhất.

4. Quy hoạch động của tôi có chu trình phụ thuộc — trạng thái A cần B mà B lại cần A. Làm sao?

| Đó là dấu hiệu bài toán không còn là quy hoạch động thuần túy. Ba hướng xử lý. Nếu các chuyển có “chi phí không âm” và bạn tìm cực tiểu, hãy dùng Dijkstra trên đồ thị trạng thái (C/17) — thứ tự tính được xác định động thay vì tĩnh. Nếu là bài kỳ vọng có chu trình, hãy lập và giải hệ phương trình tuyến tính. Nếu chu trình đến từ việc trạng thái thiếu chiều, hãy thêm chiều để phá chu trình — ví dụ thêm chiều “số bước đã đi”.

5. Tôi cần in ra lời giải cụ thể chứ không chỉ giá trị tối ưu. Làm thế nào?

| Hai cách. Cách đơn giản: lưu thêm mảng `choice[state]` ghi quyết định đã chọn tại mỗi trạng thái, rồi đi ngược từ trạng thái đáp án. Cách tiết kiệm bộ nhớ: không lưu gì, và khi truy vết thì tính lại — tại mỗi trạng thái, thử từng quyết định và xem quyết định nào cho ra đúng giá trị đã lưu. Nếu bạn đã dùng mảng cuộn thì không truy vết trực tiếp được, và cách chuẩn cho họ hai chuỗi là kỹ thuật chia đôi của Hirschberg, cho $O(nm)$ thời gian với $O(\min(n, m))$ bộ nhớ.

6. Vì sao dãy con tăng dài nhất lại có lời giải $O(n \log n)$ trông chẳng giống quy hoạch động?

| Vì nó là quy hoạch động đã được biến đổi. Bản $O(n^2)$ có trạng thái “độ dài dãy tăng dài nhất kết thúc tại i ”. Bản $O(n \log n)$ đảo lại: với mỗi độ dài, giữ *giá trị kết thúc nhỏ nhất* có thể. Mảng các giá trị đó luôn tăng, nên cập nhật được bằng tìm kiếm nhị phân. Đây là ví dụ điển hình của mẹo đảo vai trò ở mục 4 — đổi chỗ giữa cái được tối ưu và cái làm chỉ số — và nó đáng học vì mẹo này áp dụng được cho nhiều bài khác.

7. Quy hoạch động trên cây khác gì so với trên dãy?

| Khác chủ yếu ở thứ tự tính và ở cách gộp. Thứ tự là duyệt sâu, tính cây con trước rồi gộp lên cha — tương đương với việc “từ dưới lên” trên một cấu trúc phân nhánh. Cách gộp thường có dạng “kết hợp kết quả của các con”, và khi có ràng buộc về số lượng thì phép gộp giống bài ba lô trên từng cây con. Một chi tiết kỹ thuật quan trọng: với cây sâu tới 10^5 , đệ quy có thể tràn ngăn xếp, nên hãy dùng bản lập hoặc tăng giới hạn ngăn xếp (E/25).

8. Có bài nào nhìn không giống quy hoạch động mà thực ra là không?

| Rất nhiều, và nhận ra chúng là dấu hiệu đã nắm phương pháp. Ba ví dụ: bài đếm đường đi trên lưới có vật cản (thực ra là $f[i][j]$ một cách hiển nhiên); bài chia dãy thành k đoạn tối ưu (họ khoảng, nhưng thường phải kèm tối ưu chia để trị); và bài xác suất thắng của một trò chơi (quy hoạch động kỳ vọng, với chú ý về chu trình). Ngoài ra, mọi bài tìm đường đi ngắn nhất trên đồ thị không chu trình đều là quy hoạch động — và ngược lại, mọi quy hoạch động đều là bài đường đi ngắn nhất trên đồ thị trạng thái, một cách nhìn rất đáng có (C/17).

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Số cách leo cầu thang, số cách đi trên lưới có vật cản	AtCoder DP “Frog 1”; CSES “Grid Paths”[33]	Bài nhập môn; mục tiêu là viết đủ năm bước ra giấy trước khi gõ.
● Đổi tiền: số cách và số đồng ít nhất	CSES “Coin Combinations I/II”[33]	Hai bài khác nhau đúng ở thứ tự hai vòng lặp — hãy giải thích vì sao.
● Dãy con tăng dài nhất	CSES “Increasing Subsequence”[33]	Cài cả $O(n^2)$ và $O(n \log n)$; hiểu mẹo đảo vai trò.
●● Ba lô 0/1 và ba lô không giới hạn	CSES “Book Shop”[33]	Giảm bộ nhớ bằng mảng cuộn; chú ý chiều duyệt w cho hai biến thể.
●● Khoảng cách chỉnh sửa	CSES “Edit Distance”[33]	Họ hai chuỗi; sau đó thử truy vết ra dãy thao tác cụ thể.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Dây con chung dài nhất của hai chuỗi	Bài kinh điển	Cùng họ; so sánh cách truy vết với và không có mảng cuộn.
•• Tập độc lập lớn nhất trên cây	Bài kinh điển	Quy hoạch động trên cây với hai trạng thái mỗi đỉnh; chú ý độ sâu đệ quy.
•• Cắt gậy để tổng chi phí nhỏ nhất	Bài kinh điển	Họ khoảng; nhận ra sự tương đồng với bài nhân dây ma trộn.
••• Người bán hàng với $n \leq 20$	CSES “Hamiltonian Flights”-loại[33]	Quy hoạch động mặt nạ bit; ước lượng $2^n n^2$ trước khi cài.
••• Chia dây thành k đoạn sao cho tổng chi phí nhỏ nhất	Bài thi đấu kinh điển	Bản $O(kn^2)$ trước, rồi thêm tối ưu chia để trị; kiểm tính đơn điệu điểm tối ưu bằng thực nghiệm.
••• Đếm số nguyên trong $[a, b]$ thoả một điều kiện về chữ số	Bài kinh điển	Quy hoạch động trên chữ số với cờ “đang bám biên”; đây là họ có nhiều trường hợp biên nhất.
••• Xác suất thắng của một trò chơi có chu trình trạng thái	Bài kinh điển	Lập hệ phương trình tuyến tính; ví dụ cho thấy ranh giới của quy hoạch động thuần túy.
••• Ghép cặp hoàn hảo trọng số nhỏ nhất với $n \leq 20$	Bài kinh điển	Mặt nạ bit; so với thuật toán Hungarian ở C/18 về cả độ phức tạp lẫn độ dài code.
••• Bài toán ba lô với W tới 10^{18} nhưng tổng giá trị nhỏ	Bài kinh điển	Đảo vai trò hai chiều; bài mẫu cho mẹo ở mục 4.

Kết chương và đọc tiếp

Quy hoạch động là kỹ thuật có tỉ lệ xuất hiện cao nhất trong mọi kỳ thi và phỏng vấn, và toàn bộ độ khó của nó tập trung vào một câu: trạng thái phải đủ. Ba điều đáng mang theo: quy trình năm bước viết ra thành lời trước khi gõ; ba cách kiểm tra tính đủ của trạng thái; và cách nhìn “chi phí bằng số trạng thái nhân chi phí chuyển”, vì nó chỉ ra ngay phải tấn công thừa số nào khi quá chậm.

Chương tiếp theo đổi ngôn ngữ. Rất nhiều bài toán trông rất khác nhau trở thành cùng một bài khi được phát biểu lại bằng đỉnh và cạnh — và như câu hỏi cuối ở mục hỏi đáp đã gợi ý, quy hoạch động và đường đi ngắn nhất là hai cách nói về cùng một thứ. Đồ thị là chương của kỹ năng mô hình hoá, kỹ năng đáng giá hơn việc thuộc thêm mười thuật toán (C/17).

I Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Yao, “Speed-Up in Dynamic Programming”	SIAM Journal on Algebraic and Discrete Methods, 1982	Nguồn của điều kiện tứ giác ở dạng tổng quát. Đọc phần phát biểu định lý và ví dụ; đây là bài nên đọc nếu bạn hay gặp quy hoạch động trên khoảng.
nền tảng	Hirschberg, “A Linear Space Algorithm for Computing Maximal Common Subsequences”	Communications of the ACM, 1975	Ba trang. Đọc để có kỹ thuật truy vết trong bộ nhớ tuyến tính — thứ mà mảng cuộn lấy đi.
tra khi cần	Knuth, “Optimum Binary Search Trees”	Acta Informatica, 1971	Nơi tính đơn điệu của điểm chia xuất hiện lần đầu, trong một bài toán cụ thể.
tra khi cần	Aggarwal, Klawe, Moran, Shor, Wilber, “Geometric Applications of a Matrix-Searching Algorithm”	Algorithmica, 1987	Thuật toán SMAWK. Đọc khi truy hồi của bạn viết được thành “cực tiểu theo hàng của một ma trận đơn điệu toàn phần”.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Backurs & Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time”	SIAM Journal on Computing, 2018 (bản hội nghị STOC 2015)	Đọc khi bạn tự hỏi vì sao khoảng cách chỉnh sửa mãi vẫn là $O(nm)$: có lý do, và nó nằm ở D/24 .

Đồ thị: mô hình hoá và duyệt (Graphs: Modeling and Traversal)

PHỤ THUỘC B/07 cho ba loại kho chờ — chương này là nơi chúng sinh ra ba thuật toán; B/11 cho DSU; C/15 cho chứng minh tham lam của Dijkstra và Kruskal; C/16 cho quan hệ giữa quy hoạch động và đường đi ngắn nhất.

TÓM TẮT

Kỹ năng quan trọng nhất của chương này không phải thuộc thuật toán mà là *mô hình hoá*: nhận ra rằng bài toán trước mặt, dù nói về gì, thực chất là bài toán về đỉnh và cạnh. Khi làm được điều đó, hàng chục bài trông khác nhau sụp xuống thành vài bài đã có lời giải. Chương đi qua hai phép duyệt nền tảng và những thứ chúng để lộ ra — thứ tự tô-pô, thành phần liên thông mạnh, cầu và khớp — rồi tới bốn thuật toán đường đi ngắn nhất, mỗi cái ứng với một giả thiết khác nhau về trọng số. Phần cuối là các mẹo mô hình hoá: đồ thị trạng thái, đồ thị nhiều tầng, tách đỉnh. Nếu chỉ nhớ một điều: khi bí, hãy hỏi “cái gì là đỉnh, cái gì là cạnh” — câu hỏi đó giải được nhiều bài hơn bất kỳ thuật toán riêng lẻ nào.

KIẾN THỨC CẦN CÓ TRƯỚC

- **đồ thị** — tập đỉnh và tập cạnh nối chúng; có hướng nếu cạnh có chiều, có trọng số nếu cạnh mang một giá trị.
- **danh sách kề** — cách lưu phổ biến nhất: với mỗi đỉnh, một danh sách các đỉnh kề. Bộ nhớ $O(n+m)$, hợp với đồ thị thưa.
- **duyet rộng (BFS)** — duyệt theo lớp khoảng cách bằng hàng đợi; cho đường đi ngắn nhất trên đồ thị không trọng số.
- **duyet sâu (DFS)** — đi sâu hết một nhánh rồi quay lại, dùng ngăn xếp hoặc đệ quy; thời điểm vào/ra của mỗi đỉnh mang thông tin cấu trúc.
- **thứ tự tô-pô** — cách xếp các đỉnh của một đồ thị có hướng không chu trình sao cho mọi cạnh đi từ trái sang phải.
- **thành phần liên thông mạnh** — tập đỉnh mà từ đỉnh nào cũng tới được mọi đỉnh khác trong tập; nén chúng lại cho một đồ thị không chu trình.

CÂU HỎI MỞ ĐẦU

- Q1. Nhiều bài toán không hề nhắc tới “đồ thị” vẫn giải bằng đồ thị. Dấu hiệu nào cho biết nên phát biểu lại bài toán theo đỉnh và cạnh?
- Q2. Duyệt rộng và duyệt sâu chỉ khác nhau ở một dòng code. Vì sao chúng để lộ ra những thông tin hoàn toàn khác nhau?
- Q3. Dijkstra sai với cạnh trọng số âm. Chỗ nào trong lập luận của nó dùng tới giả thiết không âm, và Bellman–Ford làm khác thế nào?
- Q4. Vì sao “đường đi ngắn nhất” và “quy hoạch động” lại là hai cách nói về cùng một thứ?
- Q5. Khi đồ thị có tới 10^5 đỉnh và cần trả lời hàng trăm nghìn truy vấn về tổ tiên chung trên cây, làm sao trả lời mỗi truy vấn trong $O(\log n)$?

Có một khoảnh khắc rất đặc trưng khi luyện thuật toán: bạn đọc một đề bài nói về những người bạn trao đổi quà, hoặc về các công việc phụ thuộc nhau, hoặc về các trạng thái của một trò xếp hình — và đột nhiên nhận ra đó là bài toán tìm chu trình, hoặc sắp thứ tự tô-pô, hoặc đường đi ngắn nhất. Sau khoảnh khắc đó, bài toán gần như đã xong: phần còn lại là gọi một thuật toán bạn đã biết.

Chương này viết cho chính khoảnh khắc đó. Các thuật toán đồ thị đều đã được chuẩn hoá và bạn tra được ở bất cứ đâu; cái không tra được là kỹ năng nhìn ra rằng bài toán của mình *là* một bài đồ thị. Vì vậy tôi sẽ trình bày thuật toán ở mức đủ dùng, và dành trọng lượng cho câu hỏi mô hình hoá — cái gì là đỉnh, cái gì là cạnh, và cạnh mang trọng số gì.

17.1 Mô hình hoá: cái gì là đỉnh, cái gì là cạnh

Ba dấu hiệu cho thấy bài toán nên được phát biểu lại thành đồ thị.

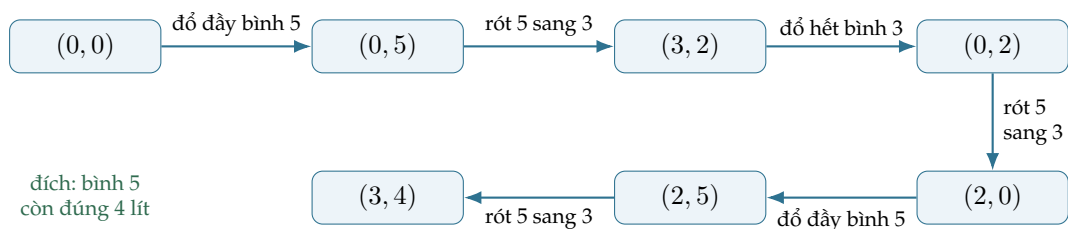
Dấu hiệu thứ nhất — quan hệ đôi một. Đề nói về những cặp có quan hệ với nhau: hai người quen nhau, hai thành phố có đường nối, hai công việc xung khắc. Đỉnh là đối tượng, cạnh là quan hệ. Đây là trường hợp hiển nhiên nhất.

Dấu hiệu thứ hai — phụ thuộc và thứ tự. Đề nói “việc A phải làm trước việc B”. Đỉnh là việc, cạnh có hướng là ràng buộc; câu hỏi “có xếp được thứ tự không” trở thành “đồ thị có chu trình không”.

Dấu hiệu thứ ba — và đây là dấu hiệu đáng giá nhất — trạng thái và bước chuyển. Đề mô tả một hệ có thể ở nhiều tình huống, và có các thao tác đưa từ tình huống này sang tình huống khác. Lúc đó *đỉnh là trạng thái, cạnh là một thao tác hợp lệ*, và câu hỏi “ít thao tác nhất để tới đích” chính là đường đi ngắn nhất.

Dấu hiệu thứ ba mở ra rất nhiều bài mà nhìn bề ngoài không có đồ thị nào: trò xếp hình, các bài đổ nước giữa các bình, bài biến chuỗi này thành chuỗi kia bằng các phép biến đổi, bài robot di chuyển trên lưới có luật. Điều cần cẩn thận là *kích thước không gian trạng thái*: nếu số trạng thái là 10^6 thì duyệt rộng chạy tốt; nếu là 10^{15} thì phải tìm cấu trúc khác hoặc dùng tìm kiếm có định hướng.

Hình 17.1 ghi rõ từng thao tác; kiểm từng cạnh là cách kiểm mô hình trước khi viết BFS.



Hình 17.1. Một đường đi hợp lệ gồm sáu thao tác từ hai bình rỗng tới 4 lít trong bình 5 lít. Trạng thái là (lượng trong bình 3, lượng trong bình 5). Hình chỉ vẽ một đường đi, không phải toàn bộ đồ thị BFS.

CẦN NHỚ

Ba câu hỏi khi nghi ngờ bài là bài đồ thị: (i) đỉnh là gì — đối tượng hay *trạng thái*; (ii) cạnh là gì, và có hướng không; (iii) cạnh có trọng số gì — không trọng số, trọng số 0/1, trọng số dương, hay có thể âm. Câu thứ ba quyết định thuật toán, và bảng ở mục 4 là bản đồ.

17.2 Hai phép duyệt và những gì chúng để lộ

Duyệt rộng và duyệt sâu chỉ khác nhau ở kho chờ — hàng đợi hay ngăn xếp, đúng như chương B/07 đã nói — nhưng thông tin chúng cho lại rất khác.

Duyệt rộng xử lý các đỉnh theo lớp khoảng cách tăng dần. Hệ quả trực tiếp: nó cho *đường đi ngắn nhất trên đồ thị không trọng số*, và khoảng cách tìm được là chính xác chứ không phải xấp xỉ. Hai biến thể đáng chú ý: *duyet rộng nhiều nguồn* — đẩy tất cả các nguồn vào hàng đợi lúc đầu, cho “khoảng cách tới nguồn gần nhất” trong một lượt; và *duyet rộng 0-1* — khi trọng số chỉ là 0 hoặc 1, dùng deque và đẩy vào đầu khi cạnh trọng số 0, cho $O(n + m)$ thay vì $O(m \log n)$ của Dijkstra.

Duyệt sâu không cho khoảng cách, nhưng nó cho một thứ khác có giá trị hơn: *cấu trúc*. Khi ghi lại thời điểm vào và thời điểm ra của mỗi đỉnh, ta được một bộ khung để đọc ra rất nhiều tính chất. Đây chính là đóng góp khái niệm lớn của Tarjan năm 1972[64]: biến duyệt sâu từ “một cách đi lòng vòng” thành một công cụ phân tích có hệ thống.

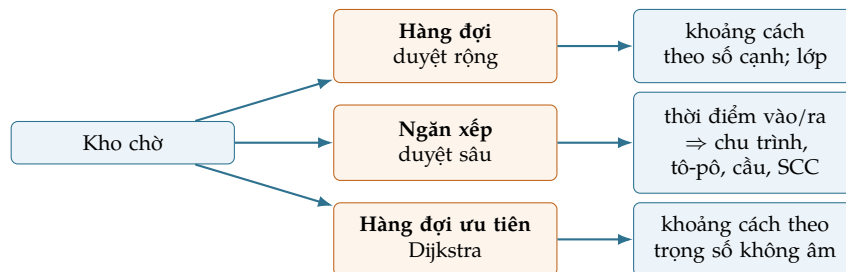
Bốn thứ mà thời điểm vào/ra của duyệt sâu để lộ ra:

Chu trình. Trong đồ thị có hướng, có chu trình khi và chỉ khi tồn tại cạnh đi ngược về một đỉnh đang nằm trên ngăn xếp hiện tại.

Thứ tự tô-pô. Trên đồ thị có hướng không chu trình (DAG), xếp các đỉnh theo thời điểm *ra* giảm dần cho một thứ tự tô-pô hợp lệ. Nếu phát hiện chu trình thì không có thứ tự tô-pô.

Cầu và khớp. Một cạnh là cầu khi cây con bên dưới nó không có đường ngược nào lên phía trên — điều kiện này viết được bằng thời điểm vào và một giá trị “lên cao nhất có thể”.

Thành phần liên thông mạnh. Cả hai thuật toán chuẩn — Kosaraju với hai lượt duyệt, Tarjan với một lượt — đều dựa trên cùng bộ thông tin đó. Sau khi nén mỗi thành phần thành một đỉnh, ta được đồ thị không chu trình, và mọi kỹ thuật quy hoạch động trên đồ thị không chu trình dùng lại được.



Hình 17.2. Cùng một khung “lấy ra, xử lý, đẩy vào”, đổi kho chờ thì được ba thuật toán với ba loại kết quả khác nhau. Nhận ra điều này giúp nhớ ít hơn và suy được nhiều hơn — và nó cũng giải thích ngay vì sao Dijkstra cần trọng số không âm (B/07).

17.3 Bốn thuật toán đường đi ngắn nhất, bốn giả thiết

Chọn thuật toán đường đi ngắn nhất không phải chuyện sở thích: mỗi thuật toán ứng với một giả thiết về trọng số, và dùng sai giả thiết cho kết quả sai chứ không phải chậm.

Không trọng số → duyệt rộng, $O(n + m)$. Đơn giản nhất, nhanh nhất; đừng dùng Dijkstra ở đây.

Trọng số 0 hoặc 1 → duyệt rộng 0-1 với deque, $O(n + m)$.

Trọng số không âm → Dijkstra với hàng đợi ưu tiên, $O(m \log n)$. Nó là thuật toán tham lam: chốt đỉnh có nhãn nhỏ nhất và không xét lại. Chứng minh dùng giả thiết không âm đúng một lần — ở chỗ khẳng định rằng một đường đi vòng qua các đỉnh chưa chốt không thể ngắn hơn — và đó chính là chỗ nó hỏng khi có cạnh âm.

Có trọng số âm $\rightarrow$ Bellman–Ford, $O(nm)$. Nó không chốt gì cả: nó thư giãn mọi cạnh $n - 1$ lần, và ý tưởng là “đường đi ngắn nhất có tối đa $n - 1$ cạnh, nên sau $n - 1$ vòng mọi nhãn đã ổn định”. Lợi thế kèm theo: nếu ở vòng thứ n vẫn còn cạnh thư giãn được thì đồ thị có chu trình âm — một thứ mà Dijkstra không phát hiện được.

Mọi cặp đỉnh $\rightarrow$ Floyd–Warshall, $O(n^3)$. Đáng chú ý vì cách nghĩ của nó là quy hoạch động thuần túy: $f[k][i][j]$ là đường ngắn nhất từ i tới j chỉ dùng các đỉnh trung gian trong k đỉnh đầu tiên. Ba vòng lặp lồng nhau, và thứ tự của chúng không hoán đổi được — vòng k phải nằm ngoài cùng, một lỗi rất phổ biến.

Ngoài bốn cái trên còn hai công cụ nên biết. A^* là Dijkstra cộng một hàm ước lượng khoảng cách còn lại; nếu hàm đó không bao giờ đánh giá quá thực tế thì kết quả vẫn tối ưu, còn nếu đánh giá tốt thì số đỉnh phải xét giảm rất nhiều. Đây là thuật toán chuẩn cho tìm đường của robot và trò chơi. Johnson biến đổi trọng số để loại bỏ số âm rồi chạy Dijkstra từ mọi đỉnh — hợp cho đồ thị thưa có cạnh âm.

Giả thiết trọng số	Thuật toán	Chi phí	Điều cần nhớ
Không trọng số	Duyệt rộng	$O(n + m)$	Nhanh nhất; nhiều nguồn cùng lúc rất tiện
Chỉ 0 và 1	Duyệt rộng 0–1	$O(n + m)$	Deque: cạnh 0 đẩy vào đầu, cạnh 1 vào cuối
Không âm	Dijkstra	$O(m \log n)$	Nhớ bỏ qua bản ghi lỗi thời khi lấy ra
Có thể âm	Bellman–Ford	$O(nm)$	Phát hiện được chu trình âm ở vòng thứ n
Mọi cặp, n nhỏ	Floyd–Warshall	$O(n^3)$	Vòng k bắt buộc ở ngoài cùng
Có ước lượng tốt	A^*	phụ thuộc heuristic	Heuristic không được đánh giá quá thực tế

17.4 Cây: các kỹ thuật riêng

Cây là đồ thị liên thông không chu trình, và chính vì thiếu chu trình mà nhiều bài trở nên dễ hơn hẳn. Ba kỹ thuật đáng thuộc.

Nhảy nhị phân cho tổ tiên chung gần nhất. Tiền xử lý $up[v][k] =$ tổ tiên thứ 2^k của v , tốn $O(n \log n)$; sau đó mỗi truy vấn tổ tiên chung gần nhất trả lời trong $O(\log n)$ bằng cách nâng hai đỉnh về cùng độ sâu rồi nhảy đồng thời. Kỹ thuật này còn dùng để tính nhanh cực trị hoặc tổng trên đường đi giữa hai đỉnh — chỉ cần lưu thêm thông tin tổng hợp cùng với bảng nhảy.

Dãy Euler. Duyệt sâu và ghi lại đỉnh mỗi khi đi qua, ta được một dãy mà bài toán tổ tiên chung trở thành bài toán cực tiểu trên đoạn — tức là chuyển hẳn sang công cụ của chương B/10. Đây là một trong những phép quy dẫn đẹp nhất giữa hai miền, và nó đi cả hai chiều: mọi bài cực tiểu trên đoạn cũng dựng được thành bài tổ tiên chung trên cây Cartesian.

Phân rã theo cạnh nặng. Chia cây thành các chuỗi dọc sao cho mọi đường đi từ gốc xuống lá chỉ cắt qua $O(\log n)$ chuỗi; nhờ đó truy vấn và cập nhật trên đường đi giữa hai đỉnh quy về $O(\log n)$ truy vấn đoạn trên các chuỗi. Xếp vào loại tham khảo nhưng rất mạnh khi cần.

17.5 Ba mẹo mô hình hoá đáng giá nhất

Ba mẹo dưới đây biến những bài trông không giải được thành bài quen thuộc.

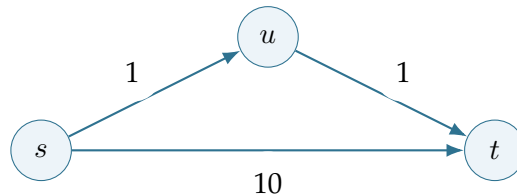
Đồ thị trạng thái. Đã nói ở mục 1, nhưng đáng nhắc lại kèm ví dụ: bài “có hai bình dung tích 3 và 5 lít, đong ra đúng 4 lít” có đỉnh là cặp (lượng nước bình 1, lượng nước bình 2) và cạnh là các thao tác đổ. Bài trở thành duyệt rộng trên vài chục đỉnh.

Đồ thị nhiều tầng. Khi bài cho phép một số hành động đặc biệt với số lần hạn chế — “được đi miễn phí tối đa k cạnh” — hãy nhân bản đồ thị thành $k + 1$ tầng, mỗi tầng ứng với số lần đã dùng, và cho cạnh đặc biệt nối tầng i sang tầng $i + 1$. Đường đi ngắn nhất trên đồ thị mở rộng chính là đáp án. Mẹo này là ứng dụng trực tiếp của nguyên tắc “trạng thái phải đủ” ở chương C/16: số lần đã dùng là một phần của trạng thái.

Tách đỉnh. Khi ràng buộc nằm ở *đỉnh* chứ không ở *cạnh* — “mỗi thành phố chỉ đi qua một lần”, “mỗi máy chỉ xử lý một việc” — hãy tách mỗi đỉnh thành hai nửa nối bởi một cạnh mang đúng ràng buộc đó. Kỹ thuật này đặc biệt quan trọng trong các bài luồng ở chương C/18.

17.6 Phản ví dụ ba đỉnh để chọn đúng đường đi ngắn nhất

Hình 17.3 có đường một cạnh $s \rightarrow t$ giá 10 và đường hai cạnh $s \rightarrow u \rightarrow t$ giá 2. BFS tối thiểu số cạnh sẽ chọn đường giá 10; Dijkstra tối thiểu tổng trọng số chọn đường giá 2. Vì vậy thay queue bằng heap không đủ: phải thay cả luật cập nhật khoảng cách bằng phép nối lỏng.



Hình 17.3. Ít cạnh nhất không đồng nghĩa tổng trọng số nhỏ nhất. Cả ba cạnh có hướng và trọng số không âm; Dijkstra tìm khoảng cách 2 tới t .

Khi dùng heap có phần tử cũ, mỗi lần lấy (d, u) , bỏ qua nếu $d \neq \text{dist}[u]$. Chỉ dừng sớm khi lấy đích ra với khoảng cách còn hợp lệ, không dừng lúc vừa nhìn thấy cạnh tới đích. Muốn thấy vì sao cấm cạnh âm, đổi $s \rightarrow t$ thành 2, $s \rightarrow u$ thành 5 và $u \rightarrow t$ thành -10 : chốt t ở 2 sẽ bỏ lỡ đường giá -5 . Đồ thị này là DAG nên vẫn giải được bằng thứ tự tô-pô.

Bảng 17.1. Bổ sung hai trường hợp dễ bị bỏ sót khi chọn đường đi ngắn nhất. n là số đỉnh, m là số cạnh, dùng danh sách kề.

Thuật toán	Giả thiết	Chi phí	Lỗi cần tránh
BFS	Mọi cạnh cùng giá dương	$O(n + m)$	Tối thiểu số cạnh trên đồ thị có giá khác nhau
0–1 BFS	Giá cạnh chỉ 0 hoặc 1	$O(n + m)$	Dùng visited để chặn mọi lần cải thiện
DP trên DAG	Có hướng, không chu trình; cho phép cạnh âm	$O(n + m)$	Dùng thứ tự DFS khi chưa loại chu trình
Dijkstra + heap nhị phân	Giá cạnh không âm	$O((n + m) \log n)$ trên đồ thị đơn	Chốt đỉnh lúc thêm vào heap
Bellman–Ford	Cho phép cạnh âm	$O(nm)$	Bỏ qua chu trình âm tới được từ nguồn

Bảng 17.1 tách giả thiết về trọng số khỏi giả thiết về cấu trúc. Nếu chu trình âm tới được từ nguồn và đi tiếp được tới đích, không tồn tại khoảng cách hữu hạn nhỏ nhất tới đích.

17.7 Dijkstra có phải là tận cùng không: rào cản sắp xếp

Bảng trên ghi Dijkstra là $O(m \log n)$ và dừng ở đó, như mọi giáo trình. Nhưng cái log ấy đến từ đâu, và có bắt buộc không? Câu hỏi này có một lịch sử dài và một câu trả lời rất mới, đáng kể lại vì nó cho thấy cách một “sự thật hiển nhiên” bị lật.

Nguồn của log là hàng đợi ưu tiên: Dijkstra rút các đỉnh ra *theo đúng thứ tự khoảng cách tăng dần*, nghĩa là chạy xong thì nó đã sắp xếp n đỉnh theo khoảng cách. Mà sắp xếp n số bằng so sánh cần $\Omega(n \log n)$ (D/24). Vậy mọi thuật toán làm đúng như Dijkstra — chốt đỉnh theo thứ tự — đều bị chặn dưới bởi chi phí sắp xếp. Người ta gọi đó là *rào cản sắp xếp*, và với hàng đợi Fibonacci[36] thì cận $O(m + n \log n)$ chính là chạm đúng rào cản ấy.

Cách vượt rào thứ nhất là bỏ mô hình so sánh. Nếu trọng số là số nguyên và máy được phép làm số học trên chúng, cận dưới sắp xếp không còn áp dụng — đúng như sắp xếp đếm không mâu thuẫn với cận $n \log n$. Thorup[65] khai thác đúng chỗ đó và cho thuật toán *tuyến tính* $O(m)$ cho đồ thị *vô hướng* với trọng số nguyên dương, bằng cách xây một cấu trúc phân tầng theo bit và chấp nhận xử lý các đỉnh “theo nhóm không cần đúng thứ tự”. Chi tiết đáng nhớ là chỗ giả thiết được dùng: vô hướng và nguyên, cả hai đều thiết yếu.

Cách vượt rào thứ hai mới xuất hiện gần đây và mạnh hơn về mặt khái niệm, vì nó *không* bỏ mô hình so sánh. Duan, Mao, Mao, Shu và Yin[66] đưa ra thuật toán tất định $O(m \log^{2/3} n)$ cho đồ thị *có hướng* với trọng số thực không âm, trong mô hình chỉ có so sánh và cộng — tức là phá vỡ cận $O(m + n \log n)$ của Dijkstra trên đồ thị thưa và chứng minh rằng Dijkstra *không* tối ưu cho bài toán đường đi ngắn nhất. Ý tưởng cốt lõi là làm ít hơn cái Dijkstra làm: thuật toán tránh việc sắp xếp đầy đủ mọi đỉnh theo khoảng cách, và chỉ giữ đủ thứ tự để các nhãn vẫn đúng. Bài được trao giải bài báo xuất sắc tại STOC 2025.

Đừng cài các thuật toán này. Giá trị của mục này nằm ở ba câu hỏi mà nó dạy bạn tự đặt cho *bất kỳ* thuật toán quen thuộc nào. Một, thuật toán này có đang làm nhiều hơn việc đề bài yêu cầu không — Dijkstra sắp xếp toàn bộ đỉnh trong khi đề chỉ hỏi khoảng cách. Hai, cận dưới mà tôi vừa viện dẫn thuộc mô hình nào, và bài toán của tôi có thật sự nằm trong mô hình đó không. Ba, khi một cận đứng yên hàng chục năm, đó là dấu hiệu của một định lý hay chỉ là dấu hiệu chưa ai thử đúng chỗ? Với đường đi ngắn nhất, câu trả lời hoá ra là về sau — sau sáu mươi sáu năm.

17.8 Phản biện, nhược điểm và rủi ro triển khai

Đồ thị là ngôn ngữ mô hình hoá mạnh nhất trong thuật toán, nhưng sự phổ quát của nó thường khiến kỹ sư chủ quan trước các cái giá về phần cứng và các chế độ hỏng thảm hoạ trong môi trường thực tế.

Phản biện: sự lạm dụng của tư duy rời rạc hoá. Mô hình hoá thành đồ thị là con dao hai lưỡi. Trong các bài toán thực tế của robot và xe tự hành (như dẫn đường trong không gian liên tục), việc băm không gian thành lưới rồi chạy Dijkstra hay A^* thường sinh ra các đường đi zíc-zắc gây khúc (discretization artifacts), không khả thi về mặt động học trơn của xe. Khi số bậc tự do tăng lên (như cánh tay robot 6–7 khớp), không gian trạng thái bùng nổ theo hàm mũ khiến mọi đồ thị rời rạc đều cạn kiệt bộ nhớ; lúc đó các thuật toán tối ưu liên tục (như TrajOpt, CHOMP) hoặc cây ngẫu nhiên hoá (RRT*, PRM) vượt trội hơn hẳn. Ngoài ra, với A^* , niềm tin rằng “ A^* luôn nhanh hơn Dijkstra” là ảo tưởng: nếu hàm heuristic $h(u)$ tính toán tốn kém hoặc không định hướng tốt ($h \equiv 0$), A^* chạy chậm hơn Dijkstra vì gánh thêm chi phí tính toán cho từng đỉnh; còn nếu $h(u)$ vô tình đánh giá cao hơn chi phí thật (vi phạm tính chấp nhận được / admissibility), thuật toán sẽ âm thầm trả về đường đi không tối ưu mà không hề báo lỗi.

Nhược điểm cốt lõi về tài nguyên và phân cấp bộ nhớ:

- *Thảm hoạ trượt cache của danh sách kề (Pointer Chasing)*: Cách cài đặt phổ biến `vector<int> adj[N]` tạo ra N mảng con nằm phân tán rải rác trên heap. Khi duyệt đồ thị có hàng triệu đỉnh, CPU liên tục phải nhảy con trỏ ngẫu nhiên khắp bộ nhớ, gây trượt cache L1/L2 liên tục và làm chậm chương trình từ 3 tới 5 lần so với cấu trúc mảng nén liên tục CSR (Compressed Sparse Row).
- *Phình to bộ nhớ trong Dijkstra dùng hàng đợi ưu tiên chuẩn*: Thư viện chuẩn C++ (`std::priority_queue`) không hỗ trợ thao tác giảm khoá (`decrease-key`). Để cập nhật nhãn, ta buộc phải đẩy một cặp (d, u) mới vào heap, khiến số phần tử trong heap có thể lên tới m thay vì n . Trên đồ thị dày ($m \approx 10^7$), heap ngốn hàng trăm megabyte RAM và làm tăng chi phí mỗi lần rút đỉnh lên $O(\log m)$.
- *Sự thoái hoá của SPFA (Shortest Path Faster Algorithm)*: SPFA (bản cải tiến dùng hàng đợi của Bellman–Ford) chạy rất nhanh trên đồ thị ngẫu nhiên (trung bình $O(km)$), khiến nhiều người nhầm tưởng nó thay thế được Dijkstra. Nhưng trong trường hợp xấu nhất do đối thủ cố ý dựng (đồ thị dạng lưới hoặc chuỗi lồng nhau), SPFA thoái hoá về đúng $O(nm)$, thậm chí bị treo hoàn toàn khi các tối ưu như SLF/LLL gặp dữ liệu đối kháng.

Rủi ro vận hành và các lỗi im lặng nguy hiểm:

- *Tràn ngăn xếp đệ quy (Stack Overflow) trong DFS*: DFS đệ quy ngây thơ là nguyên nhân số một gây lỗi sập chương trình đột ngột (Segmentation Fault). Trên các hệ thống Linux với giới hạn ngăn xếp mặc định 8 MB, một cây suy biến thành đường thẳng có $n \geq 10^5$ đỉnh sẽ làm tràn ngăn xếp chỉ sau vài phần mười giây. Trong hệ thống thật, DFS trên đồ thị lớn bắt buộc phải được viết ở dạng lặp với một ngăn xếp tường minh cấp phát trên heap.
- *Tràn số nguyên trong Floyd–Warshall*: Để biểu diễn “không có đường nối”, ta khởi tạo ma trận khoảng cách bằng ∞ . Nếu chọn $\infty = 2 \cdot 10^9$ (gần `INT_MAX`), phép cộng $dist[i][k] + dist[k][j]$ sẽ tràn số học thành một số âm cực lớn, khiến thuật toán nhầm tưởng rằng giữa i và j tồn tại một đường đi âm siêu ngắn. Quy tắc: dùng hằng số an toàn như `0x3f3f3f3f` (cho kiểu 32-bit) hoặc 10^{18} (cho kiểu 64-bit).
- *Lập vô hạn trong Dijkstra trên số thực*: Khi cạnh mang trọng số thực và đồ thị xuất hiện các chu trình có chi phí xấp xỉ 0 do sai số làm tròn số thực dấu phẩy động (ví dụ sai số -10^{-16}), điều kiện kiểm tra khoảng cách có thể bị đánh lừa liên tục, đẩy thuật toán vào vòng lặp vô tận. Bắt buộc phải có ngưỡng dung sai ε (eps) khi so sánh các nhãn khoảng cách thực.
- *Bẫy đồ thị không liên thông*: Rất nhiều chương trình chỉ chạy BFS/DFS từ đỉnh 1 và ngầm giả định đã duyệt hết mọi đỉnh. Khi đồ thị bị đứt gãy thành nhiều thành phần liên thông độc lập, các đỉnh còn lại sẽ hoàn toàn bị bỏ quên, dẫn tới nghiệm sai lệch âm thầm khi tìm cầu, khớp hoặc thành phần liên thông mạnh.

CẠM BÃY

Trong Dijkstra với heap chuẩn C++, dòng kiểm tra `if (d > dist[u]) continue;` không phải là một tối ưu nhỏ để chạy nhanh hơn — nó là điều kiện sống còn để ngăn chặn thuật toán duyệt lại các đỉnh cũ và làm độ phức tạp bùng nổ theo cấp số nhân trên các đồ thị có nhiều đường đi thay thế.

17.9 Gốc rễ: bảy cây cầu, một quán cà phê, và một con robot

Lý thuyết đồ thị có một ngày sinh khá rõ ràng: năm 1736, Euler giải bài toán bảy cây cầu ở Königsberg — có đi qua mỗi cây cầu đúng một lần rồi về chỗ cũ được không. Điều làm bài báo đó thành khởi đầu của cả một ngành không phải câu trả lời (không) mà là *cách trả lời*: Euler bỏ đi toàn bộ hình dạng địa lý, giữ lại đúng quan hệ “vùng đất nào nối với vùng đất nào”, và chứng minh kết

luận chỉ từ tính chắn lẻ của số cầu tại mỗi vùng. Đó chính xác là thao tác mô hình hoá mà mục 1 nói tới, thực hiện lần đầu tiên.

Thuật toán Dijkstra có một nguồn gốc rất đời và được chính tác giả kể lại nhiều lần: năm 1956, ông đang ngồi ở một quán cà phê tại Amsterdam cùng vợ chưa cưới, và cần một bài toán trình diễn cho máy tính ARMAC sao cho khán giả không chuyên hiểu được. Ông nghĩ ra bài đường đi ngắn nhất giữa hai thành phố Hà Lan, và tìm ra thuật toán trong khoảng hai mươi phút, không giấy bút. Ông xuất bản nó ba năm sau trong một bài báo hai trang[67]. Chi tiết đáng chú ý về mặt phương pháp: ràng buộc “phải giải thích được cho người không chuyên” đã đẩy ông tới một thuật toán đơn giản — và sự đơn giản đó là lý do nó vẫn được dùng khắp nơi sau bảy mươi năm.

Duyệt sâu thì cổ hơn nhiều — nó xuất hiện trong các phương pháp giải mê cung từ thế kỷ 19 — nhưng đóng góp quyết định là của Tarjan năm 1972[64]. Ông chỉ ra rằng nếu ghi lại thời điểm vào và ra của mỗi đỉnh thì duyệt sâu trở thành một *khung phân tích*: cùng một lượt duyệt cho ra thành phần liên thông mạnh, cầu, khớp, và nhiều tính chất khác. Trước đó mỗi bài toán có một thuật toán riêng; sau đó chúng trở thành các cách đọc khác nhau của cùng một cấu trúc dữ liệu.

Cuối cùng, A^* ra đời năm 1968 tại Viện Nghiên cứu Stanford, từ một nhu cầu rất cụ thể: robot Shakey — một trong những robot di động đầu tiên có khả năng lập kế hoạch — cần tìm đường trong một môi trường thật với tài nguyên tính toán rất hạn chế. Hart, Nilsson và Raphael thêm vào Dijkstra một hàm ước lượng khoảng cách còn lại và chứng minh điều kiện để kết quả vẫn tối ưu. Đây là lý do A^* tới nay vẫn là thuật toán lập kế hoạch đường đi mặc định trong robot học và trong trò chơi.

■ Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1736 — Euler	Bảy cây cầu Königsberg: đi qua mỗi cầu đúng một lần được không?	Khai sinh lý thuyết đồ thị; và quan trọng hơn, khai sinh thao tác <i>bỏ chi tiết, giữ quan hệ</i> — chính là mô hình hoá.
1956 — Dijkstra	Cần một bài trình diễn máy tính mà khán giả không chuyên hiểu được.	Thuật toán đường đi ngắn nhất, nghĩ ra trong 20 phút không giấy bút; ràng buộc “phải dễ hiểu” dẫn tới sự đơn giản khiến nó sống lâu.
1958–62 — Bellman, Ford, Floyd, Warshall 1968 — Hart, Nilsson, Raphael (SRI)	Mạng có chi phí âm (chiết khấu, chênh lệch tỉ giá) và nhu cầu tính mọi cặp. Robot Shakey phải lập kế hoạch đường đi thật với tài nguyên rất hạn chế.	Bellman–Ford và Floyd–Warshall; cả hai là quy hoạch động thuần túy trên đồ thị. A^* : Dijkstra cộng ước lượng khoảng cách còn lại; vẫn là thuật toán mặc định trong robot học và trò chơi.
1972 — Tarjan	Mỗi tính chất của đồ thị lại có một thuật toán riêng.	Duyệt sâu với thời điểm vào/ra trở thành khung phân tích chung: SCC, cầu, khớp, tô-pô đều đọc ra từ cùng một lượt duyệt.
Thập niên 2000 — định tuyến bản đồ	Tìm đường trên mạng đường bộ hàng trăm triệu cạnh, trả lời trong mili-giây.	Các kỹ thuật tiền xử lý như phân cấp co rút; cho thấy Dijkstra thuần túy không đủ ở quy mô thật, nhưng vẫn là nền của mọi cải tiến.

■ 17.10 Liên hệ ngang

■ Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
SLAM và robot	Đồ thị tư thế: đỉnh là tư thế, cạnh là ràng buộc đo được	Đây là mô hình hoá đúng nghĩa: bài toán ước lượng trở thành bài toán trên đồ thị, và việc đóng vòng chính là thêm một cạnh. Khác với chương này: ở đó ta không tìm đường đi mà <i>tối ưu vị trí các đỉnh</i> sao cho tổng sai lệch nhỏ nhất — một bài bình phương tối thiểu trên cấu trúc đồ thị.
Lập kế hoạch đường đi	A*, lưới chiếm dụng, đồ thị khả kiến	Đúng thuật toán của mục 3, với đỉnh là ô lưới hoặc trạng thái tư thế. Bài học chuyển được: chất lượng hàm ước lượng quyết định số đỉnh phải xét, và một hàm đánh giá quá thực tế sẽ phá vỡ tính tối ưu.
Trình biên dịch	Đồ thị luồng điều khiển, đồ thị phụ thuộc, tô màu để cấp phát thanh ghi	Sắp thứ tự tô-pô dùng để xếp lịch lệnh; phát hiện chu trình dùng để tìm vòng lặp; tô màu đồ thị dùng để gán biến vào thanh ghi — một bài NP-khó được giải bằng heuristic trong mọi trình biên dịch bạn dùng.
Mạng máy tính	Giao thức định tuyến	Định tuyến theo trạng thái liên kết chính là Dijkstra chạy phân tán; định tuyến theo vector khoảng cách chính là Bellman–Ford chạy phân tán. Đây là ví dụ hiếm hoi mà hai thuật toán sách giáo khoa tương ứng đúng với hai họ giao thức thực tế.
Quản lý dự án	Sơ đồ mạng công việc, đường găng	Đường dài nhất trên đồ thị không chu trình — giải được vì không có chu trình, đúng như C/16 đã lưu ý. Đường găng là các công việc mà chậm một ngày thì cả dự án chậm một ngày.
Web và mạng xã hội	Xếp hạng trang, phát hiện cộng đồng, đường đi ngắn nhất xã hội	Xếp hạng trang là bước ngẫu nhiên trên đồ thị, tức là chuỗi Markov — nổi chương này với xác suất ở A/05. Quy mô ở đây buộc phải dùng thuật toán xấp xỉ và tính toán phân tán.
Hoá học và sinh học	Đồ thị phân tử, mạng phản ứng, mạng tương tác protein	Bài toán đẳng cấu đồ thị — hai phân tử có cùng cấu trúc không — là một trong số ít bài toán chưa được xếp dứt khoát vào P hay NP-đầy đủ (D/23).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Ba dấu hiệu. Một, đề nói về quan hệ đôi một giữa các đối tượng. Hai, đề nói về ràng buộc thứ tự hoặc phụ thuộc — lúc đó câu hỏi “có xếp được không” chính là “có chu trình không”. Ba, và đáng giá nhất, đề mô tả một hệ có nhiều *trạng thái* và các thao tác chuyển giữa chúng; lúc đó đỉnh là trạng thái, cạnh là thao tác, và “ít bước nhất” là đường đi ngắn nhất. Dấu hiệu thứ ba mở ra các bài không nhắc gì tới đồ thị: trò xếp hình, đong nước, biến đổi chuỗi.
- A2. Vì thứ tự xử lý quyết định thông tin được đảm bảo. Hàng đợi cho thứ tự theo lớp khoảng cách, nên lần đầu chạm tới một đỉnh là đường ngắn nhất — một tính chất mạnh và trực tiếp. Ngăn xếp cho thứ tự lồng nhau: một đỉnh chỉ ra khỏi ngăn xếp sau khi mọi thứ dưới nó đã xong, nên khoảng thời gian vào/ra của các đỉnh có quan hệ bao nhau đúng theo cấu trúc cây duyệt. Chính tính bao nhau đó là thứ cho ra thứ tự tô-pô, cầu và thành phần liên thông mạnh.
- A3. Dijkstra dùng giả thiết không âm ở đúng một chỗ: khi chốt đỉnh có nhãn nhỏ nhất, nó khẳng định rằng không có đường nào tới đỉnh đó rẻ hơn, vì mọi đường khác phải đi qua một đỉnh chưa chốt có nhãn lớn hơn hoặc bằng, và các cạnh sau đó *chỉ làm tăng* chi phí. Với cạnh âm, các cạnh sau có thể làm giảm chi phí nên lập luận sụp. Bellman–Ford không chốt gì cả: nó thư giãn mọi cạnh $n - 1$ lần, dựa trên nhận xét đường đi ngắn nhất có tối đa $n - 1$ cạnh — và nếu vòng thứ n vẫn cải thiện được thì tồn tại chu trình âm.
- A4. Vì mọi bài quy hoạch động đều là bài đường đi ngắn nhất trên *đồ thị trạng thái*: đỉnh là trạng thái, cạnh là chuyển, trọng số là chi phí của chuyển đó, và thứ tự tính hợp lệ tồn tại đúng khi đồ thị không có chu trình. Ngược lại, đường đi ngắn nhất trên đồ thị không chu trình là quy hoạch động theo thứ tự tô-pô. Cách nhìn này có giá trị thực dụng: khi quy hoạch động của bạn có chu trình phụ thuộc, hãy chuyển sang Dijkstra trên đồ thị trạng thái thay vì cố ép một thứ tự tính.

- A5.** Bằng nhảy nhị phân: tiền xử lý bảng $up[v][k]$ — tổ tiên thứ 2^k của v — trong $O(n \log n)$, rồi mỗi truy vấn nâng hai đỉnh về cùng độ sâu và nhảy đồng thời từ bước lớn xuống bước nhỏ, tốn $O(\log n)$. Cách thứ hai là chuyển sang dãy Euler và biến bài toán thành cực tiểu trên đoạn $(B/10)$, sau đó dùng sparse table cho truy vấn $O(1)$ — lựa chọn tốt khi số truy vấn rất lớn.

NÓI LẠI THẬT ĐƠN GIẢN

Đồ thị chỉ là cách vẽ lại một bài toán thành các điểm và các đường nối. Điều đáng nói không phải bản thân hình vẽ mà là việc rất nhiều bài toán khác nhau, sau khi vẽ lại, hoá ra là cùng một bài.

Hãy nghĩ tới bản đồ tàu điện. Các ga là điểm, các tuyến là đường nối. Câu hỏi “đi từ ga A tới ga B ít trạm nhất” được trả lời bằng cách loang dân: đánh dấu mọi ga cách A một trạm, rồi hai trạm, rồi ba — lần đầu chạm tới B là đáp án. Đó là duyệt rộng, và nó đúng vì bạn xét theo đúng thứ tự tăng dần của số trạm.

Nếu mỗi tuyến mất thời gian khác nhau thì cách loang đều không còn đúng, và bạn phải luôn đi tiếp từ ga có *thời gian tích lũy nhỏ nhất* trong số các ga đang chờ. Đó là Dijkstra. Điều kiện để nó đúng: đi thêm một tuyến không bao giờ làm giảm tổng thời gian. Nếu có tuyến “thường” làm giảm tổng thời gian — tức trọng số âm — thì cách này sai, và phải dùng cách chậm hơn nhưng chắc chắn: cứ cải thiện mọi tuyến nhiều lượt cho tới khi không cải thiện được nữa.

Còn mẹo mạnh nhất của chương này thì thế này: đỉnh *không nhất thiết là địa điểm*. Nếu bài toán nói bạn có hai cái bình 3 lít và 5 lít và cần đóng đúng 4 lít, thì hãy coi mỗi “tình huống” — ví dụ bình một có 2 lít, bình hai có 5 lít — là một điểm, và mỗi thao tác đổ là một đường nối. Bỗng nhiên bài đổ mẹo trở thành bài tìm đường trên một bản đồ vài chục điểm, và bạn giải nó bằng đúng cách loang ở trên.

Đó là toàn bộ ý của mô hình hoá, và nó cũng chính là điều Euler làm năm 1736 khi ông xoá bản đồ thành phố đi và chỉ giữ lại xem vùng đất nào nối với vùng đất nào.

Một câu: Khi bí, hãy hỏi “*cái gì là đỉnh, cái gì là cạnh*” — và nhớ rằng *đỉnh có thể là một trạng thái, không chỉ là một địa điểm*.

Chỗ không rút gọn được: Với đồ thị trạng thái, việc ước lượng số trạng thái phải làm bằng tay cho từng bài; không có cách nào tự động biết nó là 10^6 hay 10^{15} .

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ba dấu hiệu bài đồ thị	Quan hệ đôi một; ràng buộc thứ tự/phụ thuộc; và <i>trạng thái với bước chuyển</i> — dấu hiệu thứ ba đáng giá nhất.
••• Bảng chọn thuật toán đường đi ngắn nhất	Không trọng số: BFS. Trọng số 0/1: BFS 0–1 với deque. Không âm: Dijkstra. Có âm: Bellman–Ford. Mọi cặp, n nhỏ: Floyd–Warshall.
••• Chỗ Dijkstra dùng giả thiết không âm	Khi chốt đỉnh nhãn nhỏ nhất, nó tin rằng đường khác đi qua đỉnh chưa chốt chỉ có thể đắt hơn — đúng vì cạnh không làm giảm chi phí.
••• Bellman–Ford và chu trình âm	Thư giãn mọi cạnh $n - 1$ lần (đường ngắn nhất có $\leq n - 1$ cạnh); nếu vòng thứ n vẫn cải thiện thì có chu trình âm.
••• Duyệt sâu để lộ những gì	Thời điểm vào/ra cho: phát hiện chu trình, thứ tự tô-pô (ra giảm dần), cầu và khớp, thành phần liên thông mạnh.
••• Quy hoạch động và đường đi ngắn nhất	Là hai cách nói về cùng một thứ: đỉnh là trạng thái, cạnh là chuyển. Quy hoạch động có chu trình phụ thuộc $\Rightarrow$ dùng Dijkstra trên đồ thị trạng thái.
•• Ba mẹo mô hình hoá	Đồ thị trạng thái; đồ thị nhiều tầng (nhân bản theo số lần dùng khả năng đặc biệt); tách đỉnh (khi ràng buộc nằm ở đỉnh).
•• Duyệt rộng nhiều nguồn	Đẩy mọi nguồn vào hàng đợi lúc khởi tạo $\Rightarrow$ khoảng cách tới nguồn gần nhất trong một lượt $O(n + m)$.
•• Nhảy nhị phân	$up[v][k]$ = tổ tiên thứ 2^k ; tiền xử lý $O(n \log n)$, truy vấn tổ tiên chung $O(\log n)$; mở rộng được cho cực trị trên đường đi.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Dãy Euler	Biến bài tổ tiên chung thành bài cực tiểu trên đoạn ($B/10$); phép quy dẫn đi được cả hai chiều.
•• A*	Dijkstra cộng ước lượng khoảng cách còn lại; tối ưu nếu ước lượng không bao giờ vượt quá thực tế.
•• Lỗi Floyd–Warshall phổ biến	Vòng k phải ở ngoài cùng; đảo thứ tự vòng lặp cho kết quả sai.
• Phân rã theo cạnh nặng	Chia cây thành các chuỗi để truy vấn trên đường đi quy về $O(\log n)$ truy vấn đoạn; mạnh nhưng thuộc loại tham khảo.
• Rào cản sắp xếp	Dijkstra chốt đỉnh theo đúng thứ tự khoảng cách nên thừa hưởng cận dưới của sắp xếp. Vượt được bằng cách đổi mô hình (trọng số nguyên, đồ thị vô hướng) hoặc bằng cách không sắp xếp đầy đủ — $O(m \log^{2/3} n)$, 2025.

••• phải thuộc •• nên nắm • biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Khi nào tôi nên dùng ma trận kề thay vì danh sách kề?

| Chỉ khi n nhỏ (vài nghìn trở xuống) và đồ thị dày, hoặc khi bạn cần hỏi “có cạnh giữa u và v không” rất nhiều lần trong $O(1)$. Ma trận tốn $O(n^2)$ bộ nhớ nên với $n = 10^5$ là không khả thi. Trong hầu hết bài thi đấu, danh sách kề là lựa chọn mặc định, và nếu cần kiểm tra cạnh nhanh thì dùng thêm một tập băm các cặp.

2. Dijkstra của tôi cho kết quả sai trên đồ thị có cạnh trọng số 0. Có phải vì trọng số 0?

| Không, trọng số 0 hoàn toàn hợp lệ với Dijkstra — giả thiết là *không âm*, và 0 không âm. Lỗi gần như chắc chắn nằm ở chỗ khác: quên bỏ qua bản ghi lỗi thời khi lấy ra khỏi hàng đợi ưu tiên, hoặc cập nhật nhãn nhưng không đẩy vào hàng đợi, hoặc so sánh sai khi hai nhãn bằng nhau. Nếu đồ thị chỉ có trọng số 0 và 1 thì hãy dùng duyệt rộng 0–1, vừa nhanh hơn vừa ít chỗ sai hơn.

3. Làm sao đếm số đường đi ngắn nhất chứ không chỉ độ dài?

| Thêm một mảng $\text{cnt}[v]$. Khi thư giãn cạnh (u, v) : nếu tìm được đường ngắn hơn thì $\text{cnt}[v] = \text{cnt}[u]$; nếu bằng đúng khoảng cách hiện tại thì $\text{cnt}[v] += \text{cnt}[u]$. Cách này hoạt động với cả duyệt rộng và Dijkstra, với điều kiện bạn xử lý mỗi đỉnh theo đúng thứ tự khoảng cách tăng dần. Nhớ lấy modulo nếu số đường đi lớn.

4. Bài của tôi có 10^6 đỉnh trạng thái nhưng tôi không muốn sinh trước toàn bộ đồ thị. Làm sao?

| Sinh cạnh *khi cần* — gọi là đồ thị ẩn. Thay vì lưu danh sách kề, viết một hàm “cho tôi các trạng thái kề của trạng thái này” và gọi nó trong vòng lặp duyệt. Cách này tiết kiệm bộ nhớ rất nhiều và là cách chuẩn cho các bài trò xếp hình hay bài đong nước. Điều cần thêm là một cấu trúc để đánh dấu trạng thái đã thăm — thường là bảng băm nếu trạng thái không ánh xạ gọn về số nguyên.

5. Thứ tự tô-pô có thể có nhiều kết quả. Làm sao lấy kết quả nhỏ nhất theo thứ tự từ điển?

| Dùng thuật toán Kahn nhưng thay hàng đợi bằng hàng đợi ưu tiên: luôn lấy ra đỉnh có chỉ số nhỏ nhất trong số các đỉnh không còn phụ thuộc. Chi phí thành $O(m \log n)$. Lưu ý đây là một câu hỏi khác với “thứ tự tô-pô bất kỳ”, và nhiều bài cổ tình hỏi bản khó hơn — hãy đọc kỹ đề.

6. Vì sao cùng một bài lại có bài giải bằng luồng cực đại, bài giải bằng Dijkstra?

| Vì hai họ bài toán trả lời hai câu hỏi khác nhau. Đường đi ngắn nhất hỏi về *một* đường; luồng hỏi về *nhiều* đường đồng thời với ràng buộc sức chứa. Dấu hiệu phân biệt trên đề: nếu có khái niệm “sức chứa”, “mỗi thứ chỉ dùng một lần”, “phân công” thì đó là luồng hoặc ghép đôi ([C/18](#)); nếu chỉ có chi phí đi lại thì đó là đường đi ngắn nhất.

7. Trong hệ thống thật, Dijkstra có đủ cho định tuyến bản đồ không?

| Không ở quy mô thật. Mạng đường bộ một quốc gia có hàng chục triệu đỉnh, và Dijkstra thuần túy mất hàng giây cho một truy vấn — quá chậm cho dịch vụ trực tuyến. Các hệ thống thật dùng tiền xử lý nặng: phân cấp co rút, nhãn khoảng cách, hoặc phân vùng — những kỹ thuật đổi hàng giờ tiền xử lý lấy truy vấn dưới mili-giây. Nhưng tất cả đều xây trên Dijkstra và đều dùng đúng tính chất tham lam của nó, nên hiểu Dijkstra vẫn là điều kiện cần.

8. Phát hiện chu trình trong đồ thị vô hướng và có hướng khác nhau thế nào?

| Khác ở tiêu chí. Trong đồ thị vô hướng, gặp một đỉnh đã thăm mà không phải cha trực tiếp là có chu trình — hoặc đơn giản hơn, dùng DSU (B/11): cạnh nối hai đỉnh cùng nhóm tạo chu trình. Trong đồ thị có hướng, phải phân biệt “đã thăm xong” với “đang trên ngăn xếp”: chỉ cạnh trở về một đỉnh *đang trên ngăn xếp* mới tạo chu trình. Nhầm hai trạng thái này là một trong những lỗi phổ biến nhất khi cài.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Đếm số thành phần liên thông	CSES “Counting Rooms”[33]	Duyệt sâu hoặc rộng trên lưới; chú ý độ sâu đệ quy với lưới lớn.
● Đường đi ngắn nhất trên lưới có vật cản	CSES “Labyrinth”[33]	Duyệt rộng kèm truy vết đường đi; bài mẫu cho việc lưu “đỉnh trước”.
● Kiểm tra đồ thị hai phía	CSES “Building Teams”[33]	Tô màu bằng duyệt rộng; so với cách DSU chặn lẻ ở B/11.
●● Đường đi ngắn nhất có trọng số	CSES “Shortest Routes I”[33]	Dijkstra với hàng đợi ưu tiên; nhớ dòng bỏ qua bản ghi lỗi thời.
●● Phát hiện chu trình âm	CSES “Cycle Finding”[33]	Bellman–Ford và truy vết chu trình; chú ý đỉnh không tới được từ nguồn.
●● Sắp thứ tự tô-pô và đường đi dài nhất trên đồ thị không chu trình	CSES “Course Schedule”, “Longest Flight Route”[33]	Quy hoạch động theo thứ tự tô-pô — ví dụ trực tiếp cho quan hệ ở mục 3.
●● Bài đong nước với hai bình	Bài kinh điển	Đồ thị trạng thái; ước lượng số trạng thái trước khi cài.
●● Số đường đi ngắn nhất giữa hai đỉnh	CSES “Investigation”[33]	Dijkstra kèm bốn đại lượng cùng lúc; luyện việc mở rộng thuật toán chuẩn.
●●● Thành phần liên thông mạnh và nén đồ thị	CSES “Planets and Kingdoms”[33]	Kosaraju hoặc Tarjan; sau khi nén thì chạy quy hoạch động trên đồ thị không chu trình.
●●● Tìm cầu và khớp	Bài kinh điển	Duyệt sâu với giá trị “lên cao nhất”; chú ý xử lý cạnh song song.
●●● Tổ tiên chung gần nhất với 10^5 truy vấn	CSES “Company Queries II”[33]	Nhảy nhị phân; rồi thử lại bằng dãy Euler cộng sparse table và so thời gian.
●●● Đường đi ngắn nhất khi được đi miễn phí tối đa k cạnh	Bài thi đấu kinh điển	Đồ thị nhiều tầng; đây là bài mẫu cho mẹo ở mục 5.
●●● A^* cho bài xếp hình 15 ô	Bài kinh điển	So số đỉnh phải xét giữa duyệt rộng, Dijkstra và A^* với hai hàm ước lượng khác nhau.

Kết chương và đọc tiếp

Đồ thị là chương của kỹ năng mô hình hoá, và ba câu hỏi ở khối *Cần nhớ* — đỉnh là gì, cạnh là gì, trọng số là gì — đáng thành phần xạ. Hai điều nữa đáng mang theo: đổi kho chờ thì đổi thuật toán, nên duyệt rộng, duyệt sâu và Dijkstra là ba biến thể của một khung; và quy hoạch động chính là đường đi ngắn nhất trên đồ thị trạng thái, nên hai chương này là hai cách nói về cùng một thứ.

Chương tiếp theo mở rộng mô hình đồ thị theo một hướng mới: thay vì hỏi về *một* đường đi, ta hỏi về *nhiều* đường đi đồng thời với ràng buộc sức chứa. Đó là luồng cực đại, và điều bất ngờ là rất nhiều bài không liên quan gì tới “dòng chảy” — phân công công việc, chọn tập đại diện, một số bài chia đôi tối ưu — đều quy về đúng mô hình đó (C/18).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Dijkstra, “A Note on Two Problems in Connexion with Graphs”	Numerische Mathematik, 1959	Ba trang, không có ký hiệu tiệm cận nào; đọc để thấy thuật toán được nghĩ ra như thế nào trước khi ngành có ngôn ngữ để nói về nó.
nền tảng	Duan, Mao, Mao, Shu, Yin, “Breaking the Sorting Barrier for Directed Single-Source Shortest Paths”	STOC, 2025 (Best Paper)	Đọc phần mở đầu: nó giải thích rào cản sắp xếp và ý tưởng tránh sắp xếp rõ hơn bất kỳ bài giới thiệu nào.
tra khi cần	Thorup, “Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time”	Journal of the ACM, 1999	Ví dụ mẫu mực về việc đổi mô hình (số nguyên, vô hướng) để thoát một cận dưới.
tra khi cần	Tarjan, “Depth-First Search and Linear Graph Algorithms”	SIAM Journal on Computing, 1972	Bài gốc của thành phần liên thông mạnh; đọc kèm mục 2 của chương này.
tra khi cần	Fredman & Tarjan, “Fibonacci Heaps...”	Journal of the ACM, 1987	Nguồn của cận $O(m + n \log n)$; xem thêm phần đo thực nghiệm ở B/07 trước khi quyết định dùng.

Luồng cực đại và ghép đôi (Max Flow, Matching, Min Cut)

PHỤ THUỘC C/17 bắt buộc — toàn bộ chương này là mô hình đồ thị mở rộng; A/04 cho tính đúng của Ford–Fulkerson và cho chứng minh định lý luồng cực đại–lát cắt nhỏ nhất.

TÓM TẮT

Luồng cực đại là ví dụ mạnh nhất trong quyển sách này cho sức mạnh của việc mô hình hoá: rất nhiều bài không hề nói tới “dòng chảy” — phân công người vào việc, chọn dự án có lợi nhất, phân đoạn ảnh, kiểm tra một đội bóng còn cơ hội vô địch — đều quy về đúng một mô hình. Trung tâm lý thuyết của chương là định lý luồng cực đại bằng lát cắt nhỏ nhất, một sự trùng khớp giữa bài toán tối đa và bài toán tối thiểu; đó cũng là lý do luồng vừa cho lời giải vừa cho *chứng cứ* rằng lời giải là tối ưu. Chương tập trung vào mô hình hoá và các mẹo dựng đồ thị, còn phần cài đặt chỉ ở mức đủ dùng. Nếu chỉ nhớ một điều: khi đề bài có “sức chứa”, “mỗi thứ dùng một lần”, hoặc “chọn một trong hai phía sao cho ít mâu thuẫn nhất”, hãy nghĩ tới luồng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mạng luồng** — đồ thị có hướng với một đỉnh nguồn, một đỉnh đích, mỗi cạnh có sức chứa. Luồng là cách gán giá trị cho các cạnh sao cho không vượt sức chứa và mọi đỉnh trung gian có lượng vào bằng lượng ra.
- **đồ thị thặng dư** — đồ thị phụ ghi lại phần sức chứa còn lại và các cạnh ngược cho phép “rút lại” luồng đã đẩy — ý tưởng then chốt làm thuật toán đúng.
- **đường tăng luồng** — đường từ nguồn tới đích trong đồ thị thặng dư; còn đường như vậy thì luồng còn tăng được.
- **lát cắt** — cách chia tập đỉnh thành hai phần, một chứa nguồn một chứa đích; giá của lát cắt là tổng sức chứa các cạnh đi từ phần nguồn sang phần đích.
- **ghép đôi** — tập cạnh đôi một không chung đỉnh. Ghép đôi cực đại trên đồ thị hai phía quy về luồng cực đại với mọi sức chứa bằng 1.
- **phủ đỉnh** — tập đỉnh chạm tới mọi cạnh. Trên đồ thị hai phía, phủ đỉnh nhỏ nhất bằng ghép đôi lớn nhất — định lý König.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao luồng cực đại lại đúng bằng lát cắt nhỏ nhất — hai bài toán trông không liên quan gì tới nhau?
- Q2. Cạnh ngược trong đồ thị thặng dư cho phép “hoàn tác” luồng đã đẩy. Nếu bỏ chúng đi thì thuật toán hỏng ở đâu?
- Q3. Bài phân công n người vào n việc không nhắc gì tới dòng chảy. Vì sao nó lại là bài luồng?
- Q4. Ford–Fulkerson có thể không dừng với sức chứa vô tỉ. Điều đó nói gì về vai trò của “đại lượng giảm” trong chứng minh tính dừng?
- Q5. Vì sao trong phân đoạn ảnh, người ta lại đi tìm lát cắt nhỏ nhất chứ không tìm luồng?

Chương trước kết thúc bằng nhận xét rằng đường đi ngắn nhất hỏi về *một* đường. Nhưng rất nhiều bài toán thực tế hỏi về nhiều đối tượng cùng lúc, cạnh tranh nhau một nguồn lực có hạn: nhiều gói tin cùng chạy qua một mạng có băng thông giới hạn, nhiều người cùng được phân vào nhiều việc mà mỗi việc chỉ nhận một người, nhiều đường ống cùng dẫn nước qua các van có sức chứa khác nhau.

Mô hình luồng ra đời cho đúng lớp bài toán đó, và điều làm nó có giá trị vượt xa mục đích ban đầu là một hiện tượng lặp đi lặp lại: những bài toán trông chẳng liên quan gì tới dòng chảy hoá ra lại có cùng cấu trúc. Vì vậy chương này sẽ dành phần lớn trọng lượng cho câu hỏi *nhận diện và dựng mô hình*, và chỉ nói về thuật toán ở mức đủ để bạn chọn đúng và cài được.

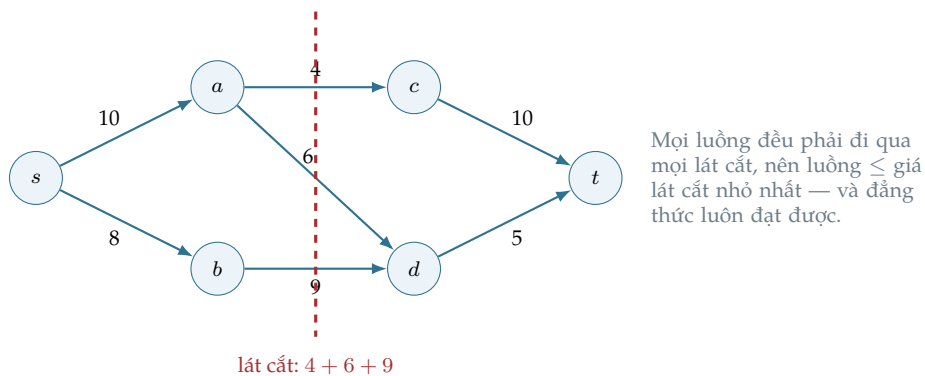
18.1 Luồng, lát cắt, và một sự trùng khớp đáng kinh ngạc

Bài toán: cho mạng với sức chứa trên mỗi cạnh, đẩy nhiều nhất bao nhiêu đơn vị từ nguồn tới đích?

Thuật toán cơ bản rất trực giác: tìm một đường từ nguồn tới đích còn chỗ trống, đẩy thêm luồng dọc đường đó, lặp lại. Điều duy nhất không hiển nhiên — và là ý tưởng then chốt của cả lý thuyết — là mỗi khi đẩy x đơn vị qua một cạnh, ta phải thêm một *cạnh ngược* sức chứa x vào đồ thị thặng dư. Cạnh ngược cho phép các bước sau “rút lại” quyết định cũ nếu nó hoá ra không tối ưu.

Vì sao cần? Vì nếu không có nó, thuật toán trở thành tham lam không hồi tiếc, và như chương C/15 đã chỉ ra, tham lam sai khi lựa chọn hôm nay phá huỷ khả năng tương lai. Đường đi đầu tiên có thể chiếm mất một cạnh hẹp mà lẽ ra nên để dành cho luồng khác. Cạnh ngược biến thuật toán tham lam thành thuật toán có khả năng tự sửa, và đó chính xác là điều làm nó tối ưu.

Kết quả trung tâm là *định lý luồng cực đại bằng lát cắt nhỏ nhất*: giá trị luồng lớn nhất đúng bằng giá của lát cắt nhỏ nhất. Chiều dễ hiểu ngay: mọi luồng đều phải đi qua mọi lát cắt, nên luồng không thể vượt giá của lát cắt nào. Chiều khó là chứng minh tồn tại một lát cắt *đạt* giá trị đó, và nó có một chứng minh rất đẹp: khi thuật toán dừng vì không còn đường tăng, hãy lấy tập đỉnh còn tới được từ nguồn trong đồ thị thặng dư — tập đó chính là một lát cắt có giá bằng luồng hiện tại.



Hình 18.1. Một mạng luồng và một lát cắt. Lát cắt vừa là chặn trên cho luồng vừa là *lời giải* của nhiều bài toán mô hình hoá (mục 4). Khi thuật toán dừng, tập đỉnh còn tới được từ s trong đồ thị thặng dư cho đúng một lát cắt đạt giá trị luồng.

TRỰC GIÁC

Vì sao một bài toán tối đa lại bằng một bài toán tối thiểu? Đây là hiện tượng *đối ngẫu*, và nó phổ biến hơn nhiều người nghĩ. Trực giác: luồng là “đẩy được bao nhiêu”, lát cắt là “chặn tồn ít nhất bao nhiêu”. Mỗi lát cắt là một *bằng chứng* chặn trên cho luồng; nếu tìm được lát

cắt có giá bằng đúng luồng hiện tại thì hai bên kẹp nhau và cả hai đều tối ưu. Đây là lý do luồng có tính chất hiếm: nó không chỉ cho lời giải mà còn cho *chứng cứ kiểm tra được* rằng lời giải là tối ưu. Cùng hiện tượng xuất hiện trong quy hoạch tuyến tính, trong lý thuyết trò chơi, và trong nhiều bài tối ưu tổ hợp khác.

18.2 Ba thuật toán và giả thiết của chúng

Ford–Fulkerson là khung chung: cứ tìm đường tăng bất kỳ. Chi phí phụ thuộc cách chọn đường: với sức chứa nguyên, mỗi vòng tăng ít nhất một đơn vị nên số vòng bị chặn bởi giá trị luồng — đó là *đại lượng giảm* theo nghĩa chương A/04. Nhưng nếu sức chứa là số vô tỉ thì các bước tăng có thể nhỏ dần vô hạn và thuật toán *không dừng*. Đây là một ví dụ hiếm và rất đáng nhớ về việc chứng minh tính dừng phụ thuộc vào giả thiết số học chứ không chỉ vào cấu trúc.

Edmonds–Karp chỉ định thêm một quy tắc: luôn chọn đường tăng *ngắn nhất* bằng duyệt rộng. Điều đó cho chặn $O(nm^2)$ độc lập với sức chứa, và giải quyết luôn vấn đề tính dừng. Bài học phương pháp rất đáng chú ý: một thuật toán không có đảm bảo tốt trở nên có đảm bảo chỉ nhờ *cổ định thứ tự lựa chọn*.

Dinic nhanh hơn nhiều và là lựa chọn thực tế: nó xây đồ thị phân tầng bằng duyệt rộng, rồi đẩy luồng chặn trên đồ thị đó bằng duyệt sâu. Chi phí $O(n^2m)$ nói chung, và $O(m\sqrt{n})$ trên mạng đơn vị của bài ghép đôi hai phía. Chỉ giả thiết mọi sức chứa bằng 1 chưa đủ để suy ra chặn này trên mạng tổng quát; cấu trúc vào/ra của các đỉnh trung gian cũng quan trọng[7]. Trong thực hành thi đấu, *Dinic* gần như luôn là lựa chọn đúng.

18.3 Ghép đôi hai phía và ba định lý anh em

Trường hợp riêng quan trọng nhất: đồ thị hai phía, mọi sức chứa bằng 1. Bài toán ghép đôi cực đại — phân công nhiều người vào nhiều việc sao cho số cặp lớn nhất — quy về luồng bằng cách thêm nguồn nối tới mọi đỉnh trái và đích nhận từ mọi đỉnh phải.

Ba định lý đi kèm, và chúng đáng thuộc vì chúng biến bài toán này thành bài toán khác:

Định lý König: trên đồ thị hai phía, ghép đôi lớn nhất bằng phủ đỉnh nhỏ nhất. Hệ quả: bài “chọn ít đỉnh nhất chạm mọi cạnh” — vốn NP-khó trên đồ thị tổng quát — lại giải được trong thời gian đa thức trên đồ thị hai phía.

Hệ quả về tập độc lập: tập độc lập lớn nhất bằng n trừ ghép đôi lớn nhất. Rất nhiều bài “chọn nhiều ô nhất trên bàn cờ sao cho không có hai ô xung khắc” rơi vào đúng đây.

Định lý Hall: tồn tại ghép đôi bão hoà phía trái khi và chỉ khi mọi tập con S của phía trái có số láng giềng ít nhất bằng $|S|$. Đây là công cụ để *chứng minh* một phân công là bất khả thi, chứ không chỉ để tìm phân công.

Với bài phân công có *chi phí* — mỗi người làm mỗi việc tốn một giá khác nhau, tìm phân công tổng chi phí nhỏ nhất — ta cần luồng chi phí nhỏ nhất hoặc thuật toán Hungarian[68]. Với $n \leq 20$, quy hoạch động mặt nạ bit (C/16) thường ngắn hơn và đủ nhanh; với n lớn hơn thì phải dùng thuật toán chuyên.

18.4 Mô hình hoá: nơi chương này thật sự trả công

Đây là mục quan trọng nhất. Dưới đây là các mẫu dựng mô hình hay gặp, kèm dấu hiệu nhận biết trên đề.

Mẫu 1 — phân công. Dấu hiệu: hai loại đối tượng, mỗi phần tử một loại ghép với một phần tử loại kia. Đồ thị hai phía, sức chứa 1.

Mẫu 2 — ràng buộc ở đỉnh. Dấu hiệu: “mỗi thành phố chỉ đi qua một lần”, “mỗi máy xử lý tối đa k việc”. Tách đỉnh thành hai nửa nối bởi một cạnh có sức chứa bằng đúng giới hạn đó — kỹ thuật đã nhắc ở C/17 và ở đây nó thành công cụ chính.

Mẫu 3 — chọn dự án có lợi nhất. Dấu hiệu: có các dự án cho lợi nhuận, mỗi dự án đòi hỏi một số thiết bị tốn chi phí, thiết bị dùng chung được. Đây là bài đóng lớn nhất, và nó quy về lát cắt nhỏ nhất: nối nguồn tới dự án với sức chứa bằng lợi nhuận, nối thiết bị tới đích với sức chứa bằng chi phí, nối dự án tới thiết bị nó cần với sức chứa vô hạn. Lợi nhuận lớn nhất bằng tổng lợi nhuận trừ lát cắt nhỏ nhất.

Mẫu 4 — chia hai phía sao cho ít mâu thuẫn nhất. Dấu hiệu: mỗi đối tượng phải được gán một trong hai nhãn; gán sai nhãn cho một đối tượng tốn một giá; hai đối tượng kề nhau mà khác nhãn cũng tốn một giá. Đây chính là mô hình lát cắt nhỏ nhất, và nó là nền của phân đoạn ảnh bằng cắt đồ thị.

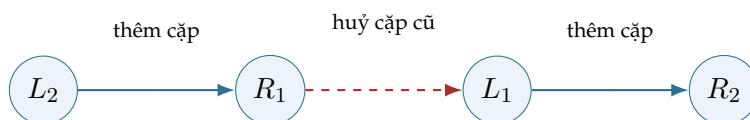
Mẫu 4 đáng được nói kỹ vì nó là cầu nối đẹp giữa chương này và thị giác máy tính. Khi ta muốn tách vật thể khỏi nền trong một bức ảnh, mỗi điểm ảnh phải nhận nhãn “vật” hoặc “nền”. Mỗi điểm ảnh có một chi phí gán nhãn dựa trên màu của nó, và hai điểm ảnh kề nhau có một chi phí nếu chúng nhận nhãn khác nhau — phần này thể hiện giả định rằng vùng ảnh thường liền mạch. Tổng chi phí đúng bằng giá của một lát cắt trong đồ thị mà đỉnh là điểm ảnh, nên tìm cách gán nhãn tốt nhất chính là tìm lát cắt nhỏ nhất. Đây là lý do câu hỏi mở đầu thứ năm có câu trả lời: người ta quan tâm tới lát cắt vì lát cắt chính là lời giải, còn luồng chỉ là phương tiện tính ra nó.

CẠM BẮY

Cạm bẫy phổ biến nhất khi dùng luồng không phải cài sai thuật toán mà là *dựng sai mô hình*, và cái sai đó im lặng: chương trình chạy, cho ra một con số, và con số đó trả lời một bài toán khác. Hai cách phòng. Thứ nhất, sau khi dựng xong, hãy kiểm tra bằng lời: “một đơn vị luồng trong mô hình này tương ứng với cái gì trong bài toán thật?” — nếu không trả lời được bằng một câu thì mô hình sai. Thứ hai, stress test với vết cặn trên dữ liệu rất nhỏ (E/25), vì mô hình sai gần như luôn lộ ra ở $n \leq 5$.

18.5 Cạnh ngược là quyền sửa một quyết định cũ

Cho hai người L_1, L_2 và hai việc R_1, R_2 ; các cặp hợp lệ là $L_1 R_1, L_1 R_2, L_2 R_1$. Nếu ghép $L_1 R_1$ trước, L_2 chưa có việc. Hình 18.2 cho thấy đường tăng dùng cạnh ngược để sửa cặp cũ.



Hình 18.2. Đường tăng xen kẽ $L_2 \rightarrow R_1 \rightarrow L_1 \rightarrow R_2$, lược bỏ nguồn và đích. Cạnh nét đứt là cạnh dư ngược của cặp đang ghép. Kết quả có hai cặp $L_2 R_1, L_1 R_2$, tăng từ 1 lên 2.

Khi đẩy Δ qua cạnh dư $u \rightarrow v$, giảm sức chứa dư cạnh đó và tăng cạnh đối ứng cùng lượng. Chỉ xoá bớt sức chứa chiều thuận sẽ biến thuật toán thành lựa chọn không thể sửa và có thể dừng ở luồng chưa cực đại. Trong cài đặt, lưu chỉ số cạnh đối ứng; hai cạnh gốc ngược chiều vẫn phải có các cặp cạnh dư riêng để xử lý chính xác đa cạnh. Sau khi chạy, kiểm sức chứa, bảo toàn luồng ở đỉnh trung gian và giá trị lát cắt từ các đỉnh còn tới được trong mạng dư.

18.6 Biên giới hiện nay: từ đường tăng tới tối ưu hoá liên tục

Dinic là lựa chọn thực hành, nhưng câu hỏi “luồng cực đại thật sự tốn bao nhiêu” có một lịch sử dài và vừa có bước ngoặt, và bước ngoặt ấy đáng kể lại vì nó đến từ một hướng hoàn toàn khác.

Suốt nửa thế kỷ, mọi cải tiến đều nằm trong khuôn tổ hợp: vẫn là đường tăng, luồng chặn, đẩy và gán nhãn, chỉ khác ở cách chọn và cách tổ chức dữ liệu. Đỉnh của dòng ấy là kết quả của Orlin[69], đưa luồng cực đại về $O(nm)$ cho mọi cặp n, m — con số mà nhiều người xem là điểm dừng tự nhiên của lối tiếp cận tổ hợp.

Bước ngoặt đến khi người ta ngừng coi luồng là bài toán tổ hợp. Nhìn từ tối ưu hoá, tìm luồng cực đại là giải một bài quy hoạch tuyến tính có cấu trúc rất đặc biệt, và người ta có thể đi tới lời giải bằng những bước *liên tục* — mỗi bước giải một bài toán dòng điện trên mạng, rồi sửa dần. Chen, Kyng, Liu, Peng, Probst Gutenberg và Sachdeva[70] hoàn thiện hướng này: luồng cực đại và luồng chi phí nhỏ nhất trên đồ thị có hướng với sức chứa và chi phí nguyên bị chặn đa thức, giải được trong thời gian $m^{1+o(1)}$ — *gần tuyến tính*, tức là chỉ hơn chi phí đọc dữ liệu vào một thừa số dưới mọi lũy thừa. Bài được trao giải bài báo xuất sắc tại FOCS 2022.

Hai điều đáng học ở đây, và không điều nào là “hãy cài thuật toán này”.

Thứ nhất, về phương pháp: bước nhảy có được nhờ *đổi ngôn ngữ* — rời khuôn tổ hợp sang khuôn tối ưu hoá liên tục — chứ không nhờ tinh chỉnh khuôn cũ. Và điều mĩ mai đẹp đẽ là cái làm cho hướng liên tục chạy được lại chính là cấu trúc dữ liệu động: mỗi bước liên tục chỉ rẽ nếu ta duy trì được thông tin về đồ thị đang thay đổi mà không tính lại từ đầu, đúng loại công việc của phần B. Lý thuyết thuật toán hiện đại có hình dạng như vậy: tối ưu hoá liên tục ở tầng trên, cấu trúc dữ liệu ở tầng dưới.

Thứ hai, về cách đọc kết quả: $m^{1+o(1)}$ giấu một thừa số dưới đa thức có thể rất lớn ở kích thước thật, nên trong mọi kỳ thi và gần như mọi hệ thống, Dinic vẫn là lựa chọn đúng, và ghép đôi hai phía vẫn nên dùng Hopcroft–Karp[71] với $O(m\sqrt{n})$. Nhưng kết quả này đổi một điều quan trọng cho *người mô hình hoá*: phát biểu lại bài toán của mình thành luồng chi phí nhỏ nhất giờ đây là một phép quy dẫn “gần như miễn phí” về mặt lý thuyết, nên rào cản còn lại chỉ là mô hình có đúng hay không — đúng thông điệp của mục 4.

18.7 Gốc rễ: mạng đường sắt Liên Xô và một bài toán về hôn nhân

Bài toán luồng cực đại có một nguồn gốc rất cụ thể và ít người biết. Năm 1955, trong bối cảnh Chiến tranh Lạnh, hai nhà nghiên cứu tại RAND Corporation là Harris và Ross viết một báo cáo phân tích năng lực vận chuyển của mạng đường sắt Liên Xô và các nước Đông Âu. Bài toán họ quan tâm có hai mặt: mạng này chuyển được tối đa bao nhiêu hàng, và — mặt được nhấn mạnh hơn — *cần cắt những tuyến nào, tốn ít nhất bao nhiêu, để chặn dòng vận chuyển*. Nghĩa là bài toán lát cắt nhỏ nhất được đặt ra như một bài toán quân sự, đồng thời với bài toán luồng cực đại.

Ford và Fulkerson, cũng tại RAND, đưa ra thuật toán và chứng minh định lý luồng cực đại bằng lát cắt nhỏ nhất vào năm 1956. Sự trùng khớp giữa hai bài toán — một của bên vận chuyển, một của bên phá hoại — vì thế không phải phát hiện lý thuyết thuần túy mà là câu trả lời cho một

câu hỏi thực tế được đặt ra từ cả hai phía cùng lúc. Đây là ví dụ mẫu mực cho việc bối cảnh sinh ra khái niệm: hai nửa của định lý đối ngẫu ứng với hai bên của một tình huống thật.

Nhánh ghép đôi thì cổ hơn và đến từ toán học thuần túy. König chứng minh định lý về ghép đôi và phủ đỉnh trên đồ thị hai phía năm 1931; Hall đưa ra điều kiện tồn tại ghép đôi bão hoà năm 1935, trong một bài toán thường được kể dưới cái tên “định lý hôn nhân”. Cả hai kết quả ra đời trước khi có máy tính và không nhằm mục đích thuật toán nào. Đến năm 1955, Kuhn kết hợp các ý tưởng của König và Egerváry thành một thuật toán cho bài toán phân công — và ông đặt tên nó là “phương pháp Hungary” để ghi công hai nhà toán học Hungary đó[68], một cử chỉ hiếm thấy trong việc đặt tên thuật toán.

Cuối cùng, một liên hệ hiện đại đáng chú ý: đầu những năm 2000, giới thị giác máy tính phát hiện ra rằng bài toán gán nhãn ảnh với ràng buộc trơn có thể giải *chính xác* bằng lát cắt nhỏ nhất, thay vì bằng các phương pháp lặp không có đảm bảo. Điều này biến một thuật toán vận trù học của thập niên 1950 thành công cụ chủ lực cho phân đoạn ảnh và cho ước lượng độ sâu từ ảnh lập thể trong suốt một thập kỷ, cho tới khi học sâu thay đổi bức tranh.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1931 — König	Quan hệ giữa ghép đôi và phủ đỉnh trên đồ thị hai phía.	Định lý König; biến bài phủ đỉnh trên đồ thị hai phía thành bài đa thức.
1935 — Hall	Khi nào thì ghép được mọi phần tử của một phía?	Điều kiện Hall; công cụ để chứng minh <i>bất khả thi</i> , không chỉ để tìm lời giải.
1955 — Harris & Ross (RAND)	Năng lực mạng đường sắt Liên Xô, và cần cắt tuyến nào để chặn.	Bài toán luồng cực đại và lát cắt nhỏ nhất được đặt ra cùng lúc, từ hai phía của một tình huống thật.
1955 — Kuhn	Bài toán phân công người vào việc với chi phí.	Phương pháp Hungary, dựa trên kết quả của König và Egerváry.
1956 — Ford & Fulkerson	Cần thuật toán và cần biết khi nào dừng được.	Thuật toán đường tăng và định lý luồng cực đại bằng lát cắt nhỏ nhất — luồng vừa cho lời giải vừa cho chứng cứ tối ưu.
1970–72 — Dinic; Edmonds & Karp	Ford–Fulkerson có thể rất chậm hoặc không dừng.	Cổ định thứ tự chọn đường (ngắn nhất trước) cho chặn độc lập với sức chứa; Dinic thêm đồ thị phân tầng, cho $O(m\sqrt{n})$ trên mạng đơn vị.
Đầu thập niên 2000 — thị giác máy tính	Gán nhãn điểm ảnh với ràng buộc trơn, các phương pháp lặp không có đảm bảo.	Cắt đồ thị cho lời giải <i>chính xác</i> cho một lớp bài năng lượng; luồng trở thành công cụ chủ lực của phân đoạn ảnh và ảnh lập thể suốt một thập kỷ.

18.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính	Cắt đồ thị cho phân đoạn ảnh và ảnh lập thể	Mô hình mẫu 4 ở mục 4: chi phí gán nhãn cộng chi phí khác nhãn giữa hai điểm kề. Điểm đáng nhớ: đây là một trong số ít bài năng lượng trong thị giác máy tính có lời giải <i>tối ưu toàn cục</i> trong thời gian đa thức.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Quy hoạch tuyến tính	Đối ngẫu: bài cực đại và bài cực tiểu có cùng giá trị tối ưu	Luồng-lát cắt là một trường hợp riêng đẹp của đối ngẫu tuyến tính. Ý tưởng chung rất mạnh: nghiệm của bài đối ngẫu là <i>chứng cứ</i> cho tính tối ưu của bài gốc.
Kinh tế học	Bài toán vận tải, phân bổ nguồn lực, thị trường ghép cặp	Cùng mô hình; ghép cặp ổn định (bệnh viện–bác sĩ nội trú, trường–học sinh) là họ hàng gần và đã được dùng trong các hệ thống phân bổ thật.
Mạng máy tính	Bảng thông, định tuyến đa đường, độ tin cậy mạng	Bài toán gốc. Số đường đi rời cạnh giữa hai đỉnh bằng luồng cực đại với mọi sức chứa bằng 1 — định lý Menger, một cách phát biểu khác của cùng kết quả.
Thể thao, lập lịch	Đội nào còn cơ hội vô địch	Bài toán loại trừ: dựng mạng phân phối các trận còn lại; nếu luồng không bão hoà thì đội đang xét chắc chắn bị loại. Ví dụ điển hình cho việc bài toán không có “dòng chảy” nào vẫn là bài luồng.
Sản xuất	Lập lịch máy, phân công ca	Ràng buộc ở đỉnh (mỗi máy tối đa k việc) xử lý bằng tách đỉnh; ràng buộc cận dưới (mỗi ca phải có ít nhất 2 người) xử lý bằng luồng có cận dưới.
SLAM và robot	Chọn tập ràng buộc nhất quán, phân công nhiệm vụ cho đội robot	Phân công nhiệm vụ nhiều robot là bài ghép đôi có trọng số — đúng bài toán Hungarian. Việc chọn tập quan sát nhất quán thì gần với các bài chọn tập con hơn (C/15), nhưng khi ràng buộc là “mỗi điểm mốc ghép với đúng một quan sát” thì lại quay về ghép đôi.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì mỗi lát cắt là một *chặn trên* cho mọi luồng — luồng nào cũng phải đi qua nó — nên luồng cực đại không vượt lát cắt nhỏ nhất. Chiều còn lại được chứng minh bằng cách xây dựng: khi thuật toán dừng vì không còn đường tăng, tập các đỉnh còn tới được từ nguồn trong đồ thị thặng dư tạo thành một lát cắt có giá đúng bằng luồng hiện tại. Hai bên kẹp nhau nên bằng nhau. Đây là hiện tượng đối ngẫu, và ý nghĩa thực dụng là luồng cho ta cả lời giải lẫn chứng cứ kiểm tra được rằng nó tối ưu.
- A2. Thuật toán trở thành tham lam không hối tiếc và sẽ cho kết quả dưới mức tối ưu. Đường tăng đầu tiên có thể chiếm một cạnh hẹp mà lẽ ra nên dành cho luồng khác, và không có cách nào sửa. Cạnh ngược cho phép các bước sau rút lại một phần quyết định cũ, biến thuật toán thành có khả năng tự sửa — đúng thứ mà chương C/15 nói là điều kiện để tham lam đúng, chỉ ở đây ta đạt được bằng cách thêm khả năng hối tiếc thay vì bằng chứng minh.
- A3. Vì bài phân công có đúng cấu trúc “mỗi bên dùng một lần”: dựng đồ thị hai phía, nguồn nối tới mọi người với sức chứa 1, mỗi người nối tới các việc họ làm được, mỗi việc nối tới đích với sức chứa 1. Sức chứa 1 trên các cạnh từ nguồn và tới đích chính là cách mã hoá ràng buộc “mỗi người một việc, mỗi việc một người”. Một đơn vị luồng tương ứng với một cặp được ghép — và câu này chính là bài kiểm tra ở khối *Cạm bẫy*.
- A4. Nó cho thấy tính dừng không phải là hệ quả tự nhiên của việc “mỗi bước đều cải thiện”. Với sức chứa nguyên, mỗi bước tăng ít nhất một đơn vị nên có một đại lượng nguyên không âm giảm — và đó là chứng minh dừng. Với sức chứa vô tỉ, các bước tăng có thể nhỏ dần theo cấp số nhân và tổng hội tụ về một giá trị nhỏ hơn tối ưu, nên thuật toán chạy mãi. Đại lượng giảm phải bị chặn dưới bởi một lượng dương cố định, không chỉ đơn thuần là “giảm”.
- A5. Vì trong bài phân đoạn ảnh, lát cắt chính là lời giải: nó nói mỗi điểm ảnh thuộc phía nào, tức là được gán nhãn gì. Luồng chỉ là phương tiện để tính ra lát cắt nhỏ nhất, và giá trị luồng chỉ là tổng chi phí. Đây là một chỗ đáng nhớ về mặt tư duy: khi mô hình hoá bằng luồng, hãy luôn hỏi bạn cần *giá trị* hay cần *cấu hình* — và nếu cần cấu hình thì nó thường nằm ở phía lát cắt.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một hệ thống ống nước: một bể nguồn, một bể đích, và nhiều đoạn ống nối nhau, mỗi đoạn chịu được một lưu lượng tối đa. Câu hỏi là bơm được nhiều nhất bao nhiêu nước từ nguồn tới đích.

Cách làm rất tự nhiên: tìm một đường ống còn chỗ trống từ nguồn tới đích, bơm thêm nước vào đó, rồi lặp lại. Chỉ có một mẹo phải nhớ, và đó là điểm tinh tế duy nhất của cả chương: mỗi khi đã bơm nước qua một đoạn ống, hãy ghi nhớ rằng ta có *quyền rút lại* phần nước đó nếu về sau thấy nên dùng ống ấy cho luồng khác. Không có quyền rút lại thì lần bơm đầu tiên có thể lỡ chiếm mất một đoạn ống hẹp quan trọng, và ta bị kẹt ở kết quả không tốt nhất.

Điều đẹp nhất của bài toán này là một sự trùng khớp. Nếu bạn hỏi ngược lại — “muốn chặn hoàn toàn dòng nước thì phải cắt những đoạn ống nào cho ít tổn kém nhất” — thì câu trả lời của bài chặn đúng bằng câu trả lời của bài bơm. Trục giác thì rõ: mọi giọt nước đều phải đi qua chỗ hẹp nhất, nên chỗ hẹp nhất quyết định lượng nước. Điều đáng giá là chiều ngược lại cũng đúng, nên khi tìm ra cách chặn rẻ nhất là bạn có luôn bằng chứng rằng không thể bơm nhiều hơn.

Và đây là lý do mô hình này quan trọng hơn hẳn phạm vi ống nước. Bài phân công nhân viên vào công việc chính là bài này với mỗi ống chịu được đúng một đơn vị. Bài tách vật thể khỏi nền trong ảnh cũng là bài này: mỗi điểm ảnh phải chọn một trong hai phía, và “chi phí cắt” chính là chi phí đặt ranh giới. Khi bạn thấy đề bài nói tới sức chứa, hoặc “mỗi thứ dùng một lần”, hoặc “chia hai phía sao cho ít mâu thuẫn nhất” — hãy nghĩ tới ống nước.

Một câu: *Cắt chỗ hẹp nhất và bơm nhiều nhất là hai câu hỏi cho cùng một đáp số.*

Chỗ không rút gọn được: Việc dựng đúng mô hình cho một bài mới vẫn đòi kinh nghiệm; mẫu ở mục 4 chỉ phủ những dạng hay gặp, không phủ hết.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng then chốt của thuật toán	Cạnh ngược trong đồ thị thặng dư: cho phép rút lại luồng đã đẩy, biến tham lam thành thuật toán tự sửa được.
••• Định lý luồng-lát cắt	Luồng cực đại bằng lát cắt nhỏ nhất; lát cắt là chặn trên, và tập đỉnh tới được từ nguồn khi thuật toán dừng cho lát cắt đạt đúng giá trị đó.
••• Dấu hiệu bài luồng trên đề	Có “sức chứa”; “mỗi thứ dùng đúng một lần”; “phân công hai loại đối tượng”; “chia hai phía sao cho ít mâu thuẫn nhất”.
••• Ghép đôi hai phía	Quy về luồng với mọi sức chứa 1; Dinic cho $O(m\sqrt{n})$. Một đơn vị luồng = một cặp được ghép.
••• Định lý König và hệ quả	Trên đồ thị hai phía: ghép đôi lớn nhất = phủ đỉnh nhỏ nhất; tập độc lập lớn nhất = n trừ ghép đôi lớn nhất.
••• Bốn mẫu mô hình hoá	Phân công; tách đỉnh khi ràng buộc ở đỉnh; chọn dự án (bài đóng lớn nhất) qua lát cắt; gán nhãn hai phía với chi phí trơn.
•• Vì sao Ford–Fulkerson có thể không dừng	Với sức chứa vô tỉ, các bước tăng nhỏ dần vô hạn; đại lượng giảm phải bị chặn dưới bởi một lượng dương cố định.
•• Edmonds–Karp	Luôn chọn đường tăng ngắn nhất (duyệt rộng) $\Rightarrow O(nm^2)$, độc lập với sức chứa. Bài học: cố định thứ tự lựa chọn tạo ra đảm bảo.
•• Định lý Hall	Ghép bão hoà phía trái tồn tại khi và chỉ khi mọi tập con S có ít nhất $ S $ láng giềng — dùng để chứng minh <i>bất khả thi</i> .
•• Bài phân công có chi phí	Luồng chi phí nhỏ nhất hoặc Hungarian; với $n \leq 20$ thì quy hoạch động mặt nạ bit thường ngắn hơn ($C/16$).
•• Cạm bẫy lớn nhất	Dựng sai mô hình, và sai im lặng. Kiểm bằng câu “một đơn vị luồng tương ứng với cái gì” và bằng stress test ở $n \leq 5$.
• Định lý Menger	Số đường đi rời cạnh giữa hai đỉnh bằng số cạnh ít nhất phải bỏ để ngắt liên thông — một cách phát biểu khác của định lý luồng-lát cắt.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
Luồng cực đại tốn bao nhiêu	Dòng tổ hợp dừng ở $O(nm)$ (2013); hướng tối ưu hoá liên tục cộng cấu trúc dữ liệu động cho $m^{1+o(1)}$ (2022). Thực hành vẫn là Dinic, và ghép đôi hai phía vẫn là Hopcroft–Karp.

● ● ● phải thuộc ● ● nên nắm ● biết là có

▮ Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên cài thuật toán luồng nào cho thi đấu?

| Dinic là một lựa chọn khởi đầu thực dụng. Nó đủ nhanh cho gần như mọi bài có giới hạn hợp lý, đặc biệt mạnh trên mạng ghép đôi hai phía nơi nó cho $O(m\sqrt{n})$. Edmonds–Karp đáng biết về mặt lý thuyết nhưng chậm hơn nhiều trong thực tế. Ford–Fulkerson thuần tuý thì dừng dừng vì chi phí phụ thuộc sức chứa. Hãy có sẵn một bản Dinic đã stress test trong thư viện riêng (A/02).

2. Làm sao xử lý ràng buộc “mỗi đỉnh chỉ đi qua một lần”?

| Tách đỉnh: mỗi đỉnh v thành $v_{\text{vào}}$ và v_{ra} , nối bởi một cạnh sức chứa 1 (hoặc bằng giới hạn của đỉnh). Mọi cạnh vào v nay nối tới $v_{\text{vào}}$, mọi cạnh ra khỏi v xuất phát từ v_{ra} . Kỹ thuật này chuyển ràng buộc từ đỉnh sang cạnh — nơi mô hình luồng biểu diễn được — và nó là một trong hai ba mẹo cần thuộc của chương.

3. Bài của tôi cần mỗi cạnh có cả cận dưới lẫn cận trên. Làm sao?

| Đó là bài luồng có cận dưới, và có một phép biến đổi chuẩn: với mỗi cạnh có cận dưới l , trừ l khỏi sức chứa và bù lại bằng cách thêm luồng cưỡng bức vào hai đầu qua một nguồn và đích phụ. Sau biến đổi, việc tồn tại luồng hợp lệ tương đương với việc mạng phụ bão hoà. Kỹ thuật này không khó nhưng nhiều chi tiết, nên hãy cài một lần cho kỹ và cất vào thư viện.

4. Vì sao bài chọn dự án lại là lát cắt chứ không phải luồng?

| Vì cái ta cần là *tập dự án được chọn*, tức là một cách chia đỉnh thành hai phía — đúng định nghĩa của lát cắt. Cách dựng: nguồn nối tới mỗi dự án với sức chứa bằng lợi nhuận, mỗi thiết bị nối tới đích với sức chứa bằng chi phí, và dự án nối tới thiết bị nó cần với sức chứa vô hạn. Cạnh vô hạn bảo đảm không thể chọn dự án mà không trả tiền thiết bị. Lợi nhuận lớn nhất bằng tổng lợi nhuận trừ giá lát cắt nhỏ nhất.

5. Khi nào bài ghép đôi *không* quy về luồng được?

| Khi đồ thị không phải hai phía. Ghép đôi cực đại trên đồ thị tổng quát cần thuật toán Blossom của Edmonds, phức tạp hơn hẳn và hiếm khi xuất hiện trong thi đấu. Đây cũng là chỗ ranh giới lý thuyết thú vị: nhiều bài dễ trên đồ thị hai phía lại khó hơn hẳn hoặc NP-khó trên đồ thị tổng quát — phủ định là ví dụ rõ nhất (D/23).

6. Tôi dựng mô hình xong nhưng không chắc đúng. Kiểm thế nào?

| Ba bước, theo thứ tự rẻ dần. Một, trả lời bằng lời câu “một đơn vị luồng ứng với gì” và “một lát cắt ứng với gì” — nếu áp ứng thì mô hình sai. Hai, thử tay một ví dụ $n = 2$ hoặc $n = 3$ và so với đáp án tính tay. Ba, stress test với vết cận trên $n \leq 5$. Bước ba tốn mười phút và bắt được gần như mọi lỗi mô hình — rẻ hơn nhiều so với ngồi đọc lại code.

7. Luồng chi phí nhỏ nhất khác luồng cực đại thế nào về mặt tư duy?

| Luồng cực đại hỏi “đẩy được bao nhiêu”; luồng chi phí nhỏ nhất hỏi “đẩy đúng lượng này với tổng chi phí nhỏ nhất”. Thuật toán chuẩn thay việc tìm đường tăng bất kỳ bằng việc tìm đường tăng *rẻ nhất* — tức là chạy Bellman–Ford hoặc Dijkstra với thế năng trên đồ thị thặng dư. Điều cần chú ý: đồ thị thặng dư có cạnh ngược với chi phí âm, nên Dijkstra thuần tuý không dùng được và phải chuyển đổi trọng số (C/17).

▮ Bài tập (mức: ● khởi động ● ● chuẩn ● ● ● khó)

Bài	Nguồn	Điều cần nhận ra
• Ghép đôi cực đại trên đồ thị hai phía	CSES “School Dance”[33]	Dựng mạng và chạy Dinic; giải thích ý nghĩa của mỗi đơn vị luồng.
•• Luồng cực đại cơ bản	CSES “Download Speed”[33]	Chú ý cạnh song song và cạnh hai chiều.
•• Số đường đi rời cạnh giữa hai đỉnh	CSES “Distinct Routes”[33]	Sức chứa 1; truy vết các đường sau khi có luồng.
•• Phủ đỉnh nhỏ nhất và tập độc lập lớn nhất trên đồ thị hai phía	Bài kinh điển	Dùng định lý König; dựng phủ đỉnh từ ghép đôi bằng cách duyệt xen kẽ.
•• Đặt quân xe trên bàn cờ có ô cấm sao cho không ăn nhau	Bài kinh điển	Hàng và cột là hai phía — một bài mà mô hình hoá là toàn bộ lời giải.
••• Bài chọn dự án có lợi nhuận lớn nhất	Bài kinh điển	Lát cắt nhỏ nhất với cạnh vô hạn; tính lợi nhuận bằng tổng trừ lát cắt.
••• Phân công với chi phí, $n \leq 200$	Bài kinh điển	Luồng chi phí nhỏ nhất hoặc Hungarian; so với quy hoạch động mặt nạ bit ở $n \leq 20$.
••• Đội bóng nào còn cơ hội vô địch	Bài kinh điển	Bài loại trừ thể thao; ví dụ đẹp nhất cho việc bài không có “dòng chảy” nào vẫn là bài luồng.
••• Phân đoạn ảnh nhị phân bằng lát cắt nhỏ nhất	Bài tập ứng dụng	Dựng chi phí gán nhãn và chi phí trơn; chạy trên ảnh nhỏ và xem kết quả bằng mắt.
••• Luồng có cận dưới trên mỗi cạnh	Bài thi đấu kinh điển	Phép biến đổi chuẩn; cài một lần cho kỹ rồi cắt vào thư viện riêng.

Kết chương và đọc tiếp

Luồng là chương mà công sức bỏ ra chủ yếu nằm ở việc *dựng mô hình*, không ở việc cài thuật toán. Ba điều đáng mang theo: cạnh ngược là ý tưởng làm cho thuật toán tự sửa được; định lý luồng–lát cắt cho ta cả lời giải lẫn chứng cứ tối ưu; và bốn mẫu mô hình ở mục 4, đặc biệt mẫu “gán nhãn hai phía với chi phí trơn” vì nó xuất hiện ở những nơi bất ngờ nhất.

Hai chương còn lại của phần C là hai công cụ cắt ngang, dùng được cho mọi chương trước. Chương C/19 nói về một kỹ thuật có tỉ lệ “biến bài khó thành bài dễ” cao nhất trong cả quyển sách: khi bài toán tối ưu khó, hãy đổi nó thành bài toán kiểm tra và tìm nhị phân trên đáp án.

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Edmonds & Karp, “Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems”	Journal of the ACM, 1972	Đọc để thấy một thuật toán không có đảm bảo trở nên có đảm bảo chỉ nhờ cố định thứ tự chọn đường tăng.
nền tảng	Hopcroft & Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs”	SIAM Journal on Computing, 1973	Thuật toán ghép đôi nên dùng trong thực hành; đọc phần lập luận về các pha đường tăng ngắn nhất.
tra khi cần	Orlin, “Max Flows in $O(nm)$ Time, or Better”	STOC, 2013	Điểm dừng của dòng thuật toán tổ hợp; đọc phần mở đầu để có bản đồ lịch sử các cận.
tra khi cần	Chen, Kyng, Liu, Peng, Probst Gutenberg, Sachdeva, “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time”	FOCS, 2022 (Best Paper)	Chỉ cần đọc mục giới thiệu: nó nói rõ vì sao hướng liên tục thắng và cấu trúc dữ liệu động đóng vai trò gì.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Ahuja, Magnanti, Orlin, <i>Network Flows</i>	Prentice-Hall, 1993	Sách tra cứu cho các biến thể mô hình hoá: luồng chi phí nhỏ nhất, luồng có cận dưới, bài toán vận tải.

Tìm kiếm trên đáp án (Binary Search on the Answer)

PHỤ THUỘC A/01 cho heuristic “đảo ngược câu hỏi”; A/04 cho bất biến của tìm kiếm nhị phân; B/06 cho các phép quét dùng trong hàm kiểm tra.

TÓM TẮT

Đây là chương có tỉ lệ “biến bài khó thành bài dễ” cao nhất trong quyển sách, và ý tưởng chỉ gồm một câu: thay vì hỏi “đáp án là bao nhiêu”, hãy hỏi “đáp án có đạt được mức x không” — câu thứ hai thường dễ hơn nhiều bậc. Nếu câu trả lời cho câu thứ hai có tính đơn điệu theo x thì tìm nhị phân trên x cho lời giải với chi phí bằng chi phí kiểm tra nhân log. Chương cũng nói về hai họ hàng: tìm kiếm ba phân cho hàm một cực trị, và các phương pháp tìm kiếm cục bộ khi không có cấu trúc đơn điệu nào. Nếu chỉ nhớ một điều: dấu hiệu nhận biết là đề bài có dạng “nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất” — gặp cụm từ đó thì gần như chắc chắn nên tìm nhị phân trên đáp án.

KIẾN THỨC CẦN CÓ TRƯỚC

- **vi tử đơn điệu** — hàm $P(x)$ nhận giá trị đúng/sai sao cho nếu $P(x)$ đúng thì $P(y)$ đúng với mọi $y > x$ (hoặc mọi $y < x$). Đây là điều kiện sống còn của cả chương.
- **tìm nhị phân trên đáp án** — nhị phân trên miền giá trị của kết quả, mỗi bước gọi một hàm kiểm tra “có đạt được mức này không”.
- **hàm kiểm tra (predicate)** — hàm trả lời câu hỏi quyết định; nó thường là một phép quét tham lam hoặc một quy hoạch động đơn giản.
- **hàm một cực trị (unimodal)** — hàm tăng rồi giảm (hoặc ngược lại), không có cực trị địa phương nào khác — điều kiện để dùng tìm kiếm ba phân.
- **tìm kiếm ba phân** — thu hẹp khoảng bằng cách so hai điểm bên trong; dùng cho hàm một cực trị, tương tự nhị phân nhưng cho bài tối ưu thay vì bài quyết định.
- **tìm kiếm cục bộ** — lớp phương pháp đi từ một lời giải và cải thiện dần bằng các thay đổi nhỏ; không có bảo đảm tối ưu nhưng dùng được khi không còn cấu trúc nào khác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao câu hỏi “đáp án có $\geq x$ không” lại thường dễ hơn hẳn câu hỏi “đáp án là bao nhiêu”?
- Q2. Kỹ thuật này đòi tính đơn điệu. Làm sao kiểm tra tính đơn điệu *trước khi* cài, và nó hỏng thế nào nếu bạn nhầm?
- Q3. Tìm nhị phân trên số thực khác gì trên số nguyên, và vì sao “lặp cho tới khi hai đầu bằng nhau” lại là một lỗi?
- Q4. Vì sao tìm kiếm ba phân cần hàm *một cực trị* chứ không chỉ cần “có cực đại”?
- Q5. Khi bài toán không có tính đơn điệu và cũng không có cấu trúc nào khác, còn lại lựa chọn gì?

Trong danh sách mười hai heuristic ở chương A/01 có một câu: “thay vì hỏi đáp án là bao nhiêu, hãy hỏi với giá trị x cho trước thì đáp án có $\geq x$ không”. Chương này khai thác trọn vẹn câu đó, và lý do nó đáng có một chương riêng là vì hiệu lực bất thường của nó: rất nhiều bài tối ưu trông không có lời giải nào lại có một hàm kiểm tra hiển nhiên chỉ dài năm dòng.

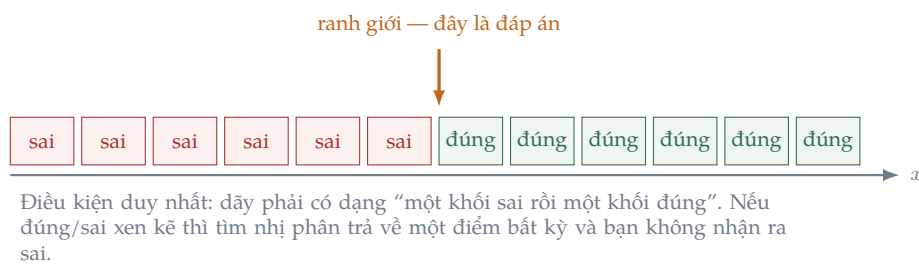
Cơ chế đằng sau khá rõ. Bài toán tối ưu buộc bạn phải *xây dựng* lời giải tốt nhất, tức là vừa phải quyết định cấu hình vừa phải bảo đảm không có cấu hình nào tốt hơn. Bài toán quyết định thì nhẹ hơn nhiều: bạn được cho sẵn ngưỡng, và chỉ cần trả lời có hoặc không — thường bằng một phép quét tham lam, vì khi ngưỡng đã cố định thì mọi lựa chọn trở nên rõ ràng. Đổi bài tối ưu thành bài quyết định là chuyển gánh nặng từ “tìm” sang “kiểm”, và kiểm thì rẻ hơn nhiều.

19.1 Khuôn chuẩn và điều kiện áp dụng

Khuôn gồm ba phần. *Một*: xác định miền giá trị của đáp án — cận dưới và cận trên chắc chắn. *Hai*: viết hàm $P(x)$ trả lời “có phương án nào đạt mức x không”. *Ba*: nhị phân trên x , giữ bất biến “ P sai với mọi giá trị nhỏ hơn lo , đúng với hi ” như ở chương A/04.

Điều kiện sống còn là *tính đơn điệu* của P : nếu đạt được mức x thì cũng đạt được mọi mức dễ hơn. Cách kiểm nhanh và đáng tin: phát biểu P thành lời và hỏi “nếu tôi làm được với ngân sách x , tôi có chắc làm được với ngân sách $x + 1$ không?”. Trong phần lớn bài đúng dạng, câu trả lời là hiển nhiên vì *chính phương án cũ vẫn hợp lệ*. Nếu bạn phải suy nghĩ lâu để trả lời thì rất có thể tính đơn điệu không có, và kỹ thuật này sẽ cho kết quả sai một cách im lặng.

Chi phí là $O(\log(\text{miền giá trị}) \times \text{chi phí kiểm tra})$. Thừa số log hầu như luôn nhỏ — với miền 10^{18} thì chỉ 60 vòng — nên bạn được phép có hàm kiểm tra khá đắt. Đây là điểm đáng nhớ: kỹ thuật này cho phép bạn “mua” một bậc log để đổi lấy một bài toán dễ hơn hẳn.



Hình 19.1. Tìm nhị phân trên đáp án không tìm trên dữ liệu mà tìm trên *miền giá trị của kết quả*. Việc bạn thật sự làm là tìm ranh giới giữa vùng “không đạt được” và vùng “đạt được”. Toàn bộ tính đúng nằm ở giả thiết rằng ranh giới đó tồn tại và chỉ có một.

19.2 Dấu hiệu nhận biết trên đề

Ba dấu hiệu, xếp theo độ tin cậy.

Dấu hiệu thứ nhất, gần như chắc chắn: đề có cụm “nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất”. Ví dụ: chia mảng thành k đoạn sao cho tổng lớn nhất của một đoạn là nhỏ nhất; đặt k trạm sao cho khoảng cách xa nhất từ một nhà tới trạm là nhỏ nhất. Dạng “cực tiểu của cực đại” hầu như luôn giải bằng kỹ thuật này, vì hàm kiểm tra “có chia được sao cho mọi đoạn $\leq x$ không” là một phép quét tham lam khi các phần tử không âm và số đoạn được giới hạn từ trên. Với số âm, phải thiết kế lại hàm kiểm tra; từ khoá minimax tự nó không chứng minh rằng kiểm tra là dễ.

Dấu hiệu thứ hai: đề hỏi giá trị tối ưu mà việc *kiểm tra* một giá trị cụ thể lại dễ. Ví dụ: tốc độ ăn nhỏ nhất để kịp thời hạn; kích thước lớn nhất của k miếng cắt bằng nhau.

Dấu hiệu thứ ba: đề hỏi “lớn nhất/nhỏ nhất sao cho một tính chất vẫn giữ được”, và tính chất đó rõ ràng dễ mất đi khi ta đòi hỏi nhiều hơn.

Có một biến thể quan trọng: *nhị phân trên tỉ số*. Với các bài tối ưu tỉ số kiểu “tìm tập con có trung bình lớn nhất”, hãy nhị phân trên giá trị tỉ số x và kiểm tra bằng cách trừ x khỏi mỗi phần tử rồi hỏi có tập con nào có tổng không âm không. Kỹ thuật này biến bài tỉ số — vốn không cộng dồn được — thành bài tổng thông thường, và nó giải cả họ bài “chu trình trung bình nhỏ nhất” hay “đoạn con có trung bình lớn nhất với độ dài tối thiểu”.

19.3 Ba cạm bẫy cài đặt

Cạm bẫy 1 — số thực và điều kiện dừng. Trên số thực, đừng lặp cho tới khi $lo = hi$ vì điều đó có thể không bao giờ xảy ra do sai số biểu diễn. Hai cách đúng: lặp một số vòng cố định — chọn k sao cho $(hi_0 - lo_0)/2^k \leq \varepsilon$ trong số học chính xác — hoặc dừng khi $hi - lo$ nhỏ hơn một ngưỡng đã chọn dựa trên yêu cầu của đề. Số vòng cần thiết phụ thuộc độ rộng ban đầu và sai số yêu cầu. Với dấu phẩy động, còn phải dừng nếu trung điểm bằng một đầu mút do làm tròn; tăng số vòng không vượt qua được độ phân giải của kiểu số.

Cạm bẫy 2 — biên và kiểu số. Miền giá trị phải chứa đáp án: cận dưới thường là 0 hoặc giá trị nhỏ nhất, cận trên phải là một giá trị chắc chắn đạt được. Với dữ liệu lớn, $(lo + hi)/2$ có thể tràn — hãy viết $lo + (hi - lo)/2$. Và kiểm tra kiểu: nếu đáp án tới 10^{18} thì cả biên lẫn giá trị trung gian trong hàm kiểm tra đều phải là 64 bit.

Cạm bẫy 3 — hàm kiểm tra sai một cách tinh vi. Đây là lỗi hay gặp nhất và khó thấy nhất, vì phần nhị phân thì đúng còn phần kiểm tra thì sai ở một trường hợp biên. Cách phòng hiệu quả nhất: viết hàm kiểm tra trước, test riêng nó với vài giá trị x cụ thể mà bạn tính được đáp án bằng tay, rồi mới ghép vào vòng nhị phân.

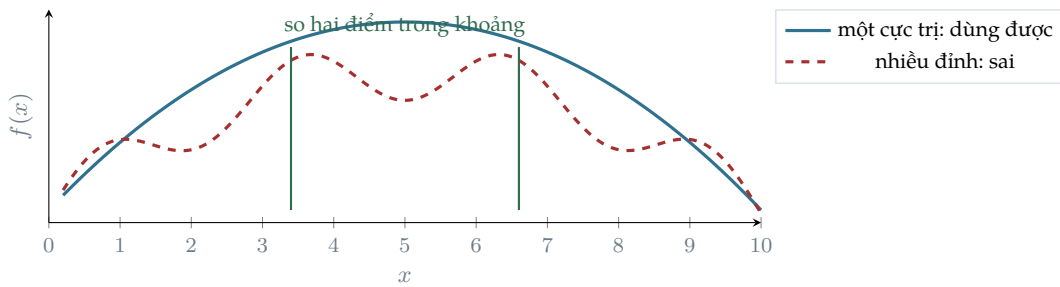
19.4 Tìm kiếm ba phân: khi hàm có một đỉnh

Nếu bài toán không cho một vị từ đơn điệu mà cho một *hàm số* tăng rồi giảm, ta không nhị phân được nhưng vẫn thu hẹp khoảng được. Ý tưởng: lấy hai điểm $m_1 < m_2$ bên trong khoảng; nếu $f(m_1) < f(m_2)$ thì cực đại không nằm bên trái m_1 , nên bỏ đoạn đầu; ngược lại bỏ đoạn cuối.

Điều kiện là hàm phải *một cực trị* — tăng rồi giảm, không có gợn nào khác. Đây là điều kiện mạnh hơn “có cực đại” rất nhiều, và đó là câu trả lời cho câu hỏi mở đầu thứ tư: nếu hàm có nhiều đỉnh địa phương thì phép so sánh hai điểm không còn nói được gì về vị trí cực đại toàn cục, và thuật toán trả về một đỉnh bất kỳ.

Trên miền số nguyên, tìm kiếm ba phân dễ sai ở những bước cuối khi khoảng còn vài phần tử; cách an toàn là thu hẹp tới khi còn dưới ba phần tử rồi thử hết. Trên số thực, dùng số vòng lặp cố định như ở mục trước.

Một biến thể đáng biết cho các bài tối ưu lồi: nếu hàm *lồi* thì có thể nhị phân trên *đạo hàm* rồi rạc $f(i+1) - f(i)$, vốn đơn điệu — và như vậy quay về bài toán nhị phân thông thường. Cách nhìn này nối chương với các tối ưu quy hoạch động ở C/16, nơi tính lồi cũng là điều kiện then chốt.



Hình 19.2. Tìm kiếm ba phân cần hàm *một cực trị*, không chỉ “có cực đại”. Với đường liền, việc so hai điểm bên trong cho biết cực đại nằm về phía nào. Với đường nét đứt nhiều gợn, cùng phép so chỉ nói lên xu hướng *cực bộ*, nên thuật toán hội tụ về một đỉnh bất kỳ.

19.5 Khi không có cấu trúc nào: tìm kiếm cực bộ

Nếu không có tính đơn điệu, không có tính lồi, và không có cấu trúc tổ hợp nào khai thác được, ta rơi vào lớp bài mà không thuật toán chính xác nào chạy nổi. Lúc đó còn một họ phương pháp: bắt đầu từ một lời giải bất kỳ và cải thiện dần bằng các thay đổi nhỏ.

Leo đồi luôn nhận thay đổi làm tốt lên; nó nhanh nhưng kẹt ở cực trị địa phương — đúng thất bại mà chương C/15 đã mô tả cho tham lam. *Mô phỏng luyện kim* thỉnh thoảng chấp nhận cả thay đổi làm tệ đi, với xác suất giảm dần theo thời gian; điều đó cho phép thoát khỏi cực trị địa phương ở giai đoạn đầu và ổn định ở giai đoạn sau. *Khởi động lại nhiều lần* thì đơn giản hơn: chạy leo đồi từ nhiều điểm xuất phát ngẫu nhiên và giữ kết quả tốt nhất.

Cần thẳng thắn về vị trí của nhóm này. Chúng không có bảo đảm chất lượng, kết quả phụ thuộc tham số và may rủi, và trong thi đấu thì hiếm khi là lời giải mong muốn. Nhưng trong công việc thật — lập lịch, bố trí, hiệu chỉnh tham số — chúng thường là thứ duy nhất chạy được trong thời gian cho phép. Nguyên tắc dùng: chỉ chuyển sang nhóm này sau khi đã chứng minh được rằng bài toán không có cấu trúc khai thác được, không phải vì lười tìm.

19.6 Một vết chạy đầy đủ: chia mảng để giảm tổng lớn nhất

Lấy $[7, 2, 5, 10, 8]$, chia thành tối đa hai đoạn không rỗng; mọi phần tử không âm. Miền đáp án là $[10, 32]$. Hàm $P(x)$ quét trái sang phải, chỉ mở đoạn mới khi thêm phần tử kế tiếp làm tổng vượt x . Quét như vậy dùng ít đoạn nhất: mỗi đoạn lấy tiền tố dài nhất còn hợp lệ; không âm bảo đảm phần đuôi không khó hơn khi ta lấy thêm phần tử vào đoạn trước.

Bảng 19.1. Tìm giá trị đúng đầu tiên, với $mid = lo + \lfloor (hi - lo)/2 \rfloor$. Đúng thì $hi = mid$, sai thì $lo = mid + 1$.

lo	hi	mid	Số đoạn	Chia tham lam tại mid
10	32	21	2: đúng	$[7, 2, 5] \mid [10, 8]$
10	21	15	3: sai	$[7, 2, 5] \mid [10] \mid [8]$
16	21	18	2: đúng	$[7, 2, 5] \mid [10, 8]$
16	18	17	3: sai	$[7, 2, 5] \mid [10] \mid [8]$

Bảng 19.1 kết thúc ở $lo = hi = 18$. Hai phép kiểm quan trọng nhất là $P(18)$ đúng và $P(17)$ sai: chúng kiểm ranh giới, không chỉ một điểm ở xa đáp án. Nếu đề yêu cầu đúng k đoạn, điều kiện “tối đa k ” tương đương khi $1 \leq k \leq n$ và các phần tử không âm: có thể tách thêm đoạn mà không tăng tổng lớn nhất.

19.7 Gốc rễ: từ giải phương trình tới ủ kim loại

Tìm kiếm nhị phân là ý tưởng cổ hơn máy tính rất nhiều: nó chính là *phương pháp chia đôi* của giải tích số để tìm nghiệm phương trình, và ý tưởng “giá trị đối dấu giữa hai đầu thì có nghiệm ở giữa” đã được dùng từ lâu trước thế kỷ 20. Nhưng phiên bản rời rạc trên mảng lại có một lịch sử đáng nhớ về sự khó cài: Knuth ghi nhận rằng thuật toán được công bố lần đầu vào năm 1946, nhưng phiên bản đầu tiên xử lý đúng *mọi* kích thước mảng chỉ xuất hiện vào năm 1962 — mười sáu năm cho một thuật toán mà ai cũng nghĩ là hiển nhiên. Cùng với quan sát của Bentley[6] rằng rất nhiều lập trình viên chuyên nghiệp viết sai nó khi phải viết từ đầu, đây là bằng chứng mạnh cho luận điểm ở chương A/04: sự đơn giản của ý tưởng không nói gì về độ khó của việc cài đúng.

Ý tưởng *tìm nhị phân trên đáp án* thì không có tác giả rõ ràng — nó là một kỹ thuật hiển nhiên với người đã quen nhìn bài toán tối ưu như bài toán quyết định. Nhưng cách nhìn đó lại có nguồn gốc lý thuyết rất rõ: trong lý thuyết độ phức tạp, mối quan hệ giữa bài toán tối ưu và bài toán quyết định là quan hệ nền tảng — người ta định nghĩa lớp NP qua bài toán quyết định chính vì lý do đó, và mọi bài tối ưu đều có một phiên bản quyết định đi kèm (D/23). Kỹ thuật của chương này là ứng dụng thực dụng của cùng ý tưởng: nếu bài quyết định dễ hơn thì hãy giải nó và tìm ranh giới.

Mô phỏng luyện kim có một gốc rễ liên ngành đẹp: năm 1983, Kirkpatrick và các cộng sự tại IBM đề xuất nó bằng cách mượn thẳng một hiện tượng vật lý. Khi làm nguội kim loại nóng chảy thật chậm, các nguyên tử có thời gian sắp xếp về cấu hình năng lượng thấp và tinh thể tạo thành đều đẹp; làm nguội nhanh thì kim loại đông cứng ở trạng thái năng lượng cao và có nhiều khuyết tật. Họ nhận ra rằng cùng cơ chế — cho phép hệ tạm thời đi lên mức năng lượng cao hơn, với xác suất giảm dần theo “nhiệt độ” — có thể dùng để thoát khỏi cực trị địa phương trong bài toán tối ưu tổ hợp. Đây là một trong những ví dụ rõ nhất về việc mượn cơ chế từ ngành khác, và nó cũng giải thích tên gọi cùng các tham số của phương pháp.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Trước thế kỷ 20 — phương pháp chia đôi 1946–1962 — tìm kiếm nhị phân rời rạc	Tìm nghiệm phương trình khi không có công thức đóng. Tra cứu trong bảng đã sắp.	Ý tưởng nhị phân trên miền liên tục; tổ tiên trực tiếp của cả chương này. Công bố 1946, nhưng bản đúng cho mọi kích thước mảng phải tới 1962 — bằng chứng rằng ý tưởng đơn giản không đồng nghĩa với cài đặt dễ.
Thập niên 1970 — lý thuyết độ phức tạp	Cần một cách chuẩn hoá để so độ khó của các bài toán.	Quan hệ giữa bài tối ưu và bài quyết định trở thành khái niệm nền; kỹ thuật của chương này là ứng dụng thực dụng của nó (D/23).
1983 — Kirkpatrick và cộng sự	Bài tối ưu tổ hợp lớn, leo đồi kẹt ở cực trị địa phương.	Mô phỏng luyện kim: mượn cơ chế làm nguội chậm trong vật lý để cho phép tạm thời đi ngược hướng cải thiện.

19.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Giải tích số	Phương pháp chia đôi, phương pháp Newton	Cùng ý tưởng thu hẹp khoảng chứa nghiệm. Khác: Newton hội tụ nhanh hơn nhiều nhưng cần đạo hàm và có thể phân kỳ; chia đôi chậm mà chắc — đúng đánh đổi giữa tốc độ và tính bền vững.
Gỡ lỗi phần mềm	git bisect: tìm commit đầu tiên làm hỏng	Đúng thuật toán của chương, áp lên trục thời gian của kho mã. Điều kiện áp dụng cũng giống hệt: tính đơn điệu — trước điểm đó thì tốt, sau điểm đó thì hỏng. Nếu lỗi xuất hiện rồi biến mất rồi lại xuất hiện thì công cụ trả về kết quả vô nghĩa.
Vật lý	Ủ kim loại, hạ nhiệt độ chậm để đạt trạng thái năng lượng thấp	Nguồn gốc trực tiếp của mô phỏng luyện kim, kể cả tên gọi và tham số “nhiệt độ”. Bài học chuyển được: cho phép tạm thời đi ngược hướng cải thiện là cách thoát khỏi cực trị địa phương.
Học máy	Dò tham số, dừng sớm, tìm tốc độ học	Kỹ thuật tìm tốc độ học bằng cách tăng dần và quan sát chính là tìm ranh giới trên một đại lượng gần đơn điệu. Cạm bẫy cũng giống: nếu quan hệ không đơn điệu thì kết luận sai.
Ước lượng robot, SLAM	Chọn ngưỡng loại ngoại lai, hiệu chỉnh tham số	Nhiều tham số ngưỡng có quan hệ đơn điệu với tỉ lệ chấp nhận, nên tìm nhị phân dùng được. Cạm bẫy quen thuộc: quan hệ đó chỉ đơn điệu <i>trung bình</i> , còn trên một tập dữ liệu cụ thể thì có nhiễu — và nhị phân trên hàm nhiễu cho kết quả không ổn định.
Kinh tế học, đấu giá	Tìm giá cân bằng bằng cách dò tăng dần	Cùng cấu trúc: cầu giảm khi giá tăng — một vị từ đơn điệu — nên giá cân bằng tìm được bằng chia đôi. Đây là cơ chế của nhiều thuật toán đấu giá thật.
Tối ưu hoá	Tìm kiếm cục bộ, thuật toán tiến hoá	Cùng họ với mục 5. Nhận xét đáng nhớ: các phương pháp này không có bảo đảm, nên chúng chỉ nên được dùng <i>sau khi</i> đã xác nhận bài toán không có cấu trúc khai thác được.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bài tối ưu đòi bạn vừa xây dựng cấu hình vừa bảo đảm không có cấu hình nào tốt hơn, còn bài quyết định cho sẵn ngưỡng nên mọi lựa chọn trở nên rõ ràng. Khi đã biết “mỗi đoạn không được vượt x ”, việc chia mảng thành ít đoạn nhất trở thành một phép quét tham lam hiển nhiên: cứ nhồi vào đoạn hiện tại tới khi vượt thì mở đoạn mới. Nói cách khác, ngưỡng cố định đã loại bỏ phần cần cân nhắc, và cái còn lại thường chỉ là một lượt duyệt.
- A2. Kiểm bằng cách phát biểu vị từ thành lời rồi hỏi “nếu làm được với ngân sách x thì có chắc làm được với $x + 1$ không”. Trong các bài đúng dạng, câu trả lời hiển nhiên vì chính phương án cũ vẫn hợp lệ. Nếu nhầm — vị từ không đơn điệu — thì tìm nhị phân vẫn chạy và vẫn trả về một số, nhưng số đó là một điểm chuyển bất kỳ chứ không phải đáp án; đây là lỗi im lặng, và chỉ stress test với vết cạn mới bắt được.
- A3. Trên số thực, phép chia đôi có thể không bao giờ làm hai đầu bằng nhau do sai số biểu diễn, nên vòng lặp treo. Hai cách đúng: lập một số vòng cố định (100 vòng là quá đủ, vì mỗi vòng giảm khoảng đi một nửa) hoặc dừng khi $hi - lo$ nhỏ hơn một ngưỡng chọn theo yêu cầu độ chính xác của đề. Cách lập cố định an toàn hơn vì độ chính xác tương đối không phụ thuộc độ lớn của giá trị.
- A4. Vì phép so sánh hai điểm bên trong chỉ cho thông tin về vị trí cực đại nếu hàm không có gợn nào khác. Với hàm nhiều đỉnh, việc $f(m_1) < f(m_2)$ không loại trừ được khả năng cực đại toàn cục nằm bên trái m_1 — nó chỉ nói rằng *quanh đó* hàm đang đi lên. Kết quả là thuật toán hội tụ về một đỉnh địa phương nào đó. “Có cực đại” là điều kiện quá yếu; phải là tăng rồi giảm, không có ngoại lệ.
- A5. Còn nhóm tìm kiếm cục bộ: leo đồi, mô phỏng luyện kim, khởi động lại nhiều lần, các thuật toán tiến hoá. Chúng không có bảo đảm chất lượng và kết quả phụ thuộc tham số, nên chỉ nên dùng sau khi đã xác nhận không còn cấu trúc nào khai thác được. Trong công việc thật — lập

lich, bố trí, hiệu chỉnh — chúng thường là thứ duy nhất chạy được trong thời gian cho phép, và lúc đó việc cần làm là đo phân phối kết quả thay vì tin vào một lần chạy.

NÓI LẠI THẬT ĐƠN GIẢN

Có một mẹo rất đơn giản mà lại giải được rất nhiều bài khó: thay vì hỏi “đáp án tốt nhất là bao nhiêu”, hãy hỏi “liệu có đạt được mức này không”.

Ví dụ: bạn cần chia một dãy số thành phần liên tiếp sao cho phần nặng nhất càng nhẹ càng tốt. Nghĩ trực tiếp thì rối. Nhưng nếu tôi hỏi “có chia được sao cho mọi phần đều không quá 100 không?” thì bạn trả lời được ngay: cứ nhồi số vào phần hiện tại tới khi sắp vượt 100 thì mở phần mới, cuối cùng đếm xem có quá năm phần không. Chỉ vài dòng.

Bây giờ để ý điều này: nếu chia được với ngưỡng 100 thì chắc chắn cũng chia được với ngưỡng 120 — ngưỡng rộng hơn thì chỉ dễ hơn. Nghĩa là câu trả lời “được / không được” xếp thành một dãy: không, không, không, rồi được, được, được. Đáp án chính là chỗ chuyển từ “không” sang “được”, và tìm chỗ chuyển đó thì đúng là trò đoán số: hỏi ở giữa, rồi thu hẹp một nửa.

Đó là toàn bộ kỹ thuật. Dấu hiệu để nhận ra nó trên đề bài rất dễ nhớ: khi thấy chữ “nhỏ nhất của cái lớn nhất” hoặc “lớn nhất của cái nhỏ nhất”, gần như chắc chắn là nó.

Có đúng một điều phải kiểm tra, và nếu bỏ qua thì chương trình vẫn chạy nhưng ra số sai: dãy trả lời phải thật sự có dạng “một khối không rồi một khối có”. Nếu “được” và “không được” xen kẽ nhau thì trò đoán số vô nghĩa, vì bạn sẽ dừng lại ở một chỗ chuyển ngẫu nhiên nào đó.

Một câu: Đổi câu hỏi “bao nhiêu” thành câu hỏi “có đạt được mức này không”, rồi đoán số trên mức đó.

Chỗ không rút gọn được: Việc chứng minh vị từ đơn điệu vẫn phải làm riêng cho từng bài; không có dấu hiệu bề ngoài nào bảo đảm điều đó.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng cốt lõi	Đổi bài tối ưu thành bài quyết định: “có đạt được mức x không”; ngưỡng cố định làm mọi lựa chọn trở nên rõ ràng, hàm kiểm tra thường chỉ là một lượt quét tham lam.
••• Điều kiện sống còn	Vị từ phải đơn điệu: đạt được x thì đạt được mọi mức dễ hơn. Kiểm bằng câu “phương án cũ có còn hợp lệ với ngân sách rộng hơn không”.
••• Dấu hiệu trên đề	“Nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất” — gần như chắc chắn là kỹ thuật này.
••• Chi phí	$O(\log(\text{miền giá trị}) \times \text{chi phí kiểm tra})$; log của 10^{18} chỉ là 60, nên hàm kiểm tra được phép khá đắt.
••• Ba cạm bẫy cài đặt	Số thực: lập số vòng cố định thay vì chờ $lo = hi$. Biên và tràn: dùng $lo + (hi - lo)/2$, kiểu 64 bit. Hàm kiểm tra sai tinh vi: test riêng nó trước khi ghép.
•• Nhị phân trên tỉ số	Bài “trung bình lớn nhất”: nhị phân trên x , trừ x khỏi mỗi phần tử, hỏi có tổng không âm không — biến bài tỉ số thành bài tổng.
•• Tìm kiếm ba phần	Cho hàm một cực trị; so hai điểm trong khoảng để loại một đầu. Nhiều đỉnh địa phương thì sai.
•• Hàm lồi thì nhị phân được	Nhị phân trên đạo hàm rời rạc $f(i+1) - f(i)$, vốn đơn điệu — quay về bài nhị phân thường.
•• Tìm kiếm cục bộ	Leo đồi (kết cực trị địa phương); mô phỏng luyện kim (chấp nhận bước xấu với xác suất giảm dần); khởi động lại nhiều lần.
•• Khi nào dùng tìm kiếm cục bộ	Chỉ sau khi xác nhận bài toán không có cấu trúc khai thác được; không có bảo đảm chất lượng nên phải đo phân phối kết quả.
• Liên hệ với git bisect	Cùng thuật toán trên trục thời gian của kho mã; cùng yêu cầu đơn điệu — lỗi phải xuất hiện từ một điểm và không biến mất.

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Nhị phân trên đáp án và nhị phân trên mảng khác nhau thế nào?

| Khác ở *cái được tìm trên*. Nhị phân trên mảng tìm một phần tử trong dữ liệu đã sắp; nhị phân trên đáp án tìm một giá trị trong *miền kết quả*, và dữ liệu có thể hoàn toàn không sắp. Điểm chung là cả hai đều cần một vị từ đơn điệu, và cả hai đều dùng cùng khuôn bất biến ở A/04. Nhận ra chúng là một giúp bạn viết cả hai bằng cùng một mẫu code và ít sai hơn.

2. Tôi nên nhị phân trên số nguyên hay số thực khi đáp án là số thực?

| Nếu đề cho phép sai số, hãy xem có thể nhân lên thành số nguyên không — ví dụ đáp án luôn là bội của 0,001 thì nhân 1000 và làm việc trên số nguyên, tránh hẳn mọi vấn đề sai số. Nếu không, dùng số thực với số vòng lặp cố định. Một mẹo phụ: nhiều bài trông có đáp án thực nhưng đáp án thật sự luôn nằm trong một tập hữu hạn giá trị (một hiệu, một tỉ số của các số nguyên trong dữ liệu) — lúc đó nhị phân trên tập đó cho kết quả chính xác tuyệt đối.

3. Hàm kiểm tra của tôi tốn $O(n \log n)$. Tổng $O(n \log n \log C)$ có quá chậm không?

| Thường là không, và đây là lý do kỹ thuật này mạnh. Với $n = 10^5$ và $C = 10^9$: $10^5 \times 17 \times 30 \approx 5 \cdot 10^7$ — nằm trong ngân sách. Điều cần kiểm là hằng số của hàm kiểm tra: nếu nó cấp phát bộ nhớ hoặc sắp xếp lại từ đầu mỗi lần thì hãy chuyển việc sắp xếp ra *ngoài* vòng nhị phân, vì dữ liệu không đổi giữa các lần kiểm.

4. Làm sao nhận ra tính đơn điệu khi nó không hiển nhiên?

| Ba cách. Một, viết vị từ ra thành lời và tìm một lập luận kiểu “phương án hợp lệ cho x vẫn hợp lệ cho $x + 1$ ”. Hai, tính tay giá trị vị từ cho vài x liên tiếp trên một ví dụ nhỏ và nhìn dãy kết quả. Ba, viết bộ sinh test nhỏ, tính vị từ cho mọi x trong miền, và kiểm bằng máy rằng dãy có dạng “khởi sai rồi khỏi đúng”. Cách ba tốn năm phút và cho câu trả lời chắc chắn — đáng làm khi nghi ngờ.

5. Bài của tôi có hai đại lượng cần tối ưu cùng lúc. Kỹ thuật này còn dùng được không?

| Có, theo hai cách. Nếu hai đại lượng có thứ tự ưu tiên — tối ưu cái thứ nhất trước, cái thứ hai chỉ để phá hoà — thì nhị phân trên cái thứ nhất, rồi trong hàm kiểm tra hãy trả về giá trị tốt nhất của cái thứ hai. Nếu bài yêu cầu tối ưu một tổ hợp tuyến tính có tham số, đó là bài toán tìm kiếm tham số và thường giải bằng nhị phân trên tham số cộng một tính chất lồi — cùng ý tưởng với nhị phân trên tỉ số ở mục 2.

6. Mô phỏng luyện kim có đáng học cho thi đấu không?

| Đáng biết là có, nhưng hiếm khi cần. Nó xuất hiện trong một số kỳ thi cho phép lời giải xấp xỉ và chấm theo điểm chất lượng, và trong các bài tối ưu hình học nhỏ. Trong các kỳ thi chấm đúng/sai thì hầu như không dùng. Ngược lại, trong công việc thật thì nó rất hữu ích — lập lịch, bố trí, hiệu chỉnh tham số — nên nó thuộc loại “học một lần cho biết cách dùng, không cần luyện nhiều”.

7. Có bài nào nhìn giống nhị phân trên đáp án mà thực ra không phải không?

| Có, và dấu hiệu là vị từ không đơn điệu. Ví dụ: “tìm giá trị x sao cho số cách đạt đúng x là lớn nhất” — vị từ ở đây không phải dạng “đạt được hay không” mà là một hàm đếm, và nó có thể lên xuống tùy ý. Một ví dụ khác: các bài mà tăng ngân sách lại làm phương án cũ *không còn hợp lệ* vì có ràng buộc kiểu “phải dùng hết”. Khi thấy chữ “đúng bằng” hoặc “dùng hết” trong đề, hãy dừng lại kiểm tính đơn điệu cẩn thận.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tìm căn bậc hai nguyên của một số	Bài kinh điển	Nhị phân trên đáp án; chú ý tràn khi bình phương.
● Tốc độ nhỏ nhất để hoàn thành trong h giờ	LeetCode “Koko Eating Bananas”	Bài mẫu nhập môn; viết vị từ ra thành lời trước.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Chia mảng thành k đoạn sao cho tổng lớn nhất nhỏ nhất	CSES “Array Division”[33]	Dạng “cực tiểu của cực đại” kinh điển; hàm kiểm tra là một lượt tham lam.
•• Đặt k trạm sao cho khoảng cách xa nhất nhỏ nhất	Bài kinh điển	Cùng dạng; chú ý biên của miền giá trị.
•• Số nhỏ nhất x sao cho có ít nhất k bội số của các số cho trước không vượt x	CSES “Factory Machines”[33]	Nhị phân trên thời gian; cẩn thận tràn khi cộng số lượng.
•• Chuỗi con lặp lại dài nhất	Bài kinh điển	Nhị phân trên độ dài cộng bấm chuỗi (B/12); tính đơn điệu ở đây rất rõ.
••• Đoạn con có trung bình lớn nhất với độ dài tối thiểu k	Bài kinh điển	Nhị phân trên tỉ số: trừ x khỏi mỗi phần tử rồi hỏi có đoạn tổng không âm không.
••• Chu trình có trung bình trọng số nhỏ nhất	Bài kinh điển	Cùng mẹo tỉ số, kết hợp phát hiện chu trình âm bằng Bellman–Ford (C/17).
••• Tìm điểm cực tiểu của một hàm một cực trị trên số thực	Bài kinh điển	Tìm kiếm ba phân với số vòng cố định; so với cách nhị phân trên đạo hàm rời rạc.
••• Bài toán tối ưu nhỏ giải bằng mô phỏng luyện kim, so với vét cạn	Bài tập thực nghiệm	Chạy 100 lần và vẽ phân phối kết quả — mục tiêu là thấy tận mắt “không có bảo đảm” nghĩa là gì.
••• Kiểm tra tính đơn điệu của một vị từ bằng máy trên dữ liệu nhỏ	Bài tập kiểm thử	Viết vòng lặp tính vị từ cho mọi x và khẳng định đây có dạng “sai rồi đúng”; công cụ này nên có sẵn trong thư viện của bạn.

Kết chương và đọc tiếp

Kỹ thuật của chương này rẻ tới mức nên là một trong những thứ đầu tiên bạn thử khi gặp bài tối ưu: đọc đề, tìm cụm “nhỏ nhất của lớn nhất”, và hỏi “nếu tôi cố định ngưỡng thì việc kiểm tra có dễ không”. Hai điều đáng mang theo: tính đơn điệu là điều kiện duy nhất nhưng bắt buộc, và nó phải được kiểm chứ không được giả định; và log là cái giá rất rẻ để đổi lấy một bài toán dễ hơn hẳn.

Chương cuối của phần C là công cụ cắt ngang thứ hai, và nó đã xuất hiện thấp thoáng ở bốn chương trước: bấm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui. Chương [C/20](#) phát biểu nguyên lý chung đằng sau cả bốn — *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc sang một nguồn ngẫu nhiên mà bạn kiểm soát* — và chỉ ra rằng ngẫu nhiên hoá không phải sự cầu may mà là một công cụ thiết kế có bảo đảm.

Ngẫu nhiên hoá (Randomized Algorithms)

PHỤ THUỘC A/05 cho kỳ vọng tuyến tính và các bất đẳng thức đuôi; B/08 và B/09 cho hai ứng dụng đã gặp; C/14 cho quicksort ngẫu nhiên.

TÓM TẮT

Ngẫu nhiên hoá đã xuất hiện bốn lần trong quyển sách này — băm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui — và chương này phát biểu nguyên lý chung đằng sau cả bốn: *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc sang một nguồn ngẫu nhiên mà bạn kiểm soát*. Đây không phải sự cầu may: kết quả là các đảm bảo đúng cho mọi dữ liệu vào, kể cả dữ liệu do đối thủ chọn. Chương phân biệt hai loại thuật toán ngẫu nhiên, đi qua các đại diện — kiểm tra nguyên tố, kiểm tra tích ma trận, lát cắt nhỏ nhất của Karger, lấy mẫu dòng dữ liệu — và nói về cách khuếch đại xác suất đúng bằng cách lặp lại. Nếu chỉ nhớ một điều: ngẫu nhiên hoá đôi “luôn đúng, đôi khi rất chậm” lấy “gần như luôn nhanh, và bạn kiểm soát được xác suất sai”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thuật toán Las Vegas** — luôn cho kết quả đúng; thời gian chạy là biến ngẫu nhiên. Quicksort ngẫu nhiên và treap thuộc loại này.
- **thuật toán Monte Carlo** — luôn chạy trong thời gian định trước; kết quả có thể sai với xác suất nhỏ. Kiểm tra nguyên tố Miller–Rabin thuộc loại này.
- **khuếch đại xác suất** — lặp lại một thuật toán Monte Carlo k lần độc lập; nếu mỗi lần sai với xác suất p thì k lần cùng sai chỉ với xác suất p^k — giảm theo hàm mũ.
- **lấy mẫu hồ chứa** — kỹ thuật chọn k phần tử đều ngẫu nhiên từ một dòng dữ liệu chưa biết độ dài, chỉ dùng $O(k)$ bộ nhớ.
- **khử ngẫu nhiên** — quá trình biến một thuật toán ngẫu nhiên thành tất định, thường bằng cách thay nguồn ngẫu nhiên thật bằng một họ hàm nhỏ.
- **đối thủ thích ứng** — mô hình trong đó dữ liệu được chọn sau khi biết thuật toán — lý do mọi phân tích “trung bình theo dữ liệu” sụp đổ trong thực tế.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao ngẫu nhiên hoá lại cho đảm bảo *mạnh hơn* phân tích trung bình, dù cả hai đều nói về xác suất?
- Q2. Một thuật toán trả lời sai với xác suất một phần tư nghe rất tệ. Vì sao nó vẫn dùng được trong thực tế?
- Q3. Kiểm tra xem $A \times B = C$ có đúng không mà không nhân ma trận — làm sao có thể nhanh hơn phép nhân?
- Q4. Trong lập trình thi đấu, vì sao dùng hàm sinh số ngẫu nhiên cố định hạt giống lại nguy hiểm?
- Q5. Thuật toán lát cắt nhỏ nhất của Karger có ngẫu nhiên các cạnh và thường cho kết quả sai. Vì sao nó vẫn là một thuật toán tốt?

Ngẫu nhiên hoá là chủ đề dễ bị hiểu sai nhất trong quyển sách này, vì cái tên gợi ý sự may rủi. Thực tế thì ngược lại: nó là công cụ để *loại bỏ* may rủi.

Hãy nhớ lại tình huống lặp đi lặp lại ở phần B. Bảng băm nhanh “trung bình”, nhưng trung bình đó giả định dữ liệu ngẫu nhiên — và dữ liệu thật thì không ngẫu nhiên, thậm chí có thể do người khác chọn để hại bạn. Cây tìm kiếm nhị phân cân bằng “trung bình”, nhưng dữ liệu đã sắp sẵn là chuyện thường ngày. Quicksort nhanh “trung bình”, với cùng lỗ hổng. Trong cả ba trường hợp, phân tích trung bình là một lời hứa dựa trên giả định về dữ liệu, và giả định đó không do bạn kiểm soát.

Ngẫu nhiên hoá sửa đúng chỗ đó: thay vì hy vọng dữ liệu ngẫu nhiên, hãy *tự tạo ra ngẫu nhiên* ở chỗ bạn kiểm soát — chọn hàm băm, chọn độ ưu tiên, chọn chốt. Khi đó hành vi của thuật toán không còn phụ thuộc thứ tự dữ liệu, và đảm bảo trở thành đúng cho mọi dữ liệu vào. Đây là khác biệt then chốt so với phân tích trung bình, và nó là câu trả lời cho câu hỏi mở đầu thứ nhất.

20.1 Hai loại và cách chọn giữa chúng

Phân biệt dưới đây theo khuôn Las Vegas / Monte Carlo của Motwani và Raghavan[40].

Las Vegas luôn đúng, thời gian chạy ngẫu nhiên. Bạn không bao giờ phải lo về tính đúng; bạn chỉ phải chấp nhận rằng thỉnh thoảng nó chạy lâu hơn. Quicksort với chốt ngẫu nhiên, treap, và thuật toán chọn phần tử thứ k đều thuộc loại này.

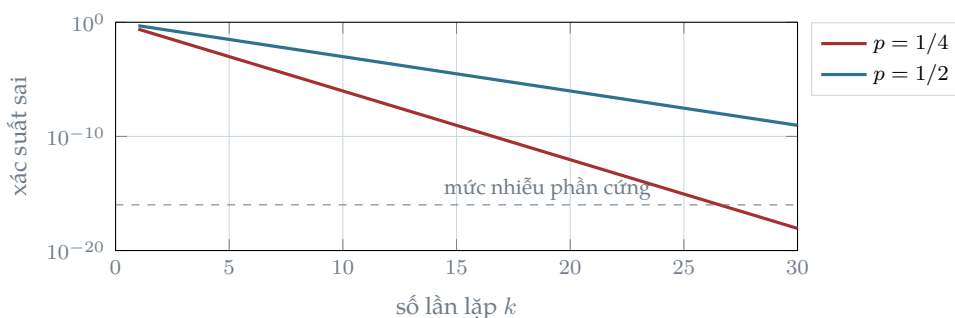
Monte Carlo luôn nhanh, kết quả có thể sai. Bạn kiểm soát được xác suất sai và làm nó nhỏ tùy ý bằng cách lặp lại. Kiểm tra nguyên tố, kiểm tra tích ma trận, băm chuỗi đều thuộc loại này.

Có một quy tắc chuyển đổi hữu ích: nếu kết quả *kiểm chứng được nhanh*, thì mọi thuật toán Monte Carlo biến thành Las Vegas bằng cách lặp lại tới khi kiểm chứng thành công. Ví dụ: một thuật toán ngẫu nhiên đề xuất một cách ghép cặp; kiểm tra cách ghép có hợp lệ hay không thì rẻ, nên cứ thử lại tới khi được. Ngược lại, khi kết quả không kiểm chứng được — “số này có phải nguyên tố không” — thì bạn buộc phải sống với xác suất sai.

20.2 Khuếch đại: vì sao “sai một phần tư” vẫn dùng được

Đây là kỹ thuật làm cho toàn bộ hướng tiếp cận trở nên khả thi, và nó dựa trên một tính toán rất đơn giản.

Giả sử thuật toán trả lời sai với xác suất tối đa $1/4$, và các lần chạy độc lập. Chạy k lần và lấy kết quả xuất hiện nhiều nhất; xác suất tất cả k lần cùng sai là $(1/4)^k$. Với $k = 30$, con số đó nhỏ hơn 10^{-18} — tức là nhỏ hơn xác suất phần cứng của bạn bị lỗi bit trong lúc chạy.



Hình 20.1. Khuếch đại xác suất. Xác suất sai giảm theo *hàm mũ* của số lần lặp, nên vài chục lần là đủ để đưa nó xuống dưới mức mà các nguồn sai khác — kể cả lỗi phần cứng — đã chi phối. Điều kiện bắt buộc: các lần chạy phải độc lập.

Điều đáng nhấn mạnh về mặt kỹ thuật: đây là dạng đơn giản nhất của các bất đẳng thức đuôi ở chương A/05, và nó cho thấy vì sao Chernoff quan trọng — xác suất sai giảm theo *hàm mũ* của số lần lặp, chứ không theo tỉ lệ nghịch. Chính điều đó cho phép ta trả một chi phí nhỏ (vài chục lần lặp) để đổi lấy độ tin cậy thực tế tuyệt đối.

Một điều kiện dễ bị quên: các lần chạy phải *độc lập*. Nếu bạn lặp lại thuật toán với cùng hạt giống, hoặc với các lựa chọn tương quan nhau, thì phép nhân xác suất không còn đúng và bạn chỉ đang lãng phí thời gian.

CẦN NHỚ

Trong lập trình thi đấu, luôn khởi tạo bộ sinh ngẫu nhiên bằng một nguồn không đoán trước — thời gian đồng hồ hệ thống ở độ phân giải cao, hoặc một giá trị phụ thuộc địa chỉ bộ nhớ. Dùng hạt giống cố định biến thuật toán ngẫu nhiên của bạn thành tất định, và khi đó người ra đề (hoặc người viết test phá) có thể dựng dữ liệu chạm đúng trường hợp xấu — đúng lỗ hổng mà ngẫu nhiên hoá lẽ ra đã bịt.

20.3 Bốn đại diện đáng biết

Kiểm tra nguyên tố. Muốn biết n có phải số nguyên tố, thử chia tới $\sqrt{n}$ là $O(\sqrt{n})$ — quá chậm với số 60 bit. Kiểm tra Miller–Rabin chọn ngẫu nhiên vài “nhân chứng” và dùng một tính chất số học để phát hiện hợp số; mỗi nhân chứng phát hiện hợp số với xác suất ít nhất $3/4$, nên vài chục nhân chứng cho độ tin cậy tuyệt đối trong thực tế. Đây là thuật toán chạy trong mọi thư viện mật mã, và là ví dụ chuẩn cho việc ngẫu nhiên hoá không phải giải pháp tạm mà là công cụ sản xuất.

Kiểm tra tích ma trận (Freivalds). Cho ba ma trận A, B, C cỡ $n \times n$, hỏi $AB = C$ có đúng không. Tính AB tốn $O(n^3)$. Mẹo: chọn một vector ngẫu nhiên r gồm các bit, và so $A(Br)$ với Cr — cả hai về chỉ cần các phép nhân ma trận–vector, tốn $O(n^2)$. Nếu $AB \neq C$ thì phép so sai với xác suất tối đa $1/2$, và lặp vài chục lần là đủ. Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó minh hoạ một ý sâu hơn: *kiểm tra một đẳng thức rẻ hơn tính ra nó*, vì ta chỉ cần kiểm nó trên một hướng ngẫu nhiên thay vì trên mọi hướng.

Lát cắt nhỏ nhất của Karger. Lặp lại việc co một cạnh ngẫu nhiên cho tới khi còn hai đỉnh; các cạnh còn lại là một lát cắt. Xác suất ra đúng lát cắt nhỏ nhất chỉ khoảng $2/n^2$ — rất thấp — nhưng vì đó là xác suất chứ không phải sự bế tắc, ta chỉ cần lặp $O(n^2 \log n)$ lần để có độ tin cậy cao. Điều làm thuật toán này nổi tiếng không phải hiệu năng mà là sự đơn giản: nó không dùng luồng, không dùng lý thuyết nào ngoài việc co cạnh, và nó chứng minh rằng một cách tiếp cận “ngây thơ có ngẫu nhiên” vẫn có thể cho kết quả sâu sắc.

Lấy mẫu hồ chứa. Chọn đều ngẫu nhiên k phần tử từ một dòng dữ liệu mà bạn không biết trước độ dài và không lưu được. Giữ k phần tử đầu; với phần tử thứ $i > k$, giữ nó với xác suất k/i và nếu giữ thì thay thế một phần tử ngẫu nhiên trong hồ. Chứng minh tính đúng bằng quy nạp là một bài tập đẹp, và kỹ thuật này là công cụ chuẩn trong xử lý dòng dữ liệu (D/24).

20.4 Ngẫu nhiên hoá như một mẫu thiết kế

Nhìn lại toàn bộ quyển sách, ngẫu nhiên hoá xuất hiện dưới bốn vai trò khác nhau. Nhận ra bốn vai trò này giúp bạn biết khi nào nên nghĩ tới nó.

Vai trò 1 — phá vỡ trường hợp xấu do dữ liệu. Băm với hạt giống ngẫu nhiên (B/08), treap (B/09), chốt quicksort (C/14). Dấu hiệu: thuật toán của bạn nhanh trừ khi dữ liệu có cấu trúc đặc biệt.

Vai trò 2 — kiểm tra rẻ hơn tính toán. Freivalds, băm chuỗi (B/12), so sánh hai đa thức bằng cách thay một giá trị ngẫu nhiên. Dấu hiệu: bạn cần kiểm một đẳng thức chứ không cần biết giá trị.

Vai trò 3 — lấy mẫu thay cho vét cạn. Ước lượng kích thước cây quay lui (C/13), lấy mẫu hồ chứa, các phương pháp Monte Carlo cho tích phân. Dấu hiệu: không gian quá lớn để duyệt nhưng bạn chỉ cần một ước lượng.

Vai trò 4 — thoát khỏi cấu trúc xấu. Karger, khởi động lại nhiều lần trong tìm kiếm cục bộ (C/19), RANSAC trong thị giác máy tính. Dấu hiệu: có nhiều lời giải khả dĩ và bạn chỉ cần tìm được một cái tốt, chứ không cần chứng minh nó tốt nhất.

CẠM BẮY

Ngẫu nhiên hoá không sửa được một thuật toán *sai*. Nếu ý tưởng của bạn có phần ví dụ về mặt logic thì việc thêm ngẫu nhiên chỉ làm nó sai một cách khó dự đoán hơn. Ngẫu nhiên hoá chỉ giải quyết vấn đề *phân phối dữ liệu*, không giải quyết vấn đề tính đúng. Trước khi thêm ngẫu nhiên, hãy chắc chắn bạn đang ở đúng loại vấn đề.

20.5 Gốc rễ: một ván bài trong bệnh viện và bài toán số nguyên tố

Phương pháp Monte Carlo có một nguồn gốc được kể lại khá thống nhất. Năm 1946, Stanislaw Ulam đang nằm dưỡng bệnh và chơi bài một mình. Ông tự hỏi xác suất để một ván bài kiểu solitaire thắng là bao nhiêu, và nhận ra rằng việc tính chính xác bằng tổ hợp là gần như bất khả thi — trong khi việc *chơi thử một trăm ván và đếm* thì rất dễ. Ý tưởng đó, mang sang cho các bài toán khuếch tán neutron mà ông và von Neumann đang làm tại Los Alamos, trở thành phương pháp Monte Carlo. Cái tên được đặt theo sông bạc ở Monaco, nơi một người họ hàng của Ulam hay lui tới.

Bài học phương pháp ở đây rất đáng nhớ và nó áp dụng cho cả việc giải thuật toán: khi một đại lượng khó tính chính xác nhưng dễ *lấy mẫu*, hãy đổi bài toán tính sang bài toán ước lượng. Đây chính là vai trò thứ ba ở mục 5.

Nhánh thuật toán ngẫu nhiên trong khoa học máy tính bắt đầu nghiêm túc vào giữa thập niên 1970 với bài toán kiểm tra nguyên tố. Solovay và Strassen, rồi Rabin, đưa ra các thuật toán xác suất nhanh cho một bài toán mà khi đó chưa ai biết có lời giải đa thức tất định hay không. Điều này đặt ra một câu hỏi lý thuyết quan trọng: ngẫu nhiên có thực sự cho thêm sức mạnh tính toán, hay mọi thuật toán ngẫu nhiên đều khử ngẫu nhiên được? Câu chuyện có một kết cục thú vị: năm 2002, ba nhà nghiên cứu Ấn Độ công bố một thuật toán *tất định* thời gian đa thức cho kiểm tra nguyên tố, giải quyết đúng bài toán mở đầu của cả nhánh — dù trong thực tế thì các thuật toán ngẫu nhiên vẫn nhanh hơn nhiều và vẫn là thứ được dùng.

Câu hỏi tổng quát vẫn còn mở nhưng nghiêng về một phía: phần lớn giới nghiên cứu tin rằng ngẫu nhiên *không* cho thêm sức mạnh về mặt lớp độ phức tạp, tức là mọi thuật toán ngẫu nhiên đa thức đều có phiên bản tất định đa thức. Điều đó không làm ngẫu nhiên hoá mất giá trị — các phiên bản tất định thường phức tạp và chậm hơn nhiều bậc trong thực tế — nhưng nó nói rằng sức mạnh thật sự của ngẫu nhiên nằm ở *sự đơn giản*, không ở khả năng tính toán.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1946 — Ulam, von Neumann	Tính xác suất thắng của một ván bài; rồi bài toán khuếch tán neutron ở Los Alamos.	Phương pháp Monte Carlo: khi khó tính chính xác nhưng dễ lấy mẫu, hãy đổi sang ước lượng.
1975–76 — Solovay, Strassen, Rabin	Kiểm tra số lớn có nguyên tố không — chưa ai biết lời giải đa thức tất định.	Kiểm tra nguyên tố xác suất; khai sinh nhánh thuật toán ngẫu nhiên trong khoa học máy tính.
1977 — Freivalds	Kiểm tra $AB = C$ mà không muốn tốn $O(n^3)$.	Kiểm bằng một vector ngẫu nhiên trong $O(n^2)$; ý tưởng “kiểm rẻ hơn tính”.
1979 — Carter & Wegman	Bảng băm nhanh trung bình nhưng không có đảm bảo với dữ liệu thù địch.	Băm phổ dụng: ngẫu nhiên nằm ở lựa chọn hàm, không ở dữ liệu (B/08).
1993 — Karger	Tìm lát cắt nhỏ nhất mà không dùng luồng.	Co cạnh ngẫu nhiên; xác suất thành công thấp nhưng lặp lại được — minh hoạ rằng đơn giản cộng ngẫu nhiên vẫn cho kết quả sâu.
2002 — Agrawal, Kayal, Saxena	Kiểm tra nguyên tố có lời giải đa thức tất định không?	Có — nhưng chậm hơn bản ngẫu nhiên nhiều bậc trong thực tế. Sức mạnh của ngẫu nhiên nằm ở sự đơn giản, không ở lớp độ phức tạp.

20.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính, SLAM	RANSAC: lấy mẫu ngẫu nhiên tập con nhỏ để ước lượng mô hình	Vai trò 4 ở mục 5, và là ứng dụng ngẫu nhiên hoá quan trọng nhất trong ngành này. Cùng lập luận khuếch đại: xác suất một lần lấy mẫu “sạch” thì thấp, nhưng lặp đủ nhiều thì gần như chắc chắn có ít nhất một lần sạch.
Ước lượng trạng thái	Bộ lọc hạt: biểu diễn phân phối bằng một tập mẫu	Monte Carlo áp cho bài toán ước lượng liên tục. Cùng tinh thần với mục 4: khi phân phối khó mô tả bằng công thức, hãy mô tả nó bằng mẫu. Cạm bẫy đặc thù là suy thoái mẫu, cần lấy mẫu lại.
Thống kê	Bootstrap, MCMC, kiểm định hoán vị	Cùng ý tưởng gốc của Ulam: thay tính toán giải tích bằng mô phỏng lặp lại. Khác: ở đó mục tiêu là ước lượng độ bất định, ở đây là ước lượng một đại lượng.
Mật mã	Sinh khoá, kiểm tra nguyên tố, giao thức xác suất	Kiểm tra nguyên tố ngẫu nhiên chạy mỗi lần bạn tạo một khoá. Điểm khác biệt quan trọng: ở đó chất lượng nguồn ngẫu nhiên là vấn đề an ninh, không chỉ là vấn đề hiệu năng.
Học máy	Khởi tạo ngẫu nhiên, bỏ ngẫu nhiên nơ-ron, trộn dữ liệu, rừng ngẫu nhiên	Ngẫu nhiên ở đây phục vụ tính tổng quát hoá chứ không phục vụ tốc độ: nó phá vỡ sự tương quan và ngăn mô hình bám vào cấu trúc giả. Cùng nguyên lý phá vỡ cấu trúc xấu, khác mục tiêu.
Xử lý dòng dữ liệu	Lấy mẫu hồ chứa, sơ đồ đếm xấp xỉ, HyperLogLog	Ngẫu nhiên là điều kiện để làm việc với bộ nhớ nhỏ hơn dữ liệu nhiều bậc (D/24). Cùng đánh đổi: chấp nhận sai số nhỏ để đổi lấy khả năng chạy được.
Trò chơi và AI	Tìm kiếm cây Monte Carlo	Thay vì duyệt cây trò chơi đầy đủ, lấy mẫu các ván chơi tới cuối và thống kê. Cùng vai trò 3; đây là kỹ thuật đứng sau bước nhảy vọt của các chương trình chơi cờ vây.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai loại phân tích nói về hai nguồn ngẫu nhiên khác nhau. Phân tích trung bình giả định dữ liệu được rút từ một phân phối nào đó — một giả định về thể giới, và nó sai khi dữ liệu có cấu trúc hoặc do đối thủ chọn. Ngẫu nhiên hoá đặt nguồn ngẫu nhiên vào *lựa chọn của thuật toán*, thứ mà bạn kiểm soát và đối thủ không biết. Kết quả là một đảm bảo đúng cho mọi dữ liệu vào, không kèm giả định nào về phân phối dữ liệu.
- A2. Vì xác suất sai giảm theo hàm mũ khi lặp lại. Chạy k lần độc lập, xác suất tất cả cùng sai là $(1/4)^k$; với $k = 30$ thì nhỏ hơn 10^{-18} , tức là nhỏ hơn xác suất máy tính bị lỗi phần cứng trong lúc chạy. Điều kiện là các lần chạy phải thật sự độc lập — dùng lại cùng hạt giống hoặc các lựa chọn tương quan sẽ phá vỡ phép nhân xác suất và bạn chỉ đang lặp lại cùng một sai lầm.
- A3. Vì để kiểm một đẳng thức, ta không cần kiểm nó theo mọi hướng — chỉ cần theo một hướng ngẫu nhiên là đủ để phát hiện sai lệch với xác suất cao. Cụ thể, thay vì tính AB rồi so với C , ta chọn vector ngẫu nhiên r và so $A(Br)$ với Cr ; cả hai vế chỉ cần nhân ma trận với vector, tốn $O(n^2)$. Nếu hai ma trận khác nhau thì chúng khác nhau trên phần lớn các vector, nên xác suất bỏ sót là nhỏ và giảm theo hàm mũ khi lặp lại.
- A4. Vì hạt giống cố định làm thuật toán trở thành tất định, và mọi thứ tất định đều dựng được test phá. Người viết test chỉ cần chạy đúng chương trình của bạn để biết nó sẽ chọn chốt nào, chọn hàm băm nào, rồi dựng dữ liệu chạm đúng trường hợp xấu. Toàn bộ giá trị của ngẫu nhiên hoá nằm ở chỗ đối thủ *không đoán được*, nên hạt giống phải lấy từ nguồn không đoán trước như đồng hồ độ phân giải cao.
- A5. Vì thất bại của nó là thất bại có xác suất, không phải bế tắc, nên lặp lại giải quyết được. Xác suất một lần chạy ra đúng lát cắt nhỏ nhất là khoảng $2/n^2$; lặp $O(n^2 \log n)$ lần cho xác suất bỏ sót nhỏ tùy ý. Giá trị của nó không nằm ở hiệu năng — các thuật toán dựa trên luồng nhanh hơn — mà ở chỗ nó cho thấy một thủ tục cực kỳ đơn giản, không dùng lý thuyết nào ngoài việc co cạnh, vẫn giải được bài toán mà trước đó chỉ có lời giải phức tạp.

NÓI LẠI THẬT ĐƠN GIẢN

Có một hiểu lầm phổ biến: nghĩ rằng dùng số ngẫu nhiên là phó mặc cho may rủi. Thực ra thì ngược lại — người ta dùng ngẫu nhiên để *khỏi phụ thuộc vào may rủi*.

Hãy nhớ lại chuyện cái tủ hồ sơ ở chương về băm. Nếu quy tắc xếp hồ sơ của bạn là cố định thì người khác có thể mang tới toàn những hồ sơ rơi vào cùng một ngăn, và tủ của bạn thành vô dụng. Nhưng nếu mỗi sáng bạn trộn thêm một con số bí mật vào quy tắc thì họ không đoán được nữa. Bạn không hy vọng gặp may; bạn đã *loại bỏ* khả năng bị hại.

Cùng logic đó áp cho việc sắp xếp: chọn phần tử làm mốc một cách ngẫu nhiên thì không có dây đầu vào nào là “dây xấu” nữa, vì cái xấu bây giờ phụ thuộc vào đồng xu của bạn chứ không phụ thuộc vào dữ liệu.

Loại thứ hai của ngẫu nhiên hoá thì khác: chấp nhận trả lời sai đôi khi, để đổi lấy tốc độ. Nghe đáng lo, cho tới khi bạn làm phép tính: nếu mỗi lần trả lời sai với xác suất một phần tư, thì ba mươi lần liên tiếp cùng sai có xác suất nhỏ hơn xác suất máy tính của bạn hỏng giữa chừng. Ở mức đó, “có thể sai” trở thành một khái niệm không còn ý nghĩa thực tế.

Loại thứ ba là lấy mẫu, và nó đến từ một ý rất đời: nếu bạn muốn biết trong nồi canh đủ mặn chưa, bạn không uống cả nồi — bạn ném một thìa. Với điều kiện là đã khuấy đều. Toàn bộ nghệ thuật của các phương pháp lấy mẫu nằm ở đúng chữ “khuấy đều” đó.

Một câu: Đừng hy vọng dữ liệu ngẫu nhiên; hãy tự tạo ngẫu nhiên ở chỗ bạn kiểm soát.

Chỗ không rút gọn được: Chất lượng của nguồn ngẫu nhiên là chuyện thật, không phải chi tiết: một bộ sinh kém hoặc một hạt giống đoán được sẽ phá huỷ mọi đảm bảo ở trên.

Thuật ngữ / câu hỏi	Trả lời
●●● Nguyên lý cốt lõi	Khi trường hợp xấu phụ thuộc dữ liệu, chuyển sự phụ thuộc sang nguồn ngẫu nhiên bạn kiểm soát; đảm bảo trở thành đúng cho <i>mọi</i> dữ liệu vào.
●●● Ngẫu nhiên hoá khác phân tích trung bình	Trung bình giả định về phân phối <i>dữ liệu</i> (không kiểm soát được); ngẫu nhiên hoá đặt ngẫu nhiên vào <i>lựa chọn của thuật toán</i> .
●●● Las Vegas so với Monte Carlo	Las Vegas: luôn đúng, thời gian ngẫu nhiên. Monte Carlo: luôn nhanh, có thể sai. Kết quả kiểm chứng được nhanh thì chuyển Monte Carlo thành Las Vegas bằng cách lặp.
●●● Khuếch đại xác suất	Sai với xác suất p mỗi lần, lặp k lần độc lập thì p^k ; $k = 30$ với $p = 1/4$ cho dưới 10^{-18} . Bắt buộc: các lần chạy độc lập.
●●● Hạt giống trong thi đấu	Phải lấy từ nguồn không đoán trước (đồng hồ độ phân giải cao); hạt giống cố định biến thuật toán thành tất định và dựng test phá được.
●● Bốn vai trò của ngẫu nhiên hoá	Phá trường hợp xấu do dữ liệu; kiểm rẻ hơn tính; lấy mẫu thay vết cặn; thoát khỏi cấu trúc xấu.
●● Freivalds	Kiểm $AB = C$ bằng vector ngẫu nhiên trong $O(n^2)$ thay vì $O(n^3)$; ý tưởng “kiểm một đẳng thức chỉ cần một hướng ngẫu nhiên”.
●● Miller–Rabin	Chọn nhân chứng ngẫu nhiên; mỗi nhân chứng phát hiện hợp số với xác suất $\geq 3/4$; là thuật toán dừng trong mọi thư viện mật mã.
●● Karger	Co cạnh ngẫu nhiên tới khi còn hai đỉnh; xác suất đúng $\approx 2/n^2$, lặp $O(n^2 \log n)$ lần. Giá trị nằm ở sự đơn giản, không ở tốc độ.
●● Lấy mẫu hồ chứa	Giữ k phần tử đầu; phần tử thứ $i > k$ được giữ với xác suất k/i , thay thế ngẫu nhiên một phần tử trong hồ. Bộ nhớ $O(k)$, không cần biết độ dài dòng.
●● Ngẫu nhiên không sửa được cái gì	Không sửa được thuật toán <i>sai</i> về logic; nó chỉ giải quyết vấn đề phân phối dữ liệu.
● Ngẫu nhiên và lớp độ phức tạp	Phần lớn giới nghiên cứu tin rằng ngẫu nhiên không thêm sức mạnh về lớp; giá trị thật của nó là sự đơn giản và hằng số nhỏ.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên dùng bộ sinh ngẫu nhiên nào trong C++?

| Dùng `mt19937` (hoặc `mt19937_64` cho số 64 bit) với hạt giống lấy từ đồng hồ độ phân giải cao. Tránh `rand()`: chất lượng kém, khoảng giá trị nhỏ trên một số nền tảng, và cách lấy ngẫu nhiên trong một khoảng bằng phép chia dư gây lệch phân phối. Để lấy số ngẫu nhiên trong $[a, b]$, dùng `uniform_int_distribution` thay vì phép chia dư.

2. Trộn ngẫu nhiên mảng trước khi sắp có thực sự cần không?

| Cần, trong hai tình huống. Thứ nhất, khi bạn tự cài quicksort mà không chọn chốt ngẫu nhiên — lúc đó trộn trước là cách rẻ nhất để loại bỏ trường hợp xấu. Thứ hai, khi dùng hàm sắp xếp thư viện trong một kỳ thi mà bạn biết có test phá nhắm vào cài đặt cụ thể của thư viện đó. Chi phí là một dòng và $O(n)$, còn lợi ích là loại hẳn một lớp thất bại — một đánh đổi rất đáng.

3. Làm sao chứng minh một thuật toán ngẫu nhiên là đúng?

| Chia làm hai phần và đừng lẫn chúng. Phần tính đúng: với Las Vegas thì chứng minh như thuật toán tất định — bất biến, quy nạp ([A/04](#)); ngẫu nhiên chỉ ảnh hưởng thời gian. Với Monte Carlo thì phải chặn xác suất sai, thường bằng cách chỉ ra “một lựa chọn ngẫu nhiên là xấu chỉ khi rơi vào một tập nhỏ”. Phần chi phí: dùng kỳ vọng tuyến tính và các bất đẳng thức đuôi ở [A/05](#), và nhớ nói rõ kỳ vọng lấy theo cái gì.

4. Nếu ngẫu nhiên hoá tốt như vậy, vì sao không dùng nó ở mọi nơi?

| Vì nó có ba cái giá. Một, kết quả không tái lập được giữa các lần chạy, gây khó cho gỡ lỗi — cách chữa là ghi lại hạt giống của mỗi lần chạy để tái hiện. Hai, với Monte Carlo, bạn phải chấp nhận xác suất sai khác không; trong các hệ thống quan trọng, điều này cần được nêu rõ chứ không được giấu. Ba, một số bài đơn giản là không có cấu trúc để ngẫu nhiên hoá khai thác. Ngẫu nhiên là công cụ, không phải liều thuốc chung.

5. Trong hệ thống thật, tôi phải cẩn thận gì với ngẫu nhiên?

| Ba điều. Một, ghi lại hạt giống để tái lập được sự cố — đây là điều khác biệt lớn nhất giữa hệ thống có kỷ luật và hệ thống không. Hai, phân biệt bộ sinh ngẫu nhiên thông thường với bộ sinh an toàn mật mã: dùng loại thứ nhất cho khoá hay mã phiên là một lỗ hổng bảo mật thật sự. Ba, trong hệ nhiều luồng, chia sẻ một bộ sinh giữa các luồng vừa gây tranh chấp vừa có thể phá tính độc lập — mỗi luồng nên có bộ sinh riêng với hạt giống khác nhau.

6. RANSAC có phải thuật toán ngẫu nhiên theo nghĩa của chương này không?

| Đúng, và nó thuộc vai trò thứ tư. Bài toán là ước lượng một mô hình từ dữ liệu có nhiều điểm ngoại lai; vết cận mọi tập con là bất khả thi. RANSAC lấy ngẫu nhiên một tập con nhỏ vừa đủ để xác định mô hình, đếm số điểm ủng hộ, và lặp lại. Lập luận về số lần lặp cần thiết chính là lập luận khuếch đại ở mục 3: nếu xác suất một lần lấy mẫu “sạch” là p thì sau k lần, xác suất chưa từng sạch là $(1 - p)^k$, và ta chọn k để con số đó đủ nhỏ.

7. Có cách nào biến thuật toán ngẫu nhiên thành tất định không?

| Có, và nhánh nghiên cứu đó gọi là khử ngẫu nhiên. Kỹ thuật thường dùng là thay nguồn ngẫu nhiên thật bằng một họ hàm nhỏ có tính chất tương tự — ví dụ dùng một họ hàm băm độc lập đôi một thay vì hàm ngẫu nhiên hoàn toàn, khi phân tích chỉ cần tính độc lập đôi một. Trong thực hành thì đầu, khử ngẫu nhiên hiếm khi đáng làm; giá trị của nó nằm ở lý thuyết và ở các tình huống bắt buộc phải tái lập được kết quả.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Trộn ngẫu nhiên một mảng đúng cách	Bài tập cài đặt	Thuật toán Fisher–Yates; kiểm bằng thống kê rằng mọi hoán vị xuất hiện đều nhau trên $n = 4$.
● Cài quicksort với chốt ngẫu nhiên và đo trên mảng đã sắp	Bài tập thực nghiệm	So với chốt cố định trên cùng dữ liệu — khác biệt sẽ rất rõ.
●● Kiểm tra nguyên tố Miller–Rabin cho số tới 10^{18}	Bài kinh điển	Chú ý phép nhân modulo tránh tràn; so thời gian với thử chia tới $\sqrt{n}$.
●● Kiểm tra $AB = C$ bằng thuật toán Freivalds	Bài kinh điển	Đo thời gian so với nhân ma trận thật; lặp 30 lần và ước lượng xác suất sai.
●● Lấy mẫu hồ chứa trên một dòng dữ liệu dài	Bài tập cài đặt	Chứng minh tính đều bằng quy nạp trước khi cài; kiểm bằng thống kê sau khi cài.
●● Tìm phần tử xuất hiện quá nửa bằng phương pháp ngẫu nhiên	Bài kinh điển	Chọn ngẫu nhiên một phần tử và kiểm; xác suất trúng $> 1/2$ nên vài chục lần là đủ. So với thuật toán bỏ phiếu tất định.
●●● Cài lát cắt nhỏ nhất của Karger	Bài kinh điển	Dùng DSU cho phép co cạnh ($B/11$); đo tỉ lệ ra đúng theo số lần lặp.
●●● Băm chuỗi với cơ số ngẫu nhiên, so với cơ số cố định trên test phá	Bài tập an ninh	Tự dựng test phá cho cơ số cố định — bài tập cho thấy rõ nhất vì sao phải ngẫu nhiên hoá.
●●● Ước lượng số nút cây quay lui bằng phương pháp Knuth	Chương C/13	Vai trò 3 của ngẫu nhiên hoá; so ước lượng với số thật trên bài n quân hậu.
●●● RANSAC ước lượng một đường thẳng từ dữ liệu có 50% điểm ngoại lai	Bài tập ứng dụng	Tính số lần lặp cần thiết trước khi chạy, rồi kiểm chứng bằng thực nghiệm.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Tìm kiếm cây Monte Carlo cho một trò chơi nhỏ	Bài tập cài đặt	So với duyệt đầy đủ có cắt tia (C/13); quan sát chất lượng theo số lần mô phỏng.

Kết chương và đọc tiếp

Phần C khép lại ở đây. Chương này gom lại một sợi chỉ đã chạy suốt từ phần B: ngẫu nhiên hoá không phải sự cầu may mà là cách chuyển sự phụ thuộc từ dữ liệu — thứ bạn không kiểm soát — sang các lựa chọn của chính thuật toán — thứ bạn kiểm soát. Hai điều đáng mang theo: bốn vai trò ở mục 5 để biết khi nào nghĩ tới nó, và phép tính khuếch đại cho thấy vì sao “có thể sai” lại trở thành khái niệm không có ý nghĩa thực tế sau vài chục lần lặp.

Tám chương của phần C đều trả lời câu hỏi “dùng khuôn tư duy nào để sinh ra lời giải”. Phần D đổi câu hỏi sang một hướng khác và ít dễ chịu hơn: *khi nào thì nên ngừng tìm lời giải nhanh*. Nó bắt đầu bằng hai miền có bấy số học riêng — số học và hình học — rồi tới chương quan trọng nhất của phần: cách nhận ra một bài toán là NP-khó, và bốn cách sống chung với sự thật đó ([D/23](#)).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Karger & Stein, “A New Approach to the Minimum Cut Problem”	Journal of the ACM, 1996	Thuật toán co cạnh ngẫu nhiên kèm mẹo đệ quy làm xác suất thành công tăng vọt. Đọc phần phân tích xác suất — ngắn, đẹp, và là mẫu mực cho việc khuếch đại.
nền tảng	Motwani & Raghavan, <i>Randomized Algorithms</i>	Cambridge University Press, 1995	Sách chuẩn của lĩnh vực; đọc chương về thuật toán Las Vegas và Monte Carlo cùng chương về bất đẳng thức tập trung.
tra khi cần	Karp & Rabin, “Efficient Randomized Pattern-Matching Algorithms”	IBM Journal of Research and Development, 1987	Băm chuỗi như một thuật toán ngẫu nhiên Monte Carlo; đọc kèm B/08 .
tra khi cần	Alon, Matias, Szegedy, “The Space Complexity of Approximating the Frequency Moments”	Journal of Computer and System Sciences, 1999	Bài mở đường cho thuật toán luồng dữ liệu; đọc khi cần ước lượng thống kê với bộ nhớ nhỏ (D/24).
tra khi cần	Mitzenmacher & Upfal, <i>Probability and Computing</i>	Cambridge University Press, 2005	Dễ vào hơn Motwani–Raghavan và dạy luôn phần xác suất cần thiết.

GIỚI HẠN VÀ MIỀN CHUYÊN



TÓM TẮT

Phần D trả lời hai câu hỏi mà phần C cố tình gác lại: “khi nào thì nên ngừng tìm lời giải nhanh hơn” và “những miền nào có bộ công cụ riêng mà kinh nghiệm chung không chuyển sang được”. Chương 21 là số học và đại số tính toán, chương 22 là hình học tính toán — hai miền mà cạm bẫy chủ yếu không nằm ở ý tưởng mà ở số học hữu hạn. Chương 23 là độ khó và quy dẫn: cách nhận ra một bài là NP-khó và cách sống chung với nó. Chương 24 là cận dưới và các mô hình tính toán khác — luồng dữ liệu, trực tuyến, song song, bộ nhớ ngoài. Nếu chỉ nhớ một điều: biết một bài là NP-khó không phải là thất bại, đó là thông tin đắt giá nhất bạn có thể có trước khi bỏ ra ba ngày tìm lời giải đa thức không tồn tại.

Ba chương đầu của phần này có một điểm chung dễ bị bỏ qua: chúng đều nói về chỗ mà *mô hình lý tưởng gặp thực tế*. Trong chương 21, mô hình lý tưởng nói “cộng hai số mất $O(1)$ ”, còn thực tế nói số nguyên có 64 bit và phép nhân tràn im lặng. Trong chương 22, mô hình lý tưởng nói “ba điểm thẳng hàng hoặc không”, còn thực tế nói số thực dấu phẩy động không có khái niệm bằng nhau. Trong chương 23, mô hình lý tưởng nói “có thuật toán đa thức hoặc không”, còn thực tế nói bài NP-khó vẫn được giải hàng ngày trong công nghiệp bằng heuristic và bộ giải SAT.

Chương 24 đứng riêng vì nó phá bỏ giả định lớn nhất của cả quyển sách: giả định rằng dữ liệu nằm vừa trong bộ nhớ, đọc được nhiều lần, và máy chỉ có một lỗi. Khi giả định đó sai — dữ liệu chảy qua một lần, quyết định phải ra ngay khi chưa thấy tương lai, hoặc đĩa chậm hơn RAM hàng nghìn lần — thì bảng xếp hạng thuật toán thay đổi hoàn toàn. Đây cũng là chương gần với công việc kỹ thuật thật nhất trong cả phần D.

Bản đồ phần D — Giới hạn và miền chuyên

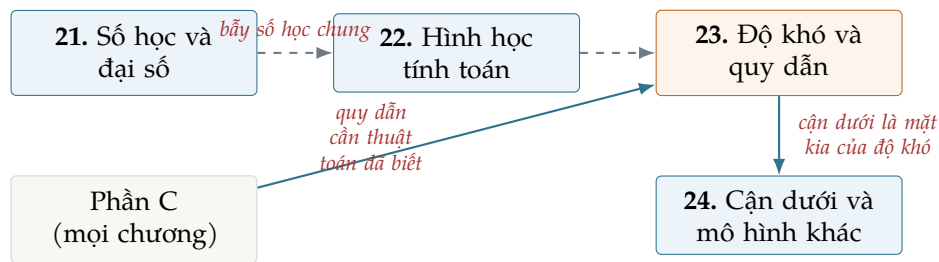
TÓM TẮT

Bốn chương, hai vai trò. Chương 21 và 22 là hai *miền chuyên* có bấy số học riêng. Chương 23 và 24 là hai cách nói về *giới hạn*: giới hạn do bản chất bài toán (NP-khó) và giới hạn do mô hình máy (bộ nhớ, một lượt đọc, không biết tương lai). Chương 23 là chương nên đọc kỹ nhất của phần này vì nó thay đổi cách bạn phân bổ thời gian khi gặp bài lạ.

A. Phần này có gì

- **21-số-học-và-dai-so** — ước chung lớn nhất, số học modulo, nghịch đảo, sàng, phân tích thừa số, định lý phần dư Trung Hoa, lũy thừa ma trận, FFT/NTT, đại số tuyến tính trên $\mathbb{F}_2$.
- **22-hình-học-tính-toan** — vị từ định hướng, bao lồi, quét đường, giao đoạn thẳng, cặp điểm gần nhất, và vấn đề bền vững số học.
- **23-do-kho-và-quy-dan** — P, NP, NP-đầy đủ, cách dựng một quy dẫn, danh mục bài NP-khó hay gặp, và bốn cách sống chung: xấp xỉ, tham số hoá, heuristic, bộ giải.
- **24-can-duoi-và-mô-hình-khac** — cận dưới cây quyết định, lập luận đối thủ, thuật toán luồng dữ liệu, thuật toán trực tuyến và phân tích cạnh tranh, mô hình bộ nhớ ngoài và cache-oblivious, song song.

B. Phụ thuộc và thứ tự đọc



Hình 20.2. Phần D chủ yếu phụ thuộc vào phần C: muốn quy dẫn bài mới về một bài NP-đầy đủ, trước hết phải biết kho bài toán chuẩn. Chương 23 (tô hồ phách) là chương cốt lõi của phần.

C. Cốt lõi hay tham khảo

Cốt lõi: số học modulo, lũy thừa nhanh và nghịch đảo (21); vị từ định hướng và bao lồi (22); toàn bộ phần nhận diện NP-khó và bốn cách đối phó (23). **Tham khảo:** FFT/NTT ở mức cài đặt (21); quét đường cho giao đoạn thẳng (22); chứng minh Cook–Levin (23); toàn bộ chương 24 trừ mục phân cấp bộ nhớ, vốn đã xuất hiện ở chương 3.

Ở **D/22**, dùng bộ hình chạm đầu mút, chồng đoạn và tách rời để kiểm quy ước giao trước khi cài; phép quay không loại bỏ thẳng hàng.

Số học và đại số tính toán (Number Theory and Algebra)

PHỤ THUỘC A/05 cho số học modulo cơ bản; C/14 cho chia để trị — FFT là ứng dụng sâu nhất của nó; C/20 cho kiểm tra nguyên tố xác suất.

TÓM TẮT

Miền này có một đặc điểm riêng: cạm bẫy chủ yếu không nằm ở ý tưởng mà ở *số học hữu hạn* — tràn số, phép chia không tồn tại, và sự khác biệt giữa số nguyên toán học với số nguyên 64 bit. Chương đi từ thuật toán Euclid và nghịch đảo modulo, qua sàng và phân tích thừa số, tới định lý phần dư Trung Hoa, lũy thừa ma trận, và biến đổi Fourier nhanh. Xuyên suốt là một cách nhìn thống nhất: rất nhiều kỹ thuật ở đây là “đổi biểu diễn để phép toán khó trở nên dễ”, đúng mẫu tư duy đã gặp ở tổng tiền tố (B/06). Nếu chỉ nhớ một điều: mọi lỗi số học trong thi đấu quy về ba nguyên nhân — tràn khi nhân, chia mà không có nghịch đảo, và kết quả âm sau khi lấy dư.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ước chung lớn nhất** — số lớn nhất chia hết cả hai; tính bằng thuật toán Euclid trong $O(\log \min(a, b))$.
- **Euclid mở rộng** — ngoài $\gcd(a, b)$ còn cho hai số x, y với $ax + by = \gcd(a, b)$; đây là cách tính nghịch đảo modulo.
- **nghịch đảo modulo** — số a^{-1} với $a \cdot a^{-1} \equiv 1 \pmod{m}$; tồn tại khi và chỉ khi $\gcd(a, m) = 1$. Thay cho phép chia.
- **sàng Eratosthenes** — đánh dấu bội của từng số nguyên tố để tìm mọi số nguyên tố tới n trong $O(n \log \log n)$.
- **định lý phần dư Trung Hoa** — biết phần dư của x theo nhiều modulo đôi một nguyên tố cùng nhau thì xác định được x theo tích của chúng.
- **biến đổi Fourier nhanh** — thuật toán $O(n \log n)$ tính tích chập hai dãy; ý tưởng là đổi sang biểu diễn mà tích chập trở thành phép nhân từng điểm.

CÂU HỎI MỞ ĐẦU

- Q1. Thuật toán Euclid ra đời hơn hai nghìn năm trước và vẫn được dùng nguyên vẹn. Điều gì làm nó bền tới vậy?
- Q2. Vì sao trong số học modulo, phép chia lại phải thay bằng phép nhân với nghịch đảo, và vì sao nghịch đảo không phải lúc nào cũng tồn tại?
- Q3. Nhân hai đa thức bậc n tốn $O(n^2)$ theo cách thông thường. FFT xuống $O(n \log n)$ bằng cách nào, và ý tưởng đó có họ hàng gì với tổng tiền tố?
- Q4. Vì sao các bài đếm hay yêu cầu modulo 998 244 353 thay vì $10^9 + 7$?
- Q5. Bài toán phân tích một số thành thừa số nguyên tố khó tới mức nào, và vì sao độ khó đó lại là nền của mã hoá hiện đại?

Miền số học có một đặc điểm khiến nó khác các chương trước: phần lớn lỗi ở đây không phải lỗi thuật toán mà là lỗi *số học hữu hạn*. Bạn viết đúng công thức toán học, nhưng máy tính làm

việc với số nguyên 64 bit và với phép chia lấy dư có dấu, và kết quả sai ở những chỗ mà toán học không hề cảnh báo.

Vì vậy chương này sẽ nhấn mạnh ba thứ song song: ý tưởng toán học, cách nó trở thành thuật toán, và *chỗ nó hỏng trên máy thật*. Cách sắp xếp đó phản ánh đúng thứ tự mà bạn sẽ gặp vấn đề trong thực tế.

21.1 Euclid: thuật toán cổ nhất còn dùng

Thuật toán Euclid tính $\gcd(a, b)$ bằng cách thay (a, b) bởi $(b, a \bmod b)$ cho tới khi số thứ hai bằng 0. Tính đúng dựa trên một bất biến rất gọn: $\gcd(a, b) = \gcd(b, a \bmod b)$. Tính đúng dựa trên một đại lượng giảm: số thứ hai giảm nghiêm ngặt. Đây là ví dụ mẫu mực cho bộ đôi công cụ ở chương A/04, và cũng là lý do thuật toán này thường được dùng làm ví dụ đầu tiên khi dạy chứng minh.

Chi phí là $O(\log \min(a, b))$, và trường hợp xấu nhất xảy ra đúng với các số Fibonacci liên tiếp — một liên hệ đẹp giữa lý thuyết số và tổ hợp.

Euclid mở rộng cho thêm hai hệ số x, y thoả $ax + by = \gcd(a, b)$. Ứng dụng quan trọng nhất là tính nghịch đảo modulo: nếu $\gcd(a, m) = 1$ thì $ax + my = 1$, tức $ax \equiv 1 \pmod{m}$, nên x chính là a^{-1} . Đây là câu trả lời cho câu hỏi mở đầu thứ hai: trong số học modulo không có phép chia; “chia cho a ” được định nghĩa là “nhân với nghịch đảo của a ”, và nghịch đảo chỉ tồn tại khi a và m nguyên tố cùng nhau.

Hệ quả thực hành rất cụ thể: khi m là số nguyên tố thì mọi số khác 0 đều có nghịch đảo, nên mọi công thức có phân số đều tính được. Đó chính là lý do đề bài luôn chọn modulo nguyên tố.

21.2 Sàng, phân tích thừa số, và ranh giới của cái dễ

Sàng Eratosthenes tìm mọi số nguyên tố tới n bằng cách đánh dấu bội của từng số nguyên tố. Chi phí là $O(n \log \log n)$ — và con số $\log \log$ này đến từ tổng $\sum_p n/p$ trên các số nguyên tố, một kết quả của lý thuyết số mà chương A/05 đã nhắc.

Hai biến thể đáng biết: sàng lưu *ước nguyên tố nhỏ nhất* của mỗi số, cho phép phân tích thừa số bất kỳ số nào $\leq n$ trong $O(\log n)$; và *sàng tuyến tính*, đảm bảo mỗi hợp số bị đánh dấu đúng một lần, cho $O(n)$ nhưng hằng số không nhỏ hơn nhiều nên hiếm khi cần.

Phân tích một số lớn thì hoàn toàn khác. Thử chia tới $\sqrt{n}$ là $O(\sqrt{n})$ — chấp nhận được tới 10^{12} nhưng không hơn. Thuật toán Pollard rho, một thuật toán ngẫu nhiên đẹp, cho kỳ vọng $O(n^{1/4})$ và xử lý được số 64 bit. Nhưng với số hàng trăm chữ số thì không thuật toán nào đã biết chạy nổi trong thời gian hợp lý.

Ranh giới đó chính là điều đáng nhớ nhất mục này, và nó là câu trả lời cho câu hỏi mở đầu thứ năm. Kiểm tra một số có nguyên tố hay không thì *dễ* (Miller–Rabin, C/20); phân tích nó ra thừa số thì *khó*. Sự bất đối xứng giữa hai bài toán tưởng như anh em ấy là nền của mã hoá khoá công khai: ai cũng nhân được hai số nguyên tố lớn với nhau, nhưng không ai tách ngược lại được.

CẠM BẮY

Ba lỗi số học chiếm gần hết các lần sai trong bài modulo. *Trên khi nhân*: với $m \approx 10^9$, tích hai phần dư đã cỡ 10^{18} , sát giới hạn số nguyên 64 bit có dấu — và nếu $m \approx 10^{18}$ thì bắt buộc phải dùng kỹ thuật nhân đặc biệt. *Kết quả âm*: trong nhiều ngôn ngữ, phép lấy dư của số âm trả về số âm, nên sau phép trừ phải cộng thêm m rồi lấy dư lần nữa. *Chia mà không*

kiểm nghịch đảo: nếu modulo không nguyên tố và số chia không nguyên tố cùng nhau với nó thì nghịch đảo không tồn tại, và công thức của bạn vô nghĩa chứ không chỉ sai số.

21.3 “Kiểm tra thì dễ” – dễ tới mức nào, và từ khi nào

Câu “kiểm tra một số có nguyên tố hay không thì dễ” ở mục trên xứng đáng được nói chính xác hơn, vì đằng sau nó là một trong những kết quả đẹp nhất của lý thuyết tính toán.

Miller–Rabin là thuật toán *ngẫu nhiên*: nó chỉ sai với xác suất nhỏ tùy ý, và trong thực tế thì thế là quá đủ. Nhưng câu hỏi lý thuyết vẫn để ngỏ suốt nhiều thập kỷ: có thuật toán *tất định*, thời gian đa thức, không cần giả thuyết chưa chứng minh nào, để kiểm tra tính nguyên tố hay không? Bản tất định của Miller chạy đa thức nhưng phải dựa vào một giả thuyết lớn của lý thuyết số, nên nó không tính là câu trả lời.

Câu trả lời tới năm 2002, từ Agrawal cùng hai sinh viên của ông là Kayal và Saxena, và được đăng trên *Annals of Mathematics* hai năm sau [72] với một nhan đề gọn tới mức nổi tiếng: “PRIMES is in P”. Thuật toán không dài, không dùng công cụ nặng, và chứng minh vừa trong mười trang. Ý nghĩa của nó với chương này là làm sắc lại đúng sự bất đối xứng đã nói ở mục 2: *kiểm tra* tính nguyên tố nằm trong P một cách vô điều kiện, trong khi *phân tích thừa số* vẫn chưa ai biết cách làm trong thời gian đa thức. Toàn bộ mã hoá khoá công khai sống trên khe hở giữa hai câu ấy.

Và như mọi lần trong quyển sách này, phải kèm câu thứ hai: không ai dùng AKS trong thực tế. Số mũ của nó lớn tới mức Miller–Rabin lặp vài chục vòng nhanh hơn nhiều bậc, với xác suất sai nhỏ hơn xác suất máy tính lỗi phần cứng. Đây là cùng bài học của C/14: kết quả trả lời câu hỏi “có thể hay không”, chứ không phải câu hỏi “nên cài gì”. Người học nên tách bạch hai câu hỏi đó ngay từ đầu, vì trộn chúng là nguồn của cả sự hoài nghi vô lý với lý thuyết lẫn sự sùng bái vô lý với các cặn mới.

21.4 Định lý phần dư Trung Hoa: ghép các mảnh thông tin

Nếu biết $x \equiv a_1 \pmod{m_1}$ và $x \equiv a_2 \pmod{m_2}$ với m_1, m_2 nguyên tố cùng nhau, thì tồn tại duy nhất một x modulo $m_1 m_2$. Ý nghĩa thực dụng: bạn có thể *tách* một phép tính lớn thành nhiều phép tính nhỏ theo các modulo nhỏ, làm độc lập, rồi ghép lại.

Ba ứng dụng hay gặp. Thứ nhất, tránh tràn: thay vì làm việc với modulo 10^{18} , làm hai lần với hai modulo cỡ 10^9 rồi ghép. Thứ hai, tăng độ tin cậy của băm chuỗi (B/08): hai modulo cho xác suất va chạm bằng tích hai xác suất. Thứ ba, trong biến đổi Fourier trên số nguyên khi modulo của bài không có tính chất cần thiết: làm với vài modulo thân thiện rồi ghép.

Nhìn ở mức trừu tượng hơn, đây lại là mẫu quen thuộc: *đổi biểu diễn để phép toán trở nên dễ*. Ta chuyển một số lớn thành bộ phần dư của nó, làm việc trong biểu diễn đó, rồi chuyển ngược. Cùng ý tưởng với tổng tiền tố và mảng hiệu ở B/06, và với FFT ở mục sau.

21.5 Luỹ thừa nhanh và luỹ thừa ma trận

Tính $a^n \bmod m$ bằng cách nhân n lần là $O(n)$; bằng bình phương liên tiếp là $O(\log n)$. Ý tưởng: $a^n = (a^{n/2})^2$ nếu n chẵn, và $a \cdot a^{n-1}$ nếu lẻ — chia để trị ở dạng đơn giản nhất.

Cùng kỹ thuật áp cho ma trận, và đó là công cụ giải các bài “tính trạng thái sau 10^{18} bước”. Điều kiện, như đã nói ở A/05, là quan hệ truy hồi phải *tuyến tính*. Khi đó $v_n = A^n v_0$ và A^n tính được trong $O(d^3 \log n)$ với d là kích thước ma trận.

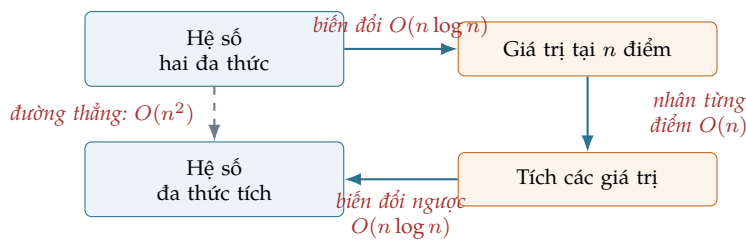
Ba mẹo mở rộng đáng biết. Nếu truy hồi có thêm hằng số cộng, hãy thêm một chiều mang giá trị 1 vào vector trạng thái. Nếu cần *tổng* của dãy, thêm một chiều tích lũy. Và nếu bài hỏi số đường đi độ dài k trong đồ thị, lũy thừa ma trận kể đếm đúng thứ đó — một liên hệ đẹp giữa đại số và đồ thị (C/17).

21.6 Biến đổi Fourier nhanh: đổi miền để nhân trở thành dễ

Bài toán: cho hai đa thức bậc n , tính tích. Cách trường học tốn $O(n^2)$. Karatsuba (C/14) cho $O(n^{1,585})$. FFT cho $O(n \log n)$.

Ý tưởng gồm ba bước, và bước giữa mới là chỗ đáng nhớ. Một đa thức bậc n được xác định hoàn toàn bởi giá trị của nó tại $n + 1$ điểm. Trong *biểu diễn theo giá trị*, phép nhân hai đa thức trở thành phép nhân từng điểm — chỉ $O(n)$. Vậy thuật toán là: đổi cả hai đa thức sang biểu diễn giá trị, nhân từng điểm, rồi đổi ngược về biểu diễn hệ số. Phần khó là làm hai phép đổi đó nhanh, và FFT làm được nhờ chọn các điểm rất đặc biệt — các căn bậc n của đơn vị — khiến phép đổi có cấu trúc chia để trị.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó cho thấy rõ mẫu tư duy chung của cả chương: *phép toán khó ở biểu diễn này lại dễ ở biểu diễn kia; hãy đi đường vòng*. Bạn đã gặp mẫu này ba lần — tổng tiền tố và mảng hiệu, định lý phần dư Trung Hoa, và bây giờ là FFT.



Hình 21.1. Mẫu “đi đường vòng” của cả chương. Phép nhân tốn $O(n^2)$ ở biểu diễn hệ số nhưng chỉ $O(n)$ ở biểu diễn giá trị, nên ta đổi biểu diễn, làm việc, rồi đổi ngược. Cùng mẫu xuất hiện ở tổng tiền tố và mảng hiệu (B/06) và ở định lý phần dư Trung Hoa.

Biến đổi trên số nguyên (NTT) thay các căn phức bằng các căn của đơn vị trong số học modulo. Điều kiện là modulo phải có căn nguyên thủy với bậc là lũy thừa đủ lớn của 2 — và đó chính là lý do 998 244 353 được ưa dùng, vì nó bằng $119 \cdot 2^{23} + 1$. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: modulo đó không phải chọn tùy tiện mà được chọn vì tính chất đại số của nó.

21.7 Đại số trên $\mathbb{F}_2$: khi cộng là XOR

Khi làm việc với các bit, phép cộng modulo 2 chính là XOR, và cả bộ máy đại số tuyến tính vẫn dùng được — chỉ khác là mọi phép toán chạy trên bit nên rất nhanh.

Ứng dụng chính là *cơ sở tuyến tính* của một tập số: duy trì một tập tối thiểu các số sao cho mọi XOR của tập gốc đều biểu diễn được. Với cơ sở đó, ta trả lời được “số này có là XOR của một tập con không”, “XOR lớn nhất có thể là bao nhiêu”, và “có bao nhiêu giá trị XOR phân biệt” — tất cả trong $O(\log C)$ mỗi thao tác.

Cùng khung áp cho giải hệ phương trình tuyến tính trên $\mathbb{F}_2$, thứ xuất hiện trong các bài về đèn và công tắc, và trong lý thuyết mã sửa lỗi.

21.8 Gốc rễ: từ Euclid tới RSA

Chương này có gốc rễ cổ nhất trong cả quyển sách. Thuật toán Euclid xuất hiện trong bộ *Cơ sở* khoảng năm 300 trước Công nguyên, và nó thường được gọi là thuật toán lâu đời nhất còn được dùng nguyên vẹn tới ngày nay — không phải một phiên bản cải tiến, mà đúng thủ tục đó. Sàng Eratosthenes cũng thuộc thời kỳ đó. Điều đáng suy nghĩ: cả hai đều là những thuật toán mà tính đúng dựa trên một bất biến rất gọn, và có lẽ chính sự gọn ấy là lý do chúng sống lâu.

Định lý phần dư Trung Hoa có tên như vậy vì dạng sớm nhất của nó xuất hiện trong một sách toán Trung Quốc khoảng thế kỷ 3–5, dưới hình thức một bài đố: tìm số mà chia 3 dư 2, chia 5 dư 3, chia 7 dư 2. Việc một bài đố dân gian trở thành công cụ tính toán hiện đại là một mẫu lặp lại trong lịch sử toán học.

Bước ngoặt lớn nhất của miền này thì rất gần đây và có tính bước ngoặt thật sự. Suốt hàng thế kỷ, lý thuyết số được coi là ngành toán “thuần túy nhất”, không có ứng dụng — một nhà toán học nổi tiếng đầu thế kỷ 20 còn tự hào về điều đó. Rồi năm 1976–78, ý tưởng mã hoá khoá công khai ra đời, và hệ RSA dựa trực tiếp lên sự bất đối xứng ở mục 2: nhân hai số nguyên tố lớn thì dễ, tách ngược thì khó. Chỉ trong vài năm, những kết quả về số nguyên tố và số học modulo từ chỗ là trò chơi trí tuệ trở thành hạ tầng của thương mại điện tử và của mọi kết nối an toàn trên Internet.

Bài học từ câu chuyện đó vượt ra ngoài số học: *một bài toán được chứng minh là khó có thể trở thành tài nguyên quý*. Trong chương [D/23](#) ta sẽ gặp lại ý này ở dạng tổng quát — độ khó không chỉ là chướng ngại, đôi khi nó là thứ ta cần.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
~300 TCN — Euclid	Tìm ước chung lớn nhất của hai đoạn thẳng (đo lường hình học).	Thuật toán cổ nhất còn dùng nguyên vẹn; tính đúng dựa trên một bất biến, tính dùng dựa trên một đại lượng giảm.
~200 TCN — Eratosthenes	Liệt kê số nguyên tố.	Sàng; vẫn là cách nhanh nhất để tìm mọi số nguyên tố trong một khoảng.
Thế kỷ 3–5 — toán học Trung Quốc	Bài đố: tìm số biết các phần dư.	Định lý phần dư Trung Hoa; nay dùng để tách phép tính lớn thành nhiều phép tính nhỏ.
1640–1801 — Fermat, Euler, Gauss	Hệ thống hoá tính chất của số nguyên tố và đồng dư.	Định lý Fermat nhỏ (nền của cả nghịch đảo modulo lẫn kiểm tra nguyên tố) và ngôn ngữ đồng dư hiện đại.
1965 — Cooley & Tukey	Xử lý tín hiệu địa chấn quy mô lớn.	FFT; ý tưởng “đổi miền để tích chập thành phép nhân từng điểm” trở thành công cụ phổ quát.
1976–78 — mã hoá khoá công khai, RSA	Trao đổi khoá an toàn qua kênh công khai.	Lý thuyết số từ ngành “thuần túy nhất” trở thành hạ tầng của Internet; sự khó của bài phân tích thừa số trở thành tài nguyên.
1975–76 — kiểm tra nguyên tố xác suất	Cần kiểm nguyên tố số hàng trăm chữ số để sinh khoá.	Miller–Rabin; ngẫu nhiên hoá trở thành công cụ sản xuất, không phải giải pháp tạm (C/20).

21.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Mật mã	RSA, trao đổi khóa, chữ ký số	Dùng đúng các công cụ của chương: lũy thừa modulo, nghịch đảo, kiểm tra nguyên tố. Khác biệt quyết định: ở đó ta dựa vào việc bài toán ngược <i>không</i> giải được nhanh — độ khó là tính năng, không phải lỗi.
Mã sửa lỗi	Reed–Solomon trong mã QR, đĩa quang, truyền tin vệ tinh	Đại số đa thức trên trường hữu hạn — cùng bộ máy với FFT/NTT. Ý tưởng chung: một đa thức bậc k được xác định bởi $k + 1$ điểm, nên gửi dư điểm thì mất vài điểm vẫn khôi phục được.
Xử lý tín hiệu	Tích chập, lọc, phổ tần số	FFT là công cụ nền. Trong thuật toán ta dùng nó để nhân đa thức; trong xử lý tín hiệu, cùng phép biến đổi cho ý nghĩa vật lý là phân tích tần số.
Đồ hoạ và học sâu	Tích chập trong mạng nơ-ron, lọc ảnh	Phép tích chập giống hệt phép nhân đa thức. Trong thực tế, thư viện chọn giữa tích chập trực tiếp và FFT tùy kích thước hạt nhân — đúng bài toán ngưỡng hoà vốn ở C/14.
Truyền thông và lưu trữ	Tổng kiểm tra CRC	Số học đa thức trên $\mathbb{F}_2$ — cùng đại số với mục 6. Đây là nơi phép XOR làm việc như phép cộng và toàn bộ lý thuyết vẫn đúng.
Hệ thống nhúng, cảm biến	Số học dấu phẩy tĩnh, tránh chia biến	Cùng mối quan tâm với mục 2: phép chia đắt và có thể mất chính xác, nên người ta thay bằng nhân với nghịch đảo đã tính trước — ý tưởng giống hệt nghịch đảo modulo, chỉ ở miền khác.
Tính toán khoa học	Số học chính xác tùy ý, hệ đại số máy tính	Khi kết quả phải chính xác tuyệt đối — ví dụ kiểm chứng một chứng minh — người ta dùng số nguyên lớn, và mọi thuật toán ở đây phải tính lại chi phí theo số bit thay vì coi mỗi phép toán là $O(1)$ (A/03).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì nó dựa trên một bất biến rất gọn — $\gcd(a, b) = \gcd(b, a \bmod b)$ — và một đại lượng giảm hiển nhiên, nên nó vừa đúng vừa dừng mà không cần giả định gì về dữ liệu. Nó cũng không có tham số nào phải tinh chỉnh, không có trường hợp xấu phụ thuộc dữ liệu, và chi phí $O(\log \min(a, b))$ là gần như tối ưu. Những thuật toán sống lâu thường có chung đặc điểm đó: ít giả định, ít tham số, chứng minh ngắn.
- A2. Vì trong tập các phần dư modulo m , phép chia không được định nghĩa như trên số hữu tỉ — ta chỉ có cộng, trừ, nhân. “Chia cho a ” được định nghĩa lại là “nhân với số a^{-1} sao cho $a \cdot a^{-1} \equiv 1$ ”. Số đó tồn tại khi và chỉ khi $\gcd(a, m) = 1$; nếu a và m có ước chung thì mọi bội của a đều nằm trong một tập con thực sự của các phần dư và không bao giờ chạm tới 1. Đây là lý do modulo nguyên tố được ưa chuộng: khi đó mọi số khác 0 đều nghịch đảo được.
- A3. Bằng cách đối biểu diễn. Một đa thức bậc n được xác định bởi giá trị tại $n + 1$ điểm, và trong biểu diễn theo giá trị thì nhân hai đa thức chỉ là nhân từng điểm, tốn $O(n)$. Toàn bộ việc còn lại là làm hai phép đối biểu diễn cho nhanh, và FFT làm được nhờ chọn các điểm là căn của đơn vị, khiến phép đối có cấu trúc chia đôi đệ quy. Họ hàng với tổng tiên tố ở chỗ cả hai đều là “đi đường vòng qua một biểu diễn khác, nơi phép toán cần làm trở thành rẻ”.
- A4. Vì $998\,244\,353 = 119 \cdot 2^{23} + 1$, nên trong số học modulo đó tồn tại căn của đơn vị bậc 2^{23} — điều kiện để chạy biến đổi Fourier trên số nguyên với độ dài là lũy thừa của 2 tới 2^{23} . Modulo $10^9 + 7$ không có tính chất đó, nên các bài cần nhân đa thức lớn buộc phải dùng modulo thân thiện với biến đổi. Nói cách khác, đó là một lựa chọn kỹ thuật có lý do đại số cụ thể, không phải quy ước tùy tiện.

A5. Rất khó theo nghĩa thực tiễn: với số hàng trăm chữ số, không thuật toán công khai nào phân tích được trong thời gian hợp lý bằng máy tính cổ điển, dù bài toán này đã được nghiên cứu hàng thế kỷ. Đồng thời, *kiểm tra* một số có nguyên tố hay không lại dễ. Sự bất đối xứng đó cho phép xây một hệ mã mà ai cũng mã hoá được (nhân hai số nguyên tố) nhưng chỉ người giữ khoá mới giải được (biết hai thừa số). Cần lưu ý: đây là độ khó *chưa ai vượt qua*, không phải độ khó đã được chứng minh — chương D/23 sẽ nói vì sao sự phân biệt đó quan trọng.

NÓI LẠI THẬT ĐƠN GIẢN

Số học trong lập trình có một luật chơi khác với toán học ở trường: mọi con số đều bị nhốt trong một cái vòng. Nếu vòng có 12 vạch như mặt đồng hồ thì 10 cộng 5 không ra 15 mà ra 3. Cộng, trừ, nhân đều chạy bình thường trong cái vòng đó. Chỉ có phép chia là rắc rối, vì “chia 2” bây giờ có nghĩa là “nhân với cái gì đó để quay về đúng vị trí ban đầu”, và cái gì đó không phải lúc nào cũng tồn tại. Nó tồn tại khi số bạn chia không có ước chung nào với kích thước vòng — đó là lý do người ta luôn chọn kích thước vòng là số nguyên tố.

Thuật toán cổ nhất còn được dùng tới hôm nay cũng nằm trong chương này, và nó đơn giản tới bất ngờ: muốn tìm ước chung lớn nhất của hai số, cứ lấy số lớn chia số bé rồi thay số lớn bằng phần dư, lặp lại tới khi dư bằng 0. Hai nghìn ba trăm năm và không ai cải tiến được đáng kể.

Còn ý tưởng sâu nhất của chương thì là chuyện đi đường vòng. Nhân hai đa thức dài thì lâu, nhưng nếu thay vì giữ chúng dưới dạng danh sách hệ số, ta ghi lại giá trị của chúng tại một loạt điểm, thì nhân chỉ còn là nhân từng cặp giá trị — rất nhanh. Vấn đề chuyển thành: làm sao đổi qua đổi lại giữa hai cách ghi cho nhanh. Câu trả lời là chọn các điểm thật khéo, và đó là toàn bộ nội dung của phép biến đổi Fourier nhanh.

Cuối cùng là một sự thật đáng nhớ: nhân hai số nguyên tố lớn thì máy tính làm trong tích tắc, nhưng tách ngược một tích ra hai thừa số thì cả thế giới chưa ai làm nổi khi số đủ lớn. Toàn bộ việc mua bán an toàn trên mạng dựa vào đúng sự chênh lệch đó.

Một câu: Trong số học máy tính, mọi thứ chạy trong một cái vòng — và phần lớn lỗi đến từ việc quên mất điều đó.

Chỗ không rút gọn được: Vì sao phân tích thừa số khó thì chưa ai chứng minh được; ta chỉ biết là chưa ai làm được.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Thuật toán Euclid	$\gcd(a, b) = \gcd(b, a \bmod b)$; bất biến cho tính đúng, số thứ hai giảm cho tính dừng; $O(\log \min(a, b))$.
••• Nghịch đảo modulo	Thay cho phép chia; tồn tại khi $\gcd(a, m) = 1$; tính bằng Euclid mở rộng hoặc bằng a^{m-2} khi m nguyên tố.
••• Ba lỗi số học kinh điển	Trần khi nhân hai phần dư cỡ 10^9 ; kết quả âm sau phép trừ rồi lấy dư; chia mà nghịch đảo không tồn tại.
••• Sàng Eratosthenes	$O(n \log \log n)$; biến thể lưu ước nguyên tố nhỏ nhất cho phép phân tích mọi số $\leq n$ trong $O(\log n)$.
••• Bất đối xứng kiểm tra / phân tích	Kiểm nguyên tố thì dễ (Miller–Rabin); phân tích thừa số thì khó — nền của mã hoá khoá công khai.
••• Luỹ thừa nhanh	$a^n = (a^{n/2})^2$; $O(\log n)$. Áp cho ma trận cho các bài truy hồi tuyến tính với n tới 10^{18} .
•• Định lý phần dư Trung Hoa	Phần dư theo các modulo nguyên tố cùng nhau xác định duy nhất số theo tích; dùng để tách phép tính lớn thành nhiều phép tính nhỏ.
•• Ý tưởng của FFT	Đa thức bậc n xác định bởi $n + 1$ giá trị; trong biểu diễn giá trị, nhân là nhân từng điểm $O(n)$; chọn căn của đơn vị để đổi biểu diễn nhanh.
•• Vì sao 998 244 353	Bằng $119 \cdot 2^{23} + 1$ nên có căn của đơn vị bậc 2^{23} — điều kiện chạy biến đổi Fourier trên số nguyên.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Cơ sở tuyến tính trên $\mathbb{F}_2$	Cộng là XOR; duy trì tập tối thiểu để trả lời “có biểu diễn được không”, “XOR lớn nhất”, “bao nhiêu giá trị phân biệt”.
•• Mẫu tư duy chung của chương	Đổi biểu diễn để phép toán khó thành dễ: tổng tiền tố, phần dư Trung Hoa, FFT — ba lần cùng một ý.
• Trường hợp xấu của Euclid	Hai số Fibonacci liên tiếp — một liên hệ giữa lý thuyết số và tổ hợp.
• PRIMES is in P	Kiểm tra tính nguyên tố có thuật toán tất định, đa thức, vô điều kiện (2004); phân tích thừa số thì chưa ai biết. Khe hở giữa hai câu đó là nền của mã hoá khoá công khai. Thực tế vẫn dùng Miller–Rabin.

••• phải thuộc •• nên nắm • biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tính $\binom{n}{k} \bmod p$ cho n tới 10^6 thì làm thế nào?

| Tiên xử lý mảng giai thừa và mảng nghịch đảo giai thừa một lần trong $O(n)$, rồi mỗi truy vấn là một phép nhân ba số trong $O(1)$. Mẹo tính nghịch đảo giai thừa nhanh: tính nghịch đảo của $n!$ một lần bằng lũy thừa, rồi đi ngược $\text{invfact}[i-1] = \text{invfact}[i] \cdot i$. Khi n lớn hơn p thì công thức này không dùng được và cần định lý Lucas.

2. Modulo trong bài của tôi không nguyên tố. Có cách nào tính phân số không?

| Có ba hướng. Nếu số chia nguyên tố cùng nhau với modulo thì nghịch đảo vẫn tồn tại, tính bằng Euclid mở rộng. Nếu không, hãy thử tách modulo thành các lũy thừa nguyên tố, tính theo từng lũy thừa rồi ghép bằng định lý phần dư Trung Hoa. Hoặc, thường là cách tốt nhất, hãy sắp xếp lại công thức để tránh phép chia — ví dụ tính tổ hợp bằng truy hồi Pascal thay vì bằng giai thừa.

3. Làm sao nhân hai số modulo 10^{18} mà không tràn?

| Ba cách. Dùng kiểu số nguyên 128 bit nếu môi trường hỗ trợ — đơn giản nhất. Dùng phép nhân kiểu “nhân bằng cách nhân đôi” theo bit, $O(\log m)$ nhưng chắc chắn. Hoặc dùng mẹo nhân qua số thực dấu phẩy động để ước lượng thương rồi hiệu chỉnh — nhanh nhưng cần cẩn thận về độ chính xác. Trong thi đấu, kiểu 128 bit là lựa chọn mặc định khi có.

4. Khi nào tôi thực sự cần FFT?

| Khi phải tính tích chập của hai dãy dài hơn vài nghìn phần tử — nhân đa thức, đếm số cặp có tổng bằng mỗi giá trị, nhân số lớn. Dưới ngưỡng đó, cách trực tiếp $O(n^2)$ thường nhanh hơn vì hằng số nhỏ hơn nhiều. Dấu hiệu trên đề: bài yêu cầu, với mỗi giá trị s , đếm số cặp (i, j) có $a_i + b_j = s$ — đó chính xác là tích chập.

5. Kiểm tra một số tới 10^{18} có nguyên tố không thì dùng gì?

| Miller–Rabin với một tập nhân chứng cố định đã được kiểm chứng là đủ cho mọi số 64 bit — đây là một trong số ít trường hợp mà thuật toán ngẫu nhiên được “khử ngẫu nhiên” bằng một danh sách nhân chứng cụ thể. Nếu cần phân tích thừa số chứ không chỉ kiểm tra, hãy kết hợp Miller–Rabin với Pollard rho.

6. Bài yêu cầu tính $a^b \bmod m$ với b là số có 10^5 chữ số. Làm sao?

| Dùng định lý Euler: nếu $\gcd(a, m) = 1$ thì $a^{\varphi(m)} \equiv 1$, nên có thể rút gọn số mũ theo modulo $\varphi(m)$. Khi $\gcd(a, m) \neq 1$ thì phải dùng phiên bản mở rộng của định lý, với điều kiện số mũ đủ lớn. Đây là một trong những chỗ dễ sai nhất của chương vì trường hợp biên rất tinh vi — hãy stress test với vết cạn ở m nhỏ.

7. Vì sao chương này lại xếp vào phần “Giới hạn” cùng với NP-khó?

Vì nó là nơi ta gặp lần đầu một bài toán mà *độ khó là điều đáng mong muốn*. Trong toàn bộ phần A–C, khó là chương ngại. Ở đây, sự khó của bài phân tích thừa số là thứ giữ an toàn cho mọi giao dịch trên mạng. Cách nhìn đó chuẩn bị cho chương D/23, nơi ta học cách *nhận ra* một bài toán là khó và coi đó là thông tin có giá trị chứ không phải thất bại.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Euclid và Euclid mở rộng, giải phương trình $ax + by = c$	CSES “Exponentiation”, bài kinh điển[33]	Chú ý điều kiện có nghiệm và cách sinh mọi nghiệm.
● Luỹ thừa nhanh modulo	CSES “Exponentiation”[33]	Chú ý tràn khi nhân và trường hợp số mũ bằng 0.
● Sàng số nguyên tố và đếm ước	CSES “Counting Divisors”[33]	Sàng lưu ước nguyên tố nhỏ nhất; so với cách thử chia từng số.
●● Tính tổ hợp modulo với truy vấn nhiều	CSES “Binomial Coefficients”[33]	Tiền xử lý giai thừa và nghịch đảo giai thừa; mỗi truy vấn $O(1)$.
●● Fibonacci thứ 10^{18} bằng luỹ thừa ma trận	CSES “Fibonacci Numbers”[33]	Kiểm chứng với giá trị nhỏ; chú ý thứ tự nhân ma trận.
●● Phân tích thừa số tới 10^{18}	Bài kinh điển	Miller–Rabin cộng Pollard rho; kiểm bằng cách nhân lại các thừa số.
●● Giải hệ đồng dư bằng định lý phần dư Trung Hoa	Bài kinh điển	Trường hợp modulo không nguyên tố cùng nhau cần xử lý riêng — đừng bỏ qua.
●●● Nhân hai đa thức bằng NTT	Bài kinh điển	Modulo 998 244 353; so thời gian với cách $O(n^2)$ để tìm ngưỡng hoà vốn.
●●● XOR lớn nhất của một tập con	Bài kinh điển	Cơ sở tuyến tính trên $\mathbb{F}_2$; thuật toán tham lam từ bit cao xuống thấp sau khi có cơ sở.
●●● Đếm số cặp có tổng bằng mỗi giá trị	Bài kinh điển	Nhận ra đây là tích chập; đây là bài mẫu để nhận diện dấu hiệu dùng FFT.
●●● Tính $a^b \bmod m$ với b rất lớn	Bài kinh điển	Định lý Euler và phiên bản mở rộng; stress test với m nhỏ vì trường hợp biên rất tinh vi.

Kết chương và đọc tiếp

Chương này là miền mà lỗi hay nằm ở số học chứ không ở ý tưởng, nên ba lỗi ở khối *Cạm bẫy* đáng thuộc hơn bất kỳ công thức nào. Hai điều nữa đáng mang theo: mẫu “đổi biểu diễn để phép toán trở nên dễ”, xuất hiện ba lần trong chương và nhiều lần trong cả quyển sách; và sự bất đối xứng giữa kiểm tra và tìm kiếm, thứ vừa là nền của mật mã vừa là lời mở đầu cho chương D/23.

Chương tiếp theo là miền chuyên thứ hai, và nó có cùng đặc điểm: cạm bẫy không nằm ở ý tưởng mà ở số học. Chỉ khác là ở đó, thủ phạm không phải phép chia dư mà là số dấu phẩy động — và hậu quả nghiêm trọng hơn, vì một phép so sánh sai có thể làm cả thuật toán hình học rơi vào trạng thái không nhất quán (D/22).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Agrawal, Kayal, Saxena, "PRIMES Is in P"	Annals of Mathematics, 2004	Mười trang, sơ cấp một cách đáng ngạc nhiên. Đọc phần mở đầu và phát biểu thuật toán, kể cả khi bỏ qua chứng minh.
nền tảng	Cooley & Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series"	Mathematics of Computation, 1965	Bài gốc của FFT, nền của mục 5. Ngắn, và cho thấy động cơ ban đầu đến từ xử lý tín hiệu.
tra khi cần	Harvey & van der Hoeven, "Integer Multiplication in Time $O(n \log n)$ "	Annals of Mathematics, 2021	Đọc phần mở đầu để nối mục 5 của chương này với biên giới hiện nay của phép nhân (C/14).
tra khi cần	Graham, Knuth, Patashnik, <i>Concrete Mathematics</i>	Addison-Wesley, tái bản 1994	Tra khi cần các tổng và đồng dư xuất hiện trong phân tích; chương về lý thuyết số sơ cấp viết rất kỹ.

Hình học tính toán (Computational Geometry)

PHỤ THUỘC B/07 cho ngăn xếp trong thuật toán bao lồi và cho hàng đợi ưu tiên trong quét đường; C/14 cho bài cặp điểm gần nhất; D/21 cho tinh thần “cạm bẫy nằm ở số học”.

TÓM TẮT

Hình học tính toán có một đặc điểm khác hẳn mọi chương trước: thuật toán thường đơn giản, nhưng *số dấu phẩy động* có thể phá hủy tính đúng theo những cách rất khó truy. Vì vậy chương bắt đầu từ vị từ định hướng — viên gạch mà gần như mọi thuật toán hình học đứng trên — và từ câu hỏi làm sao tính nó cho đáng tin. Sau đó là bao lồi, quét đường, và các bài kinh điển: giao đoạn thẳng, điểm trong đa giác, cặp điểm gần nhất, cặp điểm xa nhất. Nếu chỉ nhớ một điều: hãy làm việc với số nguyên bất cứ khi nào có thể, và khi buộc phải dùng số thực, hãy hiểu rằng “bằng nhau” không còn là một khái niệm dùng được.

KIẾN THỨC CẦN CÓ TRƯỚC

- **tích có hướng (cross product)** — với hai vector trong mặt phẳng, $u \times v = u_x v_y - u_y v_x$; dấu của nó cho biết v nằm bên trái hay bên phải u .
- **vị từ định hướng** — hàm trả lời “ba điểm này quay trái, quay phải, hay thẳng hàng” — tính bằng dấu của tích có hướng. Nền của gần như mọi thuật toán hình học.
- **bao lồi** — đa giác lồi nhỏ nhất chứa tất cả các điểm đã cho; hình dung là dây thun bọc quanh các đỉnh.
- **quét đường** — kỹ thuật cho một đường thẳng tưởng tượng quét qua mặt phẳng, duy trì một cấu trúc dữ liệu về các đối tượng đang cắt nó.
- **suy biến (degenerate case)** — cấu hình đặc biệt — ba điểm thẳng hàng, hai điểm trùng nhau, đoạn thẳng chồng nhau — nơi phần lớn lỗi hình học nằm.
- **vị từ chính xác** — cách tính vị từ đảm bảo đúng dấu, thường bằng số nguyên hoặc bằng số học nhiều độ chính xác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “ba điểm có thẳng hàng không” lại là một câu hỏi khó khi toạ độ là số thực?
- Q2. Bao lồi có thể tính bằng nhiều thuật toán khác nhau. Vì sao cách sắp xếp rồi quét lại phổ biến hơn cả?
- Q3. Kỹ thuật quét đường biến bài toán hai chiều thành bài toán một chiều. Cơ chế nào cho phép làm điều đó?
- Q4. Vì sao một lỗi làm tròn nhỏ trong hình học lại có thể gây ra kết quả hoàn toàn vô nghĩa, chứ không chỉ sai một chút?
- Q5. Khi nào nên dùng số nguyên cho bài hình học, và làm sao tránh tràn khi toạ độ tới 10^9 ?

Có một hiện tượng đặc trưng khi làm bài hình học: chương trình chạy đúng trên mọi ví dụ bạn tự nghĩ ra, rồi sai trên bộ test thật vì có ba điểm thẳng hàng, hoặc hai điểm trùng nhau, hoặc một đoạn thẳng thẳng đứng. Hình học là miền mà *trường hợp suy biến* chiếm phần lớn số lỗi, và điều

đó có lý do sâu xa: thuật toán hình học thường dựa trên các quyết định nhị phân — bên trái hay bên phải, trong hay ngoài — và ở đúng ranh giới giữa hai lựa chọn, mọi thứ trở nên mong manh.

Cộng thêm một tầng khó nữa: nếu toạ độ là số thực, các quyết định đó được tính bằng số dấu phẩy động, vốn có sai số. Một quyết định sai không chỉ làm kết quả lệch một chút mà có thể làm thuật toán rơi vào trạng thái *không nhất quán* — ví dụ tin rằng điểm A ở bên trái điểm B và đồng thời B ở bên trái A — và từ đó cho ra kết quả hoàn toàn vô nghĩa hoặc treo vòng lặp.

Vì vậy chương này ưu tiên hai thứ: các vị từ nền tảng và cách tính chúng cho đáng tin, rồi mới tới thuật toán.

22.1 Vị từ định hướng: viên gạch của mọi thứ

Cho ba điểm A, B, C , câu hỏi cơ bản nhất là: đi từ A tới B rồi tới C , ta rẽ trái, rẽ phải, hay đi thẳng? Câu trả lời nằm ở dấu của tích có hướng

$$(B - A) \times (C - A) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

Dương là rẽ trái, âm là rẽ phải, bằng 0 là thẳng hàng.

Điều đáng nói là gần như mọi thuật toán hình học trong chương này chỉ dùng vị từ đó và một hai vị từ họ hàng. Kiểm tra hai đoạn thẳng có cắt nhau, kiểm tra điểm nằm trong đa giác lồi, xây bao lồi, sắp xếp theo góc — tất cả đều là các cách kết hợp vị từ định hướng. Hệ quả thực hành rất quan trọng: nếu bạn tính vị từ này *đúng dấu* thì phần lớn thuật toán sẽ chạy đúng; nếu tính sai dấu ở các trường hợp gần suy biến thì mọi thứ sụp.

Từ đó có một quy tắc đáng thành phản xạ: *làm việc với số nguyên bất cứ khi nào có thể*. Tích có hướng của các toạ độ nguyên là số nguyên, nên dấu của nó tính được chính xác tuyệt đối — miễn là không tràn. Với toạ độ tới 10^9 , tích có hướng có thể tới $4 \cdot 10^{18}$, sát giới hạn số nguyên 64 bit có dấu; nếu toạ độ lớn hơn hoặc nếu bạn nhân thêm lần nữa thì cần kiểu 128 bit. Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

CẠM BẮY

Khi buộc phải dùng số thực, đừng bao giờ viết `if (a == b)`. Thay bằng so sánh với một ngưỡng: `if (fabs(a-b) < eps)`. Nhưng phải hiểu rằng đây là một sự thoả hiệp chứ không phải lời giải: quan hệ “gần bằng nhau” *không có tính bắc cầu*, nên nếu bạn dùng nó làm quan hệ so sánh trong một hàm sắp xếp, kết quả có thể là không xác định — kể cả sắp chương trình trong một số thư viện. Chọn ε cũng không có công thức chung: quá nhỏ thì không bắt được trường hợp suy biến, quá lớn thì gộp nhầm những điểm thật sự khác nhau.

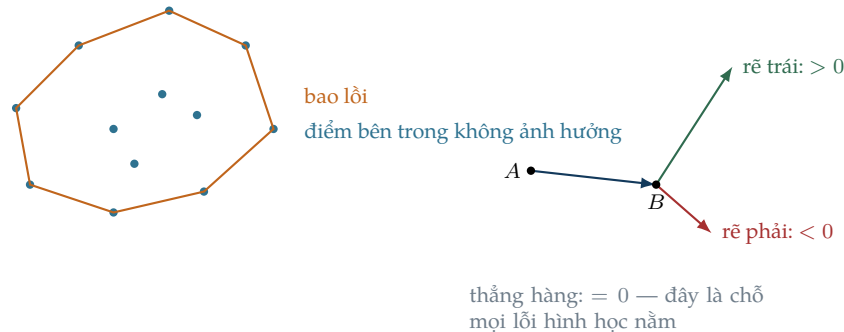
22.2 Bao lồi: thuật toán đầu tiên nên thuộc

Bài toán: cho n điểm, tìm đa giác lồi nhỏ nhất chứa tất cả. Cách hình dung tốt nhất là một sợi dây thun bọc quanh các cây đinh đóng trên bảng.

Cách phổ biến nhất — và cũng dễ cài đúng nhất — là *quét đơn điệu*: sắp các điểm theo hoành độ (rồi tung độ), sau đó xây nửa trên và nửa dưới của bao bằng cách quét một lượt và duy trì một ngăn xếp, loại bỏ mọi điểm làm ta “rẽ sai chiều”. Chi phí $O(n \log n)$, chi phối bởi bước sắp xếp.

Cấu trúc của thuật toán đáng để ý: nó là một *ngăn xếp đơn điệu* (B/07) áp lên tiêu chí hình học thay vì tiêu chí số học. Mỗi điểm vào ngăn xếp một lần và ra tối đa một lần, nên phần quét là $O(n)$ — đúng lập luận khẩu hao quen thuộc.

Bao lồi hữu ích hơn về ngoài của nó vì nhiều bài quy về nó. Cặp điểm *xa nhất* trong tập luôn nằm trên bao lồi, và tìm được bằng kỹ thuật hai con trỏ quay quanh bao. Diện tích nhỏ nhất của hình bao, hình chữ nhật nhỏ nhất bao quanh, kiểm tra một điểm có nằm trong bao — tất cả đều dựa trên bao lồi. Trong tối ưu hoá, bao lồi cũng chính là cấu trúc đẳng sau kỹ thuật tăng tốc quy hoạch động ở C/16: tập các đường thẳng cần lấy cực trị tạo thành một bao lồi.



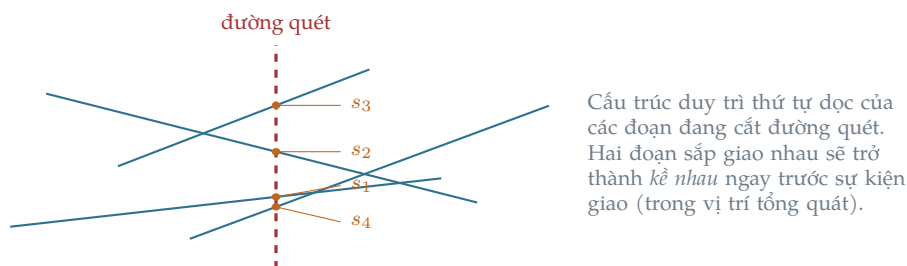
Hình 22.1. Bao lồi và vị từ định hướng. Bên phải là viên gạch: dấu của tích có hướng quyết định mọi thứ. Trường hợp bằng 0 — ba điểm thẳng hàng — là nơi phần lớn lỗi xảy ra, vì đó là ranh giới giữa hai nhánh xử lý, và cũng là nơi sai số dấu phẩy động có thể lật ngược quyết định.

22.3 Quét đường: biến hai chiều thành một chiều

Nhiều bài hình học có dạng “xét mọi cặp đối tượng” và cho $O(n^2)$ một cách hiển nhiên. Kỹ thuật quét đường thường đưa xuống $O(n \log n)$ bằng một ý tưởng chung: cho một đường thẳng đứng quét từ trái sang phải, và duy trì một cấu trúc dữ liệu chứa các đối tượng *đang cắt* đường quét đó, sắp theo tung độ.

Cơ chế làm nó hiệu quả — và đây là câu trả lời cho câu hỏi mở đầu thứ ba — là: tại mỗi thời điểm, ta chỉ cần quan tâm tới quan hệ giữa các đối tượng *kề nhau* trong thứ tự dọc. Hai đoạn thẳng chỉ có thể cắt nhau nếu tại một lúc nào đó chúng trở thành *kề nhau* trên đường quét. Nhờ đó, thay vì xét n^2 cặp, ta chỉ xử lý các sự kiện: một đối tượng bắt đầu, một đối tượng kết thúc, hai đối tượng đổi chỗ. Số sự kiện là $O(n)$ cộng số giao điểm, và mỗi sự kiện tốn $O(\log n)$.

Ba bài kinh điển giải bằng quét đường: tìm mọi giao điểm của n đoạn thẳng; hợp diện tích của n hình chữ nhật; và cặp điểm gần nhất — bài mà chương C/14 giải bằng chia để trị, còn quét đường cho một lời giải khác cũng $O(n \log n)$ nhưng dễ cài hơn với một tập hợp có thứ tự.



Hình 22.2. Quét đường. Bài toán hai chiều trở thành bài toán duy trì một tập có thứ tự trong một chiều, tức là quay về đúng công cụ của B/09. Số sự kiện phải xử lý là $O(n)$ cộng số giao điểm, thay vì $O(n^2)$ cặp.

22.4 Bốn bài nền và các biến của chúng

Điểm có nằm trong đa giác không. Với đa giác lồi: kiểm bằng vị từ định hướng cho mọi cạnh, hoặc bằng tìm kiếm nhị phân trên góc cho $O(\log n)$. Với đa giác bất kỳ: bắn một tia và đếm số lần cắt biên — lẻ là trong, chẵn là ngoài. Bẫy: tia đi qua đúng một đỉnh sẽ bị đếm hai lần hoặc không lần nào; xử lý điểm trên biên riêng, rồi chỉ xét cạnh có $(y_i > y_p) \neq (y_j > y_p)$ và giao tia về phía phải. Quy ước nửa mở này đếm một đỉnh đúng một lần; yêu cầu hai đầu nghiêm ngặt ở trên/dưới sẽ bỏ sót đỉnh nằm trên tia.

Diện tích đa giác. Công thức dây giày cho diện tích có dấu bằng tổng các tích có hướng chia đôi. Dấu cho biết chiều duyệt, và đó là thông tin hữu ích chứ không phải phiền toái. Bẫy: quên chia 2, và tràn số khi tọa độ lớn.

Hai đoạn thẳng có cắt nhau không. Trường hợp thường thì bốn lần gọi vị từ định hướng là đủ. Bẫy nằm ở trường hợp suy biến: bốn điểm thẳng hàng, khi đó phải kiểm thêm xem hai đoạn có chồng lên nhau không bằng cách so hình chiếu lên hai trục.

Sắp xếp theo góc. Rất nhiều bài cần sắp các điểm quanh một tâm theo góc. Bẫy lớn nhất là dùng hàm arctan — vừa chậm vừa có sai số. Cách đúng là chia các điểm thành nửa trên và nửa dưới rồi so bằng tích có hướng, hoàn toàn bằng số nguyên.

22.5 Vì sao sai số nhỏ lại gây hậu quả lớn

Đây là mục lý giải cho câu hỏi mở đầu thứ tư, và nó là điều phân biệt hình học tính toán với các miền khác.

Trong hầu hết tính toán số, một sai số nhỏ cho một kết quả sai nhỏ — bạn tính ra 3,0001 thay vì 3,0000 và không sao. Trong hình học, các đại lượng số được dùng để đưa ra *quyết định rời rạc*: điểm này nằm trong hay ngoài, ba điểm này thẳng hàng hay không. Một sai số nhỏ ở gần ranh giới làm lật ngược quyết định, và từ một quyết định sai, cấu trúc dữ liệu của thuật toán rơi vào trạng thái không tương ứng với bất kỳ cấu hình hình học hợp lệ nào.

Hậu quả không phải là “kết quả lệch một chút” mà là các sự cố có tính chất khác hẳn: bao lỗi tự cắt chính nó, thuật toán quét đường lặp vô hạn vì thứ tự trên đường quét mâu thuẫn, hoặc chương trình sập vì một hàm sắp xếp nhận được quan hệ so sánh không nhất quán.

Có ba cách đối phó, xếp theo thứ tự nên thử. *Một*: dùng số nguyên — luôn ưu tiên, và trong lập trình thi đấu thì đề bài thường cho tọa độ nguyên chính là vì lý do này. *Hai*: dùng số học chính xác cho riêng các vị từ, tức là chỉ trả giá ở những chỗ quyết định. *Ba*: nếu chấp nhận dung sai, phải định nghĩa rõ bài toán xấp xỉ và quy tắc xử lý suy biến. Phép quay bảo toàn tính thẳng hàng nên không thể loại bỏ ba điểm thẳng hàng; nhiều tọa độ thì thay đổi dữ liệu và có thể thay đổi đáp án. Khi cần dấu chính xác, dùng vị từ thích nghi với kiểm soát sai số[73].

22.6 Vị từ thích nghi: trả giá chính xác đúng chỗ cần

Cách thứ hai ở mục trên — “dùng số học chính xác cho riêng các vị từ” — nghe như một lời khuyên chung chung, nhưng nó có một cài đặt cụ thể đã trở thành chuẩn công nghiệp, và cơ chế của nó đáng hiểu vì nó là một mẫu thiết kế dùng lại được.

Vấn đề với số học chính xác thuần túy là giá: nếu mọi phép định hướng đều tính bằng số hữu tỉ chính xác, chương trình chậm đi hàng chục lần, trong khi hầu hết các lần gọi đều ở xa ranh giới tới mức số thực thường đã cho dấu đúng. Ý tưởng của Shewchuk[73] là *thích nghi*: tính vị từ theo nhiều tầng, tầng đầu rẻ nhất, và mỗi tầng đi kèm một *chặn sai số* tính được ngay trong lúc chạy.

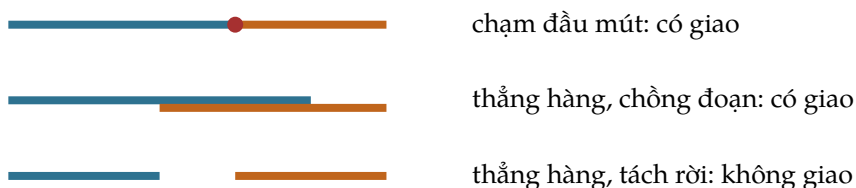
Nếu giá trị ước lượng lớn hơn chặn sai số của nó thì dấu chắc chắn đúng và ta dừng — đây là trường hợp của gần như mọi dữ liệu thật. Chỉ khi giá trị rơi vào vùng không phân biệt được với 0, thuật toán mới leo lên tầng chính xác hơn, và chỉ ở những cấu hình gần suy biến ta mới trả giá đầy đủ.

Kết quả là một hàm vừa *luôn* cho dấu đúng vừa nhanh gần bằng số thực trên dữ liệu bình thường — và đó là lý do các thư viện tam giác phân, lưới và bản đồ đều dựa trên loại vị từ này. Mẫu tư duy chung, đáng mang sang chỗ khác: *tính rẻ trước kèm một chứng chỉ về sai số, và chỉ nâng cấp chi phí khi chứng chỉ không đủ để kết luận*. Cùng khuôn ấy xuất hiện ở bộ lọc trước một phép kiểm đất tiền (B/08) và ở việc cắt nhánh trong quay lui (C/13).

Một hệ quả thực hành đi kèm, thường bị bỏ qua: nếu bạn so sánh bằng ngưỡng ε , quan hệ “bằng nhau” sinh ra *không bắc cầu* — có thể $a \approx b$ và $b \approx c$ mà $a \not\approx c$. Đưa một quan hệ so sánh như vậy cho hàm sắp xếp của thư viện là hành vi không xác định, và triệu chứng của nó là chương trình sập ở một chỗ chẳng liên quan gì tới hình học. Đây chính là loại sự cố mà mục trước gọi là “hậu quả khác hẳn”, và cách phòng duy nhất chắc chắn là đừng để ε lọt vào quan hệ thứ tự.

22.7 Bộ hình suy biến phải thử trước dữ liệu ngẫu nhiên

Hình 22.3 gom ba cấu hình mà phép kiểm chỉ dựa vào dấu nghiêm ngặt thường bỏ sót. Trước khi viết code, quyết định “giao” có tính chạm đầu mút không; hình dưới dùng đoạn đóng, nên hai trường hợp đầu đều có giao.



Hình 22.3. Ba ca kiểm thử cho giao hai đoạn đóng. Hàng giữa dịch nét màu cam xuống một chút để nhìn thấy phần chồng; dữ liệu toán học vẫn có cùng tung độ. Ba cấu hình đều có định hướng bằng 0, nên phải kiểm thêm hình chiếu hoặc điểm nằm trên đoạn.

Kinh nghiệm số học: ép sang kiểu rộng *trước* phép trừ và nhân, không đợi tới khi gán kết quả. Chẳng hạn `wide(B.x) - wide(A.x)` bảo vệ phép trừ; ép kiểu sau `B.x - A.x` không cứu được phép trừ đã tràn. Với dấu của hai định hướng, so dấu trực tiếp thay vì nhân chúng, vì tích hai định thức có thể tràn dù từng định thức còn vừa kiểu số. Thêm các ca điểm trùng nhau, đoạn dài 0 và tọa độ ở hai cực của miền cho phép.

22.8 Gốc rễ: một luận án năm 1978 và cuộc khủng hoảng về độ tin cậy

Hình học tính toán là ngành trẻ so với các chương khác: nó thường được tính là bắt đầu từ luận án tiến sĩ của Michael Shamos năm 1978, quyển đầu tiên coi “các bài toán hình học” như một lĩnh vực có phương pháp riêng thay vì một tập hợp bài lẻ. Trước đó, các thuật toán hình học tồn tại rải rác — thuật toán bao lồi của Graham ra đời năm 1972, và động cơ của nó rất thực tế: xử lý dữ liệu thống kê, nơi bao lồi được dùng để tìm các điểm cực biên.

Kỹ thuật quét đường được hệ thống hoá cuối thập niên 1970, với thuật toán tìm giao điểm các đoạn thẳng của Bentley và Ottmann năm 1979 — cùng Bentley của *Programming Pearls* ở chương

A/02. Cùng thời kỳ, các cấu trúc chia không gian như cây k -d và cây khoảng ra đời để trả lời truy vấn hình học, và chúng chính là các cây tìm kiếm được tăng cường mà **B/09** đã nói.

Nhưng câu chuyện đáng nhớ nhất của ngành này là câu chuyện về *độ tin cậy số học*, và nó đáng kể vì nó là một trường hợp hiếm hoi mà cả một cộng đồng phải thừa nhận rằng lý thuyết đã bỏ sót một điều căn bản. Suốt thập niên 1980, các thuật toán hình học được công bố với chứng minh đầy đủ, nhưng khi cài đặt bằng số thực thì chúng sập hoặc cho kết quả vô nghĩa trên dữ liệu thật. Vấn đề không phải lỗi lập trình mà là sự lệch pha giữa mô hình — “số thực chính xác vô hạn” — và thực tế — “số dấu phẩy động 64 bit”.

Phản ứng của ngành trong thập niên 1990 là xây dựng *tính toán vị từ chính xác*: các kỹ thuật tính dấu của biểu thức hình học một cách đảm bảo, thường bằng cách ước lượng trước với số dấu phẩy động rồi chỉ chuyển sang số học chính xác khi kết quả gần 0. Ý tưởng “chỉ trả giá cho độ chính xác ở nơi thật sự cần” này về sau thành chuẩn, và nó được đóng gói trong các thư viện hình học công nghiệp. Bài học cho người học: khi ai đó nói “thuật toán này đã được chứng minh”, hãy hỏi thêm “chứng minh trong mô hình số nào”.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1972 — Graham	Tìm các điểm cực biên trong dữ liệu thống kê.	Thuật toán bao lồi bằng sắp xếp theo góc và quét bằng ngăn xếp.
1978 — Shamos	Các bài toán hình học được giải lẻ tẻ, không có phương pháp chung.	Luận án khai sinh hình học tính toán như một lĩnh vực với các kỹ thuật chuẩn.
1979 — Bentley & Ottmann	Tìm mọi giao điểm của n đoạn thẳng nhanh hơn $O(n^2)$.	Quét đường: biến bài hai chiều thành chuỗi sự kiện một chiều.
Thập niên 1980 — cấu trúc truy vấn hình học	Trả lời nhiều truy vấn trên cùng tập điểm.	Cây k -d, cây khoảng, cây phân đoạn — đều là cây tìm kiếm được tăng cường (B/09 , B/10).
Thập niên 1990 — khủng hoảng độ tin cậy	Thuật toán đã chứng minh vẫn sập khi cài bằng số thực.	Nhận ra khoảng cách giữa mô hình số thực lý tưởng và số dấu phẩy động; ra đời kỹ thuật vị từ chính xác thích ứng.
Thập niên 1990–2000 — thư viện hình học	Mỗi nhóm tự cài lại và tự mắc lại cùng các lỗi.	Các thư viện đóng gói vị từ chính xác và xử lý suy biến, cho phép dùng hình học tính toán trong sản phẩm thật.

I 22.9 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Đồ hoạ máy tính	Cắt đa giác, kiểm tra va chạm, dò tia	Cùng vị từ nền tảng. Khác về ưu tiên: đồ hoạ chấp nhận sai số nhỏ để đổi lấy tốc độ khung hình, còn hình học tính toán thường cần tính đúng tuyệt đối — hai cộng đồng cùng thuật toán, khác tiêu chuẩn.
Hệ thông tin địa lý	Chồng bản đồ, truy vấn vùng, tìm điểm gần nhất	Quét đường và cây phân vùng không gian là công cụ nền. Ở đây dữ liệu suy biến rất phổ biến (đường biên chung, điểm trùng), nên xử lý suy biến quan trọng hơn cả tốc độ.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Robot và SLAM	Kiểm tra va chạm, bản đồ không gian tự do, khớp đám mây điểm	Bao lồi dùng để mô tả vùng chiếm chỗ của vật thể; cây k -d dùng để tìm điểm gần nhất trong bước khớp đám mây. Điểm khác: dữ liệu cảm biến có nhiều nên các vị từ chính xác ít giá trị hơn — cái cần là bền vững với nhiễu, không phải chính xác tuyệt đối.
Thiết kế và chế tạo	CAD, sinh lưới, in ba chiều	Sinh lưới tam giác dựa trên tam giác phân Delaunay; các lỗi suy biến ở đây gây hậu quả vật lý thật, nên ngành này là nơi vị từ chính xác được coi trọng nhất.
Tối ưu hoá	Bao lồi trong quy hoạch tuyến tính	Miền khả thi của quy hoạch tuyến tính là một đa diện lồi, và thuật toán đơn hình đi trên các đỉnh của nó. Bao lồi trong chương này là bản hai chiều của cùng đối tượng.
Quy hoạch động	Kỹ thuật bao lồi để tăng tốc chuyển trạng thái	Tập các đường thẳng cần lấy cực trị tạo thành một bao lồi; kỹ thuật ở C/16 chính là bao lồi động. Một liên hệ đẹp giữa hình học và tối ưu tổ hợp.
Học máy	Máy vector hỗ trợ, phân tách tuyến tính	Việc tìm siêu phẳng phân tách hai lớp có liên hệ trực tiếp với khoảng cách giữa hai bao lồi của hai lớp — hình học lồi là ngôn ngữ chung.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì với số thực, “bằng 0” gần như không bao giờ xảy ra đúng nghĩa: kết quả tính tích có hướng của ba điểm thẳng hàng có thể ra 10^{-16} thay vì 0 do sai số làm tròn. Bạn buộc phải so với một ngưỡng, mà ngưỡng đó không có giá trị đúng phổ quát — nó phụ thuộc độ lớn của tọa độ và số phép toán đã thực hiện. Tệ hơn, quan hệ “gần bằng” không bắc cầu, nên nó không dùng được ở những nơi cần một quan hệ so sánh nhất quán.
- A2. Vì nó dễ cài đúng và ít trường hợp riêng nhất. Sau khi sắp xếp, thuật toán chỉ là một vòng quét với ngăn xếp và một phép kiểm tra vị từ định hướng — ít chỗ để sai. Các thuật toán khác như gói quà có thể nhanh hơn khi bao có ít đỉnh, nhưng chúng nhạy cảm hơn với các điểm thẳng hàng và trùng nhau. Trong hình học, tiêu chí “ít trường hợp riêng” đáng giá hơn một chút khác biệt về tiệm cận.
- A3. Cơ chế là: tại mỗi thời điểm, chỉ các đối tượng *kề nhau* theo thứ tự dọc trên đường quét mới có thể tương tác. Hai đoạn thẳng chỉ cắt nhau nếu có lúc chúng *kề nhau* trong thứ tự đó. Nhờ vậy, thay vì n^2 cặp, ta chỉ cần xử lý các sự kiện làm thay đổi quan hệ *kề*: một đối tượng vào, một đối tượng ra, hai đối tượng đổi chỗ. Bài toán hai chiều trở thành bài toán duy trì một tập có thứ tự trong một chiều, tức là quay về cấu trúc dữ liệu của B/09.
- A4. Vì các đại lượng số ở đây được dùng để ra *quyết định rời rạc*, không phải để báo cáo giá trị. Sai số nhỏ ở gần ranh giới lật ngược quyết định, và một quyết định sai đưa cấu trúc dữ liệu vào trạng thái không tương ứng với cấu hình hình học hợp lệ nào — ví dụ thứ tự trên đường quét mâu thuẫn với chính nó. Từ đó hậu quả không phải sai số nhỏ mà là bao lồi tự cắt, vòng lặp không dừng, hoặc chương trình sập.
- A5. Luôn nên dùng số nguyên khi đề cho tọa độ nguyên — và đề thi thường cố ý cho như vậy chính vì lý do này. Với tọa độ tới 10^9 , tích có hướng có thể tới $4 \cdot 10^{18}$, vẫn nằm trong số nguyên 64 bit có dấu nhưng rất sát; nếu công thức của bạn nhân thêm một lần nữa hoặc cộng nhiều số hạng thì phải chuyển sang kiểu 128 bit. Mẹo giảm rủi ro: tính tiến toàn bộ tập điểm về gần gốc tọa độ trước khi tính, vì tích có hướng chỉ phụ thuộc hiệu các tọa độ.

NÓI LẠI THẬT ĐƠN GIẢN

Hình học trên máy tính có một điều lạ: bài toán thì trực quan, nhưng cái làm hỏng chương trình lại không nằm ở chỗ bạn nghĩ.

Gần như mọi thuật toán trong chương này đứng trên một câu hỏi duy nhất: đi từ điểm A tới B rồi tới C thì bạn rẽ trái hay rẽ phải? Có một công thức nhỏ trả lời câu đó bằng một phép nhân và một

phép trừ. Bao lỗi, kiểm tra hai đoạn cắt nhau, kiểm tra điểm nằm trong hình — tất cả chỉ là cách xâu chuỗi câu hỏi ấy lại.

Rắc rối bắt đầu khi ba điểm nằm gần như thẳng hàng. Lúc đó câu trả lời đúng là “không rẽ bên nào cả”, nhưng máy tính dùng số thực có sai số, nên nó có thể nói “rẽ trái một chút” hoặc “rẽ phải một chút” tùy vào bụi bặm của phép làm tròn. Và vì thuật toán dựa trên câu trả lời đó để quyết định, một câu trả lời sai không làm kết quả lệch chút xíu — nó làm chương trình tin vào một thế giới không tồn tại, kiểu như tin rằng A ở bên trái B và B cũng ở bên trái A. Từ đó thì bất cứ chuyện gì cũng có thể xảy ra: hình vẽ tự cắt chính nó, vòng lặp chạy mãi, hoặc chương trình sập.

Cách phòng đơn giản nhất và hiệu quả nhất: nếu toạ độ là số nguyên thì hãy giữ nguyên số nguyên từ đầu tới cuối, đừng đổi sang số thực. Phép nhân và trừ trên số nguyên cho kết quả chính xác tuyệt đối, nên câu hỏi “rẽ bên nào” luôn được trả lời đúng. Chỉ cần canh chừng một chuyện: số nguyên cũng có giới hạn, và với toạ độ cỡ một tỉ thì tích của chúng đã sát trần.

Một câu: Mọi thuật toán hình học đứng trên một câu hỏi “rẽ trái hay rẽ phải” — và tính đúng của cả chương trình phụ thuộc vào việc trả lời câu đó cho chính xác.

Chỗ không rút gọn được: Không có giá trị ε đúng phổ quát; việc chọn nó luôn là thỏa hiệp phụ thuộc dữ liệu cụ thể.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vị từ định hướng	Dấu của $(B - A) \times (C - A)$: dương là rẽ trái, âm là rẽ phải, 0 là thẳng hàng. Nền của gần như mọi thuật toán hình học.
••• Quy tắc số học	Dùng số nguyên bất cứ khi nào có thể; toạ độ tới 10^9 cho tích có hướng tới $4 \cdot 10^{18}$ — sát trần 64 bit.
••• Vì sao sai số nhỏ gây hậu quả lớn	Số được dùng để ra quyết định rồi rắc; quyết định sai đưa cấu trúc vào trạng thái không tương ứng cấu hình hợp lệ nào $\Rightarrow$ tự cắt, lặp vô hạn, sập.
••• Bao lỗi bằng quét đơn điệu	Sắp theo hoành độ rồi quét hai nửa với ngăn xếp đơn điệu; $O(n \log n)$, chi phối bởi sắp xếp; ít trường hợp riêng nhất.
••• Cơ chế của quét đường	Chỉ các đối tượng <i>kẽ nhau</i> theo thứ tự dọc mới tương tác; xử lý sự kiện vào/ra/đổi chỗ thay vì mọi cặp.
•• So sánh số thực	Không bao giờ dùng $==$; dùng ngưỡng ε nhưng nhớ rằng “gần bằng” <i>không bắc cầu</i> nên không dùng làm quan hệ sắp xếp.
•• Điểm trong đa giác	Lỗi: kiểm vị từ mọi cạnh, hoặc nhị phân theo góc $O(\log n)$. Bất kỳ: bắn tia đếm số lần cắt, lẻ là trong; bẫy là tia đi qua đỉnh.
•• Diện tích đa giác	Công thức dây giày: tổng tích có hướng chia đôi; dấu cho biết chiều duyệt.
•• Sắp xếp theo góc	Chia nửa trên / nửa dưới rồi so bằng tích có hướng — hoàn toàn số nguyên; đừng dùng arctan.
•• Cặp điểm xa nhất	Luôn nằm trên bao lỗi; tìm bằng hai con trỏ quay quanh bao sau khi có bao.
•• Ba cách đối phó sai số	Số nguyên (ưu tiên); vị từ chính xác thích ứng (chỉ trả giá ở chỗ quyết định); hoặc định nghĩa dung sai và xử lý suy biến rõ ràng. Phép quay không loại tính thẳng hàng.
• Liên hệ với quy hoạch động	Kỹ thuật bao lỗi tăng tốc chuyển trạng thái (C/16) chính là bao lỗi động trên tập đường thẳng.
•• Vị từ thích nghi	Tính rẽ trước kèm một chặn sai số tính ngay lúc chạy; chỉ leo lên tầng chính xác hơn khi giá trị không phân biệt được với 0. Kèm theo: so sánh bằng ε không bắc cầu, đừng đưa vào hàm sắp xếp.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên lưu điểm bằng kiểu gì trong bài hình học?

| Nếu đề cho toạ độ nguyên, hãy dùng số nguyên 64 bit và giữ nguyên suốt — kể cả khi công thức trung gian trông có vẻ cần chia. Rất nhiều bài tránh được phép chia bằng cách nhân chéo thay vì so hai phân số. Chỉ chuyển sang số thực ở bước cuối cùng khi phải in ra kết quả có phần thập phân. Nếu đề cho toạ độ thực, hãy cân nhắc nhân tất cả lên một hệ số rồi làm tròn — nhiều bài cho toạ độ với số chữ số thập phân cố định chính là gợi ý làm điều đó.

2. Bao lỗi của tôi sai khi có nhiều điểm thẳng hàng. Xử lý thế nào?

| Vấn đề nằm ở chỗ bạn dùng dấu “lớn hơn” hay “lớn hơn hoặc bằng” khi loại điểm khỏi ngăn xếp, và câu trả lời phụ thuộc vào việc đề muốn gì: giữ mọi điểm trên cạnh của bao, hay chỉ giữ các đỉnh thật sự. Hãy đọc kỹ đề, chọn một quy ước, và kiểm bằng một ví dụ có ba điểm thẳng hàng trên biên. Ngoài ra nhớ loại các điểm trùng nhau trước khi chạy, vì chúng gây ra tích có hướng bằng 0 ở những chỗ không mong muốn.

3. Có nên tự cài các thuật toán hình học phức tạp trong thi đấu không?

| Với bao lỗi, giao đoạn thẳng, diện tích, sắp xếp theo góc thì có — chúng ngắn và bạn nên có sẵn trong thư viện riêng. Với tam giác phân Delaunay, giao nửa mặt phẳng, hay bao lỗi ba chiều thì hãy cân nhắc: chúng dài, nhiều trường hợp suy biến, và thường có cách vòng qua. Trước khi cài một thuật toán hình học nặng, hãy dành năm phút hỏi “bài này có cách nào không cần hình học không” — nhiều bài trông hình học lại giải được bằng sắp xếp cộng một quan sát.

4. Làm sao kiểm thử một chương trình hình học?

| Ba lớp dữ liệu, và lớp thứ ba là quan trọng nhất. Một: dữ liệu ngẫu nhiên toạ độ lớn — bắt lỗi tràn. Hai: dữ liệu ngẫu nhiên toạ độ *nhỏ*, ví dụ trong $[-3, 3]$ — điều này tạo ra rất nhiều điểm thẳng hàng và trùng nhau, tức là ép trường hợp suy biến xuất hiện. Ba: dữ liệu dựng tay cho từng trường hợp suy biến bạn nghĩ ra được. Lớp thứ hai là mẹo hiệu quả nhất và bị bỏ qua nhiều nhất — toạ độ nhỏ khiến suy biến xảy ra liên tục thay vì hiếm.

5. Vì sao đề bài thi thường cho toạ độ nguyên?

| Chính vì lý do ở mục 6: với toạ độ nguyên, mọi vị từ tính được chính xác tuyệt đối, nên bài toán có một đáp án xác định và việc chấm không phụ thuộc vào cách cài số thực của thí sinh. Nếu bạn thấy đề cho toạ độ thực với số chữ số thập phân giới hạn, gần như chắc chắn ý định là bạn nhân lên thành số nguyên. Đây là một gợi ý đọc đề mà nhiều người bỏ qua.

6. Cây k -d và bao lỗi khác nhau về vai trò thế nào?

| Bao lỗi là một *kết quả* — nó mô tả hình dạng bao ngoài của tập điểm. Cây k -d là một *cấu trúc tra cứu* — nó tổ chức tập điểm để trả lời nhiều truy vấn về lân cận. Chúng trả lời hai loại câu hỏi khác nhau: “hình bao của tập này là gì” so với “điểm nào gần điểm truy vấn nhất”. Trong ứng dụng robot, bao lỗi hay dùng để mô tả vật cản, còn cây k -d dùng để khớp đám mây điểm.

7. Xoay hệ toạ độ một góc ngẫu nhiên có thực sự giúp gì không?

| Có, và nó là một mẹo thực dụng đáng biết. Rất nhiều trường hợp suy biến trong quét đường liên quan tới các đoạn thẳng đứng hoặc các điểm cùng hoành độ. Xoay toàn bộ tập điểm một góc ngẫu nhiên nhỏ làm xác suất còn tồn tại các trường hợp đó gần bằng 0, nên bạn không phải viết mã xử lý riêng cho chúng. Cái giá: bạn buộc phải chuyển sang số thực, nên đây là đánh đổi giữa hai loại rủi ro — và nó chỉ đáng khi mã xử lý suy biến phức tạp hơn nhiều so với rủi ro sai số.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Vị từ định hướng và kiểm tra hai đoạn thẳng cắt nhau	CSES “Line Segment Intersection”[33]	Xử lý đủ trường hợp thẳng hàng chồng nhau; kiểm bằng toạ độ nhỏ.
● Diện tích đa giác bằng công thức dây giày	CSES “Polygon Area”[33]	Giữ nguyên số nguyên, in ra diện tích nhân đôi để tránh số thực.

Bài	Nguồn	Điều cần nhận ra (tiếp)
• Điểm nằm trong, trên biên hay ngoài đa giác	CSES “Point in Polygon”[33]	Bắn tia; xử lý trường hợp điểm nằm đúng trên biên trước.
•• Bao lồi	CSES “Convex Hull”[33]	Quét đơn điệu; quyết định rõ quy ước về điểm thẳng hàng trên biên.
•• Cặp điểm gần nhất	CSES “Minimum Euclidean Distance”[33]	Chia để trị (C/14) hoặc quét đường với tập có thứ tự; cài cả hai nếu có thời gian.
•• Số điểm nguyên trên biên và bên trong đa giác	CSES “Polygon Lattice Points”[33]	Định lý Pick cộng gcd cho số điểm trên cạnh — một liên hệ đẹp với D/21 .
••• Tìm mọi giao điểm của n đoạn thẳng	Bài kinh điển	Quét đường với tập có thứ tự; đây là bài kiểm tra khả năng xử lý suy biến của bạn.
••• Hợp diện tích của n hình chữ nhật	Bài kinh điển	Quét đường cộng cây phân đoạn (B/10); một trong những bài ghép hai chương đẹp nhất.
••• Cặp điểm xa nhất trong tập	Bài kinh điển	Bao lồi rồi hai con trỏ quay quanh bao; chứng minh vì sao đáp án luôn nằm trên bao.
••• Hình chữ nhật nhỏ nhất bao quanh một tập điểm	Bài kinh điển	Một cạnh của hình chữ nhật tối ưu luôn trùng một cạnh của bao lồi — hãy tự chứng minh trước.
••• Kiểm thử bao lồi bằng dữ liệu toạ độ nhỏ	Bài tập kiểm thử	Sinh ngẫu nhiên toạ độ trong $[-3, 3]$ và so với vết cạn; mẹo hiệu quả nhất của chương.

Kết chương và đọc tiếp

Hình học là miền mà thuật toán đơn giản còn cài đặt thì không, và lý do nằm ở chỗ số học được dùng để ra quyết định rời rạc. Ba điều đáng mang theo: vị từ định hướng là viên gạch của mọi thứ; số nguyên là bạn còn số thực là nguồn rủi ro; và mẹo kiểm thử bằng toạ độ nhỏ để ép trường hợp suy biến xuất hiện.

Hai chương đầu của phần D là hai miền chuyên. Chương tiếp theo là chương quan trọng nhất của cả phần, và nó thay đổi cách bạn phân bổ thời gian khi gặp bài lạ: cách nhận ra một bài toán *không* có lời giải nhanh nào được biết tới, vì sao thông tin đó đáng giá, và bốn cách sống chung với nó ([D/23](#)).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates”	Discrete & Computational Geometry, 1997	Nguồn của vị từ thích nghi. Đọc phần mở đầu và phần chặn sai số; mã nguồn kèm theo dùng được trực tiếp.
nền tảng	de Berg, Cheong, van Kreveld, Overmars, <i>Computational Geometry: Algorithms and Applications</i>	Springer, tái bản 2008	Sách trực chính của chương. Đọc chương bao lồi và chương quét đường; phần suy biến nằm rải rác trong các mục “degeneracies”.
tra khi cần	Gajentaan & Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry”	Computational Geometry: Theory and Applications, 1995	Vì sao nhiều bài hình học phẳng đứng yên ở n^2 : chúng khó ngang bài 3SUM (D/24).
tra khi cần	Knuth, <i>The Art of Computer Programming</i> , tập 1	Addison-Wesley, tái bản 1997	Tra khi cần cơ sở về số học máy tính và biểu diễn dấu phẩy động.

Độ khó và quy dẫn (Hardness and Reductions)

PHỤ THUỘC C/13 cho vết cạo có cắt tia — công cụ chính khi gặp bài khó; C/15 cho xấp xỉ tham lam; C/16 cho khái niệm giả đa thức.

TÓM TẮT

Chương này thay đổi cách bạn phân bổ thời gian khi gặp bài lạ. Nó định nghĩa khái niệm P và NP theo hướng thực dụng — điểm mấu chốt là sự khác biệt giữa *tìm* lời giải và *kiểm* một lời giải cho sẵn — rồi dạy kỹ năng nhận ra một bài toán là NP-khó bằng cách quy dẫn từ một bài trong danh mục. Nửa sau là phần quan trọng hơn với người làm thực tế: bốn cách sống chung với độ khó, gồm thuật toán mũ thông minh, xấp xỉ có bảo đảm, tham số hoá, và bộ giải công nghiệp. Nếu chỉ nhớ một điều: biết một bài là NP-khó không phải thất bại mà là thông tin đắt giá nhất bạn có thể có — nó ngăn bạn mất ba ngày tìm một lời giải đa thức không tồn tại, và chuyển câu hỏi từ “giải thế nào” sang “chấp nhận đánh đổi nào”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán quyết định** — bài toán có câu trả lời đúng/sai, ví dụ “đồ thị này có tập độc lập kích thước k không”. Lý thuyết độ phức tạp phát biểu qua dạng này cho gọn.
- **lớp P** — các bài toán quyết định giải được trong thời gian đa thức theo kích thước dữ liệu vào.
- **lớp NP** — các bài toán mà một lời giải, nếu được cho sẵn, *kiểm chứng* được trong thời gian đa thức. Chú ý: NP không có nghĩa là “không đa thức”.
- **quy dẫn** — biến đổi bài toán A thành bài toán B trong thời gian đa thức, sao cho lời giải của B cho ngay lời giải của A. Ý nghĩa: B khó ít nhất bằng A.
- **NP-đầy đủ** — thuộc NP và mọi bài trong NP đều quy dẫn về được; đây là các bài “khó nhất trong NP”.
- **NP-khó** — khó ít nhất bằng mọi bài trong NP, nhưng không nhất thiết thuộc NP — ví dụ các bài tối ưu tương ứng.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao lý thuyết lại phát biểu mọi thứ qua bài toán quyết định, trong khi thực tế ta luôn muốn bài toán tối ưu?
- Q2. Bài ba lô có lời giải quy hoạch động $O(nW)$. Vì sao điều đó không mâu thuẫn với việc nó là NP-khó?
- Q3. Khi chứng minh bài của bạn là NP-khó, quy dẫn phải đi theo chiều nào — và vì sao chiều ngược lại là một lỗi vô nghĩa?
- Q4. Nếu một bài là NP-khó, vì sao trong công nghiệp người ta vẫn giải được những trường hợp hàng triệu biến?
- Q5. Biết một bài là NP-khó thì bạn thay đổi cụ thể điều gì trong cách làm việc?

Có một tình huống mà mọi người làm thuật toán đều gặp: bạn dành nhiều giờ tìm một thuật toán đa thức cho một bài, thử hết mọi hướng ở phần C, và không ra. Câu hỏi tự nhiên là “mình

còn thiếu ý tưởng gì?”. Nhưng có một khả năng khác mà người mới thường không nghĩ tới: *có thể không ai biết cách giải nhanh bài này*, và việc bạn không nghĩ ra không phải do bạn kém.

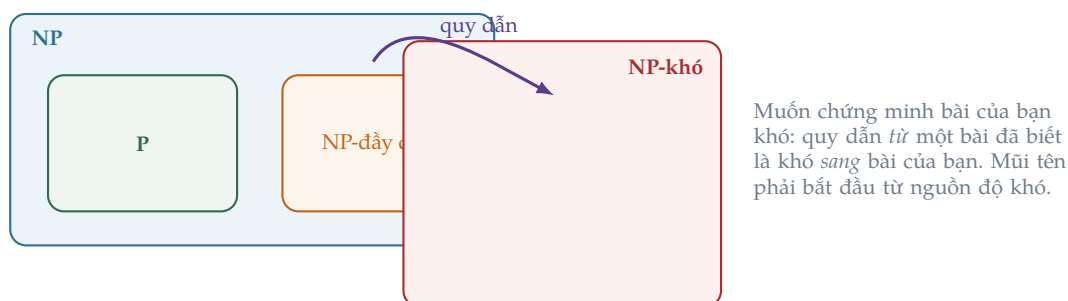
Chương này dạy cách phân biệt hai tình huống đó. Đây là kỹ năng có giá trị thực dụng rất cao — không phải vì nó giúp bạn giải thêm bài, mà vì nó giúp bạn *ngừng đúng lúc* và chuyển sang câu hỏi đúng: nếu không có lời giải nhanh chính xác, tôi sẵn sàng đánh đổi cái gì — tính tối ưu, tính tổng quát, hay thời gian chạy?

23.1 Tìm và kiểm: sự phân biệt sinh ra cả lý thuyết

Xét bài toán: cho một đồ thị và số k , có tồn tại một tập k đỉnh đôi một không kề nhau không? Tìm ra tập đó thì khó — không ai biết cách nào tốt hơn về cơ bản so với thử các tập con. Nhưng nếu ai đó *đưa* cho bạn một tập và bảo “đây là đáp án”, bạn kiểm tra trong vài giây: đếm số đỉnh, kiểm mọi cặp.

Đó chính là định nghĩa của lớp NP: các bài toán mà một lời giải, nếu được cho sẵn, kiểm chứng được nhanh. Lớp P là các bài mà bản thân việc *tìm* cũng nhanh. Rõ ràng P nằm trong NP. Câu hỏi P có bằng NP không — tức là “kiểm được nhanh thì có tìm được nhanh không” — là câu hỏi mở nổi tiếng nhất của ngành, và phần lớn giới nghiên cứu tin rằng câu trả lời là không.

Về lý do lý thuyết dùng bài toán quyết định thay vì bài toán tối ưu: đó là để có một khái niệm “kiểm chứng” sạch sẽ. Với bài quyết định, chứng cứ cho câu trả lời “có” là một cấu hình cụ thể. Với bài tối ưu, để kiểm “đây là giá trị tối ưu” bạn cần chứng minh không có gì tốt hơn — một việc khó hơn hẳn. May mắn là hai dạng liên hệ chặt với nhau qua đúng kỹ thuật ở chương C/19: giải bài tối ưu bằng cách nhị phân trên đáp án và gọi bài quyết định. Vì vậy phát biểu lý thuyết qua bài quyết định không làm mất tính tổng quát — đó là câu trả lời cho câu hỏi mở đầu thứ nhất.



Hình 23.1. Bản đồ các lớp. P nằm trong NP (tin là thực sự nhỏ hơn, nhưng chưa chứng minh được). NP-dễ đủ là các bài khó nhất *trong* NP; NP-khó rộng hơn và chứa cả những bài không thuộc NP, ví dụ các bài tối ưu tương ứng. Mũi tên tím nhắc lại chiều đúng của quy dẫn — lỗi phổ biến nhất khi lần đầu viết chứng minh độ khó.

23.2 Quy dẫn: công cụ để so độ khó

Ý tưởng trung tâm của chương: nếu tôi biến được bài A thành bài B một cách nhanh chóng, thì B *khó ít nhất bằng* A. Vì nếu B có lời giải nhanh, ghép với phép biến đổi ta được lời giải nhanh cho A.

Chiều của lập luận là chỗ mà gần như mọi người mới nhầm. Muốn chứng minh *bài của bạn khó*, bạn phải quy dẫn từ một bài đã biết là khó sang bài của bạn — tức là chỉ ra rằng bài của bạn đủ mạnh để mô phỏng bài khó kia. Quy dẫn theo chiều ngược lại — biến bài của bạn thành một bài khó đã biết — không chứng minh gì cả, vì ai cũng biến được mọi thứ thành một bài đủ tổng quát.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và cũng là lỗi phổ biến nhất khi lần đầu viết một chứng minh độ khó.

Cấu trúc của một quy dẫn gồm ba phần: mô tả phép biến đổi từ dữ liệu của A thành dữ liệu của B; chứng minh phép biến đổi chạy trong thời gian đa thức; và chứng minh *cả hai chiều* của tương đương — A có nghiệm thì B có nghiệm, và ngược lại. Phần cuối là phần dễ bị làm ẩu nhất, và nó cũng là phần chứa nội dung thật.

Kết quả nền của cả lý thuyết là định lý Cook–Levin: bài toán thoả mãn công thức lôgic (SAT) là NP-đầy đủ. Ý nghĩa: nó là bài toán “khó nhất trong NP”, và từ nó có thể quy dẫn tới mọi bài NP khác. Sau đó Karp chứng minh 21 bài toán tổ hợp quen thuộc cũng NP-đầy đủ, biến một kết quả lôgic trừu tượng thành một danh mục dùng được.

23.3 Danh mục cần thuộc

Trong thực hành, bạn không chứng minh NP-khó từ đầu mà quy dẫn từ một bài trong danh mục. Bảng dưới là những bài hay dùng nhất, kèm dạng của chúng — vì nhận ra “bài của tôi có dạng giống bài này” là kỹ năng thật sự cần.

Bài toán	Dạng	Hay dùng để quy dẫn khi
3-SAT	Gán đúng/sai cho biến thoả mọi mệnh đề	Bài có cấu trúc “chọn một trong hai” kèm ràng buộc
Tập độc lập / Clique	Chọn nhiều đỉnh đôi một không kề / đôi một kề	Bài chọn nhiều đối tượng không xung khắc
Phủ đỉnh	Chọn ít đỉnh chạm mọi cạnh	Bài “chọn ít nhất để bao phủ”
Tô màu đồ thị	Gán k màu, hai đỉnh kề khác màu	Bài phân nhóm có ràng buộc xung khắc
Chu trình Hamilton / TSP	Đi qua mọi đỉnh đúng một lần	Bài về thứ tự đi qua toàn bộ
Ba lô / Phân hoạch	Chọn tập con đạt tổng mục tiêu	Bài có ràng buộc tổng số
Phủ tập hợp	Chọn ít tập nhất phủ hết	Bài phân bổ tài nguyên bao phủ
Đóng thùng	Xếp đồ vào ít thùng nhất	Bài lập lịch, phân bổ dung lượng

Một lưu ý quan trọng và hay bị bỏ: *nhiều bài NP-khó trở nên dễ trên lớp dữ liệu hẹp hơn*. Phủ đỉnh là NP-khó trên đồ thị tổng quát nhưng đa thức trên đồ thị hai phía (định lý König, C/18) và trên cây. Tô màu là NP-khó với ba màu nhưng đa thức với hai màu (kiểm tra đồ thị hai phía). Tập độc lập là NP-khó nói chung nhưng đa thức trên cây bằng quy hoạch động. Vì vậy khi gặp một bài trông giống bài NP-khó, câu hỏi tiếp theo phải là: *cấu trúc dữ liệu của tôi có đặc biệt không?*

23.4 Bốn cách sống chung

Đây là mục thực dụng nhất chương. Khi đã xác định bài là NP-khó, bạn có bốn hướng, và việc chọn hướng nào phụ thuộc vào cái bạn sẵn sàng từ bỏ.

Hướng 1 — giữ tính chính xác, từ bỏ tính đa thức. Dùng thuật toán mũ nhưng thông minh: quy hoạch động trên mặt nạ bit cho $O(2^n n^2)$ thay vì $O(n!)$ (C/16); nhánh cận với cận chặt (C/13); gấp nhau ở giữa cho $2^{n/2}$. Hướng này đúng khi n nhỏ, và câu hỏi trở thành “hàm mũ với cơ số bao nhiêu”.

Hướng 2 — giữ tính đa thức, từ bỏ tính tối ưu. Thuật toán xấp xỉ có bảo đảm: tham lam cho phủ tập hợp với hệ số $\ln n$, thuật toán 2-xấp xỉ cho phủ đỉnh, bảo đảm $1 + 1/e$ cho hàm dưới mô-đun (C/15). Điểm quan trọng: đây là bảo đảm chứng minh được cho mọi dữ liệu, khác hẳn “thực nghiệm thấy tốt”.

Hướng 3 — giữ cả hai, từ bỏ tính tổng quát theo một tham số. Nhiều bài có lời giải $O(f(k) \cdot n^c)$ với k là một tham số nhỏ — kích thước lời giải, độ rộng cây của đồ thị, số ràng buộc vi phạm. Nếu k nhỏ trong dữ liệu thật thì thuật toán chạy nhanh dù bài là NP-khó. Hướng này gọi là tham số hoá, và nó rất hợp với thực tế vì dữ liệu thật hiếm khi là trường hợp xấu nhất.

Hướng 4 — giao cho bộ giải công nghiệp. Mã hoá bài toán thành SAT, quy hoạch nguyên, hoặc một bài toán ràng buộc, rồi dùng bộ giải sẵn có. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: các bộ giải hiện đại xử lý được những trường hợp hàng triệu biến vì dữ liệu công nghiệp có rất nhiều cấu trúc ẩn, và bộ giải khai thác chúng bằng lan truyền ràng buộc và học từ xung đột (C/13). Chúng không phá vỡ độ khó lý thuyết; chúng chỉ giải tránh trường hợp xấu.

CẦN NHỚ

Điều quý nhất mà chương này cho bạn không phải khả năng chứng minh NP-khó mà là khả năng *nhận ra sớm*. Khi một bài trông giống một mục trong danh mục ở mục 3, hãy dừng lại năm phút để kiểm tra thay vì tìm tiếp lời giải đa thức. Nếu đúng là NP-khó, câu hỏi lập tức đổi từ “giải thế nào” sang “ n bao nhiêu, tôi cần chính xác hay gần đúng, và dữ liệu có cấu trúc đặc biệt gì”.

23.5 Ngoài NP: hai điều nên biết là có

Có những bài toán không giải được, chấm hết. Bài toán dừng — cho một chương trình và dữ liệu vào, hỏi nó có dừng không — không có thuật toán nào giải được, và điều này đã được chứng minh chứ không phải chưa ai tìm ra. Đây là kết quả của Turing năm 1936, và nó có hệ quả thực dụng: nhiều câu hỏi về phân tích chương trình — “đoạn mã này có bao giờ chạy không”, “hai chương trình này có tương đương không” — là không giải được trong trường hợp tổng quát. Đó là lý do các công cụ phân tích tĩnh luôn hoặc bỏ sót hoặc báo động giả; sự không hoàn hảo đó là bắt buộc về mặt toán học, không phải do công cụ kém.

Có những bài khó hơn NP. Các bài về trò chơi hai người thường thuộc lớp cao hơn, vì chúng có cấu trúc “tồn tại nước đi sao cho với mọi phản ứng của đối thủ...” lồng nhau nhiều tầng. Đây là lý do các trò chơi tổng quát hoá — cờ vây trên bàn $n \times n$ chẳng hạn — khó hơn hẳn các bài tối ưu thông thường.

23.6 Gốc rễ: từ câu hỏi của Hilbert tới danh mục của Karp

Dòng suy nghĩ dẫn tới chương này bắt đầu từ toán học thuần túy. Đầu thế kỷ 20, Hilbert đặt ra bài toán quyết định: có tồn tại một thủ tục cơ học để xác định một mệnh đề toán học là đúng hay sai không? Năm 1936, Turing trả lời là không — và để trả lời được, ông phải *định nghĩa* thế nào là “thủ tục cơ học”, tạo ra máy Turing. Nghĩa là khái niệm thuật toán được hình thức hoá không phải để chế tạo máy tính mà để chứng minh một giới hạn.

Bước tiếp theo là phân biệt giữa “giải được” và “giải được trong thời gian chấp nhận”. Ý tưởng lấy *thời gian đa thức* làm ranh giới của cái “khả thi” được Edmonds phát biểu rõ vào năm 1965, khi ông trình bày một thuật toán ghép đôi và nhấn mạnh rằng nó chạy đa thức chứ không chỉ hữu hạn. Đây là một lựa chọn quy ước có tranh cãi — một thuật toán $O(n^{100})$ là đa thức nhưng

vô dụng — nhưng nó đã tỏ ra rất hữu ích trong thực tế, vì các thuật toán đa thức tự nhiên hầu như luôn có số mũ nhỏ.

Năm 1971, Cook chứng minh rằng SAT là bài khó nhất trong NP; Levin chứng minh kết quả tương đương một cách độc lập ở Liên Xô. Nhưng chính bài báo của Karp năm 1972 mới là thứ làm lý thuyết này lan rộng: ông chỉ ra 21 bài toán tổ hợp quen thuộc — những bài mà giới vận trù học và tổ hợp đã vật lộn nhiều năm — đều NP-đầy đủ. Tác động về mặt tâm lý rất lớn: hoá ra nhiều người đã thất bại trong việc tìm thuật toán nhanh cho những bài khác nhau, nhưng tất cả họ đang vấp phải *cùng một bức tường*.

Đó cũng là bài học lớn nhất của chương, và nó thuộc về phương pháp làm việc hơn là về lý thuyết: kết quả về độ khó không phải tin xấu mà là thông tin. Nó chuyển câu hỏi từ “tôi chưa đủ giỏi” sang “bài này đòi hỏi một loại đánh đổi”, và nó cho phép cả một cộng đồng ngừng tìm cái không có và bắt đầu xây dựng cái có — xấp xỉ, tham số hoá, bộ giải. Toàn bộ mục 4 của chương này là thành quả của sự chuyển hướng đó.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1928–1936 — Hilbert, Turing	Có thủ tục cơ học nào quyết định mọi mệnh đề toán học không?	Không — và để chứng minh, Turing phải định nghĩa “thuật toán”; máy Turing ra đời từ một kết quả <i>phủ định</i> .
1965 — Edmonds	Cần một ranh giới rõ ràng giữa “giải được” và “giải được trong thực tế”.	Thời gian đa thức làm chuẩn cho tính khả thi; một quy ước có tranh cãi nhưng hữu ích.
1971 — Cook; Levin (độc lập)	Có bài toán nào “khó nhất” trong lớp các bài kiểm chứng nhanh không?	SAT là NP-đầy đủ; mọi bài trong NP quy dẫn về nó.
1972 — Karp	Kết quả của Cook là logic trừu tượng, giới tổ hợp chưa thấy liên quan.	21 bài toán tổ hợp quen thuộc đều NP-đầy đủ; nhiều thất bại riêng lẻ hoá ra là cùng một bức tường.
Thập niên 1970–90 — xấp xỉ và tham số hoá	Bài NP-khó vẫn phải giải trong thực tế.	Thuật toán xấp xỉ có bảo đảm; lý thuyết phức tạp tham số hoá tách “khó theo n ” khỏi “khó theo k ”.
Cuối thập niên 1990–nay — bộ giải công nghiệp	Các bài logic và tối ưu thật có hàng triệu biến.	Bộ giải SAT và quy hoạch nguyên khai thác cấu trúc ẩn; NP-khó không còn đồng nghĩa với “không giải được trong thực tế”.

I 23.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Mật mã	Độ khó làm tài nguyên	Đảo ngược hoàn toàn quan hệ với độ khó: ở đây ta cần bài toán ngược khó. Điểm tinh tế: mật mã cần khó ở <i>trường hợp trung bình</i> , còn NP-đầy đủ chỉ nói về trường hợp xấu nhất — nên NP-đầy đủ không tự động dùng làm nền mật mã được.
Vận trù học	Bộ giải quy hoạch nguyên trong logistics, lập lịch bay	Hướng 4 ở mục 4, ở quy mô công nghiệp. Các bài lập lịch thật là NP-khó nhưng được giải hằng ngày — vì dữ liệu thật có cấu trúc và vì người ta chấp nhận lời giải tốt thay vì tối ưu tuyệt đối.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Kiểm chứng phần mềm và phần cứng	Bộ giải SAT/SMT, phân tích tĩnh	Hai liên hệ: hướng 4 dùng SAT làm công cụ; và kết quả không giải được của Turing giải thích vì sao mọi công cụ phân tích tĩnh đều phải chấp nhận bỏ sót hoặc báo động giả.
Sinh học tính toán	Gấp protein, lắp ráp bộ gen, cây phát sinh loài	Nhiều bài trong nhóm này là NP-khó, và ngành này sống bằng đúng bốn hướng ở mục 4 — đặc biệt là heuristic có kiểm chứng thực nghiệm và các trường hợp riêng có cấu trúc.
Kinh tế học	Độ phức tạp của việc tìm điểm cân bằng	Câu hỏi “con người có thật sự đạt tới cân bằng không” có một chiều tính toán: nếu tìm cân bằng là khó về mặt tính toán thì mô hình giả định mọi người tính được nó là đáng ngờ. Đây là ví dụ độ phức tạp làm thay đổi một ngành khác.
Trò chơi	Sudoku, dò mìn, xếp gạch tổng quát hoá đều NP-khó	Cách kiểm tra trực giác tốt: nếu một trò chơi giải đồ cảm giác “phải thử rồi quay lui” thì rất có thể phiên bản tổng quát của nó NP-khó. Các trò hai người thường còn khó hơn NP.
Robot và SLAM	Ghép dữ liệu, chọn tập ràng buộc nhất quán tối đa	Bài “chọn tập lớn nhất các quan sát nhất quán với nhau” có dạng tập độc lập — NP-khó nói chung. Đó là lý do thực tế người ta dùng RANSAC và các heuristic (C/20) thay vì tìm nghiệm tối ưu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bài quyết định cho một khái niệm “chứng cứ” sạch sẽ: câu trả lời “có” luôn kèm một cấu hình cụ thể mà ta kiểm được nhanh. Với bài tối ưu, để kiểm “đây là giá trị tối ưu” bạn phải chứng minh không có gì tốt hơn — một nghĩa vụ nặng hơn hẳn. Việc phát biểu qua dạng quyết định không làm mất tính tổng quát, vì bài tối ưu quy về bài quyết định bằng cách nhai phân trên đáp án (C/19).
- A2. Vì $O(nW)$ là giả đa thức: nó đa thức theo giá trị W , nhưng W được biểu diễn bằng $\log W$ bit trong dữ liệu vào. Với $W = 2^{100}$, dữ liệu vào vẫn ngắn còn thuật toán cần 2^{100} bước. Độ phức tạp luôn đo theo kích thước biểu diễn, nên $O(nW)$ là mũ theo thước đo đó. Điều thú vị là điều này gợi ý một cách phân loại tinh hơn: các bài NP-khó có lời giải giả đa thức thì “nhẹ” hơn các bài NP-khó cả khi số nhỏ, ví dụ bài chu trình Hamilton.
- A3. Từ bài đã biết là khó sang bài của bạn: bạn phải chỉ ra rằng bài của bạn đủ mạnh để mô phỏng một bài khó. Chiều ngược lại vô nghĩa vì việc biến bài của bạn thành một bài tổng quát đã biết là khó không nói gì cả — mọi bài đều nhúng được vào một bài đủ tổng quát. Cách nhớ: quy dẫn chuyển độ khó theo chiều mũi tên, nên mũi tên phải bắt đầu từ nguồn độ khó.
- A4. Vì NP-khó nói về trường hợp xấu nhất, còn dữ liệu công nghiệp hầu như không bao giờ là trường hợp xấu nhất — nó có rất nhiều cấu trúc: ràng buộc lặp lại, biến gần như độc lập, đối xứng. Bộ giải hiện đại khai thác chính những cấu trúc đó bằng lan truyền ràng buộc, học mệnh đề từ xung đột, và phá đối xứng (C/13). Chúng không mâu thuẫn với lý thuyết: vẫn tồn tại những trường hợp nhỏ mà chúng không giải nổi, và người ta dùng đúng những trường hợp đó để so sánh bộ giải.
- A5. Bốn thay đổi cụ thể. Một, ngừng tìm lời giải đa thức chính xác — tiết kiệm hàng giờ hoặc hàng ngày. Hai, đọc lại giới hạn của bài: nếu n nhỏ thì tìm thuật toán mũ tốt. Ba, hỏi dữ liệu có cấu trúc đặc biệt không — cây, đồ thị hai phía, tham số nhỏ — vì rất nhiều bài NP-khó trở nên dễ trên lớp hẹp hơn. Bốn, nếu tất cả đều không, chọn một trong bốn hướng ở mục 4 và nói rõ mình đang đánh đổi cái gì.

NÓI LẠI THẬT ĐƠN GIẢN

Có một sự khác nhau rất lớn giữa việc tìm ra một lời giải và việc kiểm tra một lời giải người khác đưa cho.

Hãy nghĩ tới một câu đố ghép hình khó. Tự ghép thì có thể mất cả buổi. Nhưng nếu ai đó đưa cho bạn một cách ghép và bảo “xong rồi đây”, bạn chỉ mất một phút để kiểm xem có đúng không. Rất nhiều bài toán quan trọng có tính chất này: kiểm thì nhanh, tìm thì không ai biết cách nào nhanh. Câu hỏi lớn nhất của ngành khoa học máy tính chính là: có phải cứ kiểm được nhanh thì tìm cũng được nhanh? Chưa ai trả lời được, và hầu hết mọi người tin là không.

Điều thực dụng hơn là chuyện sau. Người ta phát hiện ra rằng rất nhiều bài toán khó khác nhau — xếp lịch, chia đồ, đi qua mọi thành phố một lần, tô màu bản đồ với ràng buộc — thực chất là cùng một bài khoắc áo khác nhau. Nếu ai giải nhanh được một cái thì giải nhanh được tất cả. Điều đó nghe như tin xấu, nhưng thật ra là tin rất tốt: nó có nghĩa là khi bạn nhận ra bài của mình thuộc nhóm đó, bạn biết ngay rằng không phải mình kém, mà là cả thế giới cũng chưa ai làm được.

Và khi biết vậy, câu hỏi đổi hẳn. Thay vì hỏi “giải thế nào”, bạn hỏi: dữ liệu của tôi có nhỏ không (nếu nhỏ thì cứ thử hết một cách thông minh); tôi có chấp nhận lời giải gần đúng không (có những cách gần đúng kèm bảo đảm hẳn hoi); dữ liệu của tôi có đặc điểm gì riêng không (rất nhiều bài khó trở nên dễ khi dữ liệu có dạng cây hoặc chia được thành hai nhóm). Đó mới là các câu hỏi dẫn tới lời giải dùng được.

Một câu: *Biết bài toán là khó không phải thất bại — đó là lúc câu hỏi đổi từ “giải thế nào” sang “đánh đổi cái gì”.*

Chỗ không rút gọn được: Việc P có bằng NP hay không vẫn là câu hỏi mở; mọi điều ở trên dựa trên niềm tin phổ biến rằng chúng khác nhau, chứ không dựa trên chứng minh.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• P và NP theo cách hiểu thực dụng	P: tìm được nhanh. NP: <i>kiểm</i> được nhanh nếu có lời giải cho sẵn. NP không có nghĩa là “không đa thức”.
••• Chiều của quy dẫn	Muốn chứng minh bài của bạn khó: quy dẫn <i>từ</i> bài đã biết là khó <i>sang</i> bài của bạn. Chiều ngược lại không chứng minh gì.
••• Ba phần của một quy dẫn	Phép biến đổi; chứng minh nó đa thức; chứng minh tương đương <i>cả hai chiều</i> .
••• Giá đa thức	Đa thức theo <i>giá trị</i> nhưng mũ theo <i>số bit</i> biểu diễn; ba lô $O(n^W)$ là ví dụ. Không mâu thuẫn với NP-khó.
••• Bốn cách sống chung	Mũ thông minh (giữ chính xác); xấp xỉ có bảo đảm (giữ đa thức); tham số hoá $O(f(k)n^c)$ (giữ cả hai, hẹp theo k); bộ giải công nghiệp.
••• Câu hỏi sau khi biết bài NP-khó	n bao nhiêu; cần chính xác hay gần đúng; dữ liệu có cấu trúc đặc biệt gì (cây, hai phía, tham số nhỏ).
•• Danh mục quy dẫn hay dùng	3-SAT, tập độc lập / clique, phủ đỉnh, tô màu, Hamilton/TSP, ba lô/phân hoạch, phủ tập hợp, đóng thùng.
•• Bài khó trở nên dễ khi nào	Trên cây, trên đồ thị hai phía, khi tham số nhỏ, khi số màu bằng 2 — luôn kiểm cấu trúc dữ liệu trước khi bỏ cuộc.
•• Cook–Levin và Karp	SAT là NP-đầy đủ (1971); Karp (1972) đưa 21 bài tổ hợp quen thuộc vào cùng lớp, biến lý thuyết thành công cụ dùng được.
•• Bài toán dừng	Không giải được — đã chứng minh, không phải chưa tìm ra. Hệ quả: phân tích tĩnh buộc phải bỏ sót hoặc báo động giả.
•• Vì sao bộ giải công nghiệp mạnh	NP-khó nói về trường hợp xấu nhất; dữ liệu thật có cấu trúc, và bộ giải khai thác nó bằng lan truyền ràng buộc và học từ xung đột.
• NP-đầy đủ và mật mã	NP-đầy đủ là độ khó <i>trường hợp xấu nhất</i> , còn mật mã cần khó ở <i>trường hợp trung bình</i> — nên không dùng trực tiếp làm nền mật mã được.

••• phải thuộc •• nên nắm • biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao tôi nghi ngờ một bài là NP-khó ngay khi đọc đề?

| Ba dấu hiệu thực dụng. Một, bài yêu cầu chọn một tập con tối ưu với các ràng buộc xung khắc giữa các phần tử — rất gần tập độc lập hoặc phủ đỉnh. Hai, bài yêu cầu một thứ tự đi qua toàn bộ đối tượng với chi phí — gần TSP. Ba, và đáng tin nhất trong thi đấu: giới hạn n rất nhỏ ($n \leq 20$ hoặc $n \leq 40$) trong khi bài trông như một bài tối ưu tổng quát — người ra đề đang nói với bạn rằng không có lời giải đa thức.

2. Nếu bài là NP-khó thì trong kỳ thi tôi làm gì?

| Đọc lại giới hạn — chúng luôn cho biết lời giải mong muốn. $n \leq 20$: quy hoạch động mặt nạ bit. $n \leq 40$: gặp nhau ở giữa. n lớn nhưng có cấu trúc cây hoặc đồ thị hai phía: có lời giải đa thức cho trường hợp riêng. Tổng giá trị nhỏ: quy hoạch động giả đa thức. Nếu không dấu hiệu nào khớp thì rất có thể bạn đã hiểu sai đề — hãy đọc lại, vì đề thi hiếm khi yêu cầu giải một bài NP-khó tổng quát ở quy mô lớn.

3. Chứng minh NP-khó có xuất hiện trong phòng vẫn không?

| Chứng minh đầy đủ thì hiếm, nhưng *nhận ra* thì có và nó tạo ấn tượng tốt. Nếu người phỏng vấn đưa một bài mà bạn nhận ra là biến thể của tập độc lập hay TSP, việc nói “bài này ở dạng tổng quát là NP-khó, nên tôi sẽ hỏi thêm về ràng buộc dữ liệu, hoặc đề xuất một thuật toán xấp xỉ với bảo đảm” cho thấy đúng loại tư duy mà họ tìm — biết dừng đúng lúc và biết đặt câu hỏi về đánh đổi.

4. “Xấp xỉ 2 lần” nghĩa chính xác là gì?

| Nghĩa là với *mọi* dữ liệu vào, kết quả của thuật toán không tệ hơn hai lần giá trị tối ưu — với bài cực tiểu thì không quá gấp đôi, với bài cực đại thì không dưới một nửa. Đây là bảo đảm chứng minh được, không phải quan sát thực nghiệm. Ví dụ đơn giản: với phủ đỉnh, cứ lấy cả hai đầu của một cạnh chưa bị phủ và lặp lại — kết quả không quá gấp đôi tối ưu, vì mọi cạnh được chọn đôi một không chung đỉnh và lời giải tối ưu phải chứa ít nhất một đầu của mỗi cạnh đó.

5. Tôi nghe nói “nếu $P = NP$ thì mọi thứ sụp đổ”. Có đúng không?

| Đúng một phần và hay bị nói quá. Nếu $P = NP$ với một thuật toán *thực dụng* thì phần lớn mật mã khoá công khai hiện nay sụp, và rất nhiều bài tối ưu trở nên dễ. Nhưng một chứng minh $P = NP$ với thuật toán $O(n^{100})$ thì gần như không thay đổi gì trong thực tế. Ngược lại, nếu chứng minh được $P \neq NP$ thì cũng không thay đổi công việc hằng ngày — ta đã hành xử như thể điều đó đúng từ lâu. Giá trị của câu hỏi này chủ yếu nằm ở việc hiểu bản chất của tính toán.

6. Lý thuyết tham số hoá khác gì với việc “chỉ giải bài nhỏ”?

| Khác ở chỗ nó tách *hai* đại lượng thay vì một. Thuật toán $O(2^k \cdot n)$ với k là kích thước lời giải chạy tốt trên đồ thị hàng triệu đỉnh miễn là lời giải cần tìm nhỏ — điều rất hay gặp trong thực tế, ví dụ tìm một tập nhỏ các nút hỏng trong mạng lớn. Đây là một cách nhìn tinh hơn về độ khó: “khó theo cái gì” quan trọng hơn “khó hay dễ”.

7. Có bài toán nào chưa biết là P hay NP-đầy đủ không?

| Có, và chúng thú vị vì hiếm. Bài đẳng cấu đồ thị — hai đồ thị có cùng cấu trúc không — chưa được xếp dứt khoát: chưa ai tìm được thuật toán đa thức, cũng chưa ai chứng minh NP-đầy đủ, và có kết quả cho thấy nó gần như chắc chắn không NP-đầy đủ. Bài phân tích thừa số cũng ở trạng thái tương tự. Sự tồn tại của những bài này là một trong các lý do người ta tin rằng cấu trúc bên trong NP phong phú hơn hai lớp P và NP-đầy đủ.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết một bộ kiểm chứng đa thức cho bài tập độc lập	Bài tập khái niệm	Mục tiêu là cảm nhận rõ khác biệt giữa “kiểm” và “tìm”.
●● Quy dẫn tập độc lập về phủ đỉnh và ngược lại	Bài kinh điển	Phần bù của tập độc lập là phủ đỉnh; chứng minh cả hai chiều.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Quy dẫn 3-SAT về tô màu ba màu	Bài kinh điển	Dựng tiện ích cho mỗi mệnh đề; đây là bài tập chuẩn để học cấu trúc một quy dẫn.
●● Giải TSP chính xác với $n \leq 18$	Bài kinh điển	Quy hoạch động mặt nạ bit; ước lượng $2^n n^2$ trước khi cài.
●● Thuật toán 2-xấp xỉ cho phủ đỉnh, đo tỉ số thực tế	Bài tập thực nghiệm	So với tối ưu tìm bằng vét cạn trên đồ thị nhỏ; tỉ số thật thường tốt hơn 2 nhiều.
●● Tập độc lập lớn nhất trên cây bằng quy hoạch động	Bài kinh điển	Bài NP-khó tổng quát nhưng đa thức trên cây — ví dụ trực tiếp cho lời khuyên ở mục 3.
●● Thuật toán tham số hoá $O(2^k n)$ cho phủ đỉnh kích thước $\leq k$	[74]	Chọn một cạnh, một trong hai đầu phải thuộc lời giải; cây nhánh có độ sâu k .
●● Mã hoá một bài xếp lịch thành SAT và giải bằng bộ giải	Bài tập ứng dụng	So thời gian với vét cạn tự viết; đây là cách nhanh nhất để hiểu hướng 4 ở mục 4.
●● Tìm một bài trong công việc của bạn và xác định nó có NP-khó không	Bài tập tự do	Nếu có, hãy viết ra bốn hướng đối phó và chọn một, kèm lý do về đánh đổi.
●● Chứng minh bài đóng thùng NP-khó bằng quy dẫn từ bài phân hoạch	Bài kinh điển	Quy dẫn ngắn nhưng phải viết đủ hai chiều; kiểm lại chiều mũi tên.

Kết chương và đọc tiếp

Chương này thay đổi cách bạn phân bổ thời gian nhiều hơn bất kỳ chương kỹ thuật nào. Ba điều đáng mang theo: sự phân biệt giữa tìm và kiểm, vì nó là hạt nhân của toàn bộ lý thuyết; chiều của quy dẫn, vì đó là lỗi phổ biến nhất; và bốn hướng đối phó ở mục 4, kèm câu hỏi “tôi đang đánh đổi cái gì”.

Chương cuối của phần D nhìn vào giới hạn từ một hướng khác. Chương này nói về những bài toán mà *chưa ai* tìm được lời giải nhanh; chương tiếp theo nói về những trường hợp ta *chứng minh được* rằng không thể nhanh hơn, và về những mô hình tính toán khác — dữ liệu chảy qua một lần, quyết định phải ra khi chưa biết tương lai, bộ nhớ chậm hơn hàng nghìn lần — nơi bảng xếp hạng thuật toán thay đổi hoàn toàn (D/24).

Cận dưới và các mô hình tính toán khác (Lower Bounds and Other Models)

PHỤ THUỘC A/03 cho mô hình chi phí và phân cấp bộ nhớ; A/05 cho lập luận đếm; D/23 cho mặt kia của độ khó.

TÓM TẮT

Chương trước nói về những bài toán mà *chưa ai* tìm được lời giải nhanh. Chương này nói về hai thứ khác: những trường hợp ta *chứng minh được* rằng không thể nhanh hơn, và những mô hình tính toán mà trong đó bảng xếp hạng thuật toán thay đổi hoàn toàn. Nửa đầu trình bày ba kỹ thuật chứng minh cận dưới — cây quyết định, lập luận đối thủ, và đếm thông tin — cùng lý do vì sao cận dưới lại khó chứng minh tới vậy. Nửa sau bỏ giả định lớn nhất của cả quyển sách: rằng dữ liệu nằm vừa trong bộ nhớ, đọc được nhiều lần, và máy chỉ có một lõi. Khi giả định đó sai — bộ nhớ ngoài, luồng dữ liệu, trực tuyến, song song — thì thuật toán tốt nhất đối hẳn. Nếu chỉ nhớ một điều: mô hình chi phí quyết định thuật toán, nên khi kết quả thực nghiệm mâu thuẫn với lý thuyết, thủ phạm gần như luôn là mô hình chứ không phải phép đo.

KIẾN THỨC CẦN CÓ TRƯỚC

- **cận dưới** — phát biểu rằng *mọi* thuật toán trong một mô hình nào đó đều cần ít nhất một lượng tài nguyên nhất định. Khác hẳn với việc phân tích chi phí của một thuật toán cụ thể.
- **cây quyết định** — mô hình trong đó thuật toán chỉ được đặt câu hỏi so sánh; mỗi lá là một kết quả khả dĩ, nên số lá bị chặn dưới bởi số kết quả.
- **lập luận đối thủ** — kỹ thuật chứng minh cận dưới bằng cách hình dung một đối thủ trả lời các câu hỏi của thuật toán một cách bất lợi nhất, miễn là vẫn nhất quán.
- **mô hình bộ nhớ ngoài** — tính chi phí theo số lần chuyển khối giữa bộ nhớ nhanh và chậm, thay vì theo số phép toán.
- **thuật toán luồng dữ liệu** — đọc dữ liệu một hoặc vài lượt, dùng bộ nhớ nhỏ hơn dữ liệu rất nhiều; thường phải chấp nhận kết quả xấp xỉ.
- **tỉ lệ cạnh tranh** — với thuật toán trực tuyến, tỉ số giữa chi phí của nó và chi phí của lời giải tối ưu biết trước toàn bộ tương lai.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chứng minh “không thuật toán nào nhanh hơn” lại khó hơn nhiều so với việc tìm một thuật toán nhanh?
- Q2. Cận dưới $\Omega(n \log n)$ cho sắp xếp có mâu thuẫn với việc sắp xếp đếm chạy $O(n)$ không?
- Q3. Trong mô hình bộ nhớ ngoài, vì sao một thuật toán “chậm hơn” về số phép toán lại có thể nhanh hơn nhiều lần?
- Q4. Thuật toán trực tuyến không biết tương lai. Vậy ta đánh giá nó bằng thước đo nào cho công bằng?
- Q5. Vì sao một chương trình song song trên 32 lõi thường không nhanh gấp 32 lần?

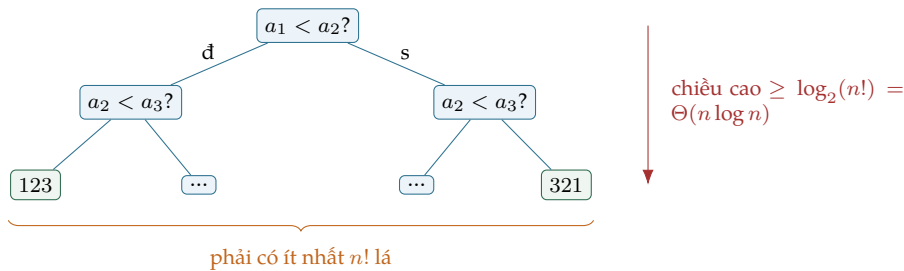
Toàn bộ quyển sách này, tới chương trước, đi theo một hướng: cho một bài toán, tìm thuật toán nhanh hơn. Chương này đi ngược lại và hỏi: *có điểm dừng không, và làm sao biết mình đã tới đó?*

Câu hỏi ấy có giá trị thực dụng lớn hơn vẻ ngoài học thuật của nó. Nếu bạn biết một bài toán cần ít nhất $\Omega(n \log n)$, bạn ngừng tìm lời giải tuyến tính. Nếu bạn biết mọi thuật toán so sánh đều cần $\Omega(n \log n)$ nhưng bài của bạn có thêm thông tin — khoá là số nguyên nhỏ — thì bạn biết chính xác phải phá vỡ giả định nào để đi nhanh hơn. Cận dưới không chỉ nói “dừng lại” mà còn nói *dừng ở đâu và vì sao*.

Nửa sau chương làm một việc khác nhưng cùng tinh thần: nó chỉ ra rằng nhiều kết luận trong các chương trước là hệ quả của một mô hình chi phí cụ thể, và khi mô hình đổi thì kết luận đổi theo.

24.1 Ba cách chứng minh cận dưới

Cách thứ nhất — cây quyết định. Nếu thuật toán chỉ được phép so sánh các phần tử, thì quá trình chạy của nó là một đường đi trên một cây nhị phân: mỗi nút là một phép so sánh, mỗi lá là một kết luận. Với bài sắp xếp n phần tử, số kết luận khả dĩ là $n!$ hoán vị, nên cây phải có ít nhất $n!$ lá, nên chiều cao ít nhất $\log_2(n!) = \Theta(n \log n)$. Đó là cận dưới nổi tiếng nhất trong ngành, và nó chỉ tốn ba câu để chứng minh — nhờ công thức Stirling ở chương A/05.



Hình 24.1. Cận dưới cho sắp xếp bằng cây quyết định. Mỗi phép so sánh chia đôi tập kết quả còn khả dĩ, mà số kết quả là $n!$, nên cây phải cao ít nhất $\log_2(n!)$. Điều bắt buộc phải nhớ kèm: kết luận này chỉ đúng *trong mô hình so sánh* — sắp xếp đếm dùng giá trị làm chỉ số nên nằm ngoài mô hình và không mâu thuẫn.

Điểm mấu chốt, và cũng là câu trả lời cho câu hỏi mở đầu thứ hai: cận này chỉ đúng *trong mô hình so sánh*. Sắp xếp đếm không so sánh phần tử với nhau mà dùng giá trị của chúng làm chỉ số mảng — một thao tác mà mô hình cây quyết định không cho phép. Không có mâu thuẫn nào; chỉ có hai mô hình khác nhau. Đây là bài học tổng quát: *mọi cận dưới đều kèm một mô hình, và bỏ qua mô hình là hiểu sai kết quả*.

Cách thứ hai — lập luận đối thủ. Hình dung một đối thủ không quyết định dữ liệu từ trước mà trả lời từng câu hỏi của thuật toán, luôn chọn câu trả lời bất lợi nhất, miễn là vẫn nhất quán với mọi câu đã trả lời. Nếu đối thủ giữ được ít nhất hai khả năng cho tới sau k câu hỏi thì thuật toán không thể kết luận sớm hơn $k + 1$. Ví dụ: tìm đồng thời cực đại và cực tiểu của n phần tử cần ít nhất $\lceil 3n/2 \rceil - 2$ phép so sánh, chứng minh bằng cách đếm lượng thông tin mà mỗi phép so sánh có thể cung cấp.

Cách thứ ba — đếm hoặc lập luận thông tin. Nếu kết quả cần phân biệt N khả năng và mỗi bước cho tối đa một bit thông tin thì cần ít nhất $\log_2 N$ bước. Đây là dạng tổng quát của cách thứ nhất, và nó cũng là ngôn ngữ chung với lý thuyết thông tin — lý do mà cận dưới cho sắp xếp và cận dưới cho nén dữ liệu có cùng hình thức.

24.2 Vì sao cận dưới khó

Câu hỏi mở đầu thứ nhất đáng được trả lời kỹ, vì nó giải thích tình trạng của cả ngành.

Để chứng minh một thuật toán *tồn tại*, bạn chỉ cần đưa ra một thuật toán — một đối tượng cụ thể. Để chứng minh *không thuật toán nào* nhanh hơn, bạn phải lập luận về *tất cả* các thuật toán khả dĩ, kể cả những cái chưa ai nghĩ ra. Đó là một mệnh đề toàn thể trên một tập vô hạn các đối tượng rất tự do, và ta có rất ít công cụ để nói điều gì đó về mọi thành viên của tập đó.

Vì vậy các cận dưới đã chứng minh được hầu như đều nằm trong những mô hình *hạn chế*: chỉ được so sánh, chỉ được truy cập theo một kiểu nhất định, chỉ có một lượt đọc. Trong mô hình tính toán tổng quát, chúng ta gần như không chứng minh được cận dưới nào đáng kể — và đó chính là lý do câu hỏi P đối với NP vẫn mở sau nửa thế kỷ. Câu chuyện Karatsuba ở chương C/14 là lời cảnh báo cụ thể: một nhà toán học hàng đầu phỏng đoán rằng nhân số cần $\Omega(n^2)$, và phỏng đoán đó bị bác bỏ trong một tuần. Trục giác về cận dưới rất dễ sai.

Một hướng hiện đại đáng biết là *độ phức tạp tính*: thay vì cố chứng minh cận dưới tuyệt đối, người ta chứng minh cận dưới *có điều kiện*, dạng “nếu bài toán A không thể giải nhanh hơn thì bài toán B cũng không”. Nhờ đó có những kết quả kiểu “khoảng cách chỉnh sửa không thể tính nhanh hơn đáng kể so với $O(n^2)$, trừ khi một giả thuyết được tin rộng rãi bị bác bỏ”. Đây là cách ngành này sống chung với sự khó của cận dưới: xây một mạng lưới các kết quả liên hệ với nhau thay vì chờ một chứng minh tuyệt đối.

24.3 Độ phức tạp tính: cận dưới có điều kiện, và cách dùng chúng

Mục trên kết thúc bằng một câu về “độ phức tạp tính”. Ý tưởng ấy đáng được nói kỹ hơn, vì nó là hướng sống động nhất của lý thuyết cận dưới hiện nay và vì nó có ích ngay cả cho người chỉ đi thi.

Cách đặt vấn đề là như sau. Ta không chứng minh nổi cận dưới tuyệt đối, vậy hãy chọn một *giả thuyết neo* mà cộng đồng tin, rồi quy dẫn để cho thấy “nếu bài toán của tôi nhanh hơn thì giả thuyết neo sụp”. Giả thuyết neo hay dùng nhất là **giả thuyết thời gian mũ mạnh** (*strong exponential time hypothesis*, SETH): không có $\varepsilon > 0$ nào để mọi bài k -SAT giải được trong $(2 - \varepsilon)^n$ — một phát biểu ra đời từ Impagliazzo và Paturi[75]. Giả thuyết neo thứ hai là bài 3SUM: cho n số, có ba số nào cộng lại bằng 0 không — tin rằng không có thuật toán $n^{2-\varepsilon}$, dù các cải tiến ở mức thừa số lôgarit thì đã có.

Kết quả đáng nhớ nhất của hướng này, ít nhất với người đọc quyển sách này, là của Backurs và Indyk[76]: nếu tính được khoảng cách chỉnh sửa của hai chuỗi độ dài n trong $O(n^{2-\varepsilon})$ thì SETH sai. Nói cách khác, bảng quy hoạch động $O(nm)$ mà bạn viết ở chương C/16 — một bài tập nhập môn — gần như chắc chắn là đã tối ưu. Ba chục năm người ta cố cải tiến nó và không ai thành công; bây giờ ta biết *vì sao*.

Nhánh 3SUM cho một câu chuyện song song trong hình học. Gajentaan và Overmars[77] chỉ ra cả một họ bài toán phẳng — có ba điểm nào thẳng hàng không, hợp của các tam giác có lỗ không, một số bài lập kế hoạch chuyển động — đều khó ít nhất bằng 3SUM. Điều đó giải thích một hiện tượng mà người thi đấu hay gặp: rất nhiều bài hình học “chỉ” có lời giải n^2 và không ai tìm được gì tốt hơn.

Giá trị thực hành nằm ở chỗ ít ai nói ra: cận dưới có điều kiện cho bạn *quyền dùng tìm*. Khi nhận ra bài mình đang vật lộn quy dẫn được về khoảng cách chỉnh sửa hay về 3SUM, câu hỏi đúng không còn là “làm sao nhanh hơn về mặt tiệm cận” mà chuyển thành ba câu khác: hằng số có giảm được không, đề bài có ràng buộc riêng nào phá được cấu trúc khó không (chuỗi ngắn, bảng chữ cái nhỏ, các điểm nằm trên lưới), và có chấp nhận được lời giải xấp xỉ không. Đó cũng chính

là bốn cách sống chung ở chương D/23, chỉ khác là ở đây ranh giới không phải giữa đa thức và mũ, mà giữa n^2 và $n^{2-\epsilon}$.

24.4 Mô hình bộ nhớ ngoài: khi đơn vị chi phí là lần đọc khối

Chương A/03 đã cảnh báo rằng mô hình RAM coi mọi truy cập bộ nhớ là như nhau, còn thực tế thì có phân cấp. Mục này lấy nhận xét đó làm mô hình chính thức.

Trong mô hình bộ nhớ ngoài, ta đếm số lần chuyển *khối* dữ liệu giữa bộ nhớ chậm và nhanh, mỗi khối chứa B phần tử, và bộ nhớ nhanh chứa được M phần tử. Với mô hình này, sắp xếp không còn là $O(n \log n)$ mà là $O(\frac{n}{B} \log_{M/B} \frac{n}{B})$ lần chuyển khối — và cơ số của lôgarit là M/B , thường hàng trăm, nên số “tầng” rất ít. Đó là nền lý thuyết cho sắp xếp trộn nhiều đường, và cũng là lý do B-cây có nút to (B/09).

Điều đáng chú ý nhất, và là câu trả lời cho câu hỏi mở đầu thứ ba: trong mô hình này, một thuật toán làm nhiều phép toán hơn nhưng truy cập *tuần tự* có thể thắng một thuật toán ít phép toán hơn nhưng nhảy ngẫu nhiên — vì chi phí thật nằm ở việc kéo khối dữ liệu, không ở việc tính toán.

Cache-oblivious là một ý tưởng đẹp trong nhánh này [78]: thiết kế thuật toán tối ưu về số lần chuyển khối mà *không cần biết* B và M là bao nhiêu. Kỹ thuật chung là chia để trị đệ quy tới mọi tỉ lệ, để ở tầng nào của phân cấp bộ nhớ thì cũng có một mức đệ quy vừa khít. Nhờ vậy một chương trình chạy tốt trên mọi máy mà không phải tinh chỉnh tham số.

24.5 Ba mô hình bỏ giả định khác

Luồng dữ liệu. Dữ liệu chảy qua một lần và quá lớn để lưu; bạn có bộ nhớ nhỏ hơn nhiều bậc. Trong mô hình này, hầu hết bài toán chính xác trở nên bất khả thi, và người ta chuyển sang xấp xỉ: đếm số phần tử phân biệt bằng HyperLogLog, tìm phần tử xuất hiện nhiều bằng sơ đồ đếm, lấy mẫu đều bằng hồ chứa (C/20). Đây là mô hình của các hệ thống đo lường và phân tích nhật ký ở quy mô lớn.

Trực tuyến. Bạn phải quyết định ngay khi mỗi dữ liệu tới, không được xét lại, và không biết tương lai. Câu hỏi mở đầu thứ tư hỏi về thước đo, và câu trả lời là *tỉ lệ cạnh tranh*: so chi phí của bạn với chi phí của một lời giải tối ưu *biết trước toàn bộ tương lai*. Ví dụ kinh điển là bài thuê hay mua: mỗi ngày trượt tuyết bạn có thể thuê ván với giá 1 hoặc mua hẳn với giá B , và bạn không biết mình sẽ trượt bao nhiêu ngày; chiến lược “thuê B ngày rồi mua” có tỉ lệ cạnh tranh 2, và người ta chứng minh được không chiến lược tắt định nào tốt hơn. Cùng khung phân tích áp cho các chính sách thay thế bộ nhớ đệm: LRU có tỉ lệ cạnh tranh phụ thuộc kích thước bộ đệm, và phân tích đó giải thích vì sao nó là lựa chọn hợp lý dù không tối ưu.

Song song. Bỏ giả định một lõi. Thước đo chuẩn gồm hai đại lượng: *tổng công việc* — số phép toán nếu chạy tuần tự — và *chiều sâu* — độ dài chuỗi phụ thuộc dài nhất, tức thời gian nếu có vô hạn bộ xử lý. Thời gian thực tế bị chặn dưới bởi chiều sâu, nên một thuật toán có chuỗi phụ thuộc dài thì song song hoá bao nhiêu cũng vô ích.

Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó có ba lớp lý do. Thứ nhất, phần tuần tự không loại bỏ được sẽ chi phối khi số lõi tăng — nếu 5% chương trình là tuần tự thì dù có vô hạn lõi cũng không nhanh hơn 20 lần. Thứ hai, chi phí điều phối: tạo luồng, đồng bộ, truyền dữ liệu đều tốn, và chúng tăng theo số lõi. Thứ ba, tranh chấp tài nguyên chung, đặc biệt là băng thông bộ nhớ — nhiều lõi cùng đọc thì bộ nhớ trở thành nút cổ chai. Kết quả thực nghiệm nổi tiếng của

McSherry và cộng sự[38] cho thấy nhiều hệ thống song song bị một chương trình đơn luồng viết cẩn thận đánh bại, đúng vì ba lý do trên.

CẦN NHỚ

Mô hình chi phí quyết định thuật toán. Bốn mô hình trong chương này — bộ nhớ ngoài, luồng, trực tuyến, song song — mỗi cái làm một họ thuật toán khác nhau trở thành tốt nhất. Khi bạn thấy một thuật toán “tốt hơn về lý thuyết” lại thua trong thực nghiệm, câu hỏi đầu tiên không phải “phép đo có sai không” mà là “tôi đang ở mô hình chi phí nào”.

24.6 Gốc rễ: từ cân đo tới các hệ thống dữ liệu lớn

Cận dưới cho sắp xếp thuộc loại kết quả xuất hiện sớm và tự nhiên: các bài toán “cân bao nhiêu lần thì tìm ra đồng xu giả” đã có trong toán giải trí từ lâu, và chúng chính là lập luận cây quyết định ở dạng dân dã. Việc hình thức hoá thành một cận dưới cho sắp xếp thuộc về giai đoạn đầu của ngành, và nó vẫn là ví dụ chuẩn để dạy khái niệm “cận dưới trong một mô hình”.

Phân tích cạnh tranh cho thuật toán trực tuyến ra đời năm 1985 trong cùng bài báo mà Sleator và Tarjan giới thiệu splay tree[35] — một sự trùng hợp đáng chú ý, vì cả hai ý tưởng đều xuất phát từ cùng một câu hỏi: làm sao đánh giá công bằng một thuật toán không có đảm bảo cho từng thao tác riêng lẻ. Phân tích khấu hao trả lời cho trường hợp offline; phân tích cạnh tranh trả lời cho trường hợp không biết tương lai. Bối cảnh thúc đẩy là rất thực tế: các chính sách quản lý bộ nhớ đệm trong hệ điều hành cần một tiêu chuẩn đánh giá, và “trường hợp xấu nhất” thuần túy thì quá bi quan vì mọi chính sách đều tệ trên dữ liệu thù địch.

Mô hình bộ nhớ ngoài được hình thức hoá vào cuối thập niên 1980 khi cơ sở dữ liệu trở thành ngành lớn và khoảng cách tốc độ giữa đĩa và bộ nhớ ngày càng rõ. Cache-oblivious ra đời năm 1999[78] với một động cơ rất thực dụng: các thư viện tối ưu theo tham số phần cứng phải viết lại cho mỗi đời máy, và người ta muốn thoát khỏi vòng lặp đó.

Thuật toán luồng dữ liệu có một mốc rõ ràng: bài báo của Alon, Matias và Szegedy giữa thập niên 1990[79] đặt nền cho việc ước lượng các đại lượng thống kê với bộ nhớ nhỏ hơn dữ liệu nhiều bậc. Động cơ ban đầu mang tính lý thuyết, nhưng chỉ một thập kỷ sau, khi lưu lượng mạng và nhật ký hệ thống vượt xa khả năng lưu trữ, các kỹ thuật đó trở thành hạ tầng của mọi hệ thống giám sát.

Điểm chung của bốn câu chuyện: mỗi mô hình mới ra đời khi một giả định của mô hình cũ trở nên sai trong thực tế. Đó là mẫu đáng nhớ nhất của chương, và nó gợi ý rằng sẽ còn những mô hình mới — năng lượng, tính toán trên dữ liệu mã hoá, hay phần cứng chuyên dụng — khi các ràng buộc mới trở nên chi phối.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Giai đoạn đầu của ngành — cận dưới sắp xếp	Có thuật toán sắp xếp nhanh hơn $n \log n$ không?	Lập luận cây quyết định: $\log_2(n!) = \Theta(n \log n)$; và bài học “cận dưới luôn kèm một mô hình”.
1967 — Amdahl	Máy nhiều bộ xử lý có thực sự nhanh hơn tỉ lệ thuận không?	Phân tuần tự không loại bỏ được sẽ chi phối; đặt kỳ vọng thực tế cho tính toán song song.
1985 — Sleator & Tarjan	Đánh giá thế nào cho công bằng một thuật toán không biết tương lai?	Tỉ lệ cạnh tranh: so với lời giải tối ưu biết trước tương lai; ra đời cùng bài báo với splay tree và phân tích khấu hao.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
Cuối thập niên 1980 — mô hình bộ nhớ ngoài	Cơ sở dữ liệu lớn hơn bộ nhớ; đĩa chậm hơn hàng nghìn lần.	Đếm chi phí theo lần chuyển khối; giải thích B-cây và sắp xếp trộn nhiều đường.
Giữa thập niên 1990 — Alon, Matias, Szegedy	Ước lượng thống kê trên dòng dữ liệu quá lớn để lưu.	Nền của thuật toán luồng; một thập kỷ sau trở thành hạ tầng giám sát hệ thống.
1999 — Frigo và cộng sự	Thư viện tối ưu theo tham số phần cứng phải viết lại cho mỗi đời máy.	Cache-oblivious: tối ưu mà không cần biết kích thước khối và bộ nhớ đệm.
2015 — McSherry và cộng sự	Hệ thống phân tán “mở rộng tốt” có thật sự nhanh không?	Nhiều hệ thống bị một chương trình đơn luồng viết tốt đánh bại; nhắc rằng chi phí điều phối không xuất hiện trong phân tích tiệm cận.

24.7 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Lý thuyết thông tin	Entropy và cận dưới cho nén	Cùng lập luận đếm: cần $\log_2 N$ bit để phân biệt N khả năng. Cận dưới sắp xếp và cận dưới nén là hai mặt của một kết quả.
Cơ sở dữ liệu	Sắp xếp ngoài, chỉ mục, tối ưu hoá truy vấn	Mô hình bộ nhớ ngoài là mô hình chính thức của ngành này. Bộ tối ưu truy vấn ước lượng số lần đọc khối chứ không ước lượng số phép toán — đúng tinh thần mục 3.
Hệ điều hành và mạng phân phối nội dung	Chính sách thay thế bộ nhớ đệm	Phân tích cạnh tranh cho biết một chính sách trực tuyến kém tối đa bao nhiêu lần so với biết trước tương lai — thước đo phù hợp hơn “trường hợp xấu nhất” thuần túy.
Hệ thời gian thực và robot	Ràng buộc thời hạn cho từng chu kỳ	Ở đây khẩu hao không dùng được và cả tỉ lệ cạnh tranh cũng không đủ: cái cần là cận trên cho <i>mỗi</i> thao tác. Đây là một mô hình chi phí riêng mà sách giáo khoa thuật toán ít nói tới.
Tính toán trên GPU	Mô hình công việc và chiều sâu, hợp nhất truy cập bộ nhớ	Chiều sâu là ràng buộc thật: thuật toán có chuỗi phụ thuộc dài không tăng tốc được dù có hàng nghìn lõi. Ngoài ra kiểu truy cập bộ nhớ quyết định nhiều hơn số phép toán — lại là mô hình bộ nhớ ngoài ở quy mô nhỏ.
Hệ thống dữ liệu lớn	Xử lý luồng, cửa sổ thời gian, sơ đồ xấp xỉ	Ứng dụng trực tiếp của mục 4. Đánh đổi đặc trưng: chấp nhận sai số vài phần trăm để đổi lấy khả năng chạy được ở quy mô mà lời giải chính xác không khả thi.
Thiết bị nhúng, robot chạy pin	Năng lượng làm đơn vị chi phí	Một mô hình gần như vắng mặt trong sách giáo khoa thuật toán nhưng chi phối trong thực tế: thuật toán ít phép toán hơn nhưng truy cập bộ nhớ nhiều hơn có thể tốn nhiều năng lượng hơn.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai loại mệnh đề có bản chất khác nhau. Chứng minh tồn tại thuật toán nhanh chỉ cần đưa ra *một* đối tượng cụ thể. Chứng minh cận dưới đòi lập luận về *mọi* thuật toán khả dĩ, kể cả những cái chưa ai nghĩ ra — một mệnh đề toàn thể trên một tập vô hạn rất tự do. Vì vậy các cận dưới đã có hầu như đều nằm trong mô hình hạn chế (chỉ so sánh, một lượt đọc), còn trong mô hình

tổng quát thì ta gần như không chứng minh được gì — đó cũng là lý do câu hỏi P đối với NP vẫn mở.

- A2. Không mâu thuẫn, vì hai kết quả nằm trong hai mô hình khác nhau. Cận $\Omega(n \log n)$ chỉ áp cho các thuật toán *chỉ được so sánh phần tử với nhau*; sắp xếp đếm không so sánh mà dùng giá trị làm chỉ số mảng, một thao tác nằm ngoài mô hình đó. Bài học tổng quát: mỗi cận dưới luôn kèm một mô hình, và muốn vượt qua nó thì phải phá vỡ đúng giả định của mô hình — ở đây là “chỉ được so sánh”.
- A3. Vì trong mô hình đó, đơn vị chi phí là *lần chuyển khối*, không phải phép toán. Đọc một khối 4 KB từ đĩa tốn như nhau dù bạn dùng một phần tử hay cả khối, nên thuật toán truy cập tuần tự tận dụng được toàn bộ khối, còn thuật toán nhảy ngẫu nhiên trả giá đầy đủ cho mỗi phần tử. Chênh lệch giữa hai kiểu truy cập có thể tới hàng nghìn lần — đủ để lấn át một khác biệt về số phép toán. Cùng hiện tượng xảy ra ở quy mô nhỏ với cache, và đó là lý do mảng thắng con trỏ (B/06).
- A4. Bằng tỉ lệ cạnh tranh: tỉ số giữa chi phí của thuật toán trực tuyến và chi phí của lời giải tối ưu *biết trước toàn bộ tương lai*. Thước đo này công bằng ở chỗ nó không đòi hỏi thuật toán phải làm điều bất khả thi, mà chỉ hỏi nó tệ hơn bao nhiêu lần so với một đối thủ được biết tương lai. Ví dụ bài thuê hay mua: chiến lược “thuê tới khi tổng tiền thuê bằng giá mua rồi mua” có tỉ lệ 2, và không chiến lược tắt định nào tốt hơn.
- A5. Ba lớp lý do. Một, phần tuần tự không loại bỏ được: nếu 5% chương trình không song song hoá được thì dù vô hạn lõi cũng chỉ nhanh tối đa 20 lần. Hai, chi phí điều phối — tạo luồng, đồng bộ, truyền dữ liệu — tăng theo số lõi và có thể vượt phần được tăng tốc. Ba, tài nguyên chung: nhiều lõi cùng đọc bộ nhớ làm băng thông thành nút cổ chai. Ba lý do này giải thích kết quả thực nghiệm rằng nhiều hệ thống song song thua một chương trình đơn luồng được viết cẩn thận[38].

NÓI LẠI THẬT ĐƠN GIẢN

Chương này nói về hai chuyện, và cả hai đều là chuyện “biết mình đang chơi theo luật nào”. Chuyện thứ nhất: đôi khi ta chứng minh được rằng không ai có thể nhanh hơn một mức nào đó. Ví dụ dễ hình dung nhất: nếu bạn chỉ được phép hỏi “số này lớn hơn số kia không”, thì để sắp xếp n món bạn buộc phải hỏi cỡ $n \log n$ câu, vì mỗi câu hỏi chỉ chia đôi số khả năng còn lại, mà số cách sắp xếp thì rất lớn. Điều quan trọng phải nhớ kèm theo: kết luận đó chỉ đúng *với luật chơi* “chỉ được hỏi so sánh”. Nếu bạn được phép dùng giá trị của món đồ làm địa chỉ ngăn tủ thì luật đã khác và bạn có thể nhanh hơn. Rất nhiều tranh cãi kiểu “sách nói không thể nhanh hơn mà tôi làm được” đến từ việc quên mất luật chơi.

Chuyện thứ hai còn thực dụng hơn: gần như toàn bộ quyển sách này ngầm giả định rằng dữ liệu nằm gọn trong bộ nhớ, đọc lại bao nhiêu lần cũng được, và máy chỉ làm một việc một lúc. Khi một trong ba giả định đó sai, bảng xếp hạng thuật toán đảo lộn.

Nếu dữ liệu nằm trên đĩa, thứ đắt không còn là phép tính mà là mỗi lần với tay xuống kho lấy một thùng — nên thuật toán tốt là thuật toán lấy ít thùng và dùng hết mọi thứ trong thùng. Nếu dữ liệu chảy qua một lần như nước và bạn chỉ có một cái ca nhỏ, bạn phải từ bỏ câu trả lời chính xác và chuyển sang ước lượng. Nếu bạn phải quyết định ngay mà chưa biết tương lai, cách đánh giá công bằng là so với một người biết trước mọi chuyện — và bạn chấp nhận tệ hơn họ vài lần. Còn nếu bạn có nhiều lõi thì hãy nhớ: chuỗi việc phải làm nối tiếp nhau sẽ quyết định thời gian, chứ không phải tổng số việc.

Một câu: Mọi kết luận về “thuật toán nào tốt hơn” đều là kết luận trong một luật chơi; đổi luật thì đổi câu trả lời.

Chỗ không rút gọn được: Chứng minh cận dưới trong mô hình tính toán tổng quát vẫn nằm ngoài khả năng hiện nay; đó là lý do nhiều câu hỏi lớn còn mở.

Thuật ngữ / câu hỏi	Trả lời
●●● Cận dưới luôn kèm mô hình	$\Omega(n \log n)$ cho sắp xếp chỉ đúng trong mô hình <i>so sánh</i> ; sắp xếp đếm không mâu thuẫn vì nó dùng giá trị làm chỉ số.
●●● Ba kỹ thuật cận dưới	Cây quyết định (số lá $\geq$ số kết quả); lập luận đối thủ (giữ nhiều khả năng càng lâu càng tốt); đếm thông tin ($\log_2 N$ bit).
●●● Vì sao cận dưới khó	Phải lập luận về <i>mọi</i> thuật toán khả dĩ, kể cả chưa ai nghĩ ra — mệnh đề toàn thể trên tập vô hạn tự do.
●●● Mô hình bộ nhớ ngoài	Đếm số lần chuyển khối, không đếm phép toán; sắp xếp là $O(\frac{n}{B} \log_{M/B} \frac{n}{B})$ lần chuyển. Giải thích B-cây và trộn nhiều đường.
●●● Hai đại lượng của song song	Tổng công việc và chiều sâu (chuỗi phụ thuộc dài nhất). Thời gian bị chặn dưới bởi chiều sâu, dù có bao nhiêu lõi.
●●● Ba lý do song song không tăng tuyến tính	Phản tuần tự chi phối; chi phí điều phối tăng theo số lõi; tranh chấp băng thông bộ nhớ.
●● Tỷ lệ cạnh tranh	So chi phí của thuật toán trực tuyến với lời giải tối ưu biết trước tương lai; bài thuê hay mua có tỉ lệ 2 và đó là tối ưu cho chiến lược tất định.
●● Mô hình luồng dữ liệu	Một lượt đọc, bộ nhớ nhỏ hơn dữ liệu nhiều bậc; hầu hết bài chính xác bất khả thi nên chuyển sang xấp xỉ (HyperLogLog, sơ đồ đếm, lấy mẫu hồ chứa).
●● Cache-oblivious	Tối ưu số lần chuyển khối mà không cần biết B và M ; đạt được bằng chia để trị đệ quy tới mọi tỉ lệ.
●● Độ phức tạp tinh	Cận dưới <i>có điều kiện</i> : “nếu bài A không nhanh hơn được thì B cũng không”; cách sống chung với việc cận dưới tuyệt đối quá khó.
●● Khi lý thuyết mâu thuẫn thực nghiệm	Nghi mô hình chi phí trước, đừng nghi phép đo — bạn có thể đang ở mô hình khác với mô hình mà kết quả lý thuyết giả định.
● Hệ thời gian thực	Cần cận trên cho <i>mỗi</i> thao tác; khấu hao và tỉ lệ cạnh tranh đều không đủ.
●● Cận dưới có điều kiện	Neo vào SETH hoặc 3SUM rồi quy dẫn: khoảng cách chính sửa không thể tính trong $O(n^{2-\epsilon})$ trừ khi SETH sai. Giá trị thực hành là biết khi nào nên ngừng tìm thuật toán nhanh hơn.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Cận dưới có ích gì cho công việc hằng ngày của tôi?

| Chủ yếu ở hai chỗ. Thứ nhất, nó cho bạn biết khi nào nên ngừng tìm — nếu bài của bạn quy về sắp xếp thì đừng tìm lời giải tuyến tính trong mô hình so sánh. Thứ hai, và hữu ích hơn, nó chỉ ra *giả định nào cần phá* để đi nhanh hơn: khoá có phải số nguyên nhỏ không (thì dùng sắp xếp đếm), dữ liệu có gần như đã sắp không (thì có thuật toán thích ứng), bạn có chấp nhận kết quả xấp xỉ không. Cận dưới không chỉ nói “không được” mà nói “không được *trong luật này*”.

2. Làm sao biết chương trình của tôi đang bị chặn bởi bộ nhớ chứ không phải bởi tính toán?

| Hai kiểm tra rẻ. Một: giảm kích thước dữ liệu xuống mức vừa trong bộ nhớ đệm và xem thời gian trên mỗi phần tử có giảm mạnh không — nếu có, bạn đang bị chặn bởi bộ nhớ. Hai: thay đổi *thứ tự truy cập* mà giữ nguyên số phép toán, ví dụ duyệt ma trận theo hàng thay vì theo cột; nếu thời gian đổi nhiều lần thì chi phí thật nằm ở truy cập. Đây là hai thí nghiệm nên làm trước khi tối ưu bất cứ thứ gì (E/26).

3. Thuật toán trực tuyến có xuất hiện trong bài tập không, hay chỉ là lý thuyết?

| Có, dưới hai dạng. Dạng thứ nhất là các bài yêu cầu trả lời truy vấn ngay mà không được xử lý ngoại tuyến — lúc đó các kỹ thuật như thuật toán Mo (B/10) không dùng được và bạn phải chọn cấu trúc dữ liệu khác. Dạng thứ hai là các bài mô phỏng chính sách quyết định, ví dụ quản lý bộ đệm.

Ngoài thi đấu thì đây là mô hình rất thực: mọi hệ thống phải quyết định khi chưa biết yêu cầu tiếp theo.

4. Bộ nhớ đệm LRU có tối ưu không?

| Không tối ưu, và người ta biết chính xác nó kém bao nhiêu. Chính sách tối ưu là loại bỏ trang sẽ được dùng lại xa nhất trong tương lai — không cài được vì cần biết tương lai. LRU có tỉ lệ cạnh tranh bằng kích thước bộ đệm trong trường hợp xấu nhất, nhưng trong thực tế nó chạy tốt vì các mẫu truy cập thật có tính cục bộ thời gian. Đây là ví dụ điển hình cho việc phân tích trường hợp xấu nhất quá bi quan, và là lý do các phân tích tinh hơn ra đời.

5. Nếu tôi muốn tăng tốc bằng song song, nên bắt đầu từ đâu?

| Bắt đầu bằng việc tính *chiều sâu* của thuật toán: nếu nó có một chuỗi phụ thuộc dài thì song song hoá không cứu được và bạn cần đổi thuật toán trước. Sau đó chọn nơi song song hoá ở mức thô nhất có thể — các nhánh trên cùng của chia để trị, các phần tử độc lập của một vòng lặp lớn — vì song song ở mức mịn thường bị chi phí điều phối nuốt hết. Cuối cùng, luôn đo trước và sau; kết quả thường khác dự đoán, đúng như bài học COST.

6. Thuật toán xấp xỉ trong mô hình luồng đáng tin tới đâu?

| Chúng đi kèm bảo đảm dạng “sai số tương đối không quá ε với xác suất ít nhất $1 - \delta$ ”, và bạn chọn ε, δ bằng cách trả thêm bộ nhớ. Đây là bảo đảm chứng minh được, không phải quan sát thực nghiệm — nên chúng đáng tin theo đúng nghĩa đã phát biểu. Điều cần cẩn thận trong thực tế: các bảo đảm này thường giả định các hàm băm đủ ngẫu nhiên và các phần tử của luồng không phụ thuộc vào lựa chọn ngẫu nhiên của bạn ([C/20](#)).

7. Có mô hình chi phí nào đang trở nên quan trọng mà sách giáo khoa chưa nói nhiều?

| Ít nhất hai. Năng lượng: trên thiết bị chạy pin và trong trung tâm dữ liệu, joule mỗi thao tác là ràng buộc thật, và nó không tỉ lệ với số phép toán — truy cập bộ nhớ tốn năng lượng hơn phép tính rất nhiều. Và tính toán trên dữ liệu được bảo vệ: khi phải tính trên dữ liệu mã hoá hoặc phân tán mà không lộ thông tin, chi phí của một phép toán thay đổi hoàn toàn, và các thuật toán tối ưu cho mô hình đó rất khác. Cả hai đều là ví dụ cho mẫu ở mục 6: mô hình mới ra đời khi một ràng buộc mới trở nên chi phối.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Chứng minh cận dưới $\Omega(n \log n)$ cho sắp xếp so sánh	[27]	Cây quyết định cộng công thức Stirling; ba câu là đủ.
● Chứng minh cần ít nhất $n - 1$ phép so sánh để tìm cực đại	Bài kinh điển	Lập luận đối thủ: mỗi phép so sánh loại tối đa một ứng viên.
●● Tìm đồng thời cực đại và cực tiểu với $\lceil 3n/2 \rceil - 2$ phép so sánh	Bài kinh điển	So từng cặp trước rồi mới so với kỷ lục; chứng minh cận dưới bằng lập luận đối thủ.
●● Sắp xếp một tệp lớn hơn bộ nhớ	Bài tập hệ thống	Trộn nhiều đường với heap (B/07); đo số lần đọc/ghi và so với dự đoán của mô hình bộ nhớ ngoài.
●● Đếm số phần tử phân biệt trong luồng bằng bộ nhớ nhỏ	[44]	Cài HyperLogLog đơn giản; đo sai số thực tế trên dữ liệu vài triệu phần tử.
●● Mô phỏng LRU và chính sách tối ưu trên cùng dãy truy cập	Bài tập thực nghiệm	Đo tỉ số trượt bộ đệm; đối chiếu với khái niệm tỉ lệ cạnh tranh.
●●● Nhân ma trận theo khối và đo hiệu ứng cache	Bài tập thực nghiệm	So ba cách: ngây thơ, đối thứ tự vòng lặp, chia khối. Số liệu sẽ nói rõ hơn mọi lý thuyết.
●●● Cài phiên bản cache-oblivious của một thuật toán quen thuộc	[78]	Ví dụ: chuyển vị ma trận bằng chia để trị; so với bản ngây thơ trên ma trận lớn.
●●● Phân tích bài thuê hay mua và chứng minh tỉ lệ 2 là tối ưu	Bài kinh điển	Dùng lập luận đối thủ: đối thủ chọn thời điểm dừng ngay sau khi bạn quyết định mua.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Đo tăng tốc thực tế của một chương trình song song theo số lõi	Bài tập thực nghiệm	Vẽ đường tăng tốc và ước lượng tỉ lệ phần tuần tự; đối chiếu với ba lý do ở mục 5.
●●● So sánh một hệ thống xử lý dữ liệu song song với một chương trình đơn luồng viết cẩn thận	[38]	Chọn một bài đồ thị vừa phải; kết quả thường gây ngạc nhiên và là bài học đáng nhớ nhất chương.

Kết chương và đọc tiếp

Phần D khép lại với một thông điệp thống nhất: mọi kết luận về hiệu năng đều là kết luận *trong một mô hình*. Chương D/23 nói về giới hạn do bản chất bài toán; chương này nói về giới hạn chứng minh được trong mô hình hạn chế, và về những mô hình khác nơi bảng xếp hạng thuật toán đảo lộn. Ba điều đáng mang theo: cận dưới luôn kèm mô hình và muốn vượt qua thì phải phá đúng giả định; đơn vị chi phí quyết định thuật toán tối ưu; và khi lý thuyết mâu thuẫn thực nghiệm, hãy nghi mô hình trước.

Phần E chuyển hẳn sang thực hành. Bốn chương cuối nói về khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng, đúng hạn”: cài đặt và gỡ lỗi (E/25), thuật toán trong hệ thống thật (E/26), chiến thuật phỏng vấn và thi đấu (E/27), và cuối cùng là lộ trình luyện tập kèm ngân hàng bài tập để mang theo sau khi gấp sách (E/28).

I Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Backurs & Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False)”	SIAM Journal on Computing, 2018 (bản hội nghị STOC 2015)	Đọc phần mở đầu và sơ đồ quy dẫn từ bài vectơ trực giao; đây là ví dụ mẫu mực của cận dưới có điều kiện.
nền tảng	Impagliazzo & Paturi, “On the Complexity of k -SAT”	Journal of Computer and System Sciences, 2001	Nơi ETH và SETH được phát biểu. Đọc để biết chính xác giả thuyết neo nói gì — và không nói gì.
tra khi cần	Gajentaan & Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry”	Computational Geometry: Theory and Applications, 1995	Danh mục các bài 3SUM-khó trong hình học; tra khi bài của bạn có mùi “ba đối tượng thẳng hàng”.
tra khi cần	Frigo, Leiserson, Prokop, Ramachandran, “Cache-Oblivious Algorithms”	FOCS, 1999	Mô hình chi phí thay thế cho mô hình RAM, dùng cho mục 3 của chương này.
tra khi cần	Borodin & El-Yaniv, <i>Online Computation and Competitive Analysis</i>	Cambridge University Press, 1998	Sách chuẩn cho tỉ lệ cạnh tranh ở mục 4; đọc chương về bài toán thuê hay mua và về bộ nhớ đệm.

HÀNH NGHỀ VÀ LUYỆN TẬP



TÓM TẮT

Phần E là khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng, đúng hạn”. Chương 25 nói về cài đặt, gỡ lỗi và kiểm thử — trong đó stress test là kỹ thuật đáng giá nhất mà rất nhiều người học muộn. Chương 26 nói về thuật toán trong hệ thống thật, nơi hằng số, bộ nhớ và tính bảo trì thường thắng độ phức tạp tiệm cận. Chương 27 là chiến thuật phỏng vấn và thi đấu: quản lý thời gian, chọn bài, nói ra suy nghĩ. Chương 28 là lộ trình luyện tập và ngân hàng bài tập lớn để mang theo sau khi gấp sách. Nếu chỉ nhớ một điều: năng lực giải thuật tăng theo số bài bạn *giải xong sau khi đã bí*, không theo số bài bạn đọc lời giải.

Phần này tồn tại vì một quan sát khó chịu: rất nhiều người hiểu thuật toán tốt hơn hẳn kết quả họ đạt được. Nguyên nhân gần như luôn nằm ở bốn chỗ, và bốn chương của phần E ứng đúng với bốn chỗ đó. Thứ nhất, code có bug mà không có quy trình tìm bug — chương 25. Thứ hai, chọn sai thứ để tối ưu trong bối cảnh thật, ví dụ đổi cấu trúc dữ liệu để giảm độ phức tạp trong khi nút cổ chai nằm ở truy cập bộ nhớ — chương 26. Thứ ba, chiến thuật kém: dành 50 phút cho bài khó nhất rồi không kịp làm hai bài dễ — chương 27. Thứ tư, luyện tập không có cấu trúc: giải nhiều mà không tiến, vì luôn giải những bài vừa sức thay vì những bài hơi quá sức — chương 28.

Ba chương đầu ngắn và thực dụng; chương 28 dài vì nó chứa ngân hàng bài tập được phân loại theo chương, kèm gợi ý và lộ trình theo tuần. Đó là chương bạn sẽ mở lại nhiều lần nhất sau khi đã đọc xong cả quyển.

Bản đồ phần E — Hành nghề và luyện tập

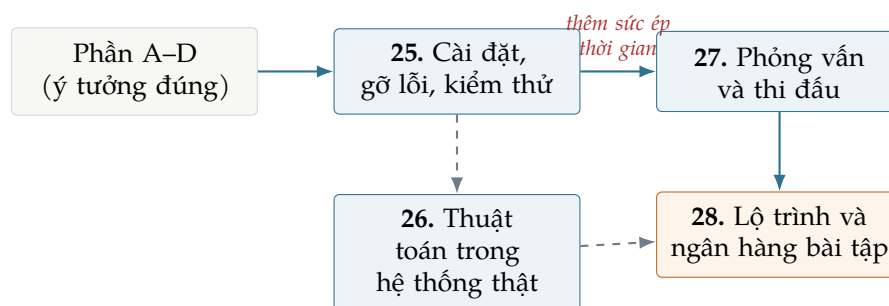
TÓM TẮT

Bốn chương ứng với bốn chỗ hay hổng giữa “hiểu thuật toán” và “có kết quả”: bug không tìm được (25), tối ưu sai chỗ trong hệ thống thật (26), chiến thuật kém khi có giới hạn thời gian (27), và luyện tập không có cấu trúc (28). Không có phụ thuộc bắt buộc giữa bốn chương; đọc theo nhu cầu. Chương 28 là chương dùng lại nhiều nhất về lâu dài.

A. Phần này có gì

- **25-cai-dat-go-loi-kiem-thu** — viết code đúng ngay từ đầu, danh mục bug kinh điển, khẳng định và bất biến trong code, stress test bằng sinh test ngẫu nhiên, thu nhỏ phân ví dụ, đo thời gian.
- **26-thuat-toan-trong-he-thong-that** — vì sao $O(n \log n)$ chậm hơn $O(n^2)$ ở n nhỏ, chi phí bộ nhớ và cache, đo trước khi tối ưu, thuật toán trong cơ sở dữ liệu, trình biên dịch, mạng, đồ hoạ và robot.
- **27-phong-van-va-thi-dau** — quy trình 45 phút của một buổi phỏng vấn, cách nói ra suy nghĩ, chọn bài và phân bổ thời gian trong contest, xử lý lúc bí, đọc bảng điểm.
- **28-lo-trinh-va-ngan-hang-bai-tap** — lộ trình 12 tuần và lộ trình 6 tháng, nguyên tắc up-solving, nhật ký lỗi, lập lại ngắt quãng, và ngân hàng bài tập phân loại theo 27 chương trước.

B. Phụ thuộc và thứ tự đọc



Hình 24.2. Phần E nhận đầu vào từ toàn bộ phần A–D và chia làm hai nhánh: nhánh kỹ thuật (25–26) và nhánh chiến thuật cùng luyện tập (27–28). Chương 28, tô hồ phách, là nơi mọi thứ quay về thành lịch luyện cụ thể.

C. Cốt lõi hay tham khảo

Cốt lõi: stress test và danh mục bug ở chương 25; nguyên tắc “đo trước khi tối ưu” ở chương 26; quy trình nói ra suy nghĩ ở chương 27; nguyên tắc upsolving và nhật ký lỗi ở chương 28. **Tham khảo:** các mục khảo sát thuật toán trong từng ngành công nghiệp ở chương 26 — đọc để biết bức tranh, không cần thuộc.

Phần thực hành bổ sung: [E/25](#) có bảng chọn lớp kiểm thử và phân biệt chia đôi với thu nhỏ phần ví dụ; [E/26](#) có phiếu benchmark và phép tính giới hạn tăng tốc toàn pipeline.

Cài đặt, gỡ lỗi và kiểm thử (Implementation, Debugging, Testing)

PHỤ THUỘC A/04 cho bất biến — công cụ phòng lỗi mạnh nhất; C/13 cho lời giải vét cạn, thứ đóng vai trò trọng tài trong stress test.

TÓM TẮT

Chương này nói về khoảng cách giữa “tôi biết phải làm gì” và “chương trình chạy đúng”. Ba phần: viết code sao cho ít lỗi ngay từ đầu — chủ yếu bằng cách viết bất biến trước khi viết vòng lặp; danh mục các lỗi kinh điển kèm cách phòng từng loại; và quan trọng nhất, một quy trình gỡ lỗi có hệ thống thay cho việc sửa mò. Trung tâm của quy trình đó là *stress test*: sinh dữ liệu nhỏ ngẫu nhiên, chạy song song lời giải nhanh và lời giải vét cạn, so kết quả, rồi thu nhỏ phản ví dụ. Nếu chỉ nhớ một điều: khi bạn không biết vì sao code sai, đừng đọc lại code — hãy để máy tìm cho bạn một phản ví dụ nhỏ, vì mười phút viết bộ sinh test gần như luôn rẻ hơn một giờ nhìn chằm chằm màn hình.

KIẾN THỨC CẦN CÓ TRƯỚC

- **stress test** — chạy hai lời giải — một nhanh, một chậm nhưng chắc đúng — trên hàng nghìn dữ liệu nhỏ sinh ngẫu nhiên, và dừng lại ở lần đầu hai kết quả khác nhau.
- **thu nhỏ phản ví dụ** — từ một dữ liệu làm chương trình sai, tìm dữ liệu nhỏ hơn vẫn giữ được lỗi — vì lỗi trên dữ liệu ba phần tử dễ hiểu hơn nhiều so với lỗi trên ba nghìn phần tử.
- **khẳng định (assertion)** — câu lệnh kiểm tra một điều kiện phải đúng tại một điểm trong chương trình; nếu sai thì dừng ngay tại chỗ phát sinh thay vì để lỗi lan ra.
- **lỗi im lặng** — lỗi không làm chương trình sập mà chỉ làm kết quả sai — loại nguy hiểm nhất, vì nó chỉ lộ ra khi có người đối chiếu.
- **dữ liệu biên** — các trường hợp ở ranh giới: rỗng, một phần tử, mọi phần tử bằng nhau, giá trị lớn nhất, giá trị âm.
- **tái lập** — khả năng chạy lại và gặp đúng lỗi cũ; với chương trình có ngẫu nhiên, điều này đòi phải ghi lại hạt giống.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “đọc lại code” là chiến lược gỡ lỗi kém, dù đó là việc ai cũng làm đầu tiên?
- Q2. Stress test cần một lời giải vét cạn — nghe như làm gấp đôi công việc. Vì sao nó vẫn tiết kiệm thời gian?
- Q3. Vì sao lỗi trên dữ liệu nhỏ lại dễ sửa hơn lỗi trên dữ liệu lớn tới mức đáng bỏ công thu nhỏ phản ví dụ?
- Q4. Loại lỗi nào sống sót qua mọi bộ test ngẫu nhiên, và phải phòng bằng cách khác?
- Q5. Khi chương trình có yếu tố ngẫu nhiên, làm sao gỡ lỗi được?

Có một cảnh quen thuộc: bạn nộp bài, nhận về “sai ở test 12”, và bắt đầu đọc lại code. Đọc ba lần, mọi dòng đều hợp lý. Bạn đổi một dấu < thành <=, nộp lại, vẫn sai. Đổi lại như cũ, thêm một trường hợp riêng, vẫn sai. Nửa giờ trôi qua và bạn không biết mình đang tiến hay lùi.

Vấn đề của cảnh này không phải kỹ năng lập trình mà là *thiếu quy trình*. Đọc lại code là cách gỡ lỗi kém vì hai lý do: bạn đọc lại chính suy nghĩ của mình, nên bạn sẽ đọc thấy điều bạn *định* viết chứ không phải điều bạn *đã* viết; và bạn không có thông tin mới nào — đúng tình trạng “lặp lại một biểu diễn” mà chương A/01 đã mô tả.

Chương này thay cảnh đó bằng một quy trình. Quy trình ấy dựa trên một nguyên tắc duy nhất: *hãy để máy tìm cho bạn một ví dụ sai nhỏ, rồi mới dùng đầu*.

25.1 Phòng bệnh: viết code ít lỗi ngay từ đầu

Cách rẻ nhất để sửa lỗi là không tạo ra nó. Bốn thói quen có hiệu quả rõ rệt.

Viết bất biến trước khi viết vòng lặp. Đây là lời khuyên của chương A/04 và nó thuộc về chương này nhiều nhất. Một dòng chú thích — “sau vòng lặp thứ i , $best$ là đáp án của $a[0..i]$ ” — biến mọi quyết định về chỉ số từ đoán mò thành suy diễn. Phần lớn lỗi lệch một đơn vị chết ngay tại đây.

Chọn quy ước và giữ nhất quán. Đánh chỉ số từ 0 hay từ 1, đoạn đóng $[l, r]$ hay nửa mở $[l, r)$ — chọn một và dùng cho toàn bài. Phần lớn lỗi biên đến từ việc trộn hai quy ước trong cùng một chương trình. Nửa mở là quy ước ít gây lỗi hơn cho các bài chia đoạn, vì độ dài bằng đúng $r - l$ và phép chia đôi không có trường hợp riêng.

Ưu tiên rõ ràng hơn ngắn. Một dòng code khéo léo tiết kiệm ba dòng nhưng tốn hai mươi phút khi phải gỡ lỗi. Trong thi đấu điều này còn quan trọng hơn ngoài đời, vì bạn không có thời gian đọc lại thứ mình không hiểu ngay.

Đặt khẳng định ở những chỗ bạn tin chắc. Nghịch lý là ở chỗ: nơi đáng đặt khẳng định nhất chính là nơi bạn *chắc chắn* nó luôn đúng, vì đó là những giả định ngầm mà bạn sẽ không bao giờ nghĩ tới khi gỡ lỗi. Khẳng định làm lỗi dừng lại tại nơi phát sinh thay vì để nó lan ra và chỉ lộ ở kết quả cuối.

25.2 Danh mục lỗi kinh điển

Phần lớn lỗi trong lập trình thuật toán rơi vào một số ít loại. Thuộc danh mục này giúp bạn kiểm nhanh trước khi nộp, và cột thứ ba là cách phòng chứ không phải cách sửa.

Loại lỗi	Dấu hiệu	Cách phòng
Tràn số	Kết quả âm bất ngờ, hoặc sai chỉ với dữ liệu lớn	Ước lượng giá trị lớn nhất <i>trước</i> ; mặc định dùng 64 bit cho tổng và tích
Lệch một đơn vị	Sai đúng ở phần tử đầu hoặc cuối	Viết bất biến; chọn một quy ước đoạn và giữ nhất quán
Chưa khởi tạo	Kết quả đổi giữa các lần chạy	Khởi tạo tường minh; đặt khẳng định về miền giá trị hợp lệ
Hàm so sánh không nhất quán	Sắp xếp cho kết quả lạ, đôi khi sập	Kiểm quan hệ có bắc cầu; không dùng “gần bằng” làm khoá sắp xếp
Đệ quy quá sâu	Sập với dữ liệu lớn, chạy tốt với dữ liệu nhỏ	Ước lượng độ sâu tối đa; chuyển sang bản lặp khi cần
Chia số nguyên	Sai với số âm hoặc mất phần lẻ	Nhân chéo thay vì chia; cẩn thận với hướng làm tròn của số âm
Phần dư âm	Kết quả âm sau phép trừ modulo	Cộng thêm m rồi lấy dư lần nữa ($D/21$)
Trường hợp $n = 0$ hoặc $n = 1$	Sai đúng ở test đầu hoặc test biên	Luôn thử tay hai ca này trước khi nộp
Đọc dữ liệu chậm	Quá thời gian dù thuật toán đúng bậc	Dùng cách đọc nhanh khi n tới hàng triệu

Có một lỗi thứ mười không nằm trong bảng vì nó thuộc loại khác: *hiểu sai đề*. Nó chiếm tỉ lệ cao hơn nhiều người thừa nhận, và cách phòng duy nhất là bước 1 của Pólya — phát biểu lại đề bằng lời của mình và tính tay ví dụ trong đề (A/01).

25.3 Stress test: kỹ thuật đánh giá nhất chương

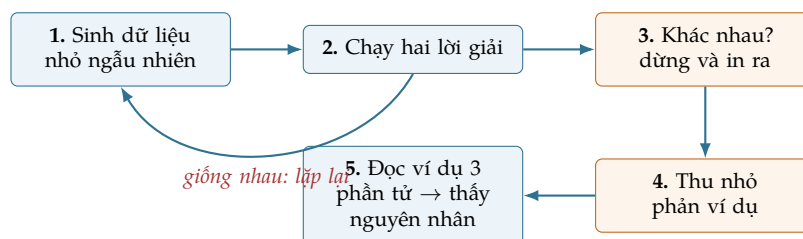
Đây là quy trình thay thế cho việc sửa mò, và nó gồm bốn phần.

Một — bộ sinh dữ liệu. Một chương trình nhỏ sinh dữ liệu ngẫu nhiên *hợp lệ* với kích thước *rất nhỏ*: $n \leq 8$, giá trị trong $[-3, 3]$. Kích thước nhỏ là điểm mấu chốt, vì nó vừa làm lỗi dễ hiểu vừa làm các trường hợp trùng lặp và biên xuất hiện thường xuyên thay vì hiếm.

Hai — lời giải trọng tài. Một lời giải vét cạn, chậm nhưng bạn tin chắc là đúng. Đây chính là lý do chương C/13 nói “luôn viết vét cạn trước”: nó không chỉ giúp bạn nghĩ, nó còn là công cụ kiểm thử.

Ba — vòng lặp so sánh. Chạy cả hai trên cùng dữ liệu, so kết quả, dừng lại ở lần đầu khác nhau và in dữ liệu đó ra. Vài dòng kịch bản, chạy vài nghìn ca trong vài giây.

Bốn — thu nhỏ. Khi đã có một dữ liệu sai, hãy tìm dữ liệu *nhỏ hơn* vẫn sai: thử bỏ từng phần tử, giảm từng giá trị, và giữ lại thay đổi nào vẫn giữ được lỗi. Việc này tự động hoá được bằng một vòng lặp đơn giản, và nó biến một phần ví dụ 8 phần tử thành 3 phần tử — ở kích thước đó bạn thường nhìn ra nguyên nhân ngay lập tức.



Hình 25.1. Vòng stress test. Điểm quan trọng nhất là bước 1 phải sinh dữ liệu *rất nhỏ* — điều này vừa làm phần ví dụ dễ đọc vừa làm các trường hợp biên xuất hiện liên tục thay vì hiếm. Bước 4 thường bị bỏ, nhưng nó là bước biến một phần ví dụ khó hiểu thành một phần ví dụ hiển nhiên.

Stress test không chỉ dùng để gỡ lỗi. Nó còn là cách kiểm chứng một *ý tưởng* trước khi tin vào nó: khi bạn nghĩ ra một tiêu chí tham lam (C/15) hoặc một công thức quy hoạch động, năm phút stress test cho câu trả lời chắc chắn hơn nhiều so với mười phút cố chứng minh trong đầu.

CẦN NHỚ

Khi bí trong lúc gỡ lỗi, thứ tự nên làm là: (i) kiểm dữ liệu biên bằng tay — $n = 0, 1$, mọi phần tử bằng nhau; (ii) viết stress test; (iii) thu nhỏ phản ví dụ; (iv) chỉ khi đó mới đọc code. Đảo ngược thứ tự này — đọc code trước — là cách phần lớn thời gian gỡ lỗi bị lãng phí.

25.4 Khi không có lời giải trọng tài

Đôi khi bài quá khó để viết vết cạn, hoặc kết quả không so sánh trực tiếp được (có nhiều đáp án đúng). Vẫn còn ba cách.

Kiểm tính chất thay vì kiểm giá trị. Nếu bài yêu cầu một cách sắp xếp thoả điều kiện, hãy viết một hàm *kiểm tra* kết quả có hợp lệ không thay vì so với đáp án. Đây là ý tưởng nền của kiểm thử dựa trên tính chất, và nó thường dễ hơn viết vết cạn nhiều.

So hai cách cài đặt khác nhau. Nếu bạn có hai thuật toán cho cùng bài — ví dụ một bản đệ quy và một bản lặp, hoặc một bản dùng cây phân đoạn và một bản dùng phân rã căn — so chúng với nhau. Hai bản có thể cùng sai vì dùng chung mô hình hoặc cùng một giả thiết. So chéo là bằng chứng bổ sung, không thay thế trọng tài độc lập và các bất biến.

Kiểm bất biến trong lúc chạy. Rải khăng định kiểm các bất biến ở những điểm then chốt, rồi chạy trên dữ liệu lớn. Cách này không cần trọng tài và bắt được lỗi ngay tại nơi cấu trúc dữ liệu bị hỏng.

25.5 Gỡ lỗi như một bài toán tìm kiếm

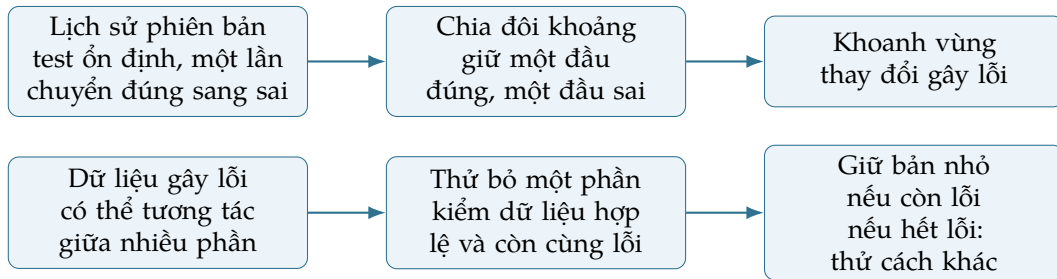
Khi đã có phản ví dụ nhỏ mà vẫn chưa thấy nguyên nhân, hãy coi việc gỡ lỗi là bài toán tìm kiếm — chọn phép thu hẹp theo cấu trúc của lỗi, thay vì mặc định mọi thứ đều đơn điệu.

Trên trục *không gian*: đặt các điểm kiểm tra ở giữa chương trình, xác nhận trạng thái tại đó đúng hay sai, rồi loại một nửa. Chỉ loại được một nửa nếu phép kiểm bảo đảm trạng thái lỗi không bị che hoặc sửa ở bước sau; nói chung trạng thái đúng/sai có thể xen kẽ.

Trên trục *thời gian*: nếu code trước đây chạy đúng, hãy tìm thay đổi đầu tiên làm hỏng bằng cách chia đôi lịch sử — đây chính là công cụ *git bisect*, và điều kiện dùng được của nó là tính đơn điệu, đúng như mọi tìm kiếm nhị phân khác.

Trên trục *dữ liệu*: đó chính là bước thu nhỏ phản ví dụ ở mục 3.

Hình 25.2 phân biệt phép chia đôi có bất biến với việc thu nhỏ dữ liệu phải kiểm lại sau mỗi lần thử.



Hình 25.2. Hai quy trình thu hẹp khác nhau. Trên: tìm ranh giới trong một lịch sử có giả thiết đơn điệu. Dưới: thu nhỏ phần ví dụ; không suy ra rằng một nửa đúng thì nửa còn lại chắc chắn chứa toàn bộ nguyên nhân.

Ba trục này dùng chung ý tưởng thu hẹp, nhưng không có cùng bảo đảm thuật toán, và nhận ra điều đó giúp bạn luôn có một nước đi tiếp theo thay vì bế tắc.

25.6 Ba việc riêng cho chương trình có ngẫu nhiên

Câu hỏi mở đầu thứ năm hỏi về việc gỡ lỗi khi chương trình có yếu tố ngẫu nhiên, và câu trả lời gồm ba việc rất cụ thể.

Ghi lại hạt giống. Mỗi lần chạy, in ra hạt giống đã dùng. Khi gặp lỗi, chạy lại với đúng hạt giống đó và bạn tái lập được. Không có bước này thì mọi lỗi phụ thuộc ngẫu nhiên đều gần như không sửa được.

Tách phần ngẫu nhiên khỏi phần logic. Nếu nguồn ngẫu nhiên được truyền vào như một tham số thay vì gọi trực tiếp trong hàm, bạn thay được nó bằng một dãy cố định khi kiểm thử.

Kiểm bằng thống kê, không bằng một lần chạy. Với thuật toán ngẫu nhiên, một lần chạy đúng không nói gì. Hãy chạy hàng nghìn lần và kiểm phân phối kết quả — ví dụ với một hàm trộn ngẫu nhiên, đếm tần suất mỗi hoán vị trên $n = 4$ và kiểm chúng gần đều nhau.

25.7 Biến mỗi lỗi thành một phép kiểm có thể chạy lại

Một lần sửa chỉ có giá trị lâu dài nếu phần ví dụ trở thành ca kiểm thử hồi quy. Bảng 25.1 đưa ra ba lớp kiểm bổ sung nhau. Với một bài trả về nhiều lỗi giải hợp lệ, hãy so giá trị mục tiêu và dùng bộ kiểm chứng tính hợp lệ, không so nguyên văn hai danh sách kết quả.

Bảng 25.1. Ba lớp kiểm thử và giới hạn của bằng chứng chúng cung cấp.

Lớp kiểm	Ví dụ	Bắt được	Chưa bảo đảm
Ca biên có chủ đích	Mảng rỗng, toàn âm, giá trị trùng, không có nghiệm	Sai khởi tạo và quy ước biên	Phối hợp bất ngờ giữa nhiều điều kiện
So với vết cặn	Duyệt mọi tập con ở $n \leq 12$ rồi so giá trị	Sai logic của thuật toán nhanh	Lỗi chỉ xuất hiện khi dữ liệu lớn
Quan hệ biến đổi	Sắp lại đầu vào không đối MST; đảo đoạn không đối tổng	Sai phụ thuộc thứ tự ngoài đặc tả	Đúng cho mọi dữ liệu

Khi test sai, lưu cả đầu vào cụ thể, kết quả mong đợi, kết quả thực tế và seed; seed một mình chưa đủ nếu generator hay phiên bản thư viện đã đổi. Thu nhỏ nhưng giữ đúng đặc tả: bỏ một cạnh có thể làm đồ thị không còn liên thông, khiến test không còn thuộc miền bài toán. Sau khi

sửa, chạy lại ca gốc, ca đã thu nhỏ và bộ đối chiếu; đừng chỉ chạy đúng ví dụ nhỏ vừa dùng để sửa.

25.8 Gốc rễ: từ khẳng định của Turing tới thu nhỏ tự động

Ý tưởng đặt khẳng định vào chương trình để kiểm tra trạng thái xuất hiện rất sớm — Turing đã dùng nó năm 1949 trong bài viết về kiểm tra một chương trình lớn, như chương A/04 đã kể. Nhưng phải mất nhiều thập kỷ để nó thành thói quen phổ biến, và lý do là bối cảnh: khi máy tính còn đắt, việc để chương trình tự dừng vì một khẳng định bị coi là lãng phí thời gian máy.

Câu nói của Dijkstra — kiểm thử cho thấy sự hiện diện của lỗi, không cho thấy sự vắng mặt — thường bị dùng như một lời chê kiểm thử, nhưng ý ông tinh tế hơn: kiểm thử là một công cụ *phát hiện* rất tốt và một công cụ *bảo chứng* rất tệ, nên đừng nhầm hai vai trò. Toàn bộ mục 3 của chương này khai thác đúng vai trò thứ nhất.

Ba kỹ thuật hiện đại đáng nhắc vì chúng đều là hình thức hoá của những việc mà người giải bài vẫn làm bằng tay. *Kiểm thử ngẫu nhiên có hệ thống* bắt đầu được nghiên cứu nghiêm túc từ đầu thập niên 1990, khi người ta phát hiện rằng ném dữ liệu ngẫu nhiên vào các tiện ích hệ thống làm sập một tỉ lệ đáng ngạc nhiên trong số chúng — một kết quả gây sốc vì các phần mềm đó đã được dùng hàng ngày trong nhiều năm. *Kiểm thử dựa trên tính chất* ra đời cuối thập niên 1990 với ý tưởng: thay vì viết từng ca kiểm thử, hãy phát biểu một tính chất phải luôn đúng và để máy sinh dữ liệu tìm phản ví dụ — đúng mục 4 ở trên. Và *thu nhỏ tự động* được hình thức hoá cuối thập niên 1990 dưới tên gỡ lỗi delta: một quy trình thử các tập con và phần bù để tìm phản ví dụ 1-tối-tiểu (bỏ thêm một đơn vị thì không còn tái hiện lỗi), không bảo đảm nhỏ nhất toàn cục[80], tức là bước 4 ở mục 3 nhưng có phân tích chi phí đàng hoàng.

Điểm chung của ba kỹ thuật: chúng đều chuyển công việc từ người sang máy, và cả ba đều dựa trên cùng một quan sát — *máy tìm phản ví dụ giỏi hơn người, còn người hiểu phản ví dụ giỏi hơn máy*. Quy trình ở mục 3 chỉ là cách phân công đúng theo quan sát đó.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1949 — Turing	Kiểm tra một chương trình dài mà chạy thử không xuể.	Khẳng định tại từng điểm; ý tưởng có từ rất sớm nhưng chỉ thành thói quen phổ biến khi thời gian máy trở nên rẻ.
Thập niên 1970 — Dijkstra	“Khủng hoảng phần mềm”: lỗi vượt khả năng kiểm soát bằng chạy thử.	Kiểm thử là công cụ phát hiện lỗi, không phải bảo chứng — phân vai đúng thay vì bỏ kiểm thử.
Đầu thập niên 1990 — kiểm thử bằng dữ liệu ngẫu nhiên	Ném dữ liệu ngẫu nhiên vào các tiện ích hệ thống làm sập nhiều bất ngờ.	Chứng minh rằng phần mềm dùng hàng ngày vẫn đầy lỗi ở dữ liệu bất thường; sinh ra cả một nhánh kiểm thử tự động.
Cuối thập niên 1990 — gỡ lỗi delta	Phản ví dụ tìm được thường quá lớn để hiểu.	Thuật toán thử tập con và phần bù để thu nhỏ dữ liệu — chính là bước 4 của stress test, có phân tích đàng hoàng.
Khoảng 2000 — kiểm thử dựa trên tính chất	Viết từng ca kiểm thử vừa tốn công vừa bỏ sót.	Phát biểu tính chất phải luôn đúng, để máy sinh dữ liệu tìm phản ví dụ; cách làm khi không có lời giải trọng tài.

25.9 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kiểm chứng phần cứng	Mô phỏng ngẫu nhiên có ràng buộc, kiểm chứng hình thức	Cùng cấu trúc với stress test nhưng ở quy mô công nghiệp: sinh kích thích ngẫu nhiên hợp lệ và so với một mô hình tham chiếu — chính là “lời giải trọng tài” của mục 3.
Hàng không	Bảng kiểm trước khi cất cánh	Danh mục lỗi ở mục 2 chính là một bảng kiểm. Bài học từ ngành hàng không: bảng kiểm hiệu quả không phải vì phi công không biết, mà vì nó chống lại việc <i>bỏ sót dưới áp lực</i> — đúng tình huống trong kỳ thi.
Y khoa	Chẩn đoán phân biệt, xét nghiệm loại trừ	Gỡ lỗi bằng cách chia đôi vùng nghi ngờ giống hệt việc loại trừ dần các chẩn đoán. Cùng nguyên tắc: chọn phép thử nào loại được nhiều khả năng nhất, không phải phép thử dễ nhất.
Khoa học thực nghiệm	Tái lập kết quả, ghi sổ thí nghiệm	Việc ghi hạt giống ngẫu nhiên ở mục 6 chính là yêu cầu tái lập của khoa học. Một kết quả không tái lập được thì không kiểm chứng được — đúng với thí nghiệm và với lỗi phần mềm.
Ước lượng trạng thái, SLAM	Chạy trên đoạn dữ liệu ngắn, kiểm bất biến của bộ lọc	Áp dụng trực tiếp mục 3 và 4: dựng “ví dụ nhỏ nhất” là một đoạn vài giây thay vì cả chuỗi, và kiểm các bất biến (hiệp phương sai xác định dương, hướng trọng lực không trôi) bằng khẳng định trong lúc chạy.
Thống kê	Kiểm định giả thuyết	Việc kiểm thuật toán ngẫu nhiên bằng phân phối kết quả (mục 6) là một bài kiểm định thống kê. Cạm bẫy cũng giống: một lần quan sát không nói gì, và cần đủ số lần lặp để kết luận có nghĩa.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai lý do. Thứ nhất, bạn đọc lại chính suy nghĩ của mình nên bạn thấy điều bạn *định* viết chứ không phải điều bạn *đã* viết — đây là hiện tượng tâm lý rất mạnh và không khắc phục được bằng cách cố gắng hơn. Thứ hai, đọc lại không tạo ra thông tin mới; bạn đang lặp lại cùng một biểu diễn, đúng trạng thái bí mà A/01 mô tả. Stress test thì tạo ra thông tin mới: một dữ liệu cụ thể mà chương trình sai, và đó là thứ thu hẹp không gian nguyên nhân.
- A2. Vì lời giải vét cạn thường chỉ tốn năm tới mười phút — nó không cần nhanh, không cần đẹp, chỉ cần đúng — trong khi việc sửa mò một lỗi không hiểu có thể tốn hàng giờ và không có gì bảo đảm kết thúc. Ngoài ra bạn thường đã viết vét cạn rồi, vì C/13 khuyên viết nó như bước đầu của quá trình nghĩ. Và nó còn dùng lại cho các bài sau: một bộ sinh test và một khung stress test viết một lần dùng mãi.
- A3. Vì số lượng thứ có thể sai tỉ lệ với kích thước dữ liệu. Trên mảng ba phần tử, bạn theo dõi được toàn bộ trạng thái bằng mắt và thường nhìn ra nguyên nhân trong một phút. Trên mảng ba nghìn phần tử, bạn không biết bắt đầu từ đâu, và các lỗi khác nhau trông giống hệt nhau ở đầu ra. Việc thu nhỏ tự động hoá được bằng một vòng lặp thử bỏ từng phần tử, nên chi phí gần như bằng không so với lợi ích.
- A4. Lỗi ở trường hợp *biên* và trường hợp *cấu trúc đặc biệt* — mảng rỗng, một phần tử, mọi phần tử bằng nhau, đồ thị không liên thông, giá trị lớn nhất gây tràn. Bộ sinh ngẫu nhiên thông thường hầu như không tạo ra chúng. Cách phòng có hai phần: sinh dữ liệu với *miền giá trị rất nhỏ* để ép trùng lặp và biên xuất hiện thường xuyên; và bổ sung một danh sách ca biên viết tay chạy trước mọi lần nộp.

- A5.** Ba việc. Ghi lại hạt giống mỗi lần chạy và in ra, để tái lập được đúng lần chạy hỏng. Truyền nguồn ngẫu nhiên vào như tham số thay vì gọi trực tiếp, để thay được bằng dãy cố định khi kiểm thử. Và kiểm bằng thống kê thay vì bằng một lần chạy — chạy hàng nghìn lần rồi kiểm phân phối, vì với thuật toán ngẫu nhiên thì một lần đúng không chứng minh gì.

NÓI LẠI THẬT ĐƠN GIẢN

Khi chương trình sai mà bạn không biết vì sao, phản xạ tự nhiên là đọc lại code. Đó gần như luôn là cách chậm nhất, vì bạn sẽ đọc thấy cái mình *định* viết chứ không phải cái mình *đã* viết — giống như tự dò lỗi chính tả trong bài văn của chính mình.

Cách nhanh hơn nhiều là bắt máy tìm lỗi. Bạn viết thêm hai chương trình rất ngắn: một cái sinh ra dữ liệu nhỏ ngẫu nhiên, và một cái giải bài theo cách ngu ngốc nhất nhưng chắc chắn đúng — thử hết mọi khả năng. Rồi cho cả hai chạy hàng nghìn lần và so đáp án. Khi chúng khác nhau, bạn có trong tay một ví dụ cụ thể mà chương trình sai.

Còn một bước nữa mà nhiều người bỏ, và nó là bước đáng giá nhất: hãy làm cho ví dụ đó *nhỏ hết mức*. Thử bỏ bớt một phần tử, nếu vẫn sai thì bỏ tiếp. Một ví dụ sai với ba phần tử thường tự nói ra nguyên nhân; cùng cái lỗi ấy trên ba nghìn phần tử thì không nói gì cả.

Cả quy trình này chỉ tốn mười phút để viết lần đầu, và bạn dùng lại nó cho mọi bài sau. So với việc ngồi sửa mò nửa tiếng và không chắc đã xong, đó là một trong những đánh đổi tốt nhất trong nghề.

Cuối cùng, gỡ lỗi là tìm kiếm bằng các phép kiểm có thể lặp lại. Dùng chia đôi khi có ranh giới đúng/sai; với dữ liệu, thử thu nhỏ và chỉ giữ thay đổi còn tái hiện cùng lỗi. Một phản ví dụ không thu nhỏ thêm được theo các phép thử hiện có chưa chắc nhỏ nhất toàn cục.

Một câu: *Đừng đọc lại code khi bí — hãy để máy tìm cho bạn một ví dụ sai nhỏ.*

Chỗ không rút gọn được: Lỗi phụ thuộc thời gian hoặc phụ thuộc môi trường (nhiều luồng, phân cứng) không tái lập được bằng stress test thông thường; đó là một lớp khó riêng.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Vì sao không nên đọc lại code trước	Bạn đọc thấy điều mình định viết, và không tạo ra thông tin mới. Stress test tạo ra thông tin mới: một dữ liệu cụ thể sai.
••• Bốn bước stress test	Sinh dữ liệu <i>rất nhỏ</i> → chạy lời giải nhanh và vét cạn → dừng ở lần khác nhau đầu tiên → thu nhỏ phản ví dụ.
••• Vì sao dữ liệu phải nhỏ	Phản ví dụ dễ đọc, và các trường hợp trùng lặp/biên xuất hiện thường xuyên thay vì hiếm. Miền giá trị nhỏ cũng quan trọng như n nhỏ.
••• Thứ tự khi gỡ lỗi	Kiểm ca biên bằng tay → stress test → thu nhỏ → mới đọc code. Đảo ngược thứ tự này là cách lãng phí thời gian phổ biến nhất.
••• Gỡ lỗi là bài toán tìm kiếm	Chia đôi trên ba trục: không gian (điểm kiểm tra giữa chương trình), thời gian (git bisect), dữ liệu (thu nhỏ phản ví dụ).
••• Danh mục lỗi kinh điển	Tràn số; lệch một đơn vị; chưa khởi tạo; hàm so sánh không nhất quán; đệ quy quá sâu; chia số nguyên; phần dư âm; $n = 0, 1$; đọc dữ liệu chậm.
•• Khi không có lời giải trọng tài	Kiểm <i>tính chất</i> thay vì giá trị; so hai cách cài đặt khác nhau; rải khẳng định kiểm bất biến trong lúc chạy.
•• Nơi nên đặt khẳng định	Ở những chỗ bạn <i>chắc chắn</i> luôn đúng — vì đó là các giả định ngầm mà bạn sẽ không nghĩ tới khi gỡ lỗi.
•• Ba việc với chương trình ngẫu nhiên	Ghi và in hạt giống; truyền nguồn ngẫu nhiên như tham số; kiểm bằng phân phối trên nhiều lần chạy chứ không bằng một lần.
•• Quy ước đoạn	Chọn nửa mở $[l, r)$ hoặc đóng $[l, r]$ và giữ nhất quán cả bài; nửa mở ít gây lỗi hơn vì độ dài bằng $r - l$.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Stress test không chỉ để gỡ lỗi	Còn để <i>kiểm chứng ý tưởng</i> : năm phút stress test một tiêu chí tham lam chắc chắn hơn mười phút cố chứng minh trong đầu.
● Câu của Dijkstra, hiểu đúng	Kiểm thử là công cụ phát hiện lỗi rất tốt và công cụ bảo chứng rất tệ — đừng nhầm hai vai trò, không phải dùng kiểm thử.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Viết bộ sinh test tốn bao lâu, và có đáng trong kỳ thi không?

| Khoảng năm phút nếu bạn đã có khung sẵn, và đáng trong hầu hết tình huống. Phép so sánh đúng là: bạn đang cân nhắc giữa năm phút chắc chắn có tiến triển và một khoảng thời gian không xác định để sửa mò. Ngoại lệ hợp lý: khi bạn còn dưới mười phút và lỗi rõ ràng là lỗi cài đặt nhỏ. Lời khuyên thực dụng: chuẩn bị sẵn khung stress test trong thư viện cá nhân (A/02) để mỗi lần chỉ phải viết bộ sinh.

2. Độ quy của tôi sập với n lớn nhưng chạy tốt với n nhỏ. Sửa thế nào?

| Đó gần như chắc chắn là tràn ngăn xếp. Ba lựa chọn: tăng giới hạn ngăn xếp nếu môi trường cho phép; chuyển sang bản lập với ngăn xếp tường minh (B/07); hoặc giảm độ sâu bằng cách đổi thứ tự duyệt — ví dụ với cây, duyệt con nặng sau cùng. Cách kiểm nhanh: chạy thử với dữ liệu hình đường thẳng (cây suy biến thành chuỗi), vì đó là trường hợp độ sâu lớn nhất và bộ sinh ngẫu nhiên hầu như không tạo ra.

3. Làm sao biết lỗi là tràn số?

| Ba dấu hiệu. Kết quả âm ở nơi phải dương; kết quả đúng với dữ liệu nhỏ và sai với dữ liệu lớn; hoặc kết quả sai lệch một lượng gần bằng lũy thừa của 2. Cách kiểm nhanh nhất là đổi tạm mọi kiểu số sang 128 bit hoặc sang số nguyên lớn và chạy lại — nếu đúng thì bạn đã xác định được nguyên nhân. Cách phòng là ước lượng giá trị lớn nhất *trước khi cài*, cùng lúc với việc ước lượng độ phức tạp (A/03).

4. Tôi nên đặt bao nhiêu hằng định?

| Đủ để mỗi cấu trúc dữ liệu quan trọng được kiểm bất biến ít nhất một lần mỗi thao tác, nhưng không nhiều tới mức làm chậm chương trình đáng kể. Quy tắc thực dụng: đặt hằng định ở ranh giới giữa các phần — đầu vào của một hàm, sau khi cập nhật một cấu trúc — và ở những giả định ngầm. Nhớ tắt chúng trong bản nộp nếu chúng tốn thời gian; nhưng đừng xóa, chỉ tắt.

5. Chương trình chạy đúng trên máy tôi, sai trên máy chấm. Nguyên nhân hay gặp?

| Bốn nguyên nhân theo thứ tự tần suất. Biến chưa khởi tạo — giá trị rác khác nhau giữa hai môi trường. Hành vi không xác định như tràn số có dấu hoặc truy cập ngoài mảng — có thể “chạy đúng” ở nơi này và sai ở nơi khác. Khác biệt về kích thước kiểu số hoặc về cách làm tròn số thực. Và giới hạn ngăn xếp khác nhau. Cả bốn đều bắt được bằng cách bật các tùy chọn kiểm tra của trình biên dịch khi chạy thử ở nhà.

6. Có cách nào kiểm thử một lời giải mà bài có nhiều đáp án đúng?

| Có, và đó là mục 4: viết một hàm *kiểm tra tính hợp lệ* thay vì so với đáp án mẫu. Nếu bài yêu cầu một cách sắp xếp thỏa điều kiện, hàm kiểm tra chỉ cần duyệt và xác nhận mọi điều kiện được thỏa. Nếu bài yêu cầu một lời giải tối ưu, hãy kiểm hai điều: kết quả hợp lệ, và giá trị của nó bằng giá trị mà lời giải vét cạn tìm được — so *giá trị* chứ không so cấu hình.

7. Trong công việc thật, quy trình này thay đổi thế nào?

| Cùng nguyên tắc, khác quy mô. Lời giải vét cạn trở thành một mô hình tham chiếu đơn giản hoặc một phiên bản cũ đã tin cậy. Bộ sinh dữ liệu trở thành sinh dữ liệu có ràng buộc theo miền nghiệp vụ. Bước thu nhỏ trở thành việc rút gọn một ca lỗi từ hệ thống thật thành một ca tái lập được. Và có

thêm hai thứ mà thi đấu không có: kiểm thử hồi quy — mỗi lỗi đã sửa trở thành một ca kiểm thử vĩnh viễn — và ghi nhật ký đủ để tái lập sự cố từ hiện trường.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết khung stress test dừng lại được cho thư viện cá nhân	Bài tập công cụ	Gồm bộ sinh, vòng so sánh, và hàm thu nhỏ; đây là bài tập có lợi ích lâu dài nhất chương.
● Viết lại tìm kiếm nhị phân và stress test nó	[6]	So với thư viện chuẩn trên hàng nghìn ca ngẫu nhiên, gồm mảng rỗng và mảng trùng lặp.
●● Dừng stress test để tìm phản ví dụ cho một tiêu chí tham lam sai	C/15	Mục tiêu kép: luyện công cụ và luyện thói quen không tin cảm giác.
●● Viết hàm thu nhỏ tự động cho phản ví dụ dạng mảng	Bài tập công cụ	Thứ bỏ từng phần tử và giảm từng giá trị; giữ thay đổi nào vẫn còn lỗi.
●● Cố tình tạo một lỗi tràn số rồi tìm nó bằng quy trình ở mục 5	Bài tập luyện quy trình	Đi đúng thứ tự bốn bước; đo thời gian để thấy quy trình nhanh hơn sửa mò.
●● Chuyển một thuật toán đệ quy sâu sang bản lặp	Bài tập cài đặt	Duyệt sâu trên đồ thị 10^5 đỉnh hình đường thẳng; kiểm cả hai bản cho cùng kết quả.
●●● Viết bộ kiểm tra tính hợp lệ cho một bài có nhiều đáp án đúng	Bài tập kiểm thử	Ví dụ: sắp xếp tô-pô, ghép đôi, tô màu; kiểm mọi ràng buộc của đề.
●●● Kiểm một hàm trộn ngẫu nhiên bằng thống kê	Bài tập kiểm thử	Đếm tần suất mọi hoán vị với $n = 4$ trên 10^6 lần chạy; phát hiện lỗi lệch phân phối nếu có.
●●● Dừng git bisect để tìm thay đổi làm hỏng một bài trong kho mã của bạn	Bài tập công cụ	Trải nghiệm trực tiếp tìm kiếm nhị phân trên trục thời gian.
●●● Lập danh sách ca biên riêng cho ba dạng bài bạn hay làm	Bài tập tự tổ chức	Ví dụ: bài mảng, bài đồ thị, bài chuỗi; biến nó thành bảng kiểm trước khi nộp.

Kết chương và đọc tiếp

Chương này thay việc sửa mò bằng một quy trình. Ba điều đáng mang theo: thứ tự đúng khi gỡ lỗi — ca biên, stress test, thu nhỏ, rồi mới đọc code; ý thức rằng máy tìm phản ví dụ giỏi hơn người còn người hiểu phản ví dụ giỏi hơn máy, nên hãy phân công theo đúng thể mạnh; và việc chọn phép thu hẹp phù hợp trên ba trục không gian, thời gian và dữ liệu.

Chương tiếp theo mở rộng phạm vi: từ “chương trình chạy đúng” sang “chương trình chạy đủ nhanh trong một hệ thống thật”. Ở đó, các kết luận của phần A về độ phức tạp tiệm cận gấp thực tế của hằng số, bộ nhớ và tính bảo trì — và bạn sẽ thấy rằng thuật toán tốt nhất trên giấy thường không phải thuật toán được chọn (E/26).

Thuật toán trong hệ thống thật (Algorithms in Production)

PHỤ THUỘC A/03 cho khoảng cách giữa tiệm cận và hằng số; D/24 cho các mô hình chi phí khác; E/25 cho kỷ luật đo đạc.

TÓM TẮT

Chương này nói về chỗ mà lý thuyết gặp thực tế, và về vì sao thuật toán tốt nhất trên giấy thường không phải thuật toán được chọn. Bốn chủ đề: hằng số và bộ nhớ có tiếng nói riêng ở kích thước thật; kỷ luật đo trước khi tối ưu, vì trực giác về nút cổ chai gần như luôn sai; các tiêu chí ngoài tốc độ — độ trễ đuôi, khả năng bảo trì, tính dự đoán được — vốn quyết định nhiều hơn trong hệ thống thật; và một khảo sát ngắn về nơi các thuật toán của quyển sách này đang thực sự chạy. Nếu chỉ nhớ một điều: trong hệ thống thật, câu hỏi không phải “thuật toán nào nhanh nhất” mà là “ràng buộc thật sự của tôi là gì” — và câu trả lời thường không phải số phép toán.

KIẾN THỨC CẦN CÓ TRƯỚC

- **hồ sơ hiệu năng (profiling)** — đo xem thời gian thực sự bị tiêu ở đâu trong chương trình, thay vì đoán.
- **độ trễ đuôi** — thời gian phản hồi ở các phân vị cao — 99% hoặc 99,9% — thay vì trung bình; trong dịch vụ trực tuyến, đây mới là con số người dùng cảm nhận.
- **thông lượng so với độ trễ** — số việc làm được mỗi giây so với thời gian hoàn thành một việc; tối ưu cái này thường làm hại cái kia.
- **ngưỡng hoà vốn** — kích thước dữ liệu mà tại đó một thuật toán bậc thấp hơn bắt đầu thắng thuật toán có hằng số nhỏ hơn.
- **tối ưu hoá vi mô** — cải thiện hằng số mà không đổi thuật toán: đổi cấu trúc dữ liệu, giảm cấp phát, cải thiện kiểu truy cập bộ nhớ.
- **tính dự đoán được** — khả năng đảm bảo thời gian chạy cho *mỗi* lần gọi, không chỉ trung bình — yêu cầu đặc trưng của hệ thời gian thực.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao trực giác về “chỗ nào chậm” trong một chương trình lại gần như luôn sai?
- Q2. Khi nào một thuật toán có độ phức tạp tệ hơn lại là lựa chọn đúng trong sản phẩm thật?
- Q3. Vì sao độ trễ trung bình lại là con số gây hiểu lầm trong dịch vụ trực tuyến?
- Q4. Cùng một thuật toán, vì sao đổi cách sắp xếp dữ liệu trong bộ nhớ có thể làm nó nhanh gấp nhiều lần?
- Q5. Trong hệ thống thời gian thực, vì sao một cấu trúc có chi phí khấu hao rất tốt lại có thể là lựa chọn sai?

Có một khoảng cách mà người luyện thuật toán thường không thấy cho tới khi đi làm: trong kỳ thi, tiêu chí duy nhất là “đúng và trong giới hạn thời gian”. Trong hệ thống thật, có ít nhất năm tiêu chí cạnh tranh nhau — tốc độ trung bình, tốc độ ở trường hợp xấu, bộ nhớ, khả năng bảo

trì, và khả năng dự đoán — và thuật toán thắng cuộc là thuật toán cân bằng chúng, không phải thuật toán tối ưu một chiều.

Điều này không làm mất giá trị của phần A tới phần D. Ngược lại: bậc tiệm cận vẫn là thứ quyết định khi dữ liệu lớn lên, và không có nó thì bạn không có cách nào so sánh các hướng đi. Nhưng nó là *điều kiện cần chứ không đủ*, và chương này nói về phần còn thiếu.

26.1 Đo trước khi tối ưu

Đây là nguyên tắc quan trọng nhất chương, và lý do nó cần được nhắc lại là vì ai cũng đồng ý với nó mà rất ít người làm.

Câu của Knuth về tối ưu hoá sớm [10] thường bị trích cụt. Ý đầy đủ có hai vế: phần lớn thời gian ta nên bỏ qua các cải thiện nhỏ, *nhưng* không được bỏ lỡ cơ hội ở phần mã thực sự quan trọng — và cách biết phần nào quan trọng là *đo*, không phải đoán.

Vì sao trực giác sai? Ba lý do. Thứ nhất, chương trình hiện đại có nhiều tầng — thư viện, cấp phát bộ nhớ, hệ điều hành — và chi phí thường nằm ở tầng bạn không viết. Thứ hai, chi phí truy cập bộ nhớ không nhìn thấy được trong mã nguồn: hai vòng lặp trông giống hệt nhau có thể chênh nhau năm lần vì kiểu truy cập. Thứ ba, chúng ta có xu hướng chú ý tới đoạn mã *phức tạp* chứ không phải đoạn mã *chạy nhiều lần*, mà hai thứ đó thường không trùng nhau.

Quy trình tối thiểu: đo tổng thời gian trước; chạy hồ sơ hiệu năng để tìm phần chiếm nhiều nhất; chỉ tối ưu phần đó; đo lại để xác nhận. Bước cuối hay bị bỏ, và nó là bước quan trọng nhất — rất nhiều “tối ưu” hoá ra không cải thiện gì, hoặc làm chậm đi vì một hiệu ứng phụ.

26.2 Hằng số, bộ nhớ và những gì tiệm cận không nói

Ba nhóm yếu tố quyết định hiệu năng thật mà ký hiệu $O(\cdot)$ bỏ qua.

Kiểu truy cập bộ nhớ. Đây là yếu tố lớn nhất và bị đánh giá thấp nhất, đúng như hình phân cấp bộ nhớ ở chương A/03. Duyệt mảng tuần tự nhanh hơn nhiều lần so với duyệt danh sách liên kết cùng số phần tử; duyệt ma trận theo hàng nhanh hơn theo cột; cấu trúc cây cài bằng mảng nhanh hơn cài bằng con trỏ. Đây là câu trả lời cho câu hỏi mở đầu thứ tư, và nó cũng là lý do “tối ưu vi mô” đầu tiên nên thử luôn là đổi bố cục dữ liệu chứ không phải đổi thuật toán.

Cấp phát bộ nhớ. Cấp phát động trong vòng lặp là một trong những nguyên nhân chậm phổ biến nhất và ít bị nghi ngờ nhất. Cấp phát trước, dùng lại vùng nhớ, và tránh tạo đối tượng tạm là những cải thiện thường cho hệ số lớn mà không đổi một dòng logic nào.

Ngưỡng hoà vốn. Mọi thuật toán “tốt hơn về tiệm cận” đều có một kích thước dưới đó nó thua. Các thư viện thật xử lý điều này bằng cách kết hợp: hàm sắp xếp chuẩn chuyển sang sắp xếp chèn khi đoạn nhỏ [14], và chuyển sang heapsort khi độ sâu đệ quy vượt ngưỡng [37]. Đây là câu trả lời cho câu hỏi mở đầu thứ hai ở dạng cụ thể nhất: thuật toán “tệ hơn” được chọn khi n nhỏ, và trong thực tế n thường nhỏ hơn ta tưởng.

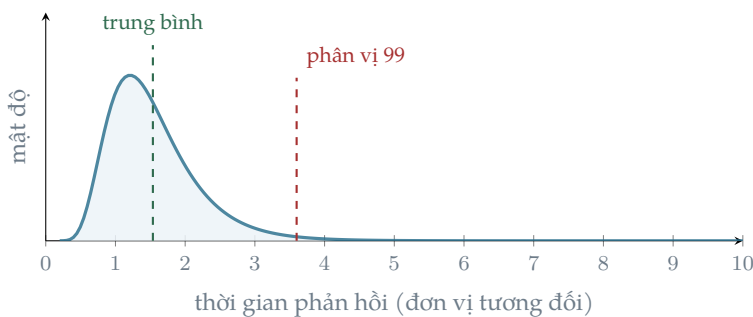
CẠM BẤY

Một cạm bẫy đặc thù của người mới đi làm sau khi luyện thi đấu: chọn cấu trúc dữ liệu tinh vi cho một bài mà dữ liệu chỉ có vài trăm phần tử. Một mảng phẳng và một vòng lặp tuyến tính thường vừa nhanh hơn vừa dễ bảo trì hơn một cây cân bằng tự cài. Câu hỏi đầu

tiên trong hệ thống thật không phải “cấu trúc nào tối ưu” mà là “dữ liệu lớn tới đâu, và nó sẽ lớn tới đâu trong hai năm nữa”.

26.3 Những tiêu chí ngoài tốc độ

Độ trễ đuôi. Trong một dịch vụ trực tuyến, một yêu cầu của người dùng thường phải chờ hàng chục thành phần con trả lời. Nếu mỗi thành phần chậm bất thường trong 1% số lần, thì xác suất *ít nhất một* thành phần chậm là rất cao — và người dùng cảm nhận đúng cái chậm nhất. Đó là lý do các hệ thống lớn đo và tối ưu theo phân vị 99 hoặc 99,9 thay vì theo trung bình. Hệ quả cho việc chọn thuật toán: một thuật toán có trung bình tốt nhưng đuôi xấu — ví dụ bảng băm phải băm lại toàn bộ khi vượt ngưỡng tải, hay bộ thu gom rác chạy đột ngột — có thể tệ hơn một thuật toán chậm hơn nhưng đều đặn.



Hình 26.1. Minh họa bằng phân phối log-normal với $\mu = 0.35$, $\sigma = 0.4$, không phải số đo hệ thống thật. Trung bình xấp xỉ 1.537 và phân vị 99 xấp xỉ 3.599: phần lớn yêu cầu nhanh, một số ít rất chậm. Khi một thao tác của người dùng chạm tới hàng chục thành phần, họ cảm nhận cái chậm nhất — nên phân vị 99 mới là con số phản ánh trải nghiệm, và tính *đều đặn* của thuật toán trở nên quan trọng ngang tốc độ trung bình.

Tính dự đoán được. Trong hệ thời gian thực — điều khiển robot, xử lý âm thanh, hệ thống an toàn — yêu cầu là *mỗi* chu kỳ phải hoàn thành trước hạn. Ở đó, chi phí khấu hao đẹp không cứu được bạn: một thao tác đột ngột tốn $O(n)$ làm lỡ hạn kỳ và hậu quả có thể là vật lý. Đây là câu trả lời cho câu hỏi mở đầu thứ năm, và nó dẫn tới các lựa chọn khác hẳn: cấu trúc có đảm bảo trường hợp xấu nhất, cấp phát tĩnh thay vì động, và tránh các cơ chế chạy nền không kiểm soát được thời điểm.

Khả năng bảo trì. Một thuật toán mà chỉ một người trong nhóm hiểu là một rủi ro vận hành. Trong thực tế, chênh lệch 20% tốc độ hiếm khi biện minh được cho việc thay một lời giải rõ ràng bằng một lời giải tinh vi. Nguyên tắc thực dụng: dùng cấu trúc phức tạp khi nó cho cải thiện *bậc*, không phải khi nó cho cải thiện hằng số.

Tính đúng dưới dữ liệu thù địch. Đây là chỗ chương [B/08](#) và [C/20](#) chạm vào hệ thống thật: nếu dữ liệu vào đến từ người dùng bên ngoài, phân tích trung bình không còn đủ. Bảng băm không ngẫu nhiên hoá, biểu thức chính quy có thể bị đưa vào trạng thái mũ, và các thuật toán sắp xếp có trường hợp xấu đoán được — cả ba đều đã từng bị khai thác để làm quá tải dịch vụ.

26.4 Thuật toán của quyển sách này đang chạy ở đâu

Mục này là khảo sát ngắn, đọc để có bức tranh chứ không cần thuộc.

Cơ sở dữ liệu là nơi tập trung nhiều thuật toán nhất: B-cây cho chỉ mục (B/09), sắp xếp ngoài và trộn nhiều đường (D/24), nối bằng băm và nối bằng trộn, và một bộ tối ưu truy vấn về bản chất là bài toán tìm kiếm trên không gian kế hoạch thực thi.

Trình biên dịch dùng đồ thị ở khắp nơi (C/17): đồ thị luồng điều khiển, sắp thứ tự tô-pô để xếp lịch lệnh, tô màu đồ thị để cấp phát thanh ghi — một bài NP-khó được giải bằng heuristic trong mọi lần biên dịch.

Mạng và hạ tầng dùng Dijkstra và Bellman–Ford ở dạng phân tán cho định tuyến, băm nhất quán cho phân mảnh dữ liệu (B/08), và các cấu trúc xấp xỉ cho giám sát lưu lượng (D/24).

Đồ hoạ và trò chơi dùng cấu trúc phân cấp không gian cho dò tia và va chạm (D/22), A* cho tìm đường, và rất nhiều tối ưu hằng số vì ràng buộc thời gian khung hình rất chặt.

Robot và hệ thống nhận thức dùng bài toán tối ưu trên đồ thị làm khung chính — đồ thị tư thế trong định vị và dựng bản đồ — cộng với cây k -d cho tìm lân cận, RANSAC cho loại ngoại lai (C/20), và A* cho lập kế hoạch. Điểm đáng chú ý: ở đây ràng buộc thời gian thực và ràng buộc năng lượng thường quyết định nhiều hơn bậc tiệm cận, đúng như mục 3 nói.

Học máy dùng đại số tuyến tính và tối ưu liên tục làm lõi, nhưng các thuật toán rời rạc của quyển sách này vẫn có mặt ở khắp phần hạ tầng: lấy mẫu, chỉ mục vector cho tìm lân cận gần đúng, cấu trúc dữ liệu cho tải dữ liệu, và các bài chọn tập con có tính dưới mô-đun (C/15).

26.5 Một phiếu đo trước khi quyết định tối ưu

Với một pipeline xử lý khung hình, hãy đo riêng đọc dữ liệu, tiền xử lý, thuật toán lõi và xuất kết quả, rồi đo lại toàn pipeline. Giả sử phép tìm lân cận chiếm 30% tổng thời gian: tăng tốc phần này hai lần chỉ cho hệ số toàn cục

$$\frac{1}{(1 - 0.3) + 0.3/2} \approx 1.18.$$

Ngay cả làm phần đó nhanh vô hạn cũng chỉ đạt $1/0.7 \approx 1.43$ lần. Đây là một phép tính từ tỉ phần thời gian giả định, không phải số đo benchmark.

Bảng 26.1. Phiếu đo để so sánh ba thay đổi thường gặp. Luôn dùng cùng tập đầu vào và kiểm kết quả trước khi so tốc độ.

Thay đổi	Dự đoán cần kiểm	Đo thêm	Đánh đổi
Mảng phẳng thay nút cấp phát riêng	Giảm cấp phát và cache miss	Bộ nhớ đỉnh, thời gian xây	Chi phí chèn/xoá, di chuyển dữ liệu
Tiền xử lý chỉ mục	Truy vấn rẻ hơn sau khi trả chi phí xây	Tổng thời gian theo số truy vấn q	Chỉ mục mất hiệu lực khi dữ liệu đổi
Xấp xỉ tìm lân cận	Đổi chất lượng lấy độ trễ thấp hơn	Recall so với tìm chính xác, p95/p99	Sai khác có thể ảnh hưởng bước tối ưu phía sau

Bảng 26.1 buộc mỗi tối ưu có một dự đoán có thể bác bỏ. Ghi kích thước và phân phối dữ liệu, phiên bản mã, chế độ biên dịch, số luồng, phần cứng và số lần lặp. Tách phép đo cache nóng/lạnh; chạy xen kẽ bản cũ và mới để giảm sai lệch do nhiệt độ hoặc tải nền. Với robot, kiểm cả tỉ lệ trễ hạn trên toàn pipeline: p99 thấp không đồng nghĩa mọi chu kỳ đều kịp hạn.

26.6 Gốc rễ: từ “đừng tối ưu sớm” tới “đuôi mới là thứ quan trọng”

Câu chuyện về mối quan hệ giữa lý thuyết và thực hành trong ngành này đi qua vài giai đoạn khá rõ.

Thập niên 1970 là giai đoạn mà tối ưu hoá thủ công còn phổ biến và thường phản tác dụng. Knuth viết câu nổi tiếng về tối ưu hoá sớm trong bối cảnh đó[10], và Bentley viết cả một quyển sách về việc viết chương trình hiệu quả[81] với thông điệp cân bằng: có kỹ thuật thật, nhưng phải áp dụng ở đúng chỗ tìm được bằng đo đạc.

Thập niên 1990 mang tới một bài học khác từ việc kỹ thuật hoá các thư viện chuẩn. Bài viết của Bentley và McIlroy về hàm sắp xếp[14] là tài liệu đáng đọc vì nó cho thấy khoảng cách giữa thuật toán trong sách và hàm dùng được: phải xử lý dữ liệu đã sắp sẵn, dữ liệu có nhiều phần tử bằng nhau, dữ liệu thù địch, và phải chuyển thuật toán theo kích thước. Cùng thời kỳ, introsort[37] giải quyết vấn đề trường hợp xấu bằng cách kết hợp ba thuật toán.

Từ giữa thập niên 2000, khi dịch vụ trực tuyến quy mô lớn trở nên phổ biến, một mối quan tâm mới nổi lên: *độ trễ đuôi*. Nhận thức rằng trung bình là con số gây hiểu lầm khi một yêu cầu phụ thuộc vào hàng chục thành phần đã thay đổi cách các hệ thống lớn được thiết kế và đo lường — và nó cũng thay đổi tiêu chí chọn thuật toán, đưa tính đều đặn lên ngang hàng với tốc độ trung bình.

Cuối cùng, bài báo về chi phí của khả năng mở rộng[38] là một lời nhắc có tính hệ thống: nhiều hệ thống xử lý dữ liệu song song, khi đo công bằng, thua một chương trình đơn luồng viết cẩn thận. Thông điệp không phải “đừng song song hoá” mà là “hãy đo so với một mốc trung thực” — và mốc đó thường là một cài đặt đơn giản, tốt, chạy trên một máy.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1974 — Knuth	Tối ưu hoá thủ công tràn lan làm mã khó đọc mà không nhanh hơn.	“Đừng tối ưu sớm” — nhưng về sau bị quên: phải đo để tìm phần thực sự quan trọng và tối ưu ở đó.
1982 — Bentley	Cần một cách có hệ thống để tăng tốc chương trình.	Danh mục kỹ thuật kèm nguyên tắc: chỉ áp dụng ở nơi đo đạc chỉ ra.
1993 — Bentley & McIlroy	Hàm sắp xếp trong thư viện chuẩn hỏng trên dữ liệu thật.	Kỹ thuật hoá: xử lý dữ liệu đã sắp, nhiều phần tử bằng nhau, và chuyển thuật toán theo kích thước.
1997 — Musser	Quicksort có trường hợp xấu $O(n^2)$ khai thác được.	Introsort: chuyển sang heapsort khi đệ quy quá sâu — kết hợp ba thuật toán để có cả tốc độ lẫn đảm bảo.
Giữa thập niên 2000 — dịch vụ quy mô lớn	Một yêu cầu phụ thuộc hàng chục thành phần; trung bình tốt mà người dùng vẫn thấy chậm.	Độ trễ đuôi trở thành tiêu chí chính; tính đều đặn được coi trọng ngang tốc độ trung bình.
2015 — McSherry và cộng sự	Hệ thống phân tán được đánh giá bằng khả năng mở rộng chứ không bằng tốc độ tuyệt đối.	So với mốc trung thực — một cài đặt đơn luồng tốt — nhiều hệ thống song song thua.

I 26.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Kỹ thuật hệ thống	Đo tải, phân vị độ trễ, ngân sách lỗi	Cùng nguyên tắc “đo trước khi quyết”. Điểm khác so với thi đấu: mục tiêu không phải giá trị nhỏ nhất mà là <i>đạt một ngưỡng cam kết một cách ổn định</i> .
Sản xuất công nghiệp	Lý thuyết nút cổ chai: cải thiện chỗ không phải nút thắt thì không tăng sản lượng	Trùng khít với nguyên tắc hồ sơ hiệu năng ở mục 1. Bài học chung: tối ưu ngoài nút cổ chai vừa vô ích vừa che mắt vấn đề thật.
Kinh tế học	Chi phí cơ hội, lợi ích cận biên giảm dần	Mỗi giờ tối ưu hằng số là một giờ không dùng cho việc khác. Trong hệ thống thật, “đủ nhanh” là một trạng thái hợp lệ, khác hẳn tinh thần thi đấu.
Hệ thời gian thực và robot	Ràng buộc hạn kỳ cứng, phân tích thời gian chạy xấu nhất	Ở đây tiêu chí là <i>trường hợp xấu nhất mỗi chu kỳ</i> , nên các cấu trúc có khâu hao đẹp bị loại. Đây là mô hình chi phí riêng mà D/24 đã nhắc.
Kiến trúc máy tính	Bộ nhớ đệm, dự đoán rẽ nhánh, xử lý theo lô	Giải thích vì sao kiểu truy cập bộ nhớ quan trọng hơn số phép toán ở kích thước thật. Hiểu sơ bộ tăng này biến “tối ưu vi mô” từ mơ mẫm thành có định hướng.
Hàng không, y tế	Thiết kế cho trường hợp xấu, không cho trường hợp trung bình	Cùng tinh thần với tính dự đoán được ở mục 3: trong hệ thống mà hậu quả nghiêm trọng, đảm bảo quan trọng hơn hiệu năng trung bình.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì ba lý do. Chi phí thường nằm ở tầng bạn không viết — thư viện, cấp phát bộ nhớ, hệ điều hành. Chi phí truy cập bộ nhớ không nhìn thấy được trong mã nguồn, nên hai vòng lặp giống hệt nhau có thể chênh nhiều lần. Và chúng ta chú ý tới đoạn mã *phức tạp* thay vì đoạn mã *chạy nhiều lần*, mà hai thứ đó thường khác nhau. Đó là lý do quy trình bắt buộc phải là đo, tối ưu, rồi đo lại — và bước đo lại là bước hay bị bỏ nhất.
- A2. Ít nhất bốn tình huống. Khi n luôn nhỏ và nằm dưới ngưỡng hoà vốn — rất phổ biến hơn ta tưởng. Khi thuật toán “tệ hơn” có kiểu truy cập bộ nhớ tốt hơn và thắng trên máy thật. Khi nó đơn giản hơn nhiều và cả nhóm hiểu được, trong khi lợi ích chỉ là hằng số. Và khi nó có *đảm bảo trường hợp xấu nhất* tốt hơn, dù trung bình tệ hơn — điều quyết định trong hệ có hạn kỳ hoặc có dữ liệu thù địch.
- A3. Vì một yêu cầu của người dùng thường phụ thuộc vào nhiều thành phần con, và người dùng cảm nhận cái chậm nhất chứ không phải cái trung bình. Nếu mỗi thành phần chậm bất thường trong 1% số lần và một yêu cầu chạm tới vài chục thành phần thì xác suất gặp ít nhất một lần chậm là rất lớn. Vì vậy các hệ thống lớn đo theo phân vị cao, và tiêu chí chọn thuật toán chuyển từ “nhanh trung bình” sang “đều đặn”.
- A4. Vì đơn vị chi phí thật trên máy hiện đại là *lần chạm bộ nhớ chậm*, không phải phép toán. Dữ liệu nằm liên tiếp cho phép phần cứng nạp sẵn phần tiếp theo và tận dụng trọn vẹn mỗi dòng cache; dữ liệu nằm rải rác làm mỗi phần tử thành một lần chờ. Chênh lệch giữa hai trường hợp có thể tới hàng chục lần mà số phép toán không đổi — đúng như mô hình ở D/24 mô tả, chỉ ở quy mô cache thay vì đĩa.
- A5. Vì khâu hao chỉ *đảm bảo tổng* chi phí của một dãy thao tác, không *đảm bảo từng* thao tác. Trong hệ thời gian thực, một lần cấp phát lại toàn bộ hoặc một lần bấm lại bảng có thể tốn $O(n)$ đúng vào chu kỳ đang có hạn kỳ, và hậu quả là lỡ hạn — trong điều khiển robot thì đó là hậu quả vật lý. Ở đó người ta chọn cấu trúc có *đảm bảo trường hợp xấu nhất* cho từng thao tác, chấp nhận hằng số lớn hơn và thông lượng thấp hơn.

NÓI LẠI THẬT ĐƠN GIẢN

Trong kỳ thi, chỉ có một câu hỏi: chương trình có chạy đúng trong giới hạn thời gian không. Ngoài đời có thêm bốn câu nữa, và chúng thường quan trọng hơn.

Câu thứ nhất: chậm ở đâu? Câu trả lời gần như luôn khác chỗ bạn đoán. Người ta hay đi tối ưu đoạn mã trông rắc rối nhất, trong khi thời gian thật lại bị tiêu ở một vòng lặp tầm thường chạy hàng

triệu lần, hoặc ở việc cấp phát bộ nhớ mà bạn không hề viết ra. Cách duy nhất để biết là đo — và đo lại sau khi sửa, vì rất nhiều “cải tiến” hoá ra không cải tiến gì.

Câu thứ hai: dữ liệu thật lớn tới đâu? Nếu chỉ có vài trăm phần tử thì một vòng lặp đơn giản gần như luôn thắng một cấu trúc tinh vi, cả về tốc độ lẫn về việc người khác trong nhóm hiểu được nó.

Câu thứ ba: bạn quan tâm tới trung bình hay tới lần chậm nhất? Một dịch vụ có thời gian phản hồi trung bình rất đẹp vẫn có thể làm người dùng bức, vì mỗi thao tác của họ đọng tới hàng chục thành phần và họ cảm nhận cái chậm nhất. Còn trong điều khiển robot thì chỉ có lần chậm nhất là quan trọng: trung bình đẹp mà một chu kỳ lỡ hạn thì hỏng việc.

Câu thứ tư: ai sẽ sửa cái này sau một năm? Một lời giải nhanh hơn 20% nhưng chỉ một người hiểu thường là lựa chọn tệ hơn một lời giải rõ ràng.

Cuối cùng, một điều rất thực dụng mà lý thuyết không nói: trên máy hiện đại, việc dữ liệu nằm liền nhau trong bộ nhớ hay nằm rải rác có thể tạo chênh lệch nhiều lần, dù số phép tính không đổi. Vì vậy khi cần tăng tốc, thứ đầu tiên nên thử thường không phải đổi thuật toán mà là đổi cách xếp dữ liệu.

Một câu: Trong hệ thống thật, câu hỏi không phải “thuật toán nào nhanh nhất” mà “ràng buộc thật sự của tôi là gì”.

Chỗ không rút gọn được: Việc cân bằng giữa tốc độ, độ phức tạp mã và khả năng bảo trì không có công thức; nó là một quyết định kỹ thuật phụ thuộc bối cảnh và con người.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Nguyên tắc số một	Đo trước khi tối ưu, và <i>đo lại</i> sau khi tối ưu. Trực giác về nút cổ chai gần như luôn sai.
●●● Ba lý do trực giác sai	Chi phí nằm ở tầng bạn không viết; chi phí bộ nhớ không thấy trong mã nguồn; ta chú ý mã <i>phức tạp</i> thay vì mã <i>chạy nhiều</i> .
●●● Yếu tố hiệu năng lớn nhất mà tiệm cận bỏ qua	Kiểu truy cập bộ nhớ. Đảo bố cục dữ liệu thường cho hệ số lớn hơn đối thuật toán, ở kích thước thật.
●●● Khi nào chọn thuật toán “tệ hơn”	n luôn nhỏ; kiểu truy cập bộ nhớ tốt hơn; đơn giản hơn nhiều mà lợi ích chỉ là hằng số; hoặc có đảm bảo xấu nhất tốt hơn.
●●● Độ trễ đuôi	Người dùng cảm nhận thành phần chậm nhất, không phải trung bình; hệ thống lớn đo phân vị 99 hoặc 99,9. Tính đều đặn ngang hàng tốc độ.
●●● Hệ thời gian thực	Cần đảm bảo cho <i>mỗi</i> thao tác; khấu hao không đủ. Ưu tiên cấu trúc cố cận trên xấp xỉ, cấp phát tĩnh, tránh cơ chế chạy nền.
●● Ngưỡng hoà vốn	Mọi thuật toán tốt hơn về tiệm cận đều thua ở kích thước nhỏ; thư viện thật giải quyết bằng cách kết hợp nhiều thuật toán theo kích thước và độ sâu.
●● Cấp phát bộ nhớ	Cấp phát trong vòng lặp là nguyên nhân chậm phổ biến và ít bị nghi ngờ; cấp phát trước và dùng lại vùng nhớ thường cho cải thiện lớn.
●● Dữ liệu thù địch	Khi dữ liệu đến từ bên ngoài, phân tích trung bình không đủ: bảng băm phải ngẫu nhiên hoá, và các trường hợp xấu đoán được đều là lỗ hổng.
●● Khả năng bảo trì	Dùng cấu trúc phức tạp khi nó cho cải thiện <i>bậc</i> , không phải khi cho cải thiện hằng số.
●● Ý đầy đủ của Knuth	Bỏ qua cải thiện nhỏ ở phần lớn mã, nhưng <i>không</i> bỏ lỡ ở phần thực sự quan trọng — và phải đo để biết phần nào.
● Bài học COST	So sánh với một mốc trung thực — một cài đặt đơn luồng tốt — trước khi kết luận rằng hệ thống song song nhanh hơn.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên bắt đầu tối ưu từ đâu khi chương trình chậm?

| Theo thứ tự này. Một, đo để biết phần nào chiếm thời gian — đừng bỏ qua bước này dù bạn “chắc chắn” biết. Hai, kiểm tra bậc tiệm cận của phần đó: nếu sai bậc thì mọi tối ưu vi mô đều vô ích. Ba, nếu bậc đúng, xem kiểu truy cập bộ nhớ và cấp phát — hai thứ này thường cho hệ số lớn nhất. Bốn, chỉ sau đó mới tới các tối ưu vi mô khác. Và luôn đo lại sau mỗi thay đổi.

2. Làm sao biết mình đang bị chặn bởi tính toán hay bởi bộ nhớ?

| Hai thí nghiệm rẻ đã nói ở D/24: giảm kích thước dữ liệu xuống mức vừa bộ nhớ đệm và xem thời gian trên mỗi phần tử có giảm mạnh không; và đổi thứ tự truy cập mà giữ nguyên số phép toán. Nếu thời gian đổi nhiều lần trong thí nghiệm thứ hai thì bạn đang bị chặn bởi bộ nhớ, và việc cần làm là đổi bố cục dữ liệu chứ không phải giảm số phép tính.

3. Trong công việc, tôi có bao giờ cần tới các thuật toán ở phần C và D không?

| Có, nhưng không đều. Những thứ dùng gần như hằng ngày: sắp xếp và tìm kiếm, bảng băm, đồ thị cơ bản, quy hoạch động ở dạng đơn giản, nhị phân trên đáp án. Những thứ dùng vài lần trong sự nghiệp nhưng khi cần thì rất đáng: luồng cực đại cho các bài phân công, cấu trúc chuỗi cho tìm kiếm văn bản, nhận diện NP-khó để ngừng đúng lúc. Giá trị lớn nhất của phần C và D không phải là bạn cài chúng thường xuyên, mà là bạn *nhận ra* khi một bài trong công việc thuộc về một trong các mẫu đó.

4. Nên tự cài hay dùng thư viện?

| Mặc định là dùng thư viện: nó đã được kỹ thuật hoá, kiểm thử, và người khác đọc được. Tự cài khi có lý do cụ thể: thư viện thiếu thao tác bạn cần (tách/ghép cây, truy vấn thứ hạng), hằng số của nó là nút cổ chai đã đo được, hoặc bạn cần đảm bảo trường hợp xấu nhất mà thư viện không cho. Trong cả ba trường hợp, hãy stress test bản tự cài với thư viện làm trọng tài (E/25).

5. “Đủ nhanh” nghĩa là gì trong thực tế?

| Nghĩa là đạt một ngưỡng đã thoả thuận, ổn định, với biên an toàn cho tăng trưởng. Cách làm đúng là xác định ngưỡng *trước*: thời gian phản hồi mục tiêu ở phân vị 99, hoặc thời gian xử lý mỗi khung hình, hoặc dung lượng bộ nhớ tối đa. Không có ngưỡng thì việc tối ưu không có điểm dừng, và mọi giờ bỏ vào đó là một giờ không dùng cho việc khác — đây là nơi tinh thần kỹ thuật khác hẳn tinh thần thi đấu.

6. Người luyện thi đấu chuyển sang làm sản phẩm hay mắc lỗi gì?

| Ba lỗi. Chọn cấu trúc tinh vi cho dữ liệu nhỏ, làm mã khó bảo trì mà không nhanh hơn. Tối ưu hằng số trước khi kiểm tra bậc, hoặc ngược lại, đổi thuật toán khi nút cổ chai nằm ở nơi khác hoàn toàn. Và bỏ qua các tiêu chí ngoài tốc độ — đặc biệt là tính đều đặn và khả năng đọc hiểu. Điểm tốt mà họ mang sang: kỷ luật ước lượng chi phí trước khi cài, và thói quen kiểm thử bằng dữ liệu biên.

7. Có cách nào giữ được kỹ năng thuật toán khi công việc ít dùng tới?

| Ba cách rẻ. Duy trì một nhịp luyện nhỏ nhưng đều — một bài mỗi tuần đủ để giữ phản xạ (E/28). Chủ động tìm chỗ áp dụng trong công việc: rất nhiều bài “tối ưu lịch”, “chọn tập đại diện”, “tìm phụ thuộc vòng” là bài trong quyển sách này khoác áo nghiệp vụ. Và đọc mã nguồn của các thư viện bạn dùng — đó là nơi lý thuyết và kỹ thuật gặp nhau, và là tài liệu học tốt nhất về nội dung của chương này.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Đo thời gian duyệt mảng theo hàng và theo cột trên ma trận lớn	Bài tập thực nghiệm	Cùng số phép toán, khác kiểu truy cập; ghi lại tỉ số trên máy của bạn.
● Chạy hồ sơ hiệu năng cho một chương trình bạn đã viết	Bài tập công cụ	So kết quả với dự đoán của chính bạn trước khi đo — thường rất khác.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Tìm ngưỡng hoà vốn giữa sắp xếp chèn và sắp xếp trộn	Bài tập thực nghiệm	Vẽ đường thời gian theo n ; so con số của bạn với ngưỡng mà thư viện chuẩn dùng.
•• So sánh danh sách liên kết và mảng động cho cùng khối lượng công việc	Bài tập thực nghiệm	Đo chèn, duyệt, xoá; giải thích kết quả bằng phân cấp bộ nhớ chứ không bằng độ phức tạp.
•• Loại bỏ cấp phát trong vòng lặp nóng và đo lại	Bài tập tối ưu	Cấp phát trước và dùng lại vùng nhớ; đây thường là tối ưu có tỉ lệ lợi ích trên công sức cao nhất.
•• Đo phân vị 50, 99 và 99,9 của thời gian phản hồi thay vì trung bình	Bài tập đo lường	Dùng một dịch vụ nhỏ tự viết; quan sát khoảng cách giữa trung bình và đuôi.
••• Dựng dữ liệu thù địch cho một cấu trúc băm không ngẫu nhiên hoá	B/08	Đo mức suy giảm hiệu năng, rồi sửa bằng hạt giống ngẫu nhiên và đo lại.
••• So một chương trình đơn luồng viết tốt với một phiên bản song song	[38]	Chọn một bài xử lý đồ thị vừa phải; báo cáo cả thời gian lẫn số lỗi dùng.
••• Thay một cấu trúc dựa trên con trỏ bằng mảng phẳng trong một chương trình có sẵn	Bài tập tối ưu	Đo trước và sau; ghi lại cả thời gian lẫn mức tiêu thụ bộ nhớ.
••• Chọn một thuật toán trong quyển sách này và tìm nơi nó chạy trong một phần mềm mã nguồn mở bạn dùng	Bài tập đọc mã	Đọc cách nó được kỹ thuật hoá so với bản trong sách; đây là bài tập học được nhiều nhất chương.

Kết chương và đọc tiếp

Chương này nối phần lý thuyết với thực tế bằng một thay đổi trong câu hỏi: không phải “thuật toán nào nhanh nhất” mà “ràng buộc thật sự của tôi là gì”. Ba điều đáng mang theo: đo trước và đo lại sau khi tối ưu; kiểu truy cập bộ nhớ thường quyết định nhiều hơn số phép toán ở kích thước thật; và các tiêu chí ngoài tốc độ — đuôi độ trễ, tính dự đoán được, khả năng bảo trì — thường là thứ chọn ra thuật toán được dùng.

Hai chương cuối quay lại với người học. Chương E/27 nói về chiến thuật khi có giới hạn thời gian và có người quan sát — phỏng vấn và thi đấu — nơi kỹ năng quan trọng nhất không phải kiến thức mà là quản lý thời gian và nói ra suy nghĩ. Sau đó, chương E/28 biến toàn bộ quyển sách thành một lịch luyện cụ thể kèm ngân hàng bài tập.

Phỏng vấn và thi đấu (Interviews and Contests)

PHỤ THUỘC A/01 cho quy trình bốn bước — ở đây nó được nén lại dưới sức ép thời gian; A/02 cho từng tự giám sát; E/25 cho bảng kiểm trước khi nộp.

TÓM TẮT

Hai bối cảnh trong chương này — phỏng vấn và thi đấu — khác nhau ở tiêu chí chấm nhưng giống nhau ở điều quyết định kết quả: không phải lượng kiến thức mà là *quản lý thời gian và quản lý bản thân dưới sức ép*. Chương đưa ra một quy trình bảy bước cho buổi phỏng vấn, trong đó bước quan trọng nhất là nói ra suy nghĩ, vì người phỏng vấn chấm quá trình chứ không chỉ chấm kết quả. Với thi đấu, chương nói về đọc toàn bộ đề trước, phân bổ thời gian, quy tắc bỏ bài, và cách xử lý khi bị sai liên tiếp. Nếu chỉ nhớ một điều: dưới sức ép, thứ khiến bạn mất điểm nhiều nhất không phải bài khó mà là hai lỗi rất tránh được — hiểu sai đề, và bám vào một hướng sai quá lâu.

KIẾN THỨC CẦN CÓ TRƯỚC

- **nói ra suy nghĩ** — diễn đạt thành lời quá trình suy nghĩ trong lúc giải, để người nghe theo được đường đi của bạn.
- **thi ảo** — tự tổ chức một kỳ thi từ đề cũ, đúng thời lượng và điều kiện thật, để luyện chiến thuật chứ không chỉ luyện kiến thức.
- **phạt thời gian** — ở nhiều thể thức thi, mỗi lần nộp sai cộng thêm một khoảng vào tổng thời gian — điều này thay đổi hẳn chiến thuật nộp bài.
- **chấm từng phần** — thể thức cho điểm theo số test qua được hoặc theo nhóm test; khuyến khích nộp lời giải chậm hơn để lấy điểm bộ phận.
- **tilt** — trạng thái mất bình tĩnh sau một thất bại, dẫn tới các quyết định tệ tiếp theo — thuật ngữ mượn từ cờ và thể thao điện tử.
- **bảng kiểm trước khi nộp** — danh sách ngắn các ca biên và lỗi kinh điển cần rà lại, dùng để chống bỏ sót dưới áp lực (E/25).

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao người phỏng vấn thường quan tâm tới quá trình suy nghĩ hơn là lời giải cuối cùng?
- Q2. Trong một kỳ thi ba tiếng với sáu bài, vì sao đọc hết cả sáu đề trước lại quan trọng hơn bắt tay vào bài đầu tiên ngay?
- Q3. Bạn đã bỏ 40 phút cho một bài và vẫn chưa ra. Quyết định đúng là gì, và làm sao biết được điều đó *trước khi hết giờ*?
- Q4. Vì sao im lặng suy nghĩ trong phỏng vấn lại là một sai lầm, kể cả khi bạn đang nghĩ tốt?
- Q5. Sau một lần nộp sai, vì sao xác suất lần nộp tiếp theo cũng sai lại cao hơn bình thường?

Hai bối cảnh trong chương này có một điểm chung mà người luyện thường không chuẩn bị: chúng không đo kiến thức thuần túy. Một kỳ thi ba tiếng đo khả năng *chọn bài, phân bổ thời gian và giữ bình tĩnh* nhiều không kém khả năng nghĩ ra thuật toán. Một buổi phỏng vấn 45 phút đo khả năng *giao tiếp và làm việc dưới quan sát* nhiều không kém năng lực kỹ thuật.

Điều đó có nghĩa là hai bối cảnh này luyện được riêng, và phần lớn người học bỏ qua việc luyện đó. Họ giải hàng trăm bài nhưng chưa bao giờ tự tổ chức một kỳ thi ảo đúng thời lượng, chưa bao giờ tập nói ra suy nghĩ trong lúc giải. Rồi đến ngày thật, họ mất điểm ở đúng những chỗ có thể luyện được.

27.1 Phỏng vấn: quy trình bảy bước

Một buổi phỏng vấn thuật toán điển hình kéo dài 40–60 phút cho một hoặc hai bài. Quy trình dưới đây nén bốn bước của Pólya (A/01) vào khung thời gian đó, và nó cũng chính là thứ người phỏng vấn mong thấy.

Bước 1 — làm rõ đề (2–4 phút). Hỏi về kích thước dữ liệu, kiểu dữ liệu, trường hợp biên, điều gì được đảm bảo. Đây là bước bị bỏ nhiều nhất và cũng là bước bị chấm điểm âm nặng nhất khi bỏ, vì trong công việc thật, viết code trước khi hiểu yêu cầu là một lỗi nghiêm trọng.

Bước 2 — ví dụ (2 phút). Tự viết một ví dụ nhỏ và giải nó bằng tay, xác nhận với người phỏng vấn rằng bạn hiểu đúng. Bước này bắt được phần lớn hiểu lầm và tốn rất ít thời gian.

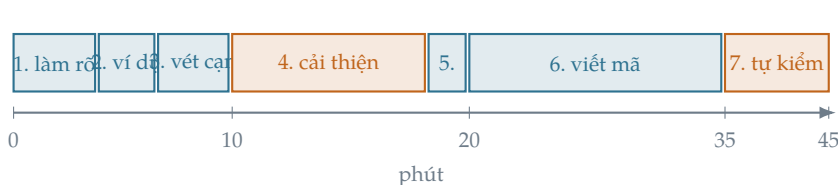
Bước 3 — lời giải vét cạn (2–3 phút). Nói ra cách giải chậm nhưng đúng, kèm độ phức tạp của nó. Đừng ngại: điều này cho thấy bạn hiểu bài, và nó tạo điểm xuất phát để cải thiện — đúng lập luận ở C/13.

Bước 4 — cải thiện (5–10 phút). Hỏi to câu “vét cạn này lãng phí ở đâu”, rồi đi theo cây quyết định ở hình của chương A/01. Đây là phần mà người phỏng vấn quan tâm nhất, và cũng là phần bạn nên nói nhiều nhất.

Bước 5 — thống nhất trước khi viết (1 phút). Tóm tắt lời giải trong ba câu và hỏi “tôi bắt đầu viết nhé”. Nếu hướng sai, người phỏng vấn thường gợi ý ở đúng lúc này — và bạn tiết kiệm được mười lăm phút.

Bước 6 — viết mã (10–15 phút). Viết sạch, đặt tên rõ, tách hàm khi hợp lý. Nói ngắn gọn khi làm điều gì đó không hiển nhiên.

Bước 7 — tự kiểm (3–5 phút). Chạy tay ví dụ ở bước 2 qua code, rồi rà bảng kiểm ca biên: mảng rỗng, một phần tử, giá trị trùng, tràn số. Chủ động làm bước này thay vì chờ được hỏi — nó tạo ấn tượng mạnh vì rất nhiều ứng viên bỏ qua.



Hai ô tô hổ phách là phần được chấm cao nhất — và là hai phần hay bị cắt nhất khi vội.

Hình 27.1. Phân bố thời gian gợi ý cho một buổi phỏng vấn 45 phút. Điều đáng chú ý là bước 5 — thống nhất hướng trước khi viết — chỉ tốn một phút nhưng ngăn được kịch bản tốn kém nhất: viết mười lăm phút theo hướng sai. Bước 7 nên chủ động làm, không đợi được hỏi.

CẦN NHỚ

Điều được chấm không phải “có ra lời giải tối ưu không” mà là “người này có phải người tôi muốn cùng gỡ một bài khó lúc hai giờ sáng không”. Bốn thứ tạo nên ấn tượng đó: hỏi làm rõ trước khi làm, nói ra suy nghĩ, phản ứng tốt với gợi ý, và tự kiểm tra mà không đợi nhắc. Cả bốn đều luyện được, và cả bốn đều không liên quan tới việc bạn thuộc bao nhiêu thuật toán.

Ba lỗi giao tiếp phổ biến

Im lặng suy nghĩ. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: người phỏng vấn không đọc được suy nghĩ của bạn, nên với họ, ba phút im lặng trông giống hết ba phút bế tắc. Tệ hơn, họ không có cơ hội gợi ý đúng chỗ. Nếu cần nghĩ yên, hãy nói trước: “cho tôi một phút suy nghĩ về cấu trúc dữ liệu”.

Nhảy vào viết code quá sớm. Viết trước khi thống nhất hướng là cách nhanh nhất để mất mười lăm phút và phải xoá làm lại.

Phỏng thủ khi được góp ý. Gợi ý của người phỏng vấn gần như luôn là gợi ý thật, không phải bẫy. Cách phản ứng tốt là dừng lại, hỏi lại cho rõ, và đi theo hướng đó.

27.2 Thi đấu: chiến thuật quan trọng ngang kiến thức

Trong một kỳ thi có nhiều bài và giới hạn thời gian, ba quyết định chiến thuật ảnh hưởng tới kết quả nhiều hơn hầu hết kiến thức kỹ thuật.

Quyết định 1 — đọc hết đề trước. Dành 5–10 phút đầu đọc toàn bộ và ước lượng độ khó từng bài. Lý do rất thực dụng: thứ tự đề *không* luôn theo độ khó, và mỗi phút bạn dùng cho một bài khó trong khi còn một bài dễ chưa đọc là một phút bị lãng phí. Đây là câu trả lời cho câu hỏi mở đầu thứ hai. Nếu bảng điểm hiển thị số người giải được từng bài, đó là nguồn thông tin về độ khó chính xác hơn cảm giác của bạn.

Quyết định 2 — đặt mốc thời gian và tuân thủ. Trước khi bắt đầu một bài, tự đặt một hạn: “nếu 25 phút nữa chưa có hướng rõ ràng, tôi chuyển bài”. Không có mốc thì bạn sẽ phát hiện mình chọn sai bài lúc chỉ còn mười phút — đúng mô hình phân bố thời gian mà Schoenfeld quan sát được ở A/01. Có mốc thì bạn mất tối đa 25 phút cho một quyết định sai.

Quyết định 3 — biết khi nào bỏ và khi nào quay lại. Bỏ bài không phải thất bại. Rất thường xuyên, ý tưởng cho bài đã bỏ xuất hiện trong lúc bạn làm bài khác — hiện tượng “áp ủ” ở A/01. Quy tắc thực dụng: khi bỏ, hãy ghi lại một dòng về chỗ bạn đang kẹt, để khi quay lại không phải nạp lại từ đầu.

Nộp bài và xử lý sai

Thẻ thức chấm quyết định chiến thuật nộp. Nếu có *phạt thời gian* cho mỗi lần nộp sai, hãy dành thêm hai phút rà bảng kiểm trước khi nộp — rẻ hơn nhiều so với tiền phạt. Nếu có *chấm từng phần*, hãy nộp lời giải chậm hơn để lấy điểm bộ phận trước khi cố tối ưu; nhiều người mất điểm vì theo đuổi lời giải đầy đủ tới hết giờ.

Sau một lần nộp sai, có một hiệu ứng tâm lý cần biết, và nó là câu trả lời cho câu hỏi mở đầu thứ năm: xác suất lần nộp tiếp theo cũng sai cao hơn bình thường. Lý do là bạn thường sửa *triệu chứng* chứ không sửa nguyên nhân, và bạn làm việc đó dưới áp lực với sự chú ý bị thu hẹp. Cách chống là bắt buộc mình dừng lại 60 giây và trả lời hai câu trước khi sửa: *ca dữ liệu nào có thể làm sai*, và *giả định nào của tôi có thể không đúng*. Nếu không trả lời được, hãy viết stress test thay vì đoán tiếp (E/25).

27.3 Luyện hai kỹ năng này thế nào

Cả hai bối cảnh đều luyện được, và cách luyện rất cụ thể.

Cho phỏng vấn: giải bài với việc *nói to* toàn bộ suy nghĩ, kể cả khi ngồi một mình — ban đầu rất gượng, nhưng đó chính là kỹ năng cần luyện. Tốt hơn nữa là luyện theo cặp, thay nhau đóng vai người phỏng vấn. Ngoài ra, tự đặt giới hạn thời gian 40 phút cho mỗi bài để làm quen với sức ép.

Cho thi đấu: thi ảo là công cụ không thể thay thế. Lấy một đề cũ, đặt đồng hồ đúng thời lượng, không tra cứu, làm nghiêm túc. Sau đó ghi lại mốc thời gian mỗi lần đổi bài và đối chiếu với hai quyết định chiến thuật ở mục 2. Rồi upsolve theo quy trình ở [A/02](#).

Cho cả hai: chuẩn bị sẵn hai thứ. Một *thư viện mẫu* đã kiểm chứng cho các cấu trúc hay dùng — DSU, cây phân đoạn, Dijkstra, luồng — vì thời gian gỡ lại từ đầu là thời gian không dùng để nghĩ. Và một *bảng kiểm trước khi nộp* gồm năm tới bảy dòng, rút từ nhật ký lỗi của chính bạn.

27.4 Gốc rễ: từ một cuộc thi ở Texas tới quy trình phỏng vấn hiện đại

Thi lập trình có tuổi đời khá dài. Các cuộc thi giữa các trường đại học bắt đầu từ đầu thập niên 1970 ở Mỹ và dần lớn thành giải vô địch lập trình sinh viên quốc tế; kỳ thi tin học quốc tế dành cho học sinh phổ thông bắt đầu năm 1989. Hai thể thức này khác nhau ở một điểm ảnh hưởng lớn tới chiến thuật: thi đồng đội theo kiểu ICPC chấm đúng/sai kèm phạt thời gian, còn IOI chấm từng phần theo nhóm test. Sự khác biệt đó tạo ra hai văn hoá luyện tập khác nhau — một bên nhấn mạnh tốc độ và độ chính xác lần nộp đầu, một bên nhấn mạnh khả năng lấy điểm bộ phận và tối ưu dần.

Phỏng vấn kỹ thuật có một lịch sử ngắn hơn và nhiều thay đổi hơn. Giai đoạn đầu của các công ty công nghệ lớn nổi tiếng với các câu đố mẹo — những câu hỏi kiểu “vì sao nắp cống hình tròn” — và cách làm đó dần bị bỏ khi có bằng chứng rằng chúng không dự báo được năng lực làm việc. Từ giữa thập niên 2000, các câu hỏi thuật toán có cấu trúc trở thành chuẩn, và bộ sách hướng dẫn phỏng vấn[32] đóng vai trò lớn trong việc chuẩn hoá kỳ vọng của cả hai phía.

Điều đáng nhớ về mặt phương pháp là vì sao quy trình bảy bước ở mục 1 lại được chăm cao. Nó không phải một nghi thức: mỗi bước tương ứng với một hành vi mà người phỏng vấn thực sự cần dự báo. Hỏi làm rõ trước khi làm dự báo việc bạn có xây sai thứ trong dự án thật hay không. Nói ra suy nghĩ dự báo việc cộng tác. Tự kiểm tra dự báo chất lượng mã bạn giao. Nhìn theo cách đó, buổi phỏng vấn là một mô phỏng thu nhỏ — và cách luyện tốt nhất là làm việc theo đúng quy trình đó cả khi không ai xem, cho tới khi nó thành phản xạ.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Đầu thập niên 1970 — thi lập trình sinh viên	Cần một cách đánh giá kỹ năng lập trình mang tính thi đấu.	Thể thức đồng đội, chấm đúng/sai kèm phạt thời gian; định hình văn hoá coi trọng độ chính xác lần nộp đầu.
1989 — kỳ thi tin học quốc tế	Đánh giá học sinh phổ thông ở quy mô quốc tế.	Chấm từng phần theo nhóm test; sinh ra chiến thuật lấy điểm bộ phận và tối ưu dần.
Thập niên 1990–2000 — phỏng vấn kiểu câu đố	Cần lọc ứng viên nhanh ở quy mô lớn.	Câu đố mẹo phổ biến rồi bị bỏ vì không dự báo được năng lực làm việc thật.
Giữa thập niên 2000 — phỏng vấn thuật toán có cấu trúc	Cần tiêu chí đánh giá lập lại được và công bằng hơn.	Bài thuật toán 45 phút thành chuẩn; sách hướng dẫn chuẩn hoá kỳ vọng của cả hai phía[32].
Thập niên 2010 — nền tảng luyện tập trực tuyến	Người học cần môi trường luyện có phản hồi tức thì.	Kho bài phân loại theo độ khó và thi ảo; độ trễ phản hồi giảm mạnh, đúng yếu tố mà A/02 nói là quan trọng nhất.

27.5 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Cờ vua	Quản lý đồng hồ, tránh áp lực thời gian	Kỳ thủ mạnh không tính sâu hơn nhiều mà <i>phân bổ thời gian</i> tốt hơn — đúng phát hiện của Schoenfeld ở A/01. Cùng bài học: quyết định nhanh ở thể đơn giản để dành thời gian cho thể phức tạp.
Thể thao	Thi đấu tập, mô phỏng điều kiện thật	Thi ảo chính là buổi tập có tính giờ. Nguyên tắc chung: kỹ năng chịu sức ép chỉ luyện được khi có sức ép, nên tập nhẹ nhàng không chuẩn bị cho ngày thi.
Hàng không	Bảng kiểm, huấn luyện buồng lái mô phỏng	Bảng kiểm trước khi nộp bài giống bảng kiểm trước cất cánh: không phải vì bạn không biết, mà để chống bỏ sót dưới áp lực. Buồng lái mô phỏng chính là thi ảo.
Y khoa	Thi lâm sàng có cấu trúc, “nghĩ to” khi chẩn đoán	Sinh viên y được dạy diễn đạt lập luận chẩn đoán thành lời — vì cùng lý do với mục 1.2: người chấm cần thấy quá trình, và bệnh nhân cần hiểu lý do.
Trình bày và bảo vệ nghiên cứu	Trả lời câu hỏi của hội đồng	Cùng ba lỗi giao tiếp: im lặng quá lâu, trả lời trước khi hiểu câu hỏi, và phòng thủ khi bị góp ý. Cách chữa cũng giống nhau.
Tâm lý học thể thao	Trạng thái mất bình tĩnh sau thất bại	Hiện tượng nộp sai liên tiếp ở mục 2.1 là một dạng của nó. Biện pháp cũng tương tự: một thủ tục cố định để chạy sau mỗi thất bại, thay vì dựa vào ý chí.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì lời giải cuối cùng nói rất ít về việc bạn sẽ làm việc thế nào. Bạn có thể đã gặp đúng bài đó, hoặc gặp một bài hoàn toàn xa lạ — hai trường hợp cho cùng kết quả nhưng nói hai điều khác nhau. Quá trình thì dự báo được: hỏi làm rõ trước khi làm dự báo bạn có xây sai thứ trong dự án thật hay không; nói ra suy nghĩ dự báo khả năng cộng tác; tự kiểm tra dự báo chất lượng mã bạn giao. Buổi phỏng vấn là một mô phỏng thu nhỏ của việc cùng làm việc.
- A2. Vì thứ tự đề không luôn theo độ khó, và bạn không thể phân bổ thời gian hợp lý cho thứ mình chưa biết. Mười phút đọc hết đề cho bạn một bức tranh để xếp thứ tự làm, và nó thường tiết kiệm được nửa giờ. Ngoài ra, việc đọc một bài rồi để đó có tác dụng phụ tốt: não bạn tiếp tục xử lý nó trong lúc bạn làm bài khác — hiện tượng ấp ủ ở A/01.
- A3. Quyết định đúng gần như luôn là chuyển sang bài khác và quay lại sau, kèm ghi một dòng về chỗ đang kẹt. Cách biết trước khi hết giờ là đặt mốc *ngay khi bắt đầu bài*: “nếu 25 phút nữa chưa có hướng rõ ràng thì chuyển”. Mốc phải đặt trước, vì khi đã bỏ 40 phút thì tâm lý tiếc công đã bỏ ra sẽ khiến bạn quyết định tệ. Đây chính là tăng tự giám sát mà A/01 mô tả, ở dạng cụ thể nhất.
- A4. Vì im lặng với bạn là suy nghĩ, còn với người phỏng vấn là không có thông tin — ba phút nghĩ tốt và ba phút bế tắc trông giống hệt nhau. Quan trọng hơn, im lặng làm họ mất cơ hội gợi ý đúng chỗ, mà gợi ý là một phần của quy trình chứ không phải sự nhân nhượng. Nếu cần yên tĩnh để nghĩ, hãy nói trước một câu: “cho tôi một phút cân nhắc về cấu trúc dữ liệu” — điều đó vừa giữ được sự yên tĩnh vừa giữ được liên lạc.
- A5. Vì bạn thường sửa triệu chứng chứ không sửa nguyên nhân: bạn đổi một dấu so sánh, thêm một trường hợp riêng, mà không biết vì sao sai. Cộng thêm áp lực làm sự chú ý thu hẹp và loại bỏ khả năng suy xét rộng. Cách chống là một thủ tục cố định: dừng 60 giây, trả lời hai câu — ca dữ liệu nào có thể sai, giả định nào của tôi có thể không đúng — và nếu không trả lời được thì viết stress test thay vì đoán tiếp (E/25).

NÓI LẠI THẬT ĐƠN GIẢN

Cả kỳ thi lẫn buổi phỏng vấn đều không đo mỗi lượng kiến thức. Chúng đo cách bạn xoay sở khi có đồng hồ chạy và có người nhìn.

Trong phỏng vấn, điều làm người nghe an tâm không phải bạn ra đáp án nhanh, mà là họ theo được bạn đang nghĩ gì. Hãy hình dung bạn đang chỉ đường cho ai đó: nếu bạn im lặng đi trước rồi bất ngờ dừng lại nói “tới rồi”, người ta không biết bạn có đi đúng đường không. Vì vậy hãy hỏi cho rõ trước khi đi, nói ra bạn định rẽ hướng nào, và khi tới nơi thì tự kiểm tra lại xem có đúng địa chỉ không. Ba việc đó quan trọng hơn việc đi nhanh.

Trong kỳ thi, sai lầm tốn kém nhất không phải làm sai một bài khó, mà là dành cả tiếng cho bài khó trong khi có một bài dễ hơn chưa đọc tới. Cách phòng rất đơn giản và ít người làm: đọc hết đề trước rồi mới chọn bài, và mỗi khi bắt đầu một bài thì tự hẹn giờ — nếu quá hạn mà chưa thấy hướng thì chuyển sang bài khác, ghi lại một dòng về chỗ đang kẹt.

Còn khi vừa nộp sai một lần, hãy cưỡng lại việc sửa ngay. Trong trạng thái sốt ruột, người ta thường sửa cái nhìn thấy chứ không sửa cái gây ra lỗi, nên lần nộp sau lại sai. Một phút dừng lại tự hỏi “dữ liệu nào có thể làm nó sai” rẻ hơn nhiều so với ba lần nộp hỏng.

Một câu: Kiến thức quyết định bạn giải được bài nào; chiến thuật quyết định bạn kịp giải bao nhiêu bài trong số đó.

Chỗ không rút gọn được: Áp lực thật không mô phỏng hoàn toàn được; thi ảo giúp nhiều nhưng không thay thế được kinh nghiệm dự thi thật.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Bày bước phỏng vấn	Làm rõ → ví dụ → vết cạn → cải thiện → thống nhất trước khi viết → viết → tự kiểm.
●●● Điều thực sự được chấm	Không phải lời giải tối ưu, mà là: hỏi trước khi làm, nói ra suy nghĩ, phản ứng tốt với gợi ý, tự kiểm tra không cần nhắc.
●●● Ba lỗi giao tiếp	Im lặng suy nghĩ; nhảy vào viết code trước khi thống nhất hướng; phòng thủ khi được góp ý.
●●● Ba quyết định chiến thuật thi đấu	Đọc hết đề trước (5–10 phút); đặt mốc thời gian trước khi bắt đầu mỗi bài; biết khi nào bỏ và ghi lại chỗ kẹt.
●●● Sau khi nộp sai	Dừng 60 giây, hỏi “ca nào có thể sai” và “giả định nào có thể không đúng”; không trả lời được thì viết stress test thay vì đoán.
●● Thẻ thức quyết định chiến thuật	Có phạt thời gian: rà bảng kiểm thêm hai phút trước khi nộp. Chấm từng phần: nộp lời giải chậm lấy điểm bộ phận trước khi tối ưu.
●● Cách luyện phỏng vấn	Giải bài với việc nói to toàn bộ suy nghĩ, kể cả khi ngồi một mình; luyện theo cặp; tự đặt giới hạn 40 phút mỗi bài.
●● Cách luyện thi đấu	Thi ảo đúng thời lượng, không tra cứu; ghi mốc thời gian mỗi lần đổi bài; upsolve sau đó theo quy trình ở A/02.
●● Hai thứ cần chuẩn bị trước	Thư viện mẫu đã stress test; bảng kiểm trước khi nộp rút từ nhật ký lỗi của chính bạn.
●● Vì sao đặt mốc thời gian trước	Vì khi đã bỏ 40 phút, tâm lý tiếc công sẽ làm bạn quyết định tệ; mốc phải đặt lúc còn khách quan.
● Hai thẻ thức, hai văn hoá	Chấm đúng/sai kèm phạt thời gian coi trọng độ chính xác lần đầu; chấm từng phần coi trọng khả năng lấy điểm bộ phận và tối ưu dần.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Nếu tôi bí hoàn toàn trong phỏng vấn thì sao?

| Nói ra chính xác chỗ bạn kẹt, dạng “tôi biết cách làm nếu... nhưng chưa biết xử lý khi...” — đúng kỹ thuật ở A/01. Điều này có ba tác dụng: cho người phỏng vấn biết bạn đang ở đâu để gợi ý đúng chỗ; cho thấy bạn suy nghĩ có cấu trúc; và rất thường xuyên, chính lúc phát biểu ra thì bạn nhìn thấy hướng đi. Điều nên tránh là im lặng, và điều tuyệt đối nên tránh là giả vờ đang tiến triển.

2. Có nên nói thẳng rằng tôi đã từng gặp bài này không?

| Nên, và hầu hết người phỏng vấn đánh giá cao sự thẳng thắn đó. Họ thường sẽ đổi bài, hoặc yêu cầu bạn giải thích sâu hơn — cả hai đều tốt hơn cho bạn so với việc giả vờ nghĩ ra rồi bị hỏi một biến thể mà bạn không hiểu bản chất. Ngoài ra, việc giả vờ rất dễ lộ: người có kinh nghiệm nhận ra ngay khi ứng viên đi thẳng tới lời giải tối ưu mà không qua bước nào.

3. Trong contest, tôi nên đọc đề theo thứ tự nào?

| Đọc lướt toàn bộ theo thứ tự đề, ghi cho mỗi bài một ước lượng độ khó và một dòng về ý tưởng nếu đã thấy. Sau đó làm theo thứ tự khó tăng dần *theo ước lượng của bạn*, không theo thứ tự đề. Nếu có bảng điểm trực tuyến, hãy dùng số người đã giải làm nguồn thông tin — nó chính xác hơn cảm giác cá nhân, đặc biệt với những bài mà đề dài làm bạn thấy đáng sợ hơn thực tế.

4. Tôi nên chuẩn bị thư viện mẫu tới mức nào?

| Đủ để không phải gõ lại những thứ dài và dễ sai: DSU, cây phân đoạn có lazy, Dijkstra, luồng cực đại, hình học cơ bản, số học modulo. Nguyên tắc quan trọng nhất: mỗi đoạn phải *do bạn viết* và đã stress test — chép về mà không hiểu thì khi cần sửa cho biến thể lạ bạn sẽ kẹt (A/02). Đừng nhồi quá nhiều: một thư viện mà bạn không nhớ có gì trong đó thì vô dụng.

5. Phỏng vấn thiết kế hệ thống có liên quan gì tới quyển sách này không?

| Có, ở phần tư duy đánh đổi chứ không ở phần thuật toán cụ thể. Các câu hỏi ở đó xoay quanh đúng những chủ đề của E/26: chọn cấu trúc theo mẫu truy cập, ước lượng quy mô bằng phép tính phong bì, độ trễ đuôi, đánh đổi giữa nhất quán và khả dụng. Kiến thức trực tiếp dùng lại: băm nhất quán, B-cây và chỉ mục, hàng đợi, và các cấu trúc xấp xỉ cho luồng dữ liệu.

6. Tôi luyện nhiều mà thi vẫn kém. Nguyên nhân hay gặp là gì?

| Ba nguyên nhân, và cả ba đều thuộc về chiến thuật chứ không kiến thức. Một, chưa bao giờ luyện có tính giờ — nên kỹ năng dưới sức ép chưa được tập. Hai, không đọc hết đề nên chọn sai bài. Ba, không có bảng kiểm nên mất điểm vào lỗi cài đặt lặp đi lặp lại. Cách chẩn đoán: sau vài kỳ thi, phân loại các bài bạn *không* giải được thành “không nghĩ ra”, “nghĩ ra nhưng không kịp”, và “làm sai”. Tỉ lệ ba nhóm cho biết bạn cần luyện gì.

7. Có nên luyện gõ code nhanh không?

| Ở mức vừa phải thì có, nhưng nó không phải nút thắt của phần lớn người học. Điều đáng luyện hơn là gõ *ít lỗi*: viết bất biến trước, giữ quy ước nhất quán, và có thư viện mẫu để không phải gõ lại thứ dài. Nếu bạn thấy mình thường xuyên hết giờ khi đang gõ chứ không khi đang nghĩ, thì hãy xem lại xem vấn đề có thật sự là tốc độ gõ hay là bạn viết lại từ đầu những thứ lẽ ra đã có sẵn.

| Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Giải một bài quen thuộc và nói to toàn bộ suy nghĩ, có ghi âm	Bài tập giao tiếp	Nghe lại và đếm số lần im lặng quá 30 giây; đó là con số cần giảm.
● Chạy quy trình bảy bước cho một bài dễ, tính giờ 40 phút	Bài tập quy trình	Mục tiêu là đủ bảy bước, không phải ra lời giải tối ưu.
●● Thi ảo một đề contest cũ, đúng thời lượng	Codeforces, AtCoder, CSES[33]	Ghi mốc thời gian mỗi lần đổi bài; đối chiếu với hai quyết định chiến thuật ở mục 2.
●● Phân loại các bài bạn không giải được trong ba kỳ thi gần nhất	Bài tập tự chẩn đoán	Ba nhóm: không nghĩ ra, không kịp, làm sai. Tỉ lệ cho biết nên luyện gì.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Luyện phỏng vấn theo cặp, thay nhau đóng vai người phỏng vấn	Bài tập giao tiếp	Đóng vai người phỏng vấn dạy nhiều không kém đóng vai ứng viên.
●● Lập bảng kiểm trước khi nộp gồm 5–7 dòng từ nhật ký lỗi của bạn	Bài tập tự tổ chức	Dùng nó trong ba kỳ thi tiếp theo và đếm số lần nó cứu bạn.
●●● Xây thư viện mẫu cho sáu cấu trúc hay dùng, mỗi cái có stress test	Bài tập cài đặt	DSU, cây phân đoạn lazy, Dijkstra, luồng, hình học cơ bản, số học modulo.
●●● Thi ảo với ràng buộc chỉ được nộp mỗi bài một lần	Bài tập kỷ luật	Luyện thói quen rà bảng kiểm; đo xem tỉ lệ đúng lần đầu của bạn là bao nhiêu.
●●● Upsolve toàn bộ một đề contest cho tới khi giải hết	Bài tập upsolving	Đây là bài tập tạo tiến bộ lớn nhất trong danh sách, và cũng là bài tập ít vui nhất.

Kết chương và đọc tiếp

Chương này về hai bối cảnh có sức ép, và luận điểm chính là chúng đo những thứ luyện được mà người học thường không luyện. Ba điều đáng mang theo: quy trình bảy bước cho phỏng vấn, trong đó bước làm rõ đề và bước tự kiểm là hai bước tạo khác biệt lớn nhất; ba quyết định chiến thuật cho thi đấu, đặc biệt việc đặt mốc thời gian *trước* khi bắt đầu một bài; và thủ tục 60 giây sau mỗi lần nộp sai.

Chương cuối cùng gom toàn bộ quyển sách thành thứ bạn mang theo được: một lộ trình luyện tập theo tuần, một cách theo dõi tiến bộ, và một ngân hàng bài tập phân loại theo 27 chương trước — để sau khi gấp sách, bạn có việc cụ thể để làm vào sáng mai ([E/28](#)).

Lộ trình luyện tập và ngân hàng bài tập (Training Plan and Problem Bank)

PHỤ THUỘC A/02 cho nguyên lý luyện tập có chủ đích và vòng upsolving; E/27 cho việc luyện dưới sức ép. Đây là chương bạn sẽ mở lại nhiều lần nhất.

TÓM TẮT

Chương cuối biến 27 chương trước thành thứ mang theo được: một lịch luyện cụ thể, một cách theo dõi tiến bộ, và một ngân hàng bài tập phân loại theo chương. Nội dung gồm ba phần. Phần một là cấu trúc một buổi luyện và một tuần luyện, cùng bốn công cụ theo dõi — nhật ký lỗi, danh sách mẫu, lịch ôn ngắt quãng, và ba chỉ số tiến bộ. Phần hai là ba lộ trình theo mục tiêu: 12 tuần cho phỏng vấn, 6 tháng cho thi đấu, và một nhịp duy trì cho người đi làm. Phần ba là ngân hàng bài tập được nhóm theo chủ đề, đủ dùng cho vài tháng. Nếu chỉ nhớ một điều: tiến bộ đến từ số bài bạn *giải xong sau khi đã bí*, nên hãy thiết kế lịch luyện quanh việc upsolving chứ không quanh số bài đã đọc lời giải.

KIẾN THỨC CẦN CÓ TRƯỚC

- **buổi luyện** — một phiên có mục tiêu hẹp, thời lượng cố định và một cách kiểm tra kết quả — khác với “ngồi giải bài tới khi chán”.
- **ôn ngắt quãng** — ôn lại một nội dung ở các khoảng cách tăng dần (1 ngày, 3 ngày, 1 tuần, 1 tháng), vì trí nhớ được củng cố mạnh nhất ngay trước lúc sắp quên.
- **danh sách mẫu** — sổ ghi các ý tưởng cốt lõi ở dạng tổng quát, mỗi mẫu một câu không nhắc tới đề bài cụ thể (A/01).
- **nhật ký lỗi** — với mỗi lần sai: hiểu sai điều gì, dấu hiệu nào lẽ ra đã gợi ý đúng, lần sau kiểm tra gì.
- **vùng luyện** — mức khó mà bạn giải được khoảng một nửa số bài; dễ hơn thì không học được gì, khó hơn thì không động lại (A/02).
- **chỉ số tiến bộ** — đại lượng đo được để biết mình có tiến không — ví dụ tỉ lệ giải được ở một mức khó cố định, chứ không phải cảm giác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao “giải càng nhiều bài càng tốt” lại là lời khuyên nguy hiểm?
- Q2. Nếu mỗi ngày chỉ có một giờ, phân bổ thế nào cho hiệu quả nhất?
- Q3. Làm sao biết mình đang tiến bộ, khi cảm giác chủ quan gần như luôn sai?
- Q4. Nên học kiến thức mới hay giải bài nhiều hơn — và tỉ lệ nào là hợp lý?
- Q5. Vì sao lộ trình nào cũng đổ vỡ sau vài tuần, và cách thiết kế nào chống lại điều đó?

Sau khi đọc xong 27 chương, câu hỏi thực tế duy nhất còn lại là: sáng mai làm gì. Chương này trả lời câu đó.

Nguyên tắc nền đã được nêu ở A/02 và đáng nhắc lại vì mọi thứ trong chương này đều suy ra từ nó: luyện tập chỉ tạo tiến bộ khi có mục tiêu hẹp, mức khó hơi vượt khả năng, phản hồi nhanh,

và có sửa sai. Bốn điều kiện đó biến thành bốn quyết định thiết kế: buổi luyện phải có mục tiêu viết ra được; bài phải chọn ở vùng bạn giải được khoảng một nửa; phải có chấm bài hoặc stress test; và phải có nhật ký lỗi cùng bước upsolving. Lịch luyện nào thiếu một trong bốn thì tiêu tốn thời gian mà không đổi được gì.

28.1 Cấu trúc một buổi và một tuần

Một buổi 60 phút có cấu trúc dùng được như sau: 10 phút ôn — đọc lại nhật ký lỗi và một bảng active recall của chương đang học; 40 phút giải một bài ở vùng luyện, có bấm giờ; 10 phút cho bước bốn — viết một câu về ý tưởng cốt lõi vào danh sách mẫu, và ghi nhật ký nếu sai. Tỷ lệ có thể xê dịch, nhưng đừng cắt 10 phút cuối: đó là phần biến một lần giải bài thành kiến thức dùng lại được.

Một tuần nên có ba loại buổi khác nhau, và sự xen kẽ này quan trọng hơn tổng số giờ. *Buổi kiến thức*: đọc một mục của quyển sách này hoặc một chủ đề mới, rồi cài thử ngay và cho vào thư viện. *Buổi bài tập*: ba tới năm bài cùng một chủ đề, để nhận diện thành phần xạ. *Buổi thi*: một kỳ thi ảo tính giờ, xen kẽ nhiều chủ đề, kèm upsolving sau đó. Tỷ lệ hợp lý cho người mới là 2:3:1 mỗi tuần; càng về sau càng nghiêng về buổi thi.

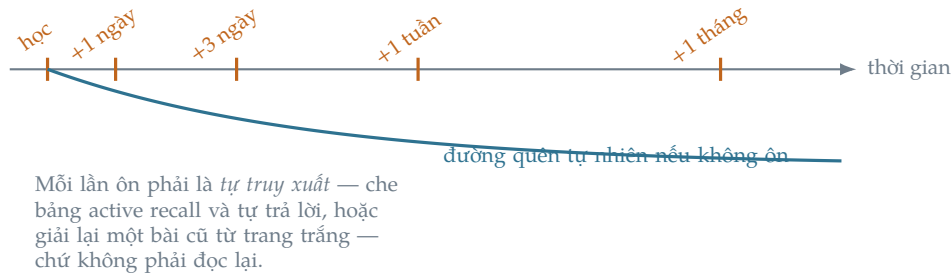
Về câu hỏi “học kiến thức mới hay giải bài”, tỉ lệ thường dùng là khoảng một phần ba thời gian cho kiến thức mới và hai phần ba cho bài tập — và tỉ lệ đó nên dịch dần về phía bài tập. Lý do: kiến thức không có nhu cầu đi kèm sẽ bay hết trong hai tuần, còn bài tập thì vừa củng cố cái cũ vừa lộ ra cái còn thiếu.

28.2 Bốn công cụ theo dõi

Nhật ký lỗi. Ba dòng cho mỗi lần sai. Sau một tháng, phân loại chúng thành ba nhóm: hiểu sai đề, sai ý tưởng, sai cài đặt. Tỷ lệ ba nhóm nói cho bạn biết phải luyện gì — và nó thường gây bất ngờ, vì phần lớn người học nghĩ mình yếu ý tưởng trong khi thực tế mất điểm nhiều nhất ở hai nhóm kia.

Danh sách mẫu. Mỗi bài giải xong, một câu tổng quát không nhắc đề bài. “Khi tìm cực trị trên mọi đoạn, cố định điểm cuối và duy trì đáp án tốt nhất kết thúc tại đó.” Sau vài tháng, danh sách này là tài sản có giá trị nhất bạn tích lũy được, và nó chính là các *khối tri thức* mà nghiên cứu về chuyên gia cờ vua mô tả ([A/02](#)).

Lịch ôn ngắt quãng. Với mỗi chủ đề đã học, ôn lại sau 1 ngày, 3 ngày, 1 tuần, 1 tháng. “Ôn” ở đây nghĩa là *tự truy xuất*: che bảng active recall và tự trả lời, hoặc giải một bài cũ từ trang trắng — không phải đọc lại.



Hình 28.1. Lịch ôn ngắt quãng. Các mốc đặt ở khoảng cách tăng dần vì trí nhớ được củng cố mạnh nhất khi việc truy xuất khó nhưng vẫn thành công: ôn quá sớm thì tốn công, ôn quá muộn thì phải học lại từ đầu.

Ba chỉ số tiến bộ. Cảm giác chủ quan gần như luôn sai, nên hãy đo. Chỉ số một: tỉ lệ giải được ở một mức khó cố định — nếu mức khó cũng tăng thì bạn không so được. Chỉ số hai: thời gian trung bình để giải một bài ở mức bạn đã quen. Chỉ số ba, và là chỉ số tốt nhất: số bài bạn *upsolve* tới nơi tới chốn mỗi tuần.

CẦN NHỚ

Ba việc chiếm chưa tới mười phút mỗi buổi nhưng tạo ra phần lớn tiến bộ: viết một câu mẫu sau mỗi bài giải xong; ghi ba dòng nhật ký sau mỗi lần sai; và *upsolve* ít nhất một bài mỗi tuần từ *trang trắng*. Nếu phải cắt lịch luyện vì bận, hãy cắt phần đọc kiến thức mới chứ đừng cắt ba việc này.

28.3 Ba lộ trình theo mục tiêu

Lộ trình 12 tuần — chuẩn bị phỏng vấn

Giả định 8–10 giờ mỗi tuần. Mục tiêu: giải được một bài mức trung bình trong 35 phút với quy trình bảy bước ở E/27.

Tuần	Nội dung	Tiêu chí kiểm tra khách quan
1	Chương A/01, A/03; mảng, hai con trỏ (B/06)	Thuộc bảng giới hạn → độ phức tạp; giải 10 bài dễ
2	Băm, tập hợp (B/08); ngăn xếp, hàng đợi (B/07)	Nhận ra dạng cửa sổ trượt trong 2 phút; 12 bài
3	Sắp xếp, tìm kiếm nhị phân (C/14, C/19)	Viết nhị phân đúng từ trang trắng, có stress test
4	Cây và đệ quy (B/09); duyệt cây	10 bài cây; viết được duyệt trước/giữa/sau không cần tra
5	Đồ thị: duyệt rộng, duyệt sâu (C/17)	Nhận ra bài đồ thị trá hình; 10 bài
6	Đồ thị: đường đi ngắn nhất, tô-pô	Chọn đúng thuật toán theo giả thiết trọng số
7	Quy hoạch động một chiều và ba lô (C/16)	Viết được năm bước ra giấy trước khi gõ, mọi bài
8	Quy hoạch động hai chuỗi và trên cây	10 bài; tự phát hiện được trạng thái thiếu
9	Tham lam (C/15); heap	Chứng minh hoặc bác bỏ được tiêu chí bằng stress test
10	Ôn tập trộn chủ đề; E/25	3 buổi thi ảo trộn chủ đề, mỗi buổi 2 bài trong 70 phút
11	Phỏng vấn thử theo cặp (E/27)	4 buổi phỏng vấn thử; ghi âm và nghe lại
12	Ôn nhật ký lỗi và danh sách mẫu	Giải lại 10 bài cũ từ trang trắng, đạt trên 80%

Lưu ý về cách dùng bảng này: cột thứ ba mới là phần quan trọng. Một tuần “xong” không phải khi bạn đọc hết nội dung mà khi bạn đạt tiêu chí kiểm tra — nếu chưa đạt thì kéo dài thêm vài ngày thay vì chạy tiếp.

Lộ trình 6 tháng — thi đấu

Giả định 10–15 giờ mỗi tuần. Cấu trúc theo tháng: tháng 1–2 phủ phần B (cấu trúc dữ liệu) và các chương C/13–C/16; tháng 3–4 phủ C/17–C/20 và D/21; tháng 5 phủ B/10, B/12, D/22; tháng 6 dành hoàn toàn cho thi ảo và upsolving.

Mỗi tuần trong sáu tháng đó nên có: một kỳ thi ảo tính giờ; upsolving toàn bộ đề đó cho tới khi giải hết hoặc hiểu hết; và hai buổi chủ đề. Đây là nhịp mà phần lớn người tiến bộ nhanh đều theo, và điểm khác biệt của họ so với người luyện nhiều mà không tiến thường nằm ở chữ “toàn bộ” trong bước upsolving.

Nhịp duy trì — người đi làm

Nếu mục tiêu là không quên và tiến chậm nhưng đều: một bài mỗi tuần ở vùng luyện, cộng một kỳ thi ảo mỗi tháng, cộng đọc lại danh sách mẫu mỗi tháng. Tổng khoảng hai giờ mỗi tuần. Điều quan trọng hơn khối lượng là tính liên tục — và câu trả lời cho câu hỏi mở đầu thứ năm nằm ở đây: lộ trình đổ vỡ vì đặt mục tiêu theo ngày bạn nhất chứ không theo ngày bình thường. Hãy đặt mức tối thiểu mà bạn chắc chắn làm được kể cả ngày tẻ nhất, rồi làm thêm khi rảnh.

28.4 Ngân hàng bài tập

Phần này gom các bài theo chủ đề. Nguồn được ghi bằng tên bài để bạn tra; các tập bài được nhắc tới gồm CSES[33], chuỗi bài quy hoạch động của AtCoder, và các bài kinh điển trên LeetCode. Cách dùng: chọn một chủ đề, làm theo thứ tự mức độ, và upsolve mọi bài không giải được trong 60 phút.

Ngân hàng — Nền tảng và mảng (B/06) (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Increasing Array, Missing Number, Repetitions	CSES Introductory[33]	Khởi động; luyện đọc đề và xử lý ca biên.
● Distinct Numbers, Apartments, Ferris Wheel	CSES Sorting[33]	Sắp xếp như nước đi mở đầu; hai con trỏ ở dạng đơn giản nhất.
●● Sum of Two Values, Sum of Three Values, Sum of Four Values	CSES Sorting[33]	Ba mức của cùng một ý: cố định bớt biến rồi hai con trỏ hoặc băm.
●● Maximum Subarray Sum, Subarray Sums I và II	CSES Sorting[33]	Tổng tiền tố; mẫu “điều kiện đoạn thành điều kiện hai điểm nút”.
●● Playlist, Traffic Lights, Room Allocation	CSES Sorting[33]	Cửa sổ trượt và tập hợp có thứ tự; kiểm tính đơn điệu trước khi cài.
●●● Subarray Divisibility, Subarray Distinct Values, Array Division	CSES Sorting[33]	Ba hướng khác nhau: modulo, cửa sổ, nhị phân trên đáp án.
●●● Trapping Rain Water, Longest Substring Without Repeating Characters	LeetCode	Hai bài phỏng vấn kinh điển; giải mỗi bài bằng hai cách.

Ngân hàng — Ngăn xếp, hàng đợi, heap (B/07) (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Valid Parentheses, Min Stack	LeetCode	Ngăn xếp cơ bản; chú ý ca dư ngoặc mở.
•• Nearest Smaller Values	CSES Sorting[33]	Ngăn xếp đơn điệu chuẩn; viết bất biến trước.
•• Sliding Window Maximum, Sliding Window Median	LeetCode; CSES[33]	Deque đơn điệu và hai heap; so hai công cụ.
•• Josephus Problem I và II, Towers	CSES Sorting[33]	Cấu trúc phù hợp quyết định độ dài code.
••• Largest Rectangle in Histogram, Maximal Rectangle	LeetCode	Ngăn xếp đơn điệu ở mức khó; bài thứ hai quy về bài thứ nhất.
••• Tasks and Deadlines, Reading Books, Factory Machines	CSES Sorting[33]	Tham lam kèm heap và nhị phân trên đáp án.

Ngân hàng – Truy vấn đoạn và DSU (B/10, B/11) (mức: • khởi động •• chuẩn ••• khó)

Bài	Nguồn	Điều cần nhận ra
• Static Range Sum Queries, Static Range Minimum Queries	CSES Range Queries[33]	Tổng tiền tố và sparse table; hiểu vì sao không hoán đổi được.
•• Dynamic Range Sum Queries, Dynamic Range Minimum Queries, Range Xor Queries	CSES Range Queries[33]	Fenwick và cây phân đoạn; chú ý phần tử trung hoà.
•• Range Update Queries, Hotel Queries, List Removals	CSES Range Queries[33]	Mảng hiệu, đi xuống trên cây, và cây chỉ số cho vị trí thứ k .
•• Road Construction, Road Reparation, Building Roads	CSES Graph[33]	DSU và cây khung nhỏ nhất; đếm số nhóm sau mỗi cạnh.
••• Range Updates and Sums, Polynomial Queries, Subarray Sum Queries	CSES Range Queries[33]	Lazy propagation; bài cuối cần thiết kế phép gộp bốn giá trị.
••• Distinct Values Queries, Salary Queries, Forest Queries II	CSES Range Queries[33]	Nén toạ độ, xử lý ngoại tuyến, và cấu trúc hai chiều.
••• New Roads Queries, Dynamic Connectivity	CSES Advanced[33]	DSU với thời gian; kết hợp B/10 và B/11.

Ngân hàng – Quy hoạch động (C/16) (mức: • khởi động •• chuẩn ••• khó)

Bài	Nguồn	Điều cần nhận ra
• Frog 1, Frog 2, Vacation	AtCoder Educational DP	Ba bài nhập môn; viết đủ năm bước ra giấy.
• Dice Combinations, Minimizing Coins, Coin Combinations I và II	CSES DP[33]	Hai bài cuối khác nhau đúng ở thứ tự vòng lặp — giải thích được vì sao.
•• Knapsack 1, Knapsack 2, LCS	AtCoder Educational DP	Ba lô hai chiều và họ hai chuỗi; chú ý mẹo đảo vai trò ở Knapsack 2.
•• Book Shop, Array Description, Edit Distance, Removal Game	CSES DP[33]	Bốn họ trạng thái khác nhau trong cùng một buổi.
•• Grid 1, Coins, Sushi, Deque	AtCoder Educational DP	Đếm, xác suất, kỳ vọng, và trò chơi — bốn kiểu mục tiêu.
••• Increasing Subsequence, Projects, Elevator Rides, Counting Towers	CSES DP[33]	Tăng tốc bằng cấu trúc dữ liệu và quy hoạch động mặt nạ bit.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Matching, Independent Set, Subtree, Grouping	AtCoder Educational DP	Mặt nạ bit và quy hoạch động trên cây, mức khó.
●●● Digit Sum, Permutation, Intervals, Tower	AtCoder Educational DP	Quy hoạch động trên chữ số và các bài cần sắp xếp trước khi quy hoạch động.

Ngân hàng – Đồ thị (C/17, C/18) (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Counting Rooms, Labyrinth, Building Teams	CSES Graph[33]	Duyệt rộng và duyệt sâu; truy vết đường đi.
●● Message Route, Round Trip, Monsters	CSES Graph[33]	Duyệt rộng nhiều nguồn và phát hiện chu trình.
●● Shortest Routes I và II, Flight Discount, High Score	CSES Graph[33]	Dijkstra, Floyd–Warshall, đồ thị nhiều tầng, và chu trình âm.
●● Course Schedule, Longest Flight Route, Game Routes	CSES Graph[33]	Quy hoạch động theo thứ tự tô-pô.
●●● Planets and Kingdoms, Giant Pizza, Flight Routes Check	CSES Graph[33]	Thành phần liên thông mạnh và ứng dụng cho bài logic hai giá trị.
●●● Company Queries I và II, Distance Queries, Counting Paths	CSES Tree[33]	Nhảy nhị phân, tổ tiên chung, và kỹ thuật hiệu trên cây.
●●● Download Speed, Police Chase, School Dance, Distinct Routes	CSES Graph[33]	Luồng cực đại, lát cắt nhỏ nhất, ghép đôi, và truy vết đường.
●●● Necessary Roads, Necessary Cities	CSES Advanced[33]	Cầu và khớp; chú ý cạnh song song.

Ngân hàng – Chuỗi, số học, hình học (B/12, D/21, D/22) (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Word Combinations, String Matching	CSES String[33]	Trie và so khớp; làm bằng cả băm lẫn hàm Z.
●● Finding Borders, Finding Periods, Longest Palindrome	CSES String[33]	Hàm tiền tố và Manacher.
●●● Distinct Substrings, Repeating Substring, Substring Order I	CSES String[33]	Mảng hậu tố và LCP; bài cuối là lý do rõ nhất để học chúng.
● Exponentiation, Counting Divisors	CSES Math[33]	Luỹ thừa nhanh và sàng lưu ước nhỏ nhất.
●● Binomial Coefficients, Distributing Apples, Bracket Sequences I	CSES Math[33]	Tổ hợp modulo và mô hình chia kẹo.
●●● Fibonacci Numbers, Graph Paths I, Exponentiation II	CSES Math[33]	Luỹ thừa ma trận và định lý Euler cho số mũ lớn.
● Point Location Test, Polygon Area	CSES Geometry[33]	Vị từ định hướng và công thức dây giày, hoàn toàn số nguyên.
●● Line Segment Intersection, Point in Polygon, Polygon Lattice Points	CSES Geometry[33]	Xử lý suy biến; kiểm bằng tọa độ nhỏ.
●●● Convex Hull, Minimum Euclidean Distance	CSES Geometry[33]	Quét đơn điệu và chia để trị.

I Ngân hàng – Tổng hợp và kỹ năng nghề (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
●● Apple Division, Meet in the Middle	CSES Introductory, Advanced[33]	Duyệt tập con và gặp nhau ở giữa — nhận ra qua giới hạn n .
●● Viết khung stress test và dùng nó tìm một phân ví dụ thật	E/25	Công cụ dùng lại cả đời; đây là bài tập có lợi ích lâu nhất.
●● Xây thư viện mẫu sáu cấu trúc, mỗi cái có stress test	E/27	DSU, cây phân đoạn lazy, Dijkstra, luồng, hình học, số học modulo.
●●● Thi ảo một đề cũ, upsolve toàn bộ	Codeforces / AtCoder	Lập lại hằng tuần; đây là nguồn tiến bộ chính.
●●● Chọn 10 bài đã giải và viết một câu mẫu cho mỗi bài	Danh sách mẫu của bạn	Kiểm tra: câu đó có dùng lại được cho bài khác không.
●●● Đọc mã nguồn một thư viện bạn dùng và tìm một thuật toán trong sách này	E/26	So bản kỹ thuật hoá với bản trong sách; ghi lại khác biệt.

I 28.5 Gốc rễ: vì sao lộ trình lại có hình dạng như vậy

Các lựa chọn thiết kế trong chương này không tùy tiện; mỗi cái đến từ một kết quả đã nhắc ở A/02.

Việc *xen kẽ chủ đề* thay vì học tuần tự từng khối đến từ một phát hiện lặp lại trong nghiên cứu về học tập: luyện xen kẽ nhiều loại bài cho kết quả kém hơn khi đo ngay sau buổi tập, nhưng tốt hơn rõ rệt khi đo sau vài tuần. Lý do là luyện tuần tự cho phép bạn dùng cùng một phương pháp cho cả loạt bài mà không phải *chọn* phương pháp — và chọn phương pháp mới là kỹ năng cần cho bài thật. Đây cũng là lý do mục 1 khuyến có buổi thi trộn chủ đề mỗi tuần.

Việc *ôn ngắt quãng* đến từ một quan sát còn cổ hơn: trí nhớ được củng cố mạnh nhất khi việc truy xuất là khó nhưng vẫn thành công — tức là ngay trước lúc sắp quên. Ôn quá sớm thì tốn công mà không thêm gì; ôn quá muộn thì phải học lại. Các khoảng cách tăng dần là cách xấp xỉ thực dụng cho điều đó.

Việc đặt *upsolving* làm trung tâm đến từ cơ chế đã giải thích ở A/02: trong lúc vật lộn, bạn tạo ra những lỗ hổng rất cụ thể trong hiểu biết, và thông tin mới chỉ dính khi có lỗ hổng để lấp. Đó cũng là lý do chỉ số tiến bộ tốt nhất không phải số bài đã đọc lời giải mà là số bài đã upsolve tới nơi.

Cuối cùng, việc đặt *mức tối thiểu rất thấp* cho nhịp duy trì đến từ một quan sát thực dụng chứ không phải nghiên cứu: lộ trình hổng không phải vì người ta lười mà vì họ thiết kế cho ngày lý tưởng. Một lịch đòi hai giờ mỗi ngày sẽ đứt ở tuần thứ ba, và cái đứt đó tốn hơn nhiều so với việc chỉ làm ba mươi phút. Tính liên tục có giá trị kép: nó giữ trí nhớ, và nó giữ thói quen — thứ khó xây lại hơn kiến thức.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Nghiên cứu về luyện xen kẽ	Học tuần tự từng khối cho cảm giác tiến nhanh hơn.	Xen kẽ kém hơn khi đo ngay, tốt hơn khi đo sau vài tuần — vì nó buộc bạn <i>chọn</i> phương pháp, kỹ năng cần cho bài thật.

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó (tiếp)
Nghiên cứu về ôn ngắt quãng	Ôn lại ngay thì tốn công, ôn muộn thì phải học lại.	Củng cố mạnh nhất khi truy xuất khó nhưng vẫn thành công; khoảng cách tăng dần là cách xấp xỉ thực dụng.
1993 — Ericsson và cộng sự	Vì sao cùng số giờ mà người tiến người không.	Bốn điều kiện của luyện có chủ đích; toàn bộ thiết kế của chương này suy ra từ đó (A/02).
Vấn hoá thi đấu hiện đại	Người luyện nhiều mà không tiến.	Upsolving tới nơi tới chốn thành chuẩn; số bài upsolve là chỉ số tốt hơn số bài đã xem lời giải.
Kinh nghiệm thực tế về duy trì thói quen	Lộ trình đổ vỡ sau vài tuần.	Đặt mức tối thiểu theo ngày tệ nhất chứ không theo ngày lý tưởng; tính liên tục quý hơn khối lượng.

28.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thể thao	Chu kỳ hoá: xen kẽ khối lượng, cường độ và phục hồi	Cấu trúc ba loại buổi mỗi tuần chính là chu kỳ hoá thu nhỏ. Cùng bài học: cường độ cao liên tục không bền, và nghỉ là một phần của kế hoạch chứ không phải sự thất bại.
Âm nhạc	Tập chậm từng đoạn khó, rồi ghép	Buổi chủ đề tương ứng với tập đoạn khó; buổi thi ảo tương ứng với chơi cả bản. Thiếu loại thứ hai thì bạn giỏi từng đoạn mà không chơi được cả bài.
Y khoa, hàng không	Rà soát sau ca / sau chuyến bay	Nhật ký lỗi là bản cá nhân của các quy trình đó. Điểm mạnh của bản thể chế: nó bắt buộc và không phụ thuộc tâm trạng — điều bạn có thể bắt buộc bằng cách gắn nó vào cuối mỗi buổi.
Học máy	Học theo chương trình, dừng sớm, chọn mẫu khó	Vùng luyện tương ứng với việc chọn mẫu có gradient hữu ích: quá dễ thì không học được gì, quá khó thì tín hiệu nhiễu. Khác biệt: bạn tự chọn được bài, mô hình thì không.
Quản lý công việc	Mục tiêu đo được, mốc kiểm tra định kỳ	Cột “tiêu chí kiểm tra khách quan” trong bảng lộ trình chính là điều này. Không có tiêu chí thì “xong một tuần” trở thành “đã đọc hết”, vốn không đo được gì.
Nghiên cứu khoa học	Số thí nghiệm, bình duyệt	Danh sách mẫu là sổ ghi kết quả; thứ người tự học thiếu nhất là bình duyệt, và thay thế gần nhất là viết editorial công khai hoặc luyện theo cặp.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì “nhiều” thường có nghĩa là nhiều bài *vừa sức*, mà vừa sức thì không tạo ra sai lệch dự đoán để học. Nó còn khuyến khích một thói quen tệ: đọc lời giải sớm để kịp chỉ tiêu số bài. Điều kiện đúng gồm bốn phần — mục tiêu hẹp, mức hơi quá sức, phản hồi nhanh, có sửa sai — và “giải nhiều” chỉ thoả phần cuối một cách hình thức. Chỉ số tốt hơn là số bài bạn giải xong *sau khi đã bí*.
- A2. 10 phút ôn (đọc nhật ký lỗi và một bảng active recall), 40 phút giải một bài ở vùng luyện có bấm giờ, 10 phút cho bước bốn — viết một câu mẫu và ghi nhật ký nếu sai. Nếu phải cắt, hãy cắt phần đọc kiến thức mới chứ đừng cắt 10 phút cuối, vì đó là phần biến một lần giải bài thành

thứ dừng lại được. Và quan trọng hơn tỉ lệ là tính đều đặn: một giờ mỗi ngày trong sáu tháng hơn hẳn tám giờ mỗi cuối tuần.

- A3.** Bằng ba chỉ số đo được. Tỉ lệ giải được ở một mức khó *cố định* — điểm mấu chốt là mức phải cố định, nếu không bạn đang so hai thứ khác nhau. Thời gian trung bình cho một bài ở mức đã quen. Và số bài upsolve tới nơi mỗi tuần, chỉ số tốt nhất trong ba cái. Cảm giác chủ quan sai theo cả hai chiều: sau một buổi trôi chảy bạn thấy mình giỏi lên, còn khi gặp một chủ đề mới bạn thấy mình tệ đi — cả hai đều không phản ánh tiến bộ thật.
- A4.** Khoảng một phần ba cho kiến thức mới, hai phần ba cho bài tập, và tỉ lệ nên dịch dần về phía bài tập theo thời gian. Lý do: kiến thức không có nhu cầu đi kèm sẽ bay hết trong hai tuần, còn bài tập vừa củng cố cái cũ vừa lộ ra cái còn thiếu — nên nó tự dẫn bạn tới nội dung cần học tiếp. Ngoại lệ: khi bắt đầu một chủ đề hoàn toàn mới, hãy dành nguyên một buổi cho kiến thức rồi cài thử ngay trong buổi đó.
- A5.** Vì phần lớn lộ trình được thiết kế cho ngày lý tưởng chứ không cho ngày bận nhất. Một lịch đòi hai giờ mỗi ngày sẽ đứt ở tuần thứ ba, và cái đứt đó tốn hơn nhiều so với việc chỉ làm ba mươi phút mỗi ngày — vì xây lại thói quen khó hơn giữ nó. Cách thiết kế chống lại điều đó gồm ba phần: đặt mức tối thiểu mà bạn chắc chắn làm được kể cả ngày tệ nhất; làm cho tiến bộ nhìn thấy được bằng một bảng đánh dấu; và gắn buổi luyện vào một khung giờ cố định thay vì “khi nào rảnh”.

NÓI LẠI THẬT ĐƠN GIẢN

Sau khi gấp sách lại, câu hỏi duy nhất còn lại là sáng mai làm gì. Câu trả lời gọn thế này. Mỗi buổi luyện, hãy dành mười phút đầu nhìn lại những lỗi cũ của mình, bốn mươi phút giải một bài đủ khó để bạn phải vật lộn, và mười phút cuối ghi lại hai thứ: ý chính của bài vừa giải viết thành một câu không nhắc tới đề bài, và nếu có sai thì sai vì sao. Mười phút cuối là phần dễ bị cắt nhất và cũng là phần tạo ra phần lớn tiến bộ.

Mỗi tuần, hãy có một buổi làm bài trộn nhiều chủ đề, có bấm giờ, như thi thật. Lý do: khi bạn luyện một chủ đề liên tiếp, bạn không phải *chọn* phương pháp — bạn biết trước bài này là quy hoạch động. Bài thật không nói cho bạn biết điều đó, nên kỹ năng chọn phải luyện riêng.

Và sau mỗi lần bí, hãy quay lại làm cho xong chứ đừng chỉ đọc lời giải rồi gật đầu. Cảm giác “à hoá ra thế” rất dễ nhầm với việc đã học được. Phép thử duy nhất đáng tin là đóng lời giải lại và tự viết từ trang trắng.

Cuối cùng, về khối lượng: đừng lên kế hoạch cho ngày đẹp trời. Hãy chọn một mức nhỏ mà bạn chắc chắn làm được kể cả hôm mệt nhất — ba mươi phút cũng được — rồi làm thêm khi rảnh. Lộ trình hồng gần như luôn vì lý do này, chứ không phải vì thiếu tài liệu hay thiếu bài tập.

Một câu: Tiến bộ đến từ những bài bạn giải xong sau khi đã bí, không phải từ những bài bạn đọc lời giải.

Chỗ không rút gọn được: Không có lịch luyện nào đúng cho mọi người; ba lộ trình ở đây là điểm xuất phát để bạn sửa theo dữ liệu của chính mình.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn điều kiện chi phối mọi lịch luyện	Mục tiêu hẹp; mức hơi quá sức; phản hồi nhanh; có sửa sai. Thiếu một thì thời gian trôi mà năng lực đứng.
••• Cấu trúc một buổi 60 phút	10 phút ôn + 40 phút giải bài có bấm giờ + 10 phút viết câu mấu và ghi nhật ký. Dừng cắt 10 phút cuối.
••• Ba loại buổi trong tuần	Buổi kiến thức, buổi bài tập cùng chủ đề, buổi thi ảo trộn chủ đề. Tỉ lệ khởi đầu 2:3:1, dịch dần về phía thi.
••• Vì sao phải trộn chủ đề	Luyện tuần tự thì bạn không phải <i>chọn</i> phương pháp; bài thật không cho biết trước nên kỹ năng chọn phải luyện riêng.
••• Bốn công cụ theo dõi	Nhật ký lỗi; danh sách mấu; lịch ôn ngắt quãng (1 ngày, 3 ngày, 1 tuần, 1 tháng); ba chỉ số tiến bộ.
••• Ba chỉ số tiến bộ	Tỉ lệ giải được ở mức khó <i>cố định</i> ; thời gian trung bình ở mức đã quen; số bài upsolve tới nơi mỗi tuần (tốt nhất).

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● Phân loại nhật ký lỗi	Hiểu sai đề / sai ý tưởng / sai cài đặt. Tỷ lệ ba nhóm quyết định bạn nên luyện gì, và nó thường gây bất ngờ.
●● Tỷ lệ kiến thức và bài tập	Khoảng 1:2, dịch dần về phía bài tập; kiến thức không có nhu cầu đi kèm sẽ quên trong hai tuần.
●● Ôn nghĩa là gì	Tự truy xuất — che bảng active recall và tự trả lời, hoặc giải lại từ trang trắng. <i>Không</i> phải đọc lại.
●● Vì sao lộ trình đồ vỡ	Thiết kế cho ngày lý tưởng thay vì ngày bận nhất. Đặt mức tối thiểu theo ngày tệ nhất, làm thêm khi rảnh.
●● Tiêu chí “xong một tuần”	Không phải đọc hết nội dung mà là đạt tiêu chí kiểm tra khách quan; chưa đạt thì kéo dài thay vì chạy tiếp.
● Nhịp duy trì cho người đi làm	Một bài mỗi tuần ở vùng luyện, một kỳ thi ảo mỗi tháng, đọc lại danh sách mẫu mỗi tháng — khoảng hai giờ mỗi tuần.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên bắt đầu từ chương nào nếu đã biết cơ bản?

| Đọc [A/01](#) và [A/02](#) trước, dù đã biết — chúng quyết định cách bạn dùng phần còn lại. Sau đó vào thẳng phần C: [C/15](#), [C/16](#), [C/17](#), [C/19](#) là bốn chương có tỉ lệ ứng dụng cao nhất. Quay lại phần B khi một bài đòi hỏi một cấu trúc bạn chưa vững, và đọc [E/25](#) sớm vì stress test là công cụ bạn nên có ngay từ đầu.

2. Làm sao chọn được bài ở đúng vùng luyện?

| Dùng hệ thống phân loại độ khó của nền tảng bạn dùng và chọn mức mà bạn giải được khoảng một nửa. Kiểm tra định kỳ: nếu ba buổi liên tiếp bạn giải hết trong dưới 15 phút thì nâng mức; nếu hai buổi liên tiếp không giải được gì và đọc lời giải vẫn mù mờ thì hạ. Với các tập bài có thứ tự như CSES[33], cứ đi tuần tự trong một chủ đề cũng đủ tạo độ khó tăng dần.

3. Tôi quên hết những gì đã học ba tháng trước. Làm sao?

| Đó là hiện tượng bình thường và có cách chữa cụ thể: ôn ngắt quãng bằng *truy xuất* chứ không phải đọc lại. Mỗi tuần, dành mười phút giải lại một bài cũ ngẫu nhiên từ trang trắng và che bảng active recall của một chương cũ để tự trả lời. Nếu bạn có danh sách mẫu, việc đọc lại nó mỗi tháng cũng rất hiệu quả vì mỗi câu mẫu là một khoá gợi nhớ cho cả một nhóm bài.

4. Nên dùng một ngôn ngữ hay nhiều ngôn ngữ?

| Một, và thành thạo nó. Trong thi đấu, việc gõ nhanh và không phải nghĩ về cú pháp là lợi thế thật; trong phỏng vấn, sự lúng túng về ngôn ngữ làm mất thời gian quý. Học ngôn ngữ thứ hai khi có lý do cụ thể — công việc yêu cầu, hoặc bạn cần một tính năng mà ngôn ngữ hiện tại không có. Điều đáng đầu tư hơn là hiểu sâu các cấu trúc dữ liệu chuẩn của ngôn ngữ bạn dùng, kể cả hằng số và hành vi ở trường hợp biên.

5. Tôi luyện một mình, không có ai đánh giá. Thay thế thế nào?

| Ba thứ thay thế được phần lớn vai trò của người hướng dẫn. Chấm bài tự động cho phản hồi đúng/sai tức thì. Stress test cho phản hồi ở đâu sai ([E/25](#)). Và việc viết editorial công khai hoặc giải thích cho người khác cho phản hồi về việc bạn có thực sự hiểu không — chỗ bạn viết không trôi chính là chỗ bạn chưa hiểu. Nếu tìm được một bạn luyện cùng để phỏng vấn thử lẫn nhau thì đó là bổ sung có giá trị nhất.

6. Bao lâu thì thấy tiến bộ rõ rệt?

| Với nhịp 8–10 giờ mỗi tuần và luyện đúng cách, phần lớn người thấy khác biệt rõ sau khoảng 8–12 tuần — đủ để nâng một bậc độ khó. Nhưng có một giai đoạn khó chịu ở khoảng tuần thứ ba tới

thứ sáu, khi bạn đã biết nhiều hơn nhưng chưa nhận diện đủ nhanh, nên cảm giác không tiến. Đó là lúc các chỉ số đo được ở mục 2 có giá trị nhất: chúng cho thấy tiến bộ mà cảm giác chưa thấy.

7. Sau khi đọc hết quyển sách này thì đọc gì tiếp?

| Chọn theo mục tiêu, và chỉ một quyển làm trực chính. Muốn nhận diện bài toán rộng hơn: sách của Skiena[15], đặc biệt nửa danh mục. Muốn học cách *thiết kế* thuật toán: Kleinberg–Tardos[28]. Muốn thi đấu: Laaksonen[17] và các bài tập kèm theo. Muốn đi sâu về chứng minh và đệ quy: Erickson[29]. Và quan trọng hơn mọi quyển sách: duy trì nhịp giải bài, vì đọc thêm mà không giải thì chỉ tăng cảm giác quen thuộc.

Kết chương và kết sách

Quyển sách này bắt đầu bằng một quan sát ở chương A/01: khi bạn bí, cái thiếu hầu như không bao giờ là một thuật toán chưa từng nghe tên, mà là một quy trình để chạy khi chưa biết làm gì. Hai mươi tám chương sau, quy trình đó đã được cụ thể hoá: bốn bước để đối mặt với bài lạ, hai thước đo để tự chấm ý tưởng, một kho cấu trúc dữ liệu chọn theo thao tác cần làm rõ, tám khuôn tư duy để sinh lời giải, một ý thức về giới hạn để biết khi nào nên dừng, và một bộ kỹ năng nghề để đưa ý tưởng thành chương trình chạy được.

Nhưng phần quyết định không nằm trong sách. Nó nằm ở việc bạn có mở một bài ra vào sáng mai hay không, có viết một câu mẫu sau khi giải xong hay không, và có quay lại làm cho xong bài mình đã bí hay không. Ba việc đó tốn mười phút mỗi ngày và chúng là toàn bộ khác biệt giữa người đọc xong một quyển sách và người giỏi lên.

Chúc bạn luyện đều, bí nhiều, và mỗi lần bí đều học được một điều mới.

Tài liệu tham khảo có chú giải

TÓM TẮT

Danh sách *Nguồn* ở cuối sách liệt kê đầy đủ mọi tài liệu được trích dẫn, xếp theo thứ tự xuất hiện. Phụ lục này làm việc khác: nó nhóm các tài liệu theo *mục đích sử dụng*, nói rõ mỗi quyển dùng để làm gì, đọc vào lúc nào, và đọc bao nhiêu là đủ. Nếu chỉ nhớ một điều: chọn *một* quyển làm trục chính theo mục tiêu của bạn và dùng những quyển còn lại để tra — đọc song song ba giáo trình là cách chắc chắn nhất để không đọc xong quyển nào.

A. Bốn quyển làm trục chính — chọn một theo mục tiêu

Tự học tổng quát, muốn nhận diện được bài toán: Skiena, *The Algorithm Design Manual*[15]. Nửa đầu dạy kỹ thuật, nửa sau là danh mục tra ngược “bài của tôi trông thế này thì nó tên là gì”. Đây là quyển hợp nhất với người đi làm. Đọc nửa đầu tuần tự, nửa sau chỉ tra.

Muốn học cách nghĩ ra thuật toán: Kleinberg và Tardos, *Algorithm Design*[28]. Điểm mạnh riêng: mỗi kỹ thuật được dẫn ra từ một bài toán thật, và phần tham lam cùng phần luồng là phần trình bày tốt nhất mà tôi biết. Đọc chương 4, 6, 7 trước.

Thi đấu: Laaksonen, *Guide to Competitive Programming*[17] làm trục chính; Halim và cộng sự, *Competitive Programming 4*[16] làm danh mục kỹ thuật kèm bài luyện. Đọc kèm bộ bài CSES[33] theo đúng thứ tự chủ đề.

Muốn nền chứng minh vững, đặc biệt về đệ quy: Erickson, *Algorithms*[29], bản điện tử miễn phí. Phần quay lui và quy hoạch động dạy cách viết chứng minh song song với thuật toán — đúng tinh thần chương A/04.

B. Tra cứu, không đọc tuần tự

Cormen và cộng sự, *Introduction to Algorithms*[27]: tham chiếu chuẩn của ngành. Dùng để tra định nghĩa chính xác và chứng minh đầy đủ. Đọc tuần tự gần như luôn thất bại; đây không phải lỗi của bạn mà là do sách được thiết kế cho vai trò khác.

Knuth, *The Art of Computer Programming*[8], [9]: phân tích tới hằng số, độ sâu không sách nào bằng. Dùng khi cần hiểu tường tận *một* chủ đề cụ thể. Bài tập có xếp hạng độ khó 0–50; hãy chọn theo mức thay vì làm hết.

Sedgewick và Wayne, *Algorithms*[22]: mạnh về cài đặt chạy được và về thực nghiệm. Hợp khi bạn muốn thấy code thật kèm số đo thật.

Graham, Knuth, Patashnik, *Concrete Mathematics*[30]: tra khi phân tích độ phức tạp đựng tổng hoặc truy hồi khó (A/05).

cp-algorithms[18] và *KACTL*[7]: cài đặt ngắn gọn đã được kiểm chứng. Dùng để *đối chiếu* sau khi tự viết, không phải để chép.

C. Tư duy giải bài và cách luyện

Pólya, How to Solve It[1]: mông, đọc trong một buổi, và nên đọc trước mọi thứ khác. Nền của chương A/01.

Zeitz, The Art and Craft of Problem Solving[4] và *Engel*, Problem-Solving Strategies[5]: luyện tư duy bằng bài toán olympiad. Đọc khi muốn nâng khả năng nghĩ chứ không phải học thêm thuật toán.

Schoenfeld, Mathematical Problem Solving[2]: nghiên cứu thực nghiệm về quá trình giải bài; nguồn của khái niệm tự giám sát ở A/01.

Ericsson và Pool, Peak[24] cùng bài báo gốc năm 1993[3]: cơ sở của toàn bộ chương E/28. Đọc bài báo nếu muốn phần lõi; đọc sách nếu muốn ví dụ.

Lehman, Leighton, Meyer, Mathematics for Computer Science[39]: nền toán rời rạc viết cho người làm tin học, bản điện tử miễn phí. Chương “State Machines” là nguồn của khung bất biến bảo toàn ở A/04; các chương về quy nạp và tổ hợp thay thế được phần lớn nội dung của A/05.

Bentley, Programming Pearls[6]: đọc rải rác, mỗi lần một chương ngắn. Là quyển gần nhất với công việc thật trong danh sách này.

Norvig, “Teach Yourself Programming in Ten Years”[26]: một bài viết ngắn, đọc mười phút, chỉnh lại kỳ vọng về tốc độ tiến bộ.

D. Chuyên sâu theo miền

Độ khó và NP: Garey và Johnson[31] vẫn là danh mục bài toán NP-đầy đủ đầy đủ nhất. Dừng khi cần tra “bài này đã biết là khó chưa”.

Ngẫu nhiên hoá: Motwani và Raghavan[40] cho chiều sâu; Mitzenmacher và Upfal[82] dễ vào hơn và dạy luôn phần xác suất cần thiết.

Xấp xỉ: Williamson và Shmoys[83] cho lý thuyết hiện đại; Vazirani[84] ngắn gọn hơn.

Tham số hoá: Cygan và cộng sự[74] — tài liệu chuẩn cho hướng “khó theo n nhưng dễ theo k ” ở D/23.

Hình học tính toán: de Berg và cộng sự[85]; đọc kèm cảnh báo về số thực ở D/22.

Cấu trúc dữ liệu nâng cao: khoá học công khai của Demaine[86] cho cây bền vững, cấu trúc thời gian, và các kết quả gần biên.

Luồng và mạng: Ahuja, Magnanti, Orlin[87] cho chiều sâu vận trù học.

Cấu trúc bất biến: Okasaki[46] nếu bạn làm việc với ngôn ngữ hàm hoặc cần cấu trúc bền vững.

E. Bài báo gốc đáng đọc trực tiếp

Nhiều kết quả trong sách này dễ hiểu hơn khi đọc bản gốc, vì tác giả nói rõ *vì sao* họ cần nó.

Hoare về Quicksort[19] và về cơ sở tiên đề cho chương trình[12]; Dijkstra về đường đi ngắn nhất[67] — hai trang, đọc trong mười phút. Tarjan về duyệt sâu[64] và về hợp nhất tập hợp[49]; Knuth về “tối ưu hoá sớm”[10] — đọc để thấy câu đó bị trích cụt thế nào. Cook[88] và Karp[89] cho khởi đầu của lý thuyết NP. Bentley và McIlroy về kỹ thuật hoá một hàm sắp xếp[14] — tài liệu tốt nhất về khoảng cách giữa thuật toán trong sách và hàm dùng được. Frigo và cộng sự về

cache-oblivious[78]. McSherry và cộng sự về chi phí của khả năng mở rộng[38] — ngắn, và thay đổi cách bạn đọc mọi báo cáo hiệu năng.

Từ đợt bổ sung 2026-09-07, mỗi chương có mục chuyên sâu đều kèm khối **Đọc thêm** riêng với đầy đủ nơi công bố và năm; danh sách dưới đây chỉ gom lại những công trình mới thêm, để tiện tra theo chủ đề.

Cấu trúc dữ liệu. Fredman và Tarjan về Fibonacci heap (*Journal of the ACM*, 1987) đọc kèm ngay với Larkin, Sen, Tarjan (*ALLENEX*, 2014) — một bên là cận, một bên là số đo, và đọc cùng lúc thì thấy rõ khoảng cách giữa hai thứ[36], [41]. Sleator và Tarjan về splay tree (*Journal of the ACM*, 1985) đọc kèm Demaine và cộng sự về Tango tree (*SIAM Journal on Computing*, 2007) cho câu hỏi tối ưu động[35], [45]. Pagh và Rodler về băm cuckoo (*Journal of Algorithms*, 2004) và Pătraşcu cùng Thorup về băm bảng tra (*Journal of the ACM*, 2012) cho phía bảo đảm của bảng băm[42], [43]. Bentley và Saxe (*Journal of Algorithms*, 1980) cho cách động hoá một cấu trúc tĩnh, và Fenwick (*Software: Practice and Experience*, 1994) cho bài gốc của cây chỉ số nhị phân[47], [90].

Giới hạn và cận dưới. Tarjan (*Journal of Computer and System Sciences*, 1979) cùng Fredman và Saks (*STOC*, 1989) cho biết “ α là tối ưu” nghĩa là gì trong từng mô hình[50], [51]. Backurs và Indyk (*SIAM Journal on Computing*, 2018; bản hội nghị *STOC* 2015) và Gajentaan cùng Overmars (*Computational Geometry*, 1995) là hai cửa vào tốt nhất cho độ phức tạp tĩnh[76], [77].

Biên giới hiện nay. Harvey và van der Hoeven về phép nhân $O(n \log n)$ (*Annals of Mathematics*, 2021); Agrawal, Kayal, Saxena về “PRIMES is in P” (*Annals of Mathematics*, 2004); Duan và cộng sự về việc phá rào cản sắp xếp cho đường đi ngắn nhất (*STOC*, 2025); Chen và cộng sự về luồng cực đại trong thời gian gần tuyến tính (*FOCS*, 2022)[56], [66], [70], [72]. Bốn bài này chỉ nên đọc phần mở đầu — ở đó các tác giả kể lại lịch sử cận trên của bài toán, và đó mới là phần có giá trị lâu dài cho người học.

Quy hoạch động và hình học. Yao (*SIAM Journal on Algebraic and Discrete Methods*, 1982) cho bất đẳng thức tứ giác, Hirschberg (*Communications of the ACM*, 1975) cho truy vết trong bộ nhớ tuyến tính, Aggarwal và cộng sự (*Algorithmica*, 1987) cho SMAWK, Shewchuk (*Discrete & Computational Geometry*, 1997) cho vị từ hình học tin cậy[61]–[63], [73].

F. Nguồn bài tập

CSES Problem Set[33]: có thứ tự, phủ đúng các chủ đề của phần B và C, và là nơi tôi lấy phần lớn bài trong ngân hàng ở E/28. Nếu chỉ chọn một nguồn, chọn nguồn này.

Chuỗi bài quy hoạch động của AtCoder: 26 bài phủ gần hết các họ trạng thái ở C/16.

Codeforces: bài phân loại theo mức và có thi ảo — nguồn tốt nhất để luyện dưới sức ép (E/27).

LeetCode: hợp cho chuẩn bị phỏng vấn hơn là cho thi đấu; dùng kèm McDowell[32] trong 4–6 tuần trước buổi phỏng vấn, không sớm hơn.

Skiena và Revilla, Programming Challenges[91]: bài cũ nhưng phân loại tốt và có phần bình luận cách nghĩ.

G. Chọn nguồn thế nào cho khỏi hỏng cây kiến thức

Một nguồn sai đưa vào cây tri thức sẽ âm thầm làm hỏng mọi thứ dựng lên trên nó, và vài năm sau bạn sẽ không nhớ nổi ý sai đến từ đâu. Vì vậy quyển này giữ một quy tắc chọn nguồn cố định.

Nguồn được tin: bài đã qua phản biện ở hội nghị hoặc tạp chí đầu ngành — với thuật toán là *Journal of the ACM*, *SIAM Journal on Computing*, *Algorithmica*, *Journal of Algorithms*, và các kỷ yếu STOC, FOCS, SODA, ALENEX; với phần toán là các tạp chí toán chuẩn — cùng các sách giáo trình đã qua nhiều lần tái bản. Ưu tiên thêm cho bài có mã nguồn công khai hoặc đã được người ngoài chạy lại.

Nguồn bị loại: hội nghị và tạp chí săn phí, bản tiền ấn chưa ai kiểm chứng và không kèm mã nguồn, bài chỉ khoe số đo mà giấu điều kiện thí nghiệm, và nội dung do mô hình ngôn ngữ sinh ra rồi đăng lại. Khi buộc phải dẫn một nguồn chưa qua phản biện — trong quyển này chỉ có đúng một chỗ, là con số mới nhất của số mũ nhân ma trận ở C/14 — thì nó được ghi rõ là tiền ấn ngay tại chỗ.

Một lưu ý riêng cho tài liệu lập trình thi đấu. cp-algorithms, KACTL và các blog kỹ thuật[7], [18] rất hữu ích và tôi dùng chúng thường xuyên, nhưng chúng là tài liệu cộng đồng, không qua phản biện. Cách dùng đúng: lấy chúng làm bản đối chiếu *cài đặt*, và lấy bài báo gốc làm nguồn cho *phát biểu và điều kiện áp dụng*. Rất nhiều lỗi “thuật toán này chạy trong $O(\cdot)$ ” lan truyền trong cộng đồng chỉ vì một điều kiện trong bài gốc bị bỏ quên khi viết lại.

CẦN NHỚ

Thứ tự đọc gợi ý nếu bạn bắt đầu từ đầu: Pólya (một buổi) → quyển trực chính theo mục tiêu → giải bài song song ngay từ tuần đầu → tra CLRS hoặc cp-algorithms khi vướng → đọc các bài báo gốc ở mục E khi đã quen với chủ đề tương ứng. Đừng đọc quá hai quyển cùng lúc.

Bảng thuật ngữ

TÓM TẮT

Bảng này gom các thuật ngữ dùng xuyên suốt quyển sách, kèm dạng tiếng Anh và một câu giải thích. Cột cuối trỏ tới chương nói kỹ nhất về khái niệm đó. Quy ước dùng trong sách: thuật ngữ xuất hiện lần đầu ở dạng *tiếng Việt (English)*, sau đó dùng nhất quán một dạng; tên thuật toán và tên cấu trúc dữ liệu giữ nguyên tiếng Anh vì đó là dạng bạn gặp khi tra cứu và khi đọc mã.

A. Tư duy và phân tích

Tiếng Việt	English	Nghĩa ngắn	Chương
heuristic	heuristic	Mẹo tìm đường, không đảm bảo thành công nhưng tăng xác suất ra ý tưởng	A/01
tự giám sát	control / metacognition	Khả năng biết mình đang đi hướng nào và khi nào phải bỏ hướng đó	A/01
luyện tập có chủ đích	deliberate practice	Luyện có mục tiêu hẹp, mức hơi quá sức, phản hồi ngay, có sửa sai	A/02
upsolving	upsolving	Giải lại tới cùng những bài đã bí, sau kỳ thi	A/02
khối tri thức	chunk	Cụm thông tin đã ghép thành một đơn vị nhớ duy nhất	A/02
mô hình RAM	RAM model	Mô hình chi phí giả định mọi phép toán và truy cập ô nhớ tốn một đơn vị	A/03
độ phức tạp tiệm cận	asymptotic complexity	Tốc độ tăng của chi phí khi dữ liệu lớn lên	A/03
hệ thức truy hồi	recurrence	Phương trình mô tả chi phí đệ quy theo chi phí trên dữ liệu nhỏ hơn	A/03
khấu hao	amortized analysis	Chi phí trung bình mỗi thao tác trên một dãy thao tác xấu nhất	A/03
giả đa thức	pseudo-polynomial	Đa thức theo giá trị số nhưng mũ theo số bit biểu diễn	C/16
bất biến	invariant	Mệnh đề luôn đúng tại một điểm cố định trong thuật toán	A/04
đại lượng giảm	variant / measure	Số nguyên không âm giảm mỗi vòng, dùng chứng minh tính dừng	A/04
lập luận đổi chỗ	exchange argument	Biến lời giải tối ưu thành lời giải tham lam mà không làm tệ đi	A/04
lập luận cắt-dán	cut-and-paste	Chứng minh cấu trúc con tối ưu bằng phản chứng	A/04
kỳ vọng tuyến tính	linearity of expectation	Kỳ vọng của tổng bằng tổng kỳ vọng, kể cả khi phụ thuộc	A/05
biến chỉ báo	indicator variable	Biến 0/1 cho một sự kiện; kỳ vọng của nó bằng xác suất	A/05
nguyên lý chuồng bồ câu	pigeonhole principle	$n + 1$ vật vào n ngăn thì có ngăn chứa từ hai vật	A/05

B. Cấu trúc dữ liệu

Tiếng Việt	English	Nghĩa ngắn	Chương
tổng tiền tố	prefix sum	Mảng tổng lũy tích cho phép lấy tổng đoạn trong $O(1)$	B/06
mảng hiệu	difference array	Phép ngược của tổng tiền tố; cập nhật đoạn trong $O(1)$	B/06
hai con trỏ	two pointers	Hai chỉ số cùng quét, mỗi cái chỉ đi một chiều	B/06
cửa sổ trượt	sliding window	Duy trì một đoạn liên tiếp thoả điều kiện	B/06
ngăn xếp đơn điệu	monotonic stack	Ngăn xếp giữ thứ tự đơn điệu; tìm “phần tử lớn hơn kế tiếp”	B/07
hàng đợi ưu tiên	priority queue	Giao diện “lấy phần tử ưu tiên nhất”; thường cài bằng heap	B/07
heap nhị phân	binary heap	Cây gần đầy lưu trong mảng, mỗi nút không lớn hơn các con	B/07
bảng băm	hash table	Ép khoá thành chỉ số mảng để tra cứu $O(1)$ trung bình	B/08
băm phổ dụng	universal hashing	Chọn ngẫu nhiên hàm băm từ một họ, chống dữ liệu thù địch	B/08
nghịch lý ngày sinh	birthday paradox	Va chạm đáng kể khi số phần tử đạt cỡ $\sqrt{m}$	B/08
băm nhất quán	consistent hashing	Ảnh xạ khoá tới máy chủ ổn định khi số máy thay đổi	B/08
phép quay	rotation	Thao tác cục bộ giữ thứ tự nhưng đổi chiều cao cây	B/09
tăng cường	augmentation	Lưu thêm thông tin ở mỗi nút để trả lời truy vấn mới	B/09
treap	treap	Cây kết hợp BST theo khoá và heap theo độ ưu tiên ngẫu nhiên	B/09
B-cây	B-tree	Cây nhiều nhánh với nút vừa một khối đĩa	B/09
cây phân đoạn	segment tree	Cây lưu kết quả gộp của các đoạn; truy vấn và cập nhật $O(\log n)$	B/10
lazy propagation	lazy propagation	Hoãn đẩy cập nhật xuống con cho tới khi cần	B/10
Fenwick	binary indexed tree	Cấu trúc tiền tố động; khuôn tổng/XOR lấy hiệu hai tiền tố để hỏi đoạn	B/10
sparse table	sparse table	Bảng tiền xử lý cho truy vấn $O(1)$ trên mảng tĩnh, phép lũy đẳng	B/10
tính kết hợp	associativity	Điều kiện để gộp kết quả hai nửa thành kết quả cả đoạn	B/10
hợp nhất tập hợp	disjoint set union	Cấu trúc trả lời “cùng nhóm không” và “gộp hai nhóm”	B/11
nén đường	path compression	Nối mọi nút trên đường tìm lên thẳng gốc	B/11
gộp nhỏ vào lớn	small to large	Luôn đổ tập nhỏ vào tập lớn; tổng chi phí $O(n \log n)$	B/11
hàm tiền tố	prefix function	$\pi[i]$ = độ dài biên dài nhất; nền của KMP	B/12
biên	border	Chuỗi vừa là tiền tố vừa là hậu tố thực sự	B/12
mảng hậu tố	suffix array	Các hậu tố sắp theo thứ tự từ điển; biến bài chuỗi thành bài mảng	B/12

C. Chiến lược thiết kế

Tiếng Việt	English	Nghĩa ngắn	Chương
vét cạn	brute force	Duyệt mọi khả năng; luôn đúng, thường quá chậm	C/13
quay lui	backtracking	Xây lời giải từng bước, huỷ khi gặp ngõ cụt	C/13
cắt tỉa	pruning	Loại cả một nhánh khi chứng minh được nó vô ích	C/13
nhánh cận	branch and bound	Quay lui kèm cận để cắt nhánh không thể tốt hơn	C/13
gặp nhau ở giữa	meet in the middle	Chia đôi, liệt kê mỗi nửa $2^{n/2}$, rồi ghép	C/13
chia để trị	divide and conquer	Chia thành bài con độc lập, giải rồi ghép	C/14
tham lam	greedy	Lập lại lựa chọn tốt nhất cục bộ, không xét lại	C/15
matroid	matroid	Cấu trúc mà trên đó tham lam luôn tối ưu	C/15
dưới mô-đun	submodular	Hàm có lợi ích biên giảm dần; tham lam đạt $1 - 1/e$	C/15
quy hoạch động	dynamic programming	Vét cạn cộng ghi nhớ theo trạng thái	C/16
trạng thái	state	Bản tóm tắt quá khứ, đủ để quyết định tương lai	C/16
cấu trúc con tối ưu	optimal substructure	Lời giải tối ưu chứa lời giải tối ưu của bài con	C/16
mảng cuộn	rolling array	Chỉ giữ vài tầng gần nhất để giảm bộ nhớ	C/16
thứ tự tô-pô	topological order	Xếp đỉnh sao cho mọi cạnh đi từ trái sang phải	C/17
liên thông mạnh	strongly connected component	Tập đỉnh mà từ đỉnh nào cũng tới được mọi đỉnh khác	C/17
nhảy nhị phân	binary lifting	Bảng tổ tiên thứ 2^k cho truy vấn tổ tiên chung $O(\log n)$	C/17
đồ thị nhiều tầng	layered graph	Nhân bản đồ thị theo số lần dùng khả năng đặc biệt	C/17
luồng cực đại	maximum flow	Lượng lớn nhất đẩy được từ nguồn tới đích	C/18
lát cắt nhỏ nhất	minimum cut	Cách chia đỉnh rẻ nhất để ngắt nguồn khỏi đích	C/18
đồ thị thặng dư	residual graph	Đồ thị phụ với cạnh ngược cho phép rút lại luồng	C/18
ghép đôi	matching	Tập cạnh đôi một không chung đỉnh	C/18
tìm kiếm trên đáp án	binary search on answer	Đổi bài tối ưu thành bài quyết định rồi nhị phân	C/19
vị từ đơn điệu	monotone predicate	Đúng với mọi giá trị dễ hơn; điều kiện của nhị phân trên đáp án	C/19
tìm kiếm ba phân	ternary search	Thu hẹp khoảng cho hàm một cực trị	C/19
mô phỏng luyện kim	simulated annealing	Tìm kiếm cục bộ có chấp nhận bước xấu với xác suất giảm dần	C/19
Las Vegas	Las Vegas algorithm	Luôn đúng, thời gian chạy ngẫu nhiên	C/20
Monte Carlo	Monte Carlo algorithm	Luôn nhanh, kết quả có thể sai với xác suất nhỏ	C/20
khuyến đại xác suất	probability amplification	Lập lại để xác suất sai giảm theo hàm mũ	C/20
lấy mẫu hồ chứa	reservoir sampling	Chọn đều k phần tử từ dòng dữ liệu với $O(k)$ bộ nhớ	C/20

Tiếng Việt	English	Nghĩa ngắn	Chương
nghịch đảo modulo	modular inverse	Số thay cho phép chia trong số học modulo	D/21
định lý phần dư Trung Hoa	Chinese remainder theorem	Ghép các phần dư theo modulo nguyên tố cùng nhau	D/21
biến đổi Fourier nhanh	fast Fourier transform	Tính tích chập trong $O(n \log n)$ bằng cách đổi biểu diễn	D/21
vị từ định hướng	orientation predicate	Ba điểm quay trái, quay phải hay thẳng hàng	D/22
bao lồi	convex hull	Đa giác lồi nhỏ nhất chứa mọi điểm	D/22
quét đường	sweep line	Cho một đường quét qua mặt phẳng, duy trì cấu trúc theo chiều dọc	D/22
suy biến	degenerate case	Cấu hình biên: thẳng hàng, trùng nhau, chồng nhau	D/22
lớp P	class P	Bài toán quyết định giải được trong thời gian đa thức	D/23
lớp NP	class NP	Bài toán mà lời giải cho sẵn kiểm chứng được nhanh	D/23
quy dẫn	reduction	Biến bài A thành bài B; chuyển độ khó theo chiều mũi tên	D/23
NP-đầy đủ	NP-complete	Thuộc NP và mọi bài trong NP quy dẫn về được	D/23
xấp xỉ có bảo đảm	approximation ratio	Kết quả không tệ hơn tối ưu quá một hệ số chứng minh được	D/23
tham số hoá	parameterized complexity	Chi phí $O(f(k)n^c)$ với k là tham số nhỏ	D/23
cận dưới	lower bound	Mọi thuật toán trong một mô hình đều cần ít nhất chừng này	D/24
cây quyết định	decision tree	Mô hình chỉ cho phép so sánh; nền của cận dưới sắp xếp	D/24
lập luận đối thủ	adversary argument	Đối thủ trả lời bất lợi nhất mà vẫn nhất quán	D/24
bộ nhớ ngoài	external memory	Mô hình đếm số lần chuyển khối thay vì số phép toán	D/24
cache-oblivious	cache-oblivious	Tối ưu số lần chuyển khối mà không cần biết tham số phần cứng	D/24
tỉ lệ cạnh tranh	competitive ratio	So thuật toán trực tuyến với lời giải biết trước tương lai	D/24
công việc và chiều sâu	work and span	Tổng phép toán và chuỗi phụ thuộc dài nhất trong tính toán song song	D/24
stress test	stress testing	Chạy lời giải nhanh và vét cạn trên dữ liệu nhỏ ngẫu nhiên, so kết quả	E/25
thu nhỏ phản ví dụ	test case reduction	Tìm dữ liệu nhỏ nhất vẫn giữ được lỗi	E/25
khẳng định	assertion	Kiểm điều kiện tại một điểm; dừng ngay nơi phát sinh lỗi	E/25
độ trễ đuôi	tail latency	Thời gian phản hồi ở phân vị cao, thay vì trung bình	E/26
ngưỡng hoà vốn	crossover point	Kích thước mà tại đó thuật toán bậc thấp hơn bắt đầu thắng	E/26
hồ sơ hiệu năng	profiling	Đo thời gian thực sự bị tiêu ở đâu thay vì đoán	E/26
thi ảo	virtual contest	Tự tổ chức kỳ thi từ đề cũ, đúng thời lượng và điều kiện	E/27
ôn ngắt quãng	spaced repetition	Ôn lại ở các khoảng cách tăng dần, bằng cách tự truy xuất	E/28
vùng luyện	training zone	Mức khó mà bạn giải được khoảng một nửa số bài	E/28

Kết luận: cây này nói gì, và dùng tiếp thế nào

TÓM TẮT

Phần kết này làm ba việc. Một, gom lại những sợi chỉ chạy xuyên suốt 28 chương — có năm sợi, và chúng là thứ đáng nhớ hơn bất kỳ thuật toán riêng lẻ nào. Hai, nói rõ những gì quyển sách này *không* làm và nên đọc gì để bù. Ba, hướng dẫn cách mở rộng cây khi bạn học thêm, để nó lớn lên cùng bạn thay vì trở thành một tài liệu đóng băng.

Năm sợi chỉ xuyên suốt

Đọc lại 28 chương, có năm ý quay lại nhiều lần ở những chỗ rất khác nhau. Chúng là phần đáng mang theo nhất, vì thuật toán cụ thể thì tra được còn các ý này thì phải thấm.

Sợi thứ nhất — mọi lời giải nhanh bắt đầu từ một lời giải chậm. Vết cạn không phải bước lùi mà là điểm xuất phát bắt buộc: nó buộc bạn phát biểu chính xác bài toán, nó làm trọng tài cho stress test, và quan trọng nhất, nó đổi câu hỏi mơ hồ “giải thế nào” thành câu hỏi sắc bén “vì sao nó chậm”. Cả bảy chương còn lại của phần C đều là các câu trả lời cho câu hỏi thứ hai (C/13).

Sợi thứ hai — hãy dùng cấu trúc yếu nhất còn đủ dùng. Heap rẻ hơn cây cân bằng vì nó chỉ biết ai nhỏ nhất. DSU gần như miễn phí vì nó chỉ trả lời hai câu hỏi. Băm nhanh hơn cây vì nó vứt bỏ thứ tự. Mỗi khả năng bạn đòi thêm đều có giá, và rất nhiều lời giải chậm là do đòi hỏi những khả năng không dùng tới (B/07–B/11).

Sợi thứ ba — khi trường hợp xấu phụ thuộc dữ liệu, hãy chuyển nó sang một nguồn ngẫu nhiên bạn kiểm soát. Ý này xuất hiện năm lần: băm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui, và cuối cùng được phát biểu thành nguyên lý ở C/20. Nó không phải sự cầu may mà là cách tước bỏ khả năng dự đoán của đối thủ.

Sợi thứ tư — mô hình chi phí quyết định thuật toán. Cùng bài toán sắp xếp, đổi đơn vị chi phí từ “phép so sánh” sang “lần chuyển khối” thì lời giải tốt nhất đổi hình dạng. B-cây, cache-oblivious, thuật toán luồng dữ liệu, và cả sự khác biệt giữa thi đấu và hệ thống thời gian thực — tất cả đều là hệ quả của việc đổi mô hình (A/03, D/24, E/26).

Sợi thứ năm — năng lực đến từ vòng phản hồi, không từ số giờ. Bốn trường phái ở A/02 khác nhau ở thước đo nhưng giống nhau ở chỗ ai cũng có một thước đo khách quan và ai cũng sửa cách làm theo kết quả đo. Với người học, vòng phản hồi ấy là chấm bài, stress test, nhật ký lỗi và upsolving — và toàn bộ E/28 chỉ là cách vận hành nó.

Những gì quyển sách này không làm

Trung thực về giới hạn cũng là một phần của tài liệu tốt.

Nó *không* là tài liệu tra cứu đầy đủ. Nhiều thuật toán quan trọng chỉ được nhắc tên: cây Blossom cho ghép đôi tổng quát, link-cut tree, suffix automaton ở mức cài đặt, các thuật toán xấp xỉ chuyên sâu. Khi cần, hãy tra ở [27] cho chứng minh chuẩn, [15] cho việc nhận diện bài toán, và các nguồn cộng đồng như [18] cho cài đặt.

Nó *không* thay được việc giải bài. Mỗi chương có bài tập vì lý do đó, và chương [E/28](#) nói thẳng rằng đọc mà không giải chỉ tạo cảm giác quen thuộc.

Nó *không* phủ các mảng gần kề: tối ưu hoá liên tục, học máy, lý thuyết mã, thuật toán phân tán ở mức sâu. Những chỗ đó được nhắc trong các bảng *Liên hệ ngang*, đủ để bạn biết ranh giới nằm ở đâu và mở đúng cửa khi cần.

Cuối cùng, nó không phủ *thiết kế hệ thống* — một chủ đề đủ lớn để có cây riêng, và các khái niệm chuẩn bị cho nó nằm rải ở [B/08](#), [B/09](#), [D/24](#) và [E/26](#).

Mở rộng cây này thế nào

Cây tri thức chỉ có giá trị nếu nó lớn lên cùng người viết. Bốn quy tắc để mở rộng mà không làm hỏng cấu trúc.

Thêm chương mới thành thư mục, không thành file rời. Mỗi chủ đề mới là một thư mục NN-ten-chu-de trong phần phù hợp, với một file `.tex` cùng tên. Khi một mục trong chương phình to tới mức có nhánh con riêng, hãy nâng nó thành thư mục con có `00-map.tex` của chính nó.

Giữ nguyên khuôn của chương. Tóm tắt, kiến thức nền, câu hỏi mở đầu, thân bài, gốc rễ, liên hệ ngang, trả lời câu hỏi, nói lại thật đơn giản, active recall, hỏi đáp, bài tập, kết chương. Khuôn này không phải nghi thức: mỗi khối phục vụ một chức năng học tập cụ thể, và bỏ khối nào thì mất chức năng đó.

Cập nhật bản đồ khi thêm nội dung. Mỗi `00-map.tex` nói nhánh gồm gì, vì sao chia như vậy, phụ thuộc ra sao, và phần nào cốt lõi. Thêm chương mà không cập nhật bản đồ thì cây trở lại thành một thư mục.

Ghi ngày ôn gần nhất ở đầu file. Dòng metadata ở đầu mỗi file có trường “Ôn gần nhất” vì một lý do thực dụng: sau một năm, bạn cần biết chương nào lâu chưa quay lại. Đó cũng là cách rẻ nhất để vận hành lịch ôn ngắt quãng ở [E/28](#).

Một câu cuối

Quyển sách này mở đầu bằng nhận xét rằng khi bí, cái thiếu hầu như không bao giờ là một thuật toán chưa từng nghe tên. Sau 28 chương, nhận xét đó vẫn đúng — chỉ là bây giờ bạn có công cụ cụ thể để thay thế: một quy trình để chạy khi chưa biết làm gì, hai thước đo để tự chấm ý tưởng trước khi gõ code, một bộ khuôn tư duy để sinh lời giải, và một vòng phản hồi để mỗi lần sai đều đổi được thành một thứ dùng lại được.

Phần còn lại không nằm trong sách. Nó nằm ở việc sáng mai bạn có mở một bài ra hay không.

Nguồn

- [1] G. Pólya, *How to Solve It: A New Aspect of Mathematical Method*. Princeton University Press, 1945.
- [2] A. H. Schoenfeld, *Mathematical Problem Solving*. Academic Press, 1985.
- [3] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, "The Role of Deliberate Practice in the Acquisition of Expert Performance," *Psychological Review*, vol. 100, no. 3, pp. 363–406, 1993.
- [4] P. Zeitz, *The Art and Craft of Problem Solving*, 2nd ed. Wiley, 2007.
- [5] A. Engel, *Problem-Solving Strategies*. Springer, 1998.
- [6] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [7] KTH Royal Institute of Technology. "KACTL: KTH Algorithm Competition Template Library." (2026), address: <https://github.com/kth-competitive-programming/kactl> (visited on 09/06/2026).
- [8] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Addison-Wesley, 1997.
- [9] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.
- [10] D. E. Knuth, "Structured Programming with go to Statements," *ACM Computing Surveys*, vol. 6, no. 4, pp. 261–301, 1974.
- [11] E. W. Dijkstra, *A Discipline of Programming*. Prentice-Hall, 1976.
- [12] C. A. R. Hoare, "An Axiomatic Basis for Computer Programming," *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969.
- [13] D. Gries, *The Science of Programming*. Springer, 1981.
- [14] J. L. Bentley and M. D. McIlroy, "Engineering a Sort Function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.
- [15] S. S. Skiena, *The Algorithm Design Manual*, 3rd ed. Springer, 2020.
- [16] S. Halim, F. Halim, and S. Effendy, *Competitive Programming 4*. Lulu, 2020.
- [17] A. Laaksonen, *Guide to Competitive Programming: Learning and Improving Algorithms Through Contests*. Springer, 2017.
- [18] e-maxx / cp-algorithms contributors. "cp-algorithms: Algorithms for Competitive Programming." (2026), address: <https://cp-algorithms.com/> (visited on 09/06/2026).
- [19] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [20] R. E. Tarjan, *Data Structures and Network Algorithms*. SIAM, 1983.
- [21] R. E. Tarjan, "Amortized Computational Complexity," *SIAM Journal on Algebraic and Discrete Methods*, vol. 6, no. 2, pp. 306–318, 1985.
- [22] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.
- [23] R. Sedgewick and P. Flajolet, *An Introduction to the Analysis of Algorithms*, 2nd ed. Addison-Wesley, 2013.
- [24] K. A. Ericsson and R. Pool, *Peak: Secrets from the New Science of Expertise*. Houghton Mifflin Harcourt, 2016.

- [25] R. W. Hamming, *The Art of Doing Science and Engineering: Learning to Learn*. Gordon and Breach, 1997.
- [26] P. Norvig. “Teach Yourself Programming in Ten Years.” (2001), address: <https://norvig.com/21-days.html> (visited on 09/06/2026).
- [27] T. H. Cormen, C. E. Leiserson, R. L. Rivest, et al., *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
- [28] J. Kleinberg and É. Tardos, *Algorithm Design*. Addison-Wesley, 2006.
- [29] J. Erickson, *Algorithms*. Independently published, 2019.
- [30] R. L. Graham, D. E. Knuth, and O. Patashnik, *Concrete Mathematics: A Foundation for Computer Science*, 2nd ed. Addison-Wesley, 1994.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] G. L. McDowell, *Cracking the Coding Interview*, 6th ed. CareerCup, 2015.
- [33] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).
- [34] M. Blum, R. W. Floyd, V. Pratt, et al., “Time Bounds for Selection,” *Journal of Computer and System Sciences*, vol. 7, no. 4, pp. 448–461, 1973.
- [35] D. D. Sleator and R. E. Tarjan, “Self-Adjusting Binary Search Trees,” *Journal of the ACM*, vol. 32, no. 3, pp. 652–686, 1985.
- [36] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms,” *Journal of the ACM*, vol. 34, no. 3, pp. 596–615, 1987.
- [37] D. R. Musser, “Introspective Sorting and Selection Algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [38] F. McSherry, M. Isard, and D. G. Murray, “Scalability! But at What COST?” In *15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.
- [39] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. MIT, 2018, sách giáo trình, bản điện tử miễn phí.
- [40] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.
- [41] D. H. Larkin, S. Sen, and R. E. Tarjan, “A Back-to-Basics Empirical Study of Priority Queues,” in *Proceedings of the 16th Workshop on Algorithm Engineering and Experiments (ALENEX)*, SIAM, 2014.
- [42] R. Pagh and F. F. Rodler, “Cuckoo Hashing,” *Journal of Algorithms*, vol. 51, no. 2, pp. 122–144, 2004.
- [43] M. Pătraşcu and M. Thorup, “The Power of Simple Tabulation Hashing,” *Journal of the ACM*, vol. 59, no. 3, 14:1–14:50, 2012.
- [44] P. Flajolet, É. Fusy, O. Gandouet, et al., “HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm,” in *Conference on Analysis of Algorithms (AofA)*, 2007.
- [45] E. D. Demaine, D. Harmon, J. Iacono, et al., “Dynamic Optimality—Almost,” *SIAM Journal on Computing*, vol. 37, no. 1, pp. 240–251, 2007.
- [46] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [47] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [48] AtCoder. “AC Library: Lazy Segtree.” (), address: https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html (visited on 09/07/2026).

- [49] R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [50] R. E. Tarjan, "A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets," *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [51] M. L. Fredman and M. E. Saks, "The Cell Probe Complexity of Dynamic Data Structures," in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [52] D. E. Knuth, J. H. Morris, and V. R. Pratt, "Fast Pattern Matching in Strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.
- [53] A. V. Aho and M. J. Corasick, "Efficient String Matching: An Aid to Bibliographic Search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–340, 1975.
- [54] U. Manber and G. Myers, "Suffix Arrays: A New Method for On-line String Searches," *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935–948, 1993.
- [55] V. Strassen, "Gaussian Elimination Is Not Optimal," *Numerische Mathematik*, vol. 13, pp. 354–356, 1969.
- [56] D. Harvey and J. van der Hoeven, "Integer Multiplication in Time $O(n \log n)$," *Annals of Mathematics*, vol. 193, no. 2, pp. 563–617, 2021.
- [57] J. Alman and V. Vassilevska Williams, "A Refined Laser Method and Faster Matrix Multiplication," in *Proceedings of the 32nd ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2021.
- [58] J. Alman, R. Duan, V. Vassilevska Williams, et al. "Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve." bản tiền ấn arXiv:2608.16884, chưa qua phản biện. (2026), address: <https://arxiv.org/abs/2608.16884> (visited on 09/07/2026).
- [59] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [60] D. E. Knuth, "Optimum Binary Search Trees," *Acta Informatica*, vol. 1, pp. 14–25, 1971.
- [61] F. F. Yao, "Speed-Up in Dynamic Programming," *SIAM Journal on Algebraic and Discrete Methods*, vol. 3, no. 4, pp. 532–540, 1982.
- [62] A. Aggarwal, M. M. Klawe, S. Moran, et al., "Geometric Applications of a Matrix-Searching Algorithm," *Algorithmica*, vol. 2, pp. 195–208, 1987.
- [63] D. S. Hirschberg, "A Linear Space Algorithm for Computing Maximal Common Subsequences," *Communications of the ACM*, vol. 18, no. 6, pp. 341–343, 1975.
- [64] R. E. Tarjan, "Depth-First Search and Linear Graph Algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [65] M. Thorup, "Undirected Single-Source Shortest Paths with Positive Integer Weights in Linear Time," *Journal of the ACM*, vol. 46, no. 3, pp. 362–394, 1999.
- [66] R. Duan, J. Mao, X. Mao, et al., "Breaking the Sorting Barrier for Directed Single-Source Shortest Paths," in *Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC)*, Best Paper Award, 2025.
- [67] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [68] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.
- [69] J. B. Orlin, "Max Flows in $O(nm)$ Time, or Better," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 765–774.

- [70] L. Chen, R. Kyng, Y. P. Liu, et al., “Maximum Flow and Minimum-Cost Flow in Almost-Linear Time,” in *Proceedings of the 63rd IEEE Symposium on Foundations of Computer Science (FOCS)*, Best Paper Award, 2022.
- [71] J. E. Hopcroft and R. M. Karp, “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs,” *SIAM Journal on Computing*, vol. 2, no. 4, pp. 225–231, 1973.
- [72] M. Agrawal, N. Kayal, and N. Saxena, “PRIMES Is in P,” *Annals of Mathematics*, vol. 160, no. 2, pp. 781–793, 2004.
- [73] J. R. Shewchuk, “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates,” *Discrete & Computational Geometry*, vol. 18, pp. 305–363, 1997.
- [74] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.
- [75] R. Impagliazzo and R. Paturi, “On the Complexity of k-SAT,” *Journal of Computer and System Sciences*, vol. 62, no. 2, pp. 367–375, 2001.
- [76] A. Backurs and P. Indyk, “Edit Distance Cannot Be Computed in Strongly Subquadratic Time (Unless SETH Is False),” *SIAM Journal on Computing*, vol. 47, no. 3, 2018, bản hội nghị: STOC 2015.
- [77] A. Gajentaan and M. H. Overmars, “On a Class of $O(n^2)$ Problems in Computational Geometry,” *Computational Geometry: Theory and Applications*, vol. 5, no. 3, pp. 165–185, 1995.
- [78] M. Frigo, C. E. Leiserson, H. Prokop, et al., “Cache-Oblivious Algorithms,” in *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 1999, pp. 285–297.
- [79] N. Alon, Y. Matias, and M. Szegedy, “The Space Complexity of Approximating the Frequency Moments,” *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 1999.
- [80] A. Zeller and R. Hildebrandt, “Simplifying and Isolating Failure-Inducing Input,” *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [81] J. Bentley, *Writing Efficient Programs*. Prentice-Hall, 1982.
- [82] M. Mitzenmacher and E. Upfal, *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.
- [83] D. P. Williamson and D. B. Shmoys, *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
- [84] V. V. Vazirani, *Approximation Algorithms*. Springer, 2001.
- [85] M. de Berg, O. Cheong, M. van Kreveld, et al., *Computational Geometry: Algorithms and Applications*, 3rd ed. Springer, 2008.
- [86] E. D. Demaine. “MIT 6.851: Advanced Data Structures.” (2021), address: <https://courses.csail.mit.edu/6.851/> (visited on 09/06/2026).
- [87] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice-Hall, 1993.
- [88] S. A. Cook, “The Complexity of Theorem-Proving Procedures,” in *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 1971, pp. 151–158.
- [89] R. M. Karp, “Reducibility Among Combinatorial Problems,” in *Complexity of Computer Computations*, Plenum Press, 1972, pp. 85–103.
- [90] P. M. Fenwick, “A New Data Structure for Cumulative Frequency Tables,” *Software: Practice and Experience*, vol. 24, no. 3, pp. 327–336, 1994.
- [91] S. S. Skiena and M. A. Revilla, *Programming Challenges: The Programming Contest Training Manual*. Springer, 2003.